

Claude session transcript: MAIIC raw query scripts

This is the complete record of the part of a Claude Code session (5 and 6 October 2026) that dealt with the three raw query scripts MAIIC sent for Extracts A, B and C. It is saved so that another member of staff, with or without an AI tool, can follow what was done, in what order, and why.

It starts at the request to analyse the scripts and runs to the audit of the eighteen follow-up extracts MAIIC returned on 7 October 2026 and the notes written from it. Earlier, unrelated parts of the same session are left out.

The work falls into six stages, in time order:

1. Analysis of the three scripts and tests against the delivered extract files (5 October).
2. The data dictionary query pack and its run order.
3. The plain-language PDF report, version 1, and the session transcript and build files saved for review.
4. The independent review by Codex, the response to it, and version 2 of the report and query pack, including suggested changes to MAIIC's scripts and to the Duplex engine code.
5. The data dictionary results returned by MAIIC on 6 October, what they showed, and the follow-up extract queries written from them.
6. The audit of the eighteen follow-up extracts received on 7 October: date formats, reconciliations to the balance history, loan book, Extract C and trial balance, the interest rebuild, the repayment-profile tests, and the notes to Barry and Credit that followed.

What is included: every user message, every reply shown to the user, every tool call with its full input (commands, scripts and file contents), and every tool result in full. Where a long result was stored by the tooling in a side file, that file is reproduced here.

What is left out: the assistant's internal reasoning, automatic system reminders (they carry unrelated workspace context), the text of the PDF skill, and image outputs (page renders used for visual checks).

How to follow it: the files produced along the way are in the same folder as this transcript. The Build files subfolder holds the Python scripts, with a README. The data dictionary results themselves are in the Query Request Responses 6 Oct 2026 folder next to this one.

Corrections made during the session, left in place where they were made: (a) the first analysis said Extract A shows one disbursement tranche per loan, which a later test showed was wrong; (b) version 1 of the report overstated several points that the Codex review identified, and version 2 corrects them. Both are explained in the report's Appendix E.

Times are UTC. Source: the session log ee984144-7b28-4df1-9a6e-560f112869e9.jsonl, entries 483 to 1410. Events: 373.

USER (2026-10-05 13:02:59 UTC)

"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts"

These are the scripts providing extract a. b and c that we requested from MAIIC. Please analyse

Tool call: PowerShell

description: List the raw query scripts and the MAIIC project folder

--- command ---

```
Get-ChildItem -LiteralPath "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts" -Recurse |  
Select-Object LastWriteTime, Length, @{n='Path';e={$_.FullName.Replace('C:\Users\wadza\OneDrive\2026\Projects\MAIIC\','')}} |  
Format-Table -AutoSize -Wrap; "-----"; Get-ChildItem -LiteralPath "C:\Users\wadza\OneDrive\2026\Projects\MAIIC" -Depth 1 |  
Select-Object LastWriteTime, Length, @{n='Path';e={$_.FullName.Replace('C:\Users\wadza\OneDrive\2026\Projects\MAIIC\','')}} |  
Format-Table -AutoSize -Wrap
```

Result of PowerShell

LastWriteTime	Length	Path
05/10/2026 13:10:20	9616	2. Documents from clients\Raw Query Scripts\CASHFLOW_REGISTER.txt
05/10/2026 13:10:20	6999	2. Documents from clients\Raw Query Scripts\FACILITY_REGISTER.txt
05/10/2026 13:10:20	3306	2. Documents from clients\Raw Query Scripts\INTEREST_RECON.txt

LastWriteTime	Length	Path
10/09/2026 16:39:53		1. Engagement Contracting
05/10/2026 13:14:32		2. Documents from clients
27/09/2026 21:45:11		3. Project Execution
10/09/2026 15:58:14		4. Reporting & Deliverables
10/09/2026 12:35:06		5. Project Completion & Lessons Learnt
27/09/2026 21:45:11		6. Prior Engagements (ESG and 2025 ECL)
27/09/2026 21:44:31		9. Archive
24/09/2026 10:05:52	2905	README.md
10/09/2026 12:35:06		1. Engagement Contracting\Award Letter
10/09/2026 12:35:06		1. Engagement Contracting\Claude Session
10/09/2026 12:35:06		1. Engagement Contracting\Contract Reviews
10/09/2026 12:35:06		1. Engagement Contracting\Emails Received
10/09/2026 12:35:06		1. Engagement Contracting\Final Contract
10/09/2026 12:35:06		1. Engagement Contracting\Other Public Documents
10/09/2026 12:28:52		1. Engagement Contracting\Pitch
10/09/2026 12:28:52		1. Engagement Contracting\Reference Contracts
07/09/2026 13:06:00	5973086	1. Engagement Contracting\DUPELIX INSTITUTE CONTRACT IFRS9 TOOL 2026.pdf
04/10/2026 18:50:00	5807004	1. Engagement Contracting\DUPELIX INSTITUTE SIGNED CONTRACT FOR IFRS 9 TOOL.pdf
21/01/2025 07:37:16	58121	1. Engagement Contracting\Engagement Letter - MAIIC.docx
21/01/2025 07:38:39	139659	1. Engagement Contracting\Engagement Letter - MAIIC.pdf
06/01/2025 20:02:01	169300	1. Engagement Contracting\Purchase Order - Dupleix ESG 041224.pdf
27/08/2024 08:23:59	130828	1. Engagement Contracting\Standard Terms and Conditions - MAIIC.pdf
21/09/2026 12:33:41		2. Documents from clients\AFS 2025
10/09/2026 12:35:06		2. Documents from clients\Annual Reports
10/09/2026 13:38:40		2. Documents from clients\Database extracts
27/09/2026 21:44:33		2. Documents from clients\EIR Assessment
28/09/2026 10:43:01		2. Documents from clients\EMI CHARTS
10/09/2026 12:35:06		2. Documents from clients\Image and Logos
17/09/2026 16:21:43		2. Documents from clients\Loan Books
28/09/2026 10:51:02		2. Documents from clients\Loan Books- Ebanker Format
01/10/2026 14:29:49		2. Documents from clients\Meetings and Transcripts
11/09/2026 13:13:10		2. Documents from clients\New Doc Received 11 Sep
10/09/2026 12:35:31		2. Documents from clients\New Doc Received 19 Aug
21/09/2026 10:38:02		2. Documents from clients\New Doc Received 21 Sep
18/09/2026 08:33:07		2. Documents from clients\Offer Letters
05/10/2026 13:14:01		2. Documents from clients\Raw Query Scripts
10/09/2026 12:59:06		2. Documents from clients\Trial Balances
21/07/2026 11:35:00	180198	2. Documents from clients\26100.04 Interest Income - EIR Vs Contractual rate(2025-05-28 11.36.03).xlsx
21/07/2026 11:37:00	423737	2. Documents from clients\Assessment of EIR monthly basis.xlsx
21/07/2026 11:38:03	692224	2. Documents from clients\FW Calculation on Impact of using contractual rate v EIR.msg
03/08/2026 14:43:46	92672	2. Documents from clients\FW MAIIC EIR Assessment as of 31st December 2025.msg
17/09/2026 16:01:02	11534	2. Documents from clients\Interest Rate Change History.xlsx
11/09/2026 11:59:27	50607	2. Documents from clients\MAIIC EIR - Source Screen Field Inventory.docx
09/09/2026 09:03:47	87552	2. Documents from clients\printPRCReportDSC(90).xls
23/07/2026 11:14:00	11710	2. Documents from clients\Script Preparation_Dupleix.xlsx
23/07/2026 11:26:19	9611	2. Documents from clients\Script Preparation_Dupleix_REVIEWED.xlsx
16/09/2026 11:24:02		3. Project Execution\Amortisation Analysis
11/09/2026 13:27:27		3. Project Execution\Correspondence - Information Requests
27/09/2026 21:44:02		3. Project Execution\Data Review and Meeting Preparation 24 Sep 2026
10/09/2026 12:55:08		3. Project Execution\Database Backups
27/09/2026 21:43:46		3. Project Execution\Reference Rates
10/09/2026 12:35:06		3. Project Execution\specs
10/09/2026 12:35:06		3. Project Execution\VPN
17/09/2026 11:41:12	30723	3. Project Execution\contract_master_2025_with_fees.csv
18/08/2026 15:43:34	393774	3. Project Execution\MAIIC_Data_Status_and_Guidance_2026-08-18.pdf
10/09/2026 16:13:40		4. Reporting & Deliverables\December 2025 Review 2026-09-10
10/09/2026 17:14:10		6. Prior Engagements (ESG and 2025 ECL)\MAIIC 2025 Project Folder
10/09/2026 12:25:54	347816	9. Archive\Dupleix 2.zip

Tool call: Read

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\FACILITY_REGISTER.txt

Result of Read

```
1 CREATE OR REPLACE PROCEDURE PROC_BUILD_MAIIC_FACILITY_REGISTER (
2     p_as_of_date IN DATE DEFAULT TRUNC(SYSDATE)
3 ) IS
4     v_rowcount NUMBER;
5 BEGIN
6
7     DELETE FROM MAIIC_FACILITY_REGISTER WHERE AS_OF_DATE = TRUNC(p_as_of_date);
8
9     INSERT INTO MAIIC_FACILITY_REGISTER (
10         AS_OF_DATE, CUSTOMER_ID, CUSTOMER_NAME, LOAN_ACCOUNT_NUMBER, SUB_ACCOUNT_NO,
11         GL_ACCOUNT_CODE, GL_ACCOUNT_TITLE, PORTFOLIO, PRODUCT_TYPE, CURRENCY,
12         LOAN_START_DATE, SANCTIONED_AMOUNT, PRINCIPAL_DISBURSED, DISBURSEMENT_TRANCHES,
13         INTEREST_RATE, RATE_BASIS, COMPOUNDING, DAY_COUNT_BASIS, REPAYMENT_FREQUENCY,
14         TENOR, NUM_INSTALLMENTS, FIRST_REPAYMENT_DATE, CONTRACTUAL_MATURITY_DATE,
15         ACTUAL_CLOSURE_DATE, WRITEOFF_DATE, WRITEOFF_FLAG, WRITEOFF_AMOUNT,
16         ACCOUNT_STATUS, RESTRUCTURE_DATE, PRINCIPAL_GRACE_PERIOD, INTEREST_GRACE_PERIOD,
17         BALANCE_SNAPSHOT_DATE, OPENING_BALANCE, BALANCE_DATA_NOTE
18     )
19     SELECT
20         TRUNC(p_as_of_date),
21         a.customer_id,
22         c.customer_name,
23         a.new_ac_number,
24         a.sub_ac_number,
25         a.glcode,
26         gm.gltitle,
27         CASE WHEN UPPER(s.scheme_name) LIKE 'MAIIC%' THEN 'MAIIC'
28              WHEN UPPER(s.scheme_name) LIKE 'FINES%' THEN 'FINES'
29              ELSE 'OTHER' END,
30         s.scheme_name,
31         a.currency_type,
32         a.account_open_date,
33         al.sanction_amount,
34         (SELECT SUM(d.disb_schedule_amount) FROM loan_disbursement_schedule d
35          WHERE d.new_ac_number = a.new_ac_number AND d.sub_account_no = a.sub_ac_number
36          AND d.delete_flag = 'N'),
37         (SELECT LISTAGG(TO_CHAR(d.disb_schedule_date, 'DD-Mon-YYYY') || ':' ||
38             TO_CHAR(d.disb_schedule_amount, 'FM999999999999.00'), ', ');
39          WITHIN GROUP (ORDER BY d.disb_schedule_date)
40         FROM loan_disbursement_schedule d
41         WHERE d.new_ac_number = a.new_ac_number AND d.sub_account_no = a.sub_ac_number
42         AND d.delete_flag = 'N'),
43         a.interest_rate,
44         CASE s.floating_flag WHEN 'F' THEN 'Fixed' WHEN 'I' THEN 'Variable' ELSE s.floating_flag END,
45         CASE s.compound_simple_flag WHEN 'C' THEN 'Compound' WHEN 'S' THEN 'Simple' ELSE s.compound_simple_flag END,
46         s.intdays_per_year,
47         CASE mi.installment_pay_mode
48             WHEN 'M' THEN 'Monthly' WHEN 'Q' THEN 'Quarterly' WHEN 'H' THEN 'Half-Yearly'
49             WHEN 'Y' THEN 'Yearly' WHEN 'W' THEN 'Weekly' WHEN 'D' THEN 'Daily'
50             ELSE mi.installment_pay_mode END,
51         al.period_years || 'y ' || al.period_months || 'm ' || al.period_days || 'd',
52         (SELECT COUNT(*) FROM fixloan_inst_chart f
53          WHERE f.new_ac_number = a.new_ac_number AND f.sub_account_no = a.sub_ac_number
54          AND f.delete_flag = 'N'),
55         (SELECT MIN(f.installment_date) FROM fixloan_inst_chart f
56          WHERE f.new_ac_number = a.new_ac_number AND f.sub_account_no = a.sub_ac_number
57          AND f.delete_flag = 'N'),
58         al.expiry_date,
59         CASE WHEN a.status_code = 'C' THEN a.account_status_date END,
60         (SELECT MAX(w.writeoff_date) FROM loan_writeoff_master w
61          WHERE w.new_ac_number = a.new_ac_number AND w.sub_ac_number = a.sub_ac_number
62          AND w.delete_flag = 'N'),
63         (SELECT CASE WHEN COUNT(*) > 0 THEN 'Y' ELSE 'N' END FROM loan_writeoff_master w
64          WHERE w.new_ac_number = a.new_ac_number AND w.sub_ac_number = a.sub_ac_number
65          AND w.delete_flag = 'N'),
66         (SELECT SUM(w.writeoff_amount) FROM loan_writeoff_master w
67          WHERE w.new_ac_number = a.new_ac_number AND w.sub_ac_number = a.sub_ac_number
68          AND w.delete_flag = 'N'),
69         CASE a.status_code WHEN 'A' THEN 'Active' WHEN 'C' THEN 'Closed' WHEN 'D' THEN 'Dormant'
70             ELSE a.status_code END,
71         (SELECT MAX(r.transaction_date) FROM loan_reschedule_details r
72          WHERE r.new_ac_number = a.new_ac_number AND r.sub_account_no = a.sub_ac_number
73          AND r.delete_flag = 'N'),
74         al.pmoratorium_period || ' ' || al.pmoratorium_unit,
75         al.imoratorium_period || ' ' || al.imoratorium_unit,
76         bal.transaction_date,
77         CASE WHEN bal.principal_balance IS NOT NULL THEN ABS(bal.principal_balance)
78             WHEN a.status_code = 'C' AND a.account_status_date <= TRUNC(p_as_of_date) THEN 0
79             ELSE NULL END,
80         CASE WHEN a.account_open_date > TRUNC(p_as_of_date)
81             THEN 'N/A - loan opened after ' || TO_CHAR(TRUNC(p_as_of_date), 'DD-Mon-YYYY')
82             WHEN bal.principal_balance IS NOT NULL THEN NULL
83             WHEN a.status_code = 'C' AND a.account_status_date <= TRUNC(p_as_of_date)
84             THEN 'Assumed zero - closed before as-of date, no snapshot recorded'
85             ELSE 'DATA GAP - no balance snapshot found, needs manual verification' END
86     FROM acmaster a
87     JOIN acloanmaster al ON al.new_ac_number = a.new_ac_number AND al.sub_account_no = a.sub_ac_number
88     JOIN customer_mast c ON c.customer_id = a.customer_id
89     JOIN scheme_master s ON s.scheme_mst_id = a.scheme_mst_id
```

```

90 LEFT JOIN glmaster gm ON gm.cbs_glcode = a.glcode
91 LEFT JOIN (
92   -- Inline view: the ACCOUNT_BALANCE snapshot nearest on/before p_as_of_date,
93   -- per account. Written this way (not as a subquery inside the outer join's
94   -- ON clause) to avoid ORA-01799 "a column may not be outer-joined to a
95   -- subquery", which Oracle raises when a static PL/SQL statement outer-joins
96   -- to a table whose join predicate itself contains a correlated subquery.
97   SELECT b1.new_ac_number, b1.sub_ac_number, b1.transaction_date, b1.principal_balance
98   FROM account_balance b1
99   WHERE b1.transaction_date = (
100     SELECT MAX(b2.transaction_date) FROM account_balance b2
101     WHERE b2.new_ac_number = b1.new_ac_number AND b2.sub_ac_number = b1.sub_ac_number
102     AND b2.transaction_date <= TRUNC(p_as_of_date)
103   )
104 ) bal
105 ON bal.new_ac_number = a.new_ac_number AND bal.sub_ac_number = a.sub_ac_number
106 LEFT JOIN (
107   -- Inline view (same ORA-01799-avoidance pattern as the balance lookup
108   -- above): the most recent installment-plan revision per facility.
109   SELECT m1.new_ac_number, m1.sub_account_no, m1.installment_pay_mode
110   FROM multi_inst_details m1
111   WHERE m1.delete_flag = 'N'
112   AND m1.inst_det_id = (
113     SELECT MAX(m2.inst_det_id) FROM multi_inst_details m2
114     WHERE m2.new_ac_number = m1.new_ac_number AND m2.sub_account_no = m1.sub_account_no
115     AND m2.delete_flag = 'N'
116   )
117 ) mi
118 ON mi.new_ac_number = a.new_ac_number AND mi.sub_account_no = a.sub_ac_number
119 WHERE UPPER(s.scheme_name) LIKE 'MAIIC%' OR UPPER(s.scheme_name) LIKE 'FINES%';
120
121 v_rowcount := SQL%ROWCOUNT;
122 COMMIT;
123
124 DBMS_OUTPUT.PUT_LINE('MAIIC_FACILITY_REGISTER rebuilt for as-of date ' ||
125   TO_CHAR(TRUNC(p_as_of_date), 'DD-Mon-YYYY') || ': ' ||
126   v_rowcount || ' facilities.');
```

```

127
128 EXCEPTION
129   WHEN OTHERS THEN
130     ROLLBACK;
131     RAISE_APPLICATION_ERROR(-20001,
132       'PROC_BUILD_MAIIC_FACILITY_REGISTER failed for as-of date ' ||
133       TO_CHAR(p_as_of_date, 'DD-Mon-YYYY') || ': ' || SQLERRM);
134 END PROC_BUILD_MAIIC_FACILITY_REGISTER;
135
```

Tool call: Read

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\CASHFLOW_REGISTER.txt

Tool call: Read

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\INTEREST_RECON.txt

Result of Read

```
1  CREATE OR REPLACE PROCEDURE PROC_BUILD_MAIIC_GL_INTEREST_RECON (
2    p_start_date IN DATE DEFAULT TO_DATE('01-01-2025','DD-MM-YYYY'),
3    p_end_date IN DATE DEFAULT TO_DATE('31-12-2026','DD-MM-YYYY'),
4    p_run_id OUT NUMBER
5  ) IS
6    v_rowcount NUMBER;
7  BEGIN
8
9    SELECT SEQ_MAIIC_GL_INT_RUN.NEXTVAL INTO p_run_id FROM DUAL;
10
11    INSERT INTO MAIIC_GL_INTEREST_RECOGNIZED (
12      RUN_ID, CUSTOMER_ID, LOAN_ACCOUNT_NUMBER, SUB_ACCOUNT_NO, GL_ACCOUNT_CODE,
13      PERIOD_TYPE, PERIOD_YEAR, PERIOD_MONTH, INTEREST_INCOME_POSTED,
14      TRANSACTION_COUNT, POSTING_REFERENCES, ROW_NOTE
15    )
16    WITH scope_accounts AS (
17      SELECT a.new_ac_number, a.sub_ac_number, a.customer_id, a.glcode
18      FROM acmaster a
19      JOIN scheme_master s ON s.scheme_mst_id = a.scheme_mst_id
20      WHERE UPPER(s.scheme_name) LIKE 'MAIIC%' OR UPPER(s.scheme_name) LIKE 'FINES%'
21    ),
22    interest_postings AS (
23      SELECT
24        sa.customer_id,
25        cv.new_ac_number,
26        cv.sub_account_no,
27        sa.glcode AS gl_account_code,
28        cv.transaction_date,
29        EXTRACT(YEAR FROM cv.transaction_date) AS period_year,
30        EXTRACT(MONTH FROM cv.transaction_date) AS period_month,
31        -1 * cv.transamt AS interest_income_signed,
32        sa.glcode || '-' || cv.cumvouch_det_id AS gl_posting_ref
33      FROM cumvouch cv
34      JOIN scope_accounts sa ON sa.new_ac_number = cv.new_ac_number AND sa.sub_ac_number = cv.sub_account_no
35      WHERE cv.ac_glcode = sa.glcode
36        AND cv.delete_flag = 'N'
37        AND cv.trantype IN (120, 303, 308, 309, 310, 311)
38        AND cv.transaction_date BETWEEN p_start_date AND p_end_date
39    )
40    SELECT
41      p_run_id,
42      customer_id,
43      new_ac_number,
44      sub_account_no,
45      gl_account_code,
46      CASE WHEN GROUPING(period_month) = 1 THEN 'YEAR' ELSE 'MONTH' END,
47      period_year,
48      CASE WHEN GROUPING(period_month) = 1 THEN 0 ELSE period_month END,
49      SUM(interest_income_signed),
50      COUNT(*),
51      LISTAGG(gl_posting_ref, ';' ON OVERFLOW TRUNCATE '...(truncated)' WITH COUNT)
52      WITHIN GROUP (ORDER BY transaction_date, gl_posting_ref),
53      CASE WHEN GROUPING(period_month) = 1
54        THEN 'Annual total of the monthly interest-income postings below.'
55        ELSE 'Net of TRANTYPE 303/120/308/309/310/311 postings to the loan''s own GL this month.'
56      END
57    FROM interest_postings
58    GROUP BY GROUPING SETS (
59      (customer_id, new_ac_number, sub_account_no, gl_account_code, period_year, period_month),
60      (customer_id, new_ac_number, sub_account_no, gl_account_code, period_year)
61    );
62
63    v_rowcount := SQL%ROWCOUNT;
64    COMMIT;
65
66    DBMS_OUTPUT.PUT_LINE('MAIIC_GL_INTEREST_RECOGNIZED built, RUN_ID=' || p_run_id ||
67      ', range ' || TO_CHAR(p_start_date,'DD-Mon-YYYY') || ' to ' ||
68      TO_CHAR(p_end_date,'DD-Mon-YYYY') || ': ' || v_rowcount || ' rows (month+year combined).');
69
70  EXCEPTION
71    WHEN OTHERS THEN
72      ROLLBACK;
73      RAISE_APPLICATION_ERROR(-20003,
74        'PROC_BUILD_MAIIC_GL_INTEREST_RECON failed for range ' ||
75        TO_CHAR(p_start_date,'DD-Mon-YYYY') || ' to ' || TO_CHAR(p_end_date,'DD-Mon-YYYY') ||
76        ': ' || SQLERRM);
77  END PROC_BUILD_MAIIC_GL_INTEREST_RECON;
78
```

Result of Read

```
1  CREATE OR REPLACE PROCEDURE PROC_BUILD_MAIIC_CASHFLOW_REGISTER (
2      p_start_date IN DATE DEFAULT TO_DATE('01-01-2025','DD-MM-YYYY'),
3      p_end_date   IN DATE DEFAULT TO_DATE('31-12-2026','DD-MM-YYYY'),
4      p_run_id     OUT NUMBER
5  ) IS
6      v_rowcount NUMBER;
7  BEGIN
8
9      SELECT SEQ_MAIIC_CASHFLOW_RUN.NEXTVAL INTO p_run_id FROM DUAL;
10
11
12      INSERT INTO MAIIC_CASHFLOW_REGISTER (
13          RUN_ID, CUSTOMER_ID, LOAN_ACCOUNT_NUMBER, SUB_ACCOUNT_NO, GL_POSTING_REF,
14          TRANSACTION_DATE, TRANSACTION_TYPE, PRINCIPAL_COMPONENT, INTEREST_COMPONENT,
15          FEE_COMPONENT, TOTAL_AMOUNT, SCHEDULED_ACTUAL_FLAG, BALANCE_AFTER_TRANSACTION, ROW_NOTE
16      )
17      WITH scope_accounts AS (
18          SELECT a.new_ac_number, a.sub_ac_number, a.customer_id, a.glcode, a.interest_rate,
19                 a.account_open_date, s.intdays_per_year
20          FROM acmaster a
21          JOIN scheme_master s ON s.scheme_mst_id = a.scheme_mst_id
22          WHERE UPPER(s.scheme_name) LIKE 'MAIIC%' OR UPPER(s.scheme_name) LIKE 'FINES%'
23      ),
24      ledger AS (
25          SELECT cv.cumvouch_det_id, cv.new_ac_number, cv.sub_account_no, cv.transaction_date,
26                 cv.trantype, cv.transamt,
27                 sa.customer_id, sa.glcode, sa.interest_rate, sa.intdays_per_year, sa.account_open_date,
28                 LAG(cv.transaction_date) OVER (PARTITION BY cv.new_ac_number, cv.sub_account_no
29                                                  ORDER BY cv.transaction_date, cv.cumvouch_det_id) AS prev_txn_date,
30                 SUM(cv.transamt) OVER (PARTITION BY cv.new_ac_number, cv.sub_account_no
31                                         ORDER BY cv.transaction_date, cv.cumvouch_det_id) AS running_balance_after
32          FROM cumvouch cv
33          JOIN scope_accounts sa ON sa.new_ac_number = cv.new_ac_number AND sa.sub_ac_number = cv.sub_account_no
34          WHERE cv.ac_glcode = sa.glcode
35                AND cv.delete_flag = 'N'
36      ),
37      actual_rows AS (
38          SELECT
39              l.customer_id,
40              l.new_ac_number,
41              l.sub_account_no,
42              l.glcode || '-' || l.cumvouch_det_id AS gl_posting_ref,
43              l.transaction_date,
44              CASE l.trantype
45                  WHEN 301 THEN 'Disbursement'
46                  WHEN 302 THEN 'Principal'
47                  WHEN 303 THEN 'Interest'
48                  WHEN 305 THEN 'Principal+Interest'
49                  WHEN 340 THEN 'Penalty'
50                  WHEN 341 THEN 'Penalty'
51                  WHEN 900 THEN 'Fee'
52                  WHEN 901 THEN 'Fee'
53                  ELSE 'Other/Adjustment'
54              END AS transaction_type,
55              CASE l.trantype
56                  WHEN 301 THEN ABS(l.transamt)
57                  WHEN 302 THEN ABS(l.transamt)
58                  WHEN 305 THEN ABS(l.transamt) -
59                      LEAST(ABS(l.transamt), ROUND(ABS(l.running_balance_after - l.transamt) *
60                                                         (l.interest_rate/100) *
61                                                         (l.transaction_date - NVL(l.prev_txn_date, l.account_open_date)) /
62                                                         l.intdays_per_year, 2))
63              ELSE NULL
64              END AS principal_component,
65              CASE l.trantype
66                  WHEN 303 THEN ABS(l.transamt)
67                  WHEN 305 THEN LEAST(ABS(l.transamt), ROUND(ABS(l.running_balance_after - l.transamt) *
68                                                         (l.interest_rate/100) *
69                                                         (l.transaction_date - NVL(l.prev_txn_date, l.account_open_date)) /
70                                                         l.intdays_per_year, 2))
71              ELSE NULL
72              END AS interest_component,
73              CASE l.trantype
74                  WHEN 900 THEN ABS(l.transamt)
75                  WHEN 901 THEN ABS(l.transamt)
76                  ELSE NULL
77              END AS fee_component,
78              ABS(l.transamt) AS total_amount,
79              'Actual' AS scheduled_actual_flag,
80              ABS(l.running_balance_after) AS balance_after_transaction,
81              CASE WHEN l.trantype = 305
82                  THEN 'Interest component ESTIMATED from amortization formula (rate x balance x days/day-count); not a stored
split.'
83                  ELSE NULL END AS row_note
84          FROM ledger l
85          WHERE l.transaction_date BETWEEN p_start_date AND p_end_date
86      ),
87      writeoff_rows AS (
88          SELECT
```

```

89      sa.customer_id,
90      w.new_ac_number,
91      w.sub_ac_number                                AS sub_account_no,
92      sa.glcode || '-WO-' || w.loan_writeoff_mst_id   AS gl_posting_ref,
93      w.writeoff_date                                AS transaction_date,
94      'Write-off'                                    AS transaction_type,
95      NULL                                            AS principal_component,
96      NULL                                            AS interest_component,
97      NULL                                            AS fee_component,
98      w.writeoff_amount                              AS total_amount,
99      'Actual'                                       AS scheduled_actual_flag,
100     NULL                                           AS balance_after_transaction,
101     'From LOAN_WRITEOFF_MASTER header - no matching LOAN_WRITEOFF_TRANS ledger detail exists for this facility.' AS
row_note
102     FROM loan_writeoff_master w
103     JOIN scope_accounts sa ON sa.new_ac_number = w.new_ac_number AND sa.sub_ac_number = w.sub_ac_number
104     WHERE w.delete_flag = 'N'
105           AND w.writeoff_date BETWEEN p_start_date AND p_end_date
106 ),
107 scheduled_rows AS (
108     SELECT
109         sa.customer_id,
110         f.new_ac_number,
111         f.sub_account_no,
112         sa.glcode || '-SCH-' || f.inst_chart_det_id   AS gl_posting_ref,
113         f.installment_date                            AS transaction_date,
114         'Scheduled Repayment'                        AS transaction_type,
115         f.prin_installment_amt                        AS principal_component,
116         f.intr_installment_amt                       AS interest_component,
117         NULL                                           AS fee_component,
118         f.installment_amt                             AS total_amount,
119         'Scheduled'                                   AS scheduled_actual_flag,
120         NULL                                           AS balance_after_transaction,
121         'Not yet realized - projected balance not computed (would require forward amortization).' AS row_note
122     FROM fixloan_inst_chart f
123     JOIN scope_accounts sa ON sa.new_ac_number = f.new_ac_number AND sa.sub_ac_number = f.sub_account_no
124     WHERE f.delete_flag = 'N'
125           AND (f.inst_recovered IS NULL OR f.inst_recovered < f.installment_amt)
126           AND f.installment_date BETWEEN p_start_date AND p_end_date
127 )
128 SELECT p_run_id, customer_id, new_ac_number, sub_account_no, gl_posting_ref, transaction_date,
129        transaction_type, principal_component, interest_component, fee_component, total_amount,
130        scheduled_actual_flag, balance_after_transaction, row_note
131 FROM actual_rows
132 UNION ALL
133 SELECT p_run_id, customer_id, new_ac_number, sub_account_no, gl_posting_ref, transaction_date,
134        transaction_type, principal_component, interest_component, fee_component, total_amount,
135        scheduled_actual_flag, balance_after_transaction, row_note
136 FROM writeoff_rows
137 UNION ALL
138 SELECT p_run_id, customer_id, new_ac_number, sub_account_no, gl_posting_ref, transaction_date,
139        transaction_type, principal_component, interest_component, fee_component, total_amount,
140        scheduled_actual_flag, balance_after_transaction, row_note
141 FROM scheduled_rows;
142
143 v_rowcount := SQL%ROWCOUNT;
144 COMMIT;
145
146 DBMS_OUTPUT.PUT_LINE('MAIIC_CASHFLOW_REGISTER built, RUN_ID=' || p_run_id ||
147                      ', range ' || TO_CHAR(p_start_date,'DD-Mon-YYYY') || ' to ' ||
148                      TO_CHAR(p_end_date,'DD-Mon-YYYY') || ': ' || v_rowcount || ' cash flows.');
```

```

149
150 EXCEPTION
151     WHEN OTHERS THEN
152         ROLLBACK;
153         RAISE_APPLICATION_ERROR(-20002,
154             'PROC_BUILD_MAIIC_CASHFLOW_REGISTER failed for range ' ||
155             TO_CHAR(p_start_date,'DD-Mon-YYYY') || ' to ' || TO_CHAR(p_end_date,'DD-Mon-YYYY') ||
156             ': ' || SQLERRM);
157 END PROC_BUILD_MAIIC_CASHFLOW_REGISTER;
158
159
160 -----
161 -- 3. Usage
162 -----
163 -- VAR v_run NUMBER
164 -- EXEC PROC_BUILD_MAIIC_CASHFLOW_REGISTER(TO_DATE('01-01-2025','DD-MM-YYYY'), TO_DATE('31-12-2026','DD-MM-YYYY'),
:v_run);
165 -- SELECT * FROM MAIIC_CASHFLOW_REGISTER WHERE RUN_ID = :v_run ORDER BY loan_account_number, transaction_date;
166
167 -----
168 -- 4. Known limitations
169 -----
170 -- - GL_POSTING_REF is a synthetic reference (GL code + internal row id), not a
171 --   dedicated journal/voucher number - CUMVOUCH.TXN_NUMBER was blank on the
172 --   rows checked, so it was not usable as the reference.
173 -- - "Principal+Interest" (TRANTYPE 305) rows carry an ESTIMATED split - see
174 --   ROW_NOTE on those rows.
175 -- - Write-off cash flows come solely from LOAN_WRITEOFF_MASTER; there is no
176 --   ledger (CUMVOUCH/LOAN_WRITEOFF_TRANS) posting to cross-check them against.
177 -- - BALANCE_AFTER_TRANSACTION is NULL for write-off and scheduled rows.
```

Tool call: Read

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\README.md

Result of Read

```

1  # MAIIC - project folder
2
3  Malawi Agricultural and Industrial Investment Corporation plc. Duplex Institute builds and supports their IFRS 9 system:
the expected credit loss (ECL) engine, and since July 2026 the effective interest rate (EIR) engine. Project sponsor: Dr
Thomson Kumwenda, CFO. ICT contact: Barry Makumba.
4
5  Tidied on 25 September 2026. Nothing was deleted; every move is listed in `..\_folder_reorganisation_2026-09-25\
manifest.csv`, and `undo.ps1` in the same place puts everything back.
6
7  ## Where things live
8
9  | Folder | What is in it |
10 |---|---|
11 | `1. Engagement Contracting` | The award letter, the signed contract and its reviews, the pitch, reference contracts,
and the emails that set the engagement up |
12 | `2. Documents from clients` | Everything MAIIC has sent us: loan books, database extracts, offer letters, trial
balances, annual reports and the signed accounts, the EIR assessment, meeting transcripts, logos, and the dated "New Doc
Received" drops |
13 | `3. Project Execution` | Our own working material: the specifications, the analysis, the reference rates, the meeting
packs, the VPN notes, a database backup, and the information requests we have sent |
14 | `4. Reporting & Deliverables` | What we have issued to the client |
15 | `5. Project Completion & Lessons Learnt` | Empty until the project closes |
16 | `6. Prior Engagements (ESG and 2025 ECL)` | The 2024 and 2025 work: the ESG policy engagement and the earlier ECL model
review, with their own contracts, deliverables, collateral files and database backups |
17 | `9. Archive` | Superseded material kept for reference |
18
19  ## Start here
20
21  - **The current meeting pack:** `3. Project Execution\Data Review and Meeting Preparation 24 Sep 2026` - open `READ ME
FIRST.md` in it. It holds the deck for the 25 September 2026 progress meeting, the engine specification, the field reference,
the extracts workbook and all the supporting analysis.
22  - **The specification:** `3. Project Execution\specs\MAIIC_EIR_Engine_Specification_v3_2026-09-24.pdf`. Section 3 is what
has been agreed, section 4 is what has not.
23  - **What we have asked MAIIC for, and why:** `3. Project Execution\Correspondence - Information Requests` holds the
emails themselves; the meeting pack's field workbook explains each request in plain language.
24  - **The code:** GitHub repository `MAICC-IFRS9`, branch `eir_revenue_recognition`. The specification and the branch
analysis are also committed there under `docs/`.
25
26  ## Two things worth knowing
27
28  1. Dates in several of the extracts were scrambled by Excel on the way out of the core banking system, not by the system
itself. The repaired reference-rate file and the list of rows for Barry to confirm are in `3. Project Execution\Reference Rates\
`.
29  2. The monthly Loan Book Report is the cleanest feed we have. If the vendor adds the missing columns to it, that single
monthly file becomes the whole integration.
30

```

Tool call: PowerShell

description: List request correspondence, meeting notes, specs and database extracts

--- command ---

```

$r='C:\Users\wadza\OneDrive\2026\Projects\MAIIC'; foreach ($d in '3. Project Execution\Correspondence - Information Requests',
'3. Project Execution\Data Review and Meeting Preparation 24 Sep 2026','2. Documents from clients\Meetings and Transcripts','3.
Project Execution\specs','2. Documents from clients\Database extracts','3. Project Execution\Database Backups') { "==" $d";
Get-ChildItem -LiteralPath ($r+$d) -Recurse -File | Sort-Object LastWriteTime -Descending | Select-Object -First 25
LastWriteTime, Length, @{n='Path';e={$_.FullName.Replace($r+$d+'\',')}} | Format-Table -AutoSize -Wrap -HideTableHeaders }

```

Result of PowerShell

== 3. Project Execution\Correspondence - Information Requests

```
24/09/2026 16:36:38 5792 Follow-up email after the EIR meeting (draft).md
15/09/2026 12:49:29 347017 RE_UPDATED_ MAIIC IFRS 9 consolidated information request, revised 11 September 2026
after your loan books and amortisation schedules. Replaces the version sent earlier
today.eml
14/09/2026 17:11:22 53810 MAIIC System Walk-through - Information Requests (11 September 2026) - Draft.eml
14/09/2026 10:19:06 182013 2026-09-11 MAIIC EIR - Consolidated Information Request UPDATED (to MAIIC).eml
14/09/2026 10:14:26 20474 2026-09-10 Ebanker loan books - November 2025 to August 2026 & Take-on amortisation
schedules (to Tamanda).eml
14/09/2026 09:59:23 3956771 RE_Ebanker loan books - November 2025 to August 2026 & Take-on amortisation schedules.eml
11/09/2026 13:27:44 156620 SUPERSEDED 2026-09-11 MAIIC EIR - Consolidated Information Request (to MAIIC).eml
11/09/2026 13:26:26 70535 2026-09-11 MAIIC EIR - Consolidated Information Request UPDATED (to MAIIC).html
11/09/2026 12:17:03 54785 SUPERSEDED 2026-09-11 MAIIC EIR - Consolidated Information Request (to MAIIC).html
```

== 3. Project Execution\Data Review and Meeting Preparation 24 Sep 2026

```
24/09/2026 17:06:15 93122 MAIIC_EIR_Engine_Specification_v3_2026-09-24.md
24/09/2026 17:06:15 719661 MAIIC_EIR_Engine_Specification_v3_2026-09-24.pdf
24/09/2026 16:56:48 99479 TB_to_AFS_what_it_tells_us_2026-09-24.pdf
24/09/2026 16:56:46 7485 TB_to_AFS_what_it_tells_us_2026-09-24.md
24/09/2026 16:07:20 8988 MAIIC_Fee_Import_Template_2026-09-25.xlsx
24/09/2026 13:29:00 345994 MAIIC_EIR_Agenda_Enhanced_2026-09-25_Revised_Status_Comments.pptx
24/09/2026 13:05:41 90666 Deck slide previews\Slide17.PNG
24/09/2026 13:05:41 76721 Deck slide previews\Slide14.PNG
24/09/2026 13:05:41 73353 Deck slide previews\Slide13.PNG
24/09/2026 13:05:41 84661 Deck slide previews\Slide11.PNG
24/09/2026 13:05:41 12719 PROGRESS.md
24/09/2026 13:05:22 329922 Supporting analysis\deck backup before the 24 Sep reports session was folded in.pptx
24/09/2026 12:47:50 8318 READ ME FIRST.md
24/09/2026 12:47:49 77777 Deck slide previews\Slide22.PNG
24/09/2026 12:47:49 68503 Deck slide previews\Slide21.PNG
24/09/2026 12:47:49 70059 Deck slide previews\Slide03_contents_updated.PNG
24/09/2026 12:47:49 90030 Deck slide previews\Slide23.PNG
24/09/2026 10:36:50 317196 Supporting analysis\deck backup before the three evidence slides were added.pptx
24/09/2026 09:43:12 315824 Supporting analysis\deck backup before notes were added on slide 3.pptx
24/09/2026 06:54:27 4974395 MAIIC_Extracts_Field_by_Field_2026-09-24.pdf
24/09/2026 06:51:47 87882 MAIIC_Extracts_Field_Reference_Workbook_2026-09-24.xlsx
24/09/2026 06:49:33 364752 Supporting analysis\MAIIC_Extracts_Research_Report_2026-09-24.pdf
24/09/2026 06:49:31 42788 Supporting analysis\MAIIC_Extracts_Research_Report_2026-09-24.md
24/09/2026 06:48:06 170394 Supporting analysis\Loan_Book_Date_Scan_2026-09-24.pdf
24/09/2026 06:48:05 194004 Supporting analysis\Extract_B_vs_Offer_Letters_2026-09-24.pdf
```

== 2. Documents from clients\Meetings and Transcripts

```
01/10/2026 13:11:40 14716 28 September Ebanker Walkthrough\Ebanker and LOS Refresher
Training-20260929_110757-Meeting Recording-en-US.docx
01/10/2026 12:02:26 235317473 28 September Ebanker Walkthrough\Ebanker and LOS Refresher
Training-20260929_110757-Meeting Recording.mp4
11/09/2026 11:47:02 22829 3 September- System Walkthrough with Vendor (Core Banking, EMS, LOS)\System
Walkthrough-20260903_144511-Meeting Recording-en-US.docx
11/09/2026 11:17:59 2105817 11 September - Amortisation Schedules\ECL Calculation - Amortization Schedules .docx
11/09/2026 06:51:54 42038 3 September- System Walkthrough with Vendor (Core Banking, EMS, LOS)\System Walkthrough
- Timestamped Transcript.pdf
11/09/2026 06:51:53 8909 3 September- System Walkthrough with Vendor (Core Banking, EMS, LOS)\System Walkthrough
- Detailed Summary.pdf
11/09/2026 06:51:53 5393 3 September- System Walkthrough with Vendor (Core Banking, EMS, LOS)\System Walkthrough
- Summary.pdf
09/09/2026 16:41:40 160888738 3 September- System Walkthrough with Vendor (Core Banking, EMS, LOS)\System
Walkthrough-20260903_144511-Meeting Recording.mp4
```

== 3. Project Execution\specs

```
24/09/2026 17:06:16 719661 MAIIC_EIR_Engine_Specification_v3_2026-09-24.pdf
24/09/2026 17:06:15 93122 MAIIC_EIR_Engine_Specification_v3_2026-09-24.md
24/09/2026 06:49:31 42788 MAIIC_Extracts_Research_Report_2026-09-24.md
24/09/2026 03:35:10 4755809 MAIIC_EIR_Field_Reference_2026-09-24.pdf
24/09/2026 03:35:07 6135549 MAIIC_EIR_Field_Reference_2026-09-24.docx
24/09/2026 03:18:43 8489 loan_book_date_scan_2026-09-24.md
20/08/2026 09:52:09 76379 MAIIC_EIR_Revenue_Recognition_Engine_Spec.md
05/08/2026 12:09:28 9781 MAIIC_EIR_Revenue_Recognition_Engine_Spec_Explained.pdf
05/08/2026 12:09:28 39048 MAIIC_EIR_Revenue_Recognition_Engine_Spec_Explained.docx
05/08/2026 12:06:50 8654 MAIIC_EIR_Revenue_Recognition_Engine_Spec_Explained.md
05/08/2026 11:47:55 22976 Development_of_EIR.md
05/08/2026 11:47:55 20696 EIR_Build.md
04/08/2026 15:59:55 30149 MAIIC_EIR_Revenue_Recognition_Engine_Spec_v1_2026-08-04.md
```

== 2. Documents from clients\Database extracts

```
10/09/2026 13:38:27 606957 Extract B.xlsx
26/08/2026 10:49:05 300069 ExtractC_Jan2025-July2026.xlsx
18/08/2026 13:43:00 67584 TB Extracts\MAIIC Agricultural Loans.xls
18/08/2026 13:43:00 98304 TB Extracts\GL Code_1050201 - FInES Agricultural Loans.xls
18/08/2026 13:43:00 137728 TB Extracts\MAIIC Industrial Loans.xls
18/08/2026 13:43:00 43520 TB Extracts\MAIIC Term Loan.xls
```

18/08/2026 13:43:00 111616 TB Extracts\TB Format 1.xls
18/08/2026 13:43:00 79872 TB Extracts\TB Format 4.xls
18/08/2026 13:43:00 146944 TB Extracts\GL Code 1050202 - FInES Industrial Loans.xls
18/08/2026 13:42:00 22528 TB Extracts\TB Format 14.xls
18/08/2026 13:42:00 22528 TB Extracts.xls
18/08/2026 13:41:38 771584 Fw Request October November December 2025 Trial Balance Data for Reconciliation.msg
04/08/2026 12:04:39 469504 RE Follow-Up Meeting on Data Extract Progress and Integration (Barry - extracts delivered
2026-08-03 1752).msg
04/08/2026 11:52:00 11648 Extract C.xlsx
04/08/2026 11:51:00 73489 Extract A.xlsx

== 3. Project Execution\Database Backups

10/09/2026 12:52:05 9010342 maiic_ifrs9.sql

Tool call: Read

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\specs\MAIIC_Extracts_Research_Report_2026-09-24.md

Tool call: Read

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Correspondence - Information Requests\Follow-up
email after the EIR meeting (draft).md

Result of Read

```
1  # Draft follow-up email after the EIR meeting
2
3  **To:** Dr Thomson Kumwenda; Tamanda; Dibwa; Barry Makumba
4  **Cc:** Kundai Muriwo; Wadzanai
5  **Subject:** MAIIC EIR: what is coming to us, and what is still open
6
7  ---
8
9  Dear all,
10
11  Thank you for this morning. It was the most useful session we have had, and it settled several things we had been
12  guessing at. Below is the short list, so that nothing rests on memory.
13
14  ## 1. Coming to us, as agreed in the meeting
15
16  | # | What | Who | When was said |
17  |---|---|---|---|
18  | 1 | The restructured loans register, the Excel that runs from inception | Tamanda | Straight after the meeting |
19  | 2 | The EMI charts for the ten sample loans | Barry | Today, before close of business |
20  | 3 | The signed offer letters for the same ten loans, showing the fees and the margin | Credit, Dibwa | With the
walkthrough |
21  | 4 | Extract B refreshed with the debit and credit columns | Barry, with the vendor | The query already exists |
22  | 5 | Extract C for all accounts to August 2026, with the accounts that were missing | Barry | A session to walk through
what arrived |
23  | 6 | The income statement mapping column on the trial balance workbook, as you did for the balance sheet | Dr Thom |
Being added |
24  | 7 | The Unpass Register, and the Batch Detail and Cash Book, as CSV | Tamanda | From yesterday's session |
25
26  ## 2. The walkthrough
27
28  Dibwa, Tamanda and Barry, about forty minutes, tomorrow. To get the most from it, please may we take one of the ten
29  sample loans and follow that single loan from beginning to end: the credit approval, where the amount, the rate, the
30  reference rate and the margin are entered, where the arrangement and legal fees are entered, how the schedule is
31  produced, how the offer letter is produced, what is changed by hand afterwards, the disbursement, the deduction of the
32  fees, and the posting to the ledger.
33
34  We will write down the screen and the field at each step. That record becomes the integration specification, so the
35  detail is worth the time.
36
37  ## 3. Still open, and why each one matters
38
39  | # | What we need | Why |
40  |---|---|---|
41  | 1 | Interest Policy for every loan account, with the Floating Flag, the interest calculation basis, the instalment
basis, the moratorium type and the moratorium period. The scheme code plus the settings once per scheme would do just as well |
This is the field that decides whether a loan follows the Reserve Bank rate. Without it we are inferring repricing from
behaviour |
42  | 2 | The Loan Book Report for every month from October 2024, or at least the months we do not have, which are
November 2024 to November 2025 | You confirmed go-live as October 2024 and that the report runs for past months. These fourteen
months are the hole in the middle of the 2024 to 2026 recalculation |
43  | 3 | The GL-wise Interest Posted Report, monthly from October 2024 | It is the source of Extract C and gives us the
interest side of those same months |
44  | 4 | The completed fee template, attached | Fees come off the top at disbursement, and they are what make an
effective rate differ from the rate on the letter. No system extract carries them against a loan, which is why we have made it
a spreadsheet |
45  | 5 | The materiality threshold used in the 2024 and 2025 EIR assessments | So the engine can reproduce your historic
conclusion as well as produce the detailed calculation. That continuity is what the auditors will look for |
46  | 6 | Written confirmation of which record is the contractual one: the signed offer letter, or the record in the
system | The signed letter carries the fees and the margin; the system schedule carries neither. For the effective interest
rate, this decides which terms we calculate on |
47  | 7 | Every future extract as CSV with dates written year first, for example 2026-09-25, and never opened and
re-saved in Excel first | Dates in the files we hold have been scrambled on the way out. We have repaired the rate file and can
hand back a corrected copy with the twenty rows for Barry to confirm |
48
49  ## 4. For Virtual Galaxy
50
51  Three items are for the vendor rather than for your team, and we suggest one conversation covering all of them.
52
53  - The Reschedule Report (2839) does not work. The download failed when we ran it.
54  - The IFRS9 Detail Report (3863) runs and downloads but returns no data at all.
55  - The Audit Trail Report (692) exports one client at a time, with no option to pull all accounts. We need either a
bulk export or a direct extract of the interest rate change history.
56
57  The fourth item is the change request we raised previously: adding the missing fields to the Loan Book Report, so that
58  the monthly file carries everything and the data requests stop.
59
60  ## 5. From us
61
62  Attached is the fee import template. It needs only three columns to work: the loan account number, the fee type and the
63  amount. The other columns help us tie your rows back to the ledger, so please fill them in where you have them. There is
64  a worked example row, and a sheet explaining each column.
65
66  We will keep building against what we already hold. For the record, four parts of the engine were completed this week
67  without needing anything from you: the governance settings, the reference rate series with the spread added to the prime
68  rate (margin) derived from your own loan books, interest charged the way your core system charges it, and the moratorium
69  shapes with the drawdowns.
70
71  One last point worth noting. You mentioned the 2026 EIR assessment has not been done because this system is replacing
72  it. That means the 2026 figure our engine produces will not be a check against another number. It will be the number. It
```

74 is the reason the fee file matters more than anything else on this list.
75
76 Kind regards,
77
78 Edward Mazibuko CA(SA)
79 Dupleix Institute
80

Result of Read

```
1  # MAIIC Data Extracts - Research Report
2
3  **Prepared:** 24 September 2026, for the meeting with Dr Thomson Kummwenda (CFO) and Barry Makumba (ICT)
4  **Purpose:** What we received, what is good, what needs clarifying, what is wrong, how E-Banker appears to be structured
underneath, and how our database should be adjusted to match it.
5  **Sources:** Extracts A, B, C (delivered 3 and 26 Aug 2026); loan books Dec 2025 to Aug 2026; take-on amortisation
schedules as at 31 Oct 2024; Interest Rate Change History (17 Sep 2026); E-Banker and LOS user manuals (21 Sep 2026); the 11
Sep 2026 consolidated information request; the `MAIIC-IFRS9` codebase as built.
6
7  ---
8
9  ## 0. Summary in one page
10
11  **Good news, and there is a lot of it**
12
13  - The three extracts prove that every field we need exists in the core system and can be queried. The account key
(`LOAN_ACCOUNT_NUMBER` + `SUB_ACCOUNT_NO`) joins all three cleanly.
14  - The monthly loan book is a genuine E-Banker report (Menu ID 3868) from December 2025 onward, and its `Repayments`
column is a true running total that ties exactly to the movement in `Outstanding Balance`. We can build the monthly cash and
arrears picture from it without waiting for anything else.
15  - The rate behaviour of the book is now proven from data, not assumed: MAIIC Agricultural and Industrial loans move with
the Reserve Bank rate exactly, month for month, eight months out of eight; FinES and Term Loans never move. From this we can
derive each floating loan's spread over prime (margin) ourselves.
16  - The take-on schedules are far richer than "schedules as issued": they are a complete hand-built amortisation engine
with daily accrual, dated rate steps, actual repayments, arrears and restructures. They cover the pre-migration life of about
100 loans.
17  - The manuals tell us the exact name, screen and page of every setting we have been asking for, which turns vague
requests into precise ones.
18
19  **Needs clarification**
20
21  - The one field that decides whether a loan follows the Reserve Bank rate - `Interest Policy` - has never been extracted.
What we were sent (`RATE_BASIS`) is a different field, `Floating Flag`, which was designed for deposits and contradicts
observed behaviour.
22  - The scheduled-repayment rows in Extract B carry no running balance at all, and for 10 of 90 loans every scheduled row
carries the same date. They cannot be used as a schedule as delivered.
23  - `NUM_INSTALMENTS` in Extract A is always the tenor in months plus one (37 for a three-year loan). We need to know
whether that counts the disbursement row or is an off-by-one.
24  - Account status codes `H` (42 rows) and `F` (6 rows) are undocumented.
25  - Every loan in Extract A shows exactly one disbursement tranche, yet the take-on schedules show loans drawn in two and
three parts. The tranche history is not coming through.
26  - `SUB_ACCOUNT_NO` is `1` on all 362 rows, although the vendor told us a restructure creates a new sub-account, and
`RESTRUCTURE_DATE` is blank everywhere.
27  - We hold no core-banking loan book for November 2024 to November 2025 - fourteen months of the period the CFO has asked
us to recalculate.
28
29  **Errors we found**
30
31  - **Dates are corrupted in every extract**, not just the rate file: 1,134 business-critical dates have day and month
transposed (Extract A 376, Extract B 686, rate file 20, Dec-2025 assessment 52). It is an Excel locale artefact on the way out,
not a core-banking fault. Extract A and B need re-supplying with ISO dates.
32  - Extract B has no debit/credit indicator and no negative values, so the direction of every transaction is unstated. 539
of its 2,478 rows say their principal/interest split was estimated by a formula rather than read from the system.
33  - The first Extract C covered 3 accounts. The re-run covers 120 of 181.
34
35  ---
36
37  ## 1. Inventory - what we hold
38
39  | Dataset | Received | Grain | Rows / accounts | Period |
40  |---|---|---|---|---|
41  | Extract A - facility master | 3 Aug 2026 | one row per account per snapshot | 362 rows / 181 accounts | snapshots at 1
Jan 2025 and 31 Dec 2025 |
42  | Extract B - transactions | 3 Aug 2026 | one row per cash flow (actual or scheduled) | 2,478 rows / 114 accounts |
calendar 2025 only |
43  | Extract C - interest posted | 26 Aug 2026 (re-run) | one row per account per month | 3,996 rows / 120 accounts | Jan
2025 to Jul 2026 |
44  | Loan Book Report (Menu ID 3868) | 11 Sep 2026 | one row per account | ~134 to 146 per month | Dec 2025 to Aug 2026 |
45  | Finance's own Excel loan books | earlier | one row per loan, serial IDs, no account numbers | 77 to 99 per month | Jan
2024 to Oct 2025 |
46  | Take-on amortisation schedules | 11 Sep 2026 | one block per loan, one row per period | 103 blocks / 3,875 rows | first
drawdown Jul 2020 out to Mar 2034 |
47  | Interest Rate Change History | 17 Sep 2026 | one row per rate event | 48 rows (26 real changes) | Mar 2020 to Sep 2026 |
48  | Dec-2025 EIR assessment | 19 Aug 2026 | one row per facility with fees | 67 facilities | FY2025 |
49  | E-Banker and LOS manuals | 21 Sep 2026 | - | 84 + 153 pages | - |
50
51  ---
52
53  ## 2. Extract by extract
54
55  ### 2.1 Extract A - facility master
56
57  **What it is.** Thirty-six columns, one row per loan account at each of two snapshot dates. Effectively the loan account
master table joined to the customer master and a balance snapshot.
58
59  **Positive**
60  - Clean, stable keys: `CUSTOMER_ID`, `LOAN_ACCOUNT_NUMBER` (15 digits), `SUB_ACCOUNT_NO`, `GL_ACCOUNT_CODE`.
61  - Product-to-GL mapping is one-to-one and consistent: MAIIC Agricultural 1050101, MAIIC Industrial 1050102, MAIIC
Investment 1050103, FinES Agricultural 1050201, FinES Industrial 1050202, FinES Investment 1050203, all MAIIC Term Loans
```

1050401.

62 - Both grace fields are populated on all 362 rows (`PRINCIPAL\_GRACE\_PERIOD`, `INTEREST\_GRACE\_PERIOD`) and, read together, tell us the moratorium shape.

63 - `COMPOUNDING` (all Compound) and `DAY\_COUNT\_BASIS` (365 on 336 rows, 360 on 26) are stated explicitly. This settled our day-count decision.

64 - `REPAYMENT\_FREQUENCY` is present on 337 of 362 rows (Monthly 320, Half-Yearly 10, Quarterly 4, Yearly 3).

65 - `OPENING\_BALANCE` is populated on 294 rows with a note explaining the rest ("closed before as-of date").

66

67 ****Needs clarification****

68 - `RATE\_BASIS` (Fixed 158 / Variable 204) is the ****Floating Flag****, not the field that controls repricing. See section 4.

69 - `NUM\_INSTALLMENTS` is tenor-in-months plus one on every row we checked (37 / 73 / 25 / 49 against 3y / 6y / 2y / 4y). Does it count month zero, or is it an off-by-one in the query?

70 - `ACCOUNT\_STATUS` values: Active 172, Closed 92, Dormant 50, ****H 42, F 6****. H and F are not in either manual.

71 - `DISBURSEMENT\_TRANCHES` is a delimited string (`24-Jan-2023:90900000.00`) and holds exactly one entry on all 314 rows where it is populated. Yet the take-on workbook shows several loans drawn in two or three tranches. Either the query takes only the first, or E-Banker books later drawdowns differently.

72 - `SUB\_ACCOUNT\_NO` = 1 on every row; `RESTRUCTURE\_DATE` blank on every row. Restructure lineage is being lost on extraction.

73 - One FinES Investment loan carries GL code 3065, which is not a 105xxxx loan code. Probably an equity or preference-share instrument - it should be excluded, and the engine already refuses equity.

74 - `REPAYMENT\_FREQUENCY` is blank on 25 rows.

75 - Grace periods arrive as strings in two formats: ``0`` and ``0 M``, ``3 M``, ``6 M``. Trivial, but the importer must normalise it.

76 - ****No fee fields of any kind.**** The single biggest gap, already request 4 of 11 Sep.

77

78 ****Errors****

79 - Five date columns corrupted (day/month transposed on 376 business-critical values): `LOAN\_START\_DATE` 142, `FIRST\_REPAYMENT\_DATE` 74, `CONTRACTUAL\_MATURITY\_DATE` 108, `ACTUAL\_CLOSURE\_DATE` 52. Plus all 362 `GENERATED\_ON` timestamps.

80

81 **### 2.2 Extract B - transactions**

82

83 ****What it is.**** Sixteen columns. Actual postings to the loan (`Actual`, 1,703 rows) mixed with projected future instalments (`Scheduled`, 775 rows). Types: Interest 1,034; Principal+Interest 539; Scheduled Repayment 775; Disbursement 97; Other/Adjustment 31; Fee 2.

84

85 ****Positive****

86 - `GL\_POSTING\_REF` (1050102-36994) is GL code plus posting number - a genuine ledger reference, which is exactly the audit trail Deloitte will want.

87 - 97 disbursement rows: the actual cash-out events, with dates. This is the origin of every EIR.

88 - 1,034 interest rows and 539 combined rows give us actual cash received per loan per month for 2025, the only year we have transaction-level data for.

89 - The `Scheduled` rows prove the system holds a forward schedule per loan with a principal/interest split.

90

91 ****Needs clarification****

92 - ****The scheduled rows are not usable as a schedule.**** `BALANCE\_AFTER\_TRANSACTION` is empty on 100% of them ("Not yet realized - projected balance not computed"), and on 10 of the 90 loans every scheduled row carries the identical date. We need to know whether the EMI chart is stored as a table or generated on demand - the manual's wording ("click on Show button... report is display on screen", p.55; "view EMI chart before posting", p.63) suggests it is computed on the fly from parameters, not persisted.

93 - No `DR\_CR\_INDICATOR` and ****no negative values anywhere****, so cash direction is unstated. A disbursement and a repayment both appear as positive numbers.

94 - 539 rows (22%) carry a note saying the principal/interest split was "ESTIMATED from amortization formula" rather than read from the system - yet the EMI Chart Wise Data report holds that split as stored data.

95 - Only 2 fee rows in 2,478. Fees are being captured somewhere else (Finance keys them manually, per the 11 Sep note).

96 - Nothing after December 2025.

97

98 ****Errors****

99 - `TRANSACTION\_DATE` corrupted on 686 of 2,478 rows (28%). All 2,478 `GENERATED\_ON` values too.

100

101 **### 2.3 Extract C - interest posted by loan**

102

103 ****What it is.**** Fourteen columns, one row per account per month: `INTEREST\_INCOME\_POSTED`, `TRANSACTION\_COUNT`, `POSTING\_REFERENCES`, keyed on account, sub-account, GL code, period.

104

105 ****Positive****

106 - The re-run (Jan 2025 to Jul 2026) covers 120 accounts and 3,996 rows across all five loan GL codes.

107 - The `ROW\_NOTE` is unusually informative: ****Net of TRANTYPE 303/120/308/309/310/311 postings to the loan's own GL this month.**** That gives us six E-Banker transaction-type codes and tells us the figure is netted at loan level. This is the contractual-basis interest the GL actually recognised - the "what MAIIC did" side of the reconciliation.

108 - No date corruption (dates are held as year/month integers).

109

110 ****Needs clarification****

111 - 120 of 181 accounts. Which 61 are missing, and why - closed, dormant, or never posted interest?

112 - No fee or commission column, so arrangement and legal fee income (GL 4871/4873) cannot be tied to a loan from this extract.

113 - Nothing for August 2026.

114 - What each of TRANTYPE 303, 120, 308, 309, 310, 311 means. We need the code list.

115

116 ****Errors**** - none found in the re-run. The original delivery (28 rows, 3 accounts) was simply incomplete.

117

118 **### 2.4 Loan Book Report (Menu ID 3868), Dec 2025 onward**

119

120 ****Positive****

121 - Twenty-nine columns including `A/C Number`, `Customer ID`, `National ID`, `Value Date`, `Maturity Date`, `Moratorium (Months)`, `Interest Rate`, `Approved`, `Disbursed`, `Not Yet Disbursed`, `Repayments`, `Carrying Amount`, overdue split and four ageing buckets.

122 - The `Tenure` column carries the moratorium type as a tag: `(Cap\_only)` 42 accounts, `(Cap+Int)` 92, matching LOS's two options exactly.

123 - `Repayments` is cumulative and self-validating against `Outstanding Balance`. Monthly cash can be derived.

124 - `Not Yet Disbursed` totals MWK 3,472,435,397 across eight facilities - the undrawn-commitment population for IFRS 9.

125

126 ****Needs clarification****
127 - On 26 of 110 active accounts the `Repayments` running total drops to zero between months (MUSECO 677m to nil, Lenziemill 569m to nil). Settlement, restructure or re-booking?
128 - Loan type appears only as a section heading (`Loan Type : 1050101-MAIIC Agricultural Loans`), not as a column.
129 - No repayment frequency, no reference rate or margin held separately, no sub-account, no restructure date, no fees.
130
131 ****Errors**** - none. Scanned on 24 Sep 2026 (`loan\_book\_date\_scan\_2026-09-24.md`): all 2,018 Value Date and Maturity Date cells across the nine `.xls` files are dd/mm/yyyy text, 0 Excel datetime cells, 0 accounts whose date changes between months, 0 wrong dates. This family never went through Excel's locale conversion and does not need re-supplying; the loader must simply parse the strings day-first.
132
133 **### 2.5 Finance's Excel loan books, Jan 2024 to Oct 2025**
134
135 Not core-banking output. Seventeen to nineteen columns, serial `customer\_id` (1, 2, 3...), no account numbers, and IFRS 9 working fields (`internal\_grade`, `industry\_code`, `carrying\_amount`, ageing buckets). Same lineage as the take-on schedules. Useful as a cross-check, but cannot be joined to E-Banker except by name.
136
137 **### 2.6 Take-on amortisation schedules, as at 31 Oct 2024**
138
139 ****Positive**** - 103 loan blocks with daily accrual on a 365 basis, dated rate-change steps (3 to 6 per loan), actual repayments, accumulated arrears, and twin parameter blocks on the four restructures. This is the pre-migration history for the legacy book.
140
141 ****Needs clarification**** - only 4 of 103 blocks carry a loan name and none carries an account number. Is there a mapping? Do the 103 cover the whole pre-migration population (Finance's Oct 2024 book has 79 rows, Extract A has 181 accounts)?
142
143 **### 2.7 Interest Rate Change History**
144
145 ****Positive**** - 26 genuine rate changes from 4 Mar 2020 to 3 Sep 2026, range 12.0% to 25.4%, with a free-text `Particulars` column that shows most changes were actioned on a request from Credit.
146
147 ****Errors**** - 20 of 48 effective dates transposed. Repaired copy built and validated (zero chronological breaks after repair, eleven before). Barry to confirm the 20 rows. The column names (`SrNo`, `Appli From Date`, `Active`) are raw database columns; there is no such report in the manual, so this was a one-off query.
148
149 ---
150
151 **## 3. How E-Banker is structured, as far as we can tell**
152
153 Nothing below is documented as a schema. It is inferred from the manuals' screen hierarchy, the extract column names, and the GL posting references.
154
155 **### 3.1 The hierarchy: Product, Scheme, Account**
156
157 The E-Banker manual is organised as ****Product Creation** (section 2, p.7) - **Scheme Master** (section 3, p.17 and p.26) - **Account transactions** (sections 6-7)**. So:
158
159 - A ****Product**** is the top level (a loan product type, mapped to a GL code: 1050101, 1050102...).
160 - A ****Scheme**** is a variant of a product with its own configuration. ****This is where `Interest Policy`, `Floating Flag`, `Loan Interest Cal. Base On`, `Installment Based On` and `Moratorium Period` live**** (pp.19-31 are the Scheme Master screens).
161 - An ****Account**** (`LOAN\_ACCOUNT\_NUMBER`) is opened on a scheme and inherits its settings. Sub-accounts (`SUB\_ACCOUNT\_NO`) hang off the account - the vendor said a restructure creates a new one.
162
163 ****Consequence for us:**** "Interest Policy per account" really means "which scheme is each account on, and what is that scheme's Interest Policy". Asking for the ****scheme code on the loan book**** would let us hold the scheme settings once and inherit them, exactly as E-Banker does. That is a cleaner ask than repeating six settings on every account row.
164
165 **### 3.2 The tables we can infer**
166
167 | Inferred table | Evidence | What it holds |
168 |---|---|---|
169 | Customer master | `CUSTOMER\_ID`, `CUSTOMER\_NAME`, `National ID` | one row per customer |
170 | Product master | section 2.1; `GL\_ACCOUNT\_CODE`, `PRODUCT\_TYPE` | product to GL code |
171 | Scheme master | section 3.1 / 3.5; the dropdowns on pp.19-31 | interest policy, floating flag, calc base, instalment basis, moratorium min/max |
172 | Loan account master | Extract A columns | account, scheme, customer, dates, sanction, tenor, frequency, grace, status |
173 | Sub-account | `SUB\_ACCOUNT\_NO`; Reschedule Report (Menu ID 2839) | restructure lineage |
174 | Disbursement | `DISBURSEMENT\_TRANCHES`; section 7.2 (p.60) | date and amount per drawdown |
175 | Transaction ledger | Extract B; `GL\_POSTING\_REF`; TRANTYPE codes 303/120/308/309/310/311 | every posting, with GL reference |
176 | EMI chart | "Show" button p.55; section 7.2 p.63; Extract B `Scheduled` rows | the forward schedule - ****possibly computed, not stored**** |
177 | Interest slab / PLR master | `Interest Policy = Slab / Link with PLR`; the rate change file's `SrNo`, `Appli From Date`, `Active` columns | reference rates by effective date |
178 | Auto-post charges | section 4.1 (p.35) | fee and charge rules |
179 | GL | Trial Balance (Menu ID 557); Extract C | the ledger the loan module posts into |
180
181 **### 3.3 The LOS to E-Banker handoff**
182
183 LOS runs Proposal Initiation → Application Entry → Enrichment → Underwriting → Assessment → ****Offer Issue** (p.112, PDF only, with or without schedule)** → Offer Accept/Reject → ****Account to be Created in LMS (CBS)**** (p.129)**. That last step is the handoff to E-Banker. ****Application No.` and `Process Ref. No.` are the only keys that survive it.**** The fees (Agreement Fee, Legal Fee, Other Fee on the Facility Sought tab, pp.79-80) and the reference rate and margin (Funding Proposal tab) are captured in LOS and printed on the offer letter only partially.
184
185 **### 3.4 Why the labels looked wrong**
186
187 `RATE\_BASIS` in Extract A is derived from the Floating Flag, whose `Fixed (I)` option is documented as ****applicable for Time Deposit Account**** (p.20). It was designed for the deposit side. The field that governs loan repricing is `Interest Policy`, which has never been extracted. Neither label is wrong; we were sent the wrong field.
188

189 ---
190
191 ## 4. The columns we are asking for, one by one, in plain language
192
193 Each entry: what it is, what it decides, where it lives (manual page and screen), and why we need it. Menu IDs are given where the manual has one; the scheme screens (pp.17-31) are not numbered in the manual's Menu ID scheme.
194
195 ****Interest Policy**** - E-Banker p.19 (Scheme Master Opening, section 3.1) and p.27 (Scheme Master Opening for Loan Account, section 3.5)
196 A dropdown that says where the loan's interest rate comes from. Nine options; the ones that will appear on MAIIC's book are ****Link with PLR (P)**** - the rate follows the Reserve Bank prime rate and moves with it automatically; ****Fixed (F)**** - "interest policy assigned will never change"; and ****Manual (M)**** - a person types the rate in and changes it by hand. Without it we cannot tell which loans must be recalculated each time the Reserve Bank moves.
197
198 ****Floating Flag**** - E-Banker pp.19-20 and p.28
199 A second, separate setting: Not applied (N), Floating (F), Fixed (I, documented for time deposits), Both (B). This is what `RATE\_BASIS` currently reports. We want it alongside Interest Policy so the two can be compared.
200
201 ****Reference rate % and the spread added to the prime rate (margin) %**** - LOS Funding Proposal tab
202 The two halves of a floating rate, held separately. The loan book shows only the blended total. Without the split a floating loan cannot be repriced. ****Note:**** the word "margin" in the manuals means collateral margin (LOS p.63) or Margin Period (E-Banker pp.23, 31). Always ask for "the spread added to the prime rate (margin)".
203
204 ****Loan Interest. Cal. Base On**** - E-Banker p.30
205 Options: N - not applied, ****S** - sanction amount wise, ****B** - balance wise. In plain words: is interest charged on the full approved amount, or only on what the customer has actually drawn? On a loan that is only part-drawn the two give very different figures - Lake Malawi Aquaculture is approved for 1,055m and has drawn 297m; at 34.75% the difference is about MWK 22 million a month. Nothing in the output looks wrong either way. We need the setting per loan.
206
207 ****Installment Based On**** - E-Banker p.22
208 Options: sanction amount / disbursement amount. The same question again, but for the size of the instalment rather than the interest. A separate setting - the two can differ on the same loan.
209
210 ****Moratorium Period**** - E-Banker p.31 (scheme minimum/maximum) and the loan book `Moratorium (Months)`
211 The manual's definition, verbatim: "used in start of Term Loans repayment from the date of first Disbursement. If loan is disbursed in the Month of May but the installment will start from the Month August, then this period is defined as Moratorium period." So it is measured from first disbursement, not from approval.
212
213 ****Moratorium Type**** - LOS p.76 (Repayment Schedule tab, Table 58)
214 Verbatim: "Available options are as Principle Only and Both (Interest + Principle)." Principle Only = the customer keeps paying interest but not capital. Both = the customer pays nothing and the interest is added to the loan. Exactly two options; the system cannot express an interest-only moratorium. Already visible in the loan book as `(Cap\_only)` / `(Cap+Int)`.
215
216 ****Grace Period (In Month)**** - LOS p.77
217 A third, separate deferral field on the same screen as Moratorium. We need to know how it interacts with Moratorium.
218
219 ****Repayment Frequency**** - LOS p.76: Monthly, Quarterly, Half Yearly, Yearly
220 ****Type of Repayment**** - LOS p.76: EMI, lump sum (and others the host supports)
221 ****Interest Start Date**** - LOS p.76: when interest begins running, which can be before the first instalment
222 ****Installment Date**** - LOS p.77: the first instalment date; `Maturity Date` is auto-calculated from it
223
224 ****Sub Account Number**** - E-Banker pp.10-12; Reschedule Report Menu ID 2839
225 The suffix that identifies a restructured arrangement on the same main account. Today it is `1` everywhere, so the link between an old arrangement and its replacement is lost on the way out.
226
227 ****Restructure Date**** - Reschedule Report Menu ID 2839
228 When the terms changed. Blank on every Extract A row.
229
230 ****Application No. and Process Ref. No.**** - LOS p.13, pp.34-41, p.129
231 The origination system's references. They are the only link from a core-banking account back to the offer letter, the fees and the rate build-up in LOS.
232
233 ****Arrangement / Legal / Other fee, amount and date**** - LOS pp.79-80 (Facility Sought tab); posted to GL 4873 / 4871 / 4872; Auto Post Charges section 4.1 (p.35)
234 Deducted from the amount advanced. The single biggest reason the effective rate differs from the contractual rate.
235
236 ****Account Status**** - E-Banker p.14; Loan Inquiry Menu ID 542 (p.55)
237 Active / Closed / Dormant are self-explanatory. `H` and `F` are not documented; we need the code list.
238
239 ****Funded / Non-funded**** - LOS p.74
240 Guarantees and letters of credit are not amortised-cost loans and must be excluded before any rate is solved.
241
242 ****Int. Calc. Freq / Interest Freq / Int. Calc. Flag**** - E-Banker pp.20-22
243 How often interest is calculated, how often it is charged, and whether it accrues at all. Daily-calculated-monthly-charged produces a different schedule from monthly-calculated.
244
245 ****Disbursement (per drawdown: date, amount, reference)**** - E-Banker section 7.2 "Transaction for Disbursement Transaction", p.60; Finance Team Pending Disbursement Forward, section 6.2, p.52
246 Cannot be a loan-book column because one loan can have several. Needs its own small extract.
247
248 ---
249
250 ## 5. Can the extracts reproduce a repayment schedule?
251
252 The LOS schedule has five columns: date, instalment, principal, interest, outstanding balance (LOS p.77). Extract B's `Scheduled` rows carry four of them (`TRANSACTION\_DATE`, `TOTAL\_AMOUNT`, `PRINCIPAL\_COMPONENT`, `INTEREST\_COMPONENT`) but ****not** the balance, and on 10 of 90 loans not the dates either.
253
254 ****Verdict:**** not as delivered. But we do not actually need the system's schedule as a table. Given the parameters that the loan-book extension would carry (sanction, drawn, rate, frequency, tenor, moratorium period and type, first instalment date, calculation base) plus the EMI type from p.30 (`E` - EMI based, `P` - principal based, `F` - flat EMI based), ****our generator**

reproduces the schedule and the system's own scheduled rows become the check against it**, not the source. That is the right way round for audit purposes: two independent constructions that must agree.

255

256 What we still need for that: the `EMI Calc Type` (p.30) per scheme, and a straight answer on whether the EMI chart is stored or computed.

257

258 ---

259

260

261 ## 5A. Reconstruction test - the 10 offer letters vs the GL (2026-09-24) - THE HEADLINE RESULT

262

263 All 10 offer letters link to E-Banker accounts (by name, then sanctioned amount): L01 Ebenezer 104430000084 · L02 GOJET 104450000098 · L03 JAT 104430000087 · L04 JVD 104460000120 · L05 Lake Malawi 106050000009 · L06 Police SACCO 106050000006 · L07 Microloan 104460000116 · L08 Milele 106050000007 · L09 Mphunzitsi 106050000014 · L10 Promenade 106050000004. Eight of the ten have Extract C interest postings (47 account-months); L02 and L09 originated after Jul 2026.

264

265 **E-Banker's interest convention is now PROVEN TO THE CENT:** posted interest = **prior month-end outstanding balance x annual rate x days in month / 365**. Exact matches (difference < MWK 0.20): JVD Apr-Jul 2026 (4,692,858.90 vs 4,692,859.03 ...), Lake Malawi May-Jul, Microloan Feb-Jul, Police SACCO May-Jun. 32 of 47 months within 2% on actual/365; **rate/12 (30/360) is WRONG** - off 8-9% in February (ratios 0.91-0.92). Confirms the day-count decision with evidence.

266

267 **Capitalising moratorium is MONTHLY:** on `Both`-type loans in grace (JVD, Lake Malawi, Microloan) the outstanding balance rises each month by exactly the interest posted (JVD 570.96m -> 575.66m -> 580.55m -> 585.32m). Interest is posted and added to principal at every month-end, compounding monthly. **Settles governance setting 5 with evidence.**

268

269 **The 15 non-matching months are all explainable, and each one is a lesson:**

270 - First-month partials (JVD Mar 0.03, Lake Malawi Apr 0.10, Milele Mar 0.22): loan disbursed late in the month, interest for a few days only. We need the disbursement DATE, not just the month.

271 - Catch-up postings (Police SACCO Feb 1.46; **Promenade Feb 4.09** = ~4 months' interest posted at once, and the balance jumped 425.8m -> 471.8m by exactly that amount): interest accrued for several months then posted and capitalised in one go. Ask why - delayed activation of the interest run?

272 - Milele (ratios 1.10 / 1.05 / 1.42 / 0.85) with balance 262m -> 534m: **tranche disbursements within the month**. Prior-month-end balance x rate cannot match when money is drawn mid-month. Proves the per-drawdown extract is needed for exactness.

273 - Ebenezer / JAT Jul 2026 (0.96 / 0.99): small drift, probably the instalment date vs month-end.

274

275 **What it means for the engagement:** our generator can reproduce E-Banker's own interest postings from the loan book plus the rate, without the transaction file, on a normal month. Where it cannot, the gap is diagnostic (disbursement date, catch-up, tranche). This is the demonstration for Dr Thom: "we took your offer letters, rebuilt the interest, and it matches your GL to the cent."

276

277 **The 31 October 2024 date on the mislabelled Reschedule figure:** that figure is a real `GLwise Interest Posted Report`, Post Date 31-Oct-2024, for GL 1050101 MAIIC Agricultural Loans, 19 rows of REAL accounts (Nsanama 000104420000006, Lenziemill ...007, Mach Milk ...009, Kajikhomele, Nature's Gift, Happie Foods, Hortinet ...) with interest for 01/10/2024-31/10/2024. **So E-Banker was posting production interest in October 2024** - go-live was on or before 1 Oct 2024, and 31 Oct 2024 (the take-on date) was the first month-end in the new system. (The Loan Book screenshot "AsOn 15/03/2024" is then either a test run or a parallel-run - ask.)

278

279 Working file: `3. Project Execution/Amortisation Analysis/offer\_letter\_reconstruction\_test\_2026-09-24.csv`.

280

281 ### 5B. The three MAIIC reports, corrected

282

283 - **Audit Trail Report (692)** is a field-level change log (table, column, old value, new value, entered by, passed by, timestamp). Run it on the loan account interest-rate field and on the scheme `Interest Policy` field: it gives the rate history from the system of record and shows whether each change was made by a person or by the system.

284 - **Reschedule Report (2839)**: the manual's figure is mislabelled (it shows the GL-wise Interest Posted Report). Columns unknown - request a sample run. By name it is the restructure population for modification accounting and the source of the blank `RESTRUCTURE\_DATE`.

285 - **Loan Book Report (3868)** takes an `AsOn Date` parameter, so it can be re-run for past months.

286

287

288 ## 5D. Extract B against four of the offer letters (2026-09-24)

289

290 A second check took Extract B, the 2025 transaction ledger, for four of the ten offer letters (Ebenezer Midian, JAT Group, Microloan Foundation, Promenade Medical Centre) after repairing its dates (775 of 2,478 transaction dates were Excel datetimes with day and month swapped; 11 of the 39 rows for these four accounts). Full tables: `extract\_b\_offer\_letter\_check\_2026-09-24.md` and `.csv` in `Amortisation Analysis`.

291

292 **What ties.** The approved amount reaches the core account to the kwacha on all four (314,900,900 in five tranches; 209,336,236 in two; 400,000,000 in four; 470,795,000 in one). The balance chain in Extract B is internally consistent on every row. Where a first-month interest posting exists it equals the disbursed amount x annual rate x days from the disbursement day, inclusive, / 365, to the cent (Ebenezer Midian: 314,900,900 x 31% x 24/365 = 6,418,801.91 against 6,418,801.92 posted; Microloan Foundation on 14 days, exact). No fee rows and no fees on the letters, so fees are consistent (nil both sides).

293

294 **What does not tie, and what it means.**

295

296 | Finding | Evidence | Consequence |

297 | ---|---|---|

298 | The offer-letter schedules start interest on the 1st of the month; E-Banker starts on the disbursement day | Ebenezer Midian schedule charges 1-7 Aug the system never charged; Microloan 1-17 Dec | The LOS schedule is not the cash flow the core account produces. Expected cash flows must be generated from the core disbursement date, with the schedule kept as reference |

299 | Two of four loans carry a different rate in the core from the letter | Ebenezer Midian 31% in the core (the schedule rate) against 25.1% typed on the letter; JAT Group 33% in the core against 25.3% on both letter and schedule | Ask why. For JAT the core rate is not on any document we hold |

300 | JAT Group's two disbursement rows are dated before its own application | 4 Aug and 5 Sep 2025 against an application of 23 Sep and approval of 30 Sep; GL references and the one-day September interest show the account went live around 30 Sep | Disbursement value dates in Extract B can be back-dated; the posting reference sequence is the safer clock |

301 | Promenade Medical Centre had no interest posted at all in 2025 | Disbursed 13 Nov 2025; nothing in Nov or Dec; the only movement is a 25,000,000 repayment on 30 Dec | Explains the February 2026 catch-up posting seen in the reconstruction test |

302 | Repayments do not follow the schedules | Ebenezer paid 5,220,495.30 twice on 12 Sep, 16,731,448.80 on 1 Oct, 10,440,

990.72 on 29 Oct against a level instalment of 10,382,383 on the 1st | Actual cash must come from the loan book or the ledger, never from the schedule |

303 | Later months on Ebenezer Midian land at an implied 29 to 31 percent and do not reproduce under either a month-end or a daily reading at 31% | Sep to Dec 2025 | Needs the system's own accrual log (Audit Trail Report) rather than more arithmetic on our side |

304

305 ## 5C. Standard reports to run first - "STANDARD REPORTS WE WANT TO SEE RUN" (2026-09-24)

306

307 Framing for the slide: *Before we ask for anything new, we want to run the reports E-Banker already has. Some of what we need may already be sitting in them. For each one: who can run it, has it been run before, and what it gives us.*

308

309 Standing questions to ask on EVERY report: **Who has the menu rights to run it? Has it ever been run in production? Can it be exported to CSV/Excel rather than PDF? Can it be run for a past date?*

310

311 | # | Report | Menu ID | Manual | What it gives | Which of our questions it answers | Use in the build |

312 |---|---|---|---|---|---|---|

313 | 1 | **Audit Trail Report** | **692** (Report ID 248) | E-Banker p.75 | Field-level change log: table (Option Name), column, old value -> new value, entered by, passed by (maker-checker), timestamp, IP, account/sub-account, From/To date filter | Rate history per loan (req 7) from the system of record; which loans are on `Manual` vs `Link with PLR`; PLR master history with who-changed-what (the rate file's `APPLI\_DATE` naming matches its `CURRENCY\_RATE\_MASTER` sample); any change to `Interest Policy` on a scheme | Populates `rate\_reset\_events` and `reference\_rate\_series` with audited provenance; control evidence for Deloitte. **Ask to run for: loan account master / interest-rate column / all accounts / 1 Jan 2024 to date; scheme master / Interest Policy; the PLR rate master.** |

314 | 2 | **Reschedule Report** | **2839** | E-Banker p.75-76 (figure mislabelled - shows the GL-wise Interest Posted Report instead) | Unknown columns. By name: loans whose terms were changed, with dates | The restructure population (req 12); the `RESTRUCTURE\_DATE` blank on every Extract A row; sub-account lineage | Modification accounting under IFRS 9 5.4.3 - the `Modification` sheet of the workbook, per loan. **Ask for a sample run, all restructures since 1 Jan 2024.** |

315 | 3 | **IFRS9 Detail Report** | **3863** | E-Banker p.76 | 36+ columns; visible 25-36: tenure/terms, interest rate, **interest compound (monthly/semi-annual/annual)***, loan series, instalment date/amount, first/last instalment date, arrear days, arrear amount split P/I/penalties. Point-in-time ("Report As on date") | Compounding frequency per loan; instalment dates and amounts; arrears split - several of the Section 3 fields in one export | Input feed for staging/ECL and a cross-check on our schedule. **NOT an EIR - no EIR, amortised cost or ECL columns.** **Ask for a full-column sample and whether anyone at MAIIC has used it.** |

316 | 4 | **Loan Book Report** | **3868** | E-Banker p.77 | 29 columns per account; parameters: Branch, GL Code (one at a time), **AsOn Date**, Summary/Detail, Figures In Actual; output Laser/Dot Matrix/PDF | Month-end position, moratorium type tag, undrawn amount, cumulative repayments, arrears buckets. **Can it be re-run for Nov 2024 - Nov 2025?*

317 | 5 | **GL-wise Interest Posted Report** | *(no Menu ID captured - it is the figure wrongly captioned "Reschedule Report" on p.76)* | E-Banker p.76 | Per account per month: A/C number, name, applicable from/upto date, total interest product, interest amount, transfer-to account, posted by, TDS. Filter by GL code and post date | The source of Extract C. The 31-Oct-2024 sample proves E-Banker was posting production interest in October 2024 (go-live evidence) | The "what MAIIC actually recognised" side of the EIR-vs-GL reconciliation. **Ask for its Menu ID and a monthly run, all GL codes, Oct 2024 to date.** |

318 | 6 | **Loan Inquiry** | **542** | E-Banker p.55 | Single-account screen: "Enter Rate of Interest and click Show - report displayed EMI wise" - the on-demand EMI chart | Whether the EMI chart is stored or computed; the system's own schedule for a given loan | The cross-check against our generated schedule for the 10 sample loans. **Ask for a screen capture / export for the 10 offer-letter accounts.** |

319 | 7 | **Trial Balance** | **557** | E-Banker p.68 | GL balances, period-stamped (the 19-month corpus we hold) | Control totals; the cumulative-YTD rule on 4xxx accounts | Already in use. Ask only for August 2026 onward, same layout. |

320 | 8 | **Income Statement** | **560** | E-Banker p.71 | P&L by GL code | Interest income 4215/4216/4218/4219 and fee income 4871/4872/4873 by month | Reconciliation targets. |

321 | 9 | **Unpass Register** | **572** | E-Banker p.77 | Unauthorised (pending) entries, real time | Whether rate changes or postings are sitting unapproved - explains lag on the accounts that did not reprice | Data-quality check before month-end runs. |

322 | 10 | **Overdue / NPA Dashboards** | *(no Menu ID captured)* | E-Banker p.79-82 | Overdue and non-performing exposure by product and date | Arrears and staging cross-check against the loan book ageing buckets | Independent check on the arrears we derive from `Repayments`. |

323 | 11 | **Batch Detail / Cash Book** | **553 / 554** | E-Banker p.66-67 | Posting batches; daily cash book | Disbursement and repayment DATES within the month (the first-month partial and tranche cases in the reconstruction test) | The per-drawdown extract may already be obtainable from here. |

324

325 Reports in the LOS we should also ask to see: **Loan Application Status (3.9.2, p.147)***, **Offer Issue with Schedule (3.5.3, p.136)***, **Deal Summary (3.5.7, p.140)*** - these carry Application No. / Process Ref. No. and the fees and reference rate + spread captured at origination.

326

327 ## 6. Aligning our database to E-Banker's structure

328

329 The `MAICC-IFRS9` build already has `contract\_eir`, `contract\_cashflow\_schedule` (versioned), `contract\_fees` (with an `integral` flag and an `ON\_APPROVED` / `ON\_DRAWN` basis), `eir\_amortisation`, `rate\_reset\_events`, `gl\_interest\_postings` and the import mappings. It is a good base. The gaps are where our model diverges from how E-Banker actually holds the data.

330

331 | Concept | E-Banker holds it as | We hold it as today | Change |

332 |---|---|---|---|

333 | Rate behaviour | `Interest Policy` (9 codes) + `Floating Flag` (4 codes), at scheme level | `rate\_type` enum {FIXED, FLOATING} populated from `RATE\_BASIS` | Add `interest\_policy` and `floating\_flag` as raw codes; **derive** `rate\_type` from `interest\_policy`, never from the flag |

334 | Scheme | Product -> Scheme -> Account | nothing | Add `scheme\_code` on the contract; new `schemes` table holding the six scheme-level settings once |

335 | Moratorium | `Moratorium Period` (months, from first disbursement) + `Moratorium Type` {Principle Only, Both} + separate `Grace Period` | `moratorium\_months` only | Add `moratorium\_type` enum {PRINCIPAL_ONLY, BOTH} **using E-Banker's two values verbatim***, `grace\_period\_months`, and `moratorium\_from` = first disbursement date. Derive our internal shape (INTEREST_ONLY / CAPITALISE) from theirs rather than modelling it independently |

336 | Interest basis | `Loan Interest Cal. Base On` {N, S, B} | nothing | Add `interest\_calc\_base` |

337 | Instalment basis | `Installment Based On` {sanction, disbursement} | `contract\_fees.basis` has the idea for fees only | Add `installment\_based\_on` on the contract |

338 | EMI type | `EMI Calc Type` {E, P, F} p.30 | nothing (generator assumes level EMI) | Add `emi\_calc\_type`; teach `ScheduleGeneratorService` the three |

339 | Drawdowns | one row per disbursement transaction (section 7.2) | `disbursement\_tranches` text blob | New child table `contract\_disbursements` (date, amount, reference, tranche_no) |

340 | Restructure | new `SUB\_ACCOUNT\_NO` on the same account; Reschedule Report | `sub\_account\_no`, `last\_restructure\_date`

```

on `contract_eir` | Keep, but key the contract on account **and** sub-account, and add `predecessor_sub_account` for lineage |
341 | Reference rate | PLR master by effective date | nothing (only `reference_rate_at_origination` and a reset log) | New
`reference_rate_series` (index, effective_date, rate, source_row, as_delivered, interpretation) |
342 | EIR over time | n/a (E-Banker has no EIR) | one EIR per contract | New `eir_rate_periods` child table (already
decided) |
343 | LOS link | `Application No.`, `Process Ref. No.` | nothing | Add `los_application_no`, `los_process_ref` |
344 | Status | `ACCOUNT_STATUS` incl. H and F | nothing on `contract_eir` | Add `account_status_code` raw, plus a lookup
once the codes are explained |
345 | Dates | dd/mm text, corrupted on export | date columns | Importer must **reject** any date column that is mixed-type
or ambiguous, and accept ISO only |
346
347 The principle throughout: **store E-Banker's own codes verbatim, then derive our internal enums from them.** Our
teaching workbook modelled moratorium as three shapes; E-Banker has two. We align to two.
348
349 ---
350
351 ## 7. What goes on the loan book, and what has to stay a separate extract
352
353 **Add as columns to the Loan Book Report (Menu ID 3868):** scheme code; Interest Policy; Floating Flag; reference rate %;
spread added to the prime rate (margin) %; Loan Interest Cal. Base On; Installment Based On; EMI Calc Type; Repayment
Frequency; Type of Repayment; Interest Start Date; Installment (first) Date; Moratorium Type as its own field; Grace Period;
Sub Account Number; Restructure Date; Application No.; Process Ref. No.; loan type / GL code as a column; Account Status code;
Funded/Non-funded; currency; days past due as a number.
354
355 **Must remain separate extracts (one-to-many):** disbursements per drawdown; fees per posting (or the 4871/4872/4873
spool with an account-number column); the transaction ledger (Extract B, re-supplied with a DR/CR indicator and ISO dates);
interest posted by loan (Extract C, all accounts); the reference-rate series (ISO dates, repeatable).
356
357 **Reports to request that exist but were never asked for:** Reschedule Report (2839), Audit Trail Report (692), IFRS9
Detail Report (3863) with its full column list.
358
359 ---
360
361 ## 8. Open clarifications for MAIIC and Virtual Galaxy
362
363 1. Is the EMI chart stored as a table or computed on demand when "Show" is clicked?
364 2. What do TRANTYPE codes 303, 120, 308, 309, 310 and 311 mean?
365 3. What do account status codes `H` and `F` mean?
366 4. Why is `NUM_INSTALLMENTS` always tenor-in-months plus one?
367 5. Why does every loan show exactly one disbursement tranche when several were drawn in parts?
368 6. Why is `SUB_ACCOUNT_NO` = 1 and `RESTRUCTURE_DATE` blank on every row, when restructures exist?
369 7. Which scheme is each account on, and can the scheme code be added to the loan book?
370 8. How was the rate change file produced, and can it be made a repeatable extract with ISO dates?
371 9. What is the E-Banker go-live date, and can the Loan Book Report be re-run for Nov 2024 to Nov 2025?
372 10. How was the Loan Book Report originally customised, and how long would adding the columns in section 7 take?
373
374 ---
375
376 *Prepared by Dupleix Institute. Figures quoted are from the files as received on the dates stated; repaired or derived
figures are marked as such.*
377

```

Tool call: Bash

description: Inspect the layout of the delivered Extract A, B and C workbooks

```

--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && ./songenv/Scripts/python -m pip install -q openpyxl pandas 2>&1 | tail -1; cat > maiic_check.py <<'PYEOF'
import pandas as pd, warnings
warnings.filterwarnings("ignore")
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
for f in ["Extract A.xlsx", "Extract B.xlsx", "ExtractC_Jan2025-July2026.xlsx"]:
    x = pd.ExcelFile(D + "\\\" + f)
    for sh in x.sheet_names:
        df = x.parse(sh, header=None, nrows=4)
        print("==", f, "|", sh, "|", x.parse(sh).shape)
        print(df.iloc[:2].to_string(max_cols=40, max_colwidth=22))
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maiic_check.py 2>&1 | cut -c1-900
{
"timeout": 300000
}

```

Result of Bash

```
== Extract A.xlsx | Sheet1 | (362, 36)
0      7      8      9      10     11     12     13
14     15     16     17     18     19     20     21
22     23     24     25     26     27     28
29     30     31     32     33     34
35
0      AS_OF_DATE  CUSTOMER_ID  CUSTOMER_NAME  LOAN_ACCOUNT_NUMBER  SUB_ACCOUNT_NO  GL_ACCOUNT_CODE
GL_ACCOUNT_TITLE  PORTFOLIO  PRODUCT_TYPE  CURRENCY  LOAN_START_DATE  SANCTIONED_AMOUNT  PRINCIPAL_DISBURSED
DISBURSEMENT_TRANCES  INTEREST_RATE  RATE_BASIS  COMPOUNDING  DAY_COUNT_BASIS  REPAYMENT_FREQUENCY  TENOR  NUM_INSTALLMENTS
FIRST_REPAYMENT_DATE  CONTRACTUAL_MATURI...  ACTUAL_CLOSURE_DATE  WRITEOFF_DATE  WRITEOFF_FLAG  WRITEOFF_AMOUNT  ACCOUNT_STATUS
RESTRUCTURE_DATE  PRINCIPAL_GRACE_PE...  INTEREST_GRACE_PERIOD  BALANCE_SNAPSHOT_DATE  OPENING_BALANCE  BALANCE_DATA_NOTE
GENERATED_ON
1  1  2025-01-01 00:00:00  93  Kwithu Producers a...  000104450000053  1  1050201  FInES
Agricultural...  FInES  FInES  AGRICULTURAL...  MWK  23-01-2023  90900000  90900000
24-Jan-2023:909000...  10  Variable  Compound  365  Monthly  3y 0m 0d  37
24-01-2023  23-01-2026  NaN  NaN  N  NaN  F
NaN  3 M  0  31-12-2024  38768954.06  NaN
2026-03-08 15:32:09
== Extract B.xlsx | Sheet1 | (2478, 16)
0      1      2      3      4      5      6      7
8      9      10     11     12     13     14
15
0      RUN_ID  CUSTOMER_ID  LOAN_ACCOUNT_NUMBER  SUB_ACCOUNT_NO  GL_POSTING_REF  TRANSACTION_DATE  TRANSACTION_TYPE
PRINCIPAL_COMPONENT  INTEREST_COMPONENT  FEE_COMPONENT  TOTAL_AMOUNT  SCHEDULED_ACTUAL_FLAG  BALANCE_AFTER_TRAN...  ROW_NOTE
GENERATED_ON
1  1  1  308  000104430000002  1  1050102-36994  31-01-2025  Interest
NaN  10672784.21  NaN  10672784.21  Actual  365097578.94  NaN  2026-03-08
15:36:17
== Extract B.xlsx | Sheet3 | (117, 5)
0  1  2  3  4
0 NaN NaN NaN NaN NaN
1 NaN NaN NaN NaN NaN
== Extract B.xlsx | Sheet1 (2) | (2478, 18)
0      1      2      3      4      5      6      7      8
9      10     11     12     13     14     15
16     17
0      RUN_ID  CUSTOMER_ID  LOAN_ACCOUNT_NUMBER  SUB_ACCOUNT_NO  GL_POSTING_REF  GL Code added  TRANSACTION_DATE  PERIOD
TRANSACTION_TYPE  PRINCIPAL_COMPONENT  INTEREST_COMPONENT  FEE_COMPONENT  TOTAL_AMOUNT  SCHEDULED_ACTUAL_FLAG
BALANCE_AFTER_TRAN...  ROW_NOTE  GENERATED_ON
1  1  1  308  000104430000002  1  1050102-36994  1050102  31-01-2025  1
Interest  NaN  10672784.21  NaN  10672784.21  Actual  365097578.94
NaN  2026-03-08 15:36:17
== ExtractC_Jan2025-July2026.xlsx | Sheet1 | (3996, 14)
0      1      2      3      4      5      6      7      8
9      10     11     12     13
0      RUN_ID  CUSTOMER_ID  LOAN_ACCOUNT_NUMBER  SUB_ACCOUNT_NO  GL_ACCOUNT_CODE  PERIOD_TYPE  PERIOD_YEAR  PERIOD_MONTH
INTEREST_INCOME_PO...  TRANSACTION_COUNT  POSTING_REFERENCES  ROW_NOTE  GENERATED_ON
1  1  7  1434  000104430000062  1  1050102  MONTH  2025  2
19428764.01  1  1050102-40429  Net of TRANTYPE 30...  2026-03-08 16:14:18
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

description: Test the script-reading hypotheses against the delivered extracts

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && cat > maiic_check2.py <<'PYEOF'
import pandas as pd, numpy as np, warnings
warnings.filterwarnings("ignore")
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 30)
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
A = pd.read_excel(D + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
B = pd.read_excel(D + r"\Extract B.xlsx", sheet_name="Sheet1", dtype={"LOAN_ACCOUNT_NUMBER": str})
C = pd.read_excel(D + r"\ExtractC_Jan2025-July2026.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
A["snap"] = A["AS_OF_DATE"].astype(str).str[:10]
print("\n1) RATE BASIS by GL title (one snapshot):")
a1 = A[A.snap == A.snap.max()]
print(pd.crosstab(a1["GL_ACCOUNT_TITLE"], a1["RATE_BASIS"]).to_string())
print("\n2) Fields that differ between the two snapshots for the same account:")
s = sorted(A.snap.unique()); x = A[A.snap == s[0]].set_index("LOAN_ACCOUNT_NUMBER"); y = A[A.snap == s[1]].set_index("LOAN_ACCOUNT_NUMBER")
common = x.index.intersection(y.index); print("    snapshots", s, "| accounts in both", len(common))
for col in A.columns:
    if col in ("AS_OF_DATE", "snap", "GENERATED_ON", "Unnamed: 0"): continue
    d = (x.loc[common, col].astype(str) != y.loc[common, col].astype(str)).sum()
    if d: print(f"    {col}: differs on {d}")
print("\n3) PRINCIPAL_DISBURSED vs SANCTIONED_AMOUNT (latest snapshot):")
p = a1.dropna(subset=["PRINCIPAL_DISBURSED"])
print("    rows with disbursed populated", len(p), "| equal to sanctioned", int((p.PRINCIPAL_DISBURSED.round(2) == p.SANCTIONED_AMOUNT.round(2)).sum()),
      "| less", int((p.PRINCIPAL_DISBURSED < p.SANCTIONED_AMOUNT - 1).sum()), "| more", int((p.PRINCIPAL_DISBURSED > p.SANCTIONED_AMOUNT + 1).sum()))
lm = a1[a1.LOAN_ACCOUNT_NUMBER.str.endswith("10605000009")]
print("    Lake Malawi:", lm[["SANCTIONED_AMOUNT", "PRINCIPAL_DISBURSED", "DISBURSEMENT_TRANCHES", "OPENING_BALANCE"]].to_dict("records"))
print("\n4) FIRST_REPAYMENT_DATE equals LOAN_START_DATE (as text):", int((a1.FIRST_REPAYMENT_DATE.astype(str) == a1.LOAN_START_DATE.astype(str)).sum()), "of", int(a1.FIRST_REPAYMENT_DATE.notna().sum()))
print(a1[["LOAN_START_DATE", "FIRST_REPAYMENT_DATE", "CONTRACTUAL_MATURITY_DATE", "TENOR", "NUM_INSTALMENTS", "PRINCIPAL_GRACE_PERIOD"]].head(6).to_string())
print("\n5) Extract B 2025 interest vs Extract C 2025, by account:")
act = B[B.SCHEDULED_ACTUAL_FLAG == "Actual"]
print("    B actual rows by type:", act.TRANSACTION_TYPE.value_counts().to_dict())
bi = act[act.TRANSACTION_TYPE == "Interest"].groupby("LOAN_ACCOUNT_NUMBER").TOTAL_AMOUNT.sum()
bo = act[act.TRANSACTION_TYPE == "Other/Adjustment"].groupby("LOAN_ACCOUNT_NUMBER").TOTAL_AMOUNT.sum()
cy = C[(C.PERIOD_TYPE == "YEAR") & (C.PERIOD_YEAR == 2025)].groupby("LOAN_ACCOUNT_NUMBER").INTEREST_INCOME_POSTED.sum()
cm = C[(C.PERIOD_TYPE == "MONTH") & (C.PERIOD_YEAR == 2025)].groupby("LOAN_ACCOUNT_NUMBER").INTEREST_INCOME_POSTED.sum()
print("    C: YEAR rows", int((C.PERIOD_TYPE == "YEAR").sum()), "| MONTH rows", int((C.PERIOD_TYPE == "MONTH").sum()), "| sum of all rows", round(C.INTEREST_INCOME_POSTED.sum()), "| sum MONTH only", round(C[(C.PERIOD_TYPE == "MONTH").INTEREST_INCOME_POSTED.sum()]))
print("    C 2025: YEAR total", round(cy.sum()), "| MONTH total", round(cm.sum()), "| negative monthly rows (all years)", int((C[(C.PERIOD_TYPE == "MONTH").INTEREST_INCOME_POSTED < 0].sum())))
j = pd.concat([bi.rename("B_interest"), bo.rename("B_other"), cm.rename("C_2025")], axis=1).fillna(0)
j["diff"] = j.B_interest - j.C_2025
print("    accounts: in B interest", int((j.B_interest > 0).sum()), "| in C 2025", int((j.C_2025 != 0).sum()), "| totals B", round(j.B_interest.sum()), "C", round(j.C_2025.sum()))
print("    accounts where B interest = C (within 1):", int((j["diff"].abs() < 1).sum()), "| differ:", int((j["diff"].abs() >= 1).sum()))
dd = j[j["diff"].abs() >= 1].copy(); dd["diff_minus_other"] = dd["diff"].abs() - dd.B_other
print(dd.sort_values("diff", key=abs, ascending=False).head(8).round(0).to_string())
print("\n6) Accounts in A but not in C, by status (latest snapshot):")
missing = a1[~a1.LOAN_ACCOUNT_NUMBER.isin(C.LOAN_ACCOUNT_NUMBER.unique())]
print("    ", len(missing), "missing |", missing.ACCOUNT_STATUS.value_counts().to_dict())
print("    in C by status:", a1[a1.LOAN_ACCOUNT_NUMBER.isin(C.LOAN_ACCOUNT_NUMBER.unique())].ACCOUNT_STATUS.value_counts().to_dict())
print("\n7) Scheduled rows in B:")
sch = B[B.SCHEDULED_ACTUAL_FLAG == "Scheduled"]
print("    rows", len(sch), "| accounts", sch.LOAN_ACCOUNT_NUMBER.nunique(), "| accounts with actual rows", act.LOAN_ACCOUNT_NUMBER.nunique())
print("    305 rows", int((act.TRANSACTION_TYPE == "Principal+Interest").sum()), "| of which interest component = total (capped):", int(((act.TRANSACTION_TYPE == "Principal+Interest") & ((act.INTEREST_COMPONENT - act.TOTAL_AMOUNT).abs() < 0.01)).sum()))
print("    status H/F rows: rate basis", a1[a1.ACCOUNT_STATUS.isin(["H", "F"])].groupby("ACCOUNT_STATUS").OPENING_BALANCE.agg(["count", "sum"]).round(0).to_dict())
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maiic_check2.py 2>&1 | cut -c1-330
{
  "timeout": 300000
}
```

Result of Bash

```
1) RATE_BASIS by GL title (one snapshot):
RATE_BASIS           Fixed  Variable
GL_ACCOUNT_TITLE
FINES Agricultural Loans      0      50
FINES Industrial Loans       0      51
FINES Investment Loan        4       0
MAIIC Agricultural Loans     16       0
MAIIC Industrial Loans       41       0
MAIIC Term Loan              13       0
NASCOMEX Limited             0       1
Quasi Equity Investment       5       0
```

```
2) Fields that differ between the two snapshots for the same account:
   snapshots ['2025-01-01', '31-12-2025'] | accounts in both 181
   : differs on 181
```

Traceback (most recent call last):

```
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\indexes\base.py", line 3641, in get_loc
    return self._engine.get_loc(casted_key)
~~~~~
File "pandas/_libs/index.pyx", line 168, in pandas._libs.index.IndexEngine.get_loc
File "pandas/_libs/index.pyx", line 197, in pandas._libs.index.IndexEngine.get_loc
File "pandas/_libs/hashtable_class_helper.pxi", line 7668, in pandas._libs.hashtable.PyObjectHashTable.get_item
File "pandas/_libs/hashtable_class_helper.pxi", line 7676, in pandas._libs.hashtable.PyObjectHashTable.get_item
KeyError: 'LOAN_ACCOUNT_NUMBER'
```

The above exception was the direct cause of the following exception:

Traceback (most recent call last):

```
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\maii_check2.py", line 17, in <module>
    d = (x.loc[common, col].astype(str) != y.loc[common, col].astype(str)).sum()
~~~~~
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\indexing.py", line 1200, in _getitem__
    return self._getitem_tuple(key)
~~~~~
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\indexing.py", line 1386, in _getitem_tuple
    return self._getitem_lowerdim(tup)
~~~~~
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\indexing.py", line 1093, in _getitem_lowerdim
    section = self._getitem_axis(key, axis=i)
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\indexing.py", line 1449, in _getitem_axis
    return self._get_label(key, axis=axis)
~~~~~
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\indexing.py", line 1399, in _get_label
    return self.obj.xs(label, axis=axis)
~~~~~
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\generic.py", line 4227, in xs
    return self[key]
~~~~~
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\frame.py", line 4378, in _getitem__
    indexer = self.columns.get_loc(key)
File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\indexes\base.py", line 3648, in get_loc
    raise KeyError(key) from err
KeyError: 'LOAN_ACCOUNT_NUMBER'
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

CLAUDE (2026-10-05 13:07:27 UTC)

Reading the scripts against the delivered data. One check already confirms a finding: the Fixed/Variable labels in Extract A look inverted. Finishing the remaining checks.

Tool call: Bash

description: Rerun the remaining data checks

```
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp\htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && sed -i 's/      if col in ("AS_OF_DATE", "snap", "GENERATED_ON", "Unnamed: 0"): continue/      if col in
("AS_OF_DATE", "snap", "GENERATED_ON", "LOAN_ACCOUNT_NUMBER") or str(col).strip() == "" or str(col).startswith("Unnamed"):
continue/' maii_check2.py && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maii_check2.py 2>&1 | sed -n '13,80p' | cut
-c1-300
{
  "timeout": 300000
}
```

Result of Bash

2) Fields that differ between the two snapshots for the same account:
snapshots ['2025-01-01', '31-12-2025'] | accounts in both 181
PRINCIPAL_DISBURSED: differs on 24
DISBURSEMENT_TRANCHES: differs on 24
REPAYMENT_FREQUENCY: differs on 42
FIRST_REPAYMENT_DATE: differs on 53
CONTRACTUAL_MATURITY_DATE: differs on 1
ACTUAL_CLOSURE_DATE: differs on 135
WRITEOFF_DATE: differs on 181
WRITEOFF_AMOUNT: differs on 181
RESTRUCTURE_DATE: differs on 181
BALANCE_SNAPSHOT_DATE: differs on 179
OPENING_BALANCE: differs on 128
BALANCE_DATA_NOTE: differs on 133

3) PRINCIPAL_DISBURSED vs SANCTIONED_AMOUNT (latest snapshot):
rows with disbursed populated 157 | equal to sanctioned 138 | less 12 | more 6
Lake Malawi: [{ 'SANCTIONED_AMOUNT': 1055473655.0, 'PRINCIPAL_DISBURSED': 297161905.0, 'DISBURSEMENT_TRANCHES':
'28-Apr-2026:297161905.00', 'OPENING_BALANCE': nan}]

4) FIRST_REPAYMENT_DATE equals LOAN_START_DATE (as text): 63 of 128

	LOAN_START_DATE	FIRST_REPAYMENT_DATE	CONTRACTUAL_MATURITY_DATE	TENOR	NUM_INSTALMENTS	PRINCIPAL_GRACE_PERIOD
124	2022-04-07 00:00:00	NaN	2024-06-07 00:00:00	2y 0m 0d	0	3 M
125	2022-11-07 00:00:00	NaN	2024-12-07 00:00:00	2y 0m 0d	0	3 M
126	2022-01-04 00:00:00	NaN	30-09-2028	7y 0m 0d	0	0
127	28-07-2020	28-07-2020	27-07-2022	2y 0m 0d	25	6 M
128	21-04-2021	21-04-2021	21-04-2026	0y 60m 0d	61	0
129	30-08-2022	2025-01-06 00:00:00	29-08-2025	0y 36m 0d	3	0

5) Extract B 2025 interest vs Extract C 2025, by account:
B actual rows by type: {'Interest': 1034, 'Principal+Interest': 539, 'Disbursement': 97, 'Other/Adjustment': 31, 'Fee': 2}
C: YEAR rows 408 | MONTH rows 3588 | sum of all rows 20805539201 | sum MONTH only 10402769601
C 2025: YEAR total 7663172090 | MONTH total 7663172090 | negative monthly rows (all years) 6
accounts: in B interest 107 | in C 2025 107 | totals B 2639557380 C 7663172090
accounts where B interest = C (within 1): 1 | differ: 107

	B_interest	B_other	C_2025	diff	diff_minus_other
LOAN_ACCOUNT_NUMBER					
000104430000062	239578671.0	0.0	718736013.0	-479157342.0	479157342.0
000104430000071	214935266.0	0.0	644805797.0	-429870532.0	429870532.0
000104430000070	137891972.0	0.0	413675916.0	-275783944.0	275783944.0
000104420000005	118947126.0	200000000.0	356841378.0	-237894252.0	37894252.0
000104430000069	118137980.0	0.0	354413940.0	-236275960.0	236275960.0
000104430000072	109392039.0	0.0	328176117.0	-218784078.0	218784078.0
000104430000002	117296781.0	10457152.0	320518886.0	-203222105.0	192764953.0
000104430000073	90522529.0	2395118.0	264382235.0	-173859705.0	171464587.0

6) Accounts in A but not in C, by status (latest snapshot):
61 missing | {'Closed': 46, 'Active': 10, 'Dormant': 4, 'H': 1}
in C by status: {'Active': 76, 'Dormant': 21, 'H': 20, 'F': 3}

7) Scheduled rows in B:
rows 775 | accounts 90 | accounts with actual rows 112
305 rows 539 | of which interest component = total (capped): 8
status H/F rows: rate basis {'count': {'F': 3, 'H': 21}, 'sum': {'F': 172981552.0, 'H': 38356803.0}}

Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

Tool call: Bash

description: Check for repeated runs in Extract C and compare with Extract B after de-duplication

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && cat > maiic_check3.py <<'PYEOF'
import pandas as pd, numpy as np, warnings
warnings.filterwarnings("ignore")
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 30); pd.set_option("display.max_colwidth", 60)
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
A = pd.read_excel(D + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
B = pd.read_excel(D + r"\Extract B.xlsx", sheet_name="Sheet1", dtype={"LOAN_ACCOUNT_NUMBER": str})
C = pd.read_excel(D + r"\ExtractC_Jan2025-July2026.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
print("C RUN_ID counts:", C.RUN_ID.value_counts().to_dict(), "| GENERATED_ON:",
C.GENERATED_ON.astype(str).str[:16].value_counts().head(5).to_dict())
print("B RUN_ID counts:", B.RUN_ID.value_counts().to_dict())
M = C[C.PERIOD_TYPE == "MONTH"]
k = ["LOAN_ACCOUNT_NUMBER", "PERIOD_YEAR", "PERIOD_MONTH"]
g = M.groupby(k).size()
print("C monthly rows", len(M), "| distinct account-months", len(g), "| rows per account-month:", g.value_counts().to_dict())
print("    by year, rows per account-month:", M.groupby(["PERIOD_YEAR"]).apply(lambda d:
d.groupby(k).size().mean()).round(2).to_dict())
ex = M[(M.LOAN_ACCOUNT_NUMBER == "000104430000062") & (M.PERIOD_YEAR == 2025)].sort_values("PERIOD_MONTH")
print(ex[["RUN_ID", "PERIOD_MONTH", "INTEREST_INCOME_POSTED", "TRANSACTION_COUNT", "POSTING_REFERENCES"]].head(9).to_string())
bx = B[(B.LOAN_ACCOUNT_NUMBER == "000104430000062") & (B.SCHEDULED_ACTUAL_FLAG == "Actual")]
print(bx[["GL_POSTING_REF", "TRANSACTION_DATE", "TRANSACTION_TYPE", "TOTAL_AMOUNT",
"BALANCE_AFTER_TRANSACTION"]].head(8).to_string())
# de-duplicated C vs B
Cd = M.drop_duplicates(subset=k + ["INTEREST_INCOME_POSTED", "POSTING_REFERENCES"])
print("C monthly after dropping identical repeats:", len(Cd), "| 2025 total", round(Cd[Cd.PERIOD_YEAR ==
2025].INTEREST_INCOME_POSTED.sum()), "| 2026 total", round(Cd[Cd.PERIOD_YEAR == 2026].INTEREST_INCOME_POSTED.sum()))
act = B[B.SCHEDULED_ACTUAL_FLAG == "Actual"]
bi = act[act.TRANSACTION_TYPE == "Interest"].groupby("LOAN_ACCOUNT_NUMBER").TOTAL_AMOUNT.sum()
cd = Cd[Cd.PERIOD_YEAR == 2025].groupby("LOAN_ACCOUNT_NUMBER").INTEREST_INCOME_POSTED.sum()
j = pd.concat([bi.rename("B"), cd.rename("C")], axis=1).fillna(0); j["diff"] = j.B - j.C
print("after de-dup: B interest total", round(j.B.sum()), "| C 2025", round(j.C.sum()), "| accounts equal within 1:",
int((j["diff"].abs() < 1).sum()), "of", len(j))
print(j[j["diff"].abs() >= 1].sort_values("diff", key=abs, ascending=False).head(6).round(0).to_string())
# snapshot differences sample
A["snap"] = A["AS_OF_DATE"].astype(str).str[:10]; s = sorted(A.snap.unique())
x = A[A.snap == s[0]].set_index("LOAN_ACCOUNT_NUMBER"); y = A[A.snap == s[1]].set_index("LOAN_ACCOUNT_NUMBER")
for col in ["PRINCIPAL_DISBURSED", "REPAYMENT_FREQUENCY", "INTEREST_RATE", "ACCOUNT_STATUS"]:
    d = x[col].astype(str) != y[col].astype(str)
    print(col, "differs on", int(d.sum()), "| sample:", list(zip(x[col][d].head(3).tolist(), y[col][d].head(3).tolist())))
print("GENERATED_ON by snapshot:", A.groupby("snap").GENERATED_ON.agg(lambda v:
v.astype(str).str[:16].unique().tolist()).to_dict())
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maiic_check3.py 2>&1 | cut -c1-420
{
  "timeout": 300000
}
```

Result of Bash

```
C RUN_ID counts: {41: 1758, 7: 1119, 25: 1119} | GENERATED_ON: {'25-08-2026 15:40': 1758, '2026-03-08 16:14': 1119, '2026-03-08 16:15': 1119}
B RUN_ID counts: {1: 2478}
C monthly rows 3588 | distinct account-months 1564 | rows per account-month: {3: 1012, 1: 552}
Traceback (most recent call last):
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\maiiic_check3.py", line 14, in <module>
    print("    by year, rows per account-month:", M.groupby(["PERIOD_YEAR"]).apply(lambda d:
d.groupby(k).size().mean()).round(2).to_dict())
~~~~~
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\songenv\Lib\site-packages\pandas\core\groupby\groupby.py", line 1650, in apply
    return self._python_apply_general(f, self._obj_with_exclusions)
~~~~~
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\songenv\Lib\site-packages\pandas\core\groupby\groupby.py", line 1687, in _python_apply_general
    values, mutated = self._grouper.apply_groupwise(f, data)
~~~~~
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\songenv\Lib\site-packages\pandas\core\groupby\ops.py", line 1035, in apply_groupwise
    res = f(group)
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\maiiic_check3.py", line 14, in <lambda>
    print("    by year, rows per account-month:", M.groupby(["PERIOD_YEAR"]).apply(lambda d:
d.groupby(k).size().mean()).round(2).to_dict())
~~~~~
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\songenv\Lib\site-packages\pandas\util\_decorators.py", line 336, in wrapper
    return func(*args, **kwargs)
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\songenv\Lib\site-packages\pandas\core\frame.py", line 10835, in groupby
    return DataFrameGroupBy(
        obj=self,
        ...<6 lines>...
        dropna=dropna,
    )
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\songenv\Lib\site-packages\pandas\core\groupby\groupby.py", line 1095, in __init__
    grouper, exclusions, obj = get_grouper(
        ~~~~~^
        obj,
        ^^^^
    ...<4 lines>...
        dropna=self.dropna,
        ^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\songenv\Lib\site-packages\pandas\core\groupby\grouper.py", line 901, in get_grouper
    raise KeyError(gpr)
KeyError: 'PERIOD_YEAR'
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

```
description: Rerun the Extract C run analysis with the latest run isolated
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp\htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python - <<'PYEOF' 2>&1 | cut -c1-400
import re
s = open("maiiic_check3.py", encoding="utf-8").read()
s = re.sub(r'print\(("    by year, rows per account-month:".*\n', 'print("    runs by year:", M.groupby(["RUN_ID",
"PERIOD_YEAR"]).size().to_dict())\n', s)
s = s.replace('Cd = M.drop_duplicates(subset=k + ["INTEREST_INCOME_POSTED", "POSTING_REFERENCES"])', 'Cd = M[M.RUN_ID ==
M.RUN_ID.max()]\nprint("run 7 = run 25 identical on amounts:", M[M.RUN_ID ==
7].set_index(k).INTEREST_INCOME_POSTED.round(2).equals(M[M.RUN_ID == 25].set_index(k).INTEREST_INCOME_POSTED.round(2)))\nm7 =
M[M.RUN_ID == 7].set_index(k).INTEREST_INCOME_POSTED; m41 = Cd.set_index(k).INTEREST_INCOME_POSTED\nboth =
m7.index.intersection(m41.index); print("run 7 account-months", len(m7), "| also in run 41", len(both), "| amounts differ",
int(((m7[both] - m41[both]).abs() > 0.01).sum()), "| accounts run 7", M[M.RUN_ID == 7].LOAN_ACCOUNT_NUMBER.nunique(), "| run
41", Cd.LOAN_ACCOUNT_NUMBER.nunique())')
s = s.replace('"C monthly after dropping identical repeats:"', '"C latest run only (41):"')
exec(s)
PYEOF
{
  "timeout": 30000
}
```

Result of Bash

```
C RUN_ID counts: {41: 1758, 7: 1119, 25: 1119} | GENERATED_ON: {'25-08-2026 15:40': 1758, '2026-03-08 16:14': 1119, '2026-03-08 16:15': 1119}
B RUN_ID counts: {1: 2478}
C monthly rows 3588 | distinct account-months 1564 | rows per account-month: {3: 1012, 1: 552}
  runs by year: {(7, 2025): 1012, (25, 2025): 1012, (41, 2025): 1012, (41, 2026): 552}
    RUN_ID PERIOD_MONTH INTEREST_INCOME_POSTED TRANSACTION_COUNT POSTING_REFERENCES
1401      25           1          21497946.96              1      1050102-36989
485       7           1          21497946.96              1      1050102-36989
3148     41           1          21497946.96              1      1050102-36989
0        7           2          19428764.01              1      1050102-40429
3149     41           2          19428764.01              1      1050102-40429
1402     25           2          19428764.01              1      1050102-40429
1403     25           3          20803886.90              1      1050102-48211
1        7           3          20803886.90              1      1050102-48211
3150     41           3          20803886.90              1      1050102-48211
    GL_POSTING_REF TRANSACTION_DATE TRANSACTION_TYPE TOTAL_AMOUNT BALANCE_AFTER_TRANSACTION
502  1050102-36989      31-01-2025           Interest    21497946.96      7.885312e+08
503  1050102-40041      18-02-2025 Principal+Interest  53589482.23      7.349418e+08
504  1050102-40429      28-02-2025           Interest    19428764.01      7.543705e+08
505  1050102-44495      25-03-2025 Principal+Interest  26794741.11      7.275758e+08
506  1050102-44509      25-03-2025 Principal+Interest  26794741.11      7.007810e+08
507  1050102-48211      31-03-2025           Interest    20803886.90      7.215849e+08
508  1050102-49377      30-04-2025           Interest    19571755.50      7.411567e+08
509  1050102-53400      2025-06-05 00:00:00 Principal+Interest  26794741.11      7.143619e+08
run 7 = run 25 identical on amounts: False
run 7 account-months 1012 | also in run 41 1012 | amounts differ 0 | accounts run 7 107 | run 41 120
C latest run only (41): 1564 | 2025 total 2554390697 | 2026 total 2739597510
after de-dup: B interest total 2639557380 | C 2025 2554390697 | accounts equal within 1: 101 of 107
      B          C      diff
LOAN_ACCOUNT_NUMBER
000104430000068      131899214.0      62568404.0      69330810.0
000104430000002      117296781.0      106839629.0      10457152.0
000104430000073      90522529.0      88127412.0      2395118.0
000104420000062      38421399.0      36402698.0      2018700.0
000104420000052      42329780.0      41416239.0      913541.0
000104420000032      29893579.0      29842217.0      51362.0
Traceback (most recent call last):
  File "<stdin>", line 6, in <module>
  File "<string>", line 35, in <module>
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\ops\common.py", line 85, in new_method
    return method(self, other)
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\arraylike.py", line 46, in __ne__
    return self._cmp_method(other, operator.ne)
~~~~~
  File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\songenv\Lib\site-packages\pandas\core\series.py", line 6730, in _cmp_method
    raise ValueError("Can only compare identically-labeled Series objects")
ValueError: Can only compare identically-labeled Series objects
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

CLAUDE (2026-10-05 13:09:16 UTC)

I read the three scripts against what we requested and tested what they imply on the delivered Extract A, B and C files. The scripts explain most of our open questions, and they expose three problems that affect numbers we may already have used. The files are partial: the table definitions and the export step are not included, so I have not seen how the tables reach Excel.

Problems that affect our numbers

- **Extract C is triple-counted for 2025.** The script adds a new run each time and never clears old ones, and the file we received holds three runs (IDs 7, 25 and 41). All three carry 2025 with identical amounts, so summing the file gives 7.66bn of 2025 interest against a true 2.55bn. Only run 41 (25 August, 120 accounts, Jan 2025 to Jul 2026) should be used. I have not checked whether our import filtered on run ID.
- **The Fixed/Variable label in Extract A is inverted.** The script labels Floating Flag F as "Fixed" and I as "Variable", but the manual defines F as Floating and I as Fixed. In the data, every MAIIC Agricultural and Industrial loan shows "Fixed" and every FlnES Agricultural and Industrial loan shows "Variable", the exact opposite of the repricing we observed. Term Loans still don't fit after flipping, so Interest Policy is still needed.
- **Extract B's "Interest" rows are interest charged, not cash received.** They are the same postings Extract C reports as income: on run 41, 101 of 107 accounts agree within MWK 1 for 2025. In the ledger these rows increase the balance and the "Principal+Interest" rows reduce it. Our research report describes them as cash received, which needs correcting.

What else the scripts explain

```
| Open question | What the script shows |
|---|---|
| No debit/credit, no negatives in B | The ledger amount is signed; the script wraps every amount in `ABS()`. Removing it restores direction. |
```

"Estimated" split on 539 rows	The script computes it as balance × the loan's current rate × days since the previous posting. It ignores rate history and interest already charged, so it is not usable.
Scheduled rows unusable	The script only pulls instalments not yet fully recovered, within the date range, so it is the unpaid remainder and not the schedule.
Is the EMI chart stored?	Yes, in a table (`fixloan\_inst\_chart`) with date, instalment, principal and interest per row.
`NUM\_INSTALLMENTS` = tenor + 1	It is a plain count of chart rows. The chart appears to include a row at the start date: first repayment date equals loan start date on 63 of 128 rows.
One tranche per loan	The script reads the disbursement schedule table, not ledger drawdowns. Actual tranches are the type 301 postings in the ledger.
Restructure date blank	It reads `loan\_reschedule\_details`. Blank everywhere means that table has no rows for these loans, so restructures are not being recorded there.
Status H and F	Raw codes the script does not decode (it only handles A, C, D). We still need the code list.
61 accounts missing from C	Accounts with no interest posting in the period produce no row. Of the 61, 46 are closed, 10 active, 4 dormant and 1 is H. The 10 active ones are worth asking about.
Only 2 fee rows	Both ledger scripts read only postings on the loan's own GL code. Fees posted to the fee income GLs are never picked up.
Scrambled dates	The scripts hold proper Oracle dates, so the corruption happens at export.

Two further points:

- **Extract A is not a true snapshot.** Only the balance is taken as at the date. Rate, status, sanctioned amount and tenor are identical in both snapshots, and Lake Malawi's 31 December 2025 row shows a drawdown dated 28 April 2026.
- **GL_POSTING_REF is an internal row ID, not a voucher number.** The script's own note says the voucher number field was blank.

What to ask Barry for

These are now small edits to existing queries, because we know the table names:

1. Export Extract C for the latest run only, and Extract B with signed amounts and the transaction type code.
2. Add the scheme code and Interest Policy to Extract A and correct the Fixed/Variable decode.
3. Add a full dump of the EMI chart table without the "unrecovered" filter.
4. Add a per-drawdown list from the type 301 ledger postings.
5. Run B and C from October 2024, which is only a parameter change.
6. Export as CSV with year-first dates.

I can draft the amended queries and the note to Barry next.

USER (2026-10-05 13:14:04 UTC)

Before you draft amended queries, can you give me the scripts that give me the data dictionary

Tool call: Write

```
file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Data Dictionary Scripts\
MAIIC_EBanker_Data_Dictionary_Queries.sql
--- content ---
-----
-- MAIIC E-Banker: data dictionary queries
-- Prepared by Dupleix Institute, 5 October 2026
--
-- PURPOSE
-- To give us the structure of the E-Banker tables behind Extracts A, B and C
-- (the table and column names, data types, keys and code values), so that the
-- amended extract queries can be written against the real structure.
--
-- SAFE TO RUN
-- Every statement is a SELECT. Nothing is created, changed or deleted.
-- Queries DD_01 to DD_08 read Oracle's own catalogue views (ALL_TABLES and so
-- on), not business data. DD_09 to DD_15 read codes and counts. DD_16 reads the
-- rows of ONE sample loan.
--
-- HOW TO RUN
-- 1. Log in as the same user that runs PROC_BUILD_MAIIC_FACILITY_REGISTER.
-- 2. Run each query on its own and export the result grid as CSV (not Excel),
-- using the file name given above the query, for example DD_01_schema.csv.
-- 3. Please do not open and re-save the CSV files in Excel before sending:
-- that is what scrambled the dates in the earlier extracts.
--
-- THE SCHEMA
-- The queries find the schema by looking for the table ACMASTER, so no schema
-- name has to be typed in. If DD_01 returns no rows, the login cannot see
-- ACMASTER: please tell us which schema owns it.
-----

-----
-- PART 1. STRUCTURE (Oracle catalogue only, no business data)
-----

-- DD_01_schema.csv
-- Which schema owns the tables our three extract procedures read.
SELECT t.owner,
       t.table_name,
       t.num_rows,
       TO_CHAR(t.last_analyzed, 'YYYY-MM-DD') AS stats_date
FROM   all_tables t
WHERE  t.table_name IN ('ACMASTER', 'ACLOANMASTER', 'CUSTOMER_MAST', 'SCHEME_MASTER', 'GLMASTER',
                        'CUMVOUCH', 'ACCOUNT_BALANCE', 'FIXLOAN_INST_CHART', 'MULTI_INST_DETAILS',
                        'LOAN_DISBURSMENT_SCHEDULE', 'LOAN_RESCHEDULE_DETAILS',
                        'LOAN_WRITEOFF_MASTER', 'LOAN_WRITEOFF_TRANS')
ORDER  BY t.owner, t.table_name;

-- DD_02_tables.csv
-- Every table in the schema, with its approximate row count and any description.
SELECT t.owner,
       t.table_name,
       t.num_rows,
       TO_CHAR(t.last_analyzed, 'YYYY-MM-DD') AS stats_date,
       c.comments AS table_description
FROM   all_tables t
LEFT   JOIN all_tab_comments c
       ON c.owner = t.owner AND c.table_name = t.table_name
WHERE  t.owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
ORDER  BY t.table_name;

-- DD_03_columns.csv
-- Every column of every table in the schema: the data dictionary itself.
-- NUM_DISTINCT and NUM_NULLS come from optimiser statistics and show which
-- columns are actually populated.
SELECT col.table_name,
       col.column_id AS column_no,
       col.column_name,
       col.data_type,
       col.data_length,
       col.data_precision,
       col.data_scale,
       col.nullable,
       col.num_distinct,
       col.num_nulls,
       cc.comments AS column_description
FROM   all_tab_columns col
JOIN   all_tables t
       ON t.owner = col.owner AND t.table_name = col.table_name
LEFT   JOIN all_col_comments cc
       ON cc.owner = col.owner AND cc.table_name = col.table_name AND cc.column_name = col.column_name
WHERE  col.owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
ORDER  BY col.table_name, col.column_id;

-- DD_04_keys.csv
```

```

-- Primary keys (P), unique keys (U) and foreign keys (R), with the table each
-- foreign key points to. This is how the tables join.
SELECT c.table_name,
       c.constraint_name,
       c.constraint_type,
       cc.position              AS column_position,
       cc.column_name,
       r.table_name             AS references_table,
       rc.column_name          AS references_column,
       c.status
FROM   all_constraints c
JOIN   all_cons_columns cc
      ON cc.owner = c.owner AND cc.constraint_name = c.constraint_name
LEFT   JOIN all_constraints r
      ON r.owner = c.r_owner AND r.constraint_name = c.r_constraint_name
LEFT   JOIN all_cons_columns rc
      ON rc.owner = r.owner AND rc.constraint_name = r.constraint_name AND rc.position = cc.position
WHERE  c.owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
AND    c.constraint_type IN ('P', 'U', 'R')
ORDER  BY c.table_name, c.constraint_type, c.constraint_name, cc.position;

-- DD_05_indexes.csv
-- Indexes and their columns. Where no keys are declared (DD_04 comes back short),
-- the unique indexes show what identifies a row.
SELECT i.table_name,
       i.index_name,
       i.uniqueness,
       ic.column_position,
       ic.column_name
FROM   all_indexes i
JOIN   all_ind_columns ic
      ON ic.index_owner = i.owner AND ic.index_name = i.index_name
WHERE  i.table_owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
ORDER  BY i.table_name, i.index_name, ic.column_position;

-- DD_06_account_key_tables.csv
-- Every table that carries the loan account key or one of the other keys we join
-- on. This finds the tables we have not been told about (fees, charges, rate
-- history, restructures, the link to the loan origination system).
SELECT col.table_name,
       col.column_name,
       col.data_type,
       col.data_length,
       t.num_rows
FROM   all_tab_columns col
JOIN   all_tables t
      ON t.owner = col.owner AND t.table_name = col.table_name
WHERE  col.owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
AND    col.column_name IN ('NEW_AC_NUMBER', 'SUB_AC_NUMBER', 'SUB_ACCOUNT_NO', 'SCHEME_MST_ID',
                          'CUSTOMER_ID', 'TRANSTYPE', 'GLCODE', 'AC_GLCODE', 'CBS_GLCODE',
                          'STATUS_CODE', 'FLOATING_FLAG')
ORDER  BY col.column_name, col.table_name;

-- DD_07_tables_by_subject.csv
-- Tables whose names suggest the subjects we still need: interest policy and
-- rates, fees and charges, instalments, disbursements, restructures, write-offs,
-- status and transaction-type code lists, audit trail, loan origination, IFRS 9.
SELECT t.table_name,
       t.num_rows,
       c.comments              AS table_description
FROM   all_tables t
LEFT   JOIN all_tab_comments c
      ON c.owner = t.owner AND c.table_name = t.table_name
WHERE  t.owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
AND    ( t.table_name LIKE '%LOAN%'   OR t.table_name LIKE '%SCHEME%'
        OR t.table_name LIKE '%INST%' OR t.table_name LIKE '%EMI%'
        OR t.table_name LIKE '%INT%'  OR t.table_name LIKE '%RATE%'
        OR t.table_name LIKE '%PLR%'  OR t.table_name LIKE '%SLAB%'
        OR t.table_name LIKE '%POLICY%' OR t.table_name LIKE '%CHARGE%'
        OR t.table_name LIKE '%FEE%'   OR t.table_name LIKE '%DISB%'
        OR t.table_name LIKE '%RESCH%' OR t.table_name LIKE '%RESTRUCT%'
        OR t.table_name LIKE '%WRITEOFF%' OR t.table_name LIKE '%MORAT%'
        OR t.table_name LIKE '%TRANSTYPE%' OR t.table_name LIKE '%STATUS%'
        OR t.table_name LIKE '%AUDIT%'  OR t.table_name LIKE '%APPLICATION%'
        OR t.table_name LIKE '%PROPOSAL%' OR t.table_name LIKE '%SANCTION%'
        OR t.table_name LIKE '%NPA%'    OR t.table_name LIKE '%IFRS%'
        OR t.table_name LIKE '%VOUCH%'  OR t.table_name LIKE '%BALANCE%')
ORDER  BY t.table_name;

-- DD_08_stored_code.csv
-- Views, procedures, functions and packages that read the tables our extracts
-- use. Names only, no source code. This shows which existing reports already sit
-- on the same data (for example the Loan Book and IFRS9 Detail reports).
SELECT d.type              AS object_type,
       d.name              AS object_name,
       d.referenced_name   AS reads_table
FROM   all_dependencies d

```

```

WHERE d.referenced_owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
AND d.referenced_type = 'TABLE'
AND d.referenced_name IN ('ACMASTER', 'ACLOANMASTER', 'SCHEME_MASTER', 'CUMVOUCH', 'ACCOUNT_BALANCE',
                          'FIXLOAN_INST_CHART', 'MULTI_INST_DETAILS', 'LOAN_DISBURSMENT_SCHEDULE',
                          'LOAN_RESCHEDULE_DETAILS', 'LOAN_WRITEOFF_MASTER', 'LOAN_WRITEOFF_TRANS')
AND d.type IN ('VIEW', 'PROCEDURE', 'FUNCTION', 'PACKAGE', 'PACKAGE BODY', 'TRIGGER')
ORDER BY d.referenced_name, d.type, d.name;

```

```

-----
-- PART 2. CODES AND COUNTS (what the values in the columns mean)
-- From here on the queries use the table names directly, as the extract
-- procedures do, so they must be run as the same user as the procedures.
-----

```

```

-- DD_09_schemes_all.csv
-- Every loan scheme with the number of accounts on it. Shows whether any scheme
-- falls outside the "MAIIC% or FINES%" name filter the extracts use.
SELECT s.scheme_mst_id,
       s.scheme_name,
       CASE WHEN UPPER(s.scheme_name) LIKE 'MAIIC%' OR UPPER(s.scheme_name) LIKE 'FINES%'
            THEN 'Y' ELSE 'N' END           AS in_extract_scope,
       COUNT(a.new_ac_number)               AS accounts
FROM   scheme_master s
LEFT JOIN acmaster a ON a.scheme_mst_id = s.scheme_mst_id
GROUP BY s.scheme_mst_id, s.scheme_name
ORDER BY s.scheme_name;

```

```

-- DD_10_scheme_settings.csv
-- Every setting held on the MAIIC and FINES schemes, all columns. This is where
-- Interest Policy, Floating Flag, Loan Interest Cal. Base On, Installment Based
-- On, EMI Calc Type and the moratorium limits are stored.
SELECT s.*
FROM   scheme_master s
WHERE  UPPER(s.scheme_name) LIKE 'MAIIC%' OR UPPER(s.scheme_name) LIKE 'FINES%'
ORDER BY s.scheme_name;

```

```

-- DD_11_account_status_codes.csv
-- Account status codes in use on the MAIIC and FINES accounts (A, C, D, H, F ...).
SELECT a.status_code,
       COUNT(*)                               AS accounts,
       TO_CHAR(MIN(a.account_status_date), 'YYYY-MM-DD') AS earliest_status_date,
       TO_CHAR(MAX(a.account_status_date), 'YYYY-MM-DD') AS latest_status_date
FROM   acmaster a
JOIN   scheme_master s ON s.scheme_mst_id = a.scheme_mst_id
WHERE  UPPER(s.scheme_name) LIKE 'MAIIC%' OR UPPER(s.scheme_name) LIKE 'FINES%'
GROUP BY a.status_code
ORDER BY a.status_code;

```

```

-- DD_12_transaction_types.csv
-- Transaction type codes posted on the MAIIC and FINES loan accounts, with the
-- direction of the amounts. POSITIVE_ROWS and NEGATIVE_ROWS show the sign
-- convention for each code.
SELECT cv.trantype,
       COUNT(*)                               AS postings,
       SUM(CASE WHEN cv.transamt > 0 THEN 1 ELSE 0 END) AS positive_rows,
       SUM(CASE WHEN cv.transamt < 0 THEN 1 ELSE 0 END) AS negative_rows,
       SUM(cv.transamt)                             AS net_amount,
       SUM(CASE WHEN cv.ac_glcode = a.glcode THEN 1 ELSE 0 END) AS rows_on_loan_gl,
       TO_CHAR(MIN(cv.transaction_date), 'YYYY-MM-DD') AS first_posting,
       TO_CHAR(MAX(cv.transaction_date), 'YYYY-MM-DD') AS last_posting
FROM   cumvouch cv
JOIN   acmaster a
      ON a.new_ac_number = cv.new_ac_number AND a.sub_ac_number = cv.sub_account_no
JOIN   scheme_master s ON s.scheme_mst_id = a.scheme_mst_id
WHERE  (UPPER(s.scheme_name) LIKE 'MAIIC%' OR UPPER(s.scheme_name) LIKE 'FINES%')
AND    cv.delete_flag = 'N'
GROUP BY cv.trantype
ORDER BY cv.trantype;

```

```

-- DD_13_gl_codes_posted.csv
-- Every GL code the MAIIC and FINES loan accounts post to, by transaction type.
-- The extracts read only the loan's own GL code; this shows what else is posted
-- against the same accounts (fee income, penalties, interest in suspense).
SELECT cv.ac_glcode,
       gm.gltitle,
       cv.trantype,
       COUNT(*)                               AS postings,
       SUM(cv.transamt)                             AS net_amount,
       TO_CHAR(MIN(cv.transaction_date), 'YYYY-MM-DD') AS first_posting,
       TO_CHAR(MAX(cv.transaction_date), 'YYYY-MM-DD') AS last_posting
FROM   cumvouch cv
JOIN   acmaster a
      ON a.new_ac_number = cv.new_ac_number AND a.sub_ac_number = cv.sub_account_no
JOIN   scheme_master s ON s.scheme_mst_id = a.scheme_mst_id
LEFT JOIN glmaster gm ON gm.cbs_glcode = cv.ac_glcode
WHERE  (UPPER(s.scheme_name) LIKE 'MAIIC%' OR UPPER(s.scheme_name) LIKE 'FINES%')

```

```

AND    cv.delete_flag = 'N'
GROUP  BY cv.ac_glcode, gm.gltitle, cv.trantype
ORDER  BY cv.ac_glcode, cv.trantype;

-- DD_14_other_codes.csv
-- The remaining code columns the extracts decode: repayment frequency and the
-- moratorium units.
SELECT 'MULTI_INST_DETAILS.INSTALLMENT_PAY_MODE' AS code_column,
       m.installment_pay_mode                     AS code_value,
       COUNT(*)                                  AS rows_using_it
FROM   multi_inst_details m
WHERE  m.delete_flag = 'N'
GROUP  BY m.installment_pay_mode
UNION ALL
SELECT 'ACLOANMASTER.PMOROTORIUM_UNIT', al.pmoratorium_unit, COUNT(*)
FROM   acloanmaster al
GROUP  BY al.pmoratorium_unit
UNION ALL
SELECT 'ACLOANMASTER.IMOROTORIUM_UNIT', al.imoratorium_unit, COUNT(*)
FROM   acloanmaster al
GROUP  BY al.imoratorium_unit
ORDER  BY 1, 2;

-- DD_15_row_counts.csv
-- Exact row counts for the smaller loan tables. A zero against
-- LOAN_RESCHEDULE_DETAILS would explain why RESTRUCTURE_DATE is blank on every
-- row of Extract A.
SELECT 'LOAN_RESCHEDULE_DETAILS' AS table_name, COUNT(*) AS row_count FROM loan_reschedule_details
UNION ALL
SELECT 'LOAN_WRITEOFF_MASTER',      COUNT(*) FROM loan_writeoff_master
UNION ALL
SELECT 'LOAN_WRITEOFF_TRANS',      COUNT(*) FROM loan_writeoff_trans
UNION ALL
SELECT 'LOAN_DISBURSMENT_SCHEDULE', COUNT(*) FROM loan_disbursement_schedule
UNION ALL
SELECT 'FIXLOAN_INST_CHART',        COUNT(*) FROM fixloan_inst_chart
UNION ALL
SELECT 'MULTI_INST_DETAILS',        COUNT(*) FROM multi_inst_details
UNION ALL
SELECT 'SCHEME_MASTER',             COUNT(*) FROM scheme_master;

=====
-- PART 3. ONE SAMPLE LOAN, EVERY COLUMN
-- The rows held for a single loan in each table, so that we can see what each
-- column actually contains. The account is Ebenezer Midian, one of the ten
-- sample loans already shared with us. Change the account number in each query
-- if another loan is preferred. Export each result as its own CSV.
=====

-- DD_16a_sample_acmaster.csv
SELECT * FROM acmaster WHERE new_ac_number = '000104430000084';

-- DD_16b_sample_acloanmaster.csv
SELECT * FROM acloanmaster WHERE new_ac_number = '000104430000084';

-- DD_16c_sample_disbursement_schedule.csv
SELECT * FROM loan_disbursement_schedule WHERE new_ac_number = '000104430000084'
ORDER BY disb_schedule_date;

-- DD_16d_sample_installment_chart.csv
SELECT * FROM fixloan_inst_chart WHERE new_ac_number = '000104430000084'
ORDER BY installment_date, inst_chart_det_id;

-- DD_16e_sample_installment_plan.csv
SELECT * FROM multi_inst_details WHERE new_ac_number = '000104430000084'
ORDER BY inst_det_id;

-- DD_16f_sample_ledger.csv
-- Every posting for the loan on every GL code, not only the loan's own.
SELECT * FROM cumvouch WHERE new_ac_number = '000104430000084'
ORDER BY transaction_date, cumvouch_det_id;

-- DD_16g_sample_balances.csv
SELECT * FROM account_balance WHERE new_ac_number = '000104430000084'
ORDER BY transaction_date;

-- DD_16h_sample_reschedule.csv
SELECT * FROM loan_reschedule_details WHERE new_ac_number = '000104430000084';

-- DD_16i_sample_writeoff.csv
SELECT * FROM loan_writeoff_master WHERE new_ac_number = '000104430000084';

-----
-- End of file. Twenty-four result sets in all: DD_01 to DD_15, and DD_16a to DD_16i.
-----

```

Result of Write

File created successfully at: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Data Dictionary Scripts\MAIIC_EBanker_Data_Dictionary_Queries.sql (file state is current in your context – no need to Read it back)

CLAUDE (2026-10-05 13:16:34 UTC)

Written for: Barry Makumba or the vendor's database person, who will run the queries on E-Banker.

The data dictionary scripts are saved as one SQL file with 24 read-only queries. I could not test them, because I have no access to the Oracle database, so expect that one or two may need a small fix on first run.

The file has three parts:

- **Structure (DD_01 to DD_08):** these read only Oracle's own catalogue. They list every table in the schema, every column with its type and how populated it is, the keys and indexes that show how tables join, and the views and procedures that already read the loan tables. Two of them search for tables we have not been told about, such as fees, rate history, restructures and the link to the loan origination system.
- **Codes and counts (DD_09 to DD_15):** these answer what the values mean.
 - DD_10 returns every setting on the MAIIC and FinES schemes, which is where Interest Policy and the other scheme settings should be.
 - DD_12 lists the transaction type codes with the sign of their amounts.
 - DD_13 lists every GL code the loan accounts post to, which should show where the fees go.
 - DD_15 counts the rows in the reschedule table, to confirm why the restructure date is blank.
- **One sample loan (DD_16a to DD_16i):** every column of every loan table for Ebenezer Midian, one of the ten sample loans, including its full ledger on all GL codes.

Things to know before sending it:

- **Schema name:** the queries find the schema by looking for the ACMaster table, so nobody has to type it in. If DD_01 returns nothing, the login cannot see that table and we need the schema owner's name.
- **Same login as the extracts:** parts 2 and 3 use the table names directly, so they must be run as the user that runs the extract procedures.
- **CSV only:** the header asks for each result to be exported as CSV and not re-saved in Excel, to avoid the date scrambling.
- **Possible failure:** DD_15 counts LOAN_WRITEOFF_TRANS, which I only know from a comment in the cashflow script. If that table does not exist, remove that line.
- **Vendor sensitivity:** the vendor may be reluctant to release the full schema listing (DD_02, DD_03). The rest still gives us what we need for the amended extracts.

Files are in C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Data Dictionary Scripts:

- MAIIC_EBanker_Data_Dictionary_Queries.sql

USER (2026-10-05 13:17:19 UTC)

Can you open the data dictionary query in notepad

Tool call: PowerShell

description: Open the data dictionary SQL file in Notepad

--- command ---

```
Start-Process notepad.exe -ArgumentList '"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Data Dictionary Scripts\MAIIC_EBanker_Data_Dictionary_Queries.sql"'
```

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-05 13:17:32 UTC)

I've sent MAIIC_EBanker_Data_Dictionary_Queries.sql to Notepad, so it should be open on your screen now. The command returned without an error, but I can't see your screen to confirm the window appeared.

USER (2026-10-05 13:25:57 UTC)

Add the scheme code and Interest Policy to Extract A and correct the Fixed/Variable decode. => have you identified the table that stores these?

Partly: two of the three are identified from the scripts, but the Interest Policy column is not.

- **Scheme code:** it is in `scheme_master`. The facility script already joins that table through `acmaster.scheme_mst_id`, so the scheme ID is available today, and the scheme name is already being output as `PRODUCT_TYPE`. I don't know whether there is also a separate user-facing scheme code column.
- **Fixed/Variable decode:** it comes from `scheme_master.floating_flag`. The fix is in the script's own CASE statement, which labels F as Fixed and I as Variable, so there is no table to find.
- **Interest Policy:** not identified. None of the three scripts reads it. The manual puts it on the Scheme Master screen, so my expectation is a column on `scheme_master`, but I have not seen the column name and it could also be held per account.

The dictionary queries are what settle this. `DD_10` returns every column of `scheme_master` for the MAIIC and FInES schemes, and `DD_03` lists every column in the schema, so a policy column held per account would show up there too.

which schema query to run first?

Run **DD_01** first. It confirms the login can see `ACMASTER` and the other tables the extracts use, and shows which schema owns them. If it returns no rows, none of the other catalogue queries will return anything either.

After that, the order that gets answers fastest is:

1. **DD_10** — every column of `scheme_master` for the MAIIC and FInES schemes. This is the one most likely to show the Interest Policy column and whether there is a scheme code besides the ID.
2. **DD_03** — every column of every table. This is the full dictionary, and it catches Interest Policy if it is held per account instead.
3. **DD_06 and DD_07** — the tables we have not been told about (fees, rate history, restructures).

The rest can follow in file order.

Can you create a subfolder in the raw queries onedrive folder to keep these query requests

Tool call: PowerShell

```
description: Create the query requests subfolder and move the data dictionary file into it
--- command ---
$new = "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC"
$oldDir = "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Data Dictionary Scripts"
$file = "MAIIC_EBanker_Data_Dictionary_Queries.sql"
New-Item -ItemType Directory -Force -Path $new | Out-Null
Get-ChildItem -LiteralPath $oldDir | Select-Object Name, Length, LastWriteTime | Format-Table -AutoSize
Move-Item -LiteralPath (Join-Path $oldDir $file) -Destination (Join-Path $new $file)
if (-not (Get-ChildItem -LiteralPath $oldDir)) { Remove-Item -LiteralPath $oldDir -Confirm:$false; "old empty folder removed" }
Get-ChildItem -LiteralPath "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts" -Recurse |
Select-Object @{n='Path';e={$_.FullName.Replace('C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw
Query Scripts\',')}}}, Length, LastWriteTime | Format-Table -AutoSize
```

Result of PowerShell

Name	Length	LastWriteTime
MAIIC_EBanker_Data_Dictionary_Queries.sql	16173	05/10/2026 15:16:15

Remove-Item : Cannot remove item C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Data Dictionary Scripts: You do not have sufficient access rights to perform this operation.
At line:7 char:50
+ ... \$oldDir)) { Remove-Item -LiteralPath \$oldDir -Confirm:\$false; "old e ...
+ ~~~~~
+ CategoryInfo : PermissionDenied: (C:\Users\wadza\...tionary Scripts:DirectoryInfo) [Remove-Item], IOExc
ption
+ FullyQualifiedErrorId : RemoveFileSystemItemUnauthorizedAccess,Microsoft.PowerShell.Commands.RemoveItemCommand
old empty folder removed

Path	Length	LastWriteTime
Query Requests to MAIIC		05/10/2026 15:35:51
CASHFLOW_REGISTER.txt	9616	05/10/2026 13:10:20
FACILITY_REGISTER.txt	6999	05/10/2026 13:10:20
INTEREST_RECON.txt	3306	05/10/2026 13:10:20
Query Requests to MAIIC\MAIIC_EBanker_Data_Dictionary_Queries.sql	16173	05/10/2026 15:16:15

CLAUDE (2026-10-05 13:35:58 UTC)

The subfolder is created and the data dictionary file is now in it:

2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_EBanker_Data_Dictionary_Queries.sql

I moved the file rather than copying it, so there is only one version.

- **Old folder left behind:** the empty 3. Project Execution\Data Dictionary Scripts folder would not delete (Windows refused access), so it is still there and can be removed by hand.
- **Notepad:** the window you have open still points at the old location. Close it and reopen the file from the new folder before making any edits.

The amended extract queries will go in the same subfolder when I draft them.

USER (2026-10-05 13:36:52 UTC)

Can you extract the queries we want to run in priority order and why so that we start running them

Tool call: Bash

```
description: Build a run-order SQL file from the existing queries, with the reason for each
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && cat > runorder.py <<'PYEOF'
import re
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC"
src = open(D + r"\MAIIC_EBanker_Data_Dictionary_Queries.sql", encoding="utf-8").read().replace("\r\n", "\n")
blocks = {}
cur = None
for line in src.split("\n"):
    m = re.match(r"^-- (DD_\d+[a-z]?)(\S+)\.csv\s*$", line)
    if m:
        cur = m.group(1); blocks[cur] = dict(file=f"{m.group(1)}_{m.group(2)}.csv", lines=[]); continue
    if line.startswith("--====") or line.startswith("-----"):
        cur = None; continue
    if cur:
        blocks[cur]["lines"].append(line)
sql = {}
for k, b in blocks.items():
    body = [l for l in b["lines"] if not l.startswith("--")]
    sql[k] = "\n".join(body).strip()
    assert sql[k].upper().startswith("SELECT") and sql[k].endswith(";"), k
assert len(sql) == 24, len(sql)

ORDER = [
    ("DD_01", "Gate check. Confirms this login can see ACMaster and the other tables the extracts read, and shows which schema owns them."),
    ("If it returns no rows, stop: the catalogue queries below will also return nothing, and we need the schema owner's name."),
    ("DD_10", "Finds Interest Policy. Returns every setting on the MAIIC and FInES schemes, which is where the manual says Interest Policy, Floating Flag, the interest calculation base, the instalment base and the EMI type are held."),
    ("Look for a policy column with values such as P, F or M, and for a scheme code besides SCHEME_MST_ID. Compare FLOATING_FLAG with the Fixed/Variable labels in Extract A."),
    ("DD_03", "The data dictionary itself: every column of every table. Needed before any amended extract can be written, and it catches Interest Policy if it is held per account and not per scheme."),
    ("Large result. Export it and carry on; we search it afterwards."),
    ("DD_12", "Transaction type codes and the sign of their amounts. Settles the direction of every row in Extract B and what the six interest codes in Extract C are."),
    ("For each code, are the amounts all positive, all negative or mixed? Codes 301, 302, 303, 305, 120 and 308 to 311 matter most."),
    ("DD_13", "Shows every GL code the loan accounts post to. The extracts read only the loan's own GL, so this is how we find where fees, penalties and suspended interest go."),
    ("Look for postings on the fee income codes 4871, 4872 and 4873 and on the interest income codes 4215 to 4219."),
    ("DD_15", "Row counts for the small loan tables. A zero on LOAN_RESTRUCTURE_DETAILS explains the blank restructure date on every row of Extract A."),
    ("If LOAN_WRITEOFF_TRANS does not exist, delete that line and run again."),
    ("DD_11", "Account status codes in use. Gives the counts behind the undocumented H and F codes."),
    ("We still need the meaning of each code from the vendor."),
    ("DD_09", "Every scheme with its account count. Proves whether the MAIIC% / FInES% name filter leaves any loan scheme out of the extracts."),
    ("Any scheme marked N with loan accounts on it is outside the extracts today."),
    ("DD_16f", "Sample loan (Ebenezer Midian): the full ledger on every GL code. Shows the real posting pattern, the signs, and any fee postings for one loan we already know well."),
    ("We expect five disbursement postings totalling 314,900,900."),
    ("DD_16d", "Sample loan: the stored instalment chart. Confirms the schedule is held as a table, shows whether it has a row at the start date, and whether it carries a balance column."),
    ("DD_16c", "Sample loan: the disbursement schedule table. Extract A showed one tranche for a loan the ledger shows drawn in five."),
    ("DD_16a", "Sample loan: the account master row, every column. Shows what else is held per account (rate, status, any policy or margin fields)."),
    ("DD_16b", "Sample loan: the loan master row, every column. Moratorium fields, sanction details and any link to the origination system."),
    ("DD_16e", "Sample loan: the instalment plan rows (repayment frequency)."),
    ("DD_16g", "Sample loan: the balance history. Shows which balances are held besides principal."),
    ("DD_16h", "Sample loan: reschedule rows, if any."),
    ("DD_16i", "Sample loan: write-off rows, if any."),
    ("DD_06", "Finds every table that carries the loan account key. This is how we discover tables nobody has mentioned: fees, charges, rate history."),
    ("DD_07", "Finds tables by subject from their names (rate, PLR, charge, fee, audit, application, IFRS)."),
    ("DD_14", "Remaining code lists: repayment frequency and moratorium units."),
    ("DD_04", "Declared keys and foreign keys: how the tables join."),
    ("Vendor systems often declare few keys; a short result is normal."),
    ("DD_05", "Indexes. Where keys are not declared, the unique indexes show what identifies a row."),
    ("DD_02", "The full table list with row counts."),
    ("DD_08", "Existing views and procedures that read the loan tables, names only. Shows which standard reports sit on the same data."),
]
assert sorted(k for k, _, _ in ORDER) == sorted(sql), set(sql) ^ {k for k, _, _ in ORDER}
out = ["-----"]
-- MAIIC E-Banker: data dictionary queries IN RUN ORDER
-- Prepared by Dupleix Institute, 5 October 2026
--
-- The same 24 read-only queries as MAIIC_EBanker_Data_Dictionary_Queries.sql,
-- re-ordered so that the ones that unblock the most come first. Nothing here
-- creates, changes or deletes anything.
--
-- Run as the user that runs PROC_BUILD_MAIIC_FACILITY_REGISTER.
-- Run one query at a time. Export each result as CSV (not Excel) under the file
-- name shown, and do not re-save the CSV in Excel.
--
```

```

-- TIER 1 (runs 1 to 8)  answers the open questions on Extracts A, B and C
-- TIER 2 (runs 9 to 17) one sample loan, every column
-- TIER 3 (runs 18 to 24) the rest of the dictionary
-----
"""
def wrap(prefix, text, width=76):
    words, lines, cur = text.split(), [], ""
    for w in words:
        if len(cur) + len(w) + 1 > width:
            lines.append(cur); cur = w
        else:
            cur = (cur + " " + w).strip()
    lines.append(cur)
    pad = " " * len(prefix)
    return "\n".join(("-- " + (prefix if i == 0 else pad) + 1) for i, l in enumerate(lines))
for i, (k, why, look) in enumerate(ORDER, 1):
    if i in (1, 9, 18):
        tier = {1: "TIER 1. ANSWERS THE OPEN QUESTIONS ON EXTRACTS A, B AND C", 9: "TIER 2. ONE SAMPLE LOAN, EVERY COLUMN
(Ebenezer Midian, 000104430000084)", 18: "TIER 3. THE REST OF THE DICTIONARY"}[i]
        out.append("\n--" + "=" * 78 + "\n-- " + tier + "\n--" + "=" * 78 + "\n")
        out.append(f"-- RUN {i} of 24 | save as {blocks[k]['file']}")
        out.append(wrap("WHY: ", why))
        if look:
            out.append(wrap("NOTE: ", look))
        out.append(sql[k] + "\n\n")
open(D + r"\RUN ORDER - data dictionary queries by priority.sql", "w", encoding="utf-8", newline="\r\n").write("\n".join(out))
print("written; queries:", len(ORDER))
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python runorder.py && sed -n '1,60p' "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2.
Documents from clients/Raw Query Scripts/Query Requests to MAIIC/RUN ORDER - data dictionary queries by priority.sql" && grep
-c "^-- RUN " "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/
RUN ORDER - data dictionary queries by priority.sql"
{
    "timeout": 120000
}

```

Result of Bash

written; queries: 24

```
-----
-- MAIIC E-Banker: data dictionary queries IN RUN ORDER
-- Prepared by Dupleix Institute, 5 October 2026
--
-- The same 24 read-only queries as MAIIC_EBanker_Data_Dictionary_Queries.sql,
-- re-ordered so that the ones that unblock the most come first. Nothing here
-- creates, changes or deletes anything.
--
-- Run as the user that runs PROC_BUILD_MAIIC_FACILITY_REGISTER.
-- Run one query at a time. Export each result as CSV (not Excel) under the file
-- name shown, and do not re-save the CSV in Excel.
--
-- TIER 1 (runs 1 to 8)  answers the open questions on Extracts A, B and C
-- TIER 2 (runs 9 to 17) one sample loan, every column
-- TIER 3 (runs 18 to 24) the rest of the dictionary
-----

=====
-- TIER 1. ANSWERS THE OPEN QUESTIONS ON EXTRACTS A, B AND C
=====

-- RUN 1 of 24 | save as DD_01_schema.csv
-- WHY: Gate check. Confirms this login can see ACMASTER and the other tables the
-- extracts read, and shows which schema owns them.
-- NOTE: If it returns no rows, stop: the catalogue queries below will also return
-- nothing, and we need the schema owner's name.
SELECT t.owner,
       t.table_name,
       t.num_rows,
       TO_CHAR(t.last_analyzed, 'YYYY-MM-DD') AS stats_date
FROM   all_tables t
WHERE  t.table_name IN ('ACMASTER', 'ACLOANMASTER', 'CUSTOMER_MAST', 'SCHEME_MASTER', 'GLMASTER',
                        'CUMVOUCH', 'ACCOUNT_BALANCE', 'FIXLOAN_INST_CHART', 'MULTI_INST_DETAILS',
                        'LOAN_DISBURSMENT_SCHEDULE', 'LOAN_RESCHEDULE_DETAILS',
                        'LOAN_WRITEOFF_MASTER', 'LOAN_WRITEOFF_TRANS')

ORDER BY t.owner, t.table_name;

-- RUN 2 of 24 | save as DD_10_scheme_settings.csv
-- WHY: Finds Interest Policy. Returns every setting on the MAIIC and FINES schemes,
-- which is where the manual says Interest Policy, Floating Flag, the interest
-- calculation base, the instalment base and the EMI type are held.
-- NOTE: Look for a policy column with values such as P, F or M, and for a scheme
-- code besides SCHEME_MST_ID. Compare FLOATING_FLAG with the Fixed/Variable
-- labels in Extract A.
SELECT s.*
FROM   scheme_master s
WHERE  UPPER(s.scheme_name) LIKE 'MAIIC%' OR UPPER(s.scheme_name) LIKE 'FINES%'
ORDER BY s.scheme_name;

-- RUN 3 of 24 | save as DD_03_columns.csv
-- WHY: The data dictionary itself: every column of every table. Needed before any
-- amended extract can be written, and it catches Interest Policy if it is held
-- per account and not per scheme.
-- NOTE: Large result. Export it and carry on; we search it afterwards.
SELECT col.table_name,
       col.column_id          AS column_no,
       col.column_name,
24
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: PowerShell

description: Open the run-order file in Notepad

--- command ---

```
Start-Process notepad.exe -ArgumentList '"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\RUN ORDER - data dictionary queries by priority.sql"'
```

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-05 13:37:57 UTC)

Written for: whoever runs the queries on E-Banker with you (Barry or the vendor).

The queries are now in a second file in run order, each with its reason and the file name to save the result under. It holds the same 24 queries as the original, untested for the same reason: I have no Oracle access. I've sent it to Notepad so you can copy one query at a time.

Tier 1: answers the open questions on Extracts A, B and C

| Run | Query | Why it comes here |

```

|---|---|---|
| 1 | DD_01 schema | Gate check: confirms the login can see the tables. If it returns nothing, stop and get the schema owner's name. |
| 2 | DD_10 scheme settings | Most likely place to find Interest Policy, plus the Floating Flag values and any scheme code. |
| 3 | DD_03 all columns | The dictionary itself. Needed before any amended extract is written, and it catches Interest Policy if it is held per account. |
| 4 | DD_12 transaction types | The sign of each transaction code, which settles the direction of every Extract B row. |
| 5 | DD_13 GL codes posted | Shows where fees, penalties and suspended interest go, since the extracts read only the loan's own GL. |
| 6 | DD_15 row counts | A zero on the reschedule table explains the blank restructure dates. |
| 7 | DD_11 status codes | Counts behind the undocumented H and F codes. |
| 8 | DD_09 all schemes | Proves whether the name filter leaves any loan scheme out. |

```

Tier 2: one sample loan, every column (runs 9 to 17)

These are the DD_16 queries for Ebenezer Midian, ledger first. The ledger should show five disbursement postings totalling 314,900,900, which checks the sign convention and the tranche question on a loan we already know.

Tier 3: the rest of the dictionary (runs 18 to 24)

Table discovery (DD_06, DD_07), remaining code lists (DD_14), keys and indexes (DD_04, DD_05), the full table list (DD_02) and existing reports on the same tables (DD_08). None of these block the amended extracts.

If time is short, runs 1 to 5 are enough for me to start drafting the amended Extract A and B queries.

Files are in 2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC:

- RUN ORDER - data dictionary queries by priority.sql
- MAIIC_EBanker_Data_Dictionary_Queries.sql

USER (2026-10-05 13:42:17 UTC)

in the raw queries sub folder, write a plain language pdf report analysis of the raw queries provided, what they do and where they come short and why we need the data queries to be run first and then the other queries to answer some questions. Please be comprehensive

Tool call: Skill

skill: anthropic-skills:pdf

Result of Skill

Launching skill: anthropic-skills:pdf

Tool call: Bash

description: Pull the exact evidence figures for the report from the delivered extracts

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && cat > maiic_check4.py <<'PYEOF'
import pandas as pd, numpy as np, warnings
warnings.filterwarnings("ignore")
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 30); pd.set_option("display.max_colwidth", 45)
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
A = pd.read_excel(D + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
B = pd.read_excel(D + r"\Extract B.xlsx", sheet_name="Sheet1", dtype={"LOAN_ACCOUNT_NUMBER": str})
C = pd.read_excel(D + r"\ExtractC_Jan2025-July2026.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
A["snap"] = A["AS_OF_DATE"].astype(str).str[:10]
x = A[A.snap == "2025-01-01"].set_index("LOAN_ACCOUNT_NUMBER"); y = A[A.snap == "31-12-2025"].set_index("LOAN_ACCOUNT_NUMBER")
print("GENERATED_ON:", {s: A[A.snap == s].GENERATED_ON.astype(str).str[:16].unique().tolist() for s in A.snap.unique()})
for col in ["INTEREST_RATE", "ACCOUNT_STATUS", "SANCTIONED_AMOUNT", "TENOR", "NUM_INSTALMENTS", "RATE_BASIS",
"PRINCIPAL_DISBURSED", "REPAYMENT_FREQUENCY", "DISBURSEMENT_TRANCES"]:
    xa, ya = x[col], y[col].reindex(x.index)
    d = ~(xa == ya) | (xa.isna() & ya.isna())
    print(f"{col}: differs on {int(d.sum())} | one side blank {int((xa.isna() ^ ya.isna()).sum())} | sample
{list(zip(xa[d].head(3).tolist(), ya[d].head(3).tolist()))}")
print("open dates of accounts where disbursed differs:", x.loc[(x.PRINCIPAL_DISBURSED.isna() ^
y.PRINCIPAL_DISBURSED.reindex(x.index).isna())].LOAN_START_DATE.astype(str).str[:10].head(8).tolist())
print("accounts closed (status Closed) with closure date in 2025 per latest snapshot:", int(y[y.ACCOUNT_STATUS ==
"Closed"].ACTUAL_CLOSURE_DATE.astype(str).str.contains("2025").sum()), "| status in Jan-2025 snapshot for those:",
x.loc[y[(y.ACCOUNT_STATUS == "Closed") &
y.ACTUAL_CLOSURE_DATE.astype(str).str.contains("2025").index].ACCOUNT_STATUS.value_counts().to_dict()])
print("status counts latest:", y.ACCOUNT_STATUS.value_counts().to_dict(), "| NUM_INSTALMENTS zero:", int((y.NUM_INSTALMENTS ==
0).sum()), "| repayment freq blank:", int(y.REPAYMENT_FREQUENCY.isna().sum()))
act = B[B.SCHEDULED_ACTUAL_FLAG == "Actual"]
bx = act[act.LOAN_ACCOUNT_NUMBER == "000104430000062"].head(4)
for _, r in bx.iterrows(): print(" ledger 062:", r.GL_POSTING_REF, str(r.TRANSACTION_DATE)[:10], r.TRANSACTION_TYPE,
f"{r.TOTAL_AMOUNT:,.2f}", f"{r.BALANCE_AFTER_TRANSACTION:,.2f}")
print("B negative amounts:", int((B.TOTAL_AMOUNT < 0).sum()), "| 305 note rows:",
int(B.ROW_NOTE.astype(str).str.contains("ESTIMATED").sum()))
sch = B[B.SCHEDULED_ACTUAL_FLAG == "Scheduled"]
print("scheduled rows with balance:", int(sch.BALANCE_AFTER_TRANSACTION.notna().sum()), "of", len(sch))
M = C[(C.PERIOD_TYPE == "MONTH") & (C.RUN_ID == 41)]
miss = y[~y.index.isin(C.LOAN_ACCOUNT_NUMBER.unique())]
print("active accounts with no interest row in C:")
print(miss[miss.ACCOUNT_STATUS == "Active"][["CUSTOMER_NAME", "GL_ACCOUNT_TITLE", "LOAN_START_DATE", "SANCTIONED_AMOUNT",
"OPENING_BALANCE"]].to_string())
bi = act[act.TRANSACTION_TYPE == "Interest"].groupby("LOAN_ACCOUNT_NUMBER").TOTAL_AMOUNT.sum()
bo = act[act.TRANSACTION_TYPE == "Other/Adjustment"].groupby("LOAN_ACCOUNT_NUMBER").TOTAL_AMOUNT.sum()
cd = M[M.PERIOD_YEAR == 2025].groupby("LOAN_ACCOUNT_NUMBER").INTEREST_INCOME_POSTED.sum()
j = pd.concat([bi.rename("B"), bo.rename("B_other"), cd.rename("C")], axis=1).fillna(0); j["diff"] = j.B - j.C
print(j[j["diff"].abs() >= 1].join(y.CUSTOMER_NAME).round(2).to_string())
print("C run 41 negative month rows:", int((M.INTEREST_INCOME_POSTED < 0).sum()), "| months covered:",
sorted(set(zip(M.PERIOD_YEAR, M.PERIOD_MONTH))[0], sorted(set(zip(M.PERIOD_YEAR, M.PERIOD_MONTH)))[-1])
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maiic_check4.py 2>&1 | cut -c1-330
{
  "timeout": 300000
}
```

Result of Bash

```
GENERATED_ON: {'2025-01-01': ['2026-03-08 15:32'], '31-12-2025': ['2026-03-08 16:29']}
INTEREST_RATE: differs on 0 | one side blank 0 | sample []
ACCOUNT_STATUS: differs on 0 | one side blank 0 | sample []
SANCTIONED_AMOUNT: differs on 0 | one side blank 0 | sample []
TENOR: differs on 0 | one side blank 0 | sample []
NUM_INSTALMENTS: differs on 0 | one side blank 0 | sample []
RATE_BASIS: differs on 0 | one side blank 0 | sample []
PRINCIPAL_DISBURSED: differs on 0 | one side blank 0 | sample []
REPAYMENT_FREQUENCY: differs on 42 | one side blank 25 | sample [('Monthly', nan), ('Monthly', nan), ('Monthly', nan)]
DISBURSEMENT_TRANCHES: differs on 0 | one side blank 0 | sample []
open dates of accounts where disbursed differs: []
accounts closed (status Closed) with closure date in 2025 per latest snapshot: 0 | status in Jan-2025 snapshot for those: {}
status counts latest: {'Active': 86, 'Closed': 46, 'Dormant': 25, 'H': 21, 'F': 3} | NUM_INSTALMENTS zero: 53 | repayment freq
blank: 25
  ledger 062: 1050102-36989 31-01-2025 Interest 21,497,946.96 788,531,241.57
  ledger 062: 1050102-40041 18-02-2025 Principal+Interest 53,589,482.23 734,941,759.34
  ledger 062: 1050102-40429 28-02-2025 Interest 19,428,764.01 754,370,523.35
  ledger 062: 1050102-44495 25-03-2025 Principal+Interest 26,794,741.11 727,575,782.24
B negative amounts: 0 | 305 note rows: 539
scheduled rows with balance: 0 of 775
active accounts with no interest row in C:
      CUSTOMER_NAME      GL_ACCOUNT_TITLE      LOAN_START_DATE  SANCTIONED_AMOUNT
OPENING_BALANCE
LOAN_ACCOUNT_NUMBER
000104420000003      Micholess Creamery  MAIIC Agricultural Loans      28-07-2020      2.000000e+07
  0.00
000104450000012      Maluso Cooperative Union  FInES Agricultural Loans      22-12-2021      1.151150e+07
  0.00
000104440000005      Belk Inc  Quasi Equity Investment  2024-01-10 00:00:00      6.900000e+07
  NaN
000104470000004      NASCOMEX Limited      NASCOMEX Limited      25-03-2024      2.500000e+09
  NaN
000104430000081  PINNACLE FINANCIAL SERVICES LIMITED  MAIIC Industrial Loans      24-07-2025      1.000000e+09
  NaN
000104430000088      ANCHOR PROCESSORS LIMITED  MAIIC Industrial Loans      29-10-2025      3.000000e+08
  NaN
000104430000083      JAT GROUP LTD  MAIIC Industrial Loans  2025-05-08 00:00:00      2.092700e+08
  NaN
000105810000003      ECOGEN LIMITED      Fines Investment Loan      16-05-2025      1.000000e+08
  NaN
000105810000004      ECOGEN LIMITED      Fines Investment Loan      22-05-2025      1.050600e+08
  NaN
000104460000111      ECOGEN LIMITED  FInES Industrial Loans      26-05-2025      1.050600e+08
90457186.24
      B      B_other      C      diff      CUSTOMER_NAME
LOAN_ACCOUNT_NUMBER
000104420000032      2.989358e+07      51362.16      2.984222e+07      51362.16      African Honey products Industries
000104420000052      4.232978e+07      913541.02      4.141624e+07      913541.02      Kambewu Organic Fertilizer Investments
000104420000062      3.842140e+07      9018700.44      3.640270e+07      2018700.44      Happie Foods Limited
000104430000002      1.172968e+08      10457152.33      1.068396e+08      10457152.33      Edge View Academy Limited
000104430000068      1.318992e+08      69330809.54      6.256840e+07      69330809.54      Nyamunyamu processors limited
000104430000073      9.052253e+07      2395117.82      8.812741e+07      2395117.82      SAILE FINANCIAL SERVICES LIMITED COMPANY
C run 41 negative month rows: 2 | months covered: (2025, 1) (2026, 7)
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\maaic_report.py
--- content ---
"""Plain-language report on the three MAIIC raw query scripts (PDF, reportlab)."""
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.enums import TA_LEFT
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import (BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Table, TableStyle,
                                PageBreak, KeepTogether, Image, ListFlowable, ListItem)

F = r"C:\Windows\Fonts"
pdfmetrics.registerFont(TTFont("Seg", F + r"\segoeui.ttf"))
pdfmetrics.registerFont(TTFont("Seg-B", F + r"\segoeuib.ttf"))
pdfmetrics.registerFont(TTFont("Seg-I", F + r"\segoeuii.ttf"))
pdfmetrics.registerFont(TTFont("Seg-BI", F + r"\segoeuiiz.ttf"))
pdfmetrics.registerFontFamily("Seg", normal="Seg", bold="Seg-B", italic="Seg-I", boldItalic="Seg-BI")
pdfmetrics.registerFont(TTFont("Mono", F + r"\consola.ttf"))

NAVY = colors.HexColor("#1B2A41"); ORANGE = colors.HexColor("#E07B00"); GREY = colors.HexColor("#5B6470")
LIGHT = colors.HexColor("#F4F6F9"); LINE = colors.HexColor("#C9D1DB"); AMBER = colors.HexColor("#FFF4E0")

S = {
    "title": ParagraphStyle("title", fontName="Seg-B", fontSize=24, leading=29, textColor=NAVY, spaceAfter=4),
    "sub": ParagraphStyle("sub", fontName="Seg", fontSize=13, leading=17, textColor=GREY, spaceAfter=10),
    "meta": ParagraphStyle("meta", fontName="Seg", fontSize=9.5, leading=13, textColor=GREY),
    "h1": ParagraphStyle("h1", fontName="Seg-B", fontSize=15, leading=19, textColor=NAVY, spaceBefore=14, spaceAfter=6),
    "h2": ParagraphStyle("h2", fontName="Seg-B", fontSize=11.5, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4),
    "p": ParagraphStyle("p", fontName="Seg", fontSize=10, leading=14.2, textColor=colors.black, spaceAfter=6,
alignment=TA_LEFT),
    "li": ParagraphStyle("li", fontName="Seg", fontSize=10, leading=14, textColor=colors.black),
    "cell": ParagraphStyle("cell", fontName="Seg", fontSize=8.6, leading=11.4),
    "cellb": ParagraphStyle("cellb", fontName="Seg-B", fontSize=8.6, leading=11.4),
    "head": ParagraphStyle("head", fontName="Seg-B", fontSize=8.8, leading=11.4, textColor=colors.white),
    "box": ParagraphStyle("box", fontName="Seg", fontSize=10, leading=14.2, textColor=colors.black),
    "note": ParagraphStyle("note", fontName="Seg-I", fontSize=8.8, leading=12, textColor=GREY, spaceAfter=6),
}
W = A4[0] - 36 * mm
story = []

def H1(t): story.append(Paragraph(t, S["h1"]))
def H2(t): story.append(Paragraph(t, S["h2"]))
def P(t): story.append(Paragraph(t, S["p"]))
def NOTE(t): story.append(Paragraph(t, S["note"]))
def GAP(h=4): story.append(Spacer(1, h))

def BUL(items, numbered=False):
    story.append(ListFlowable([ListItem(Paragraph(i, S["li"]), leftIndent=14, spaceAfter=3) for i in items],
                              bulletType="1" if numbered else "bullet", bulletFontName="Seg", bulletFontSize=9,
                              start=None if numbered else "*", leftIndent=14, bulletColor=ORANGE if not numbered else NAVY))
    GAP(4)

def TAB(header, rows, widths, bold_first=False, keep=False):
    data = [[Paragraph(h, S["head"]) for h in header]]
    for r in rows:
        data.append([Paragraph(str(c), S["cellb"]) if (bold_first and i == 0) else S["cell"] for i, c in enumerate(r)])
    tw = [W * x for x in widths]
    t = Table(data, colWidths=tw, repeatRows=1)
    st = [("BACKGROUND", (0, 0), (-1, 0), NAVY), ("VALIGN", (0, 0), (-1, -1), "TOP"),
          ("LEFTPADDING", (0, 0), (-1, -1), 5), ("RIGHTPADDING", (0, 0), (-1, -1), 5),
          ("TOPPADDING", (0, 0), (-1, -1), 4), ("BOTTOMPADDING", (0, 0), (-1, -1), 5),
          ("LINEBELOW", (0, 0), (-1, -1), 0.4, LINE), ("BOX", (0, 0), (-1, -1), 0.6, LINE)]
    for i in range(1, len(data)):
        if i % 2 == 0:
            st.append(("BACKGROUND", (0, i), (-1, i), LIGHT))
    t.setStyle(TableStyle(st))
    story.append(KeepTogether([t]) if keep else t)
    GAP(8)

def BOX(paras, colour=AMBER):
    t = Table([[[Paragraph(x, S["box"]) for x in paras]]], colWidths=[W])
    t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, -1), colour), ("LINEBEFORE", (0, 0), (0, -1), 3, ORANGE),
                          ("LEFTPADDING", (0, 0), (-1, -1), 10), ("RIGHTPADDING", (0, 0), (-1, -1), 10),
                          ("TOPPADDING", (0, 0), (-1, -1), 8), ("BOTTOMPADDING", (0, 0), (-1, -1), 8)]))
    story.append(t); GAP(8)

# ----- cover block
LOGO = r"C:\Users\wadza\OneDrive\2026\Tenders\PFA\Excel & Data Analysis Training\Basic Excel Course\3 Source Files\img\
dupleix-logo.png"
try:
    im = Image(LOGO); r = im.imageHeight / im.imageWidth; im.drawWidth = 42 * mm; im.drawHeight = 42 * mm * r; im.hAlign =
```

```

"LEFT"
    story.append(im); GAP(10)
except Exception:
    pass
story.append(Paragraph("MAIIC E-Banker extract scripts", S["title"]))
story.append(Paragraph("What the three scripts do, where they fall short, and which queries to run next", S["sub"]))
story.append(Paragraph("Prepared by Duplex Institute for MAIIC Finance and ICT · 5 October 2026", S["meta"]))
story.append(Paragraph("Based on: the three script files received on 5 October 2026; Extracts A and B as delivered on 3 August 2026; "
    "Extract C as re-run and delivered on 26 August 2026; the E-Banker user manual. We have read the scripts
and "
    "tested them against those files. We have not seen the database itself.", S["meta"])))
GAP(10)

# ----- 1 summary
H1("1. Summary")
P("MAIIC has sent us the three database scripts that produce Extract A (the facility register), Extract B (the cash flow "
    "register) and Extract C (the interest posted on each loan). We read each script line by line and then tested what it "
    "implies against the extract files we already hold.")
P("The scripts are tidy, and they prove that the data we need is held in E-Banker. They also explain, for the first time, "
    "most of the puzzles we raised in September. But each script makes choices that hide or change information the effective "
    "interest rate (EIR) engine depends on, and some of the fields we need are not read by any of the three.")
H2("Three findings that affect figures already in use")
BUL([
    "<b>The Extract C file counts 2025 three times.</b> The script keeps every earlier run, and the file we received holds
three "
    "runs. Added up as delivered, 2025 interest is MWK 7,663,172,090. The correct figure, from the latest run alone, is "
    "MWK 2,554,390,697.",
    "<b>The Fixed and Variable labels in Extract A are the wrong way round.</b> The script labels the system's code for
Floating "
    "as \"Fixed\" and its code for Fixed as \"Variable\". This is why the labels contradicted what the loan books show.",
    "<b>The \"Interest\" rows in Extract B are interest charged to the loan, not cash received.</b> They are the same postings "
    "that Extract C reports as interest income. Treating them as money received would count the interest twice.",
], numbered=True)
H2("What we are asking for")
P("Two short rounds of read-only queries, before any extract is rewritten:")
BUL([
    "<b>First, the data dictionary queries.</b> These list the tables and columns E-Banker holds. We need them first because
the "
    "fields still missing, starting with Interest Policy, do not appear in any of the three scripts. We do not yet know what "
    "they are called or which table they sit in, so an amended extract cannot be written.",
    "<b>Second, the question queries.</b> These return codes, counts and one sample loan. They answer specific open questions: "
    "which way each transaction type moves the balance, where the fees are posted, what the status codes are, and why the "
    "restructure date is blank.",
    "<b>Then the amended extracts,</b> written once, against the real structure.",
], numbered=True)
P("All 24 queries are in the file <b>RUN ORDER - data dictionary queries by priority.sql</b>, in the order to run them. "
    "The first five are enough for us to start drafting the amended extracts.")

# ----- 2 what we received
H1("2. What we received")
P("Each file is a stored procedure: a routine saved inside the database that fills a results table when someone runs it. "
    "The results table is then copied out to Excel and sent to us.")
TAB(["File", "Builds", "What it produces"], [
    ["FACILITY_REGISTER.txt", "Extract A", "One row for each loan account as at a chosen date: who the customer is, the terms
of the loan, and the balance."],
    ["CASHFLOW_REGISTER.txt", "Extract B", "One row for each posting on a loan in a date range, with a running balance,
followed by write-offs and by scheduled instalments not yet paid."],
    ["INTEREST_RECON.txt", "Extract C", "The interest posted on each loan, totalled for each month and for each year."],
], [0.26, 0.12, 0.62], bold_first=True)
P("Two things are missing from the files. The definitions of the results tables are not included, and neither is the step "
    "that copies the results out to Excel. The second one matters: it is where the dates were scrambled (section 6).")
H2("The E-Banker tables the scripts read")
P("This is the first time we have seen E-Banker's real table names. Between them the three scripts read thirteen tables.")
TAB(["Table", "What it holds, in plain language", "What the scripts take from it"], [
    ["ACMASTER", "The account master: one row for each account", "Account number, customer, GL code, interest rate, opening
date, status, currency, scheme"],
    ["ACLOANMASTER", "The loan details for each account", "Sanctioned amount, tenor, expiry date, moratorium periods"],
    ["CUSTOMER_MAST", "The customer master", "Customer name"],
    ["SCHEME_MASTER", "The scheme (the product variant) each account is opened on", "Scheme name, Floating Flag, compound or
simple, days in the year"],
    ["GLMASTER", "The list of GL codes", "GL title"],
    ["CUMVOUCH", "The ledger: every posting", "Date, transaction type code, amount, GL code"],
    ["ACCOUNT_BALANCE", "Balance history by date", "Principal balance"],
    ["FIXLOAN_INST_CHART", "The instalment chart (the EMI chart)", "Instalment date, amount, principal part, interest part,
amount recovered"],
    ["MULTI_INST_DETAILS", "The instalment plan", "Repayment frequency"],
    ["LOAN_DISBURSMENT_SCHEDULE", "The disbursement schedule", "Date and amount"],
    ["LOAN_RESCHEDULE_DETAILS", "Reschedules (restructures)", "Latest reschedule date"],
    ["LOAN_WRITEOFF_MASTER", "The write-off register", "Write-off date and amount"],
    ["LOAN_WRITEOFF_TRANS", "Write-off postings", "Named in a comment only; not read"],
], [0.27, 0.36, 0.37], bold_first=True)
H2("Which loans are included")
P("All three scripts pick loans the same way: any account whose scheme name begins with \"MAIIC\" or \"FINES\". A loan on a "
    "scheme named differently would be left out, and the scripts cannot tell us whether any exist. The same rule also brings in "
    "accounts that are not loans: Extract A includes five Quasi Equity Investment accounts and one NASCOMEX Limited account.")

# ----- 3 Extract A
H1("3. Extract A: the facility register")
H2("What the script does")
P("For the date it is given, the script removes any earlier result for that date and builds it again. For each account it "

```

```

"collects the loan's details from the account, loan, customer and scheme masters. It adds up the disbursement schedule, "
"counts the rows in the instalment chart, picks up any write-off and the latest reschedule date, and looks up the most "
"recent principal balance on or before the date.")
H2("What is good about it")
BUL([
    "The account number and sub-account number are carried through cleanly, so Extract A joins to B and C.",
    "Where a balance is missing, the script says why in a note: the loan opened after the date, it closed before the date, "
    "or no balance was found and it needs checking by hand.",
    "The day-count basis and whether interest is compound or simple come straight from the scheme, which settled our day-count
decision.",
    "It can be run again for the same date without creating duplicates.",
])
H2("Where it falls short")
TAB(["#", "What the script does", "Why it matters", "What the delivered file shows"], [
    ["1", "<b>Labels the rate type the wrong way round.</b> It reads the scheme's Floating Flag and labels code F as \"Fixed\"
and code I as \"Variable\". The manual (pages 19 to 20) gives F as Floating and I as Fixed.",
        "The label says the opposite of how the loan behaves, so it cannot be used to decide which loans reprice.",
        "All 57 MAIIC Agricultural and Industrial accounts are labelled Fixed, and all 101 FinES Agricultural and Industrial
accounts Variable. The loan books show the reverse. The 13 Term Loans still do not fit after the labels are swapped, so the
flag is not the deciding field."],
    ["2", "<b>Leaves out Interest Policy and the scheme code.</b> It reads the scheme master but takes only four settings from
it.",
        "Interest Policy is the setting that decides whether a loan follows the Reserve Bank rate. The scheme identifier is
already in the script's hands and is simply not output.", "Not in the file."],
    ["3", "<b>Only the balance is taken as at the date.</b> The rate, status, amounts and dates are read as they stand on the
day the script is run.",
        "The file looks like a picture of the book at an earlier date, but most of it is today's picture. It cannot show what a
loan's rate was at 1 January 2025.",
        "Rate, status, sanctioned amount and tenor are identical in the two snapshots for all 181 accounts. Lake Malawi
Aquaculture's 31 December 2025 row lists a drawdown dated 28 April 2026."],
    ["4", "<b>Takes disbursements from the disbursement schedule table,</b> not from the ledger.",
        "The schedule holds one planned entry. Loans drawn in several parts show as one, so the dates money actually went out are
lost.",
        "Every populated row has exactly one tranche. Ebenezer Midian shows one, while the ledger shows five."],
    ["5", "<b>Counts the rows of the instalment chart</b> for the number of instalments, and takes the earliest chart date as
the first repayment date.",
        "The chart appears to start with a row at the loan's start date. That row is counted as an instalment and reported as the
first repayment.",
        "The count is the tenor in months plus one. The first repayment date equals the loan start date on 63 of 128 rows. 53
accounts have no chart rows at all."],
    ["6", "<b>Reads the reschedule table for the restructure date.</b>", "If that table is empty, restructures are not being
recorded in E-Banker's reschedule function, and the history lives only in Finance's spreadsheet.",
        "Blank on every row."],
    ["7", "<b>Translates only three status codes</b> (Active, Closed, Dormant). Any other code is passed through as it is.",
        "The meaning of the other codes still has to come from the vendor.",
        "H on 21 accounts and F on 3."],
    ["8", "<b>Removes the sign from the balance,</b> and reads the principal balance only.", "An overpaid loan would look the
same as one that is owed. Any interest balance held separately is not shown. The column is also headed \"opening balance\"
although it is the balance at the date.", ""],
    ["9", "<b>Does not limit write-offs and closures to the date.</b>", "A write-off made after the date appears in an earlier
snapshot.", ""],
    ["10", "<b>Joins the moratorium period and its unit into text,</b> and leaves out the moratorium type.", "Values arrive as \"
3 M\" or \"0\", and we cannot tell a principal-only moratorium from a full one.", "Both formats appear."],
    ["11", "<b>Reads no fee table at all.</b>", "Fees are the main reason an effective rate differs from the contract rate.",
        "No fee fields."],
], [0.04, 0.34, 0.30, 0.32])
NOTE("One further observation: the repayment frequency differs between the two snapshots on 42 accounts, although the script "
    "does not use the date to find it. The two snapshots were generated about an hour apart, so the script was probably "
    "changed between the two runs.")

```

```

# ----- 4 Extract B
H1("4. Extract B: the cash flow register")
H2("What the script does")
P("Each run is given a new run number. The script then gathers three kinds of row for the date range it is given:")
BUL([
    "<b>Actual rows:</b> every ledger posting made on the loan's own GL code, with a running balance. It names each row from "
    "its transaction type code: 301 Disbursement, 302 Principal, 303 Interest, 305 Principal+Interest, 340 and 341 Penalty, "
    "900 and 901 Fee, and anything else \"Other/Adjustment\".",
    "<b>Write-off rows:</b> taken from the write-off register.",
    "<b>Scheduled rows:</b> rows of the instalment chart that have not yet been fully recovered.",
])
H2("What is good about it")
BUL([
    "The running balance is built from the loan's whole history before the date range is applied, so the balance is right "
    "even when only one year is extracted.",
    "Every row carries a reference back to the ledger.",
    "The script is honest about its limits. It marks every estimated figure in a note and lists its own known limitations at
the end.",
    "Its date range already runs to 31 December 2026 by default, so later periods need no rewrite.",
])
H2("Where it falls short")
TAB(["#", "What the script does", "Why it matters", "What the delivered file shows"], [
    ["1", "<b>Removes the direction of every amount.</b> The ledger holds signed amounts; the script outputs each one without
its sign.",
        "A disbursement and a repayment look the same. Direction has to be guessed from the type label.", "No negative value
anywhere in 2,478 rows."],
    ["2", "<b>Presents interest charged as if it were a cash flow.</b> Type 303 is the monthly interest charge added to the
loan.",
        "For the effective interest rate, the cash flows are the money lent and the money repaid. Interest charged is not cash.
Counting it as a receipt would count the interest twice.",
        "On one account: interest of 21,497,946.96 on 31 January 2025 takes the balance up to 788,531,241.57; a repayment of 53,

```

589,482.23 on 18 February takes it down to 734,941,759.34."],

["3", "Estimates the split of each repayment (type 305) between principal and interest with a formula: balance, times the loan's rate today, times the days since the previous posting.",

"The formula uses today's rate, not the rate at the time, and counts days from the last posting of any kind. The interest has in any case already been charged separately. The split has no meaning, and the engine does not need it: it needs the amount, the date and the direction.",

"539 rows carry the \"estimated\" note."],

["4", "Puts interest adjustments under \"Other/Adjustment\" with no sign.",

"These rows reduce interest. Extract C subtracts them; Extract B shows them as positive amounts under a vague label.",

"On six accounts they are exactly the difference between B and C, MMW 85,166,683 in total (Appendix B)."],

["5", "Reads only postings on the loan's own GL code.", "Fees, penalties or suspended interest posted to other GL codes for the same loan are never seen.", "2 fee rows in 2,478."],

["6", "Includes only instalments not yet fully recovered, inside the date range, with no balance.", "The scheduled rows are the unpaid remainder, not the schedule. The full chart is stored in the table and can be extracted whole.", "No balance on any of the 775 scheduled rows."],

["7", "Builds the reference from an internal row number.", "It can be traced in the database, but it is not a voucher number a user can look up on screen. The script's own note says the voucher number field was blank.", ""],

["8", "Takes write-offs from the register only.", "There is no ledger posting to check them against, and a write-off that is also in the ledger would appear twice.", ""],

], [0.04, 0.34, 0.30, 0.32])

----- 5 Extract C

H1("5. Extract C: interest posted by loan")

H2("What the script does")

P("Each run is given a new run number, and earlier runs are kept. The script adds up ledger postings of six transaction " types (120, 303, 308, 309, 310 and 311) made on the loan's own GL code. It reverses their sign so that interest charged " shows as positive income, and totals them for each loan for each month. It also writes a yearly total row for each loan " into the same table.")

H2("What is good about it")

BUL([

"It keeps the signs when adding, so reversals and adjustments are netted off correctly.",

"Each row lists the ledger references it was built from.",

"For 2025 it agrees with the interest rows of Extract B within MWK 1 on 101 of 107 accounts, so the two extracts support each other.",

])

H2("Where it falls short")

TAB(["#", "What the script does", "Why it matters", "What the delivered file shows"], [

["1", "Keeps every earlier run in the same table. The file sent to us was exported without choosing one run.",

"Any total taken from the file as delivered counts 2025 three times.",

"Three runs (see the table below). 1,012 loan-months appear three times with identical amounts."],

["2", "Writes monthly rows and yearly total rows together.", "Adding the whole column counts every amount twice.",

"All rows add up to twice the monthly rows."],

["3", "Measures one side only. It adds up interest charged to loan accounts. It does not read the interest income GL codes.",

"It is not yet a reconciliation. Its total still has to be agreed to the movement on the income codes in the trial balance. Interest charged to a loan but credited to suspense would be counted here as income.", ""],

["4", "Leaves out accounts with no interest posting in the period.", "A missing account and an account with no interest look the same.",

"61 of 181 accounts are absent: 46 closed, 10 active, 4 dormant and 1 with status H (Appendix C)."],

["5", "Uses six type codes whose meanings we have not been given.", "We can see that 303 is the monthly charge and that the others include reductions, but not what each one is.", ""],

["6", "Groups by the date of posting.", "Interest posted late lands in the month it was posted, not the month it was earned.", "The catch-up postings we found in February 2026."],

], [0.04, 0.34, 0.30, 0.32])

TAB(["Run number", "Generated", "Rows", "Accounts", "Period covered", "2025 interest, MWK"], [

["7", "3 August 2026", "1,119", "107", "2025", "2,554,390,697"],

["25", "3 August 2026", "1,119", "107", "2025", "2,554,390,697"],

["41", "25 August 2026", "1,758", "120", "January 2025 to July 2026", "2,554,390,697"],

["As delivered", "", "3,996", "", "", "7,663,172,090"],

], [0.15, 0.17, 0.10, 0.12, 0.26, 0.20], keep=True)

P("Run 41 is the one to use. Its January to July 2026 total is MWK 2,739,597,510, which appears once and is not affected.")

----- 6 dates

H1("6. Where the dates were scrambled")

P("In September we reported that dates were transposed in every extract, with the day and month swapped. The scripts show " that this does not happen inside the database: all three hold proper database dates from start to finish. The damage " happens afterwards, when the results are copied out through Excel. The cure is in the export, not the scripts: save as " CSV, write dates year first (2026-10-05), and do not open and re-save the file in Excel.")

----- 7 three findings

H1("7. The three findings, and what to do about each")

TAB(["Finding", "What to do now"], [

["Extract C counts 2025 three times when the delivered file is added up.",

"Use run 41 only, and monthly rows only. Any 2025 interest figure already taken from this file should be checked, including the figures loaded into our own engine. We have not yet confirmed whether our import selected a single run."],

["Fixed and Variable are the wrong way round in Extract A.",

"Do not use the label. The engine will keep deriving each loan's rate behaviour from the monthly loan books until Interest Policy is extracted."],

["\"Interest\" rows in Extract B are charges, not receipts.",

"Our 24 September research report described these rows as cash received, and we will correct it. In the engine, cash out is the disbursements and cash in is the repayments."],

], [0.36, 0.64], keep=True)

----- 8 settled / open

H1("8. What the scripts settle, and what is still open")

TAB(["Question", "What the scripts tell us", "Status", "Query that closes it"], [

["Is the instalment chart stored, or worked out on screen?", "Stored, in a table, with the principal and interest split. 53 accounts have no rows.", "Settled", "Run 10 confirms the columns"],

["Why is the number of instalments always the tenor plus one?", "It is a count of chart rows, and the chart appears to begin with a row at the start date.", "Explained", "Run 10"],

["Why does every loan show one disbursement tranche?", "The script reads the disbursement schedule, not the ledger.",

```

"Explained", "Runs 9 and 11"],
["Why is the restructure date blank everywhere?", "The script reads the reschedule table, which appears to be empty.",
"Explained", "Run 6"],
["Why is there no debit or credit direction in Extract B?", "The script strips the sign from every amount.", "Explained",
"Run 4"],
["Why are 61 accounts missing from Extract C?", "Accounts with no interest posting produce no row.", "Explained", "None
needed"],
["Why were the dates scrambled?", "Not in the scripts; in the export through Excel.", "Explained", "None needed"],
["Which scheme is each account on?", "The scheme identifier is already read by the script. It only needs to be output.",
"Available", "Run 2"],
["Where is Interest Policy held?", "No script reads it.", "Open", "Runs 2 and 3"],
["Where are fees held against a loan?", "No script reads a fee table, and both ledger scripts look at the loan's own GL
code only.", "Open", "Runs 5, 18 and 19"],
["What do transaction types 120 and 308 to 311 mean?", "The script names eight codes. These five are grouped as interest in
Extract C and as \"Other\" in Extract B.", "Open", "Run 4, then the vendor"],
["What do status codes H and F mean?", "The script passes them through without a meaning.", "Open", "Run 7, then the
vendor"],
["Is there a table of rate changes?", "No script reads one.", "Open", "Runs 18 and 19"],
["Does the scheme name rule leave any loans out?", "Cannot be told from the scripts.", "Open", "Run 8"],
], [0.27, 0.37, 0.11, 0.25])

# ----- 9 why dictionary first
H1("9. Why the data dictionary queries have to come first")
P("A data dictionary is the database's own list of what it holds: every table, every column in each table, the kind of data "
  "in each column, and how the tables link to one another. Oracle keeps this list automatically. Reading it does not touch "
  "any customer or transaction record.")
P("Everything we know about how E-Banker is built comes from the three scripts: thirteen table names, and only the columns "
  "those scripts happen to use. The fields we still need are, by definition, the ones the scripts do not use. The manual "
  "tells us that Interest Policy is on the Scheme Master screen, but the name on a screen is not the name of a column, and a "
  "query can only ask for a column by its real name.")
H2("Three reasons the order matters")
BUL([
  "<b>An amended extract cannot be written against a column we cannot name.</b> A query that guesses a column name fails when
  "
  "it is run. Each failed attempt costs a round trip between Duplex, MAIIC and the vendor. The August extracts took two "
  "deliveries, three weeks apart, to reach their present state.",
  "<b>One pass answers many questions.</b> The same listing shows where Interest Policy sits, whether the instalment chart "
  "carries a balance, whether there is a fee table, whether there is a rate history table, and what links a loan back to "
  "the origination system. Asking for these one at a time is how the last two months were spent.",
  "<b>It is the lowest-risk request we can make.</b> The queries only read. Most of them read Oracle's catalogue and not "
  "MAIIC's data, and they take minutes to run.",
], numbered=True)
H2("Why the question queries are still needed afterwards")
P("The dictionary gives structure, not meaning. It will tell us that a column called TRANTYPE exists and holds a number. It "
  "will not tell us what 308 means, or whether a repayment is stored as a positive or a negative amount. Vendor databases "
  "rarely carry written descriptions. So the second round asks the data itself: which codes are in use, how often, in which "
  "direction, and what a single well-understood loan looks like in every table.")
BOX(["<b>In short:</b> the dictionary tells us what can be asked for. The question queries tell us what the answers mean. "
  "Only then can the three extracts be rewritten once, correctly."])

# ----- 10 the queries
H1("10. The queries, in the order to run them")
P("The run numbers below match the file <b>RUN ORDER - data dictionary queries by priority.sql</b>. Each query in that file "
  "is headed with the reason for it and the file name to save its result under.")
H2("Tier 1: answers the open questions on Extracts A, B and C")
TAB(["Run", "Query", "What it returns, in plain language", "The question it answers"], [
  ["1", "DD_01", "Whether this login can see the thirteen tables, and who owns them", "The gate check. If it returns nothing,
the catalogue queries will too, and we need the owner's name."],
  ["2", "DD_10", "Every setting held on the MAIIC and FInES schemes", "Where is Interest Policy? What are the real Floating
Flag values? Is there a scheme code?"],
  ["3", "DD_03", "Every column of every table: the dictionary itself", "What is each field called and where does it sit?"],
  ["4", "DD_12", "Each transaction type code, how often it is used, and whether its amounts are positive or negative", "Which
way does each posting move the balance? What are the unexplained codes used for?"],
  ["5", "DD_13", "Every GL code the loan accounts post to, by transaction type", "Where do fees, penalties and suspended
interest go?"],
  ["6", "DD_15", "The number of rows in the smaller loan tables", "Is the reschedule table empty?"],
  ["7", "DD_11", "The account status codes in use, with counts", "How many accounts carry H and F?"],
  ["8", "DD_09", "Every scheme, with the number of accounts on it", "Does the \"MAIIC or FInES\" name rule leave any loan
scheme out?"],
], [0.07, 0.10, 0.42, 0.41])
H2("Tier 2: one sample loan, every column")
P("Runs 9 to 17 return every row held for one loan in each table: the ledger on all GL codes, the instalment chart, the "
  "disbursement schedule, the account and loan masters, the instalment plan, the balance history, and any reschedule or "
  "write-off. The loan is Ebenezer Midian, one of the ten sample loans. We chose it because we already know from Extract B "
  "that it was drawn in five tranches totalling MWK 314,900,900, so the result can be checked the moment it arrives.")
H2("Tier 3: the rest of the dictionary")
P("Runs 18 to 24 find the tables nobody has mentioned (any table carrying the loan account number, and tables named for "
  "rates, fees, charges, audit or applications), and list the remaining code values, the keys and indexes that show how "
  "tables join, the full table list, and the existing reports that already read the loan tables. None of these holds up the "
  "amended extracts.")
H2("Practical notes for whoever runs them")
BUL([
  "Every query is read-only. Nothing is created, changed or deleted.",
  "Run them with the same login that runs the three extract procedures.",
  "Run one query at a time and export each result as CSV, not Excel. Please do not open and re-save the CSV in Excel.",
  "Run 6 counts a table (LOAN_WRITEOFF_TRANS) that we know only from a comment in the cash flow script. If it does not "
  "exist, remove that line and run the query again.",
  "The vendor may prefer not to release the full listing of tables and columns (runs 3 and 23). The other queries still "
  "give us most of what we need.",
  "We wrote the queries without access to the database, so they have not been run. One or two may need a small correction "
  "the first time.",

```

```

])

# ----- 11 what follows
H1("11. What follows: the amended extracts")
P("Once the Tier 1 results are back, the extracts can be amended. These are small changes to queries that already exist.")
TAB(["Extract", "Changes"], [
    ["<b>A: facility register</b>", "Add the scheme identifier and Interest Policy, with the other scheme settings. Output the
raw Floating Flag and the raw status code instead of a translated label. Take drawdowns from the ledger. Keep the sign on the
balance. Add the moratorium type. Say plainly which fields are as at the date and which are as at the day of the run."],
    ["<b>B: cash flow register</b>", "Output signed amounts and the raw transaction type code. Drop the estimated principal and
interest split. Include postings on every GL code for the loan accounts, or add a separate fee extract. Run it from October
2024."],
    ["<b>New: instalment chart</b>", "The whole stored chart for every loan, paid and unpaid rows alike, with the amount
recovered against each."],
    ["<b>C: interest posted</b>", "Export the latest run only, and monthly rows only. Show accounts with no postings as zero.
Run it from October 2024 to date. Agree its total to the interest income codes in the trial balance."],
    ["<b>All of them</b>", "Deliver as CSV with dates written year first."],
], [0.24, 0.76], bold_first=False)
H2("The sequence")
BUL([
    "MAIIC runs Tier 1 (runs 1 to 8) and sends the eight CSV files. Runs 1 to 5 are enough for us to begin.",
    "Duplex drafts the amended extract queries against the real table and column names.",
    "MAIIC runs Tiers 2 and 3 when convenient. They refine the work but do not hold it up.",
    "MAIIC runs the amended extracts. Duplex loads them and reconciles interest to the trial balance.",
], numbered=True)

# ----- appendices
story.append(PageBreak())
H1("Appendix A. The tests behind this report")
P("Each statement about the delivered files was tested on the files themselves. These are the results.")
TAB(["Test", "Result"], [
    ["Extract A: rows, accounts and snapshots", "362 rows; 181 accounts; two snapshots, 1 January 2025 and 31 December 2025"],
    ["Extract A: rate label by product (31 December 2025 snapshot)", "Fixed: MAIIC Agricultural 16, MAIIC Industrial 41, MAIIC
Term Loan 13, Fines Investment Loan 4, Quasi Equity Investment 5. Variable: FINEs Agricultural 50, FINEs Industrial 51,
NASCOMEX Limited 1"],
    ["Extract A: fields compared between the two snapshots", "Interest rate, account status, sanctioned amount, tenor, number
of instalments, rate label, principal disbursed and tranche list: identical for all 181 accounts"],
    ["Extract A: Lake Malawi Aquaculture, 31 December 2025 snapshot", "Sanctioned 1,055,473,655; one tranche of 297,161,905
dated 28 April 2026"],
    ["Extract A: principal disbursed against sanctioned amount", "Populated on 157 accounts; equal to the sanctioned amount on
138"],
    ["Extract A: first repayment date against loan start date", "The same on 63 of the 128 accounts with a first repayment
date"],
    ["Extract A: accounts with no instalment chart rows", "53"],
    ["Extract A: account status (31 December 2025 snapshot)", "Active 86, Closed 46, Dormant 25, H 21, F 3"],
    ["Extract B: rows by kind", "2,478 rows. Actual 1,703: Interest 1,034, Principal+Interest 539, Disbursement 97, Other/
Adjustment 31, Fee 2. Scheduled 775, across 90 accounts"],
    ["Extract B: negative amounts", "None"],
    ["Extract B: scheduled rows carrying a balance", "0 of 775"],
    ["Extract C: runs in the delivered file", "Run 7: 1,119 rows. Run 25: 1,119 rows. Run 41: 1,758 rows"],
    ["Extract C: loan-months", "1,564 distinct loan-months; 1,012 appear three times and 552 once"],
    ["Extract C: run 7 against run 41 for 2025", "1,012 loan-months in both; amounts differ on none"],
    ["Extract C: 2025 interest", "All runs added together 7,663,172,090; run 41 alone 2,554,390,697"],
    ["Extract B interest rows against Extract C (run 41), 2025", "107 accounts in both. 101 agree within MWK 1. Extract B total
2,639,557,380; difference 85,166,683"],
    ["Extract C: accounts in Extract A with no row in Extract C", "61: Closed 46, Active 10, Dormant 4, H 1"],
], [0.42, 0.58])

H1("Appendix B. The six accounts where Extract B and Extract C disagree")
P("For 2025, the interest rows in Extract B exceed the interest in Extract C on six accounts. On five of them the difference "
"is exactly the account's \"Other/Adjustment\" rows in Extract B. Extract C has subtracted those postings; Extract B "
"shows them as positive amounts.")
TAB(["Account", "Customer", "Extract B interest", "Extract B other", "Extract C interest", "Difference"], [
    ["000104430000068", "Nyamunyamu Processors Limited", "131,899,213.54", "69,330,809.54", "62,568,404.00", "69,330,809.54"],
    ["000104430000002", "Edge View Academy Limited", "117,296,781.33", "10,457,152.33", "106,839,629.00", "10,457,152.33"],
    ["000104430000073", "Saile Financial Services Limited Company", "90,522,529.82", "2,395,117.82", "88,127,412.00", "2,395,
117.82"],
    ["000104420000062", "Happie Foods Limited", "38,421,398.44", "9,018,700.44", "36,402,698.00", "2,018,700.44"],
    ["000104420000052", "Kambewu Organic Fertilizer Investments", "42,329,780.02", "913,541.02", "41,416,239.00", "913,541.02"],
    ["000104420000032", "African Honey Products Industries", "29,893,579.16", "51,362.16", "29,842,217.00", "51,362.16"],
], [0.17, 0.27, 0.15, 0.13, 0.15, 0.13], keep=True)
NOTE("The Extract B and Extract C interest columns are rounded from the files to the nearest unit shown; the differences and "
"the \"other\" amounts are exact. On Happie Foods the \"other\" rows include a further 7,000,000 that is not an interest
adjustment.")

H1("Appendix C. Active accounts with no interest posted in Extract C")
P("These ten accounts are marked Active in Extract A but have no interest posting on their own GL code between January 2025 "
"and July 2026. Some may not have been drawn yet and some are not loans. Each one is worth a line of explanation from MAIIC.")
TAB(["Account", "Customer", "Product", "Sanctioned amount, MWK"], [
    ["000104470000004", "NASCOMEX Limited", "NASCOMEX Limited", "2,500,000,000"],
    ["000104430000081", "Pinnacle Financial Services Limited", "MAIIC Industrial Loans", "1,000,000,000"],
    ["000104430000088", "Anchor Processors Limited", "MAIIC Industrial Loans", "300,000,000"],
    ["000104430000083", "JAT Group Ltd", "MAIIC Industrial Loans", "209,270,000"],
    ["000104460000111", "Ecogen Limited", "FINEs Industrial Loans", "105,060,000"],
    ["000105810000004", "Ecogen Limited", "Fines Investment Loan", "105,060,000"],
    ["000105810000003", "Ecogen Limited", "Fines Investment Loan", "100,000,000"],
    ["000104440000005", "Belk Inc", "Quasi Equity Investment", "69,000,000"],
    ["000104420000003", "Micholess Creamery", "MAIIC Agricultural Loans", "20,000,000"],
    ["000104450000012", "Maluso Cooperative Union", "FINEs Agricultural Loans", "11,511,500"],
], [0.20, 0.36, 0.24, 0.20], keep=True)
NOTE("Sanctioned amounts are shown as rounded in our working of the file.")

```

```
H1("Appendix D. Terms used in this report")
TAB(["Term", "Meaning"], [
    ["Script, stored procedure", "A set of database instructions saved inside the database and run on request."],
    ["Query", "A question put to the database. A read-only query returns data and changes nothing."],
    ["Table, column", "A table is a list of records of one kind, such as accounts. A column is one field in that list, such as
the interest rate."],
    ["Data dictionary, catalogue", "The database's own list of its tables and columns."],
    ["Scheme", "In E-Banker, the product variant an account is opened on. Settings such as Interest Policy are held on the
scheme."],
    ["GL code", "The general ledger account a posting is made to. Each loan product has its own."],
    ["Transaction type code", "A number E-Banker gives each posting to say what kind it is, such as 301 for a disbursement."],
    ["Run number", "A number the script gives each time it is run, so that the results of different runs can be told apart."],
    ["Snapshot, as at date", "The position on a stated date, as opposed to the position today."],
    ["Instalment chart, EMI chart", "The loan's repayment schedule as held in E-Banker."],
    ["Effective interest rate (EIR)", "The rate that spreads all of a loan's cash flows, including fees, evenly over its life.
It is what IFRS 9 requires interest income to be recognised at."],
], [0.28, 0.72], bold_first=True)
```

```
# ----- build
def footer(canvas, doc):
    canvas.saveState()
    canvas.setStrokeColor(LINE); canvas.setLineWidth(0.5)
    canvas.line(18 * mm, 14 * mm, A4[0] - 18 * mm, 14 * mm)
    canvas.setFont("Seg", 8); canvas.setFillColor(GREY)
    canvas.drawString(18 * mm, 9.5 * mm, "MAIIC E-Banker extract scripts: analysis · Dupleix Institute · 5 October 2026")
    canvas.drawRightString(A4[0] - 18 * mm, 9.5 * mm, f"Page {doc.page}")
    canvas.restoreState()

OUT = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf"
doc = BaseDocTemplate(OUT, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=20 * mm,
    title="MAIIC E-Banker extract scripts: analysis", author="Dupleix Institute",
    subject="What the three raw query scripts do, where they fall short, and which queries to run next")
doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, id="f",
    leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)],
onPage=footer)])
doc.build(story)
print("written", OUT)
```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\maiic_report.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Get the exact figures for the report's appendix tables

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && ./songenv/Scripts/python -m pip install -q reportlab pypdf 2>&1 | tail -1; cat > maic_exact.py <<'PYEOF'
import pandas as pd, warnings
warnings.filterwarnings("ignore")
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
A = pd.read_excel(D + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
B = pd.read_excel(D + r"\Extract B.xlsx", sheet_name="Sheet1", dtype={"LOAN_ACCOUNT_NUMBER": str})
C = pd.read_excel(D + r"\ExtractC_Jan2025-July2026.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
y = A[A["AS_OF_DATE"].astype(str).str[:10] == "31-12-2025"].set_index("LOAN_ACCOUNT_NUMBER")
act = B[B.SCHEDULED_ACTUAL_FLAG == "Actual"]
bi = act[act.TRANSACTION_TYPE == "Interest"].groupby("LOAN_ACCOUNT_NUMBER").TOTAL_AMOUNT.sum()
bo = act[act.TRANSACTION_TYPE == "Other/Adjustment"].groupby("LOAN_ACCOUNT_NUMBER").TOTAL_AMOUNT.sum()
M = C[(C.PERIOD_TYPE == "MONTH") & (C.RUN_ID == 41)]
cd = M[M.PERIOD_YEAR == 2025].groupby("LOAN_ACCOUNT_NUMBER").INTEREST_INCOME_POSTED.sum()
j = pd.concat([bi.rename("B"), bo.rename("O"), cd.rename("C")], axis=1).fillna(0); j["d"] = j.B - j.C
for k, r in j[j.d.abs() >= 1].sort_values("d", ascending=False).iterrows():
    print(f'["{k}", "{y.CUSTOMER_NAME[k]}", "{r.B:.2f}", "{r.O:.2f}", "{r.C:.2f}", "{r.d:.2f}"],')
print("totals: B", f"{j.B.sum():.2f}", "C", f"{j.C.sum():.2f}", "diff", f"{j.d.sum():.2f}", "| sum of 6 diffs",
f"{j[j.d.abs() >= 1].d.sum():.2f}", "| max abs diff among the 101:", f"{j[j.d.abs() < 1].d.abs().max():.4f}")
for run in (7, 25, 41):
    m = C[(C.PERIOD_TYPE == "MONTH") & (C.RUN_ID == run)]
    print("run", run, "rows", int((C.RUN_ID == run).sum()), "accounts", m.LOAN_ACCOUNT_NUMBER.nunique(), "2025",
f"{m[m.PERIOD_YEAR == 2025].INTEREST_INCOME_POSTED.sum():.2f}", "2026", f"{m[m.PERIOD_YEAR ==
2026].INTEREST_INCOME_POSTED.sum():.2f}", "gen", C[C.RUN_ID == run].GENERATED_ON.astype(str).str[:16].unique().tolist())
allm = C[(C.PERIOD_TYPE == "MONTH")]
print("all runs 2025 monthly:", f"{allm[allm.PERIOD_YEAR == 2025].INTEREST_INCOME_POSTED.sum():.2f}", "| all rows sum",
f"{C.INTEREST_INCOME_POSTED.sum():.2f}", "| monthly rows sum", f"{allm.INTEREST_INCOME_POSTED.sum():.2f}")
miss = y[~y.index.isin(C.LOAN_ACCOUNT_NUMBER.unique())]
for k, r in miss[miss.ACCOUNT_STATUS == "Active"].sort_values("SANCTIONED_AMOUNT", ascending=False).iterrows():
    print(f'["{k}", "{r.CUSTOMER_NAME}", "{r.GL_ACCOUNT_TITLE}", "{r.SANCTIONED_AMOUNT:.2f}"],')
lm = y.loc["000106050000009"]; print("Lake Malawi:", lm.CUSTOMER_NAME, f"{lm.SANCTIONED_AMOUNT:.2f}", lm.DISBURSEMENT_TRANCHES)
eb = y.loc["000104430000084"]; print("Ebenezer:", eb.CUSTOMER_NAME, eb.DISBURSEMENT_TRANCHES, "| ledger disbursement rows:",
int((act.LOAN_ACCOUNT_NUMBER == "000104430000084") & (act.TRANSACTION_TYPE == "Disbursement")).sum()),
f"{act[(act.LOAN_ACCOUNT_NUMBER == '000104430000084') & (act.TRANSACTION_TYPE == 'Disbursement')].TOTAL_AMOUNT.sum():.2f}")
print("tranche entries per row:", y.DISBURSEMENT_TRANCHES.dropna().str.count(";").add(1).value_counts().to_dict(), "| ledger
062 customer:", y.CUSTOMER_NAME["000104430000062"])
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maic_exact.py 2>&1 | cut -c1-260
{
    "timeout": 300000
}
```

Result of Bash

```
["000104430000068", "Nyamunyamu processors limited", "131,899,213.80", "69,330,809.54", "62,568,404.26", "69,330,809.54"],
["000104430000002", "Edge View Academy Limited", "117,296,781.00", "10,457,152.33", "106,839,628.67", "10,457,152.33"],
["000104430000073", "SAILE FINANCIAL SERVICES LIMITED COMPANY", "90,522,529.35", "2,395,117.82", "88,127,411.53", "2,395,
117.82"],
["000104420000062", "Happie Foods Limited", "38,421,398.75", "9,018,700.44", "36,402,698.31", "2,018,700.44"],
["000104420000052", "Kambewu Organic Fertilizer Investments", "42,329,779.82", "913,541.02", "41,416,238.80", "913,541.02"],
["000104420000032", "African Honey products Industries", "29,893,579.18", "51,362.16", "29,842,217.02", "51,362.16"],
totals: B 2,639,557,380.04 C 2,554,390,696.73 diff 85,166,683.31 | sum of 6 diffs 85,166,683.31 | max abs diff among the 101:
0.0000
run 7 rows 1119 accounts 107 2025 2,554,390,696.73 2026 0.00 gen ['2026-03-08 16:14']
run 25 rows 1119 accounts 107 2025 2,554,390,696.73 2026 0.00 gen ['2026-03-08 16:15']
run 41 rows 1758 accounts 120 2025 2,554,390,696.73 2026 2,739,597,510.33 gen ['25-08-2026 15:40']
all runs 2025 monthly: 7,663,172,090.19 | all rows sum 20,805,539,201.04 | monthly rows sum 10,402,769,600.52
["000104470000004", "NASCOMEX Limited", "NASCOMEX Limited", "2,500,000,000.00"],
["000104430000081", "PINNACLE FINANCIAL SERVICES LIMITED", "MAIIC Industrial Loans", "1,000,000,000.00"],
["000104430000088", "ANCHOR PROCESSORS LIMITED", "MAIIC Industrial Loans", "300,000,000.00"],
["000104430000083", "JAT GROUP LTD", "MAIIC Industrial Loans", "209,270,000.00"],
["000104460000111", "ECOGEN LIMITED", "FINES Industrial Loans", "105,060,000.00"],
["000105810000004", "ECOGEN LIMITED", "Fines Investment Loan", "105,060,000.00"],
["000105810000003", "ECOGEN LIMITED", "Fines Investment Loan", "100,000,000.00"],
["000104440000005", "Belk Inc", "Quasi Equity Investment", "69,000,000.00"],
["000104420000003", "Micholess Creamery", "MAIIC Agricultural Loans", "20,000,000.00"],
["000104450000012", "Maluso Cooperative Union", "FINES Agricultural Loans", "11,511,500.00"],
Lake Malawi: Lake Malawi Aquaculture Limited 1,055,473,655.00 28-Apr-2026:297161905.00
Ebenezer: Ebenezer midian ltd 15-Jul-2025:1519500.00; 15-Jul-2025:298480500.00; 15-Jul-2025:5900000.00; 15-Jul-2025:900.00;
15-Jul-2025:9000000.00 | ledger disbursement rows: 5 314,900,900.00
tranche entries per row: {1: 128, 4: 10, 3: 8, 2: 6, 5: 4, 10: 1} | ledger 062 customer: Ifracon Limited
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

```
description: Recheck the disbursement tranche claim against Extract A and the ledger
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && cat > maiic_tr.py <<'PYEOF'
import pandas as pd, warnings, re
warnings.filterwarnings("ignore")
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
A = pd.read_excel(D + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
B = pd.read_excel(D + r"\Extract B.xlsx", sheet_name="Sheet1", dtype={"LOAN_ACCOUNT_NUMBER": str})
y = A["AS_OF_DATE"].astype(str).str[:10] == "31-12-2025"].set_index("LOAN_ACCOUNT_NUMBER")
dis = B[(B.SCHEDULED_ACTUAL_FLAG == "Actual") & (B.TRANSACTION_TYPE == "Disbursement")]
e = dis[dis.LOAN_ACCOUNT_NUMBER == "000104430000084"]
print("Ebenezer ledger disbursements:", [(str(d)[:19], f"{a:,.2f}") for d, a in zip(e.TRANSACTION_DATE, e.TOTAL_AMOUNT)])
print("Ebenezer A:", y.DISBURSEMENT_TRANCHES["000104430000084"], "| start", y.LOAN_START_DATE["000104430000084"])
g = dis.groupby("LOAN_ACCOUNT_NUMBER").agg(n=("TOTAL_AMOUNT", "size"), amt=("TOTAL_AMOUNT", "sum"), dates=("TRANSACTION_DATE",
lambda s: sorted(set(str(x)[:10] for x in s))))
rows = []
for k, r in g.iterrows():
    if k not in y.index: continue
    t = y.DISBURSEMENT_TRANCHES[k]
    if not isinstance(t, str): rows.append((k, r.n, None, r.amt, None, None, r.dates)); continue
    parts = [p.strip() for p in t.split(";")]
    adates = sorted(set(p.split(":")[0] for p in parts)); aamt = sum(float(p.split(":")[1]) for p in parts)
    rows.append((k, r.n, len(parts), r.amt, aamt, adates, r.dates))
df = pd.DataFrame(rows, columns=["acct", "ledger_n", "A_n", "ledger_amt", "A_amt", "A_dates", "ledger_dates"])
print("accounts with 2025 ledger disbursements:", len(df), "| A tranche list blank:", int(df.A_n.isna().sum()))
d2 = df.dropna(subset=["A_n"])
print("same count:", int((d2.ledger_n == d2.A_n).sum()), "| ledger more:", int((d2.ledger_n > d2.A_n).sum()), "| A more:",
int((d2.ledger_n < d2.A_n).sum()), "| same total amount:", int(((d2.ledger_amt - d2.A_amt).abs() < 1).sum()))
print(d2.head(8).to_string())
both = A.DISBURSEMENT_TRANCHES.dropna()
print("all 362 rows: populated", len(both), "| entries per row:", both.str.count(";").add(1).value_counts().to_dict())
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maiic_tr.py 2>&1 | cut -c1-400
{
  "timeout": 300000
}
```

Result of Bash

```
Ebenezer ledger disbursements: [('2025-08-08 00:00:00', '298,480,500.00'), ('2025-08-08 00:00:00', '9,000,000.00'),
('2025-08-08 00:00:00', '5,900,000.00'), ('2025-08-08 00:00:00', '1,519,500.00'), ('2025-08-08 00:00:00', '900.00')]
Ebenezer A: 15-Jul-2025:1519500.00; 15-Jul-2025:298480500.00; 15-Jul-2025:5900000.00; 15-Jul-2025:900.00;
15-Jul-2025:9000000.00 | start 15-07-2025
accounts with 2025 ledger disbursements: 35 | A tranche list blank: 0
same count: 28 | ledger more: 3 | A more: 4 | same total amount: 30
   acct  ledger_n  A_n  ledger_amt  A_amt  A_dates  ledger_dates
0  000104420000064      4      1  2.500000e+08  2.500000e+08  [03-Sep-2025]  [2025-05-09]
1  000104430000068      1      1  1.750000e+08  2.000000e+08  [14-Aug-2024]  [18-02-2025]
2  000104430000070      2      1  5.000000e+08  5.000000e+08  [09-Dec-2024]  [25-02-2025, 31-01-2025]
3  000104430000071      2      1  1.045292e+09  1.045292e+09  [16-Apr-2025]  [2025-02-05, 2025-06-05]
4  000104430000072      1      1  1.046210e+09  1.046210e+09  [22-Apr-2025]  [2025-07-05]
5  000104430000073      1      1  5.000000e+08  5.000000e+08  [23-Apr-2025]  [16-05-2025]
6  000104430000074      2      2  3.000000e+08  3.000000e+08  [08-Aug-2025, 30-Apr-2025]  [2025-08-08, 31-05-2025]
7  000104430000075      1      1  1.000000e+08  1.000000e+08  [14-Apr-2025]  [23-05-2025]
all 362 rows: populated 314 | entries per row: {1: 256, 4: 20, 3: 16, 2: 12, 5: 8, 10: 2}
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

```
description: Correct the tranche finding, build the PDF and list its pages
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && cat > patch_report.py <<'PYEOF'
p = "maic_report.py"
s = open(p, encoding="utf-8").read()
def rep(old, new, n=1):
    global s
    assert s.count(old) == n, (old[:60], s.count(old))
    s = s.replace(old, new)
# section 3, row 4: disbursements
rep('    "The schedule holds one planned entry. Loans drawn in several parts show as one, so the dates money actually went out are lost.",
    "Every populated row has exactly one tranche. Ebenezer Midian shows one, while the ledger shows five.",'',
    '    "The schedule gives the planned dates. The date money actually went out, which is when interest starts to run, is in the ledger and can be different.",
    "Ebenezer Midian: five entries dated 15 July 2025 in Extract A; the ledger shows the same five amounts paid out on 8 August 2025. Of the 35 accounts with ledger disbursements in 2025, the totals agree on 30 and the number of entries on 28."',
    '')
rep('    "The count is the tenor in months plus one. The first repayment date equals the loan start date on 63 of 128 rows. 53 accounts have no chart rows at all."', '',
    '    "Where a chart exists, the count is the tenor in months plus one on the rows we checked. The first repayment date equals the loan start date on 63 of 128 rows. 53 accounts have no chart rows at all."', '')
# section 4 evidence
rep('On one account: interest of 21,497,946.96', '"On one account (Ifracon Limited): interest of 21,497,946.96')
# section 8 row
rep('    ["Why does every loan show one disbursement tranche?", "The script reads the disbursement schedule, not the ledger.", "Explained", "Runs 9 and 11"],'',
    '    ["Do the disbursement tranches come through?", "Partly. 58 of the 314 populated rows list two to ten entries, so our September statement that every row had one was wrong. But the entries come from the disbursement schedule, with planned dates, not from the ledger.", "Corrected", "Runs 9 and 11"],''')
# tier 2 text
rep('We chose it because we already know from Extract B "
    "that it was drawn in five tranches totalling MKW 314,900,900, so the result can be checked the moment it arrives."', '',
    'We chose it because we already know two things about "
    "it: the ledger shows five amounts totalling MKW 314,900,900 paid out on 8 August 2025, and its disbursement schedule "
    "shows the same five amounts dated 15 July 2025. The result can be checked the moment it arrives."')
# section 11
rep("Take drawdowns from the ledger.", "Take drawdown dates and amounts from the ledger.")
# appendix A
rep('    ["Extract A: principal disbursed against sanctioned amount", "Populated on 157 accounts; equal to the sanctioned amount on 138"],'',
    '    ["Extract A: principal disbursed against sanctioned amount", "Populated on 157 accounts; equal to the sanctioned amount on 138"],
    ["Extract A: entries in the tranche list", "Populated on 314 of 362 rows. One entry on 256 rows; two to ten entries on 58"],
    ["Extract A tranche list against ledger disbursements in Extract B", "35 accounts have ledger disbursements in 2025. Total amount agrees on 30; number of entries agrees on 28. Ebenezer Midian: 15 July 2025 in Extract A, 8 August 2025 in the ledger"],
    '')
# appendix B exact rows
i = s.index('    ["0001044300000068", "Nyamunyamu Processors Limited"); j = s.index(']', [0.17, 0.27, 0.15, 0.13, 0.15, 0.13], keep=True)')
s = s[i:] + '    ["0001044300000068", "Nyamunyamu Processors Limited", "131,899,213.80", "69,330,809.54", "62,568,404.26", "69,330,809.54"],
    ["0001044300000002", "Edge View Academy Limited", "117,296,781.00", "10,457,152.33", "106,839,628.67", "10,457,152.33"],
    ["0001044300000073", "Saile Financial Services Limited Company", "90,522,529.35", "2,395,117.82", "88,127,411.53", "2,395,117.82"],
    ["0001044200000062", "Happie Foods Limited", "38,421,398.75", "9,018,700.44", "36,402,698.31", "2,018,700.44"],
    ["0001044200000052", "Kambewu Organic Fertilizer Investments", "42,329,779.82", "913,541.02", "41,416,238.80", "913,541.02"],
    ["0001044200000032", "African Honey Products Industries", "29,893,579.18", "51,362.16", "29,842,217.02", "51,362.16"],
    '    + s[j:]
rep('NOTE("The Extract B and Extract C interest columns are rounded from the files to the nearest unit shown; the differences and "
    "the \\\"other\\\" amounts are exact. On Happie Foods the \\\"other\\\" rows include a further 7,000,000 that is not an interest adjustment."', '',
    'NOTE("The six differences add up to MKW 85,166,683.31, which is the whole difference between the two extracts for 2025. On "
    "Happie Foods the \\\"other\\\" rows include a further 7,000,000 that Extract C does not subtract."')
rep('NOTE("Sanctioned amounts are shown as rounded in our working of the file.")\n'', '')
open(p, "w", encoding="utf-8").write(s)
# run-order file: correct the DD_16c reason
q = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\RUN ORDER - data dictionary queries by priority.sql"
t = open(q, encoding="utf-8", newline="").read()
old = "-- WHY: Sample loan: the disbursement schedule table. Extract A showed one tranche for\r\n--      a loan the ledger shows drawn in five."
assert t.count(old) == 1, t.count(old)
t = t.replace(old, "-- WHY: Sample loan: the disbursement schedule table. Extract A lists five entries\r\n--      dated 15 July 2025; the ledger shows the same amounts paid out on 8 August 2025.")
open(q, "w", encoding="utf-8", newline="").write(t)
print("patched")
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python patch_report.py && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maic_report.py && ./songenv/Scripts/python -c "
from pypdf import PdfReader
r = PdfReader(r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf')
print('pages', len(r.pages))
for i, p in enumerate(r.pages, 1):
    t = p.extract_text()
```

```

    print(i, len(t), '|', ' '.join(t.split())[:90])
}
{
  "timeout": 300000
}

```

Result of Bash [error]

Exit code 1
 Traceback (most recent call last):
 File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\patch_report.py", line 51, in <module>
 assert t.count(old) == 1, t.count(old)
 ^^^^^^^^^^^^^^^^^^^
 AssertionError: 0

CLAUDE (2026-10-05 13:49:24 UTC)

Building the PDF report now. While checking figures I found one of my earlier statements was wrong (the disbursement tranches), so I'm correcting it in the report and in the run-order file.

Tool call: Grep

```

path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\RUN ORDER
- data dictionary queries by priority.sql
pattern: disbursement schedule table
{
  "output_mode": "content",
  "-A": 2
}

```

Result of Grep

```

197:-- WHY: Sample loan: the disbursement schedule table. Extract A showed one tranche
198:--           for a loan the ledger shows drawn in five.
199:SELECT * FROM loan_disbursement_schedule WHERE new_ac_number = '000104430000084'

```

Tool call: Edit

```

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\RUN ORDER
ORDER - data dictionary queries by priority.sql
--- old_string ---
-- WHY: Sample loan: the disbursement schedule table. Extract A showed one tranche
--           for a loan the ledger shows drawn in five.
--- new_string ---
-- WHY: Sample loan: the disbursement schedule table. Extract A lists five entries
--           dated 15 July 2025; the ledger shows the same amounts paid out on 8 August 2025.
{
  "replace_all": false
}

```

Result of Edit [error]

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

Tool call: Bash

```

description: Build the PDF, list its pages and render sample pages for a visual check
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && grep -c "Corrected\|Ifracon\|131,899,213.80\|planned dates" maiic_report.py && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maiic_report.py && ./songenv/Scripts/python -c "
from pypdf import PdfReader
r = PdfReader(r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf')
print('pages', len(r.pages))
for i, p in enumerate(r.pages, 1):
    t = p.extract_text()
    print(i, len(t), '|', ' '.join(t.split())[:80])
" && ./songenv/Scripts/python -m pip install -q pypdfium2 2>&1 | tail -1; ./songenv/Scripts/python -c "
import pypdfium2 as pdfium
pdf = pdfium.PdfDocument(r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf')
for i in (0, 2, 3, 5):
    if i < len(pdf):
        pdf[i].render(scale=1.15).to_pil().save(f'rp{i+1}.png')
print('rendered')
" 2>&1 | tail -2
{
  "timeout": 300000
}

```

Result of Bash

4
written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf
pages 12
1 3090 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
2 2524 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
3 3601 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
4 2739 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
5 2984 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
6 3119 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
7 2881 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
8 3507 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
9 2852 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
10 2930 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
11 2505 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
12 890 | MAIIC E-Banker extract scripts: analysis Dupleix Institute 5 October 2026 Pa
rendered
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

Tool call: Read

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\RUN
ORDER - data dictionary queries by priority.sql
offset: 194
limit: 8

Result of Read

194
195
196 -- RUN 11 of 24 | save as DD_16c_sample_disbursement_schedule.csv
197 -- WHY: Sample loan: the disbursement schedule table. Extract A showed one tranche
198 -- for a loan the ledger shows drawn in five.
199 SELECT * FROM loan_disbursment_schedule WHERE new_ac_number = '000104430000084'
200 ORDER BY disb_schedule_date;
201

Tool call: Read

file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\rp1.png

Result of Read

[image output: not reproduced in this transcript]

Tool call: Read

file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\rp3.png

Result of Read

[image output: not reproduced in this transcript]

Tool call: Edit

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\RUN
ORDER - data dictionary queries by priority.sql
--- old_string ---
-- WHY: Sample loan: the disbursement schedule table. Extract A showed one tranche
-- for a loan the ledger shows drawn in five.
--- new_string ---
-- WHY: Sample loan: the disbursement schedule table. Extract A lists five entries
-- dated 15 July 2025; the ledger shows the same amounts paid out on 8 August 2025.
{
"replace_all": false
}

Tool call: Bash

description: Keep headings with their text, rebuild, and make a contact sheet of all pages

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && sed -i 's/textColor=NAVY, spaceBefore=14, spaceAfter=6),/textColor=NAVY, spaceBefore=14, spaceAfter=6, keepWithNext=1),/; s/textColor=ORANGE, spaceBefore=10, spaceAfter=4),/textColor=ORANGE, spaceBefore=10, spaceAfter=4, keepWithNext=1),/' maiic_report.py && grep -c "keepWithNext=1" maiic_report.py && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maiic_report.py && ./songenv/Scripts/python -c "import pypdfium2 as pdfium
pdf = pdfium.PdfDocument(r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf')
print('pages', len(pdf))
from PIL import Image
ims = [pdf[i].render(scale=0.62).to_pil() for i in range(len(pdf))]
w, h = ims[0].size
sheet = Image.new('RGB', (w * 4, h * 3), 'white')
for i, im in enumerate(ims[:12]): sheet.paste(im, ((i % 4) * w, (i // 4) * h))
sheet.save('contact.png'); print(sheet.size)
pdf[5].render(scale=1.1).to_pil().save('rp6.png')
"
{
  "timeout": 300000
}
```

Result of Edit

The file C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\RUN ORDER - data dictionary queries by priority.sql has been updated successfully.

Result of Bash

```
2
written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf
pages 14
(1480, 1566)
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Read

file_path: C:\Users\wadza\AppData\Local\Temp\claude/c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\contact.png

Result of Read

[image output: not reproduced in this transcript]

Tool call: Bash

description: Fix page breaks around long tables and rebuild

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && cat > patch2.py <<'PYEOF'
p = "maiic_report.py"; s = open(p, encoding="utf-8").read()
def rep(old, new, n=1):
    global s
    assert s.count(old) == n, (old[:50], s.count(old)); s = s.replace(old, new)
rep("PageBreak, KeepTogether, Image, ListFlowable, ListItem", "PageBreak, KeepTogether, Image, ListFlowable, ListItem, CondPageBreak")
rep('def H2(t): story.append(Paragraph(t, S["h2"]))',
    'def H2(t): story.append(Paragraph(t, S["h2"]))\n\n\ndef H2T(t):\n    """Heading that sits directly above a long table: break first if little room is left."""\n    story.append(CondPageBreak(62 * mm)); story.append(Paragraph(t, S["h2t"]))')
rep('    "p": ParagraphStyle("p",', '    "h2t": ParagraphStyle("h2t", fontName="Seg-B", fontSize=11.5, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4),\n    "p": ParagraphStyle("p",')
rep('H2("Where it falls short")', 'H2T("Where it falls short")', 3)
rep('H2("Tier 1: answers the open questions on Extracts A, B and C")', 'H2T("Tier 1: answers the open questions on Extracts A, B and C")')
rep('H1("8. What the scripts settle, and what is still open")\n', 'H1("8. What the scripts settle, and what is still open")\n\n("The questions below are the ones we raised in September, with what the scripts now tell us about each and the query "\n"that would close it. Run numbers refer to section 10.")\n')
rep('H1("7. The three findings, and what to do about each")\n', 'H1("7. The three findings, and what to do about each")\n\n("These are the three points from the summary, with the action each one calls for.")\n')
open(p, "w", encoding="utf-8").write(s); print("ok")
PYEOF
./songenv/Scripts/python patch2.py && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maiic_report.py && ./songenv/Scripts/python -c "import pypdfium2 as pdfium
from PIL import Image
pdf = pdfium.PdfDocument(r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf')
n = len(pdf); print('pages', n)
ims = [pdf[i].render(scale=0.5).to_pil() for i in range(n)]
w, h = ims[0].size; cols = 5; rows = (n + cols - 1) // cols
sheet = Image.new('RGB', (w * cols, h * rows), 'white')
for i, im in enumerate(ims): sheet.paste(im, ((i % cols) * w, (i // cols) * h))
sheet.save('contact2.png'); print(sheet.size)
"
{
  "timeout": 300000
}
```

Result of Bash

```
ok
written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf
pages 13
(1490, 1263)
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

CLAUDE (2026-10-05 13:51:05 UTC)

The PDF is built at 13 pages; doing a last visual check of the page layout before handing it over.

Tool call: Read

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\contact2.png
```

Result of Read

[image output: not reproduced in this transcript]

CLAUDE (2026-10-05 13:51:25 UTC)

Written for: MAIIC Finance and ICT (Dr Thom, Barry) and the Duplex team, in plain language.

The report is saved as a 13-page PDF in the Query Requests to MAIIC subfolder of the raw queries folder. I checked the page layout visually; a few pages have white space where a long table starts on the next page.

One correction to what I told you earlier. I said Extract A shows one disbursement tranche per loan and that Ebenezer Midian shows one against five in the ledger. That was wrong, and so is the same statement in our 24 September research report. Checked against the file:

- 58 of the 314 populated rows list two to ten tranches.
- Ebenezer Midian shows all five amounts in Extract A, dated 15 July 2025.
- The ledger shows the same five amounts paid out on 8 August 2025.

So the tranche list comes from the disbursement schedule with planned dates, and the actual payout dates are in the ledger. The report states the corrected finding, and I fixed the matching note in the run-order SQL file.

The report covers:

- **Summary:** the three findings that affect figures already in use, and what we are asking for.
- **What we received:** the three scripts and the thirteen E-Banker tables they read, in plain language.
- **Each extract in turn:** what the script does, what is good, and where it falls short, with the evidence from the delivered files.
- **What is settled and what is still open:** each September question, with the query that closes it.
- **Why the data dictionary queries come first,** and why the question queries are still needed after them.
- **The queries in run order,** practical notes for whoever runs them, and the amended extracts that follow.
- **Appendices:** the tests behind every figure, the six accounts where Extracts B and C disagree, ten active accounts with no interest posted, and a glossary.

Two things in it you may want to look at before it goes to MAIIC:

- **Ten active accounts with no interest posted (Appendix C):** it names the accounts, including Pinnacle (MWK 1bn) and NASCOMEX (MWK 2.5bn), and asks MAIIC to explain each.
- **Our own import:** the report says we have not yet confirmed whether our engine's import of Extract C selected a single run. That is still unchecked.

Files are in 2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC:

- MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf
- RUN ORDER - data dictionary queries by priority.sql

USER (2026-10-05 14:45:06 UTC)

Can you also save in the subfolder a pdf with this claude session for the raw queries in full so that codex and copilot can review. also save the build files in the sub queries

Tool call: Bash

description: Locate this session's transcript file and inspect its structure

```
--- command ---
cd "/c/Users/wadza/.claude/projects/c--xampp-htdocs-Stress-Testing-App" && ls -la *.jsonl 2>/dev/null | sort -k6,8 | tail -5;
ls -la ee984144-7b28-4df1-9a6e-560f112869e9* 2>/dev/null | head; cd "/c/Users/wadza/AppData/Local/Temp/claude/
c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && ./songenv/Scripts/python - <<'PYEOF'
import json, collections
p = r"C:\Users\wadza\.claude\projects\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9.jsonl"
rows = [json.loads(l) for l in open(p, encoding="utf-8") if l.strip()]
print("lines", len(rows), "| types", collections.Counter(r.get("type") for r in rows).most_common(8))
start = None
for i, r in enumerate(rows):
    if r.get("type") == "user":
        c = r.get("message", {}).get("content")
        txt = c if isinstance(c, str) else " ".join(b.get("text", "") for b in c if isinstance(b, dict) and b.get("type") ==
"text")
        if "Raw Query Scripts" in txt and "extract a. b and c" in txt:
            start = i; print("start line", i, "| ts", r.get("timestamp"), "|", txt[:120].replace("\n", " "))
            break
kinds = collections.Counter()
for r in rows[start:]:
    c = r.get("message", {}).get("content")
    if isinstance(c, list):
        for b in c: kinds[(r.get("type"), b.get("type"))] += 1
    elif isinstance(c, str): kinds[(r.get("type"), "str")] += 1
print(kinds)
print("keys of a row:", list(rows[start].keys()))
print("isSidechain/isMeta flags:", collections.Counter((r.get("isSidechain"), r.get("isMeta")) for r in rows[start:]))
PYEOF
```

Result of Bash

```
-rw-r--r-- 1 wadza 197609 571632 Oct 5 11:53 bd9f499a-dd1f-4d01-94ef-cd75825e28a8.jsonl
-rw-r--r-- 1 wadza 197609 2006517 Oct 5 12:10 e702d200-6694-434c-b9f3-22b716e7838d.jsonl
-rw-r--r-- 1 wadza 197609 1352575 Oct 5 12:33 be48ad55-9c52-4f47-96a8-21335e84d124.jsonl
-rw-r--r-- 1 wadza 197609 8804660 Oct 5 16:45 ee984144-7b28-4df1-9a6e-560f112869e9.jsonl
-rw-r--r-- 1 wadza 197609 36437498 Sep 29 14:29 2c3ae602-da4f-4ac9-9369-e652f1c50258.jsonl
-rw-r--r-- 1 wadza 197609 8804660 Oct 5 16:45 ee984144-7b28-4df1-9a6e-560f112869e9.jsonl

ee984144-7b28-4df1-9a6e-560f112869e9:
total 8
drwxr-xr-x 1 wadza 197609 0 Oct 5 09:40 .
drwxr-xr-x 1 wadza 197609 0 Oct 5 12:03 ..
drwxr-xr-x 1 wadza 197609 0 Oct 5 11:55 tool-results
lines 703 | types [('assistant', 247), ('user', 145), ('atis-latch', 59), ('last-prompt', 58), ('ai-title', 58),
('queue-operation', 56), ('attachment', 50), ('file-history-snapshot', 20)]
start line 483 | ts 2026-10-05T13:02:59.476Z | "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query
Scripts" These are the scripts providi
Counter({'assistant', 'tool_use'): 36, ('user', 'tool_result'): 35, ('assistant', 'thinking'): 30, ('assistant', 'text'): 11,
('user', 'text'): 10})
keys of a row: ['parentUuid', 'isSidechain', 'promptId', 'type', 'message', 'uuid', 'timestamp', 'permissionMode', 'origin',
'promptSource', 'turnOrigin', 'turnPosition', 'userType', 'entrypoint', 'cwd', 'sessionId', 'version', 'gitBranch']
isSidechain/isMeta flags: Counter(({False, None): 134, (None, None): 85, (False, True): 1})
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\build_transcript.py
--- content ---
"""Export the MAIIC raw-query part of the Claude Code session to PDF and Markdown, for review by other tools."""
import json, re, os, html
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Preformatted, Table, TableStyle

SESSION = r"C:\Users\wadza\claude\projects\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9.jsonl"
OUTDIR = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC"
NAME = "Claude_Session_Transcript_Raw_Queries_2026-10-05"
START_MARK = "These are the scripts providing extract a. b and c"
END_MARK = "save in the subfolder a pdf with this claude session"

rows = [json.loads(l) for l in open(SESSION, encoding="utf-8") if l.strip()]

def user_text(r):
    c = r.get("message", {}).get("content")
    if isinstance(c, str):
        return c
    return "\n".join(b.get("text", "") for b in c if isinstance(b, dict) and b.get("type") == "text")

start = next(i for i, r in enumerate(rows) if r.get("type") == "user" and START_MARK in user_text(r))
end = next((i for i, r in enumerate(rows) if i > start and r.get("type") == "user" and END_MARK in user_text(r)), len(rows))

def clean_user(t):
    t = re.sub(r"<system-reminder>.*?</system-reminder>", "", t, flags=re.S)
    t = re.sub(r"<ide_opened_file>.*?</ide_opened_file>", "", t, flags=re.S)
    t = re.sub(r"<ide_selection>.*?</ide_selection>", "", t, flags=re.S)
    return t.strip()

def result_text(content):
    if isinstance(content, str):
        return content
    out = []
    for b in content or []:
        if b.get("type") == "text":
            out.append(b.get("text", ""))
        elif b.get("type") == "image":
            out.append("image output: not reproduced in this transcript")
    return "\n".join(out)

def expand_persisted(t):
    """Where the harness stored a long output in a side file, inline the whole file."""
    m = re.search(r"Full output saved to: (\\S.*\\.txt)", t)
    if m and os.path.exists(m.group(1)):
        return "[long output, reproduced in full from the saved file]\n" + open(m.group(1), encoding="utf-8",
errors="replace").read()
    return t

# ----- collect events
events = [] # (kind, title, body) kind: user | assistant | tool | result | note
tool_names = {}
for r in rows[start:end]:
    typ = r.get("type")
    if typ not in ("user", "assistant") or r.get("isMeta"):
        continue
    ts = (r.get("timestamp") or "")[:19].replace("T", " ")
    c = r.get("message", {}).get("content")
    blocks = [{"type": "text", "text": c} if isinstance(c, str) else (c or [])]
    for b in blocks:
        bt = b.get("type")
        if typ == "user" and bt == "text":
            raw = b.get("text", "")
            if "<task-notification>" in raw or "SYSTEM NOTIFICATION" in raw:
                events.append(("note", f"System notification ({ts} UTC)", re.sub(r"<[^>+>", " ", raw).strip()))
                continue
            t = clean_user(raw)
            if t:
                events.append(("user", f"USER ({ts} UTC)", t))
        elif typ == "user" and bt == "tool_result":
            name = tool_names.get(b.get("tool_use_id"), "tool")
            body = expand_persisted(result_text(b.get("content")))
            body = re.sub(r"<system-reminder>.*?</system-reminder>", "", body, flags=re.S).rstrip()
            events.append(("result", f"Result of {name}" + (" [error]" if b.get("is_error") else ""), body or "(no output)"))
        elif typ == "assistant" and bt == "text":
            if b.get("text", "").strip():
                events.append(("assistant", f"CLAUDE ({ts} UTC)", b["text"].strip()))
```

```

elif typ == "assistant" and bt == "tool_use":
    tool_names[b.get("id")] = b.get("name")
    inp = b.get("input", {})
    parts = []
    for k in ("description", "file_path", "path", "pattern", "skill", "offset", "limit"):
        if k in inp:
            parts.append(f"{k}: {inp[k]}")
    for k in ("command", "content", "old_string", "new_string"):
        if k in inp:
            parts.append(f"--- {k} ---\n{inp[k]}")
    other = {k: v for k, v in inp.items() if k not in ("description", "file_path", "path", "pattern", "skill", "offset",
"limit", "command", "content", "old_string", "new_string")}
    if other:
        parts.append(json.dumps(other, indent=1)[:4000])
    events.append(("tool", f"Tool call: {b.get('name')}", "\n".join(parts)))

```

HEADER = f""""This is the record of the part of a Claude Code session (5 October 2026) that analysed the three raw query scripts MAIIC sent for Extracts A, B and C. It is saved so that other reviewers (Codex, Copilot) can check the work.

It starts at the user's request to analyse the scripts and ends with the delivery of the PDF analysis report. Earlier, unrelated parts of the same session are left out.

What is included: every user message, every reply shown to the user, every tool call with its full input (commands, scripts and file contents), and every tool result in full.

What is left out: the assistant's internal reasoning, automatic system reminders (they carry unrelated workspace context), the text of the PDF skill, and image outputs (page renders used for visual checks).

What to review:

1. The reading of the three scripts (FACILITY_REGISTER, CASHFLOW_REGISTER, INTEREST_RECON), reproduced in full in the tool results below.
2. The evidence tests run on Extracts A, B and C (the maic_check scripts and their outputs).
3. The 24 data dictionary queries. They were written without access to the Oracle database and have NOT been run.
4. The statements in the PDF report MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf.

Known correction made during the session: the first analysis said Extract A shows one disbursement tranche per loan. A later test showed that was wrong (58 of 314 populated rows list two to ten entries); the report and the run-order SQL file carry the corrected finding. The earlier wrong statement is left in this transcript where it was made.

Times are UTC. Source: the session log {os.path.basename(SESSION)}, entries {start} to {end - 1}. Events: {len(events)}."""

```

# ----- markdown
md = [f"# Claude session transcript: MAIIC raw query scripts\n", HEADER, "\n---\n"]
for kind, title, body in events:
    if kind in ("user", "assistant"):
        md.append(f"\n## {title}\n\n{body}\n")
    else:
        fence = "```\n"
        md.append(f"\n**{title}**\n\n{fence}\n{body}\n{fence}\n")
open(os.path.join(OUTDIR, NAME + ".md"), "w", encoding="utf-8").write("\n".join(md))

# ----- pdf
F = r"C:\Windows\Fonts"
pdfmetrics.registerFont(TTFont("Seg", F + r"\segoeui.ttf")); pdfmetrics.registerFont(TTFont("Seg-B", F + r"\segoeuib.ttf"))
pdfmetrics.registerFont(TTFont("Seg-I", F + r"\segoeuii.ttf")); pdfmetrics.registerFont(TTFont("Seg-BI", F + r"\segoeuiiz.ttf"))
pdfmetrics.registerFontFamily("Seg", normal="Seg", bold="Seg-B", italic="Seg-I", boldItalic="Seg-BI")
pdfmetrics.registerFont(TTFont("Mono", F + r"\consola.ttf"))
NAVY = colors.HexColor("#1B2A41"); ORANGE = colors.HexColor("#E07B00"); GREY = colors.HexColor("#5B6470"); LINE = colors.HexColor("#C9D1DB")
st_title = ParagraphStyle("t", fontName="Seg-B", fontSize=20, leading=25, textColor=NAVY, spaceAfter=8)
st_p = ParagraphStyle("p", fontName="Seg", fontSize=9.5, leading=13.5, spaceAfter=5)
st_user = ParagraphStyle("u", fontName="Seg-B", fontSize=10.5, leading=14, textColor=colors.white, backColor=NAVY, borderPadding=(3, 5, 3, 5), spaceBefore=12, spaceAfter=7, keepWithNext=1)
st_asst = ParagraphStyle("a", fontName="Seg-B", fontSize=10.5, leading=14, textColor=colors.white, backColor=ORANGE, borderPadding=(3, 5, 3, 5), spaceBefore=12, spaceAfter=7, keepWithNext=1)
st_tool = ParagraphStyle("t1", fontName="Seg-B", fontSize=8.5, leading=11, textColor=GREY, spaceBefore=7, spaceAfter=2, keepWithNext=1)
st_mono = ParagraphStyle("m", fontName="Mono", fontSize=6.6, leading=8.3, textColor=colors.HexColor("#202020"))
st_h = ParagraphStyle("h", fontName="Seg-B", fontSize=10.5, leading=14, textColor=NAVY, spaceBefore=6, spaceAfter=3)
BAD = re.compile(r"[\x00-\x08\x0b\x0c\x0e-\x1f\u00ff\u00ff]")

def esc(t):
    return html.escape(BAD.sub("", t), quote=False)

def inline(t):
    t = esc(t)
    t = re.sub(r"\".*?(.+)\".*\"", r"<b>\1</b>", t)
    t = re.sub(r"``([\^]+)``", r'<font name="Mono" size="8.5">\1</font>', t)
    return t

def mono(story, text, cols=128):
    lines = BAD.sub("", text.replace("\t", " ").replace("\r", "")).split("\n")
    for i in range(0, len(lines), 70):
        story.append(Preformatted("\n".join(lines[i:i + 70]), st_mono, maxLineLength=cols, splitChars=" ;|/\\", newLineChars=""))

def prose(story, text):

```

```

buf = [] # consecutive table / code lines go out as mono

def flush():
    if buf:
        mono(story, "\n".join(buf)); story.append(Spacer(1, 4)); buf.clear()
in_code = False
for line in text.split("\n"):
    s = line.rstrip()
    if s.strip().startswith("`"):
        in_code = not in_code; continue
    if in_code or s.lstrip().startswith("|"):
        buf.append(s); continue
    flush()
    if not s.strip():
        continue
    m = re.match(r"^(#{1,4})\s+(.*)", s)
    if m:
        story.append(Paragraph(inline(m.group(2)), st_h)); continue
    m = re.match(r"^(s*)([-*]|\d+\.)\s+(.*)", s)
    if m:
        ind = 10 + 10 * (len(m.group(1)) // 2)
        mark = "*" if m.group(2) in "-*" else m.group(2)
        story.append(Paragraph(f"{mark} {inline(m.group(3))}", ParagraphStyle("b", parent=st_p, leftIndent=ind,
firstLineIndent=-9, spaceAfter=2)))
        continue
    story.append(Paragraph(inline(s), st_p))
flush()

story = [Paragraph("Claude session transcript: MAIIC raw query scripts", st_title)]
for para in HEADER.split("\n\n"):
    for i, ln in enumerate(para.split("\n")):
        story.append(Paragraph(inline(ln), st_p))
story.append(Spacer(1, 6))
for kind, title, body in events:
    if kind == "user":
        story.append(Paragraph(esc(title), st_user)); prose(story, body)
    elif kind == "assistant":
        story.append(Paragraph(esc(title), st_asst)); prose(story, body)
    else:
        story.append(Paragraph(esc(title), st_tool)); mono(story, body); story.append(Spacer(1, 3))

def footer(canvas, doc):
    canvas.saveState(); canvas.setStrokeColor(LINE); canvas.setLineWidth(0.5)
    canvas.line(14 * mm, 12 * mm, A4[0] - 14 * mm, 12 * mm)
    canvas.setFont("Seg", 7.5); canvas.setFillColor(GREY)
    canvas.drawString(14 * mm, 8 * mm, "Claude session transcript: MAIIC raw query scripts · 5 October 2026 · for review")
    canvas.drawRightString(A4[0] - 14 * mm, 8 * mm, f"Page {doc.page}"); canvas.restoreState()

out = os.path.join(OUTDIR, NAME + ".pdf")
doc = BaseDocTemplate(out, pagesize=A4, leftMargin=14 * mm, rightMargin=14 * mm, topMargin=13 * mm, bottomMargin=17 * mm,
    title="Claude session transcript: MAIIC raw query scripts", author="Duplex Institute (Claude Code
session)")
doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, id="f",
    leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)],
onPage=footer)])
doc.build(story)
kinds = {}
for k, _, _ in events:
    kinds[k] = kinds.get(k, 0) + 1
print("events", kinds, "| chars", sum(len(b) for _, _, b in events))
print("first:", events[0][1], "| last:", events[-1][1], "|", events[-1][2][:80].replace("\n", " "))
print("written", out)

```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\build_transcript.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Build the session transcript as PDF and Markdown, and check it for completeness and stray private content

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python build_transcript.py 2>&1 | tail -8 && ./songenv/Scripts/python -c "
from pypdf import PdfReader
import pypdfium2 as pdfium
p = r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf'
r = PdfReader(p); print('pages', len(r.pages))
t = ' '.join((r.pages[i].extract_text() or '') for i in range(len(r.pages)))
for needle in ['system-reminder', 'wadzanaïrombe', 'ZNBS', 'PROC_BUILD_MAIIC_FACILITY_REGISTER', 'DD_16i', '13-page PDF']:
    print(needle, '->', t.count(needle))
pdf = pdfium.PdfDocument(p)
pdf[0].render(scale=1.0).to_pil().save('tp1.png'); pdf[min(40, len(pdf)-1)].render(scale=1.0).to_pil().save('tp2.png')
"; ls -la "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/" |
awk '{print $5, $9, $10, $11, $12, $13, $14, $15}'
{
  "timeout": 600000
}
```

Result of Bash

```
events {'user': 8, 'tool': 35, 'result': 35, 'assistant': 11} | chars 249843
first: USER (2026-10-05 13:02:59 UTC) | last: CLAUDE (2026-10-05 13:51:25 UTC) | Written for: MAIIC Finance and ICT (Dr Thom, Barry) and the Dupleix team, in pla
written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf
pages 54
system-reminder -> 0
wadzanaïrombe -> 0
ZNBS -> 0
PROC_BUILD_MAIIC_FACILITY_REGISTER -> 6
DD_16i -> 4
13-page PDF -> 1

0 .
0 ..
258441 Claude_Session_Transcript_Raw_Queries_2026-10-05.md
242350 Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf
16173 MAIIC_EBanker_Data_Dictionary_Queries.sql
103112 MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf
17620 RUN ORDER - data dictionary queries by
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Read

file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\tp1.png

Result of Read

[image output: not reproduced in this transcript]

Tool call: Read

file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\tp2.png

Result of Read

[image output: not reproduced in this transcript]

Tool call: Bash

description: Copy the build scripts into a Build files subfolder with a README

```
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp\htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && DST="/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Build files" && mkdir -p "$DST" && cp maiic_check.py maiic_check2.py maiic_check3.py maiic_check4.py maiic_exact.py maiic_tr.py runorder.py maiic_report.py patch_report.py patch2.py build_transcript.py "$DST/" && cat > "$DST/README.md" <<'EOF'
# Build files: MAIIC raw query scripts analysis
```

Prepared 5 October 2026 by Dupleix Institute, in a Claude Code session. These are the Python scripts behind the files in the folder above, kept so the work can be reviewed and re-run.

What each file does

File	Purpose	Output
`maiic_check.py`	Prints the layout of the delivered Extract A, B and C workbooks	Screen only
`maiic_check2.py`	First evidence tests: rate label by product, snapshot comparison, disbursed against sanctioned, Extract B interest against Extract C, accounts missing from C	Screen only
`maiic_check3.py`	Finds the three runs stacked in the Extract C file	Screen only
`maiic_check4.py`	Exact evidence figures for the report: snapshot comparison, status counts, ledger sample, the ten active accounts with no interest	Screen only
`maiic_exact.py`	Exact figures for the report's appendix tables (the six accounts where B and C differ, run totals)	Screen only
`maiic_tr.py`	Re-test of the disbursement tranche claim, Extract A against the ledger rows in Extract B	Screen only
`runorder.py`	Builds the run-order SQL file from `MAIIC_EBanker_Data_Dictionary_Queries.sql`	`RUN ORDER - data dictionary queries by priority.sql`
`maiic_report.py`	Builds the plain-language analysis report	`MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf`
`build_transcript.py`	Exports the relevant part of the Claude session log	`Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf` and `.md`
`patch_report.py`, `patch2.py`	One-off edits that were applied to `maiic_report.py` during the session	Already applied

Things a reviewer should know

- **Nothing here has touched the Oracle database.** The evidence tests read only the Excel extracts MAIIC delivered in August 2026. The 24 SQL queries in the folder above were written without database access and have not been run.
- **`maiic\_report.py` is the final version.** The two patch scripts have already been applied to it. Running them again will stop on their own checks and change nothing. Note that `runorder.py` regenerates the run-order SQL file from its own text, which would undo the one line corrected by hand afterwards (the reason given for run 11, DD_16c).
- **`maiic\_check3.py` stops with an error near the end** (a grouping line that fails on this version of pandas) after printing its main results. In the session the remaining checks were run with that line replaced; the results are in the transcript.
- **`maiic\_check2.py` contains a snapshot comparison that over-reports differences** on columns with mixed date formats.
- **`maiic\_check4.py` repeats the comparison properly and is the one the report relies on.**
- **Paths are fixed to the PC the work was done on** (the OneDrive MAIIC project folder and the Windows fonts folder). They need changing to run anywhere else.
- **`build\_transcript.py` reads the Claude Code session log** from the user profile on that PC, so it can only be re-run there.

To re-run

Python 3 with `pandas`, `openpyxl`, `reportlab` and `pypdf` (`pypdfium2` and `pillow` only if page previews are wanted).

1. `python maiic\_check4.py`, `python maiic\_exact.py` and `python maiic\_tr.py` reproduce the figures quoted in the report.
2. `python maiic\_report.py` rebuilds the PDF report.

EOF

```
ls -la "$DST" | awk '{print $5, $9}'; grep -n "DD_16c" -A2 "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Build files/runorder.py" | head -4
```

Result of Bash

```
0 .
0 ..
3332 README.md
12155 build_transcript.py
474 maiic_check.py
5050 maiic_check2.py
3154 maiic_check3.py
3667 maiic_check4.py
3019 maiic_exact.py
45284 maiic_report.py
2086 maiic_tr.py
1590 patch2.py
5497 patch_report.py
7009 runorder.py
40: ("DD_16c", "Sample loan: the disbursement schedule table. Extract A showed one tranche for a loan the ledger shows drawn in five.", ""),
41: ("DD_16a", "Sample loan: the account master row, every column. Shows what else is held per account (rate, status, any policy or margin fields).", ""),
42: ("DD_16b", "Sample loan: the loan master row, every column. Moratorium fields, sanction details and any link to the origination system.", ""),
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

description: Carry the tranche correction into the saved build script and README

```
--- command ---
DST="/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Build files" && cd "$DST" && sed -i 's/Sample loan: the disbursement schedule table. Extract A showed one tranche for a loan the ledger shows drawn in five./Sample loan: the disbursement schedule table. Extract A lists five entries dated 15 July 2025; the ledger shows the same amounts paid out on 8 August 2025./' runorder.py && grep -c "Extract A lists five entries dated 15 July 2025" runorder.py && sed -i 's/ Note that `runorder.py` regenerates the run-order SQL file from its own text, which would undo the one line corrected by hand afterwards (the reason given for run 11, DD_16c)./ `runorder.py` here already carries the corrected reason for run 11 (DD_16c), which was first fixed by hand in the SQL file./' README.md && grep -c "already carries the corrected reason" README.md && grep -n "RUN 11" -A3 "../RUN ORDER - data dictionary queries by priority.sql" | head -5
```

Result of Bash

```
1
1
196:-- RUN 11 of 24 | save as DD_16c_sample_disbursement_schedule.csv
197:-- WHY: Sample loan: the disbursement schedule table. Extract A lists five entries
198:-- dated 15 July 2025; the ledger shows the same amounts paid out on 8 August 2025.
199-SELECT * FROM loan_disbursement_schedule WHERE new_ac_number = '000104430000084'
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

CLAUDE (2026-10-05 14:48:29 UTC)

Written for: Codex and Copilot as reviewers, plus whoever at Dupleix re-runs the work.

The session transcript and the build files are saved in the Query Requests to MAIIC subfolder.

Transcript (54 pages)

- It covers the session from your request to analyse the three scripts through to delivery of the analysis report.
- It includes every message, every tool call with its full input, and every tool result in full, including the three MAIIC scripts and the outputs of each evidence test.
- I left out my internal reasoning, the automatic system reminders, the text of the PDF skill and the page-preview images. The system reminders carry unrelated workspace context, including another client's name; I checked the PDF and none of that appears in it.
- The opening page tells reviewers what to check, states that the 24 SQL queries have not been run, and flags the tranche correction.
- I also saved a Markdown copy of the same transcript, since review tools usually read plain text more reliably than PDF.

Build files

- The Build files subfolder holds the eleven Python scripts behind the evidence tests, the run-order SQL file, the analysis report and the transcript, with a README.
- The README says what each script does, how to re-run them, and what a reviewer should know: for example, one early test script stops with an error after printing its results, and all paths are fixed to this PC.
- I carried the tranche correction into the saved copy of the run-order builder, so re-running it does not bring the wrong wording back.

Files are in 2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC:

- Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf
- Claude_Session_Transcript_Raw_Queries_2026-10-05.md
- Build files\README.md

USER (2026-10-05 15:26:14 UTC)

/C:/xampp/htdocs/Stress-Testing-App/MAIIC-Claude-Review-2026-10-05.md => this is the review by codex. Can you analyse it

Tool call: Read

file_path: C:\xampp\htdocs\Stress-Testing-App\MAIIC-Claude-Review-2026-10-05.md

Result of Read

```
1  # Codex review of Claude's MAIIC extract analysis
2
3  Date: 5 October 2026
4
5  ## Scope and conclusion
6
7  Reviewed the supplied Claude session transcript, MAIIC's three Oracle extract procedures, the report builder and data
dictionary SQL pack. Re-ran `maiic_check4.py`, `maiic_exact.py` and `maiic_tr.py` against the delivered Excel extracts. Also
inspected the relevant transaction and interest import services in `C:\xampp\htdocs\MAICC-IFRS9`. This is a review of the
supplied analysis session, not a full application audit. No Oracle queries were executed and no application code or database
records were changed.
8
9  Claude's analysis is useful and its main numerical findings reproduce. Revise the claims below before treating the report
as final.
10
11  ## Findings that reproduce
12
13  - Extract A reads current loan terms and status for nominal historical snapshots; only the balance lookup is explicitly
dated.
14  - Extract B removes transaction signs, estimates repayment splits, and includes interest charges that must not
automatically be treated as cash receipts.
15  - Extract C contains overlapping runs plus monthly and annual totals.
16  - The 2025 B-C difference is MWK 85,166,683.31 across six accounts. The other 101 accounts agree exactly.
17  - Ebenezer has five entries in both extracts: 15 July 2025 in the schedule versus 8 August 2025 in the ledger. The
corrected finding is supported.
18  - The report correctly acknowledges that loan-account postings require reconciliation to income GL accounts.
19
20  ## Requested corrections and additional checks
21
22  1. **High: scheduled rows are not the unpaid remainder.** `CASHFLOW_REGISTER.txt`, lines 115-126, selects instalments not
fully recovered but exports the full original principal, interest and instalment amounts. It does not subtract
`inst_recovered`. Revise the report wording and require a verified recovery allocation before using these amounts as remaining
payments.
23
24  2. **High: running-balance correctness is not established.** The report says that building the balance from the whole
history makes it right. This remains conditional on complete ledger history, a valid opening position, correct posting scope
and reconciliation to independent balances. Replace the unconditional claim and request those checks.
25
26  3. **High: absence of interest rows is not automatically a verified zero.** Confirm account scope and extraction
completeness first. Distinguish no postings found, verified zero and unresolved coverage. The ten active accounts need
explanations, as the report already requests.
27
28  4. **Medium: verify final-day date handling.** B and C use inclusive `BETWEEN` end dates; A uses `transaction_date <=
TRUNC(p_as_of_date)`. If source dates contain times, later final-day postings can be excluded. Establish intended date
semantics and test boundary cases before amending the procedures.
29
30  5. **Medium: aggregate agreement does not establish adjustment semantics.** Five of the six B-C differences match the
Other/Adjustment totals. Happie Foods has MWK 9,018,700.44 of Other/Adjustment against a difference of MWK 2,018,700.44,
leaving MWK 7 million outside that explanation. Retain the appendix qualification and qualify the stronger wording in the main
report. Obtain raw signed postings and vendor transaction-code definitions.
31
32  6. **Medium: clarify schema and privilege assumptions in the dictionary pack.** Multiple accessible schemas can contain
ACMASTER, while several catalogue outputs omit owner identifiers. Carry owner identifiers through those outputs and establish
the intended schema. Verify direct SELECT access to business tables; permission to execute a procedure does not by itself
establish that access.
33
34  7. **Qualify the cause of repayment-frequency differences.** Forty-two accounts differ between snapshots, but this does
not establish that the procedure changed between runs. Request procedure versions/run provenance and retain other explanations
as unresolved.
35
36  8. **Define the export contract.** CSV alone does not establish unambiguous dates or protect account numbers when opened
in Excel. Specify date/timestamp formats, account identifiers as text, encoding and numeric conventions. Mixed dates remain a
material evidence limitation.
37
38  ## Application implications
39
40  - `app/Services/Eir/GLInterestImportService.php` already skips annual totals and detects duplicate monthly entries. Its
description still calls Extract C interest income; qualify this pending validation against the income GL and trial balance.
41  - `app/Services/Eir/ContractTransactionImportService.php` stores the delivered estimated principal/interest components.
Trace downstream consumption and ensure estimates are distinguished from verified cash movements before calculation use.
42  - The reproduced workbook has one B run, no duplicate A facility/snapshot keys, no accounts with multiple subaccounts in
A, and no duplicate run-41 C monthly keys at account/subaccount/GL/year/month grain. Nevertheless, reusable evidence checks
should retain the composite facility key rather than assume account number alone is always sufficient.
43
44  ## Evidence locations
45
46  - Project documents: `C:\Users\wadza\OneDrive\2026\Projects\MAIIC`
47  - Source procedures: `2. Documents from clients\Raw Query Scripts\FACILITY_REGISTER.txt`, `CASHFLOW_REGISTER.txt`,
`INTEREST_RECON.txt`
48  - Analysis deliverables: `2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC`
49  - Final report source: `Query Requests to MAIIC\Build files\maiic_report.py`
50  - Application repository: `C:\xampp\htdocs\MAICC-IFRS9`
51
52  ## Suggested response from Claude
53
54  For each comment, state whether it is accepted, qualified or disputed, with supporting source/data evidence. Update the
report and query pack as appropriate. Separate reproduced observations from assumptions and pending vendor confirmations. Do
not describe the work as a completed income reconciliation or a validated contractual cash-flow schedule.
55
```

Tool call: Bash

```
description: Run extra tests on the points the Codex review raises (running balance, transaction direction, adjustments,
repayment frequency)
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-hdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && cat > review_checks.py <<'PYEOF'
import pandas as pd, numpy as np, warnings
warnings.filterwarnings("ignore")
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 30); pd.set_option("display.max_colwidth", 40)
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
A = pd.read_excel(D + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
B = pd.read_excel(D + r"\Extract B.xlsx", sheet_name="Sheet1", dtype={"LOAN_ACCOUNT_NUMBER": str})
A["snap"] = A["AS_OF_DATE"].astype(str).str[:10]
x = A[A.snap == "2025-01-01"].set_index(["LOAN_ACCOUNT_NUMBER", "SUB_ACCOUNT_NO"]); y = A[A.snap ==
"31-12-2025"].set_index(["LOAN_ACCOUNT_NUMBER", "SUB_ACCOUNT_NO"])
act = B[B.SCHEDULED_ACTUAL_FLAG == "Actual"].copy()
act["seq"] = act.GL_POSTING_REF.str.split("-").str[-1].astype(int)
act["pos"] = range(len(act))
key = ["LOAN_ACCOUNT_NUMBER", "SUB_ACCOUNT_NO"]
# (b) direction of each transaction type, from the change in the running balance (file order within account)
act["prev"] = act.groupby(key).BALANCE_AFTER_TRANSACTION.shift(1)
act["delta"] = act.BALANCE_AFTER_TRANSACTION - act.prev
t = act.dropna(subset=["prev"])
up = (t.delta - t.TOTAL_AMOUNT).abs() < 0.02; dn = (t.delta + t.TOTAL_AMOUNT).abs() < 0.02
t = t.assign(dir=np.where(up, "balance up", np.where(dn, "balance down", "neither")))
print("(b) direction by type (rows after the first per account; file order):")
print(pd.crosstab(t.TRANSACTION_TYPE, t.dir).to_string())
print("    is file order = posting-id order within account?", bool((act.groupby(key).seq.diff().dropna() > 0).all()), "| rows
out of id order:", int((act.groupby(key).seq.diff() < 0).sum()))
# first rows: is first balance equal to first amount (i.e. history starts inside the file)?
f = act.groupby(key).head(1)
print("    first row per account: balance equals amount on", int(((f.BALANCE_AFTER_TRANSACTION - f.TOTAL_AMOUNT).abs() <
0.02).sum()), "of", len(f), "accounts (others carry history from before 2025)")
# (a) Extract B closing running balance vs Extract A balance at 31 Dec 2025 (ACCOUNT_BALANCE table)
last = act.groupby(key).tail(1).set_index(key)
j = last[["BALANCE_AFTER_TRANSACTION", "TRANSACTION_DATE"]].join(y[["OPENING_BALANCE", "BALANCE_SNAPSHOT_DATE",
"ACCOUNT_STATUS", "CUSTOMER_NAME"]], how="left")
j["diff"] = j.BALANCE_AFTER_TRANSACTION - j.OPENING_BALANCE
have = j.dropna(subset=["OPENING_BALANCE"])
print("\n(a) B closing running balance vs A balance at 31-Dec-2025: accounts with B actual rows", len(j), "| with an A balance",
len(have), "| agree within 1:", int((have["diff"].abs() < 1).sum()), "| differ:", int((have["diff"].abs() >= 1).sum()), "| no
A balance:", int(j.OPENING_BALANCE.isna().sum()))
print(have[have["diff"].abs() >= 1].sort_values("diff", key=abs, ascending=False)[["CUSTOMER_NAME", "BALANCE_AFTER_TRANSACTION",
"OPENING_BALANCE", "diff", "BALANCE_SNAPSHOT_DATE", "TRANSACTION_DATE"]].head(12).round(2).to_string())
# (c) Happie Foods other/adjustment rows
h = act[(act.LOAN_ACCOUNT_NUMBER == "000104420000062")]
print("\n(c) Happie Foods, Other/Adjustment rows and neighbours:")
print(h[h.TRANSACTION_TYPE.isin(["Other/Adjustment"])[["GL_POSTING_REF", "TRANSACTION_DATE", "TRANSACTION_TYPE",
"TOTAL_AMOUNT", "prev", "BALANCE_AFTER_TRANSACTION", "delta"]].round(2).to_string())
o = t[t.TRANSACTION_TYPE == "Other/Adjustment"]
print("    all Other/Adjustment rows:", len(act[act.TRANSACTION_TYPE == "Other/Adjustment"]), "| total",
f"{act[act.TRANSACTION_TYPE == 'Other/Adjustment'].TOTAL_AMOUNT.sum():.2f}", "| by direction:", o.dir.value_counts().to_dict(),
"| amount by direction:", o.groupby("dir").TOTAL_AMOUNT.sum().round(2).to_dict())
# (d) repayment frequency between snapshots
xa, ya = x.REPAYMENT_FREQUENCY.fillna("(blank)"), y.REPAYMENT_FREQUENCY.reindex(x.index).fillna("(blank)")
print("\n(d) repayment frequency, 1-Jan-2025 snapshot (rows) vs 31-Dec-2025 snapshot (columns):")
print(pd.crosstab(xa, ya).to_string())
d = xa != ya
print("    differing accounts by loan start year text:", x[d].LOAN_START_DATE.astype(str).str.extract(r"(20\d\
d)")[0].value_counts().to_dict(), "| NUM_INSTALMENTS zero among them:", int((x[d].NUM_INSTALMENTS == 0).sum()), "of",
int(d.sum()))
# keys
print("\n(e) keys: A duplicates on snapshot+account+sub:", int(A.duplicated(["snap", "LOAN_ACCOUNT_NUMBER",
"SUB_ACCOUNT_NO"]).sum()), "| sub-account values:", A.SUB_ACCOUNT_NO.unique().tolist())
sch = B[B.SCHEDULED_ACTUAL_FLAG == "Scheduled"]
print("(f) scheduled rows: principal+interest = total on", int(((sch.PRINCIPAL_COMPONENT.fillna(0) +
sch.INTEREST_COMPONENT.fillna(0) - sch.TOTAL_AMOUNT).abs() < 0.02).sum()), "of", len(sch))
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python review_checks.py 2>&1 | cut -c1-300
{
  "timeout": 300000
}
```

Tool call: Bash

```
description: Look at the interest import service in the MAIIC application repository
--- command ---
cd /c/xampp/htdocs/MAIIC-IFRS9 2>/dev/null && git branch --show-current && ls app/Services/Eir/ | head -40 && wc -l app/
Services/Eir/GlInterestImportService.php app/Services/Eir/ContractTransactionImportService.php && grep -n -i "YEAR\|period_type\
|duplicate\|run_id\|RUN_ID\|interest income\|skip" app/Services/Eir/GlInterestImportService.php | head -40
```

Result of Bash

```
(b) direction by type (rows after the first per account; file order):
dir          balance down balance up  neither
TRANSACTION_TYPE
Disbursement          0          63          1
Fee                   1           1          0
Interest              6         987          1
Other/Adjustment      9          18          3
Principal+Interest   485           4         12
  is file order = posting-id order within account? False | rows out of id order: 16
  first row per account: balance equals amount on 33 of 112 accounts (others carry history from before 2025)

(a) B closing running balance vs A balance at 31-Dec-2025: accounts with B actual rows 112 | with an A balance 111 | agree
within 1: 106 | differ: 5 | no A balance: 1

                                CUSTOMER_NAME  BALANCE_AFTER_TRANSACTION  OPENING_BALANCE          diff
BALANCE_SNAPSHOT_DATE  TRANSACTION_DATE
LOAN_ACCOUNT_NUMBER  SUB_ACCOUNT_NO
000104460000071      1          Design Management Consultants Ltd          1.356330e+08          1.172678e+08  18365144.99
    31-12-2025      2025-01-10 00:00:00
000104430000002      1          Edge View Academy Limited          3.544248e+08          3.411044e+08  13320371.33
    31-12-2025          31-01-2025
000104430000074      1          SYCAMORE CREDIT LIMITED          2.359271e+08          2.242864e+08  11640688.46
    31-12-2025      2025-12-12 00:00:00
000104450000077      1          Kwithu Kitchens 2          1.094197e+08          9.848029e+07  10939455.21
    31-12-2025          30-09-2025
000104450000015      1          Zaithwa Farms          1.225683e+06          3.053317e+06  -1827633.62
    31-12-2025          30-08-2025

(c) Happie Foods, Other/Adjustment rows and neighbours:
  GL_POSTING_REF TRANSACTION_DATE  TRANSACTION_TYPE  TOTAL_AMOUNT      prev  BALANCE_AFTER_TRANSACTION      delta
230  1050101-242833      31-10-2025  Other/Adjustment    7000000.00  96258650.59          1.032587e+08  7000000.00
238  1050101-299014      31-12-2025  Other/Adjustment    2018700.44  83723698.23          8.170500e+07 -2018700.44
  all Other/Adjustment rows: 31 | total 832,653,206.42 | by direction: {'balance up': 18, 'balance down': 9, 'neither': 3} |
amount by direction: {'balance down': 193191670.97, 'balance up': 257817556.81, 'neither': 203072549.64}

(d) repayment frequency, 1-Jan-2025 snapshot (rows) vs 31-Dec-2025 snapshot (columns):
REPAYMENT_FREQUENCY (blank) Half-Yearly  Monthly  Quarterly  Yearly
REPAYMENT_FREQUENCY
Monthly          25          10         139          4          3
  differing accounts by loan start year text: {'2024': 21, '2025': 15, '2021': 3, '2023': 1, '2022': 1, '2026': 1} |
NUM_INSTALLMENTS zero among them: 25 of 42

(e) keys: A duplicates on snapshot+account+sub: 0 | sub-account values: [1]
(f) scheduled rows: principal+interest = total on 761 of 775
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Result of Bash

```
eir_revenue_recognition
CalculateEirService.php
ContractMasterImportService.php
ContractTransactionImportService.php
EirCalculationService.php
EirContractInputService.php
EirCoverageService.php
EirGlReconciliationService.php
EirReadinessService.php
EirRevenueService.php
FeeImportService.php
FeeRuleMatcher.php
GlInterestImportService.php
RemainingScheduleImportService.php
ScheduleGeneratorService.php
ScheduleImportService.php
ScheduleWorkflowService.php
StagingClassifier.php
TrialBalanceImportService.php
TrialBalanceMovementService.php
229 app/Services/Eir/GlInterestImportService.php
118 app/Services/Eir/ContractTransactionImportService.php
347 total
10: * actually posted as interest income, per loan, per period.
20: * is not silently overwritten. An identical figure is a duplicate delivery;
34: * duplicate_source_rows:int, conflicting_duplicate_source_rows:int, held:array<string,string>,
35: * skipped:array<string,string>, restatements:array<string,string>,
43: $skipped = [];
46: $duplicates = 0;
47: $conflictingDuplicates = 0;
62: $skipped[$label] = 'no loan account number on the row';
78: [$year, $month] = $this->period($row);
79: if ($year === null || $month === null) {
80:     $skipped[$contractId . ' @ ' . $label] = 'posting period is missing or not a valid year/month';
85: $naturalKey = implode('|', [$contractId, $year, $month, $glAccount ?? '']);
90:     $duplicates++;
94:     $conflictingDuplicates++;
95:     $skipped[$naturalKey] = 'conflicting duplicate: the file carries different amounts or references for the
same loan, period and GL account';
112: $runId = trim((string) ($row['run_id'] ?? '')) ?: null;
116: 'period_type' => strtoupper(trim((string) ($row['period_type'] ?? 'MONTHLY'))): 'MONTHLY',
117: 'period_year' => $year,
119: 'reporting_period' => sprintf('%04d-%02d-01', $year, $month),
133: ->where('period_year', $year)
155: $periodKey = sprintf('%04d-%02d', $year, $month);
167: 'duplicate_source_rows' => $duplicates,
168: 'conflicting_duplicate_source_rows' => $conflictingDuplicates,
171: 'skipped' => $skipped,
180: * Period from explicit year/month columns, falling back to a date column
187: $year = (int) $this->amount($row['period_year'] ?? null);
190: if ($year === 0 && isset($row['reporting_period'])) {
193:     [$year, $month] = array_map('intval', explode('-', substr($date, 0, 7)));
197: if ($year < 1900 || $year > 2999 || $month < 1 || $month > 12) {
201:     return [$year, $month];
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Read

```
file_path: C:\xampp\htdocs\MAICC-IFRS9\app\Services\Eir\GlInterestImportService.php
offset: 1
limit: 160
```

Result of Read

```
1  <?php
2
3  namespace App\Services\Eir;
4
5  use App\Support\ContractId;
6  use Illuminate\Support\Facades\DB;
7
8  /**
9   * Imports the GL interest postings file (Extract C) – what MAIIC's ledger
10   * actually posted as interest income, per loan, per period.
11   *
12   * This is the evidence side of the Phase 6 reconciliation, so the service
13   * records rather than interprets:
14   *
15   * - signs are stored exactly as delivered and the negative/positive profile
16   *   is reported at import. The Extract C sign convention is an open item;
17   *   flipping a sign to "look right" would move a real misstatement into the
18   *   noise and out of the reconciliation;
19   * - a period that already carries a posting for the same loan and GL account
20   *   is not silently overwritten. An identical figure is a duplicate delivery;
21   *   a different figure is a GL restatement, applied and named individually,
22   *   because a restatement changes a reconciliation that may already have been
23   *   reviewed.
24   */
25  class GLInterestImportService
26  {
27      /** Persisted lineage: names the vendor file an auditor will ask for. */
28      public const SOURCE = 'MAIIC_EXTRACT_C';
29
30      /**
31       * @param list<array<string,mixed>> $rows mapped Extract C rows
32       * @return array{
33       *   source_rows:int, loaded_rows:int, restated_rows:int, unchanged:int,
34       *   duplicate_source_rows:int, conflicting_duplicate_source_rows:int, held:array<string,string>,
35       *   skipped:array<string,string>, restatements:array<string,string>,
36       *   annual_summary_rows:int, negative_rows:int, total_posted:float,
37       *   periods:array<string,float>
38       * }
39       */
40      public function import(array $rows): array
41      {
42          $held = [];
43          $skipped = [];
44          $restatements = [];
45          $seen = [];
46          $duplicates = 0;
47          $conflictingDuplicates = 0;
48          $loaded = 0;
49          $restated = 0;
50          $unchanged = 0;
51          $annualSummaries = 0;
52          $negatives = 0;
53          $total = 0.0;
54          $periods = [];
55          $now = now();
56
57          foreach ($rows as $index => $row) {
58              $label = 'row ' . ($index + 2);
59
60              $contractId = ContractId::normalise($row['contract_id'] ?? null);
61              if ($contractId === null) {
62                  $skipped[$label] = 'no loan account number on the row';
63                  continue;
64              }
65
66              // The full-population Extract C includes annual control totals
67              // identified by PERIOD_MONTH = 0. They repeat the monthly detail
68              // below them and must never enter a period-grain reconciliation.
69              // Keep them visible in the import evidence rather than silently
70              // discarding them or reporting them as malformed transactions.
71              $rawMonth = $row['period_month'] ?? null;
72              if ($rawMonth !== null && trim((string) $rawMonth) !== ''
73                  && (int) $this->amount($rawMonth) === 0) {
74                  $annualSummaries++;
75                  continue;
76              }
77
78              [$year, $month] = $this->period($row);
79              if ($year === null || $month === null) {
80                  $skipped[$contractId . ' @ ' . $label] = 'posting period is missing or not a valid year/month';
81                  continue;
82              }
83
84              $glAccount = trim((string) ($row['gl_account_code'] ?? '')) ?: null;
85              $naturalKey = implode('|', [$contractId, $year, $month, $glAccount ?? '']);
86              $posted = $this->amount($row['interest_income_posted'] ?? null);
87              $postingReferences = trim((string) ($row['posting_references'] ?? '')) ?: null;
88
89              if (isset($seen[$naturalKey])) {
```

```

90         $duplicates++;
91         $sameAmount = abs($seen[$naturalKey]['amount'] - $posted) < 0.005;
92         $sameReferences = $seen[$naturalKey]['references'] === $postingReferences;
93         if (! $sameAmount || ! $sameReferences) {
94             $conflictingDuplicates++;
95             $skipped[$naturalKey] = 'conflicting duplicate: the file carries different amounts or references for
the same loan, period and GL account';
96         }
97         continue;
98     }
99     $seen[$naturalKey] = ['amount' => $posted, 'references' => $postingReferences];
100
101     $loan = DB::table('loan_books')->where('contract_id', $contractId)
102         ->orderByDesc('reporting_period')->first(['contract_id']);
103     if (! $loan) {
104         $held[$contractId] = 'loan account is not present in the imported loan book';
105         continue;
106     }
107
108     if ($posted < 0) {
109         $negatives++;
110     }
111
112     $runId = trim((string) ($row['run_id'] ?? '')) ?: null;
113     $record = [
114         'contract_id' => $contractId,
115         'gl_account_code' => $glAccount,
116         'period_type' => strtoupper(trim((string) ($row['period_type'] ?? 'MONTHLY'))) ?: 'MONTHLY',
117         'period_year' => $year,
118         'period_month' => $month,
119         'reporting_period' => sprintf('%04d-%02d-01', $year, $month),
120         'interest_income_posted' => $posted,
121         'transaction_count' => max(0, (int) $this->amount($row['transaction_count'] ?? null)),
122         'posting_references' => $postingReferences,
123         'row_note' => trim((string) ($row['row_note'] ?? '')) ?: null,
124         'generated_on' => $this->date($row['generated_on'] ?? null),
125         'source_system' => self::SOURCE,
126         'source_reference' => $runId,
127         'external_transaction_id' => self::SOURCE . '|' . ($runId ?? 'no-run') . '|' . $naturalKey,
128         'updated_at' => $now,
129     ];
130
131     $existing = DB::table('gl_interest_postings')
132         ->where('contract_id', $contractId)
133         ->where('period_year', $year)
134         ->where('period_month', $month)
135         ->where(fn ($q) => $glAccount === null
136             ? $q->whereNull('gl_account_code')
137             : $q->where('gl_account_code', $glAccount))
138         ->first();
139
140     if ($existing === null) {
141         DB::table('gl_interest_postings')->insert($record + ['created_at' => $now]);
142         $loaded++;
143     } elseif (abs((float) $existing->interest_income_posted - $posted) < 0.005) {
144         $unchanged++;
145     } else {
146         $restatements[$naturalKey] = sprintf(
147             'GL restated from %s to %s',
148             number_format((float) $existing->interest_income_posted, 2),
149             number_format($posted, 2)
150         );
151         DB::table('gl_interest_postings')->where('id', $existing->id)->update($record);
152         $restated++;
153     }
154
155     $periodKey = sprintf('%04d-%02d', $year, $month);
156     $periods[$periodKey] = round(($periods[$periodKey] ?? 0) + $posted, 2);
157     $total += $posted;
158 }
159
160 ksort($periods);

```

Tool call: Bash

description: Check how the transaction import stores the estimated split and where it is used

--- command ---

```

cd /c/xampp/htdocs/MAICC-IFRS9 && sed -n '1,118p' app/Services/Eir/ContractTransactionImportService.php | grep -n -i "principal\
|interest\|estimat\|scheduled\|actual\|abs\|sign\|direction\|row_note\|transaction_type" | head -40; echo "--- downstream use
of the stored components:"; grep -rn -i "principal_component\|interest_component\|is_estimated\|scheduled_actual" app/database/
migrations --include=*.php | grep -v "ContractTransactionImportService" | cut -c1-190 | head -30

```

Tool call: Bash

description: Retest the running balance against the independent year-end balance using posting order

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python - <<'PYEOF' 2>&1 | cut -c1-260
import pandas as pd, numpy as np, warnings
warnings.filterwarnings("ignore")
pd.set_option("display.width", 250)
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
A = pd.read_excel(D + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
B = pd.read_excel(D + r"\Extract B.xlsx", sheet_name="Sheet1", dtype={"LOAN_ACCOUNT_NUMBER": str})
key = ["LOAN_ACCOUNT_NUMBER", "SUB_ACCOUNT_NO"]
y = A[A["AS_OF_DATE"].astype(str).str[:10] == "31-12-2025"].set_index(key)
act = B[B.SCHEDULED_ACTUAL_FLAG == "Actual"].copy()
act["seq"] = act.GL_POSTING_REF.str.split("-").str[-1].astype(int)
# closing balance = the row that no other row of the account follows in the balance chain
def closing(g):
    prevs = set((g.BALANCE_AFTER_TRANSACTION).round(2))
    # candidate closings: balances that are not the 'before' balance of any other row (before = after +/- amount)
    befores = set(np.round(np.concatenate([g.BALANCE_AFTER_TRANSACTION - g.TOTAL_AMOUNT, g.BALANCE_AFTER_TRANSACTION + g.TOTAL_AMOUNT]), 2))
    return sorted(prevs - befores)
rows = []
for k, g in act.groupby(key):
    a = y.OPENING_BALANCE.get(k, np.nan)
    last_file = g.BALANCE_AFTER_TRANSACTION.iloc[-1]; last_id = g.sort_values("seq").BALANCE_AFTER_TRANSACTION.iloc[-1]
    rows.append((k[0], a, last_file, last_id, any(abs(c - a) < 1 for c in closing(g)) if pd.notna(a) else None,
len(closing(g))))
df = pd.DataFrame(rows, columns=["acct", "A_bal", "last_file", "last_id", "chain_end_matches",
"chain_ends"]).dropna(subset=["A_bal"])
print("accounts", len(df), "| A balance = last row in file order:", int(((df.A_bal - df.last_file).abs() < 1).sum()), "| = row
with highest posting id:", int(((df.A_bal - df.last_id).abs() < 1).sum()), "| = an end of the balance chain:",
int(df.chain_end_matches.sum()))
bad = df[~((df.A_bal - df.last_file).abs() < 1) & ~((df.A_bal - df.last_id).abs() < 1)]
print(bad.round(2).to_string())
PYEOF
{
  "timeout": 300000
}
```

Result of Bash

```
9: * Imports the contract transactions file (Extract B) without confusing actual
10: * cash movements with contractual promises. The delivered Extract B has no
11: * usable fee component, so absent/blank fees are deliberately routed as zero;
12: * they do not manufacture a fee row or block the contractual schedule.
32:     $actualRows = [];
64:     $flag = strtoupper(trim((string) ($row['scheduled_actual_flag'] ?? '')));
66:     if ($flag === 'SCHEDULED') {
68:         'principal_due' => (float) ($row['principal_component'] ?? 0),
69:         'interest_due' => (float) ($row['interest_component'] ?? 0),
73:     if ($flag !== 'ACTUAL') {
74:         $conflicts[$account] = "unknown SCHEDULED_ACTUAL_FLAG '{$flag}'";
78:     $actualRows[] = $base + ['customer_id' => $customer ?: null, 'sub_account_no' => $subAccount ?: null,
80:         'transaction_type' => trim((string) ($row['transaction_type'] ?? 'Other')) ?: 'Other',
81:         'principal_component' => (float) ($row['principal_component'] ?? 0),
82:         'interest_component' => (float) ($row['interest_component'] ?? 0),
85:         'row_note' => trim((string) ($row['row_note'] ?? '')) ?: null];
89:         'description' => trim((string) ($row['transaction_type'] ?? 'Fee from Extract B')),
95:     $actualLoaded = $this->insertActuals($actualRows);
99:     return ['source_rows' => count($rows), 'scheduled_rows_routed' => count($scheduledRows),
100:         'actual_rows_loaded' => $actualLoaded, 'fee_rows_routed' => count($feeRows),
107:     private function insertActuals(array $rows): int
111:     if (DB::table('eir_actual_transactions')->where('source_system', self::SOURCE)
113:         DB::table('eir_actual_transactions')->insert($row + ['created_at' => now(), 'updated_at' => now()]));
--- downstream use of the stored components:
app/Console/Commands/MakeEirTestData.php:343:         'TRANSACTION_DATE', 'TRANSACTION_TYPE', 'PRINCIPAL_COMPONENT',
'INTEREST_COMPONENT',
app/Console/Commands/MakeEirTestData.php:344:         'FEE_COMPONENT', 'TOTAL_AMOUNT', 'SCHEDULED_ACTUAL_FLAG',
'BALANCE_AFTER_TRANSACTION',
app/Imports/ContractTransactionImport.php:37:         'PRINCIPAL_COMPONENT' => 'principal_component',
app/Imports/ContractTransactionImport.php:38:         'PRINCIPAL COMPONENT' => 'principal_component',
app/Imports/ContractTransactionImport.php:39:         'INTEREST_COMPONENT' => 'interest_component',
app/Imports/ContractTransactionImport.php:40:         'INTEREST COMPONENT' => 'interest_component',
app/Imports/ContractTransactionImport.php:45:         'SCHEDULED_ACTUAL_FLAG' => 'scheduled_actual_flag',
app/Imports/ContractTransactionImport.php:46:         'SCHEDULED ACTUAL FLAG' => 'scheduled_actual_flag',
app/Imports/ContractTransactionImport.php:47:         'ACTUAL_SCHEDULED_FLAG' => 'scheduled_actual_flag',
app/Services/Imports/MappedFileReader.php:65:         'transaction_type', 'principal_component', 'interest_component',
app/Services/Imports/MappedFileReader.php:66:         'total_amount', 'scheduled_actual_flag', 'gl_posting_ref',
app/Services/Imports/MappedFileReader.php:652:         if (isset($set['scheduled_actual_flag'], $set['loan_account_number'],
$set['principal_component'], $set['interest_component'])) {
database/migrations/2026_08_04_000000_add_extract_b_audit_fields.php:25:         $table->decimal('principal_component', 20,
2)->default(0);
database/migrations/2026_08_04_000000_add_extract_b_audit_fields.php:26:         $table->decimal('interest_component', 20,
2)->default(0);
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Result of Bash

```
accounts 111 | A balance = last row in file order: 106 | = row with highest posting id: 110 | = an end of the balance chain: 110
          acct      A_bal  last_file  last_id chain_end_matches chain_ends
37 000104450000015 3053316.65 1225683.03 1225683.03 False 2
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Read

```
file_path: C:\xampp\htdocs\MAICC-IFRS9\app\Services\Eir\ContractTransactionImportService.php
offset: 1
limit: 105
```

Result of Read

```
1  <?php
2
3  namespace App\Services\Eir;
4
5  use App\Support\ContractId;
6  use Illuminate\Support\Facades\DB;
7
8  /**
9   * Imports the contract transactions file (Extract B) without confusing actual
10   * cash movements with contractual promises. The delivered Extract B has no
11   * usable fee component, so absent/blank fees are deliberately routed as zero;
12   * they do not manufacture a fee row or block the contractual schedule.
13   */
14  class ContractTransactionImportService
15  {
16      /**
17       * Persisted lineage, not a display label: rows already in the database
18       * carry this value and it names the vendor file an auditor will ask for.
19       * The class was renamed for the operator; the stored provenance was not.
20       */
21      public const SOURCE = 'MAIIC_EXTRACT_B';
22
23      public function __construct(
24          private readonly RemainingScheduleImportService $remainingSchedules,
25          private readonly FeeImportService $fees,
26      ) {}
27
28      public function import(array $rows): array
29      {
30          $scheduleRows = [];
31          $feeRows = [];
32          $actualRows = [];
33          $held = [];
34          $conflicts = [];
35          $seen = [];
36          $duplicates = 0;
37
38          foreach ($rows as $index => $row) {
39              $account = ContractId::normalise($row['contract_id'] ?? null) ?? '';
40              $customer = trim((string) ($row['customer_id'] ?? ''));
41              $subAccount = trim((string) ($row['sub_account_no'] ?? ''));
42              $reference = trim((string) ($row['gl_posting_ref'] ?? ''));
43              $externalId = $reference !== '' ? $reference : implode('|', [
44                  trim((string) ($row['run_id'] ?? '')), $account, $subAccount,
45                  (string) ($row['transaction_date'] ?? ''), (string) ($index + 1),
46              ]);
47
48              if (isset($seen[$externalId])) { $duplicates++; continue; }
49              $seen[$externalId] = true;
50
51              $loan = DB::table('loan_books')->where('contract_id', $account)
52                  ->orderByDesc('reporting_period')->first(['contract_id', 'customer_id']);
53              if (!$loan) {
54                  $held[$account ? 'row ' . ($index + 2)] = 'loan account is not present in the imported loan book';
55                  continue;
56              }
57              if ($customer !== '' && trim((string) $loan->customer_id) !== '' && $customer !== trim((string)
58                  $loan->customer_id)) {
59                  $conflicts[$account] = "customer {$customer} does not match loan-book customer {$loan->customer_id}";
60                  continue;
61              }
62
63              $base = ['contract_id' => $account, 'source_system' => self::SOURCE,
64                  'source_reference' => $reference ? : null, 'external_transaction_id' => $externalId];
65              $flag = strtoupper(trim((string) ($row['scheduled_actual_flag'] ?? '')));
66
67              if ($flag === 'SCHEDULED') {
68                  $scheduleRows[] = $base + ['due_date' => $row['transaction_date'] ?? null,
69                      'principal_due' => (float) ($row['principal_component'] ?? 0),
70                      'interest_due' => (float) ($row['interest_component'] ?? 0),
71                      'fee_due' => (float) ($row['fee_component'] ?? 0)];
72                  continue;
73              }
74              if ($flag !== 'ACTUAL') {
75                  $conflicts[$account] = "unknown SCHEDULED_ACTUAL_FLAG '{$flag}'";
76                  continue;
77              }
78          }
79      }
80  }
```

```

77
78     $actualRows[] = $base + ['customer_id' => $customer ?: null, 'sub_account_no' => $subAccount ?: null,
79         'transaction_date' => $row['transaction_date'] ?? null,
80         'transaction_type' => trim((string) ($row['transaction_type'] ?? 'Other')) ?: 'Other',
81         'principal_component' => (float) ($row['principal_component'] ?? 0),
82         'interest_component' => (float) ($row['interest_component'] ?? 0),
83         'fee_component' => (float) ($row['fee_component'] ?? 0), 'total_amount' => (float) ($row['total_amount']
?? 0),
84         'balance_after_transaction' => ($row['balance_after_transaction'] ?? '') === '' ? null : (float)
$row['balance_after_transaction'],
85         'row_note' => trim((string) ($row['row_note'] ?? '')) ?: null];
86
87     if ((float) ($row['fee_component'] ?? 0) !== 0.0) {
88         $feeRows[] = $base + ['fee_type' => 'other',
89             'description' => trim((string) ($row['transaction_type'] ?? 'Fee from Extract B')),
90             'amount' => (float) $row['fee_component'], 'transaction_date' => $row['transaction_date'] ?? null,
91             'gl_account_ref' => $reference ?: null];
92     }
93 }
94
95 $actualLoaded = $this->insertActuals($actualRows);
96 $scheduleResult = $this->remainingSchedules->import($scheduleRows);
97 $feeResult = $this->fees->import($feeRows);
98
99 return ['source_rows' => count($rows), 'scheduled_rows_routed' => count($scheduleRows),
100     'actual_rows_loaded' => $actualLoaded, 'fee_rows_routed' => count($feeRows),
101     'duplicate_source_rows' => $duplicates, 'held' => $held + $scheduleResult['held'],
102     'skipped' => $conflicts + $scheduleResult['skipped'], 'loaded_contracts' =>
103     $scheduleResult['loaded_contracts'],
104     'loaded_rows' => $scheduleResult['loaded_rows'], 'remaining_schedule_result' => $scheduleResult,
105     'fee_result' => $feeResult, 'coverage' => $scheduleResult['coverage']];
106 }

```

CLAUDE (2026-10-05 15:29:26 UTC)

Codex's review is sound: I accept all eight corrections, and none of them overturns the three headline findings. I ran extra tests on the points it raised, and two of them show my report was wrong in more places than Codex said. I have not changed the report or the query pack yet.

The eight corrections

#	Codex's point	My position	Evidence
1	Scheduled rows are not the unpaid remainder	Accept	The script exports the full original instalment amounts and never subtracts the amount recovered. My wording "unpaid remainder" was wrong.
2	Running balance is not proven right	Accept	I stated it unconditionally. New test: Extract B's closing balance agrees with Extract A's 31 December 2025 balance on 110 of 111 accounts. The exception is Zaithwa Farms (1,225,683.03 against 3,053,316.65).
3	No interest rows is not a verified zero	Accept	My table marked the 61 missing accounts "Explained, none needed". The script only shows that no posting matched its filters.
4	Final-day date handling	Accept	If the dates carry a time of day, postings on the last day can drop out. This needs a test query added to the pack.
5	Agreement in total does not prove what the adjustments are	Accept, and it goes further	See below.
6	Schema and privilege assumptions	Accept	Several catalogue outputs omit the owner, so two schemas holding 'ACMASTER' would be mixed together. Being able to run the procedure does not prove the login can read the tables.
7	Cause of the repayment-frequency differences	Accept the rewording	See below; the evidence is stronger than I wrote, but still not proof.
8	Define the export contract	Accept	"CSV with year-first dates" is not enough. Account numbers must be text or the leading zeros are lost.

Where my report is more wrong than Codex said

- **"Other/Adjustment" rows are not mainly interest reductions.** My report says they reduce interest and that on six accounts they are exactly the B–C difference. There are 31 such rows totalling MWK 832.7m, against a B–C difference of 85.2m. Of them 18 raise the balance, 9 reduce it and 3 are unclear. On Happie Foods the 7,000,000 raises the balance and only the 2,018,700.44 reduces it. The exact match holds on five accounts; the general statement has to go.
- **The 1 January 2025 snapshot's repayment frequency is unusable.** It shows "Monthly" on all 181 accounts. The 31 December snapshot shows 139 Monthly, 17 other frequencies and 25 blank. The delivered script leaves accounts with no instalment plan blank, so it would not produce a uniform "Monthly" unless the data changed within the hour. Codex is right that this still needs the run history from MAIIC.

Where the new tests support the report

- **Interest rows are charges.** 987 of 994 interest rows raise the running balance, and 485 of 501 repayment rows reduce it.
- **Extract B and C still agree** on 101 of 107 accounts, as Codex reproduced.

The application points

- **Interest import:** Codex is right, and this closes the open item in my report. The import skips the yearly total rows and does not load a repeat of the same loan, month and GL code. So the tripled 2025 in the Extract C file would not have

loaded three times. I read the code on the `eir_revenue_recognition` branch; I did not inspect the loaded data.

- **Estimated split:** the import stores it, but I found no other code that reads those two columns. That was a text search, not a full trace.
- **One thing Codex did not flag:** the same import routes Extract B's "Scheduled" rows into the remaining-schedule tables as principal and interest due. Given point 1, those are full original amounts for 2025 instalments only, so that route needs tracing before the schedule is relied on.

What I would change

1. **Report:** reword points 1, 2, 3, 5 and 7, add the date-boundary and export-contract points, and mark the interest import item as checked.
2. **Query pack:** carry the owner through every catalogue output, add a check for time-of-day in the ledger dates, set an explicit date format, and add a direct read-access check.
3. **Research report of 24 September:** correct the tranche and "cash received" statements.

Say the word and I'll apply these and reissue the PDF. Separately, the review file is sitting in the root of the Stress-Testing-App repository, which is a different client's project, so it should be moved before anything is committed there.

USER (2026-10-05 15:33:46 UTC)

move the the review file to the maicc ifrs 9 laravel repo. adopt the corrections from codex and save in place the updated version. In the report please include the suggestions to the code

Tool call: Bash

```
description: Check the application repository's docs folder and how scheduled rows are imported
--- command ---
cd /c/xampp/htdocs/MAICC-IFRS9 && git status --short | head -15; echo "--- docs:"; ls docs | head -40; echo "--- remaining
schedule service:"; wc -l app/Services/Eir/RemainingScheduleImportService.php; sed -n '1,60p' app/Services/Eir/
RemainingScheduleImportService.php
```

Tool call: Bash

```
description: Trace where actual transactions, the rate label and disbursement tranches are used in the application
--- command ---
cd /c/xampp/htdocs/MAICC-IFRS9 && echo "--- who reads eir_actual_transactions:"; grep -rn "eir_actual_transactions" app
--include=*.php | grep -v "insert(" | cut -c1-200 | head -20; echo "--- rate basis / rate_type derivation:"; grep -rn -i
"rate_basis\|floating_flag\|interest_policy" app --include=*.php | cut -c1-210 | head -25; echo "--- disbursement tranches:";
grep -rn -i "disbursement_tranches\|contract_disbursements" app --include=*.php | cut -c1-200 | head -15
```

Result of Bash

```
--- docs:
Development_of_EIR.md
EIR_Build.md
EIR_Existing_vs_Proposed_Build.md
MAIIC_EIR_Revenue_Recognition_Engine_Spec.md
MAIIC_EIR_Revenue_Recognition_Engine_Spec_Explained.md
SSL_SETUP.md
UI_REPORT_SPEC.md
manuals
--- remaining schedule service:
31 app/Services/Eir/RemainingScheduleImportService.php
<?php
namespace App\Services\Eir;
use App\Support\ContractId;
use Illuminate\Support\Facades\DB;

/** Extract B future rows are validation evidence, not the original contractual schedule. */
class RemainingScheduleImportService
{
    public function import(array $rows): array
    {
        $loaded=0; $unchanged=0; $held=[]; $skipped=[]; $contracts=[];
        foreach ($rows as $index=>$row) {
            $id=ContractId::normalise($row['contract_id']??null);
            if (!$id) { $skipped['row'.($index+2)]= 'loan account is missing'; continue; }
            if (!DB::table('loan_books')->where('contract_id',$id)->exists()) { $held[$id]= 'loan account is not present in the
imported loan book'; continue; }
            if (empty($row['due_date'])) { $skipped[$id]= 'remaining-schedule due date is missing'; continue; }
            $externalId=trim((string)($row['external_transaction_id']??''));
            if ($externalId==='') $externalId=hash('sha256',implode('|',[$id,$row['due_date'],$index,$row['principal_due']??0,
$row['interest_due']??0,$row['fee_due']??0]));
            $source=trim((string)($row['source_system']??'MAIIC_EXTRACT_B'))?:'MAIIC_EXTRACT_B';
            if (DB::table('contract_remaining_cashflow_schedule')->where('source_system',
$source)->where('external_transaction_id',$externalId)->exists()) { $unchanged++; continue; }
            DB::table('contract_remaining_cashflow_schedule')->insert([
                'contract_id'=>$id, 'due_date'=>$row['due_date'], 'principal_due'=>(float)($row['principal_due']??0),
                'interest_due'=>(float)($row['interest_due']??0), 'fee_due'=>(float)($row['fee_due']??0),
                'source_system'=>$source, 'source_reference'=>($row['source_reference']??null)?:null,
                'external_transaction_id'=>$externalId, 'row_note'=>($row['row_note']??null)?:null,
                'created_at'=>now(), 'updated_at'=>now(),
            ]); $contracts[$id]=true; $loaded++;
        }
        return ['loaded_contracts'=>count($contracts), 'loaded_rows'=>$loaded, 'unchanged'=>$unchanged, 'held'=>$held,
'skipped'=>$skipped, 'coverage'=>['covered'=>count($contracts), 'total'=>0]];
    }
}
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Result of Bash

```
--- who reads eir_actual_transactions:
app/Services/Eir/ContractTransactionImportService.php:111:         if
(DB::table('eir_actual_transactions')->where('source_system', self::SOURCE)
app/Services/Eir/EirRevenueService.php:161:         $window = DB::table('eir_actual_transactions')->where('contract_id',
$contractId)
app/Services/Eir/EirRevenueService.php:173:         $rows = DB::table('eir_actual_transactions')->where('contract_id',
$contractId)
--- rate basis / rate_type derivation:
app/Console/Commands/MakeEirTestData.php:133:         'rate_basis' => 'Variable', 'fees' => [['ARRANGEMENT_FEE',
4_000_000, 'RECEIVED', 1]], 'stage_path' => 'stage3', 'provision_rate' => 0.50,
app/Console/Commands/MakeEirTestData.php:176:         'stage_path' => 'performing', 'provision_rate' => 0.0, 'rate_basis' =>
'Fixed',
app/Console/Commands/MakeEirTestData.php:312:         'SANCTIONED_AMOUNT', 'PRINCIPAL_DISBURSED', 'DISBURSEMENT_TRANCHES',
'INTEREST_RATE', 'RATE_BASIS',
app/Console/Commands/MakeEirTestData.php:329:         round($s['rate'] * 100, 4), $s['rate_basis'], 'Compound', '365',
$s['frequency'],
app/Imports/ContractMasterImport.php:68:         // The delivered RATE_BASIS column carries Fixed/Variable – it is
app/Imports/ContractMasterImport.php:70:         // the free-text rate_basis column would leave rate_type on its
app/Imports/ContractMasterImport.php:72:         'RATE_BASIS' => 'rate_type',
app/Imports/ContractMasterImport.php:76:         'INTEREST_BASIS' => 'rate_basis',
app/Imports/ContractMasterImport.php:77:         'ACCRUAL_BASIS' => 'rate_basis',
app/Models/ContractEir.php:36:         'rate_basis',
app/Services/Eir/ContractMasterImportService.php:50:         'contractual_rate', 'rate_basis', 'rate_type',
app/Services/Eir/ContractMasterImportService.php:277:         'rate_basis' => $this->text($row['rate_basis'] ?? null),
app/Services/Imports/MappedFileReader.php:85:         'contractual_rate', 'rate_basis', 'rate_type',
'reference_rate_at_origination',
--- disbursement tranches:
app/Console/Commands/MakeEirTestData.php:312:         'SANCTIONED_AMOUNT', 'PRINCIPAL_DISBURSED', 'DISBURSEMENT_TRANCHES',
'INTEREST_RATE', 'RATE_BASIS',
app/Imports/ContractMasterImport.php:82:         'DISBURSEMENT_TRANCHES' => 'disbursement_tranches',
app/Imports/ContractMasterImport.php:83:         'DISBURSEMENT_TRANCHES' => 'disbursement_tranches',
app/Models/ContractEir.php:22:         'disbursement_tranches',
app/Services/Eir/ContractMasterImportService.php:51:         'source_day_count_basis', 'source_compounding',
'disbursement_tranches',
app/Services/Eir/ContractMasterImportService.php:281:         'disbursement_tranches' =>
$this->text($row['disbursement_tranches'] ?? null),
app/Services/Imports/MappedFileReader.php:86:         'source_day_count_basis', 'source_compounding',
'disbursement_tranches',
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

description: Read how the rate label is mapped and how actual transactions feed revenue

```
--- command ---
cd /c/xampp/htdocs/MAICC-IFRS9 && echo "=== ContractMasterImport 62-78"; sed -n '62,78p' app/Imports/ContractMasterImport.php;
echo "=== rate_type normalisation"; grep -n -i "rate_type\|FLOATING\|VARIABLE" app/Services/Eir/ContractMasterImportService.php
| cut -c1-200 | head -20; echo "=== EirRevenueService 150-200"; sed -n '150,200p' app/Services/Eir/EirRevenueService.php
```

Tool call: Bash

description: Trace readers of the remaining schedule, tranche and rate fields, and the date parsing

```
--- command ---
cd /c/xampp/htdocs/MAICC-IFRS9 && echo "=== readers of contract_remaining_cashflow_schedule"; grep -rn
"contract_remaining_cashflow_schedule" app --include=*.php | cut -c1-190 | head -12; echo "=== consumers of
disbursement_tranches / rate_type"; grep -rn "disbursement_tranches\|>rate_type\|'rate_type'" app/Services --include=*.php |
grep -v "ContractMasterImportService\|MappedFileReader" | cut -c1-200 | head -20; echo "=== date parsing in the mapped reader";
grep -n -i "function date\|Carbon::\|createFromFormat\|strtotime\|excelToDate\|ambiguous\|d/m\|m/d" app/Services/Imports/
MappedFileReader.php app/Services/Eir/ContractMasterImportService.php app/Services/Eir/GlInterestImportService.php | cut
-c1-200 | head -25; echo "=== reconciliation service header"; sed -n '1,40p' app/Services/Eir/EirGlReconciliationService.php |
cut -c1-200
```

Result of Bash

```
=== ContractMasterImport 62-78
    'PRINCIPAL_DISBURSED' => 'drawn_amount',
    'DISBURSED_AMOUNT' => 'drawn_amount',

    'INTEREST_RATE' => 'contractual_rate',
    'INTEREST RATE' => 'contractual_rate',
    'CONTRACTUAL_RATE' => 'contractual_rate',
    // The delivered RATE_BASIS column carries Fixed/Variable – it is
    // the rate type, not a day-count or accrual basis. Mapping it to
    // the free-text rate_basis column would leave rate_type on its
    // FIXED default for the 204 variable-rate facilities in the book.
    'RATE_BASIS' => 'rate_type',
    'RATE BASIS' => 'rate_type',
    'RATE_TYPE' => 'rate_type',
    'RATE TYPE' => 'rate_type',
    'INTEREST_BASIS' => 'rate_basis',
    'ACCRUAL_BASIS' => 'rate_basis',
    'DAY_COUNT_BASIS' => 'source_day_count_basis',
=== rate_type normalisation
50:     'contractual_rate', 'rate_basis', 'rate_type',
282:         'rate_type' => $this->rateType($row['rate_type'] ?? null),
480:         $type === 'FLOATING', $type === 'VARIABLE' => 'FLOATING',
=== EirRevenueService 150-200
    * receipt. A disbursement is cash advanced, not collected, and is never
    * netted off. Any other delivered type is excluded and reported rather
    * than absorbed, because its direction and component split are unknown.
    *
    * @return array{amount:float,source:string,unclassified:float}
    */
private function cashReceived(string $contractId, string $period): array
{
    $start = CarbonImmutable::createFromFormat('Y-m-d', $period . '-01')->startOfMonth();
    $end = $start->endOfMonth();

    $window = DB::table('eir_actual_transactions')->where('contract_id', $contractId)
        ->selectRaw('MIN(transaction_date) as first_txn, MAX(transaction_date) as last_txn')->first();
    $covered = $window && $window->first_txn && $window->last_txn
        && $start->toDateString() <= substr((string) $window->last_txn, 0, 10)
        && $end->toDateString() >= substr((string) $window->first_txn, 0, 10);

    if (! $covered) {
        return ['amount' => $this->scheduledCash($contractId, $start, $end), 'source' => 'DERIVED', 'unclassified' => 0.0];
    }

    $collected = 0.0;
    $unclassified = 0.0;
    $rows = DB::table('eir_actual_transactions')->where('contract_id', $contractId)
        ->whereBetween('transaction_date', [$start->toDateString(), $end->toDateString()])
        ->selectRaw('transaction_type, SUM(total_amount) as amount')->groupBy('transaction_type')->get();

    foreach ($rows as $row) {
        if (in_array($row->transaction_type, self::COLLECTION_TYPES, true)) {
            $collected += (float) $row->amount;
        } elseif (! in_array($row->transaction_type, self::ADVANCE_TYPES, true)) {
            $unclassified += (float) $row->amount;
        }
    }

    return ['amount' => $collected, 'source' => 'IMPORTED', 'unclassified' => $unclassified];
}

/** The contractual promise for the period – used only where no actuals cover it. */
private function scheduledCash(string $contractId, CarbonImmutable $start, CarbonImmutable $end): float
{
    return (float) DB::table('contract_cashflow_schedule')->where('contract_id', $contractId)
        ->where('schedule_version', 1)
        ->whereBetween('due_date', [$start->toDateString(), $end->toDateString()])
        ->selectRaw('COALESCE(SUM(principal_due + interest_due + fee_due), 0) as due')->value('due');
}
```

```

private function normalisePeriod(string $period): string
{
    if (preg_match('/^\d{4})-?\d{0-2}(\d{0-2})?$/i', trim($period), $m) !== 1) {
        throw new InvalidArgumentException('Reporting period must be YYYY-MM, YYYYMM or a date beginning YYYY-MM.');
```

Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

Result of Bash

```

=== readers of contract_remaining_cashflow_schedule
app/Http/Controllers/EirDataController.php:43:         'remaining_cashflows' =>
DB::table('contract_remaining_cashflow_schedule')->count(),
app/Http/Controllers/EirScheduleController.php:23:
$remaining=DB::table('contract_remaining_cashflow_schedule')->where('contract_id',$contractEir->contract_id)
app/Models/ContractRemainingCashflowSchedule.php:7:     protected $table = 'contract_remaining_cashflow_schedule';
app/Services/Eir/RemainingScheduleImportService.php:20:         if
(DB::table('contract_remaining_cashflow_schedule')->where('source_system',$source)->where('external_transaction_id',$ext
app/Services/Eir/RemainingScheduleImportService.php:21:         DB::table('contract_remaining_cashflow_schedule')->insert([
app/Services/Eir/ScheduleWorkflowService.php:93:
$remaining=DB::table('contract_remaining_cashflow_schedule')->where('contract_id',
$contract->contract_id)->orderBy('due_date')->get();
=== consumers of disbursement_tranches / rate_type
app/Services/Ecl/EclDiscountRateService.php:88:         ->get(['contract_id', 'eir_effective_annual',
'rate_type'])->keyBy('contract_id');
app/Services/Ecl/EclDiscountRateService.php:107:         'applied_source' => $contract->rate_type === 'FLOATING'
app/Services/Ecl/TimePhasedEclService.php:75:
$rateSource=$eir->rate_type==='FLOATING'? 'EIR_ORIGINAL_FLOATING_PROXY': 'EIR_ORIGINAL';
app/Services/Eir/EirContractInputService.php:89:         'rate_type' => $contract->rate_type,
=== date parsing in the mapped reader
app/Services/Imports/MappedFileReader.php:289:         *           ['due_date' => 'date:d/m/Y', 'amount' =>
'number']
app/Services/Imports/MappedFileReader.php:429:         return ExcelDate::excelToDateTimeObject((float)
$value)->format('Y-m-d');
app/Services/Imports/MappedFileReader.php:439:         // createFromFormat is lenient about separators, so require the
app/Services/Imports/MappedFileReader.php:441:         $date = Carbon::createFromFormat($format, $raw);
app/Services/Imports/MappedFileReader.php:451:         return Carbon::parse($raw)->toDateString();
app/Services/Imports/MappedFileReader.php:648:         /** Detect only file shapes whose distinguishing headers are unambiguous. */
app/Services/Eir/ContractMasterImportService.php:425:     private function date($value): ?string
app/Services/Eir/GlInterestImportService.php:223:     private function date($value): ?string
=== reconciliation service header
<?php
```

```
namespace App\Services\Eir;
```

```

use App\Models\ContractEir;
use App\Models\EirAmortisation;
use App\Models\GlInterestPosting;
```

```

/**
 * Explains the difference between EIR interest income and what the ledger
 * posted, for one reporting period.
 *
 * A bare variance column invites the reader to assume the engine is wrong. It
 * is not enough to state that the two disagree; the report has to say which
 * moving part caused it, because only some of them are misstatements. Both
 * sides accrue interest monthly, so the difference resolves into three terms
 * that sum to the variance exactly:
 *
 * implied base      = GL posted / contractual monthly rate
 * base effect       = contractual monthly rate x (amortised opening - implied base)
 * rate effect        = (effective monthly rate - contractual monthly rate) x opening
 * impairment effect = interest accrued - effective monthly rate x opening
 *
 * The BASE EFFECT is the difference between the balance the engine amortises
 * and the balance the ledger accrued on. That base is derived from the posting
 * itself rather than assumed: a ledger posting flat interest on original
 * principal and one accruing on the declining balance are both common, and
 * hardcoding either turns the other's entire base difference into a residual.
 * The implied base is reported per row, because whether a ledger amortises is
 * itself a finding worth seeing.
 *
 * The RATE EFFECT is the yield uplift from fees integral to the EIR – the
 * amount this engine exists to recognise. It is only non-zero once integral
 * fees are classified, because without them the solved EIR de-compounds to
 * exactly the contractual monthly rate.
 *
 * The IMPAIRMENT EFFECT is the shortfall from Stage 3 accruing on the amortised
 * cost net of the loss allowance rather than on gross (IFRS 9 5.4.1). It is
 * zero on every performing row, and it is a correct measurement difference
 * rather than an error.
```

Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

Tool call: Bash

description: Read which transaction types the engine counts as cash, the rate type mapping and the date parsing

```

--- command ---
cd /c:/xampp/htdocs/MAICC-IFRS9 && grep -n "COLLECTION_TYPES\|ADVANCE_TYPES" -A3 app/Services/Eir/EirRevenueService.php | head
-24; sed -n '138,152p' app/Services/Eir/EirRevenueService.php; echo "=== rateType()"; sed -n '470,486p' app/Services/Eir/
ContractMasterImportService.php; echo "=== date()"; sed -n '420,455p' app/Services/Imports/MappedFileReader.php; echo "===
master import date()"; sed -n '425,445p' app/Services/Eir/ContractMasterImportService.php; echo "=== schedule workflow 85-100";
sed -n '86,100p' app/Services/Eir/ScheduleWorkflowService.php | cut -c1-220
```

Result of Bash

```
22: private const COLLECTION_TYPES = ['Interest', 'Principal+Interest', 'Fee'];
23:
24: /** Cash advanced to the customer – the opposite direction, never a receipt. */
25: private const ADVANCE_TYPES = ['Disbursement'];
26:
27: /**
28:  * @param bool $recalculate Supersede an existing row instead of leaving it.
29:  */
30: --
178:         if (in_array($row->transaction_type, self::COLLECTION_TYPES, true)) {
179:             $collected += (float) $row->amount;
180:         } elseif (! in_array($row->transaction_type, self::ADVANCE_TYPES, true)) {
181:             $unclassified += (float) $row->amount;
182:         }
183:     }
    }
    return 0;
}

/**
 * Cash collected in the period, with its provenance.
 *
 * Delivered actuals are authoritative wherever the feed covers the period:
 * inside that window an absent row means the customer paid nothing, which
 * the contractual schedule cannot distinguish from a month it never
 * reached. Outside the window the schedule is the only evidence there is,
 * and the row is labelled DERIVED so a reader can tell a promise from a
 * receipt. A disbursement is cash advanced, not collected, and is never
 * netted off. Any other delivered type is excluded and reported rather
 * than absorbed, because its direction and component split are unknown.
=== rateType()

    return $amount > 1 ? $amount / 100 : $amount;
}

private function rateType($value): ?string
{
    $type = strtoupper((string) $this->text($value));

    return match (true) {
        $type === 'FIXED' => 'FIXED',
        $type === 'FLOATING', $type === 'VARIABLE' => 'FLOATING',
        default => null,
    };
}

private function wholeNumber($value): ?int
{
=== date()
private function toDateString($value, ?string $format): ?string
{
    if ($value === null || $value === '' || $value === '-') {
        return null;
    }

    // Excel serial dates arrive as numerics from XLSX cells.
    if (is_numeric($value) && (float) $value > 20000 && (float) $value < 80000) {
        try {
            return ExcelDate::excelToDateTimeObject((float) $value)->format('Y-m-d');
        } catch (Throwable) {
            return null;
        }
    }

    $raw = trim((string) $value);

    if ($format !== null) {
        try {
            // createFromFormat is lenient about separators, so require the
            // round-trip to match before trusting the declared format.
            $date = Carbon::createFromFormat($format, $raw);
            if ($date !== false && $date->format($format) === $raw) {
                return $date->toDateString();
            }
        } catch (Throwable) {
            // fall through to flexible parsing
        }
    }

    try {
        return Carbon::parse($raw)->toDateString();
    } catch (Throwable) {
        return null;
    }
}

=== master import date()
private function date($value): ?string
{
    // MappedFileReader has already resolved dates to Y-m-d; anything else
    // reaching here was unparseable and must not become a term.
```

```

    $text = $this->text($value);

    return $text !== null && preg_match('/^\d{4}-\d{2}-\d{2}$/', $text) ? $text : null;
}

/**
 * Absent and zero must stay distinguishable: a blank cell means "this file
 * does not carry the term" and must not overwrite a stored value, while a
 * genuine 0 is a term in its own right.
 */
private function amount($value): ?float
{
    if (is_int($value) || is_float($value)) {
        return (float) $value;
    }

    $text = $this->text($value);
=== schedule workflow 85-100
        try { $this->generate($contract); $generated++; } catch (\Throwable $e) {
$skipped[$contract->contract_id]=$e->getMessage(); }
        });
        return ['generated'=>$generated, 'skipped'=>count($skipped), 'exceptions'=>$skipped];
    }

    public function comparison(ContractEir $contract): array
    {
        $remaining=DB::table('contract_remaining_cashflow_schedule')->where('contract_id',
$contract->contract_id)->orderBy('due_date')->get();
        if ($remaining->isEmpty()) return ['status'=>'NO_REMAINING_DATA', 'cutoff_date'=>null, 'generated_rows'=>0,
'remaining_rows'=>0,
            'principal_variance'=>null, 'interest_variance'=>null, 'matched_dates'=>0];
        $cutoff=(string)$remaining->min('due_date');
        $generated=DB::table('contract_cashflow_schedule')->where('contract_id',
$contract->contract_id)->where('schedule_version',1)
            ->whereDate('due_date', '>=', $cutoff)->get();
        $pVar=(float)$generated->sum('principal_due')-(float)$remaining->sum('principal_due');
        $iVar=(float)$generated->sum('interest_due')-(float)$remaining->sum('interest_due');
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

```

CLAUDE (2026-10-05 15:36:51 UTC)

Moving the review file first, then updating the query pack, then the report. I've also read the engine code Codex pointed at so the code suggestions in the report are specific.

Tool call: Bash

description: Move the review file into the application repository and apply the review's corrections to the SQL query pack

```
--- command ---
mkdir -p /c/xampp/htdocs/MAICC-IFRS9/docs/reviews/ && mv "/c/xampp/htdocs/Stress-Testing-App/MAIIC-Claude-Review-2026-10-05.md" /
c/xampp/htdocs/MAICC-IFRS9/docs/reviews/ && ls -la /c/xampp/htdocs/MAICC-IFRS9/docs/reviews/ && ls /c/xampp/htdocs/
Stress-Testing-App/MAIIC-Claude-Review-2026-10-05.md 2>&1 | head -1; cd /c/xampp/htdocs/MAICC-IFRS9 && git status --short |
head -5; cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && cat > patch_sql.py <<'PYEOF'
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC"
p = D + r"\MAIIC_EBanker_Data_Dictionary_Queries.sql"
s = open(p, encoding="utf-8").read().replace("\r\n", "\n")
def rep(old, new, n=1):
    global s
    assert s.count(old) == n, (old[:60], s.count(old)); s = s.replace(old, new)
T13 = """"('ACMASTER', 'ACLOANMASTER', 'CUSTOMER_MAST', 'SCHEME_MASTER', 'GLMASTER',
          'CUMVOUCH', 'ACCOUNT_BALANCE', 'FIXLOAN_INST_CHART', 'MULTI_INST_DETAILS',
          'LOAN_DISBURSMENT_SCHEDULE', 'LOAN_RESCHEDULE_DETAILS',
          'LOAN_WRITEOFF_MASTER', 'LOAN_WRITEOFF_TRANS')""""

# ---- header
rep("-- Prepared by Dupleix Institute, 5 October 2026\n--\n-- PURPOSE",
    "-- Prepared by Dupleix Institute, 5 October 2026\n-- Version 2, revised the same day after an independent review.\n--\n--
PURPOSE")
rep("""-- SAFE TO RUN
-- Every statement is a SELECT. Nothing is created, changed or deleted.
-- Queries DD_01 to DD_08 read Oracle's own catalogue views (ALL_TABLES and so
-- on), not business data. DD_09 to DD_15 read codes and counts. DD_16 reads the
-- rows of ONE sample loan.""",
    """-- SAFE TO RUN
-- Every query is a SELECT. Nothing is created, changed or deleted. The only
-- other statements are the two session settings below, which change how THIS
-- session displays dates and numbers and nothing in the database.
-- Queries DD_01 to DD_08 and DD_17 read Oracle's own catalogue views
-- (ALL_TABLES and so on), not business data. DD_09 to DD_15, DD_18 and DD_19
-- read codes, counts and checks. DD_16 reads the rows of ONE sample loan.""")
rep("""-- HOW TO RUN
-- 1. Log in as the same user that runs PROC_BUILD_MAIIC_FACILITY_REGISTER.
-- 2. Run each query on its own and export the result grid as CSV (not Excel),
--    using the file name given above the query, for example DD_01_schema.csv.
-- 3. Please do not open and re-save the CSV files in Excel before sending:
--    that is what scrambled the dates in the earlier extracts.
--
-- THE SCHEMA
-- The queries find the schema by looking for the table ACMASTER, so no schema
-- name has to be typed in. If DD_01 returns no rows, the login cannot see
-- ACMASTER: please tell us which schema owns it.
-----
""",
    """-- HOW TO RUN
-- 1. Log in as the same user that runs PROC_BUILD_MAIIC_FACILITY_REGISTER.
-- 2. Run the two session settings below first.
-- 3. Run each query on its own and export the result grid as CSV, using the
--    file name given above the query, for example DD_01_schema.csv.
--
-- HOW THE FILES SHOULD COME BACK (the export contract)
-- - CSV, comma separated, UTF-8, one header row.
-- - Dates as YYYY-MM-DD; dates with a time as YYYY-MM-DD HH:MM:SS (24 hour).
--   The session settings below produce this.
-- - Account numbers exactly as stored, as text, with their leading zeros.
-- - Amounts as plain numbers: a dot for the decimal point, no thousands
--   separators, and the minus sign kept.
-- - Please do not open and re-save the CSV files in Excel before sending.
--   Excel is what swapped days and months in the earlier extracts, and it
--   also drops the leading zeros from account numbers.
--
-- THE SCHEMA, AND WHAT THE LOGIN CAN READ
-- The catalogue queries find the schema by looking for the table ACMASTER, so
-- no schema name has to be typed in. Every catalogue result carries the OWNER,
-- because more than one schema can hold a table called ACMASTER (a test or
-- archive copy, for example).
-- - If DD_01 returns no rows, the login cannot see ACMASTER: please tell us
--   which schema owns it.
-- - If DD_01 lists more than one owner, please tell us which one is the live
--   schema before going further.
-- - DD_17 shows which schema the login's table names point to, and who owns
--   the three extract procedures. DD_18 tests that the login can read each
--   table directly. Being able to run a procedure does not prove that.
-----

-- SESSION SETTINGS: run these two lines first.
ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY-MM-DD HH24:MI:SS';
ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.,';
""")
# ---- DD_17 after DD_01
rep("""ORDER BY t.owner, t.table_name;

-- DD_02_tables.csv""",
    """ORDER BY t.owner, t.table_name;
-- DD_17_login_and_synonyms.csv
-- Who is logged in, which schema that login's table names point to, any synonyms
```



```

FROM loan_writeoff_master;
"""
rep("-- End of file. Twenty-four result sets in all: DD_01 to DD_15, and DD_16a to DD_16i.", "-- End of file. Twenty-seven
queries in all: DD_01 to DD_15, DD_16a to DD_16i, and DD_17 to DD_19.")
open(p, "w", encoding="utf-8", newline="\r\n").write(s)
print("sql pack patched; DD markers:", s.count("\n-- DD_"))
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python patch_sql.py

```

Result of Bash [error]

```

Exit code 2
/usr/bin/bash: -c: line 101: unexpected EOF while looking for matching ``'

```

Tool call: Write

```

file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\patch_sql.py
--- content ---
"""Apply the review's corrections to the data dictionary SQL pack (version 2)."""
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC"
p = D + r"\MAIIC_EBanker_Data_Dictionary_Queries.sql"
s = open(p, encoding="utf-8").read().replace("\r\n", "\n")

def rep(old, new, n=1):
    global s
    assert s.count(old) == n, (old[:60], s.count(old))
    s = s.replace(old, new)

T13 = """('ACMASTER', 'ACLOANMASTER', 'CUSTOMER_MAST', 'SCHEME_MASTER', 'GLMASTER',
'CUMVOUCH', 'ACCOUNT_BALANCE', 'FIXLOAN_INST_CHART', 'MULTI_INST_DETAILS',
'LOAN_DISBURSMENT_SCHEDULE', 'LOAN_RESCHEDULE_DETAILS',
'LOAN_WRITEOFF_MASTER', 'LOAN_WRITEOFF_TRANS')"""

# ---- header
rep("-- Prepared by Dupleix Institute, 5 October 2026\n--\n-- PURPOSE",
    "-- Prepared by Dupleix Institute, 5 October 2026\n-- Version 2, revised the same day after an independent review.\n--\n--
PURPOSE")
rep("""-- SAFE TO RUN
-- Every statement is a SELECT. Nothing is created, changed or deleted.
-- Queries DD_01 to DD_08 read Oracle's own catalogue views (ALL_TABLES and so
-- on), not business data. DD_09 to DD_15 read codes and counts. DD_16 reads the
-- rows of ONE sample loan.""",
    """-- SAFE TO RUN
-- Every query is a SELECT. Nothing is created, changed or deleted. The only
-- other statements are the two session settings below, which change how THIS
-- session displays dates and numbers and nothing in the database.
-- Queries DD_01 to DD_08 and DD_17 read Oracle's own catalogue views
-- (ALL_TABLES and so on), not business data. DD_09 to DD_15, DD_18 and DD_19
-- read codes, counts and checks. DD_16 reads the rows of ONE sample loan.""")
rep("""-- HOW TO RUN
-- 1. Log in as the same user that runs PROC_BUILD_MAIIC_FACILITY_REGISTER.
-- 2. Run each query on its own and export the result grid as CSV (not Excel),
-- using the file name given above the query, for example DD_01_schema.csv.
-- 3. Please do not open and re-save the CSV files in Excel before sending:
-- that is what scrambled the dates in the earlier extracts.
--
-- THE SCHEMA
-- The queries find the schema by looking for the table ACMASTER, so no schema
-- name has to be typed in. If DD_01 returns no rows, the login cannot see
-- ACMASTER: please tell us which schema owns it.
-----
""",
    """
    """-- HOW TO RUN
    -- 1. Log in as the same user that runs PROC_BUILD_MAIIC_FACILITY_REGISTER.
    -- 2. Run the two session settings below first.
    -- 3. Run each query on its own and export the result grid as CSV, using the
    -- file name given above the query, for example DD_01_schema.csv.
    --
    -- HOW THE FILES SHOULD COME BACK (the export contract)
    -- - CSV, comma separated, UTF-8, one header row.
    -- - Dates as YYYY-MM-DD; dates with a time as YYYY-MM-DD HH:MM:SS (24 hour).
    -- The session settings below produce this.
    -- - Account numbers exactly as stored, as text, with their leading zeros.
    -- - Amounts as plain numbers: a dot for the decimal point, no thousands
    -- separators, and the minus sign kept.
    -- - Please do not open and re-save the CSV files in Excel before sending.
    -- Excel is what swapped days and months in the earlier extracts, and it
    -- also drops the leading zeros from account numbers.
    --
    -- THE SCHEMA, AND WHAT THE LOGIN CAN READ
    -- The catalogue queries find the schema by looking for the table ACMASTER, so
    -- no schema name has to be typed in. Every catalogue result carries the OWNER,
    -- because more than one schema can hold a table called ACMASTER (a test or
    -- archive copy, for example).
    -- - If DD_01 returns no rows, the login cannot see ACMASTER: please tell us
    -- which schema owns it.
    -- - If DD_01 lists more than one owner, please tell us which one is the live
    -- schema before going further.
    """

```

```

-- - DD_17 shows which schema the login's table names point to, and who owns
-- the three extract procedures. DD_18 tests that the login can read each
-- table directly. Being able to run a procedure does not prove that.
-----

-- SESSION SETTINGS: run these two lines first.
ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY-MM-DD HH24:MI:SS';
ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.,';
""")

# ---- DD_17 straight after DD_01
rep("""ORDER BY t.owner, t.table_name;

-- DD_02_tables.csv""",
""ORDER BY t.owner, t.table_name;

-- DD_17_login_and_synonyms.csv
-- Who is logged in, which schema that login's table names point to, any synonyms
-- standing in for the thirteen tables, and who owns the three extract procedures.
-- Table names inside a procedure are read from the procedure owner's schema.
SELECT 'LOGIN' AS item,
USER AS name,
SYS_CONTEXT('USERENV', 'CURRENT_SCHEMA') AS points_to_owner,
CAST(NULL AS VARCHAR2(128)) AS points_to_table
FROM dual
UNION ALL
SELECT 'SYNONYM OWNED BY ' || s.owner, s.synonym_name, s.table_owner, s.table_name
FROM all_synonyms s
WHERE s.synonym_name IN "" + T13 + ""
UNION ALL
SELECT 'PROCEDURE', o.object_name, o.owner, CAST(NULL AS VARCHAR2(128))
FROM all_objects o
WHERE o.object_type = 'PROCEDURE'
AND o.object_name IN ('PROC_BUILD_MAIIC_FACILITY_REGISTER', 'PROC_BUILD_MAIIC_CASHFLOW_REGISTER',
'PROC_BUILD_MAIIC_GL_INTEREST_RECON')
ORDER BY 1, 2;

-- DD_02_tables.csv""")

# ---- carry the owner through every catalogue output
rep("SELECT col.table_name,\n col.column_id AS column_no,",
"SELECT col.owner,\n col.table_name,\n col.column_id AS column_no,"
rep("ORDER BY col.table_name, col.column_id;", "ORDER BY col.owner, col.table_name, col.column_id;")
rep("SELECT c.table_name,\n c.constraint_name,", "SELECT c.owner,\n c.table_name,\n c.constraint_name,"
rep("ORDER BY c.table_name, c.constraint_type, c.constraint_name, cc.position;",
"ORDER BY c.owner, c.table_name, c.constraint_type, c.constraint_name, cc.position;")
rep("SELECT i.table_name,\n i.index_name,", "SELECT i.table_owner AS owner,\n i.table_name,\n i.index_name,"
rep("ORDER BY i.table_name, i.index_name, ic.column_position;", "ORDER BY i.table_owner, i.table_name, i.index_name, ic.column_position;")
rep("SELECT col.table_name,\n col.column_name,\n col.data_type,\n col.data_length,\n t.num_rows",
"SELECT col.owner,\n col.table_name,\n col.column_name,\n col.data_type,\n col.data_length,\n t.num_rows")
rep("ORDER BY col.column_name, col.table_name;", "ORDER BY col.column_name, col.owner, col.table_name;")
rep("SELECT t.table_name,\n t.num_rows,\n c.comments AS table_description\nFROM all_tables t",
"SELECT t.owner,\n t.table_name,\n t.num_rows,\n c.comments AS table_description\nFROM all_tables t")
rep("ORDER BY t.table_name;\n\n-- DD_08", "ORDER BY t.owner, t.table_name;\n\n-- DD_08")
rep("SELECT d.type AS object_type,\n d.name AS object_name,",
"SELECT d.referenced_owner AS owner,\n d.owner AS object_owner,\n d.type AS object_type,\n d.name AS object_name,"
rep("ORDER BY d.referenced_name, d.type, d.name;", "ORDER BY d.referenced_owner, d.referenced_name, d.type, d.name;")

# ---- DD_18 and DD_19 after DD_15
T12 = ["ACMASTER", "ACLOANMASTER", "CUSTOMER_MAST", "SCHEME_MASTER", "GLMASTER", "CUMVOUCH", "ACCOUNT_BALANCE",
"FIXLOAN_INST_CHART", "MULTI_INST_DETAILS", "LOAN_DISBURSMENT_SCHEDULE", "LOAN_RESCHEDULE_DETAILS",
"LOAN_WRITEOFF_MASTER", "LOAN_WRITEOFF_TRANS"]
access = "\n".join(
"SELECT {<27} AS table_name, COUNT(*) AS rows_read FROM {<25} WHERE ROWNUM <= 1;".format("" + t + "", t.lower()) for t
in T12)
rep("""SELECT 'SCHEME_MASTER', COUNT(*) FROM scheme_master;
""",
""SELECT 'SCHEME_MASTER', COUNT(*) FROM scheme_master;

-- DD_18_read_access.csv
-- Tests that this login can read each table directly. Run the lines one at a
-- time. Each should return one row. Please note any line that fails with
-- "table or view does not exist": that table cannot be read by this login, and
-- the queries that use it will fail too. (LOAN_WRITEOFF_TRANS may simply not exist.)
"" + access + ""

-- DD_19_date_time_check.csv
-- Whether the dates the extracts filter on carry a time of day. The extracts
-- test "between the start and end date" and "on or before the as-at date". If a

```

```
-- date carries a time, a posting made during the last day can be left out.
-- A zero in ROWS_WITH_TIME_OF_DAY means the date tests are safe for that column.
SELECT 'CUMVOUCH.TRANSACTION_DATE' AS date_column,
COUNT(*) AS rows_checked,
SUM(CASE WHEN transaction_date <> TRUNC(transaction_date) THEN 1 ELSE 0 END) AS rows_with_time_of_day
FROM cumvouch
UNION ALL
SELECT 'ACCOUNT_BALANCE.TRANSACTION_DATE', COUNT(*),
SUM(CASE WHEN transaction_date <> TRUNC(transaction_date) THEN 1 ELSE 0 END)
FROM account_balance
UNION ALL
SELECT 'FIXLOAN_INST_CHART.INSTALLMENT_DATE', COUNT(*),
SUM(CASE WHEN installment_date <> TRUNC(installment_date) THEN 1 ELSE 0 END)
FROM fixloan_inst_chart
UNION ALL
SELECT 'LOAN_WRITEOFF_MASTER.WRITEOFF_DATE', COUNT(*),
SUM(CASE WHEN writeoff_date <> TRUNC(writeoff_date) THEN 1 ELSE 0 END)
FROM loan_writeoff_master;
""")
rep("-- End of file. Twenty-four result sets in all: DD_01 to DD_15, and DD_16a to DD_16i.",
"-- End of file. Twenty-seven queries in all: DD_01 to DD_15, DD_16a to DD_16i, and DD_17 to DD_19.")
open(p, "w", encoding="utf-8", newline="\r\n").write(s)
print("sql pack patched; DD markers:", s.count("\n-- DD_"))
```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\patch_sql.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Move the review file into the application repository and apply the corrections to the SQL query pack

```
--- command ---
mkdir -p /c/xampp/htdocs/MAIIC-IFRS9/docs/reviews && mv "/c/xampp/htdocs/Stress-Testing-App/MAIIC-Claude-Review-2026-10-05.md" /
c/xampp/htdocs/MAIIC-IFRS9/docs/reviews/ && ls -la /c/xampp/htdocs/MAIIC-IFRS9/docs/reviews/ | tail -2; ls /c/xampp/htdocs/
Stress-Testing-App/MAIIC-Claude-Review-2026-10-05.md 2>&1 | head -1; cd "/c/Users/wadza/AppData/Local/Temp/claude/
c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && cp "/c/Users/wadza/OneDrive/2026/
Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/MAIIC_EBanker_Data_Dictionary_Queries.sql"
sql_v1_backup.sql && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python patch_sql.py
```

Result of Bash

```
drwxr-xr-x 1 wadza 197609 0 Oct 5 17:38 ..
-rw-r--r-- 1 wadza 197609 5914 Oct 5 17:24 MAIIC-Claude-Review-2026-10-05.md
ls: cannot access '/c/xampp/htdocs/Stress-Testing-App/MAIIC-Claude-Review-2026-10-05.md': No such file or directory
sql pack patched; DD markers: 27
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\runorder.py
--- content ---
"""Build the run-order SQL file from MAIIC_EBanker_Data_Dictionary_Queries.sql (version 2, 27 queries)."""
import re

D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC"
src = open(D + r"\MAIIC_EBanker_Data_Dictionary_Queries.sql", encoding="utf-8").read().replace("\r\n", "\n")
blocks = {}
cur = None
for line in src.split("\n"):
    m = re.match(r"^-- (DD_\d+[a-z]?)(\S+)\.csv\s*$", line)
    if m:
        cur = m.group(1); blocks[cur] = dict(file=f"{m.group(1)}_{m.group(2)}.csv", lines=[]); continue
    if line.startswith("--=====") or line.startswith("-----"):
        cur = None; continue
    if cur:
        blocks[cur]["lines"].append(line)

sql = {}
for k, b in blocks.items():
    body = [l for l in b["lines"] if not l.startswith("--")]
    sql[k] = "\n".join(body).strip()
    assert sql[k].upper().startswith("SELECT") and sql[k].endswith(";"), k
assert len(sql) == 27, len(sql)

ORDER = [
    ("DD_01", "Gate check. Confirms this login can see ACMaster and the other tables the extracts read, and shows which schema owns them."),
    ("If it returns no rows, stop: the catalogue queries below will also return nothing, and we need the schema owner's name. If it lists MORE THAN ONE owner, tell us which is the live schema before going on."),
    ("DD_17", "Shows who is logged in, which schema that login's table names point to, any synonyms standing in for the tables, and who owns the three extract procedures."),
    ("This settles which schema the later queries, and the procedures themselves, actually read."),
    ("DD_18", "Tests that this login can read each table directly. Being able to run a procedure does not prove that."),
    ("Run the thirteen lines one at a time. Note any that fail. LOAN_WRITEOFF_TRANS may simply not exist."),
    ("DD_10", "Finds Interest Policy. Returns every setting on the MAIIC and FINES schemes, which is where the manual says Interest Policy, Floating Flag, the interest calculation base, the instalment base and the EMI type are held."),
    ("Look for a policy column with values such as P, F or M, and for a scheme code besides SCHEME_MST_ID. Compare FLOATING_FLAG with the Fixed/Variable labels in Extract A."),
    ("DD_03", "The data dictionary itself: every column of every table. Needed before any amended extract can be written, and it catches Interest Policy if it is held per account and not per scheme."),
    ("Large result. Export it and carry on; we search it afterwards."),
    ("DD_12", "Transaction type codes and the sign of their amounts. Settles the direction of every row in Extract B and what the six interest codes in Extract C are."),
    ("For each code, are the amounts all positive, all negative or mixed? Codes 301, 302, 303, 305, 120 and 308 to 311 matter most."),
    ("DD_13", "Shows every GL code the loan accounts post to. The extracts read only the loan's own GL, so this is how we find where fees, penalties and suspended interest go, and whether interest is posted anywhere the extracts do not look."),
    ("Look for postings on the fee income codes 4871, 4872 and 4873 and on the interest income codes 4215 to 4219."),
    ("DD_15", "Row counts for the small loan tables. A zero on LOAN_RESCCHEDULE_DETAILS explains the blank restructure date on every row of Extract A."),
    ("If LOAN_WRITEOFF_TRANS does not exist, delete that line and run again."),
    ("DD_11", "Account status codes in use. Gives the counts behind the undocumented H and F codes."),
    ("We still need the meaning of each code from the vendor."),
    ("DD_09", "Every scheme with its account count. Proves whether the MAIIC% / FINES% name filter leaves any loan scheme out of the extracts."),
    ("Any scheme marked N with loan accounts on it is outside the extracts today."),
    ("DD_19", "Checks whether the dates the extracts filter on carry a time of day. If they do, postings made during the last day of a period can be left out."),
    ("A zero in ROWS_WITH_TIME_OF_DAY means the date tests are safe for that column."),
    ("DD_16f", "Sample loan (Ebenezer Midian): the full ledger on every GL code. Shows the real posting pattern, the signs, and any fee postings for one loan we already know well."),
    ("We expect five disbursement postings totalling 314,900,900."),
    ("DD_16d", "Sample loan: the stored instalment chart. Shows whether it has a row at the start date, whether it carries a balance column, and how the amount recovered is held against each instalment."),
    ("DD_16c", "Sample loan: the disbursement schedule table. Extract A lists five entries dated 15 July 2025; the ledger shows the same amounts paid out on 8 August 2025."),
    ("DD_16a", "Sample loan: the account master row, every column. Shows what else is held per account (rate, status, any policy or margin fields)."),
    ("DD_16b", "Sample loan: the loan master row, every column. Moratorium fields, sanction details and any link to the origination system."),
    ("DD_16e", "Sample loan: the instalment plan rows (repayment frequency)."),
    ("DD_16g", "Sample loan: the balance history. Shows which balances are held besides principal, and gives an independent check on the running balance in Extract B."),
    ("DD_16h", "Sample loan: reschedule rows, if any."),
    ("DD_16i", "Sample loan: write-off rows, if any."),
    ("DD_06", "Finds every table that carries the loan account key. This is how we discover tables nobody has mentioned: fees, charges, rate history."),
    ("DD_07", "Finds tables by subject from their names (rate, PLR, charge, fee, audit, application, IFRS)."),
    ("DD_14", "Remaining code lists: repayment frequency and moratorium units."),
    ("DD_04", "Declared keys and foreign keys: how the tables join."),
    ("Vendor systems often declare few keys; a short result is normal."),
    ("DD_05", "Indexes. Where keys are not declared, the unique indexes show what identifies a row."),
    ("DD_02", "The full table list with row counts."),
    ("DD_08", "Existing views and procedures that read the loan tables, names only. Shows which standard reports sit on the same data."),
]
assert sorted(k for k, _ in ORDER) == sorted(sql), set(sql) ^ {k for k, _ in ORDER}
out = ["-----"]
```

```
-- MAIIC E-Banker: data dictionary queries IN RUN ORDER
-- Prepared by Duplex Institute, 5 October 2026
-- Version 2, revised the same day after an independent review.
--
-- The same 27 read-only queries as MAIIC_EBanker_Data_Dictionary_Queries.sql,
-- re-ordered so that the ones that unblock the most come first. Nothing here
-- creates, changes or deletes anything. These queries were written without
-- access to the database and have not yet been run.
--
-- Run as the user that runs PROC_BUILD_MAIIC_FACILITY_REGISTER.
-- Run the two session settings first, then one query at a time.
--
-- HOW THE FILES SHOULD COME BACK (the export contract)
-- - CSV, comma separated, UTF-8, one header row, saved under the name shown.
-- - Dates as YYYY-MM-DD; dates with a time as YYYY-MM-DD HH:MM:SS (24 hour).
-- - Account numbers exactly as stored, as text, with their leading zeros.
-- - Amounts as plain numbers: dot for the decimal point, no thousands
--   separators, minus sign kept.
-- - Do not open and re-save the CSV in Excel. Excel swapped days and months
--   in the earlier extracts and drops leading zeros from account numbers.
--
-- TIER 1 (runs 1 to 11)  answers the open questions on Extracts A, B and C
-- TIER 2 (runs 12 to 20) one sample loan, every column
-- TIER 3 (runs 21 to 27) the rest of the dictionary
-----
```

```
-- RUN 0 | SESSION SETTINGS: run these two lines first. They change only how
--          this session displays dates and numbers, nothing in the database.
ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY-MM-DD HH24:MI:SS';
ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.,';
"""]
```

```
def wrap(prefix, text, width=76):
    words, lines, cur = text.split(), [], ""
    for w in words:
        if len(cur) + len(w) + 1 > width:
            lines.append(cur); cur = w
        else:
            cur = (cur + " " + w).strip()
    lines.append(cur)
    pad = " " * len(prefix)
    return "\n".join(("-- " + (prefix if i == 0 else pad) + l) for i, l in enumerate(lines))
```

```
TIERS = {1: "TIER 1. ANSWERS THE OPEN QUESTIONS ON EXTRACTS A, B AND C",
         12: "TIER 2. ONE SAMPLE LOAN, EVERY COLUMN (Ebenezer Midian, 000104430000084)",
         21: "TIER 3. THE REST OF THE DICTIONARY"}
for i, (k, why, look) in enumerate(ORDER, 1):
    if i in TIERS:
        out.append("\n--" + "=" * 78 + "\n-- " + TIERS[i] + "\n--" + "=" * 78 + "\n")
        out.append(f"-- RUN {i} of {len(ORDER)} | save as {blocks[k]['file']}")
        out.append(wrap("WHY: ", why))
        if look:
            out.append(wrap("NOTE: ", look))
        out.append(sql[k] + "\n\n")
open(D + r"\RUN ORDER - data dictionary queries by priority.sql", "w", encoding="utf-8", newline="\r\n").write("\n".join(out))
print("written; queries:", len(ORDER), "| run numbers:", {k: i for i, (k, _, _) in enumerate(ORDER, 1)})
```

Result of Write

The file C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\runorder.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Regenerate the run-order SQL file and check the new queries

```
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python runorder.py && F="/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC" && grep -c "^-- RUN " "$F/RUN ORDER - data dictionary queries by priority.sql" && sed -n '/RUN 2 of 27/,/RUN 4 of 27/p' "$F/RUN ORDER - data dictionary queries by priority.sql" | cut -c1-150 | head -50 && grep -n "col.owner\|i.table_owner \|d.referenced_owner \|ALTER SESSION" "$F/MAIIC_EBanker_Data_Dictionary_Queries.sql" | head -12
```

Result of Bash

```
written; queries: 27 | run numbers: {'DD_01': 1, 'DD_17': 2, 'DD_18': 3, 'DD_10': 4, 'DD_03': 5, 'DD_12': 6, 'DD_13': 7,
'DD_15': 8, 'DD_11': 9, 'DD_09': 10, 'DD_19': 11, 'DD_16f': 12, 'DD_16d': 13, 'DD_16c': 14, 'DD_16a': 15, 'DD_16b': 16,
'DD_16e': 17, 'DD_16g': 18, 'DD_16h': 19, 'DD_16i': 20, 'DD_06': 21, 'DD_07': 22, 'DD_14': 23, 'DD_04': 24, 'DD_05': 25,
'DD_02': 26, 'DD_08': 27}
28
-- RUN 2 of 27 | save as DD_17_login_and_synonyms.csv
-- WHY: Shows who is logged in, which schema that login's table names point to, any
--       synonyms standing in for the tables, and who owns the three extract
--       procedures.
-- NOTE: This settles which schema the later queries, and the procedures themselves,
--       actually read.
SELECT 'LOGIN' AS item,
       USER AS name,
       SYS_CONTEXT('USERENV', 'CURRENT_SCHEMA') AS points_to_owner,
       CAST(NULL AS VARCHAR2(128)) AS points_to_table
FROM dual
UNION ALL
SELECT 'SYNONYM OWNED BY ' || s.owner, s.synonym_name, s.table_owner, s.table_name
FROM all_synonyms s
WHERE s.synonym_name IN ('ACMASTER', 'ACLOANMASTER', 'CUSTOMER_MAST', 'SCHEME_MASTER', 'GLMASTER',
                        'CUMVOUCH', 'ACCOUNT_BALANCE', 'FIXLOAN_INST_CHART', 'MULTI_INST_DETAILS',
                        'LOAN_DISBURSMENT_SCHEDULE', 'LOAN_RESCHEDULE_DETAILS',
                        'LOAN_WRITEOFF_MASTER', 'LOAN_WRITEOFF_TRANS')

UNION ALL
SELECT 'PROCEDURE', o.object_name, o.owner, CAST(NULL AS VARCHAR2(128))
FROM all_objects o
WHERE o.object_type = 'PROCEDURE'
AND o.object_name IN ('PROC_BUILD_MAIIC_FACILITY_REGISTER', 'PROC_BUILD_MAIIC_CASHFLOW_REGISTER',
                    'PROC_BUILD_MAIIC_GL_INTEREST_RECON')

ORDER BY 1, 2;

-- RUN 3 of 27 | save as DD_18_read_access.csv
-- WHY: Tests that this login can read each table directly. Being able to run a
--       procedure does not prove that.
-- NOTE: Run the thirteen lines one at a time. Note any that fail.
--       LOAN_WRITEOFF_TRANS may simply not exist.
SELECT 'ACMASTER' AS table_name, COUNT(*) AS rows_read FROM acmaster WHERE ROWNUM <= 1;
SELECT 'ACLOANMASTER' AS table_name, COUNT(*) AS rows_read FROM acloanmaster WHERE ROWNUM <= 1;
SELECT 'CUSTOMER_MAST' AS table_name, COUNT(*) AS rows_read FROM customer_mast WHERE ROWNUM <= 1;
SELECT 'SCHEME_MASTER' AS table_name, COUNT(*) AS rows_read FROM scheme_master WHERE ROWNUM <= 1;
SELECT 'GLMASTER' AS table_name, COUNT(*) AS rows_read FROM glmaster WHERE ROWNUM <= 1;
SELECT 'CUMVOUCH' AS table_name, COUNT(*) AS rows_read FROM cumvouch WHERE ROWNUM <= 1;
SELECT 'ACCOUNT_BALANCE' AS table_name, COUNT(*) AS rows_read FROM account_balance WHERE ROWNUM <= 1;
SELECT 'FIXLOAN_INST_CHART' AS table_name, COUNT(*) AS rows_read FROM fixloan_inst_chart WHERE ROWNUM <= 1;
SELECT 'MULTI_INST_DETAILS' AS table_name, COUNT(*) AS rows_read FROM multi_inst_details WHERE ROWNUM <= 1;
SELECT 'LOAN_DISBURSMENT_SCHEDULE' AS table_name, COUNT(*) AS rows_read FROM loan_disbursement_schedule WHERE ROWNUM <= 1;
SELECT 'LOAN_RESCHEDULE_DETAILS' AS table_name, COUNT(*) AS rows_read FROM loan_reschedule_details WHERE ROWNUM <= 1;
SELECT 'LOAN_WRITEOFF_MASTER' AS table_name, COUNT(*) AS rows_read FROM loan_writeoff_master WHERE ROWNUM <= 1;
SELECT 'LOAN_WRITEOFF_TRANS' AS table_name, COUNT(*) AS rows_read FROM loan_writeoff_trans WHERE ROWNUM <= 1;

-- RUN 4 of 27 | save as DD_10_scheme_settings.csv
51:ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY-MM-DD HH24:MI:SS';
52:ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.,';
116:SELECT col.owner,
130:      ON t.owner = col.owner AND t.table_name = col.table_name
132:      ON cc.owner = col.owner AND cc.table_name = col.table_name AND cc.column_name = col.column_name
133:WHERE col.owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
134:ORDER BY col.owner, col.table_name, col.column_id;
164:SELECT i.table_owner AS owner,
181:SELECT col.owner,
189:      ON t.owner = col.owner AND t.table_name = col.table_name
190:WHERE col.owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
194:ORDER BY col.column_name, col.owner, col.table_name;
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\maaic_report.py
--- content ---
"""Plain-language report on the three MAIIC raw query scripts (PDF, reportlab). Version 2, after the Codex review."""
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.enums import TA_LEFT
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import (BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Table, TableStyle,
                                PageBreak, KeepTogether, Image, ListFlowable, ListItem, CondPageBreak)

F = r"C:\Windows\Fonts"
pdfmetrics.registerFont(TTFont("Seg", F + r"\segoeui.ttf"))
pdfmetrics.registerFont(TTFont("Seg-B", F + r"\segoeuib.ttf"))
pdfmetrics.registerFont(TTFont("Seg-I", F + r"\segoeuii.ttf"))
pdfmetrics.registerFont(TTFont("Seg-BI", F + r"\segoeiz.ttf"))
pdfmetrics.registerFontFamily("Seg", normal="Seg", bold="Seg-B", italic="Seg-I", boldItalic="Seg-BI")
pdfmetrics.registerFont(TTFont("Mono", F + r"\consola.ttf"))

NAVY = colors.HexColor("#1B2A41"); ORANGE = colors.HexColor("#E07B00"); GREY = colors.HexColor("#5B6470")
LIGHT = colors.HexColor("#F4F6F9"); LINE = colors.HexColor("#C9D1DB"); AMBER = colors.HexColor("#FFF4E0")

S = {
    "title": ParagraphStyle("title", fontName="Seg-B", fontSize=24, leading=29, textColor=NAVY, spaceAfter=4),
    "sub": ParagraphStyle("sub", fontName="Seg", fontSize=13, leading=17, textColor=GREY, spaceAfter=10),
    "meta": ParagraphStyle("meta", fontName="Seg", fontSize=9.5, leading=13, textColor=GREY),
    "h1": ParagraphStyle("h1", fontName="Seg-B", fontSize=15, leading=19, textColor=NAVY, spaceBefore=14, spaceAfter=6,
keepWithNext=1),
    "h2": ParagraphStyle("h2", fontName="Seg-B", fontSize=11.5, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4,
keepWithNext=1),
    "h2t": ParagraphStyle("h2t", fontName="Seg-B", fontSize=11.5, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4),
    "p": ParagraphStyle("p", fontName="Seg", fontSize=10, leading=14.2, textColor=colors.black, spaceAfter=6,
alignment=TA_LEFT),
    "li": ParagraphStyle("li", fontName="Seg", fontSize=10, leading=14, textColor=colors.black),
    "cell": ParagraphStyle("cell", fontName="Seg", fontSize=8.6, leading=11.4),
    "cellb": ParagraphStyle("cellb", fontName="Seg-B", fontSize=8.6, leading=11.4),
    "head": ParagraphStyle("head", fontName="Seg-B", fontSize=8.8, leading=11.4, textColor=colors.white),
    "box": ParagraphStyle("box", fontName="Seg", fontSize=10, leading=14.2, textColor=colors.black, spaceAfter=3),
    "note": ParagraphStyle("note", fontName="Seg-I", fontSize=8.8, leading=12, textColor=GREY, spaceAfter=6),
}
W = A4[0] - 36 * mm
story = []

def H1(t): story.append(Paragraph(t, S["h1"]))
def H2(t): story.append(Paragraph(t, S["h2"]))
def P(t): story.append(Paragraph(t, S["p"]))
def NOTE(t): story.append(Paragraph(t, S["note"]))
def GAP(h=4): story.append(Spacer(1, h))

def H2T(t):
    """Heading that sits directly above a long table: break first if little room is left."""
    story.append(CondPageBreak(62 * mm)); story.append(Paragraph(t, S["h2t"]))

def BUL(items, numbered=False):
    story.append(ListFlowable([ListItem(Paragraph(i, S["li"]), leftIndent=14, spaceAfter=3) for i in items],
bulletType="1" if numbered else "bullet", bulletFontName="Seg", bulletFontSize=9,
start=None if numbered else "*", leftIndent=14, bulletColor=ORANGE if not numbered else NAVY))
    GAP(4)

def TAB(header, rows, widths, bold_first=False, keep=False):
    data = [[Paragraph(h, S["head"]) for h in header]]
    for r in rows:
        data.append([Paragraph(str(c), S["cellb"]) if (bold_first and i == 0) else S["cell"] for i, c in enumerate(r)])
    t = Table(data, colWidths=[W * x for x in widths], repeatRows=1)
    st = [("BACKGROUND", (0, 0), (-1, 0), NAVY), ("VALIGN", (0, 0), (-1, -1), "TOP"),
("LEFTPADDING", (0, 0), (-1, -1), 5), ("RIGHTPADDING", (0, 0), (-1, -1), 5),
("TOPPADDING", (0, 0), (-1, -1), 4), ("BOTTOMPADDING", (0, 0), (-1, -1), 5),
("LINEBELOW", (0, 0), (-1, -1), 0.4, LINE), ("BOX", (0, 0), (-1, -1), 0.6, LINE)]
    for i in range(1, len(data)):
        if i % 2 == 0:
            st.append(("BACKGROUND", (0, i), (-1, i), LIGHT))
    t.setStyle(TableStyle(st))
    story.append(KeepTogether([t]) if keep else t)
    GAP(8)

def BOX(paras, colour=AMBER):
    t = Table([[[Paragraph(x, S["box"]) for x in paras]]], colWidths=[W])
    t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, -1), colour), ("LINEBEFORE", (0, 0), (0, -1), 3, ORANGE),
("LEFTPADDING", (0, 0), (-1, -1), 10), ("RIGHTPADDING", (0, 0), (-1, -1), 10),
("TOPPADDING", (0, 0), (-1, -1), 8), ("BOTTOMPADDING", (0, 0), (-1, -1), 6)]))
    story.append(t); GAP(8)
```

```

def M(t):
    return f'<font name="Mono" size="8">{t}</font>'

# ----- cover block
LOGO = r"C:\Users\wadza\OneDrive\2026\Tenders\PFA\Excel & Data Analysis Training\Basic Excel Course\3 Source Files\img\
dupleix-logo.png"
try:
    im = Image(LOGO); r = im.imageHeight / im.imageWidth; im.drawWidth = 42 * mm; im.drawHeight = 42 * mm * r; im.hAlign =
"LEFT"
    story.append(im); GAP(10)
except Exception:
    pass
story.append(Paragraph("MAIIC E-Banker extract scripts", S["title"]))
story.append(Paragraph("What the three scripts do, where they fall short, and which queries to run next", S["sub"]))
story.append(Paragraph("Prepared by Dupleix Institute for MAIIC Finance and ICT · 5 October 2026 · <b>Version 2</b>",
S["meta"]))
story.append(Paragraph("Version 2 was revised on the same day after an independent review of the first version. The review's
comments, "
                        "our response to each, and what changed are in Appendix E.", S["meta"]))
story.append(Paragraph("Based on: the three script files received on 5 October 2026; Extracts A and B as delivered on 3 August
2026; "
                        "Extract C as re-run and delivered on 26 August 2026; the E-Banker user manual.", S["meta"]))
GAP(8)
BOX(["<b>What this report is, and is not.</b> It is an analysis of three extract scripts and of the files they produced. "
     "We read the scripts and tested them against those files. We have not seen the database.",
     "It is <b>not</b> a completed reconciliation of interest income, and it does <b>not</b> validate any contractual cash flow
"
     "schedule. The 27 queries it asks for were written without access to the database and have not been run. "
     "Section 2 separates what we tested from what we infer and what is still waiting on MAIIC or the vendor."])

# ----- 1 summary
H1("1. Summary")
P("MAIIC has sent us the three database scripts that produce Extract A (the facility register), Extract B (the cash flow "
  "register) and Extract C (the interest posted on each loan). We read each script line by line and then tested what it "
  "implies against the extract files we already hold.")
P("The scripts are tidy, and they prove that the data we need is held in E-Banker. They also explain, for the first time, "
  "most of the puzzles we raised in September. But each script makes choices that hide or change information the effective "
  "interest rate (EIR) engine depends on, and some of the fields we need are not read by any of the three.")
H2("Three findings that affect figures already in use")
BUL([
  "<b>The Extract C file counts 2025 three times.</b> The script keeps every earlier run, and the file we received holds
three "
  "runs. Added up as delivered, 2025 interest is MWK 7,663,172,090. The figure from the latest run alone is MWK 2,554,390,
697.",
  "<b>The Fixed and Variable labels in Extract A appear to be the wrong way round.</b> The script labels the system's code F
as "
  "\"Fixed\" and its code I as \"Variable\", while the manual gives F as Floating and I as Fixed. This would explain why the "
  "labels contradicted what the loan books show.",
  "<b>The \"Interest\" rows in Extract B are interest charged to the loan, not cash received.</b> 987 of the 994 interest
rows "
  "we could test raise the loan balance, and they are the same postings Extract C reports. They must not be treated as "
  "money received.",
], numbered=True)
H2("What we are asking for")
P("Two short rounds of read-only queries, before any extract is rewritten:")
BUL([
  "<b>First, the data dictionary queries.</b> These list the tables and columns E-Banker holds. We need them first because
the "
  "fields still missing, starting with Interest Policy, do not appear in any of the three scripts. We do not yet know what "
  "they are called or which table they sit in, so an amended extract cannot be written.",
  "<b>Second, the question queries.</b> These return codes, counts, checks and one sample loan. They answer specific open "
  "questions: which way each transaction type moves the balance, where the fees are posted, what the status codes are, "
  "whether dates carry a time of day, and why the restructure date is blank.",
  "<b>Then the amended extracts,</b> written once, against the real structure.",
], numbered=True)
P("All 27 queries are in the file <b>RUN ORDER - data dictionary queries by priority.sql</b>, in the order to run them. "
  "Runs 1 to 7 are enough for us to start drafting the amended extracts.")
P("This version also sets out suggested changes to the three scripts (section 12) and to our own engine's code (section 13).")

# ----- 2 how sure
H1("2. How sure we are")
P("Not every statement in this report rests on the same footing. This table separates the three kinds.")
TAB(["Footing", "Statements"], [
  ["<b>Tested on the delivered files.</b> Each can be re-run from the scripts in the Build files folder; the reviewer re-ran
the main ones and got the same figures.",
   "The Extract C file holds three runs with identical 2025 amounts.<br/>"
   "The rate label by product: MAIIC Agricultural and Industrial all \"Fixed\", FInES Agricultural and Industrial all \"
Variable\".<br/>"
   "Rate, status, sanctioned amount and tenor are identical in both Extract A snapshots.<br/>"
   "Extract B has no negative amounts, 539 estimated rows, and no balance on its scheduled rows.<br/>"
   "Extract B interest and Extract C agree on 101 of 107 accounts for 2025; the difference is MWK 85,166,683.31 on six
accounts.<br/>"
   "987 of 994 interest rows raise the running balance.<br/>"
   "Extract B's closing balance agrees with Extract A's year-end balance on 110 of 111 accounts.<br/>"
   "Ebenezer Midian: 15 July 2025 in the disbursement list, 8 August 2025 in the ledger."],
  ["<b>Inferred.</b> Reasoned from the scripts and the files, but not confirmed against the database or by the vendor.",
   "That the rate labels are inverted. This rests on the manual's code list (F Floating, I Fixed) and on loan book behaviour;
we have not seen the raw flag values.<br/>"
   "That interest rows are charges. This rests on how the balance moves; we have not seen the raw signed postings.<br/>"
   "That the instalment chart begins with a row at the loan's start date.<br/>"]
])

```

```

    "That the reschedule table is empty.<br/>"
    "That the dates were scrambled by Excel on the way out.<br/>"
    "What the \"Other/Adjustment\" rows are.",
    ["<b>Waiting on MAIIC or the vendor.</b>",
    "Where Interest Policy is held.<br/>"
    "What transaction types 120 and 308 to 311 mean, and what status codes H and F mean.<br/>"
    "Where fees are posted against a loan.<br/>"
    "Whether posting dates carry a time of day.<br/>"
    "Which schema the table names point to, and whether the login can read the tables directly.<br/>"
    "Why the repayment frequency differs between the two snapshots.<br/>"
    "Why ten active accounts show no interest.<br/>"
    "The reconciliation of Extract C to the interest income codes in the trial balance."],
], [0.30, 0.70])

# ----- 3 what we received
H1("3. What we received")
P("Each file is a stored procedure: a routine saved inside the database that fills a results table when someone runs it. "
  "The results table is then copied out to Excel and sent to us.")
TAB(["File", "Builds", "What it produces"], [
  ["FACILITY_REGISTER.txt", "Extract A", "One row for each loan account as at a chosen date: who the customer is, the terms
of the loan, and the balance."],
  ["CASHFLOW_REGISTER.txt", "Extract B", "One row for each posting on a loan in a date range, with a running balance,
followed by write-offs and by scheduled instalments not yet fully recovered."],
  ["INTEREST_RECON.txt", "Extract C", "The interest posted on each loan, totalled for each month and for each year."],
], [0.26, 0.12, 0.62], bold_first=True)
P("Two things are missing from the files. The definitions of the results tables are not included, and neither is the step "
  "that copies the results out to Excel. The second one matters: it is where the dates were scrambled (section 7).")
H2("The E-Banker tables the scripts read")
P("This is the first time we have seen E-Banker's real table names. Between them the three scripts read thirteen tables.")
TAB(["Table", "What it holds, in plain language", "What the scripts take from it"], [
  ["ACMASTER", "The account master: one row for each account", "Account number, customer, GL code, interest rate, opening
date, status, currency, scheme"],
  ["ACLOANMASTER", "The loan details for each account", "Sanctioned amount, tenor, expiry date, moratorium periods"],
  ["CUSTOMER_MAST", "The customer master", "Customer name"],
  ["SCHEME_MASTER", "The scheme (the product variant) each account is opened on", "Scheme name, Floating Flag, compound or
simple, days in the year"],
  ["GLMASTER", "The list of GL codes", "GL title"],
  ["CUMVOUCH", "The ledger: every posting", "Date, transaction type code, amount, GL code"],
  ["ACCOUNT_BALANCE", "Balance history by date", "Principal balance"],
  ["FIXLOAN_INST_CHART", "The instalment chart (the EMI chart)", "Instalment date, amount, principal part, interest part,
amount recovered"],
  ["MULTI_INST_DETAILS", "The instalment plan", "Repayment frequency"],
  ["LOAN_DISBURSEMENT_SCHEDULE", "The disbursement schedule", "Date and amount"],
  ["LOAN_RESCHEDULE_DETAILS", "Reschedules (restructures)", "Latest reschedule date"],
  ["LOAN_WRITEOFF_MASTER", "The write-off register", "Write-off date and amount"],
  ["LOAN_WRITEOFF_TRANS", "Write-off postings", "Named in a comment only; not read"],
], [0.27, 0.36, 0.37], bold_first=True)
H2("Which loans are included")
P("All three scripts pick loans the same way: any account whose scheme name begins with \"MAIIC\" or \"FINES\". A loan on a "
  "scheme named differently would be left out, and the scripts cannot tell us whether any exist. The same rule also brings in "
  "accounts that are not loans: Extract A includes five Quasi Equity Investment accounts and one NASCOMEX Limited account.")
# ----- 4 Extract A
H1("4. Extract A: the facility register")
H2("What the script does")
P("For the date it is given, the script removes any earlier result for that date and builds it again. For each account it "
  "collects the loan's details from the account, loan, customer and scheme masters. It adds up the disbursement schedule, "
  "counts the rows in the instalment chart, picks up any write-off and the latest reschedule date, and looks up the most "
  "recent principal balance on or before the date.")
H2("What is good about it")
BUL([
  "The account number and sub-account number are carried through cleanly, so Extract A joins to B and C. There are no "
  "repeated rows for the same account, sub-account and date.",
  "Where a balance is missing, the script says why in a note: the loan opened after the date, it closed before the date, "
  "or no balance was found and it needs checking by hand.",
  "The day-count basis and whether interest is compound or simple come straight from the scheme, which settled our day-count
decision.",
  "It can be run again for the same date without creating duplicates.",
])
H2T("Where it falls short")
TAB(["#", "What the script does", "Why it matters", "What the delivered file shows"], [
  ["1", "<b>Labels the rate type, apparently the wrong way round.</b> It reads the scheme's Floating Flag and labels code F
as \"Fixed\" and code I as \"Variable\". The manual (pages 19 to 20) gives F as Floating and I as Fixed.",
  "The label says the opposite of how the loan behaves, so it cannot be used to decide which loans reprice.",
  "All 57 MAIIC Agricultural and Industrial accounts are labelled Fixed, and all 101 FINES Agricultural and Industrial
accounts Variable. The loan books show the reverse. The 13 Term Loans still do not fit after the labels are swapped, so the
flag is not the deciding field."],
  ["2", "<b>Leaves out Interest Policy and the scheme code.</b> It reads the scheme master but takes only four settings from
it.",
  "Interest Policy is the setting that decides whether a loan follows the Reserve Bank rate. The scheme identifier is
already in the script's hands and is simply not output.", "Not in the file."],
  ["3", "<b>Only the balance is taken as at the date.</b> The rate, status, amounts and dates are read as they stand on the
day the script is run.",
  "The file looks like a picture of the book at an earlier date, but most of it is today's picture. It cannot show what a
loan's rate was at 1 January 2025.",
  "Rate, status, sanctioned amount and tenor are identical in the two snapshots for all 181 accounts. Lake Malawi
Aquaculture's 31 December 2025 row lists a drawdown dated 28 April 2026."],
  ["4", "<b>Takes disbursements from the disbursement schedule table,</b> not from the ledger.",
  "The schedule gives the planned dates. The date money actually went out, which is when interest starts to run, is in the
ledger and can be different.",
  "Ebenezer Midian: five entries dated 15 July 2025 in Extract A; the ledger shows the same five amounts paid out on 8
August 2025. Of the 35 accounts with ledger disbursements in 2025, the totals agree on 30 and the number of entries on 28."],

```

[5", "Counts the rows of the instalment chart for the number of instalments, and takes the earliest chart date as the first repayment date.",

"The chart appears to start with a row at the loan's start date. That row is counted as an instalment and reported as the first repayment.",

"Where a chart exists, the count is the tenor in months plus one on the rows we checked. The first repayment date equals the loan start date on 63 of 128 rows. 53 accounts have no chart rows at all.",

[6", "Reads the reschedule table for the restructure date.", "If that table is empty, restructures are not being recorded in E-Banker's reschedule function, and the history lives only in Finance's spreadsheet.",

"Blank on every row.",

[7", "Translates only three status codes (Active, Closed, Dormant). Any other code is passed through as it is.",

"The meaning of the other codes still has to come from the vendor.",

"H on 21 accounts and F on 3.",

[8", "Removes the sign from the balance, and reads the principal balance only.", "An overpaid loan would look the same as one that is owed. Any interest balance held separately is not shown. The column is also headed \"opening balance\" although it is the balance at the date.", ""],

[9", "Tests the balance date as on or before the as-at date with the time of day removed.", "If balance dates carry a time of day, a balance recorded during the as-at day itself is missed and the day before is used instead. We do not yet know whether the dates carry a time.", "Cannot be tested from the file. Run 11 answers it.",

[10", "Does not limit write-offs and closures to the date.", "A write-off made after the date appears in an earlier snapshot.", ""],

[11", "Joins the moratorium period and its unit into text, and leaves out the moratorium type.", "Values arrive as \"3 M\" or \"0\", and we cannot tell a principal-only moratorium from a full one.", "Both formats appear.",

[12", "Reads no fee table at all.", "Fees are the main reason an effective rate differs from the contract rate.",

"No fee fields.",

], [0.04, 0.34, 0.30, 0.32])

H2("The repayment frequency in the two snapshots")

P("The two snapshots disagree on repayment frequency for 42 accounts. The 1 January 2025 snapshot shows Monthly on all 181 " "accounts. The 31 December 2025 snapshot shows Monthly on 139, Half-Yearly on 10, Quarterly on 4, Yearly on 3 and blank on 25.")

P("The script we were sent does not use the date to find the frequency, and it leaves an account with no instalment plan " "blank. On its own it would not produce Monthly for every account. We do not know the cause. A different version of the " "procedure, a change in the data between the two runs (they were about an hour apart), or an edit after export are all " "possible. We have asked MAIIC for the procedure's version history and a record of the two runs. Until that is settled, " "the frequency in the 1 January 2025 snapshot should not be relied on.")

----- 5 Extract B

H1("5. Extract B: the cash flow register")

H2("What the script does")

P("Each run is given a new run number. The script then gathers three kinds of row for the date range it is given:")

BUL([

"Actual rows: every ledger posting made on the loan's own GL code, with a running balance. It names each row from " "its transaction type code: 301 Disbursement, 302 Principal, 303 Interest, 305 Principal+Interest, 340 and 341 Penalty, " "900 and 901 Fee, and anything else \"Other/Adjustment\".",

"Write-off rows: taken from the write-off register.",

"Scheduled rows: rows of the instalment chart that have not yet been fully recovered.",

])

H2("What is good about it")

BUL([

"The running balance is built from the loan's ledger history on its own GL code before the date range is applied, so a " "one-year extract does not restart the balance at zero. At 31 December 2025 it agrees with the balance in Extract A, " "which comes from a different table, on 110 of 111 accounts. The exception is Zaithwa Farms (1,225,683.03 against " "3,053,316.65). That is one checkpoint, not proof. The balance is only right if the ledger history is complete, the " "opening position is in it, and every relevant posting sits on the loan's own GL code. None of those is yet confirmed.",

"Every row carries a reference back to the ledger.",

"The script is honest about its limits. It marks every estimated figure in a note and lists its own known limitations at the end.",

"Its date range already runs to 31 December 2026 by default, so later periods need no rewrite.",

])

H2T("Where it falls short")

TAB(["#", "What the script does", "Why it matters", "What the delivered file shows"], [

[1", "Removes the direction of every amount. The ledger holds signed amounts; the script outputs each one without its sign.",

"A disbursement and a repayment look the same. Direction has to be worked out from the type label or from how the balance moves.", "No negative value anywhere in 2,478 rows.",

[2", "Presents interest charged as if it were a cash flow. Type 303 appears to be the monthly interest charge added to the loan.",

"For the effective interest rate, the cash flows are the money lent and the money repaid. Interest charged is not cash. Counting it as a receipt would count the interest twice.",

"987 of the 994 interest rows we could test raise the running balance; 485 of 501 repayment rows reduce it. On Ifracon Limited, interest of 21,497,946.96 on 31 January 2025 takes the balance up to 788,531,241.57, and a repayment of 53,589,482.23 on 18 February takes it down to 734,941,759.34.",

[3", "Estimates the split of each repayment (type 305) between principal and interest with a formula: balance, times the loan's rate today, times the days since the previous posting.",

"The formula uses today's rate, not the rate at the time, and counts days from the last posting of any kind. The interest has in any case already been charged separately. The split has no meaning, and the engine does not need it: it needs the amount, the date and the direction.",

"539 rows carry the \"estimated\" note.",

[4", "Puts every other transaction type under \"Other/Adjustment\" with no sign and no type code.",

"The label covers different things that cannot be told apart. Some rows reduce interest and are netted off in Extract C. Others raise the balance and are something else. What each one is needs the signed postings and the vendor's code list.",

"31 rows totalling MKW 832,653,206. Of the 30 we could test, 18 raise the balance, 9 reduce it and 3 could not be placed. On five accounts these rows equal the B to C difference exactly; on Happie Foods only 2,018,700.44 of 9,018,700.44 does (Appendix B).",

[5", "Reads only postings on the loan's own GL code.", "Fees, penalties or suspended interest posted to other GL codes for the same loan are never seen.", "2 fee rows in 2,478.",

[6", "Includes only instalments not yet fully recovered, inside the date range, at their full original amounts.",

"These rows are neither the schedule nor the amount still owed. Fully paid instalments are left out. A part-paid instalment is shown in full, because the amount already recovered is not subtracted and not shown. They must not be used as remaining payments until the recovery against each instalment is supplied and checked.",

"No balance on any of the 775 scheduled rows. On 14 of them the principal and interest parts do not add up to the total.",

[7", "Selects postings between the start date and the end date.", "If posting dates carry a time of day, postings

made during the last day are left out. Extract C uses the same test.", "Cannot be tested from the file. Run 11 answers it."),
["8", "Builds the reference from an internal row number.", "It can be traced in the database, but it is not a voucher number a user can look up on screen. The script's own note says the voucher number field was blank.", ""],
["9", "Takes write-offs from the register only.", "There is no ledger posting to check them against, and a write-off that is also in the ledger would appear twice.", ""],
], [0.04, 0.34, 0.30, 0.32])

----- 6 Extract C

H1("6. Extract C: interest posted by loan")
H2("What the script does")
P("Each run is given a new run number, and earlier runs are kept. The script adds up ledger postings of six transaction " "types (120, 303, 308, 309, 310 and 311) made on the loan's own GL code. It reverses their sign so that interest charged " "shows as positive, and totals them for each loan for each month. It also writes a yearly total row for each loan into the " "same table.")
H2("What is good about it")
BUL([
"It keeps the signs when adding, so reversals and adjustments are netted off.",
"Each row lists the ledger references it was built from.",
"For 2025 it agrees with the interest rows of Extract B on 101 of 107 accounts. Both extracts read the same ledger table, " "so this shows they are consistent with each other. It is not independent confirmation of the income figure.",
"Within the latest run there are no repeated rows for the same loan, GL code and month.",
])
H2T("Where it falls short")
TAB(["#", "What the script does", "Why it matters", "What the delivered file shows"], [
["1", "Keeps every earlier run in the same table. The file sent to us was exported without choosing one run.",
"Any total taken from the file as delivered counts 2025 three times.",
"Three runs (see the table below). 1,012 loan-months appear three times with identical amounts."],
["2", "Writes monthly rows and yearly total rows together.", "Adding the whole column counts every amount twice.",
"All rows add up to twice the monthly rows."],
["3", "Measures one side only. It adds up interest charged to loan accounts. It does not read the interest income GL codes.",
"It is not a reconciliation, and this report does not perform one. Its total still has to be agreed to the movement on the income codes in the trial balance. Interest charged to a loan but credited to suspense would be counted here as income.", ""],
["4", "Leaves out accounts with no matching posting in the period.",
"An absent account can mean three different things: no interest was posted; interest was posted where the script does not look (another GL code, another transaction type, a scheme outside the name rule); or the account was missed. The script cannot tell these apart, so an absent account is not a verified zero.",
"61 of 181 accounts are absent: 46 closed, 10 active, 4 dormant and 1 with status H (Appendix C)."],
["5", "Uses six type codes whose meanings we have not been given.", "The balance movements suggest 303 is the monthly charge and that the others include reductions, but what each one is has to come from the vendor.", ""],
["6", "Groups by the date of posting, and selects postings between the start and end date.", "Interest posted late lands in the month it was posted, not the month it was earned. If dates carry a time of day, postings on the last day are left out.", "The catch-up postings we found in February 2026."],
], [0.04, 0.34, 0.30, 0.32])
TAB(["Run number", "Generated", "Rows", "Accounts", "Period covered", "2025 interest, MWK"], [
["7", "3 August 2026", "1,119", "107", "2025", "2,554,390,697"],
["25", "3 August 2026", "1,119", "107", "2025", "2,554,390,697"],
["41", "25 August 2026", "1,758", "120", "January 2025 to July 2026", "2,554,390,697"],
["As delivered", "", "3,996", "", "", "7,663,172,090"],
], [0.15, 0.17, 0.10, 0.12, 0.26, 0.20], keep=True)
P("Run 41 is the one to use. Its January to July 2026 total is MWK 2,739,597,510, which appears once and is not affected.")

----- 7 dates and export

H1("7. Where the dates were scrambled, and how files should come back")
P("In September we reported that dates were transposed in every extract, with the day and month swapped. The scripts hold " "proper database dates from start to finish, so the damage does not happen inside them. It happens afterwards, when the " "results are copied out. We believe Excel is the cause but have not seen the export step.")
P("\Send it as CSV\ is not enough to prevent it. A CSV opened in Excel can still have its dates re-read and its account " "numbers stripped of their leading zeros. So every file should follow the same rules:")
BUL([
"CSV, comma separated, UTF-8, with one header row.",
"Dates written year first: 2026-10-05. Dates with a time: 2026-10-05 14:30:00.",
"Account numbers exactly as stored, as text, with their leading zeros.",
"Amounts as plain numbers: a dot for the decimal point, no thousands separators, and the minus sign kept.",
"Not opened and re-saved in Excel at any point before it reaches us.",
])
P("The mixed dates in the files we hold remain a limit on the evidence in this report. We avoided tests that depend on " "dates where we could, and used the ledger's own posting order instead.")

----- 8 three findings

H1("8. The three findings, and what to do about each")
P("These are the three points from the summary, with the action each one calls for.")
TAB(["Finding", "What to do now"], [
["Extract C counts 2025 three times when the delivered file is added up.",
"Use run 41 only, and monthly rows only. We have now read our engine's import: it skips the yearly rows and does not load a repeat of the same loan, month and GL code, so the three runs would not have been loaded three times. We have not inspected the loaded data itself. Any 2025 figure taken from the file by hand should still be checked."],
["Fixed and Variable appear to be the wrong way round in Extract A.",
"Do not use the label. Our engine currently stores a loan's rate type from it, which needs changing (section 13). Until Interest Policy is extracted, rate behaviour should be taken from the monthly loan books."],
["\Interest\ rows in Extract B are charges, not receipts.",
"Our 24 September research report described these rows as cash received, and we will correct it. Our engine currently counts them as cash collected, which also needs changing (section 13). Cash out is the disbursements and cash in is the repayments."],
], [0.36, 0.64], keep=True)

----- 9 settled / open

H1("9. What the scripts settle, and what is still open")
P("The questions below are the ones we raised in September, plus those raised in review, with what the scripts tell us " "about each and the query that would close it. Run numbers refer to section 11.")
TAB(["Question", "What the scripts tell us", "Status", "Query that closes it"], [

```

["Is the instalment chart stored, or worked out on screen?", "Stored, in a table, with the principal and interest split. 53
accounts have no rows.", "Settled", "Run 13 confirms the columns"],
["Why is the number of instalments always the tenor plus one?", "It is a count of chart rows, and the chart appears to
begin with a row at the start date.", "Explained, to confirm", "Run 13"],
["Do the disbursement tranches come through?", "Partly. 58 of the 314 populated rows list two to ten entries, so our
September statement that every row had one was wrong. But the entries come from the disbursement schedule, with planned dates,
not from the ledger.", "Corrected", "Runs 12 and 14"],
["Why is the restructure date blank everywhere?", "The script reads the reschedule table, which appears to be empty.",
"Explained, to confirm", "Run 8"],
["Why is there no debit or credit direction in Extract B?", "The script strips the sign from every amount.", "Explained",
"Run 6"],
["Why are 61 accounts missing from Extract C?", "Accounts with no posting that matches the script's filters produce no row.
Whether each one is a true zero is not established.", "Partly explained", "Runs 7 and 10, then MAIIC"],
["Why were the dates scrambled?", "Not in the scripts; in the export.", "Explained, to confirm", "None needed"],
["Which scheme is each account on?", "The scheme identifier is already read by the script. It only needs to be output.",
"Available", "Run 4"],
["Where is Interest Policy held?", "No script reads it.", "Open", "Runs 4 and 5"],
["Where are fees held against a loan?", "No script reads a fee table, and both ledger scripts look at the loan's own GL
code only.", "Open", "Runs 7, 21 and 22"],
["What do transaction types 120 and 308 to 311 mean?", "The script names eight codes. These five are grouped as interest in
Extract C and as \"Other\" in Extract B.", "Open", "Run 6, then the vendor"],
["What do status codes H and F mean?", "The script passes them through without a meaning.", "Open", "Run 9, then the
vendor"],
["Is there a table of rate changes?", "No script reads one.", "Open", "Runs 21 and 22"],
["Does the scheme name rule leave any loans out?", "Cannot be told from the scripts.", "Open", "Run 10"],
["Do posting dates carry a time of day?", "The scripts' date tests would drop last-day postings if they do.", "Open", "Run
11"],
["Which schema do the table names point to, and can the login read the tables?", "Not shown by the scripts. More than one
schema may hold a table of the same name.", "Open", "Runs 1 to 3"],
["Why does the repayment frequency differ between the two snapshots?", "The script we hold does not explain it.", "Open",
"MAIIC: procedure versions and run record"],
], [0.27, 0.37, 0.12, 0.24])

# ----- 10 why dictionary first
H1("10. Why the data dictionary queries have to come first")
P("A data dictionary is the database's own list of what it holds: every table, every column in each table, the kind of data "
  "in each column, and how the tables link to one another. Oracle keeps this list automatically. Reading it does not touch "
  "any customer or transaction record.")
P("Everything we know about how E-Banker is built comes from the three scripts: thirteen table names, and only the columns "
  "those scripts happen to use. The fields we still need are, by definition, the ones the scripts do not use. The manual "
  "tells us that Interest Policy is on the Scheme Master screen, but the name on a screen is not the name of a column, and a "
  "query can only ask for a column by its real name.")
H2("Three reasons the order matters")
BUL([
  "<b>An amended extract cannot be written against a column we cannot name.</b> A query that guesses a column name fails when
  "
  "it is run. Each failed attempt costs a round trip between Dupleix, MAIIC and the vendor. The August extracts took two "
  "deliveries, three weeks apart, to reach their present state.",
  "<b>One pass answers many questions.</b> The same listing shows where Interest Policy sits, whether the instalment chart "
  "carries a balance, whether there is a fee table, whether there is a rate history table, and what links a loan back to "
  "the origination system. Asking for these one at a time is how the last two months were spent.",
  "<b>It is the lowest-risk request we can make.</b> The queries only read. The dictionary queries read Oracle's catalogue, "
  "not MAIIC's data, and they take minutes to run.",
], numbered=True)
H2("Why the question queries are still needed afterwards")
P("The dictionary gives structure, not meaning. It will tell us that a column called TRANTYPE exists and holds a number. It "
  "will not tell us what 308 means, or whether a repayment is stored as a positive or a negative amount. Vendor databases "
  "rarely carry written descriptions. So the second round asks the data itself: which codes are in use, how often, in which "
  "direction, and what a single well-understood loan looks like in every table.")
BOX(["<b>In short:</b> the dictionary tells us what can be asked for. The question queries tell us what the answers mean. "
  "Only then can the three extracts be rewritten once, correctly."])

# ----- 11 the queries
H1("11. The queries, in the order to run them")
P("The run numbers below match the file <b>RUN ORDER - data dictionary queries by priority.sql</b>. Each query in that file "
  "is headed with the reason for it and the file name to save its result under. The file begins with two session settings "
  "(run 0) that make dates and numbers display in the agreed format; they change nothing in the database.")
H2("Tier 1: answers the open questions on Extracts A, B and C")
TAB(["Run", "Query", "What it returns, in plain language", "The question it answers"], [
  ["1", "DD_01", "Whether this login can see the thirteen tables, and who owns them", "The gate check. If it returns nothing,
we need the owner's name. If it lists more than one owner, we need to know which is the live schema."],
  ["2", "DD_17", "Who is logged in, which schema that login's table names point to, and who owns the three extract
procedures", "Which schema do the queries, and the procedures themselves, actually read?"],
  ["3", "DD_18", "A one-row read of each of the thirteen tables", "Can this login read the tables directly? Being able to run
a procedure does not prove it."],
  ["4", "DD_10", "Every setting held on the MAIIC and FInES schemes", "Where is Interest Policy? What are the real Floating
Flag values? Is there a scheme code?"],
  ["5", "DD_03", "Every column of every table: the dictionary itself", "What is each field called and where does it sit?"],
  ["6", "DD_12", "Each transaction type code, how often it is used, and whether its amounts are positive or negative", "Which
way does each posting move the balance? What are the unexplained codes used for?"],
  ["7", "DD_13", "Every GL code the loan accounts post to, by transaction type", "Where do fees, penalties and suspended
interest go? Is interest posted anywhere the extracts do not look?"],
  ["8", "DD_15", "The number of rows in the smaller loan tables", "Is the reschedule table empty?"],
  ["9", "DD_11", "The account status codes in use, with counts", "How many accounts carry H and F?"],
  ["10", "DD_09", "Every scheme, with the number of accounts on it", "Does the \"MAIIC or FInES\" name rule leave any loan
scheme out?"],
  ["11", "DD_19", "How many ledger, balance, instalment and write-off dates carry a time of day", "Could postings on the last
day of a period be dropped by the extracts' date tests?"],
], [0.07, 0.10, 0.42, 0.41])
H2("Tier 2: one sample loan, every column")
P("Runs 12 to 20 return every row held for one loan in each table: the ledger on all GL codes, the instalment chart, the ")

```

```

"disbursement schedule, the account and loan masters, the instalment plan, the balance history, and any reschedule or "
"write-off. The loan is Ebenezer Midian, one of the ten sample loans. We chose it because we already know two things about "
"it: the ledger shows five amounts totalling MWK 314,900,900 paid out on 8 August 2025, and its disbursement schedule "
"shows the same five amounts dated 15 July 2025. The result can be checked the moment it arrives.")
H2("Tier 3: the rest of the dictionary")
P("Runs 21 to 27 find the tables nobody has mentioned (any table carrying the loan account number, and tables named for "
"rates, fees, charges, audit or applications), and list the remaining code values, the keys and indexes that show how "
"tables join, the full table list, and the existing reports that already read the loan tables. None of these holds up the "
"amended extracts.")
H2("Practical notes for whoever runs them")
BUL([
    "Every query is read-only. Nothing is created, changed or deleted.",
    "Run them with the same login that runs the three extract procedures, and run the two session settings first.",
    "Every catalogue result carries the owner of each table, so that results from two schemas cannot be mixed up.",
    "Run one query at a time and return each result under the rules in section 7.",
    "Run 8 counts a table (LOAN_WRITEOFF_TRANS) that we know only from a comment in the cash flow script. If it does not "
    "exist, remove that line and run the query again.",
    "The vendor may prefer not to release the full listing of tables and columns (runs 5 and 26). The other queries still "
    "give us most of what we need.",
    "We wrote the queries without access to the database, so they have not been run. One or two may need a small correction "
    "the first time.",
])

# ----- 12 script changes
H1("12. Suggested changes to MAIIC's three extract scripts")
P("These are the changes we suggest to the scripts themselves, with the line of each file they apply to. They are small "
"edits to code that already exists. Several depend on the Tier 1 results and are marked. We suggest the vendor or ICT "
"makes them; we can draft the amended scripts once runs 1 to 7 are back.")
H2("FACILITY_REGISTER.txt (Extract A)")
TAB(["Line", "What it does now", "Suggested change"], [
    ["44", "Translates the Floating Flag: F to 'Fixed', I to 'Variable'", "Output the raw flag value as stored. Add "
    "Interest Policy as its own column once run 4 or 5 gives its name."],
    ["30, 89", "Reads the scheme but outputs only its name", "Also output the scheme identifier (" + M("s.scheme_mst_id") + ") "
    "and the scheme's other settings."],
    ["34 to 42", "Sums and lists the disbursement schedule", "Keep it, headed as planned disbursements. Add the actual "
    "drawdowns from the ledger (type 301 postings), each with its date, amount and reference."],
    ["52 to 57", "Counts the chart rows and takes the earliest chart date", "Leave out the opening row once run 13 confirms it. "
    "Better, supply the whole chart as its own extract."],
    ["43, 59, 60 to 68", "Reads the rate, closure date and write-offs as they stand today", "Either limit them to the as-at "
    "date, or head the columns 'as at run date' so the file cannot be misread."],
    ["69 to 70", "Translates status A, C and D only", "Output the raw status code. Add a meaning column once the code list is "
    "supplied."],
    ["77", "Removes the sign from the balance", "Keep the sign. Add any interest balance held separately."],
    ["99 to 102", "Finds the latest balance dated on or before the as-at date", "If run 11 shows dates carry a time, test for "
    "dates before the start of the next day, so the whole as-at day is included."],
    ["119", "Selects loans by scheme name beginning MAIIC or FINES", "Confirm with run 10 that no loan scheme is left out. "
    "Leave out the equity-type schemes, or flag them."],
], [0.12, 0.38, 0.50])
H2("CASHFLOW_REGISTER.txt (Extract B)")
TAB(["Line", "What it does now", "Suggested change"], [
    ["55 to 80", "Outputs every amount and the balance without its sign", "Output the signed amount as stored, and a debit or "
    "credit column."],
    ["44 to 54", "Replaces the transaction type code with a label", "Output the raw code as well as the label."],
    ["58 to 62, 67 to 70", "Estimates the principal and interest split of type 305", "Remove the estimate. If E-Banker stores "
    "the real split anywhere, output that; if not, leave the columns empty."],
    ["34", "Reads only postings on the loan's own GL code", "Add the postings on every other GL code for the same loan accounts, "
    "or supply a separate fee and charge extract. Run 7 shows what exists."],
    ["85, 105, 126", "Selects dates between the start and end date", "If run 11 shows dates carry a time, test for dates from "
    "the start date up to, but not including, the day after the end date."],
    ["107 to 127", "Lists instalments not fully recovered, at full original amounts", "Replace with a separate instalment chart "
    "extract: every row, paid or not, with the amount recovered against each."],
    ["2 to 3", "Defaults to 1 January 2025 onward", "Run it from October 2024, when E-Banker went live."],
], [0.16, 0.36, 0.48])
H2("INTEREST_RECON.txt (Extract C)")
TAB(["Line", "What it does now", "Suggested change"], [
    ["9 to 15", "Adds each run to the table and keeps the earlier ones", "Export the latest run only (" + M("WHERE run_id = "
    "...") + ") , or clear earlier runs for the same period first."],
    ["58 to 61", "Writes monthly rows and yearly total rows together", "Export monthly rows only, or put the yearly totals in a "
    "separate file."],
    ["37", "Adds six transaction types into one figure", "Show the amount for each type in its own column, so that charges and "
    "reductions can be seen separately."],
    ["35", "Reads only postings on the loan's own GL code", "Confirm with run 7 that no interest is posted elsewhere."],
    ["16 to 21, 33 to 34", "Produces no row for an account with no matching posting", "List every in-scope account, with a zero "
    "and a note where nothing was found."],
    ["38", "Selects dates between the start and end date", "As for Extract B, if dates carry a time."],
    ["2 to 3", "Defaults to 1 January 2025 onward", "Run it from October 2024 to date, and agree its total to the interest "
    "income codes in the trial balance."],
], [0.16, 0.36, 0.48])

# ----- 13 engine code
H1("13. Suggested changes to our own engine's code")
P("The review also looked at how our EIR engine loads these extracts. We then read the relevant files ourselves, in the "
"MAICC-IFRS9 repository on the branch " + M("eir_revenue_recognition") + ", on 5 October 2026. We did not run the engine or "
"inspect the data already loaded, and <b>nothing in the code has been changed</b>. These are suggestions for the Duplex "
"team, listed with the most important first.")
TAB(["#", "Where", "What the code does now", "Suggested change", "Priority"], [
    ["1", M("EirRevenueService.php") + " line 22 and the cash received routine",
    "Counts Extract B rows labelled Interest, Principal+Interest and Fee as cash collected in the month.",
    "Take Interest out of the list of collections. Those rows are interest charged to the loan; for 2025 they total MWK 2,639, "
    "557,380. Once Extract B carries signed amounts and type codes, classify cash by code and sign, not by label. Re-run any period "
    "whose cash figure came from the imported 2025 rows."],

```

```

    "High"],
    ["2", M("ContractMasterImport.php") + " line 72; " + M("ContractMasterImportService.php") + " rate type routine",
    "Loads Extract A's Fixed or Variable label as the loan's rate type (Variable becomes FLOATING). The ECL discount rate and
time-phased ECL services read that field.",
    "Stop deriving the rate type from this label. Keep the delivered label in its own field as evidence. Derive the rate type
from Interest Policy once it is extracted, and until then from observed repricing in the loan books. Correct the code comment
that refers to 204 variable-rate facilities.",
    "High"],
    ["3", M("ContractTransactionImportService.php") + " lines 66 to 71; " + M("RemainingScheduleImportService.php") + "; " +
M("ScheduleWorkflowService.php") + " comparison",
    "Loads Extract B's scheduled rows as a \"remaining schedule\" and compares their totals with our generated schedule from
the earliest due date onward.",
    "Treat these rows as what they are: instalments not fully recovered inside the extract's date range, at full original
amounts. Rename them accordingly. Limit the comparison to matching due dates inside that range, or switch it off until the full
chart extract arrives. Never feed them into a calculation.",
    "Medium"],
    ["4", M("ContractTransactionImportService.php") + " lines 78 to 85",
    "Stores each actual row's unsigned total and the estimated principal and interest parts. We found no other code that reads
those two parts.",
    "Add fields for the raw transaction type code, the signed amount, and a marker for an estimated split. Keep estimated
parts out of every calculation.",
    "Medium"],
    ["5", M("GlInterestImportService.php"),
    "Skips yearly rows and repeats of the same loan, month and GL code, keeping the first one it meets. Its description calls
the figure the interest income the ledger posted.",
    "Describe the figure as interest charged to loan accounts, not yet agreed to the income GL. Where a file holds several
runs, keep the latest run and report how many runs were in the file.",
    "Medium"],
    ["6", M("MappedFileReader.php") + " date routine, lines 420 to 454",
    "Accepts Excel date serials, then the declared format, then falls back to flexible parsing.",
    "For MAIIC extracts, accept year-first dates only. Hold the file when a date column mixes real dates with text, which is
exactly how the scrambled extracts arrive.",
    "Medium"],
    ["7", "GL reconciliation and coverage reporting",
    "Not traced in this review.",
    "Check that a loan with no row in the interest postings is reported as \"no posting found\" and not as zero interest. Keep
three states apart: posted, verified zero, and not covered.",
    "Medium"],
    ["8", M("ContractMasterImportService.php") + " line 281",
    "Stores Extract A's disbursement list as text on the contract.",
    "Take drawdown dates and amounts from the ledger disbursement rows. Keep Extract A's list as the planned schedule only. On
Ebenezer Midian the two differ by 24 days.",
    "Medium"],
    ["9", "Contract key in the import services",
    "Identifies a loan by account number. The sub-account is stored but is not part of the key.",
    "Key on account number and sub-account together. Every sub-account is 1 today, but the vendor says a restructure creates a
new one.",
    "Low"],
    ["10", "Use of Extract A terms",
    "Loads rate, status and terms from a snapshot as if they were as at the snapshot date.",
    "Record them as at the run date. Do not load the repayment frequency from the 1 January 2025 snapshot.",
    "Low"],
], [0.04, 0.21, 0.27, 0.40, 0.08])
NOTE("File and line references are to the branch as it stood on 5 October 2026. Item 1 follows from the finding that interest "
    "rows are charges, which is inferred and still to be confirmed with signed postings (section 2).")

# ----- 14 sequence
H1("14. The sequence from here")
BUL([
    "MAIIC runs Tier 1 (runs 1 to 11) and sends the results. Runs 1 to 7 are enough for us to begin.",
    "MAIIC sends the procedure version history and the record of the two Extract A runs, and a line of explanation for each of
the ten accounts in Appendix C.",
    "Duplex drafts the amended extract scripts against the real table and column names (section 12), and makes the engine
changes in section 13.",
    "MAIIC runs Tiers 2 and 3 when convenient. They refine the work but do not hold it up.",
    "MAIIC runs the amended extracts. Duplex loads them and reconciles interest to the trial balance. That reconciliation has
not yet been done.",
], numbered=True)

# ----- appendices
story.append(PageBreak())
H1("Appendix A. The tests behind this report")
P("Each statement about the delivered files was tested on the files themselves. These are the results. The scripts that "
    "produce them are in the Build files folder.")
TAB(["Test", "Result"], [
    ["Extract A: rows, accounts and snapshots", "362 rows; 181 accounts; two snapshots, 1 January 2025 and 31 December 2025. No
repeated rows for the same account, sub-account and snapshot. Sub-account is 1 on every row"],
    ["Extract A: rate label by product (31 December 2025 snapshot)", "Fixed: MAIIC Agricultural 16, MAIIC Industrial 41, MAIIC
Term Loan 13, Fines Investment Loan 4, Quasi Equity Investment 5. Variable: FINEs Agricultural 50, FINEs Industrial 51,
NASCOMEX Limited 1"],
    ["Extract A: fields compared between the two snapshots", "Interest rate, account status, sanctioned amount, tenor, number
of instalments, rate label, principal disbursed and tranche list: identical for all 181 accounts"],
    ["Extract A: repayment frequency in the two snapshots", "1 January 2025: Monthly 181. 31 December 2025: Monthly 139,
Half-Yearly 10, Quarterly 4, Yearly 3, blank 25. 42 accounts differ"],
    ["Extract A: Lake Malawi Aquaculture, 31 December 2025 snapshot", "Sanctioned 1,055,473,655; one tranche of 297,161,905
dated 28 April 2026"],
    ["Extract A: principal disbursed against sanctioned amount", "Populated on 157 accounts; equal to the sanctioned amount on
138"],
    ["Extract A: entries in the tranche list", "Populated on 314 of 362 rows. One entry on 256 rows; two to ten entries on 58"],
    ["Extract A tranche list against ledger disbursements in Extract B", "35 accounts have ledger disbursements in 2025. Total
amount agrees on 30; number of entries agrees on 28. Ebenezer Midian: 15 July 2025 in Extract A, 8 August 2025 in the ledger"],

```

["Extract A: first repayment date against loan start date", "The same on 63 of the 128 accounts with a first repayment date"],

["Extract A: accounts with no instalment chart rows", "53"],

["Extract A: account status (31 December 2025 snapshot)", "Active 86, Closed 46, Dormant 25, H 21, F 3"],

["Extract B: rows by kind", "2,478 rows, one run. Actual 1,703: Interest 1,034, Principal+Interest 539, Disbursement 97, Other/Adjustment 31, Fee 2. Scheduled 775, across 90 accounts"],

["Extract B: negative amounts", "None"],

["Extract B: direction of each row, from the change in the running balance", "Tested on every actual row after the first for its account. Interest: 987 raise the balance, 6 reduce it, 1 not placed. Principal+Interest: 485 reduce, 4 raise, 12 not placed. Disbursement: 63 raise, 1 not placed. Other/Adjustment: 18 raise, 9 reduce, 3 not placed. 16 rows in the file are out of posting order, which accounts for some of the rows not placed"],

["Extract B: Other/Adjustment rows", "31 rows totalling 832,653,206.42"],

["Extract B closing balance against Extract A balance at 31 December 2025", "111 accounts have both. 110 agree within MWK 1, taking the row with the highest posting number as the close. Exception: Zaithwa Farms, 1,225,683.03 against 3,053,316.65"],

["Extract B: scheduled rows", "0 of 775 carry a balance. Principal plus interest equals the total on 761 of 775"],

["Extract C: runs in the delivered file", "Run 7: 1,119 rows. Run 25: 1,119 rows. Run 41: 1,758 rows. No repeated loan, GL code and month within run 41"],

["Extract C: loan-months", "1,564 distinct loan-months; 1,012 appear three times and 552 once"],

["Extract C: run 7 against run 41 for 2025", "1,012 loan-months in both; amounts differ on none"],

["Extract C: 2025 interest", "All runs added together 7,663,172,090; run 41 alone 2,554,390,697"],

["Extract B interest rows against Extract C (run 41), 2025", "107 accounts in both. 101 agree exactly. Extract B total 2, 639,557,380; difference 85,166,683.31"],

["Extract C: accounts in Extract A with no row in Extract C", "61: Closed 46, Active 10, Dormant 4, H 1"],

], [0.42, 0.58]]

H1("Appendix B. The six accounts where Extract B and Extract C disagree")

P("For 2025, the interest rows in Extract B exceed the interest in Extract C on six accounts. On five of them the difference " "equals the account's \"Other/Adjustment\" rows in Extract B. On Happie Foods it equals one of the two such rows: the " "2,018,700.44 row reduces the balance, while the 7,000,000 row raises it. This agreement in amounts does not establish " "what the postings are. That needs the raw signed postings and the vendor's meaning for each transaction type.")

TAB(["Account", "Customer", "Extract B interest", "Extract B other", "Extract C interest", "Difference"], [

["000104430000068", "Nyamunyamu Processors Limited", "131,899,213.80", "69,330,809.54", "62,568,404.26", "69,330,809.54"],

["000104430000002", "Edge View Academy Limited", "117,296,781.00", "10,457,152.33", "106,839,628.67", "10,457,152.33"],

["000104430000073", "Saile Financial Services Limited Company", "90,522,529.35", "2,395,117.82", "88,127,411.53", "2,395, 117.82"],

["000104420000062", "Happie Foods Limited", "38,421,398.75", "9,018,700.44", "36,402,698.31", "2,018,700.44"],

["000104420000052", "Kambewu Organic Fertilizer Investments", "42,329,779.82", "913,541.02", "41,416,238.80", "913,541.02"],

["000104420000032", "African Honey Products Industries", "29,893,579.18", "51,362.16", "29,842,217.02", "51,362.16"],

], [0.17, 0.27, 0.15, 0.13, 0.15, 0.13], keep=True)

NOTE("The six differences add up to MWK 85,166,683.31, which is the whole difference between the two extracts for 2025.")

H1("Appendix C. Active accounts with no interest posted in Extract C")

P("These ten accounts are marked Active in Extract A but have no interest row in Extract C between January 2025 and July " "2026. That is not the same as a verified zero. Some may not have been drawn yet, some are not loans, and for some the " "interest may be posted where the script does not look. Each one needs a line of explanation from MAIIC.")

TAB(["Account", "Customer", "Product", "Sanctioned amount, MWK"], [

["000104470000004", "NASCOMEX Limited", "NASCOMEX Limited", "2,500,000,000"],

["000104430000081", "Pinnacle Financial Services Limited", "MAIIC Industrial Loans", "1,000,000,000"],

["000104430000088", "Anchor Processors Limited", "MAIIC Industrial Loans", "300,000,000"],

["000104430000083", "JAT Group Ltd", "MAIIC Industrial Loans", "209,270,000"],

["000104460000111", "Ecogen Limited", "FInES Industrial Loans", "105,060,000"],

["000105810000004", "Ecogen Limited", "Fines Investment Loan", "105,060,000"],

["000105810000003", "Ecogen Limited", "Fines Investment Loan", "100,000,000"],

["000104440000005", "Belk Inc", "Quasi Equity Investment", "69,000,000"],

["000104420000003", "Micholless Creamery", "MAIIC Agricultural Loans", "20,000,000"],

["000104450000012", "Maluso Cooperative Union", "FInES Agricultural Loans", "11,511,500"],

], [0.20, 0.36, 0.24, 0.20], keep=True)

H1("Appendix D. Terms used in this report")

TAB(["Term", "Meaning"], [

["Script, stored procedure", "A set of database instructions saved inside the database and run on request."],

["Query", "A question put to the database. A read-only query returns data and changes nothing."],

["Table, column", "A table is a list of records of one kind, such as accounts. A column is one field in that list, such as the interest rate."],

["Data dictionary, catalogue", "The database's own list of its tables and columns."],

["Schema, owner", "A named area of the database that holds a set of tables. Two schemas can hold tables with the same name."],

["Scheme", "In E-Banker, the product variant an account is opened on. Settings such as Interest Policy are held on the scheme."],

["GL code", "The general ledger account a posting is made to. Each loan product has its own."],

["Transaction type code", "A number E-Banker gives each posting to say what kind it is, such as 301 for a disbursement."],

["Run number", "A number the script gives each time it is run, so that the results of different runs can be told apart."],

["Snapshot, as at date", "The position on a stated date, as opposed to the position today."],

["Instalment chart, EMI chart", "The loan's repayment schedule as held in E-Banker."],

["Effective interest rate (EIR)", "The rate that spreads all of a loan's cash flows, including fees, evenly over its life. It is what IFRS 9 requires interest income to be recognised at."],

["Engine", "The Duplex software that calculates the effective interest rate and the related income for MAIIC."],

], [0.28, 0.72], bold_first=True)

H1("Appendix E. The independent review, and what changed in version 2")

P("The first version of this report, the query pack and the record of the working session were reviewed on 5 October 2026 " "by a second AI system (Codex). The reviewer re-ran our main tests on the delivered files and reproduced the figures. It " "raised eight corrections. We accept all eight. The review is filed in the engine's repository under " " + M("docs/reviews") + ".")

TAB(["#", "The review's comment", "Our response", "What changed"], [

["1", "Scheduled rows are not the unpaid remainder. The script exports full original amounts and does not subtract the amount recovered.", "Accepted. Our wording was wrong.", "Section 5, row 6; section 12; section 13, item 3."],

["2", "The running balance is not proven correct. It depends on complete history, the opening position, posting scope and agreement with independent balances.", "Accepted. We tested it: 110 of 111 accounts agree with Extract A at year end.", "Section 5, first bullet; Appendix A."],

["3", "No interest rows is not a verified zero.", "Accepted.", "Section 6, row 4; section 9; Appendix C; section 13, item

```

7. "],
["4", "Final-day date handling should be checked. Dates with a time of day could drop last-day postings.", "Accepted. A new
query tests it.", "Sections 4 to 6; run 11 (DD_19); section 12."],
["5", "Agreement in total does not establish what the adjustments are. Happie Foods leaves MWK 7 million unexplained.",
"Accepted, and it goes further: the \"Other/Adjustment\" rows total MWK 832.7 million and most do not reduce interest.",
"Section 5, row 4; Appendix B."],
["6", "Schema and privilege assumptions. Several schemas may hold ACMaster; running a procedure does not prove read
access.", "Accepted.", "Owner added to every catalogue result; runs 2 and 3 (DD_17, DD_18)."],
["7", "The cause of the repayment-frequency differences is not established.", "Accepted. We now state the facts and three
possible causes, and have asked for the run history.", "Section 4, closing paragraphs; section 9."],
["8", "Define the export contract. CSV alone does not protect dates or account numbers.", "Accepted.", "Section 7; the
header of both SQL files; session settings at run 0."],
], [0.04, 0.40, 0.30, 0.26])
P("The review also made three points about our engine's code, and our own reading added more. These are in section 13. "
"Two of our findings went beyond the review: the engine counts Extract B interest rows as cash collected, and it stores "
"the rate type from the label that appears to be inverted.")
P("As the review asked, this version separates what was tested from what is inferred and what is pending (section 2), and "
"states that it is not a completed income reconciliation or a validated cash flow schedule.")

```

```

# ----- build
def footer(canvas, doc):
    canvas.saveState()
    canvas.setStrokeColor(LINE); canvas.setLineWidth(0.5)
    canvas.line(18 * mm, 14 * mm, A4[0] - 18 * mm, 14 * mm)
    canvas.setFont("Seg", 8); canvas.setFillColor(GREY)
    canvas.drawString(18 * mm, 9.5 * mm, "MAIIC E-Banker extract scripts: analysis · Duplex Institute · 5 October 2026 ·
Version 2")
    canvas.drawRightString(A4[0] - 18 * mm, 9.5 * mm, f"Page {doc.page}")
    canvas.restoreState()

OUT = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf"
doc = BaseDocTemplate(OUT, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=20 * mm,
    title="MAIIC E-Banker extract scripts: analysis (version 2)", author="Duplex Institute",
    subject="What the three raw query scripts do, where they fall short, and which queries to run next")
doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, id="f",
    leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)],
onPage=footer)])
doc.build(story)
print("written", OUT)

```

Result of Write [error]

```

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write
it.</tool_use_error>

```

Tool call: Read

```

file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\maiic_report.py
limit: 12

```

Result of Read

```

1  """Plain-language report on the three MAIIC raw query scripts (PDF, reportlab)."""
2  from reportlab.lib.pagesizes import A4
3  from reportlab.lib.units import mm
4  from reportlab.lib import colors
5  from reportlab.lib.styles import ParagraphStyle
6  from reportlab.lib.enums import TA_LEFT
7  from reportlab.pdfbase import pdfmetrics
8  from reportlab.pdfbase.ttf fonts import TTFont
9  from reportlab.platypus import (BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Table, TableStyle,
10                                PageBreak, KeepTogether, Image, ListFlowable, ListItem, CondPageBreak)
11
12  F = r"C:\Windows\Fonts"

```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\maaic_report_v2.py
--- content ---
"""Plain-language report on the three MAIIC raw query scripts (PDF, reportlab). Version 2, after the Codex review."""
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.enums import TA_LEFT
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import (BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Table, TableStyle,
                                PageBreak, KeepTogether, Image, ListFlowable, ListItem, CondPageBreak)

F = r"C:\Windows\Fonts"
pdfmetrics.registerFont(TTFont("Seg", F + r"\segoeui.ttf"))
pdfmetrics.registerFont(TTFont("Seg-B", F + r"\segoeuib.ttf"))
pdfmetrics.registerFont(TTFont("Seg-I", F + r"\segoeuii.ttf"))
pdfmetrics.registerFont(TTFont("Seg-BI", F + r"\segoeiz.ttf"))
pdfmetrics.registerFontFamily("Seg", normal="Seg", bold="Seg-B", italic="Seg-I", boldItalic="Seg-BI")
pdfmetrics.registerFont(TTFont("Mono", F + r"\consola.ttf"))

NAVY = colors.HexColor("#1B2A41"); ORANGE = colors.HexColor("#E07B00"); GREY = colors.HexColor("#5B6470")
LIGHT = colors.HexColor("#F4F6F9"); LINE = colors.HexColor("#C9D1DB"); AMBER = colors.HexColor("#FFF4E0")

S = {
    "title": ParagraphStyle("title", fontName="Seg-B", fontSize=24, leading=29, textColor=NAVY, spaceAfter=4),
    "sub": ParagraphStyle("sub", fontName="Seg", fontSize=13, leading=17, textColor=GREY, spaceAfter=10),
    "meta": ParagraphStyle("meta", fontName="Seg", fontSize=9.5, leading=13, textColor=GREY),
    "h1": ParagraphStyle("h1", fontName="Seg-B", fontSize=15, leading=19, textColor=NAVY, spaceBefore=14, spaceAfter=6,
keepWithNext=1),
    "h2": ParagraphStyle("h2", fontName="Seg-B", fontSize=11.5, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4,
keepWithNext=1),
    "h2t": ParagraphStyle("h2t", fontName="Seg-B", fontSize=11.5, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4),
    "p": ParagraphStyle("p", fontName="Seg", fontSize=10, leading=14.2, textColor=colors.black, spaceAfter=6,
alignment=TA_LEFT),
    "li": ParagraphStyle("li", fontName="Seg", fontSize=10, leading=14, textColor=colors.black),
    "cell": ParagraphStyle("cell", fontName="Seg", fontSize=8.6, leading=11.4),
    "cellb": ParagraphStyle("cellb", fontName="Seg-B", fontSize=8.6, leading=11.4),
    "head": ParagraphStyle("head", fontName="Seg-B", fontSize=8.8, leading=11.4, textColor=colors.white),
    "box": ParagraphStyle("box", fontName="Seg", fontSize=10, leading=14.2, textColor=colors.black, spaceAfter=3),
    "note": ParagraphStyle("note", fontName="Seg-I", fontSize=8.8, leading=12, textColor=GREY, spaceAfter=6),
}
W = A4[0] - 36 * mm
story = []

def H1(t): story.append(Paragraph(t, S["h1"]))
def H2(t): story.append(Paragraph(t, S["h2"]))
def P(t): story.append(Paragraph(t, S["p"]))
def NOTE(t): story.append(Paragraph(t, S["note"]))
def GAP(h=4): story.append(Spacer(1, h))

def H2T(t):
    """Heading that sits directly above a long table: break first if little room is left."""
    story.append(CondPageBreak(62 * mm)); story.append(Paragraph(t, S["h2t"]))

def BUL(items, numbered=False):
    story.append(ListFlowable([ListItem(Paragraph(i, S["li"]), leftIndent=14, spaceAfter=3) for i in items],
bulletType="1" if numbered else "bullet", bulletFontName="Seg", bulletFontSize=9,
start=None if numbered else "*", leftIndent=14, bulletColor=ORANGE if not numbered else NAVY))
    GAP(4)

def TAB(header, rows, widths, bold_first=False, keep=False):
    data = [[Paragraph(h, S["head"]) for h in header]]
    for r in rows:
        data.append([Paragraph(str(c), S["cellb"]) if (bold_first and i == 0) else S["cell"] for i, c in enumerate(r)])
    t = Table(data, colWidths=[W * x for x in widths], repeatRows=1)
    st = [("BACKGROUND", (0, 0), (-1, 0), NAVY), ("VALIGN", (0, 0), (-1, -1), "TOP"),
          ("LEFTPADDING", (0, 0), (-1, -1), 5), ("RIGHTPADDING", (0, 0), (-1, -1), 5),
          ("TOPPADDING", (0, 0), (-1, -1), 4), ("BOTTOMPADDING", (0, 0), (-1, -1), 5),
          ("LINEBELOW", (0, 0), (-1, -1), 0.4, LINE), ("BOX", (0, 0), (-1, -1), 0.6, LINE)]
    for i in range(1, len(data)):
        if i % 2 == 0:
            st.append(("BACKGROUND", (0, i), (-1, i), LIGHT))
    t.setStyle(TableStyle(st))
    story.append(KeepTogether([t]) if keep else t)
    GAP(8)

def BOX(paras, colour=AMBER):
    t = Table([[[Paragraph(x, S["box"]) for x in paras]]], colWidths=[W])
    t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, -1), colour), ("LINEBEFORE", (0, 0), (0, -1), 3, ORANGE),
          ("LEFTPADDING", (0, 0), (-1, -1), 10), ("RIGHTPADDING", (0, 0), (-1, -1), 10),
          ("TOPPADDING", (0, 0), (-1, -1), 8), ("BOTTOMPADDING", (0, 0), (-1, -1), 6)]))
    story.append(t); GAP(8)
```

```

def M(t):
    return f'<font name="Mono" size="8">{t}</font>'

# ----- cover block
LOGO = r"C:\Users\wadza\OneDrive\2026\Tenders\PFA\Excel & Data Analysis Training\Basic Excel Course\3 Source Files\img\
dupleix-logo.png"
try:
    im = Image(LOGO); r = im.imageHeight / im.imageWidth; im.drawWidth = 42 * mm; im.drawHeight = 42 * mm * r; im.hAlign =
"LEFT"
    story.append(im); GAP(10)
except Exception:
    pass
story.append(Paragraph("MAIIC E-Banker extract scripts", S["title"]))
story.append(Paragraph("What the three scripts do, where they fall short, and which queries to run next", S["sub"]))
story.append(Paragraph("Prepared by Dupleix Institute for MAIIC Finance and ICT · 5 October 2026 · <b>Version 2</b>",
S["meta"]))
story.append(Paragraph("Version 2 was revised on the same day after an independent review of the first version. The review's
comments, "
                        "our response to each, and what changed are in Appendix E.", S["meta"]))
story.append(Paragraph("Based on: the three script files received on 5 October 2026; Extracts A and B as delivered on 3 August
2026; "
                        "Extract C as re-run and delivered on 26 August 2026; the E-Banker user manual.", S["meta"]))
GAP(8)
BOX(["<b>What this report is, and is not.</b> It is an analysis of three extract scripts and of the files they produced. "
     "We read the scripts and tested them against those files. We have not seen the database.",
     "It is <b>not</b> a completed reconciliation of interest income, and it does <b>not</b> validate any contractual cash flow
"
     "schedule. The 27 queries it asks for were written without access to the database and have not been run. "
     "Section 2 separates what we tested from what we infer and what is still waiting on MAIIC or the vendor."])

# ----- 1 summary
H1("1. Summary")
P("MAIIC has sent us the three database scripts that produce Extract A (the facility register), Extract B (the cash flow "
  "register) and Extract C (the interest posted on each loan). We read each script line by line and then tested what it "
  "implies against the extract files we already hold.")
P("The scripts are tidy, and they prove that the data we need is held in E-Banker. They also explain, for the first time, "
  "most of the puzzles we raised in September. But each script makes choices that hide or change information the effective "
  "interest rate (EIR) engine depends on, and some of the fields we need are not read by any of the three.")
H2("Three findings that affect figures already in use")
BUL([
  "<b>The Extract C file counts 2025 three times.</b> The script keeps every earlier run, and the file we received holds
three "
  "runs. Added up as delivered, 2025 interest is MWK 7,663,172,090. The figure from the latest run alone is MWK 2,554,390,
697.",
  "<b>The Fixed and Variable labels in Extract A appear to be the wrong way round.</b> The script labels the system's code F
as "
  "\"Fixed\" and its code I as \"Variable\", while the manual gives F as Floating and I as Fixed. This would explain why the "
  "labels contradicted what the loan books show.",
  "<b>The \"Interest\" rows in Extract B are interest charged to the loan, not cash received.</b> 987 of the 994 interest
rows "
  "we could test raise the loan balance, and they are the same postings Extract C reports. They must not be treated as "
  "money received.",
], numbered=True)
H2("What we are asking for")
P("Two short rounds of read-only queries, before any extract is rewritten:")
BUL([
  "<b>First, the data dictionary queries.</b> These list the tables and columns E-Banker holds. We need them first because
the "
  "fields still missing, starting with Interest Policy, do not appear in any of the three scripts. We do not yet know what "
  "they are called or which table they sit in, so an amended extract cannot be written.",
  "<b>Second, the question queries.</b> These return codes, counts, checks and one sample loan. They answer specific open "
  "questions: which way each transaction type moves the balance, where the fees are posted, what the status codes are, "
  "whether dates carry a time of day, and why the restructure date is blank.",
  "<b>Then the amended extracts,</b> written once, against the real structure.",
], numbered=True)
P("All 27 queries are in the file <b>RUN ORDER - data dictionary queries by priority.sql</b>, in the order to run them. "
  "Runs 1 to 7 are enough for us to start drafting the amended extracts.")
P("This version also sets out suggested changes to the three scripts (section 12) and to our own engine's code (section 13).")

# ----- 2 how sure
H1("2. How sure we are")
P("Not every statement in this report rests on the same footing. This table separates the three kinds.")
TAB(["Footing", "Statements"], [
  ["<b>Tested on the delivered files.</b> Each can be re-run from the scripts in the Build files folder; the reviewer re-ran
the main ones and got the same figures.",
   "The Extract C file holds three runs with identical 2025 amounts.<br/>"
   "The rate label by product: MAIIC Agricultural and Industrial all \"Fixed\", FInES Agricultural and Industrial all \"
Variable\".<br/>"
   "Rate, status, sanctioned amount and tenor are identical in both Extract A snapshots.<br/>"
   "Extract B has no negative amounts, 539 estimated rows, and no balance on its scheduled rows.<br/>"
   "Extract B interest and Extract C agree on 101 of 107 accounts for 2025; the difference is MWK 85,166,683.31 on six
accounts.<br/>"
   "987 of 994 interest rows raise the running balance.<br/>"
   "Extract B's closing balance agrees with Extract A's year-end balance on 110 of 111 accounts.<br/>"
   "Ebenezer Midian: 15 July 2025 in the disbursement list, 8 August 2025 in the ledger."],
  ["<b>Inferred.</b> Reasoned from the scripts and the files, but not confirmed against the database or by the vendor.",
   "That the rate labels are inverted. This rests on the manual's code list (F Floating, I Fixed) and on loan book behaviour;
we have not seen the raw flag values.<br/>"
   "That interest rows are charges. This rests on how the balance moves; we have not seen the raw signed postings.<br/>"
   "That the instalment chart begins with a row at the loan's start date.<br/>"]
])

```

```

    "That the reschedule table is empty.<br/>"
    "That the dates were scrambled by Excel on the way out.<br/>"
    "What the \"Other/Adjustment\" rows are.",
    ["<b>Waiting on MAIIC or the vendor.</b>"],
    "Where Interest Policy is held.<br/>"
    "What transaction types 120 and 308 to 311 mean, and what status codes H and F mean.<br/>"
    "Where fees are posted against a loan.<br/>"
    "Whether posting dates carry a time of day.<br/>"
    "Which schema the table names point to, and whether the login can read the tables directly.<br/>"
    "Why the repayment frequency differs between the two snapshots.<br/>"
    "Why ten active accounts show no interest.<br/>"
    "The reconciliation of Extract C to the interest income codes in the trial balance.",
], [0.30, 0.70])

# ----- 3 what we received
H1("3. What we received")
P("Each file is a stored procedure: a routine saved inside the database that fills a results table when someone runs it. "
  "The results table is then copied out to Excel and sent to us.")
TAB(["File", "Builds", "What it produces"], [
  ["FACILITY_REGISTER.txt", "Extract A", "One row for each loan account as at a chosen date: who the customer is, the terms
of the loan, and the balance."],
  ["CASHFLOW_REGISTER.txt", "Extract B", "One row for each posting on a loan in a date range, with a running balance,
followed by write-offs and by scheduled instalments not yet fully recovered."],
  ["INTEREST_RECON.txt", "Extract C", "The interest posted on each loan, totalled for each month and for each year."],
], [0.26, 0.12, 0.62], bold_first=True)
P("Two things are missing from the files. The definitions of the results tables are not included, and neither is the step "
  "that copies the results out to Excel. The second one matters: it is where the dates were scrambled (section 7).")
H2("The E-Banker tables the scripts read")
P("This is the first time we have seen E-Banker's real table names. Between them the three scripts read thirteen tables.")
TAB(["Table", "What it holds, in plain language", "What the scripts take from it"], [
  ["ACMASTER", "The account master: one row for each account", "Account number, customer, GL code, interest rate, opening
date, status, currency, scheme"],
  ["ACLOANMASTER", "The loan details for each account", "Sanctioned amount, tenor, expiry date, moratorium periods"],
  ["CUSTOMER_MAST", "The customer master", "Customer name"],
  ["SCHEME_MASTER", "The scheme (the product variant) each account is opened on", "Scheme name, Floating Flag, compound or
simple, days in the year"],
  ["GLMASTER", "The list of GL codes", "GL title"],
  ["CUMVOUCH", "The ledger: every posting", "Date, transaction type code, amount, GL code"],
  ["ACCOUNT_BALANCE", "Balance history by date", "Principal balance"],
  ["FIXLOAN_INST_CHART", "The instalment chart (the EMI chart)", "Instalment date, amount, principal part, interest part,
amount recovered"],
  ["MULTI_INST_DETAILS", "The instalment plan", "Repayment frequency"],
  ["LOAN_DISBURSEMENT_SCHEDULE", "The disbursement schedule", "Date and amount"],
  ["LOAN_RESCHEDULE_DETAILS", "Reschedules (restructures)", "Latest reschedule date"],
  ["LOAN_WRITEOFF_MASTER", "The write-off register", "Write-off date and amount"],
  ["LOAN_WRITEOFF_TRANS", "Write-off postings", "Named in a comment only; not read"],
], [0.27, 0.36, 0.37], bold_first=True)
H2("Which loans are included")
P("All three scripts pick loans the same way: any account whose scheme name begins with \"MAIIC\" or \"FINES\". A loan on a "
  "scheme named differently would be left out, and the scripts cannot tell us whether any exist. The same rule also brings in "
  "accounts that are not loans: Extract A includes five Quasi Equity Investment accounts and one NASCOMEX Limited account.")
# ----- 4 Extract A
H1("4. Extract A: the facility register")
H2("What the script does")
P("For the date it is given, the script removes any earlier result for that date and builds it again. For each account it "
  "collects the loan's details from the account, loan, customer and scheme masters. It adds up the disbursement schedule, "
  "counts the rows in the instalment chart, picks up any write-off and the latest reschedule date, and looks up the most "
  "recent principal balance on or before the date.")
H2("What is good about it")
BUL([
  "The account number and sub-account number are carried through cleanly, so Extract A joins to B and C. There are no "
  "repeated rows for the same account, sub-account and date.",
  "Where a balance is missing, the script says why in a note: the loan opened after the date, it closed before the date, "
  "or no balance was found and it needs checking by hand.",
  "The day-count basis and whether interest is compound or simple come straight from the scheme, which settled our day-count
decision.",
  "It can be run again for the same date without creating duplicates.",
])
H2T("Where it falls short")
TAB(["#", "What the script does", "Why it matters", "What the delivered file shows"], [
  ["1", "<b>Labels the rate type, apparently the wrong way round.</b> It reads the scheme's Floating Flag and labels code F
as \"Fixed\" and code I as \"Variable\". The manual (pages 19 to 20) gives F as Floating and I as Fixed.",
  "The label says the opposite of how the loan behaves, so it cannot be used to decide which loans reprice.",
  "All 57 MAIIC Agricultural and Industrial accounts are labelled Fixed, and all 101 FINES Agricultural and Industrial
accounts Variable. The loan books show the reverse. The 13 Term Loans still do not fit after the labels are swapped, so the
flag is not the deciding field."],
  ["2", "<b>Leaves out Interest Policy and the scheme code.</b> It reads the scheme master but takes only four settings from
it.",
  "Interest Policy is the setting that decides whether a loan follows the Reserve Bank rate. The scheme identifier is
already in the script's hands and is simply not output.", "Not in the file."],
  ["3", "<b>Only the balance is taken as at the date.</b> The rate, status, amounts and dates are read as they stand on the
day the script is run.",
  "The file looks like a picture of the book at an earlier date, but most of it is today's picture. It cannot show what a
loan's rate was at 1 January 2025.",
  "Rate, status, sanctioned amount and tenor are identical in the two snapshots for all 181 accounts. Lake Malawi
Aquaculture's 31 December 2025 row lists a drawdown dated 28 April 2026."],
  ["4", "<b>Takes disbursements from the disbursement schedule table,</b> not from the ledger.",
  "The schedule gives the planned dates. The date money actually went out, which is when interest starts to run, is in the
ledger and can be different.",
  "Ebenezer Midian: five entries dated 15 July 2025 in Extract A; the ledger shows the same five amounts paid out on 8
August 2025. Of the 35 accounts with ledger disbursements in 2025, the totals agree on 30 and the number of entries on 28."],

```

[5", "Counts the rows of the instalment chart for the number of instalments, and takes the earliest chart date as the first repayment date.",

"The chart appears to start with a row at the loan's start date. That row is counted as an instalment and reported as the first repayment.",

"Where a chart exists, the count is the tenor in months plus one on the rows we checked. The first repayment date equals the loan start date on 63 of 128 rows. 53 accounts have no chart rows at all.",

[6", "Reads the reschedule table for the restructure date.", "If that table is empty, restructures are not being recorded in E-Banker's reschedule function, and the history lives only in Finance's spreadsheet.",

"Blank on every row.",

[7", "Translates only three status codes (Active, Closed, Dormant). Any other code is passed through as it is.", "The meaning of the other codes still has to come from the vendor.",

"H on 21 accounts and F on 3.",

[8", "Removes the sign from the balance, and reads the principal balance only.", "An overpaid loan would look the same as one that is owed. Any interest balance held separately is not shown. The column is also headed \"opening balance\" although it is the balance at the date.", ""],

[9", "Tests the balance date as on or before the as-at date with the time of day removed.", "If balance dates carry a time of day, a balance recorded during the as-at day itself is missed and the day before is used instead. We do not yet know whether the dates carry a time.", "Cannot be tested from the file. Run 11 answers it.",

[10", "Does not limit write-offs and closures to the date.", "A write-off made after the date appears in an earlier snapshot.", ""],

[11", "Joins the moratorium period and its unit into text, and leaves out the moratorium type.", "Values arrive as \"3 M\" or \"0\", and we cannot tell a principal-only moratorium from a full one.", "Both formats appear.",

[12", "Reads no fee table at all.", "Fees are the main reason an effective rate differs from the contract rate.", "No fee fields.",

], [0.04, 0.34, 0.30, 0.32])

H2("The repayment frequency in the two snapshots")

P("The two snapshots disagree on repayment frequency for 42 accounts. The 1 January 2025 snapshot shows Monthly on all 181 " "accounts. The 31 December 2025 snapshot shows Monthly on 139, Half-Yearly on 10, Quarterly on 4, Yearly on 3 and blank on 25.")

P("The script we were sent does not use the date to find the frequency, and it leaves an account with no instalment plan " "blank. On its own it would not produce Monthly for every account. We do not know the cause. A different version of the " "procedure, a change in the data between the two runs (they were about an hour apart), or an edit after export are all " "possible. We have asked MAIIC for the procedure's version history and a record of the two runs. Until that is settled, " "the frequency in the 1 January 2025 snapshot should not be relied on.")

----- 5 Extract B

H1("5. Extract B: the cash flow register")

H2("What the script does")

P("Each run is given a new run number. The script then gathers three kinds of row for the date range it is given:")

BUL([

"Actual rows: every ledger posting made on the loan's own GL code, with a running balance. It names each row from " "its transaction type code: 301 Disbursement, 302 Principal, 303 Interest, 305 Principal+Interest, 340 and 341 Penalty, " "900 and 901 Fee, and anything else \"Other/Adjustment\".",

"Write-off rows: taken from the write-off register.",

"Scheduled rows: rows of the instalment chart that have not yet been fully recovered.",

])

H2("What is good about it")

BUL([

"The running balance is built from the loan's ledger history on its own GL code before the date range is applied, so a " "one-year extract does not restart the balance at zero. At 31 December 2025 it agrees with the balance in Extract A, " "which comes from a different table, on 110 of 111 accounts. The exception is Zaithwa Farms (1,225,683.03 against " "3,053,316.65). That is one checkpoint, not proof. The balance is only right if the ledger history is complete, the " "opening position is in it, and every relevant posting sits on the loan's own GL code. None of those is yet confirmed.",

"Every row carries a reference back to the ledger.",

"The script is honest about its limits. It marks every estimated figure in a note and lists its own known limitations at the end.",

"Its date range already runs to 31 December 2026 by default, so later periods need no rewrite.",

])

H2T("Where it falls short")

TAB(["#", "What the script does", "Why it matters", "What the delivered file shows"], [

[1", "Removes the direction of every amount. The ledger holds signed amounts; the script outputs each one without its sign.",

"A disbursement and a repayment look the same. Direction has to be worked out from the type label or from how the balance moves.", "No negative value anywhere in 2,478 rows.",

[2", "Presents interest charged as if it were a cash flow. Type 303 appears to be the monthly interest charge added to the loan.",

"For the effective interest rate, the cash flows are the money lent and the money repaid. Interest charged is not cash. Counting it as a receipt would count the interest twice.",

"987 of the 994 interest rows we could test raise the running balance; 485 of 501 repayment rows reduce it. On Ifracon Limited, interest of 21,497,946.96 on 31 January 2025 takes the balance up to 788,531,241.57, and a repayment of 53,589,482.23 on 18 February takes it down to 734,941,759.34.",

[3", "Estimates the split of each repayment (type 305) between principal and interest with a formula: balance, times the loan's rate today, times the days since the previous posting.",

"The formula uses today's rate, not the rate at the time, and counts days from the last posting of any kind. The interest has in any case already been charged separately. The split has no meaning, and the engine does not need it: it needs the amount, the date and the direction.",

"539 rows carry the \"estimated\" note.",

[4", "Puts every other transaction type under \"Other/Adjustment\" with no sign and no type code.",

"The label covers different things that cannot be told apart. Some rows reduce interest and are netted off in Extract C. Others raise the balance and are something else. What each one is needs the signed postings and the vendor's code list.",

"31 rows totalling MKW 832,653,206. Of the 30 we could test, 18 raise the balance, 9 reduce it and 3 could not be placed. On five accounts these rows equal the B to C difference exactly; on Happie Foods only 2,018,700.44 of 9,018,700.44 does (Appendix B).",

[5", "Reads only postings on the loan's own GL code.", "Fees, penalties or suspended interest posted to other GL codes for the same loan are never seen.", "2 fee rows in 2,478.",

[6", "Includes only instalments not yet fully recovered, inside the date range, at their full original amounts.",

"These rows are neither the schedule nor the amount still owed. Fully paid instalments are left out. A part-paid instalment is shown in full, because the amount already recovered is not subtracted and not shown. They must not be used as remaining payments until the recovery against each instalment is supplied and checked.",

"No balance on any of the 775 scheduled rows. On 14 of them the principal and interest parts do not add up to the total.",

[7", "Selects postings between the start date and the end date.", "If posting dates carry a time of day, postings

made during the last day are left out. Extract C uses the same test.", "Cannot be tested from the file. Run 11 answers it."),
["8", "Builds the reference from an internal row number.", "It can be traced in the database, but it is not a voucher number a user can look up on screen. The script's own note says the voucher number field was blank.", ""],
["9", "Takes write-offs from the register only.", "There is no ledger posting to check them against, and a write-off that is also in the ledger would appear twice.", ""],
], [0.04, 0.34, 0.30, 0.32])

----- 6 Extract C

H1("6. Extract C: interest posted by loan")
H2("What the script does")
P("Each run is given a new run number, and earlier runs are kept. The script adds up ledger postings of six transaction " "types (120, 303, 308, 309, 310 and 311) made on the loan's own GL code. It reverses their sign so that interest charged " "shows as positive, and totals them for each loan for each month. It also writes a yearly total row for each loan into the " "same table.")
H2("What is good about it")
BUL([
"It keeps the signs when adding, so reversals and adjustments are netted off.",
"Each row lists the ledger references it was built from.",
"For 2025 it agrees with the interest rows of Extract B on 101 of 107 accounts. Both extracts read the same ledger table, " "so this shows they are consistent with each other. It is not independent confirmation of the income figure.",
"Within the latest run there are no repeated rows for the same loan, GL code and month.",
])
H2T("Where it falls short")
TAB(["#", "What the script does", "Why it matters", "What the delivered file shows"], [
["1", "Keeps every earlier run in the same table. The file sent to us was exported without choosing one run.",
"Any total taken from the file as delivered counts 2025 three times.",
"Three runs (see the table below). 1,012 loan-months appear three times with identical amounts."],
["2", "Writes monthly rows and yearly total rows together.", "Adding the whole column counts every amount twice.",
"All rows add up to twice the monthly rows."],
["3", "Measures one side only. It adds up interest charged to loan accounts. It does not read the interest income GL codes.",
"It is not a reconciliation, and this report does not perform one. Its total still has to be agreed to the movement on the income codes in the trial balance. Interest charged to a loan but credited to suspense would be counted here as income.", ""],
["4", "Leaves out accounts with no matching posting in the period.",
"An absent account can mean three different things: no interest was posted; interest was posted where the script does not look (another GL code, another transaction type, a scheme outside the name rule); or the account was missed. The script cannot tell these apart, so an absent account is not a verified zero.",
"61 of 181 accounts are absent: 46 closed, 10 active, 4 dormant and 1 with status H (Appendix C)."],
["5", "Uses six type codes whose meanings we have not been given.", "The balance movements suggest 303 is the monthly charge and that the others include reductions, but what each one is has to come from the vendor.", ""],
["6", "Groups by the date of posting, and selects postings between the start and end date.", "Interest posted late lands in the month it was posted, not the month it was earned. If dates carry a time of day, postings on the last day are left out.", "The catch-up postings we found in February 2026."],
], [0.04, 0.34, 0.30, 0.32])
TAB(["Run number", "Generated", "Rows", "Accounts", "Period covered", "2025 interest, MWK"], [
["7", "3 August 2026", "1,119", "107", "2025", "2,554,390,697"],
["25", "3 August 2026", "1,119", "107", "2025", "2,554,390,697"],
["41", "25 August 2026", "1,758", "120", "January 2025 to July 2026", "2,554,390,697"],
["As delivered", "", "3,996", "", "", "7,663,172,090"],
], [0.15, 0.17, 0.10, 0.12, 0.26, 0.20], keep=True)
P("Run 41 is the one to use. Its January to July 2026 total is MWK 2,739,597,510, which appears once and is not affected.")

----- 7 dates and export

H1("7. Where the dates were scrambled, and how files should come back")
P("In September we reported that dates were transposed in every extract, with the day and month swapped. The scripts hold " "proper database dates from start to finish, so the damage does not happen inside them. It happens afterwards, when the " "results are copied out. We believe Excel is the cause but have not seen the export step.")
P("\Send it as CSV\ is not enough to prevent it. A CSV opened in Excel can still have its dates re-read and its account " "numbers stripped of their leading zeros. So every file should follow the same rules:")
BUL([
"CSV, comma separated, UTF-8, with one header row.",
"Dates written year first: 2026-10-05. Dates with a time: 2026-10-05 14:30:00.",
"Account numbers exactly as stored, as text, with their leading zeros.",
"Amounts as plain numbers: a dot for the decimal point, no thousands separators, and the minus sign kept.",
"Not opened and re-saved in Excel at any point before it reaches us.",
])
P("The mixed dates in the files we hold remain a limit on the evidence in this report. We avoided tests that depend on " "dates where we could, and used the ledger's own posting order instead.")

----- 8 three findings

H1("8. The three findings, and what to do about each")
P("These are the three points from the summary, with the action each one calls for.")
TAB(["Finding", "What to do now"], [
["Extract C counts 2025 three times when the delivered file is added up.",
"Use run 41 only, and monthly rows only. We have now read our engine's import: it skips the yearly rows and does not load a repeat of the same loan, month and GL code, so the three runs would not have been loaded three times. We have not inspected the loaded data itself. Any 2025 figure taken from the file by hand should still be checked."],
["Fixed and Variable appear to be the wrong way round in Extract A.",
"Do not use the label. Our engine currently stores a loan's rate type from it, which needs changing (section 13). Until Interest Policy is extracted, rate behaviour should be taken from the monthly loan books."],
["\Interest\ rows in Extract B are charges, not receipts.",
"Our 24 September research report described these rows as cash received, and we will correct it. Our engine currently counts them as cash collected, which also needs changing (section 13). Cash out is the disbursements and cash in is the repayments."],
], [0.36, 0.64], keep=True)

----- 9 settled / open

H1("9. What the scripts settle, and what is still open")
P("The questions below are the ones we raised in September, plus those raised in review, with what the scripts tell us " "about each and the query that would close it. Run numbers refer to section 11.")
TAB(["Question", "What the scripts tell us", "Status", "Query that closes it"], [

```

["Is the instalment chart stored, or worked out on screen?", "Stored, in a table, with the principal and interest split. 53
accounts have no rows.", "Settled", "Run 13 confirms the columns"],
["Why is the number of instalments always the tenor plus one?", "It is a count of chart rows, and the chart appears to
begin with a row at the start date.", "Explained, to confirm", "Run 13"],
["Do the disbursement tranches come through?", "Partly. 58 of the 314 populated rows list two to ten entries, so our
September statement that every row had one was wrong. But the entries come from the disbursement schedule, with planned dates,
not from the ledger.", "Corrected", "Runs 12 and 14"],
["Why is the restructure date blank everywhere?", "The script reads the reschedule table, which appears to be empty.",
"Explained, to confirm", "Run 8"],
["Why is there no debit or credit direction in Extract B?", "The script strips the sign from every amount.", "Explained",
"Run 6"],
["Why are 61 accounts missing from Extract C?", "Accounts with no posting that matches the script's filters produce no row.
Whether each one is a true zero is not established.", "Partly explained", "Runs 7 and 10, then MAIIC"],
["Why were the dates scrambled?", "Not in the scripts; in the export.", "Explained, to confirm", "None needed"],
["Which scheme is each account on?", "The scheme identifier is already read by the script. It only needs to be output.",
"Available", "Run 4"],
["Where is Interest Policy held?", "No script reads it.", "Open", "Runs 4 and 5"],
["Where are fees held against a loan?", "No script reads a fee table, and both ledger scripts look at the loan's own GL
code only.", "Open", "Runs 7, 21 and 22"],
["What do transaction types 120 and 308 to 311 mean?", "The script names eight codes. These five are grouped as interest in
Extract C and as \"Other\" in Extract B.", "Open", "Run 6, then the vendor"],
["What do status codes H and F mean?", "The script passes them through without a meaning.", "Open", "Run 9, then the
vendor"],
["Is there a table of rate changes?", "No script reads one.", "Open", "Runs 21 and 22"],
["Does the scheme name rule leave any loans out?", "Cannot be told from the scripts.", "Open", "Run 10"],
["Do posting dates carry a time of day?", "The scripts' date tests would drop last-day postings if they do.", "Open", "Run
11"],
["Which schema do the table names point to, and can the login read the tables?", "Not shown by the scripts. More than one
schema may hold a table of the same name.", "Open", "Runs 1 to 3"],
["Why does the repayment frequency differ between the two snapshots?", "The script we hold does not explain it.", "Open",
"MAIIC: procedure versions and run record"],
], [0.27, 0.37, 0.12, 0.24])

# ----- 10 why dictionary first
H1("10. Why the data dictionary queries have to come first")
P("A data dictionary is the database's own list of what it holds: every table, every column in each table, the kind of data "
  "in each column, and how the tables link to one another. Oracle keeps this list automatically. Reading it does not touch "
  "any customer or transaction record.")
P("Everything we know about how E-Banker is built comes from the three scripts: thirteen table names, and only the columns "
  "those scripts happen to use. The fields we still need are, by definition, the ones the scripts do not use. The manual "
  "tells us that Interest Policy is on the Scheme Master screen, but the name on a screen is not the name of a column, and a "
  "query can only ask for a column by its real name.")
H2("Three reasons the order matters")
BUL([
  "<b>An amended extract cannot be written against a column we cannot name.</b> A query that guesses a column name fails when
  "
  "it is run. Each failed attempt costs a round trip between Dupleix, MAIIC and the vendor. The August extracts took two "
  "deliveries, three weeks apart, to reach their present state.",
  "<b>One pass answers many questions.</b> The same listing shows where Interest Policy sits, whether the instalment chart "
  "carries a balance, whether there is a fee table, whether there is a rate history table, and what links a loan back to "
  "the origination system. Asking for these one at a time is how the last two months were spent.",
  "<b>It is the lowest-risk request we can make.</b> The queries only read. The dictionary queries read Oracle's catalogue, "
  "not MAIIC's data, and they take minutes to run.",
], numbered=True)
H2("Why the question queries are still needed afterwards")
P("The dictionary gives structure, not meaning. It will tell us that a column called TRANTYPE exists and holds a number. It "
  "will not tell us what 308 means, or whether a repayment is stored as a positive or a negative amount. Vendor databases "
  "rarely carry written descriptions. So the second round asks the data itself: which codes are in use, how often, in which "
  "direction, and what a single well-understood loan looks like in every table.")
BOX(["<b>In short:</b> the dictionary tells us what can be asked for. The question queries tell us what the answers mean. "
  "Only then can the three extracts be rewritten once, correctly."])

# ----- 11 the queries
H1("11. The queries, in the order to run them")
P("The run numbers below match the file <b>RUN ORDER - data dictionary queries by priority.sql</b>. Each query in that file "
  "is headed with the reason for it and the file name to save its result under. The file begins with two session settings "
  "(run 0) that make dates and numbers display in the agreed format; they change nothing in the database.")
H2("Tier 1: answers the open questions on Extracts A, B and C")
TAB(["Run", "Query", "What it returns, in plain language", "The question it answers"], [
  ["1", "DD_01", "Whether this login can see the thirteen tables, and who owns them", "The gate check. If it returns nothing,
we need the owner's name. If it lists more than one owner, we need to know which is the live schema."],
  ["2", "DD_17", "Who is logged in, which schema that login's table names point to, and who owns the three extract
procedures", "Which schema do the queries, and the procedures themselves, actually read?"],
  ["3", "DD_18", "A one-row read of each of the thirteen tables", "Can this login read the tables directly? Being able to run
a procedure does not prove it."],
  ["4", "DD_10", "Every setting held on the MAIIC and FInES schemes", "Where is Interest Policy? What are the real Floating
Flag values? Is there a scheme code?"],
  ["5", "DD_03", "Every column of every table: the dictionary itself", "What is each field called and where does it sit?"],
  ["6", "DD_12", "Each transaction type code, how often it is used, and whether its amounts are positive or negative", "Which
way does each posting move the balance? What are the unexplained codes used for?"],
  ["7", "DD_13", "Every GL code the loan accounts post to, by transaction type", "Where do fees, penalties and suspended
interest go? Is interest posted anywhere the extracts do not look?"],
  ["8", "DD_15", "The number of rows in the smaller loan tables", "Is the reschedule table empty?"],
  ["9", "DD_11", "The account status codes in use, with counts", "How many accounts carry H and F?"],
  ["10", "DD_09", "Every scheme, with the number of accounts on it", "Does the \"MAIIC or FInES\" name rule leave any loan
scheme out?"],
  ["11", "DD_19", "How many ledger, balance, instalment and write-off dates carry a time of day", "Could postings on the last
day of a period be dropped by the extracts' date tests?"],
], [0.07, 0.10, 0.42, 0.41])
H2("Tier 2: one sample loan, every column")
P("Runs 12 to 20 return every row held for one loan in each table: the ledger on all GL codes, the instalment chart, the ")

```

```

"disbursement schedule, the account and loan masters, the instalment plan, the balance history, and any reschedule or "
"write-off. The loan is Ebenezer Midian, one of the ten sample loans. We chose it because we already know two things about "
"it: the ledger shows five amounts totalling MWK 314,900,900 paid out on 8 August 2025, and its disbursement schedule "
"shows the same five amounts dated 15 July 2025. The result can be checked the moment it arrives.")
H2("Tier 3: the rest of the dictionary")
P("Runs 21 to 27 find the tables nobody has mentioned (any table carrying the loan account number, and tables named for "
"rates, fees, charges, audit or applications), and list the remaining code values, the keys and indexes that show how "
"tables join, the full table list, and the existing reports that already read the loan tables. None of these holds up the "
"amended extracts.")
H2("Practical notes for whoever runs them")
BUL([
    "Every query is read-only. Nothing is created, changed or deleted.",
    "Run them with the same login that runs the three extract procedures, and run the two session settings first.",
    "Every catalogue result carries the owner of each table, so that results from two schemas cannot be mixed up.",
    "Run one query at a time and return each result under the rules in section 7.",
    "Run 8 counts a table (LOAN_WRITEOFF_TRANS) that we know only from a comment in the cash flow script. If it does not "
    "exist, remove that line and run the query again.",
    "The vendor may prefer not to release the full listing of tables and columns (runs 5 and 26). The other queries still "
    "give us most of what we need.",
    "We wrote the queries without access to the database, so they have not been run. One or two may need a small correction "
    "the first time.",
])

# ----- 12 script changes
H1("12. Suggested changes to MAIIC's three extract scripts")
P("These are the changes we suggest to the scripts themselves, with the line of each file they apply to. They are small "
"edits to code that already exists. Several depend on the Tier 1 results and are marked. We suggest the vendor or ICT "
"makes them; we can draft the amended scripts once runs 1 to 7 are back.")
H2("FACILITY_REGISTER.txt (Extract A)")
TAB(["Line", "What it does now", "Suggested change"], [
    ["44", "Translates the Floating Flag: F to 'Fixed', I to 'Variable'", "Output the raw flag value as stored. Add "
    "Interest Policy as its own column once run 4 or 5 gives its name."],
    ["30, 89", "Reads the scheme but outputs only its name", "Also output the scheme identifier (" + M("s.scheme_mst_id") + ") "
    "and the scheme's other settings."],
    ["34 to 42", "Sums and lists the disbursement schedule", "Keep it, headed as planned disbursements. Add the actual "
    "drawdowns from the ledger (type 301 postings), each with its date, amount and reference."],
    ["52 to 57", "Counts the chart rows and takes the earliest chart date", "Leave out the opening row once run 13 confirms it. "
    "Better, supply the whole chart as its own extract."],
    ["43, 59, 60 to 68", "Reads the rate, closure date and write-offs as they stand today", "Either limit them to the as-at "
    "date, or head the columns 'as at run date' so the file cannot be misread."],
    ["69 to 70", "Translates status A, C and D only", "Output the raw status code. Add a meaning column once the code list is "
    "supplied."],
    ["77", "Removes the sign from the balance", "Keep the sign. Add any interest balance held separately."],
    ["99 to 102", "Finds the latest balance dated on or before the as-at date", "If run 11 shows dates carry a time, test for "
    "dates before the start of the next day, so the whole as-at day is included."],
    ["119", "Selects loans by scheme name beginning MAIIC or FINES", "Confirm with run 10 that no loan scheme is left out. "
    "Leave out the equity-type schemes, or flag them."],
], [0.12, 0.38, 0.50])
H2("CASHFLOW_REGISTER.txt (Extract B)")
TAB(["Line", "What it does now", "Suggested change"], [
    ["55 to 80", "Outputs every amount and the balance without its sign", "Output the signed amount as stored, and a debit or "
    "credit column."],
    ["44 to 54", "Replaces the transaction type code with a label", "Output the raw code as well as the label."],
    ["58 to 62, 67 to 70", "Estimates the principal and interest split of type 305", "Remove the estimate. If E-Banker stores "
    "the real split anywhere, output that; if not, leave the columns empty."],
    ["34", "Reads only postings on the loan's own GL code", "Add the postings on every other GL code for the same loan accounts, "
    "or supply a separate fee and charge extract. Run 7 shows what exists."],
    ["85, 105, 126", "Selects dates between the start and end date", "If run 11 shows dates carry a time, test for dates from "
    "the start date up to, but not including, the day after the end date."],
    ["107 to 127", "Lists instalments not fully recovered, at full original amounts", "Replace with a separate instalment chart "
    "extract: every row, paid or not, with the amount recovered against each."],
    ["2 to 3", "Defaults to 1 January 2025 onward", "Run it from October 2024, when E-Banker went live."],
], [0.16, 0.36, 0.48])
H2("INTEREST_RECON.txt (Extract C)")
TAB(["Line", "What it does now", "Suggested change"], [
    ["9 to 15", "Adds each run to the table and keeps the earlier ones", "Export the latest run only (" + M("WHERE run_id = "
    "...") + ") , or clear earlier runs for the same period first."],
    ["58 to 61", "Writes monthly rows and yearly total rows together", "Export monthly rows only, or put the yearly totals in a "
    "separate file."],
    ["37", "Adds six transaction types into one figure", "Show the amount for each type in its own column, so that charges and "
    "reductions can be seen separately."],
    ["35", "Reads only postings on the loan's own GL code", "Confirm with run 7 that no interest is posted elsewhere."],
    ["16 to 21, 33 to 34", "Produces no row for an account with no matching posting", "List every in-scope account, with a zero "
    "and a note where nothing was found."],
    ["38", "Selects dates between the start and end date", "As for Extract B, if dates carry a time."],
    ["2 to 3", "Defaults to 1 January 2025 onward", "Run it from October 2024 to date, and agree its total to the interest "
    "income codes in the trial balance."],
], [0.16, 0.36, 0.48])

# ----- 13 engine code
H1("13. Suggested changes to our own engine's code")
P("The review also looked at how our EIR engine loads these extracts. We then read the relevant files ourselves, in the "
"MAICC-IFRS9 repository on the branch " + M("eir_revenue_recognition") + ", on 5 October 2026. We did not run the engine or "
"inspect the data already loaded, and <b>nothing in the code has been changed</b>. These are suggestions for the Duplex "
"team, listed with the most important first.")
TAB(["#", "Where", "What the code does now", "Suggested change", "Priority"], [
    ["1", M("EirRevenueService.php") + " line 22 and the cash received routine",
    "Counts Extract B rows labelled Interest, Principal+Interest and Fee as cash collected in the month.",
    "Take Interest out of the list of collections. Those rows are interest charged to the loan; for 2025 they total MWK 2,639, "
    "557,380. Once Extract B carries signed amounts and type codes, classify cash by code and sign, not by label. Re-run any period "
    "whose cash figure came from the imported 2025 rows."],

```

```

    "High"],
    ["2", M("ContractMasterImport.php") + " line 72; " + M("ContractMasterImportService.php") + " rate type routine",
    "Loads Extract A's Fixed or Variable label as the loan's rate type (Variable becomes FLOATING). The ECL discount rate and
time-phased ECL services read that field.",
    "Stop deriving the rate type from this label. Keep the delivered label in its own field as evidence. Derive the rate type
from Interest Policy once it is extracted, and until then from observed repricing in the loan books. Correct the code comment
that refers to 204 variable-rate facilities.",
    "High"],
    ["3", M("ContractTransactionImportService.php") + " lines 66 to 71; " + M("RemainingScheduleImportService.php") + "; " +
M("ScheduleWorkflowService.php") + " comparison",
    "Loads Extract B's scheduled rows as a \"remaining schedule\" and compares their totals with our generated schedule from
the earliest due date onward.",
    "Treat these rows as what they are: instalments not fully recovered inside the extract's date range, at full original
amounts. Rename them accordingly. Limit the comparison to matching due dates inside that range, or switch it off until the full
chart extract arrives. Never feed them into a calculation.",
    "Medium"],
    ["4", M("ContractTransactionImportService.php") + " lines 78 to 85",
    "Stores each actual row's unsigned total and the estimated principal and interest parts. We found no other code that reads
those two parts.",
    "Add fields for the raw transaction type code, the signed amount, and a marker for an estimated split. Keep estimated
parts out of every calculation.",
    "Medium"],
    ["5", M("GLInterestImportService.php"),
    "Skips yearly rows and repeats of the same loan, month and GL code, keeping the first one it meets. Its description calls
the figure the interest income the ledger posted.",
    "Describe the figure as interest charged to loan accounts, not yet agreed to the income GL. Where a file holds several
runs, keep the latest run and report how many runs were in the file.",
    "Medium"],
    ["6", M("MappedFileReader.php") + " date routine, lines 420 to 454",
    "Accepts Excel date serials, then the declared format, then falls back to flexible parsing.",
    "For MAIIC extracts, accept year-first dates only. Hold the file when a date column mixes real dates with text, which is
exactly how the scrambled extracts arrive.",
    "Medium"],
    ["7", "GL reconciliation and coverage reporting",
    "Not traced in this review.",
    "Check that a loan with no row in the interest postings is reported as \"no posting found\" and not as zero interest. Keep
three states apart: posted, verified zero, and not covered.",
    "Medium"],
    ["8", M("ContractMasterImportService.php") + " line 281",
    "Stores Extract A's disbursement list as text on the contract.",
    "Take drawdown dates and amounts from the ledger disbursement rows. Keep Extract A's list as the planned schedule only. On
Ebenezer Midian the two differ by 24 days.",
    "Medium"],
    ["9", "Contract key in the import services",
    "Identifies a loan by account number. The sub-account is stored but is not part of the key.",
    "Key on account number and sub-account together. Every sub-account is 1 today, but the vendor says a restructure creates a
new one.",
    "Low"],
    ["10", "Use of Extract A terms",
    "Loads rate, status and terms from a snapshot as if they were as at the snapshot date.",
    "Record them as at the run date. Do not load the repayment frequency from the 1 January 2025 snapshot.",
    "Low"],
], [0.04, 0.21, 0.27, 0.40, 0.08])
NOTE("File and line references are to the branch as it stood on 5 October 2026. Item 1 follows from the finding that interest "
    "rows are charges, which is inferred and still to be confirmed with signed postings (section 2).")

# ----- 14 sequence
H1("14. The sequence from here")
BUL([
    "MAIIC runs Tier 1 (runs 1 to 11) and sends the results. Runs 1 to 7 are enough for us to begin.",
    "MAIIC sends the procedure version history and the record of the two Extract A runs, and a line of explanation for each of
the ten accounts in Appendix C.",
    "Dupleix drafts the amended extract scripts against the real table and column names (section 12), and makes the engine
changes in section 13.",
    "MAIIC runs Tiers 2 and 3 when convenient. They refine the work but do not hold it up.",
    "MAIIC runs the amended extracts. Dupleix loads them and reconciles interest to the trial balance. That reconciliation has
not yet been done.",
], numbered=True)

# ----- appendices
story.append(PageBreak())
H1("Appendix A. The tests behind this report")
P("Each statement about the delivered files was tested on the files themselves. These are the results. The scripts that "
    "produce them are in the Build files folder.")
TAB(["Test", "Result"], [
    ["Extract A: rows, accounts and snapshots", "362 rows; 181 accounts; two snapshots, 1 January 2025 and 31 December 2025. No
repeated rows for the same account, sub-account and snapshot. Sub-account is 1 on every row"],
    ["Extract A: rate label by product (31 December 2025 snapshot)", "Fixed: MAIIC Agricultural 16, MAIIC Industrial 41, MAIIC
Term Loan 13, Fines Investment Loan 4, Quasi Equity Investment 5. Variable: FINEs Agricultural 50, FINEs Industrial 51,
NASCOMEX Limited 1"],
    ["Extract A: fields compared between the two snapshots", "Interest rate, account status, sanctioned amount, tenor, number
of instalments, rate label, principal disbursed and tranche list: identical for all 181 accounts"],
    ["Extract A: repayment frequency in the two snapshots", "1 January 2025: Monthly 181. 31 December 2025: Monthly 139,
Half-Yearly 10, Quarterly 4, Yearly 3, blank 25. 42 accounts differ"],
    ["Extract A: Lake Malawi Aquaculture, 31 December 2025 snapshot", "Sanctioned 1,055,473,655; one tranche of 297,161,905
dated 28 April 2026"],
    ["Extract A: principal disbursed against sanctioned amount", "Populated on 157 accounts; equal to the sanctioned amount on
138"],
    ["Extract A: entries in the tranche list", "Populated on 314 of 362 rows. One entry on 256 rows; two to ten entries on 58"],
    ["Extract A tranche list against ledger disbursements in Extract B", "35 accounts have ledger disbursements in 2025. Total
amount agrees on 30; number of entries agrees on 28. Ebenezer Midian: 15 July 2025 in Extract A, 8 August 2025 in the ledger"],

```

["Extract A: first repayment date against loan start date", "The same on 63 of the 128 accounts with a first repayment date"],

["Extract A: accounts with no instalment chart rows", "53"],

["Extract A: account status (31 December 2025 snapshot)", "Active 86, Closed 46, Dormant 25, H 21, F 3"],

["Extract B: rows by kind", "2,478 rows, one run. Actual 1,703: Interest 1,034, Principal+Interest 539, Disbursement 97, Other/Adjustment 31, Fee 2. Scheduled 775, across 90 accounts"],

["Extract B: negative amounts", "None"],

["Extract B: direction of each row, from the change in the running balance", "Tested on every actual row after the first for its account. Interest: 987 raise the balance, 6 reduce it, 1 not placed. Principal+Interest: 485 reduce, 4 raise, 12 not placed. Disbursement: 63 raise, 1 not placed. Other/Adjustment: 18 raise, 9 reduce, 3 not placed. 16 rows in the file are out of posting order, which accounts for some of the rows not placed"],

["Extract B: Other/Adjustment rows", "31 rows totalling 832,653,206.42"],

["Extract B closing balance against Extract A balance at 31 December 2025", "111 accounts have both. 110 agree within MWK 1, taking the row with the highest posting number as the close. Exception: Zaithwa Farms, 1,225,683.03 against 3,053,316.65"],

["Extract B: scheduled rows", "0 of 775 carry a balance. Principal plus interest equals the total on 761 of 775"],

["Extract C: runs in the delivered file", "Run 7: 1,119 rows. Run 25: 1,119 rows. Run 41: 1,758 rows. No repeated loan, GL code and month within run 41"],

["Extract C: loan-months", "1,564 distinct loan-months; 1,012 appear three times and 552 once"],

["Extract C: run 7 against run 41 for 2025", "1,012 loan-months in both; amounts differ on none"],

["Extract C: 2025 interest", "All runs added together 7,663,172,090; run 41 alone 2,554,390,697"],

["Extract B interest rows against Extract C (run 41), 2025", "107 accounts in both. 101 agree exactly. Extract B total 2, 639,557,380; difference 85,166,683.31"],

["Extract C: accounts in Extract A with no row in Extract C", "61: Closed 46, Active 10, Dormant 4, H 1"],

], [0.42, 0.58]]

H1("Appendix B. The six accounts where Extract B and Extract C disagree")

P("For 2025, the interest rows in Extract B exceed the interest in Extract C on six accounts. On five of them the difference " equals the account's \"Other/Adjustment\" rows in Extract B. On Happie Foods it equals one of the two such rows: the " 2,018,700.44 row reduces the balance, while the 7,000,000 row raises it. This agreement in amounts does not establish " what the postings are. That needs the raw signed postings and the vendor's meaning for each transaction type.")

TAB(["Account", "Customer", "Extract B interest", "Extract B other", "Extract C interest", "Difference"], [

["000104430000068", "Nyamunyamu Processors Limited", "131,899,213.80", "69,330,809.54", "62,568,404.26", "69,330,809.54"],

["000104430000002", "Edge View Academy Limited", "117,296,781.00", "10,457,152.33", "106,839,628.67", "10,457,152.33"],

["000104430000073", "Saile Financial Services Limited Company", "90,522,529.35", "2,395,117.82", "88,127,411.53", "2,395, 117.82"],

["000104420000062", "Happie Foods Limited", "38,421,398.75", "9,018,700.44", "36,402,698.31", "2,018,700.44"],

["000104420000052", "Kambewu Organic Fertilizer Investments", "42,329,779.82", "913,541.02", "41,416,238.80", "913,541.02"],

["000104420000032", "African Honey Products Industries", "29,893,579.18", "51,362.16", "29,842,217.02", "51,362.16"],

], [0.17, 0.27, 0.15, 0.13, 0.15, 0.13], keep=True)

NOTE("The six differences add up to MWK 85,166,683.31, which is the whole difference between the two extracts for 2025.")

H1("Appendix C. Active accounts with no interest posted in Extract C")

P("These ten accounts are marked Active in Extract A but have no interest row in Extract C between January 2025 and July " 2026. That is not the same as a verified zero. Some may not have been drawn yet, some are not loans, and for some the " interest may be posted where the script does not look. Each one needs a line of explanation from MAIIC.")

TAB(["Account", "Customer", "Product", "Sanctioned amount, MWK"], [

["000104470000004", "NASCOMEX Limited", "NASCOMEX Limited", "2,500,000,000"],

["000104430000081", "Pinnacle Financial Services Limited", "MAIIC Industrial Loans", "1,000,000,000"],

["000104430000088", "Anchor Processors Limited", "MAIIC Industrial Loans", "300,000,000"],

["000104430000083", "JAT Group Ltd", "MAIIC Industrial Loans", "209,270,000"],

["000104460000111", "Ecogen Limited", "FInES Industrial Loans", "105,060,000"],

["000105810000004", "Ecogen Limited", "Fines Investment Loan", "105,060,000"],

["000105810000003", "Ecogen Limited", "Fines Investment Loan", "100,000,000"],

["000104440000005", "Belk Inc", "Quasi Equity Investment", "69,000,000"],

["000104420000003", "Micholless Creamery", "MAIIC Agricultural Loans", "20,000,000"],

["000104450000012", "Maluso Cooperative Union", "FInES Agricultural Loans", "11,511,500"],

], [0.20, 0.36, 0.24, 0.20], keep=True)

H1("Appendix D. Terms used in this report")

TAB(["Term", "Meaning"], [

["Script, stored procedure", "A set of database instructions saved inside the database and run on request."],

["Query", "A question put to the database. A read-only query returns data and changes nothing."],

["Table, column", "A table is a list of records of one kind, such as accounts. A column is one field in that list, such as the interest rate."],

["Data dictionary, catalogue", "The database's own list of its tables and columns."],

["Schema, owner", "A named area of the database that holds a set of tables. Two schemas can hold tables with the same name."],

["Scheme", "In E-Banker, the product variant an account is opened on. Settings such as Interest Policy are held on the scheme."],

["GL code", "The general ledger account a posting is made to. Each loan product has its own."],

["Transaction type code", "A number E-Banker gives each posting to say what kind it is, such as 301 for a disbursement."],

["Run number", "A number the script gives each time it is run, so that the results of different runs can be told apart."],

["Snapshot, as at date", "The position on a stated date, as opposed to the position today."],

["Instalment chart, EMI chart", "The loan's repayment schedule as held in E-Banker."],

["Effective interest rate (EIR)", "The rate that spreads all of a loan's cash flows, including fees, evenly over its life. It is what IFRS 9 requires interest income to be recognised at."],

["Engine", "The Duplex software that calculates the effective interest rate and the related income for MAIIC."],

], [0.28, 0.72], bold_first=True)

H1("Appendix E. The independent review, and what changed in version 2")

P("The first version of this report, the query pack and the record of the working session were reviewed on 5 October 2026 " by a second AI system (Codex). The reviewer re-ran our main tests on the delivered files and reproduced the figures. It " raised eight corrections. We accept all eight. The review is filed in the engine's repository under "

+ M("docs/reviews") + ".")

TAB(["#", "The review's comment", "Our response", "What changed"], [

["1", "Scheduled rows are not the unpaid remainder. The script exports full original amounts and does not subtract the amount recovered.", "Accepted. Our wording was wrong.", "Section 5, row 6; section 12; section 13, item 3."],

["2", "The running balance is not proven correct. It depends on complete history, the opening position, posting scope and agreement with independent balances.", "Accepted. We tested it: 110 of 111 accounts agree with Extract A at year end.", "Section 5, first bullet; Appendix A."],

["3", "No interest rows is not a verified zero.", "Accepted.", "Section 6, row 4; section 9; Appendix C; section 13, item

```

7. "],
["4", "Final-day date handling should be checked. Dates with a time of day could drop last-day postings.", "Accepted. A new
query tests it.", "Sections 4 to 6; run 11 (DD_19); section 12."],
["5", "Agreement in total does not establish what the adjustments are. Happie Foods leaves MWK 7 million unexplained.",
"Accepted, and it goes further: the \"Other/Adjustment\" rows total MWK 832.7 million and most do not reduce interest.",
"Section 5, row 4; Appendix B."],
["6", "Schema and privilege assumptions. Several schemas may hold ACMaster; running a procedure does not prove read
access.", "Accepted.", "Owner added to every catalogue result; runs 2 and 3 (DD_17, DD_18)."],
["7", "The cause of the repayment-frequency differences is not established.", "Accepted. We now state the facts and three
possible causes, and have asked for the run history.", "Section 4, closing paragraphs; section 9."],
["8", "Define the export contract. CSV alone does not protect dates or account numbers.", "Accepted.", "Section 7; the
header of both SQL files; session settings at run 0."],
], [0.04, 0.40, 0.30, 0.26])
P("The review also made three points about our engine's code, and our own reading added more. These are in section 13. "
"Two of our findings went beyond the review: the engine counts Extract B interest rows as cash collected, and it stores "
"the rate type from the label that appears to be inverted.")
P("As the review asked, this version separates what was tested from what is inferred and what is pending (section 2), and "
"states that it is not a completed income reconciliation or a validated cash flow schedule.")

```

```

# ----- build
def footer(canvas, doc):
    canvas.saveState()
    canvas.setStrokeColor(LINE); canvas.setLineWidth(0.5)
    canvas.line(18 * mm, 14 * mm, A4[0] - 18 * mm, 14 * mm)
    canvas.setFont("Seg", 8); canvas.setFillColor(GREY)
    canvas.drawString(18 * mm, 9.5 * mm, "MAIIC E-Banker extract scripts: analysis · Duplex Institute · 5 October 2026 ·
Version 2")
    canvas.drawRightString(A4[0] - 18 * mm, 9.5 * mm, f"Page {doc.page}")
    canvas.restoreState()

OUT = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf"
doc = BaseDocTemplate(OUT, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=20 * mm,
    title="MAIIC E-Banker extract scripts: analysis (version 2)", author="Duplex Institute",
    subject="What the three raw query scripts do, where they fall short, and which queries to run next")
doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, id="f",
    leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)],
onPage=footer)])
doc.build(story)
print("written", OUT)

```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\maaic_report_v2.py (file state is current in your context – no need to Read it back)

CLAUDE (2026-10-05 15:46:55 UTC)

Report rewritten as version 2 with the review's corrections and the code suggestions; building the PDF and checking the layout now.

Tool call: Bash

```

description: Build version 2 of the PDF report and render pages for a layout check
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && cp maaic_report.py maaic_report_v1_backup.py && cp maaic_report_v2.py maaic_report.py && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maaic_report.py && ./songenv/Scripts/python -c "
import pypdfium2 as pdfium
from PIL import Image
p = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf"
pdf = pdfium.PdfDocument(p); n = len(pdf); print('pages', n)
ims = [pdf[i].render(scale=0.5).to_pil() for i in range(n)]
w, h = ims[0].size; cols = 6; rows = (n + cols - 1) // cols
sheet = Image.new('RGB', (w * cols, h * rows), 'white')
for i, im in enumerate(ims): sheet.paste(im, ((i % cols) * w, (i // cols) * h))
sheet.save('contact3.png'); print(sheet.size)
for i in (0, 12): pdf[min(i, n - 1)].render(scale=1.0).to_pil().save(f'v2p{i+1}.png')
from pypdf import PdfReader
t = ' '.join((pg.extract_text() or '') for pg in PdfReader(p).pages)
for needle in ['Version 2', '13. Suggested changes to our own engine', 'EirRevenueService.php', 'Appendix E', '27 queries', 'unpaid remainder', 'first five']:
    print(needle, '->', t.count(needle))
"
{
  "timeout": 30000
}

```

Result of Bash

```
written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf
pages 18
(1788, 1263)
Version 2 -> 20
13. Suggested changes to our own engine -> 1
EirRevenueService.php -> 1
Appendix E -> 2
27 queries -> 2
unpaid remainder -> 1
first five -> 0
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Read

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\contact3.png
```

Result of Read

[image output: not reproduced in this transcript]

Tool call: Read

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\v2p13.png
```

Result of Read

[image output: not reproduced in this transcript]

Tool call: Bash

```
description: Widen the priority column, rebuild the PDF and refresh the saved build files
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp\htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && sed -i 's/\], \[0\.\04, 0\.\21, 0\.\27, 0\.\40, 0\.\08\]]/, [0.\04, 0.\21, 0.\26, 0.\38, 0.\11]]/' maiic_report.py &&
grep -c "0.\04, 0.\21, 0.\26, 0.\38, 0.\11" maiic_report.py && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python maiic_report.py && ./
songenv/Scripts/python -c "
import pypdfium2 as pdfium
p = r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf'
pdf = pdfium.PdfDocument(p); print('pages', len(pdf))
pdf[13].render(scale=1.0).to_pil().save('v2p14.png')
" && DST="/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC/Build
files" && cp maiic_report.py runorder.py patch_sql.py review_checks.py "$DST/" && ls "$DST"
{
  "timeout": 300000
}
```

Result of Bash

```
1
written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf
pages 18
README.md
build_transcript.py
maiic_check.py
maiic_check2.py
maiic_check3.py
maiic_check4.py
maiic_exact.py
maiic_report.py
maiic_tr.py
patch2.py
patch_report.py
patch_sql.py
review_checks.py
runorder.py
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Read

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\v2p14.png
```

Result of Read

[image output: not reproduced in this transcript]

Tool call: Read

```
file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
Build files\README.md
```

Result of Read

```
1  # Build files: MAIIC raw query scripts analysis
2
3  Prepared 5 October 2026 by Duplex Institute, in a Claude Code session. These are the Python scripts behind the files in
the folder above, kept so the work can be reviewed and re-run.
4
5  ## What each file does
6
7  | File | Purpose | Output |
8  |---|---|---|
9  | `maiiic_check.py` | Prints the layout of the delivered Extract A, B and C workbooks | Screen only |
10 | `maiiic_check2.py` | First evidence tests: rate label by product, snapshot comparison, disbursed against sanctioned,
Extract B interest against Extract C, accounts missing from C | Screen only |
11 | `maiiic_check3.py` | Finds the three runs stacked in the Extract C file | Screen only |
12 | `maiiic_check4.py` | Exact evidence figures for the report: snapshot comparison, status counts, ledger sample, the ten
active accounts with no interest | Screen only |
13 | `maiiic_exact.py` | Exact figures for the report's appendix tables (the six accounts where B and C differ, run totals) |
Screen only |
14 | `maiiic_tr.py` | Re-test of the disbursement tranche claim, Extract A against the ledger rows in Extract B | Screen only
|
15 | `runorder.py` | Builds the run-order SQL file from `MAIIC_EBanker_Data_Dictionary_Queries.sql` | `RUN ORDER - data
dictionary queries by priority.sql` |
16 | `maiiic_report.py` | Builds the plain-language analysis report | `MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf` |
17 | `build_transcript.py` | Exports the relevant part of the Claude session log |
`Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf` and `.md` |
18 | `patch_report.py`, `patch2.py` | One-off edits that were applied to `maiiic_report.py` during the session | Already
applied |
19
20 ## Things a reviewer should know
21
22 - **Nothing here has touched the Oracle database.** The evidence tests read only the Excel extracts MAIIC delivered in
August 2026. The 24 SQL queries in the folder above were written without database access and have not been run.
23 - **`maiiic_report.py` is the final version.** The two patch scripts have already been applied to it. Running them again
will stop on their own checks and change nothing. `runorder.py` here already carries the corrected reason for run 11 (DD_16c),
which was first fixed by hand in the SQL file.
24 - **`maiiic_check3.py` stops with an error near the end** (a grouping line that fails on this version of pandas) after
printing its main results. In the session the remaining checks were run with that line replaced; the results are in the
transcript.
25 - **`maiiic_check2.py` contains a snapshot comparison that over-reports differences** on columns with mixed date formats.
`maiiic_check4.py` repeats the comparison properly and is the one the report relies on.
26 - **Paths are fixed to the PC the work was done on** (the OneDrive MAIIC project folder and the Windows fonts folder).
They need changing to run anywhere else.
27 - **`build_transcript.py` reads the Claude Code session log** from the user profile on that PC, so it can only be re-run
there.
28
29 ## To re-run
30
31 Python 3 with `pandas`, `openpyxl`, `reportlab` and `pypdf` (`pypdfium2` and `pillow` only if page previews are wanted).
32
33 1. `python maiiic_check4.py`, `python maiiic_exact.py` and `python maiiic_tr.py` reproduce the figures quoted in the report.
34 2. `python maiiic_report.py` rebuilds the PDF report.
35
```

Tool call: Write

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Build files\README.md

--- content ---

Build files: MAIIC raw query scripts analysis

Prepared 5 October 2026 by Dupleix Institute, in a Claude Code session. These are the Python scripts behind the files in the folder above, kept so the work can be reviewed and re-run.

****Version 2.**** The report and the SQL query pack were revised on 5 October 2026 after an independent review by Codex. The review is filed in the engine repository at `MAICC-IFRS9\docs\reviews\MAIIC-Claude-Review-2026-10-05.md`. The response to each comment is Appendix E of the report.

What each file does

File	Purpose	Output
---	---	---
`maiic_check.py`	Prints the layout of the delivered Extract A, B and C workbooks	Screen only
`maiic_check2.py`	First evidence tests: rate label by product, snapshot comparison, disbursed against sanctioned, Extract B interest against Extract C, accounts missing from C	Screen only
`maiic_check3.py`	Finds the three runs stacked in the Extract C file	Screen only
`maiic_check4.py`	Exact evidence figures for the report: snapshot comparison, status counts, ledger sample, the ten active accounts with no interest	Screen only
`maiic_exact.py`	Exact figures for the report's appendix tables (the six accounts where B and C differ, run totals)	Screen only
`maiic_tr.py`	Re-test of the disbursement tranche claim, Extract A against the ledger rows in Extract B	Screen only
`review_checks.py`	Tests added in response to the review: direction of each transaction type from the running balance, Extract B closing balance against Extract A, the Other/Adjustment rows, repayment frequency in the two snapshots, key checks	Screen only
`patch_sql.py`	The version 2 changes to the SQL pack: owner carried through every catalogue result, the export contract, session settings, and three new queries (DD_17, DD_18, DD_19)	Already applied to `MAIIC_EBanker_Data_Dictionary_Queries.sql`
`runorder.py`	Builds the run-order SQL file (27 queries) from `MAIIC_EBanker_Data_Dictionary_Queries.sql`	`RUN ORDER - data dictionary queries by priority.sql`
`maiic_report.py`	Builds the plain-language analysis report, version 2	`MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf`
`build_transcript.py`	Exports the relevant part of the Claude session log	`Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf` and `.md`
`patch_report.py`, `patch2.py`	One-off edits to the version 1 report script	Superseded. `maiic_report.py` was rewritten for version 2 and these no longer apply

Things a reviewer should know

- ****Nothing here has touched the Oracle database.**** The evidence tests read only the Excel extracts MAIIC delivered in August 2026. The 27 SQL queries in the folder above were written without database access and have not been run.
- ****No application code was changed.**** Section 13 of the report suggests changes to the MAICC-IFRS9 engine. They come from reading the code on branch `eir\_revenue\_recognition` on 5 October 2026. The engine was not run and its loaded data was not inspected.
- ****`patch\_sql.py` has already been applied.**** Running it again stops on its own checks and changes nothing.
- ****The session transcript covers version 1 only.**** It ends before the Codex review. The review, the extra tests and the version 2 changes are not in it.
- ****`maiic\_check3.py` stops with an error near the end**** (a grouping line that fails on this version of pandas) after printing its main results. In the session the remaining checks were run with that line replaced; the results are in the transcript.
- ****`maiic\_check2.py` contains a snapshot comparison that over-reports differences**** on columns with mixed date formats. `maiic\_check4.py` and `review\_checks.py` repeat the comparison properly and are the ones the report relies on.
- ****`review\_checks.py` tests direction in file order.**** Sixteen rows in Extract B are out of posting order, which is why a few rows are reported as not placed. The closing-balance test in the report (110 of 111) takes the row with the highest posting number; `review\_checks.py` prints the file-order version (106 of 111), and the posting-order version was run inline in the session.
- ****Paths are fixed to the PC the work was done on**** (the OneDrive MAIIC project folder and the Windows fonts folder). They need changing to run anywhere else.
- ****`build\_transcript.py` reads the Claude Code session log**** from the user profile on that PC, so it can only be re-run there.

To re-run

Python 3 with `pandas`, `openpyxl`, `reportlab` and `pypdf` (`pypdfium2` and `pillow` only if page previews are wanted).

1. `python maiic\_check4.py`, `python maiic\_exact.py`, `python maiic\_tr.py` and `python review\_checks.py` reproduce the figures quoted in the report.
2. `python runorder.py` rebuilds the run-order SQL file from the main SQL pack.
3. `python maiic\_report.py` rebuilds the PDF report.

Result of Write

The file C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Build files\README.md has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: Write

```
file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
Build files\review_balance_check.py
--- content ---
"""Extract B closing running balance against Extract A's balance at 31 December 2025.

The close is taken as the actual row with the highest posting number for the account, because
sixteen rows in the file are out of posting order. Reported in the analysis as 110 of 111.
"""
import warnings
import pandas as pd

warnings.filterwarnings("ignore")
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
A = pd.read_excel(D + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
B = pd.read_excel(D + r"\Extract B.xlsx", sheet_name="Sheet1", dtype={"LOAN_ACCOUNT_NUMBER": str})
key = ["LOAN_ACCOUNT_NUMBER", "SUB_ACCOUNT_NO"]
year_end = A["AS_OF_DATE"].astype(str).str[:10] == "31-12-2025"].set_index(key)

actual = B[B.SCHEDULED_ACTUAL_FLAG == "Actual"].copy()
actual["posting_no"] = actual.GL_POSTING_REF.str.split("-").str[-1].astype(int)

rows = []
for k, g in actual.groupby(key):
    a_balance = year_end.OPENING_BALANCE.get(k)
    if pd.isna(a_balance):
        continue
    in_file_order = g.BALANCE_AFTER_TRANSACTION.iloc[-1]
    in_posting_order = g.sort_values("posting_no").BALANCE_AFTER_TRANSACTION.iloc[-1]
    rows.append((k[0], year_end.CUSTOMER_NAME[k], a_balance, in_file_order, in_posting_order))
df = pd.DataFrame(rows, columns=["account", "customer", "extract_a_balance", "b_close_file_order", "b_close_posting_order"])

print("accounts with both balances:", len(df))
print("agree within MWK 1, last row in file order: ", int((((df.extract_a_balance - df.b_close_file_order).abs() < 1).sum())))
print("agree within MWK 1, highest posting number: ", int((((df.extract_a_balance - df.b_close_posting_order).abs() <
1).sum())))
print("\nexceptions on the posting-order test:")
print(df[(df.extract_a_balance - df.b_close_posting_order).abs() >= 1].round(2).to_string(index=False))
```

Tool call: Bash

```
description: Run the reproducible balance test, update the README and confirm where the review file now sits
--- command ---
cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Build
files" && PYTHONIOENCODING=utf-8 "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/
ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad/songenv/Scripts/python" review_balance_check.py 2>&1 | cut -c1-200 && sed -i 's/
takes the row with the highest posting number; `review_checks.py` prints the file-order version (106 of 111), and the
posting-order version was run inline in the session./takes the row with the highest posting number, which
`review_balance_check.py` reproduces; `review_checks.py` prints the file-order version (106 of 111).;/ s/| `patch_sql.py` | The
version 2 changes/| `review_balance_check.py` | Extract B closing balance against Extract A at 31 December 2025, in posting
order (the 110 of 111 figure) | Screen only |\n| `patch_sql.py` | The version 2 changes;/ s/`python maiic_tr.py` and `python
review_checks.py` reproduce/`python maiic_tr.py`, `python review_checks.py` and `python review_balance_check.py` reproduce/'
README.md && grep -c "review_balance_check" README.md && cd /c/xampp/htdocs/MAIIC-IFRS9 && git status --short | head -5 && cd /
c/xampp/htdocs/Stress-Testing-App && git status --short | grep -i "MAIIC" ; echo "stress-testing repo clean of the review file:
$?"
{
  "timeout": 120000
}
```

Result of Write

File created successfully at: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Build files\review_balance_check.py (file state is current in your context – no need to Read it back)

Result of Bash

```
accounts with both balances: 111
agree within MWK 1, last row in file order:    106
agree within MWK 1, highest posting number:    110

exceptions on the posting-order test:
   account   customer  extract_a_balance  b_close_file_order  b_close_posting_order
00010445000015  Zaithwa Farms          3053316.65          1225683.03          1225683.03
3
?? docs/reviews/
?? MAIIC-Claude-Review-2026-10-05.md
?? MAIIC-Claude-Review-2026-10-05.pdf
stress-testing repo clean of the review file: 0
```

Tool call: Bash

```
description: Compare the newly appeared review files with the copy already moved
--- command ---
ls -la --time-style=long-iso MAIIC-Claude-Review-2026-10-05.* && ls -la --time-style=long-iso /c/xampp/htdocs/MAIIC-IFRS9/docs/
reviews/ && diff <(tr -d '\r' < MAIIC-Claude-Review-2026-10-05.md) <(tr -d '\r' < /c/xampp/htdocs/MAIIC-IFRS9/docs/reviews/
MAIIC-Claude-Review-2026-10-05.md) && echo "IDENTICAL to the moved copy"; date "+%Y-%m-%d %H:%M"
```

Result of Bash

```
-rw-r--r-- 1 wadza 197609 5771 2026-10-05 17:45 MAIIC-Claude-Review-2026-10-05.md
-rw-r--r-- 1 wadza 197609 6306 2026-10-05 17:45 MAIIC-Claude-Review-2026-10-05.pdf
total 12
drwxr-xr-x 1 wadza 197609    0 2026-10-05 17:38 .
drwxr-xr-x 1 wadza 197609    0 2026-10-05 17:38 ..
-rw-r--r-- 1 wadza 197609 5914 2026-10-05 17:24 MAIIC-Claude-Review-2026-10-05.md
7c7
< Reviewed the supplied Claude session, the three Oracle extract procedures, the report builder and dictionary SQL pack. Re-ran
maiiic_check4.py, maiiic_exact.py and maiiic_tr.py against the delivered Excel extracts. Also inspected the transaction and
interest import services in C:\xampp\htdocs\MAICC-IFRS9. This reviews the supplied analysis session, rather than the entire
application. No Oracle queries were executed and no database records or application code were changed.
---
> Reviewed the supplied Claude session transcript, MAIIC's three Oracle extract procedures, the report builder and data
dictionary SQL pack. Re-ran `maiiic_check4.py`, `maiiic_exact.py` and `maiiic_tr.py` against the delivered Excel extracts. Also
inspected the relevant transaction and interest import services in `C:\xampp\htdocs\MAICC-IFRS9`. This is a review of the
supplied analysis session, not a full application audit. No Oracle queries were executed and no application code or database
records were changed.
9c9
< Claude's analysis is useful and its main figures reproduce, but several claims need revision before treating the report as
final.
---
> Claude's analysis is useful and its main numerical findings reproduce. Revise the claims below before treating the report as
final.
13,18c13,18
< - Extract A largely reads current loan terms and status for nominal historical snapshots; only the balance lookup is
explicitly dated.
< - Extract B removes transaction signs and supplies estimated repayment splits. Interest-charge rows must not automatically be
treated as cash receipts.
< - Extract C contains overlapping runs and both monthly and annual totals, creating double-counting risk.
< - The 2025 B-C difference reproduces exactly: MWK 85,166,683.31 across six accounts. The other 101 accounts agree exactly.
< - Ebenezer has five entries in both extracts, with different dates: 15 July 2025 in the schedule and 8 August 2025 in the
ledger. The corrected finding is supported.
< - The report correctly acknowledges that loan-account postings still need reconciliation to income GL accounts.
---
> - Extract A reads current loan terms and status for nominal historical snapshots; only the balance lookup is explicitly dated.
> - Extract B removes transaction signs, estimates repayment splits, and includes interest charges that must not automatically
be treated as cash receipts.
> - Extract C contains overlapping runs plus monthly and annual totals.
> - The 2025 B-C difference is MWK 85,166,683.31 across six accounts. The other 101 accounts agree exactly.
> - Ebenezer has five entries in both extracts: 15 July 2025 in the schedule versus 8 August 2025 in the ledger. The corrected
finding is supported.
> - The report correctly acknowledges that loan-account postings require reconciliation to income GL accounts.
22c22
< 1. High: scheduled rows are not the unpaid remainder. CASHFLOW_REGISTER.txt, lines 115-126, selects instalments not fully
recovered but outputs their full original principal, interest and instalment amounts. It does not subtract inst_recovered.
Partially paid instalments can overstate remaining payments. Revise the wording and require verified recovery allocation before
treating these amounts as remaining payments.
---
> 1. **High: scheduled rows are not the unpaid remainder.** `CASHFLOW_REGISTER.txt`, lines 115-126, selects instalments not
fully recovered but exports the full original principal, interest and instalment amounts. It does not subtract
`inst_recovered`. Revise the report wording and require a verified recovery allocation before using these amounts as remaining
payments.
24c24
< 2. High: running-balance correctness is not established. Summing available ledger history before filtering dates is sensible,
but correctness also requires complete history, a valid opening position, verified posting scope and agreement to independent
balances. Replace the unconditional claim that the balance is right.
---
> 2. **High: running-balance correctness is not established.** The report says that building the balance from the whole history
makes it right. This remains conditional on complete ledger history, a valid opening position, correct posting scope and
reconciliation to independent balances. Replace the unconditional claim and request those checks.
26c26
< 3. High: absence of interest rows is not automatically a verified zero. Confirm account scope and extraction completeness
first. Distinguish no postings found, verified zero and unresolved coverage. Obtain explanations for the ten active accounts.
---
> 3. **High: absence of interest rows is not automatically a verified zero.** Confirm account scope and extraction completeness
first. Distinguish no postings found, verified zero and unresolved coverage. The ten active accounts need explanations, as the
report already requests.
28c28
< 4. Medium: verify final-day date handling. B and C use inclusive BETWEEN end dates; A uses transaction_date <=
TRUNC(p_as_of_date). If source dates contain times, later final-day postings can be excluded. Establish intended date semantics
and test boundary cases.
---
> 4. **Medium: verify final-day date handling.** B and C use inclusive `BETWEEN` end dates; A uses `transaction_date <=
TRUNC(p_as_of_date)`. If source dates contain times, later final-day postings can be excluded. Establish intended date
semantics and test boundary cases before amending the procedures.
30c30
< 5. Medium: aggregate agreement does not establish adjustment semantics. Five of the six differences match Other/Adjustment
totals. Happie Foods has MWK 9,018,700.44 of Other/Adjustment against a difference of MWK 2,018,700.44, leaving MWK 7 million
outside that explanation. Retain the appendix qualification and qualify the stronger main-report wording. Obtain raw signed
postings and vendor transaction-code definitions.
---
> 5. **Medium: aggregate agreement does not establish adjustment semantics.** Five of the six B-C differences match the Other/
Adjustment totals. Happie Foods has MWK 9,018,700.44 of Other/Adjustment against a difference of MWK 2,018,700.44, leaving MWK
7 million outside that explanation. Retain the appendix qualification and qualify the stronger wording in the main report.
Obtain raw signed postings and vendor transaction-code definitions.
32c32
< 6. Medium: clarify schema and privilege assumptions. Multiple accessible schemas can contain ACMASTER, while several
catalogue outputs omit owner identifiers. Include owner identifiers and establish the intended schema. Verify direct SELECT
access; permission to execute a procedure does not itself establish access to its underlying tables.
```

```

---
> 6. Medium: clarify schema and privilege assumptions in the dictionary pack. Multiple accessible schemas can contain
ACMASTER, while several catalogue outputs omit owner identifiers. Carry owner identifiers through those outputs and establish
the intended schema. Verify direct SELECT access to business tables; permission to execute a procedure does not by itself
establish that access.
34c34
< 7. Qualify the cause of repayment-frequency differences. Forty-two accounts differ between snapshots, but this does not
establish that the procedure changed between runs. Request procedure versions and run provenance.
---
> 7. Qualify the cause of repayment-frequency differences. Forty-two accounts differ between snapshots, but this does not
establish that the procedure changed between runs. Request procedure versions/run provenance and retain other explanations as
unresolved.
36c36
< 8. Define the export contract. CSV alone does not establish unambiguous dates or preserve account numbers when opened in
Excel. Specify date/timestamp formats, identifier handling, encoding and numeric conventions. Mixed dates remain an evidence
limitation.
---
> 8. Define the export contract. CSV alone does not establish unambiguous dates or protect account numbers when opened in
Excel. Specify date/timestamp formats, account identifiers as text, encoding and numeric conventions. Mixed dates remain a
material evidence limitation.
40,42c40,42
< - app/Services/Eir/GLInterestImportService.php already skips annual totals and detects duplicate monthly entries. Its
description still calls Extract C interest income; qualify that pending validation against income GL accounts and the trial
balance.
< - app/Services/Eir/ContractTransactionImportService.php stores the estimated principal/interest components delivered in B.
Trace downstream consumption and distinguish estimates from verified cash movements before calculation use.
< - Additional checks found one B run, no duplicate A facility/snapshot keys, no accounts with multiple subaccounts in A, and
no duplicate run-41 C monthly keys at account/subaccount/GL/year/month grain. Reusable checks should nevertheless preserve
composite facility keys.
---
> - `app/Services/Eir/GLInterestImportService.php` already skips annual totals and detects duplicate monthly entries. Its
description still calls Extract C interest income; qualify this pending validation against the income GL and trial balance.
> - `app/Services/Eir/ContractTransactionImportService.php` stores the delivered estimated principal/interest components. Trace
downstream consumption and ensure estimates are distinguished from verified cash movements before calculation use.
> - The reproduced workbook has one B run, no duplicate A facility/snapshot keys, no accounts with multiple subaccounts in A,
and no duplicate run-41 C monthly keys at account/subaccount/GL/year/month grain. Nevertheless, reusable evidence checks should
retain the composite facility key rather than assume account number alone is always sufficient.
46,50c46,50
< - Project documents: C:\Users\wadza\OneDrive\2026\Projects\MAIIC
< - Source procedures: 2. Documents from clients\Raw Query Scripts\FACILITY_REGISTER.txt, CASHFLOW_REGISTER.txt and
INTEREST_RECON.txt
< - Analysis deliverables: 2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC
< - Final report source: Query Requests to MAIIC\Build files\maic_report.py
< - Application repository: C:\xampp\htdocs\MAICC-IFRS9
---
> - Project documents: `C:\Users\wadza\OneDrive\2026\Projects\MAIIC`
> - Source procedures: `2. Documents from clients\Raw Query Scripts\FACILITY_REGISTER.txt`, `CASHFLOW_REGISTER.txt`,
`INTEREST_RECON.txt`
> - Analysis deliverables: `2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC`
> - Final report source: `Query Requests to MAIIC\Build files\maic_report.py`
> - Application repository: `C:\xampp\htdocs\MAICC-IFRS9`
54c54
< Address each comment as accepted, qualified or disputed, with supporting source/data evidence. Update the report and query
pack as appropriate. Separate reproduced observations from assumptions and pending vendor confirmations. The reproduced numbers
support the diagnosis; they do not yet establish a completed income reconciliation or validated contractual cash-flow schedule.
---
> For each comment, state whether it is accepted, qualified or disputed, with supporting source/data evidence. Update the
report and query pack as appropriate. Separate reproduced observations from assumptions and pending vendor confirmations. Do
not describe the work as a completed income reconciliation or a validated contractual cash-flow schedule.
2026-10-05 17:48

```

Tool call: Bash

description: Move the updated review files into the application repository and list the final deliverables

```

--- command ---
mv -f MAIIC-Claude-Review-2026-10-05.md MAIIC-Claude-Review-2026-10-05.pdf /c/xampp/htdocs/MAICC-IFRS9/docs/reviews/ && ls -la
--time-style=long-iso /c/xampp/htdocs/MAICC-IFRS9/docs/reviews/ | tail -2 && git status --short | grep -ci "MAIIC-Claude"; cd /
c/xampp/htdocs/MAICC-IFRS9 && git status --short | head -5 && ls -la --time-style=long-iso "/c/Users/wadza/OneDrive/2026/
Projects/MAIIC/2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC/" | awk '{print $5, $6, $7, $8, $9, $10, $11,
$12, $13, $14, $15, $16}' | tail -8

```

Result of Bash

```

-rw-r--r-- 1 wadza 197609 5771 2026-10-05 17:45 MAIIC-Claude-Review-2026-10-05.md
-rw-r--r-- 1 wadza 197609 6306 2026-10-05 17:45 MAIIC-Claude-Review-2026-10-05.pdf
0
?? docs/reviews/
0 2026-10-05 16:48 .
0 2026-10-05 15:35 ..
0 2026-10-05 17:48 Build files
258441 2026-10-05 16:47 Claude_Session_Transcript_Raw_Queries_2026-10-05.md
242350 2026-10-05 16:47 Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf
22821 2026-10-05 17:38 MAIIC_EBanker_Data_Dictionary_Queries.sql
147029 2026-10-05 17:47 MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf
23577 2026-10-05 17:39 RUN ORDER - data dictionary queries by priority.sql
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

```

Written for: MAIIC Finance and ICT and the Dupleix team (the report), and whoever runs the queries on E-Banker (the SQL pack).

The review file is now in the engine repository, and the report and query pack are saved in place as version 2 with all eight of Codex's corrections adopted. Nothing in the engine's code was changed and nothing was committed in either repository.

The review file

It is in C:\xampp\htdocs\MAICC-IFRS9\docs\reviews\. A newer copy of the review and a PDF of it appeared in the Stress-Testing-App folder while I was working. It has the same eight comments, reworded, so I moved both in and replaced the earlier copy.

The report (now 18 pages, same file name)

- **Scope statement on the cover:** it is not a completed income reconciliation or a validated cash flow schedule, as Codex asked.
- **New section 2, "How sure we are":** separates what was tested on the files, what is inferred, and what is waiting on MAIIC or the vendor.
- **Corrected wording** on the scheduled rows, the running balance, accounts with no interest, the "Other/Adjustment" rows and the repayment frequency.
- **New section 12:** suggested changes to MAIIC's three scripts, with the line numbers they apply to.
- **New section 13:** ten suggested changes to our engine's code, with file and line references and a priority for each.
- **Appendix E:** Codex's eight comments, our response to each, and where the report changed.

Two engine issues that are more serious than the review found

These come from reading the code on the `eir_revenue_recognition` branch; I did not run the engine or inspect loaded data.

- **Interest counted as cash:** `EirRevenueService.php` counts Extract B rows labelled "Interest" as cash collected. Those rows are interest charged to the loan, and for 2025 they total MWK 2,639,557,380.
- **Rate type from the inverted label:** the contract import stores each loan's rate type from Extract A's Fixed/Variable label, and the ECL discount rate services read that field.

Both are marked High in section 13. The first rests on the finding that interest rows are charges, which is still inferred until we see signed postings.

The query pack (now 27 queries)

- Every catalogue result now carries the schema owner.
- Three new queries: who is logged in and which schema the names point to, a direct read test on each table, and a check for time-of-day in the dates the extracts filter on.
- The header sets out how files should come back (dates, account numbers as text, number format), and two session settings at the top apply the date and number format.
- Run numbers have shifted: Tier 1 is now runs 1 to 11, and runs 1 to 7 are enough for us to start.

The queries are still untested against the database.

Not updated

- The session transcript covers version 1 only and ends before the review.
- The 24 September research report still carries the tranche and "cash received" statements that need correcting.

Files are in 2. Documents from `clients\Raw Query Scripts\Query Requests to MAIIC`:

- `MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf`
- `RUN ORDER - data dictionary queries by priority.sql`
- `MAIIC_EBanker_Data_Dictionary_Queries.sql`
- `Build files\README.md`

from this pack, do we have a query to check all the columns of the loan master file

Yes, in three places, depending on what you mean by "check":

- **The column list (structure):** run 5, DD_03, lists every column of every table in the schema, so the loan master's columns are in there. E-Banker splits the loan master across two tables: ACMASTER (the account master: account number, customer, GL code, rate, status, scheme) and ACLOANMASTER (the loan terms: sanctioned amount, tenor, expiry, moratorium). Filter the DD_03 result on those two table names.
- **The actual contents of every column for one loan:** run 15, DD_16a, returns the full ACMASTER row for Ebenezer Midian, and run 16, DD_16b, returns its full ACLOANMASTER row. Both use SELECT *, so every column comes back with its value.

There is no query that lists the loan master columns on their own. If Barry wants one that returns only those two tables without the full dictionary, this is DD_03 with a filter added:

```
SELECT col.owner, col.table_name, col.column_id AS column_no, col.column_name,
       col.data_type, col.data_length, col.data_precision, col.data_scale,
       col.nullable, col.num_distinct, col.num_nulls
FROM   all_tab_columns col
WHERE  col.owner IN (SELECT owner FROM all_tables WHERE table_name = 'ACMASTER')
AND    col.table_name IN ('ACMASTER', 'ACLOANMASTER')
ORDER BY col.table_name, col.column_id;
```

NUM_DISTINCT and NUM_NULLS are worth reading on this one: they show which columns are actually populated, which is the quickest way to spot whether something like an interest policy or margin field is in use at account level.

Thats great

Glad it helps. When the Tier 1 results come back from Barry, send them over and I'll start on the amended extract scripts.

"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Request Responses 6 Oct 2026\Duplex\Duplex" => These are the responses of the queries and we still have barry on teams. Please analyse and advise what barry can do

Tool call: PowerShell

description: List the query response files Barry sent

--- command ---

```
$d = "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Request Responses 6 Oct 2026\Duplex\Duplex"; Get-ChildItem -LiteralPath $d -Recurse | Select-Object Length, LastWriteTime, @{n='Name'; e={$_.FullName.Replace($d+'\','')}} | Format-Table -AutoSize -Wrap
```

Result of PowerShell

Length	LastWriteTime	Name
718	06/10/2026 15:28:03	DD_01_schema.csv
71688	06/10/2026 15:28:03	DD_02_tables.csv
3137718	06/10/2026 15:28:03	DD_03_columns.csv
220525	06/10/2026 15:28:03	DD_04_keys.csv
112766	06/10/2026 15:28:03	DD_05_indexes.csv
69375	06/10/2026 15:28:03	DD_06_account_key_tables.csv
9835	06/10/2026 15:28:03	DD_07_tables_by_subject.csv
95761	06/10/2026 15:28:03	DD_08_stored_code.csv
5525	06/10/2026 15:28:03	DD_09_schemes_all.csv
11895	06/10/2026 15:28:03	DD_10_scheme_settings.csv
275	06/10/2026 15:28:03	DD_11_account_status_codes.csv
891	06/10/2026 15:28:03	DD_12_transaction_types.csv
2853	06/10/2026 15:28:03	DD_13_gl_codes_posted.csv
566	06/10/2026 15:28:03	DD_14_other_codes.csv
264	06/10/2026 15:28:03	DD_15_row_counts.csv
3482	06/10/2026 15:28:03	DD_16a_sample_acmaster.csv
1800	06/10/2026 15:28:03	DD_16b_sample_acloanmaster.csv
1335	06/10/2026 15:28:03	DD_16c_sample_disbursement_schedule.csv
256088	06/10/2026 15:28:03	DD_16d_sample_instalment_chart.csv
4528	06/10/2026 15:28:03	DD_16e_sample_instalment_plan.csv
20162	06/10/2026 15:28:03	DD_16f_sample_ledger.csv
4399	06/10/2026 15:28:03	DD_16g_sample_balances.csv
617	06/10/2026 15:28:03	DD_16h_sample_reschedule.csv
383	06/10/2026 15:28:03	DD_16i_sample_writeoff.csv

Tool call: Bash

description: Read the small result files first

```
--- command ---
cd "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Request Responses 6 Oct 2026/
Dupleix/Dupleix" && for f in DD_01_schema.csv DD_11_account_status_codes.csv DD_12_transaction_types.csv
DD_13_gl_codes_posted.csv DD_14_other_codes.csv DD_15_row_counts.csv DD_16c_sample_disbursement_schedule.csv
DD_16h_sample_reschedule.csv DD_16i_sample_writeoff.csv DD_09_schemes_all.csv; do echo "=== $f"; cat "$f" | tr -d '\n' | head
-60; echo; done 2>&1 | cut -c1-220
```

Result of Bash

```
=== DD_01_schema.csv
"  ", "OWNER", "TABLE_NAME", "NUM_ROWS", "STATS_DATE"
"1", "VGCBS", "ACCOUNT_BALANCE", "75607", "2026-08-07"
"2", "VGCBS", "ACLOANMASTER", "7154", "2026-06-13"
"3", "VGCBS", "ACMASTER", "10862", "2026-09-28"
"4", "VGCBS", "CUMVOUCH", "395162", "2026-09-02"
"5", "VGCBS", "CUSTOMER_MAST", "4885", "2026-06-30"
"6", "VGCBS", "FIXLOAN_INST_CHART", "95609", "2026-08-03"
"7", "VGCBS", "GLMASTER", "533", "2026-10-05"
"8", "VGCBS", "LOAN_DISBURSMENT_SCHEDULE", "6880", "2026-06-13"
"9", "VGCBS", "LOAN_RESCHEDULE_DETAILS", "0", "2026-06-13"
"10", "VGCBS", "LOAN_WRITEOFF_MASTER", "0", "2026-06-13"
"11", "VGCBS", "LOAN_WRITEOFF_TRANS", "0", "2026-06-13"
"12", "VGCBS", "MULTI_INST_DETAILS", "8544", "2026-06-13"
"13", "VGCBS", "SCHEME_MASTER", "114", "2026-06-13"

=== DD_11_account_status_codes.csv
"  ", "STATUS_CODE", "ACCOUNTS", "EARLIEST_STATUS_DATE", "LATEST_STATUS_DATE"
"1", "A", "89", "2020-07-29", "2026-08-28"
"2", "C", "46", "2021-12-22", "2024-12-19"
"3", "D", "25", "2025-12-08", "2026-07-28"
"4", "F", "3", "2026-03-12", "2026-05-01"
"5", "H", "21", "2025-12-11", "2026-06-30"

=== DD_12_transaction_types.csv
"  ", "TRANTYPE", "POSTINGS", "POSITIVE_ROWS", "NEGATIVE_ROWS", "NET_AMOUNT", "ROWS_ON_LOAN_GL", "FIRST_POSTING", "LAST_POSTING"
"1", "120", "6", "6", "0", "85166683.31", "6", "2025-12-31", "2025-12-31"
"2", "300", "2", "2", "0", "22286434.32", "2", "2024-08-31", "2024-08-31"
"3", "301", "214", "0", "214", "-30888119103.87", "214", "2024-07-31", "2026-08-31"
"4", "303", "2159", "0", "2158", "-8202442901.56", "2159", "2022-09-01", "2026-08-31"
"5", "305", "1212", "1210", "0", "17516632447.5", "1212", "2024-07-31", "2026-09-30"
"6", "306", "37", "0", "37", "-1277458017.5", "37", "2024-11-05", "2026-09-29"
"7", "343", "2", "2", "0", "74454441.45", "2", "2026-03-12", "2026-03-12"
"8", "400", "1", "0", "1", "-21295000", "1", "2024-10-23", "2024-10-23"
"9", "401", "5", "5", "0", "535714287", "5", "2024-12-31", "2026-01-31"
"10", "900", "1", "0", "1", "-10350000", "1", "2025-05-06", "2025-05-06"
"11", "901", "1", "1", "0", "10350000", "1", "2025-05-12", "2025-05-12"

=== DD_13_gl_codes_posted.csv
"  ", "AC_GLCODE", "GLTITLE", "TRANTYPE", "POSTINGS", "NET_AMOUNT", "FIRST_POSTING", "LAST_POSTING"
"1", "1050101", "MAIIC Agricultural Loans", "120", "3", "2983603.62", "2025-12-31", "2025-12-31"
"2", "1050101", "MAIIC Agricultural Loans", "300", "1", "10244624.69", "2024-08-31", "2024-08-31"
"3", "1050101", "MAIIC Agricultural Loans", "301", "17", "-2096152586.64", "2024-07-31", "2025-09-05"
"4", "1050101", "MAIIC Agricultural Loans", "303", "299", "-1905193772.55", "2024-07-31", "2026-08-31"
"5", "1050101", "MAIIC Agricultural Loans", "305", "108", "3066629435.65", "2024-07-31", "2026-09-30"
"6", "1050101", "MAIIC Agricultural Loans", "306", "5", "-313715149.46", "2025-05-31", "2026-05-01"
"7", "1050101", "MAIIC Agricultural Loans", "900", "1", "-10350000", "2025-05-06", "2025-05-06"
"8", "1050101", "MAIIC Agricultural Loans", "901", "1", "10350000", "2025-05-12", "2025-05-12"
"9", "1050102", "MAIIC Industrial Loans", "120", "3", "82183079.69", "2025-12-31", "2025-12-31"
"10", "1050102", "MAIIC Industrial Loans", "301", "55", "-8987931809.17", "2024-07-31", "2026-06-30"
"11", "1050102", "MAIIC Industrial Loans", "303", "414", "-3327549422.9", "2022-09-01", "2026-08-31"
"12", "1050102", "MAIIC Industrial Loans", "305", "247", "7564857265.61", "2024-07-31", "2026-09-30"
"13", "1050102", "MAIIC Industrial Loans", "306", "4", "-401582668", "2024-12-23", "2026-09-29"
"14", "1050102", "MAIIC Industrial Loans", "400", "1", "-21295000", "2024-10-23", "2024-10-23"
"15", "1050201", "FInES Agricultural Loans", "300", "1", "12041809.63", "2024-08-31", "2024-08-31"
"16", "1050201", "FInES Agricultural Loans", "301", "36", "-2697890905", "2024-07-31", "2026-08-25"
"17", "1050201", "FInES Agricultural Loans", "303", "606", "-511994992.59", "2024-07-31", "2026-08-31"
"18", "1050201", "FInES Agricultural Loans", "305", "370", "2025394180.83", "2024-07-31", "2026-09-30"
"19", "1050201", "FInES Agricultural Loans", "306", "10", "-381262023.27", "2024-11-05", "2026-05-29"
"20", "1050202", "FInES Industrial Loans", "301", "64", "-5671966037.13", "2024-07-31", "2026-03-31"
"21", "1050202", "FInES Industrial Loans", "303", "764", "-820446999.35", "2024-07-31", "2026-08-31"
"22", "1050202", "FInES Industrial Loans", "305", "459", "2770639902.18", "2024-07-31", "2026-09-30"
"23", "1050202", "FInES Industrial Loans", "306", "15", "-48113128.01", "2025-02-25", "2026-03-31"
"24", "1050202", "FInES Industrial Loans", "343", "1", "9009942.39", "2026-03-12", "2026-03-12"
"25", "1050401", "MAIIC Term Loan", "301", "42", "-11434177765.93", "2025-11-07", "2026-08-31"
"26", "1050401", "MAIIC Term Loan", "303", "76", "-1637257714.17", "2025-12-31", "2026-08-31"
"27", "1050401", "MAIIC Term Loan", "305", "28", "2089111663.23", "2025-12-30", "2026-09-30"
"28", "1050401", "MAIIC Term Loan", "306", "3", "-132785048.76", "2026-03-31", "2026-08-31"
"29", "1050401", "MAIIC Term Loan", "343", "1", "65444499.06", "2026-03-12", "2026-03-12"
"30", "3065", "NASCOMEX Limited", "401", "5", "535714287", "2024-12-31", "2026-01-31"

=== DD_14_other_codes.csv
"  ", "CODE_COLUMN", "CODE_VALUE", "ROWS_USING_IT"
"1", "ACLOANMASTER.IMOROTORIUM_UNIT", "M", "6743"
"2", "ACLOANMASTER.IMOROTORIUM_UNIT", "Y", "1"
"3", "ACLOANMASTER.IMOROTORIUM_UNIT", "", "414"
"4", "ACLOANMASTER.PMOROTORIUM_UNIT", "M", "6762"
"5", "ACLOANMASTER.PMOROTORIUM_UNIT", "", "396"
"6", "MULTI_INST_DETAILS.INSTALLMENT_PAY_MODE", "H", "49"
"7", "MULTI_INST_DETAILS.INSTALLMENT_PAY_MODE", "M", "3956"
```

```
"8","MULTI_INST_DETAILS.INSTALLMENT_PAY_MODE","Q","25"
"9","MULTI_INST_DETAILS.INSTALLMENT_PAY_MODE","X","404"
"10","MULTI_INST_DETAILS.INSTALLMENT_PAY_MODE","Y","9"
```

```
=== DD_15_row_counts.csv
"","TABLE_NAME","ROW_COUNT"
"1","LOAN_RESCHEDULE_DETAILS","0"
"2","LOAN_WRITEOFF_MASTER","0"
"3","LOAN_WRITEOFF_TRANS","0"
"4","LOAN_DISBURSMENT_SCHEDULE","6890"
"5","FIXLOAN_INST_CHART","97249"
"6","MULTI_INST_DETAILS","8600"
"7","SCHEME_MASTER","114"
```

```
=== DD_16c_sample_disbursement_schedule.csv
"","LOAN_DISB_DET_ID","NEW_AC_NUMBER","SUB_ACCOUNT_NO","DISB_SCHEDULE_DATE","DISB_SCHEDULE_AMOUNT","STATUS_FLAG","REMARKS",
"DELETE_FLAG","OPERATION_ID","OPERATION_DATE","ENTER_USER_ID","ENTER_DESC","DELETE_USER_ID","D
"1","7055","0001044300000084","1","7/15/2025","5900000.00","D","","N","N4","8/8/2025","19","0-1-08/08/
2025-15:21:30-192.168.002.001","","","18","0-1-08/08/2025-17:34:44-192.168.002.001","","","7628"
"2","7053","0001044300000084","1","7/15/2025","298480500.00","D","","N","N2","8/8/2025","19","0-1-08/08/
2025-15:18:39-192.168.002.001","","","18","0-1-08/08/2025-17:34:06-192.168.002.001","","","7628"
"3","7057","0001044300000084","1","7/15/2025","900.00","D","","N","N6","8/8/2025","19","0-1-08/08/2025-15:23:35-192.168.002.001",
","","","18","0-1-08/08/2025-17:35:35-192.168.002.001","","","7628"
"4","7056","0001044300000084","1","7/15/2025","1519500.00","D","","N","N5","8/8/2025","19","0-1-08/08/
2025-15:22:41-192.168.002.001","","","18","0-1-08/08/2025-17:35:09-192.168.002.001","","","7628"
"5","7054","0001044300000084","1","7/15/2025","9000000.00","D","","N","N3","8/8/2025","19","0-1-08/08/
2025-15:20:02-192.168.002.001","","","18","0-1-08/08/2025-17:34:26-192.168.002.001","","","7628"
```

```
=== DD_16h_sample_reschedule.csv
"","LOAN_RESCH_DET_ID","TRANSACTION_DATE","OPBR_MST_ID","NEW_AC_NUMBER","SUB_ACCOUNT_NO","SANCTION_AMT","OLD_SANC_AMT",
"S_PERIOD_DAYS","S_PERIOD_MONTHS","S_PERIOD_YEARS","S_PERIOD_WEEKS","INSTALLMENT_AMT","OLD_INST_AM
```

```
=== DD_16i_sample_writeoff.csv
"","LOAN_WRITEOFF_MST_ID","OP_BRMST_ID","TRANSACTION_DATE","NEW_AC_NUMBER","SUB_AC_NUMBER","WRITEOFF_DATE","WRITEOFF_AMOUNT",
"REMARKS","WORKABLE_FLAG","WORKABLE_MARK_DATE","STATUS_FLAG","STATUS_CHANGE_DATE","ACTIVE_FL
```

```
=== DD_09_schemes_all.csv
"","SCHEME_MST_ID","SCHEME_NAME","IN_EXTRACT_SCOPE","ACCOUNTS"
"1","13","BILLS DISCOUNTING","N","0"
"2","57","BILLS DISCOUNTING - USD","N","0"
"3","40","BORROWING CC/DD","N","0"
"4","67","BORROWING COMPOUNDING","N","0"
"5","78","BORROWING INVESTMENT MONTHLY","N","0"
"6","79","BORROWING INVESTMENT SIMPLE (SHORT TERM)","N","0"
"7","66","BORROWING SIMPLE","N","0"
"8","39","BORROWING TERM LOAN","N","0"
"9","16","BRANCH ADJUSTMENT ASSETS SIDE","N","0"
"10","11","BRANCH ADJUSTMENT LIABILITY SIDE","N","0"
"11","61","BUSINESS LOAN","N","0"
"12","12","CASH CREDIT LIMIT","N","0"
"13","56","CASH CREDIT LIMIT - USD","N","0"
"14","63","CORPORATE LOAN","N","0"
"15","21","CREDIT GUARANTEE","N","0"
"16","32","CURRENT DEPOSIT INSTITUTIONS","N","0"
"17","42","CURRENT DEPOSIT INSTITUTIONS USD","N","0"
"18","4","CURRENT DEPOSITS INDIVIDUAL","N","0"
"19","43","CURRENT DEPOSITS INDIVIDUAL USD","N","0"
"20","25","DAILY DEPOSITS","N","0"
"21","2","DAM DUPPAT","N","0"
"22","23","DEAD STOCK","N","0"
"23","139","Equity ","N","0"
"24","9","FIXED DEPOSIT HALFYEARLY INTT PAID SCHEME","N","0"
"25","53","FIXED DEPOSIT HALFYEARLY INTT PAID SCHEME - USD","N","0"
"26","3","FIXED DEPOSIT MONTHLY INTT PAID SCHEME","N","0"
"27","49","FIXED DEPOSIT MONTHLY INTT PAID SCHEME - USD","N","0"
"28","7","FIXED DEPOSIT QUARTERLY CUMULATIVE","N","0"
"29","51","FIXED DEPOSIT QUARTERLY CUMULATIVE - USD","N","0"
"30","8","FIXED DEPOSIT QUARTERLY INTT SCHEME","N","0"
"31","52","FIXED DEPOSIT QUARTERLY INTT SCHEME - USD","N","0"
"32","6","FIXED DEPOSIT SIMPLE (SHORT TERM)","N","0"
"33","50","FIXED DEPOSIT SIMPLE (SHORT TERM) - USD","N","0"
"34","10","FIXED DEPOSIT YEARLY INTT PAID SCHEME","N","0"
"35","54","FIXED DEPOSIT YEARLY INTT PAID SCHEME - USD","N","0"
"36","90","FinES AGRICULTURAL LOANS","Y","51"
"37","91","FinES Agricultural Loans(Manual Int)","Y","0"
"38","92","FinES Industrial Loans","Y","51"
"39","93","FinES Industrial Loans(Manual Int)","Y","0"
"40","94","FinES Investment Loans","Y","1"
"41","95","FinES Investment Loans(Manual Intt)","Y","0"
"42","20","FLEXI DEPOSITS SCHEME","N","0"
"43","140","Fines Investment Loans","Y","4"
"44","62","HOUSING LOAN MICROFINANCE","N","0"
"45","77","INV TIME DEPOSIT","N","0"
"46","19","INVESTMENT IN GOVERNMENT SECURITY (365 Days)","N","0"
"47","28","INVESTMENT IN SHARES OF OTHER INSTITUTIONS","N","6"
"48","30","INVESTMENT OTHERS","N","0"
"49","80","INVESTMENT WITH CALL LENDING","N","0"
"50","31","INVESTMENT WITH SHORT AND CALL DEPOSITS","N","1"
"51","29","INVESTMENTS FIXED DEPOSITS WITH OTHER BANKS","N","0"
"52","27","INVESTMENTS IN GOVERNMENT SECURITIES (360DAYS)","N","0"
"53","70","INVESTMENTS IN GOVERNMENT SECURITIES BILL","N","0"
```

```
"54","71","INVESTMENTS IN GOVERNMENT SECURITIES BOND","N","0"
"55","22","LETTER OF CREDIT","N","0"
"56","136","LOAN AGAINST PROPERTY FLAT PENAL 5%","N","0"
"57","24","LOCKERS","N","0"
"58","48","LOCKERS USD","N","0"
"59","84","MAIIC Agricultural Loans ","Y","16"
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

description: Read the remaining schemes and every scheme setting

--- command ---

```
cd "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Request Responses 6 Oct 2026/Dupleix/Dupleix" && echo "=== DD_09 rest"; tr -d '\r' < DD_09_schemes_all.csv | tail -n +61 | awk -F',' '{ $4 != "0" || $3 == "Y" } | cut -c1-120; echo; echo "=== DD_10 scheme settings (transposed)"; PYTHONIOENCODING=utf-8 "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad/songenv/Scripts/python" - <<'PYEOF'
import pandas as pd
pd.set_option("display.width", 250); pd.set_option("display.max_rows", 400); pd.set_option("display.max_colwidth", 28)
d = pd.read_csv("DD_10_scheme_settings.csv", dtype=str).iloc[:, 1:]
d = d.set_index("SCHEME_NAME").T
d.columns = [c.strip()[:22] for c in d.columns]
# show only columns (settings) that are not all-blank
d = d[~d.apply(lambda r: r.fillna("").str.strip().eq("").all(), axis=1)]
print(d.to_string())
PYEOF
{
  "timeout": 120000
}
```

Result of Bash

[long output, reproduced in full from the saved file]
=== DD_09 rest
"60","85","MAIIC Agricultural Loans (Manual Int)","Y","0"
"61","86","MAIIC Industrial Loans","Y","41"
"62","87","MAIIC Industrial Loans(Manual Int)","Y","0"
"63","88","MAIIC Investment Loans","Y","5"
"64","89","MAIIC Investment Loans(Manual Intt)","Y","0"
"65","145","MAIIC TERM LOAN 10 YEAR","Y","0"
"66","141","MAIIC TERM LOAN 2 YEAR","Y","3"
"67","142","MAIIC TERM LOAN 3 YEAR","Y","3"
"68","143","MAIIC TERM LOAN 5 YEAR","Y","7"
"69","144","MAIIC TERM LOAN 7 YEAR","Y","2"
"70","99","Mega Farm CAPEX","N","0"
"71","102","Mega Farm Equipment Loans","N","270"
"72","103","Mega Farm Equipment Loans(Manual Int)","N","0"
"73","96","Mega Farm Fertilizer Loans","N","3246"
"74","101","Mega Farm Pesticides Loans","N","1191"
"75","98","Mega Farm Seed Loans","N","2118"
"76","100","Mega Farm Working Capital Loans","N","145"
"77","97","Mega Farms Irrigation","N","4"
"78","138","NTHA","N","0"
"79","5","Nominee Account Scheme","N","2631"
"80","0","OFFLINE GL","N","260"
"81","15","OVERDRAFT LIMIT","N","0"
"82","59","OVERDRAFT LIMIT - USD","N","0"
"83","65","OVERDRAFT LOAN","N","0"
"84","26","RECURRING DEPOSITS","N","0"
"85","60","SALARY LOAN","N","0"
"86","47","SAVING BANK DEPOSIT (JOINT, FIRM,CLUB,ASSOC,COPMPANY) USD","N","0"
"87","35","SAVING BANK DEPOSIT (JOINT, FIRM,CLUB,ASSOCIATION,COPMPANY)","N","0"
"88","44","SAVING BANK DEPOSITS PERSONAL USD","N","0"
"89","36","SAVING BANK STUDENT ACCOUNT","N","0"
"90","33","SAVING DEPOSIT SACCOS","N","0"
"91","45","SAVING DEPOSIT SACCOS USD","N","0"
"92","1","SHARE CAPITAL","N","114"
"93","41","SPECIAL OFFER DEPOSIT AS PER INTEREST SLAB","N","0"
"94","55","SPECIAL OFFER DEPOSIT AS PER INTEREST SLAB - USD","N","0"
"95","34","SPECIAL SAVING BANK DEPOSIT (SGL)","N","0"
"96","46","SPECIAL SAVING BANK DEPOSIT (SGL) USD","N","0"
"97","64","STAFF LOAN","N","0"
"98","17","SUNDRY CREDITORS","N","694"
"99","18","SUNDRY DEBTORS","N","0"
"100","107","Staff House Rent Advance","N","0"
"101","109","Staff Car Loan","N","0"
"102","104","Staff Education Loan","N","0"
"103","111","Staff Emergency Loan","N","0"
"104","110","Staff Handset Loan","N","0"
"105","108","Staff Medical Loan","N","0"
"106","106","Staff Mortgage Loan","N","0"
"107","105","Staff Personal Loan","N","0"
"108","14","TERM LOANS","N","0"
"109","58","TERM LOANS USD","N","0"
"110","83","TERM LOANS (MANUAL INTT.)","N","0"
"111","137","TERM LOANS NEW","N","0"
"112","69","TOD","N","0"
"113","38","WAHI ACCOUNTS","N","0"
"114","37","YOUNG SAVER ACCOUNT","N","0"

=== DD_10 scheme settings (transposed)

	FINES Industrial Loans	FINES AGRICULTURAL LOA	FINES Industrial Loans	FINES Agricultural Loa
	FINES Investment Loans	FINES Industrial Loans	Fines Investment Loans	FINES Investment Loans
	MAIIC Agricultural Loa	MAIIC Agricultural Loa	MAIIC Industrial Loans	MAIIC
	MAIIC Industrial Loans	MAIIC Investment Loans	MAIIC Investment Loans	
	MAIIC TERM LOAN 10 YEA	MAIIC TERM LOAN 2 YEAR	MAIIC TERM LOAN 3 YEAR	
	MAIIC TERM LOAN 5 YEAR	MAIIC TERM LOAN 7 YEAR		
SCHEME_MST_ID		90	91	
	92		93	94
		95	140	
84		85		86
	87		88	89
	145	141		142
	143	144		
SCHEME_NAME_OL		FINES Agricultural Loans	FINES Agricultural Loans(Manual Int)	FINES Investment Loans
	FINES Industrial Loans	FINES Industrial Loans(Manual Int)		MAIIC
	FINES Investment Loans(Manual Intt)	Fines Investment Loans		
Agricultural Loans	MAIIC Agricultural Loans (Manual Int)	MAIIC Industrial Loans		
MAIIC Industrial Loans(Manual Int)		MAIIC Investment Loans	MAIIC Investment Loans(Manual	
Intt)	MAIIC Term loan 10 Year	MAIIC Term loan 2 Year	MAIIC Term loan 3	
Year	MAIIC Term loan 5 Year	MAIIC Term loan 7 Year		
GLTYPE		140	140	140
	140		140	140
		140	140	
	140		140	140
		140		140
	140		140	
		140		140
APPLICABLE_FROM_DATE		1/1/2000	1/1/2000	1/1/2000
	1/1/2000		1/1/2000	1/1/2000

	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000
APPLICABLE_UPTO_DATE	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000	1/1/2000
	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
LOAN_TYPE	N	N	N	N	N	N	N	N	N
APPLICABLE_MIN_PERIOD	1	1	1	1	1	1	1	1	1
APPL_MIN_PERIOD_UNIT	M	M	M	M	M	M	M	M	M
APPLICABLE_MAX_PERIOD	180	180	180	180	180	180	180	180	180
APPL_MAX_PERIOD_UNIT	M	M	M	M	M	M	M	M	M
REVIEW_MIN_PERIOD	12	12	12	12	12	12	12	12	12
REVIEW_MIN_PERIOD_UNIT	M	M	M	M	M	M	M	M	M
REVIEW_MAX_PERIOD	12	12	12	12	12	12	12	12	12
REVIEW_MAX_PERIOD_UNIT	M	M	M	M	M	M	M	M	M
SACNTION_MIN_AMOUNT	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00	1.00
SANCTION_MAX_AMOUNT	1E19	1E19	1E19	1E19	1E19	1E19	1E19	1E19	1E19
FLOATING_FLAG	I	I	I	I	I	I	I	I	I

[illegible]

	N		Y		Y		N		N		N		N		Y
OPERATION_ID		N	N			M2						M20			
			M20		M20			M20		M20					M20
	M20				M20		M20					M20			
OPERATION_DATE		M20	N8			N7							N5	M20	
		N6				4/1/2024									
		7/30/2013			7/30/2013			7/30/2013					7/30/2013		7/30/2013
	7/30/2013									7/30/2013					
		7/30/2013			7/30/2013		7/30/2013					7/30/2013		7/30/2013	
ENTER_USER_ID		11/30/2025				11/30/2025						11/30/2025			
		11/30/2025				11/30/2025									
			1			1							1		
				1						1					1
	1				1							1			
		1					1						1		
ENTER_DESC		16	16			16							16		
			abcd		abcd		abcd		abcd				abcd		abcd
	abcd				abcd				abcd						
		abcd			abcd		abcd					abcd			
12/2025-10:55:17-192.168.002.001				0-1-08/12/2025-10:25:09-192.168.002.001				0-1-08/12/2025-10:33:17-192.168.002.001						0-1-08/12/2025-10:45:53-192.168.002.001	
PASS_USER_ID						9998							1		
			1					1							1
				1						1					
	1				1							1			
		1					1						1		
			52										52		1
PASS_DESC															
2013-01:39:59-192.168.020.90(Prefer				0-1-04/05/2024-14:40:49-192.167.001.031				0-1-13/09/2013-01:39:59-192.168.020.90(Prefer							0-1-13/09/2013-01:39:59-192.168.020.90(Prefer
2013-01:39:59-192.168.020.90(Prefer				0-1-13/09/2013-01:39:59-192.168.020.90(Prefer				0-1-13/09/2013-01:39:59-192.168.020.90(Prefer							0-1-13/09/2013-01:39:59-192.168.020.90(Prefer
2013-01:39:59-192.168.020.90(Prefer				0-1-13/09/2013-01:39:59-192.168.020.90(Prefer				0-1-13/09/2013-01:39:59-192.168.020.90(Prefer							0-1-13/09/2013-01:39:59-192.168.020.90(Prefer
2013-01:39:59-192.168.020.90(Prefer				0-1-13/09/2013-01:39:59-192.168.020.90(Prefer				0-1-13/09/2013-01:39:59-192.168.020.90(Prefer							0-1-13/09/2013-01:39:59-192.168.020.90(Prefer
2013-01:39:59-192.168.020.90(Prefer				0-1-13/09/2013-01:39:59-192.168.020.90(Prefer				0-1-13/09/2013-01:39:59-192.168.020.90(Prefer							0-1-13/09/2013-01:39:59-192.168.020.90(Prefer
2013-01:39:59-192.168.020.90(Prefer				0-1-08/12/2025-11:06:09-192.168.002.001				0-1-08/12/2025-11:11:18-192.168.002.001							0-1-08/12/2025-11:05:39-192.168.002.001
INTDAYS_PER_YEAR				0-1-08/12/2025-11:05:49-192.168.002.001				0-1-08/12/2025-11:06:00-192.168.002.001							
			365			365							365		
				365						365					365
	365				365							365			

[illegible]

		N				N						N
	N			N			N			N		
		N				N			N		N	
			N				N				N	
MIN_OVDPRD_PENAL						0.00					0.00	
		0.00					0.00					0.00
	0.00			0.00				0.00				
		0.00				0.00			0.00			
			0.00				0.00				0.00	
		0.00				0.00						
POST_NPA_INT_TO_REC						N					N	
			N					N				N
	N			N					N			
		N					N				N	
			N					N				
CAL_EXCESS_LOAN_INT						N					N	
			N					N				N
				N					N			
	N					N				N		
		N					N				N	
			N					N				
EMI_BASED_ON						N					N	
							N					
			0					0			0	
				0					0			0
	2					0				2		
		0					0				0	
			0					0				0
INT_ON_DAILY_MAXBAL						0					0	
			A								N	
				N				N				N
	A								N			
		N								A		
			A								N	
											A	

Tool call: Bash

description: Pick out the scheme settings that matter from the DD_10 output

```

--- command ---
F="/c/Users/wadza/.claude/projects/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/tool-results/bi62i7035.txt"; sed -n '/^"101"/,/=== DD_10/p' "$F" | awk -F"," ' $4 != "0" || $3 == "Y"' | cut -c1-110; echo "=== DD_10 settings: rows with interesting names"; grep -n -i "INTEREST_POLICY\|FLOATING\|POLICY\|INT_CAL\|INST_BASED\|INSTALLMENT_BASED\|EMI\|MORAT\|SCHEME_CODE\|SCHEME_NAME\|INTDAYS\|COMPOUND\|RATE\|PLR\|SLAB\|BASE\|FREQ\|CALC\|PENAL" "$F" | cut -c1-250 | head -60

```

Result of Bash

```
"101","109","Staff Car Loan","N","0"
"102","104","Staff Education Loan","N","0"
"103","111","Staff Emergency Loan","N","0"
"104","110","Staff Handset Loan","N","0"
"105","108","Staff Medical Loan","N","0"
"106","106","Staff Mortgage Loan","N","0"
"107","105","Staff Personal Loan","N","0"
"108","14","TERM LOANS","N","0"
"109","58","TERM LOANS  USD","N","0"
"110","83","TERM LOANS (MANUAL INTT.)","N","0"
"111","137","TERM LOANS NEW","N","0"
"112","69","TOD","N","0"
"113","38","WAHI ACCOUNTS","N","0"
"114","37","YOUNG SAVER ACCOUNT","N","0"

=== DD_10 scheme settings (transposed)
=== DD_10 settings: rows with interesting names
35:"93","41","SPECIAL OFFER DEPOSIT AS PER INTEREST SLAB","N","0"
36:"94","55","SPECIAL OFFER DEPOSIT AS PER INTEREST SLAB - USD","N","0"
61:SCHEME_NAME_OL      FInES Agricultural Loans      FInES Agricultural Loans(Manual Int)
      FInES Industrial Loans      FInES Industrial Loans(Manual Int)      FInES Investme
76:FLOATING_FLAG      I      I      I
77:COMPOUND_SIMPLE_FLAG      C      C
78:INTEREST_PRODUCT_FREQUENCY      D      D
79:INTEREST_PAYMENT_FREQUENCY      M      M
85:INTEREST_POLICY      S      M
103:MIN_MORATORIUM_PERIOD      999      0
104:MIN_MORATORIUM_PERIOD_UNIT      M      M
106:MAX_MORATORIUM_PERIOD_UNIT      M      M
122:INTDAYS_PER_YEAR      365      365
123:ADD_RECBAL_IN_INTCALC      O      O
124:POST_INT_PENAL_COMBINE      N      N
125:OVD_CALC_TYPE      I      I
128:PAYMENT_TRAN_FREQ      M      M
131:INST_BASED_ON      D      D
137:INT_CALC_FREQUENCY      N      N
142:MIN_OVDPRD_PENAL      0.00      0.00
145:EMI_BASED_ON      0      0
```

Tool call: Bash

```
description: Show the key settings for each MAIIC and FInES scheme
--- command ---
cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Request Responses 6 Oct 2026/Dupleix/Dupleix" && PYTHONIOENCODING=utf-8 "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad/songenv/Scripts/python" - <<'PYEOF' 2>&1 | cut -c1-230
import pandas as pd
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 40)
d = pd.read_csv("DD_10_scheme_settings.csv", dtype=str).iloc[:, 1:]
print("columns:", len(d.columns))
cols = [c for c in d.columns if any(k in c.upper() for k in ("POLICY", "FLOAT", "BASED", "EMI", "CALC", "FREQ", "INTDAYS", "COMPOUND", "MORAT", "SCHEME_MST_ID", "SCHEME_CODE", "GL", "PENAL", "RATE", "PLR", "SLAB"))]
print("matching columns:", cols)
d["SCHEME_NAME"] = d["SCHEME_NAME"].str.strip().str[:34]
show = ["SCHEME_MST_ID", "SCHEME_NAME", "INTEREST_POLICY", "FLOATING_FLAG", "INST_BASED_ON", "EMI_BASED_ON", "INT_CALC_FREQUENCY", "INTEREST_PRODUCT_FREQUENCY", "INTEREST_PAYMENT_FREQUENCY", "PAYMENT_TRAN_FREQ", "COMPOUND_SIMPLE_FLAG", "INTDAYS_PER_YEAR", "MIN_MORATORIUM_PERIOD", "MAX_MORATORIUM_PERIOD"]
show = [c for c in show if c in d.columns]
print(d[show].to_string(index=False))
other = [c for c in cols if c not in show]
print("\nother matching columns:", other)
PYEOF
{
  "timeout": 120000
}
```

Result of Bash

```
columns: 97
matching columns: ['SCHEME_MST_ID', 'GLTYPE', 'FLOATING_FLAG', 'COMPOUND_SIMPLE_FLAG', 'INTEREST_PRODUCT_FREQUENCY',
'INTEREST_PAYMENT_FREQUENCY', 'INTEREST_POLICY', 'MIN_MORATORIUM_PERIOD', 'MIN_MORATORIUM_PERIOD_UNIT', 'MAX_MORA
SCHEME_MST_ID          SCHEME_NAME INTEREST_POLICY FLOATING_FLAG INST_BASED_ON EMI_BASED_ON INT_CALC_FREQUENCY
INTEREST_PRODUCT_FREQUENCY INTEREST_PAYMENT_FREQUENCY PAYMENT_TRAN_FREQ COMPOUND_SIMPLE_FLAG INTDAYS_PER_
90      FInES AGRICULTURAL LOANS      S      I      D      0      N
      D      M      M      C
91 FInES Agricultural Loans(Manual In      M      I      D      0      N
      D      M      C
92      FInES Industrial Loans      S      I      D      0      N
      D      M      C
93 FInES Industrial Loans(Manual Int)      M      I      D      0      N
      D      M      C
94      FInES Investment Loans      S      I      D      0      N
      D      M      C
95 FInES Investment Loans(Manual Intt      M      I      D      0      N
      D      M      C
140      Fines Investment Loans      S      F      D      0      N
      D      M      C
84      MAIIC Agricultural Loans      P      F      D      2      N
      D      M      C
85 MAIIC Agricultural Loans (Manual I      M      F      D      0      N
      D      M      C
86      MAIIC Industrial Loans      P      F      D      2      N
      D      M      C
87 MAIIC Industrial Loans(Manual Int)      M      F      D      0      N
      D      M      C
88      MAIIC Investment Loans      S      F      D      0      N
      D      M      C
89 MAIIC Investment Loans(Manual Intt      M      F      D      0      N
      D      M      C
145      MAIIC TERM LOAN 10 YEAR      S      F      D      0      N
      D      M      C
141      MAIIC TERM LOAN 2 YEAR      S      F      D      0      N
      D      M      C
142      MAIIC TERM LOAN 3 YEAR      S      F      D      0      N
      D      M      C
143      MAIIC TERM LOAN 5 YEAR      S      F      D      0      N
      D      M      C
144      MAIIC TERM LOAN 7 YEAR      S      F      D      0      N
      D      M      C
```

other matching columns: ['GLTYPE', 'MIN_MORATORIUM_PERIOD_UNIT', 'MAX_MORATORIUM_PERIOD_UNIT', 'ADD_RECBAI_IN_INTCALC', 'POST_INT_PENAL_COMBINE', 'OVD_CALC_TYPE', 'MIN_OVDPRD_PENAL', 'PENAL_ON_OVD_PRI', 'CAL_EXP_PENAL_ON', 'EXP_PE
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

CLAUDE (2026-10-06 13:30:56 UTC)

Barry's responses answer the big question: INTEREST_POLICY is on the scheme table. Reading the rest now so I can give you a short list for him while he's on the call.

Tool call: Bash

```
description: Read the sample loan's rows in every table
--- command ---
cd "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Request Responses 6 Oct 2026/
Dupleix/Dupleix" && PYTHONIOENCODING=utf-8 "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/
ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad/songenv/Scripts/python" - <<'PYEOF' 2>&1 | cut -c1-250
import pandas as pd
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 60); pd.set_option("display.max_colwidth", 30)
def load(f): return pd.read_csv(f, dtype=str).iloc[:, 1:]
for f in ["DD_16a_sample_acmaster.csv", "DD_16b_sample_acloanmaster.csv"]:
    d = load(f); print("=== ", f, d.shape)
    r = d.iloc[0]; print({k: v for k, v in r.items() if isinstance(v, str) and v.strip() not in ("", "0", "0.00", "N")})
f = load("DD_16f_sample_ledger.csv"); print("=== ledger", f.shape); print(list(f.columns))
cols = [c for c in ["CUMVOUCH_DET_ID", "TRANSACTION_DATE", "VALUE_DATE", "TRANSTYPE", "TRANSAMT", "AC_GLCODE", "DRCR", "DR_CR",
"NARRATION", "PARTICULARS", "TXN_NUMBER", "VOUCHER_NO", "OPERATION_DATE"] if c in f.columns]
print(f[cols].head(14).to_string(index=False))
print("trantype counts:", f.TRANSTYPE.value_counts().to_dict(), "| GL codes:", f.AC_GLCODE.value_counts().to_dict())
d = load("DD_16d_sample_instalment_chart.csv"); print("=== chart", d.shape); print(list(d.columns))
cols = [c for c in d.columns if any(k in c for k in ("DATE", "AMT", "RECOVER", "BAL", "ID", "FLAG"))][:14]
print(d[cols].head(6).to_string(index=False)); print(d[cols].tail(3).to_string(index=False))
g = load("DD_16g_sample_balances.csv"); print("=== balances", g.shape); print(list(g.columns));
print(g.head(5).to_string(index=False))
e = load("DD_16e_sample_instalment_plan.csv"); print("=== plan", e.shape); print(list(e.columns));
print(e.head(3).to_string(index=False))
PYEOF
{
  "timeout": 120000
}
```

Result of Bash

```
=== DD_16a_sample_acmaster.csv (1, 141)
{'ACCOUNT_MST_ID': '11768', 'BRANCH_MST_ID': '1', 'GLCODE': '1050102', 'OLD_AC_NUMBER': '84', 'NEW_AC_NUMBER':
'000104430000084', 'SUB_AC_NUMBER': '1', 'SCHEME_MST_ID': '86', 'CUSTOMER_ID': '1588', 'ACCOUNT_NAME': 'EBENEZER MIDIAN LTD',
'TRANSACTION_
=== DD_16b_sample_acloanmaster.csv (1, 80)
{'ACLOAN_MST_ID': '7629', 'NEW_AC_NUMBER': '000104430000084', 'SUB_ACCOUNT_NO': '1', 'SANCTION_AMOUNT': '314900900.00',
'SANCTION_DATE': '7/15/2025', 'PERIOD_MONTHS': '60', 'EXPIRY_DATE': '7/15/2030', 'PMOROTORIUM_UNIT': 'M', 'IMOROTORIUM_UNIT':
'M',
=== ledger (30, 83)
['CUMVOUCH_DET_ID', 'CB_BRANCH_MST_ID', 'T_BRANCH_MST_ID', 'O_BRANCH_MST_ID', 'CB_GLCODE', 'AC_GLCODE', 'NEW_AC_NUMBER',
'SUB_ACCOUNT_NO', 'CB_GLTYPE', 'CB_ALNX_TYPE', 'TRANSACTION_DATE', 'CB_EFFECTIVE_DATE', 'TRANSACTION_TIME', 'TRANSAMT',
'INTEREST
CUMVOUCH_DET_ID TRANSACTION_DATE TRANTYPE          TRANSAMT AC_GLCODE                                PARTICULARS TXN_NUMBER
OPERATION_DATE
209790      8/8/2025      301 -298480500.00  1050102                                To Trf Loan drawdown      NaN
8/8/2025
209792      8/8/2025      301  -9000000.00  1050102                                To Trf arrangement fee-Ebenezer      NaN
8/8/2025
209794      8/8/2025      301  -5900000.00  1050102      To Trf Lega fees-Ebenezer Midian Limited      NaN
8/8/2025
209796      8/8/2025      301  -1519500.00  1050102                                To Trf      NaN
8/8/2025
209798      8/8/2025      301    -900.00  1050102                                To Trf      NaN
8/8/2025
210594      8/31/2025      303  -6418801.92  1050102      To Trf IntDR Fm 08-AUG-25 To 31-AUG-25 @ 31%      NaN
8/31/2025
225473      9/12/2025      305   5220495.30  1050102      By Trf Loan repayment September 2025 Ebenezer      NaN
9/12/2025
225475      9/12/2025      305   5220495.30  1050102      By Trf Loan repayment September 2025 Ebenezer      NaN
9/12/2025
241779      9/30/2025      303  -8002381.01  1050102      To Trf IntDR Fm 01-SEP-25 To 30-SEP-25 @ 30.9%      NaN
9/30/2025
240896      10/1/2025      305  16731448.80  1050102      By Trf Loan repayment Ebenezer investments      NaN
10/1/2025
242506      10/29/2025      305  10440990.72  1050102      By Trf LOAN REPAYMENT EBENEZER INVESTMENTS      NaN
10/29/2025
243230      10/31/2025      303  -7862400.65  1050102      To Trf IntDR Fm 01-OCT-25 To 31-OCT-25 @ 30.9%      NaN
10/31/2025
257702      11/25/2025      305  10442000.00  1050102                                By Trf EBENEZER INVESTMENTS      NaN
11/25/2025
258297      11/30/2025      303  -7555244.46  1050102      To Trf IntDR Fm 01-NOV-25 To 30-NOV-25 @ 30.9%      NaN
11/30/2025
trantype counts: {'303': 14, '305': 11, '301': 5} | GL codes: {'1050102': 30}
=== chart (926, 38)
['INST_CHART_DET_ID', 'NEW_AC_NUMBER', 'SUB_ACCOUNT_NO', 'TRANSACTION_DATE', 'INST_DET_ID', 'INST_SR_NO', 'INSTALLMENT_DATE',
'INSTALLMENT_AMT', 'INST_RECOVERED', 'LAST_OVD_CALC_DATE', 'REMARK', 'ACTIVE_FLAG', 'STATUS_CHANGE_DATE', 'STATUS_REASON', '
INST_CHART_DET_ID TRANSACTION_DATE INST_DET_ID INSTALLMENT_DATE INSTALLMENT_AMT INST_RECOVERED LAST_OVD_CALC_DATE ACTIVE_FLAG
STATUS_CHANGE_DATE DELETE_FLAG OPERATION_ID OPERATION_DATE ENTER_USER_ID DELETE_USER_ID
131446      7/15/2025      8013      7/15/2025      0.00      0.00      NaN      Y
NaN      Y      NaN      7/16/2025      55555      55555      0.00      0.00      NaN      Y
131503      7/16/2025      NaN      7/15/2025      0.00      0.00      NaN      Y
NaN      Y      NaN      7/23/2025      55555      55555      0.00      0.00      NaN      Y
131564      7/23/2025      NaN      7/15/2025      0.00      0.00      NaN      Y
NaN      Y      NaN      9/3/2025      55555      55555      0.00      0.00      NaN      Y
131625      9/3/2025      NaN      7/15/2025      0.00      0.00      NaN      Y
NaN      Y      NaN      10/1/2025      55555      55555      0.00      0.00      NaN      Y
131682      10/1/2025      NaN      7/15/2025      0.00      0.00      NaN      Y
NaN      Y      NaN      11/1/2025      55555      55555      0.00      0.00      NaN      Y
131739      11/1/2025      NaN      7/15/2025      0.00      0.00      NaN      Y
NaN      Y      NaN      2/4/2026      55555      51      0.00      0.00      NaN      Y
INST_CHART_DET_ID TRANSACTION_DATE INST_DET_ID INSTALLMENT_DATE INSTALLMENT_AMT INST_RECOVERED LAST_OVD_CALC_DATE ACTIVE_FLAG
STATUS_CHANGE_DATE DELETE_FLAG OPERATION_ID OPERATION_DATE ENTER_USER_ID DELETE_USER_ID
131563      7/16/2025      8791      7/15/2030      10555061.00      0.00      NaN      Y
NaN      Y      NaN      7/23/2025      55555      55555      0.00      0.00      NaN      Y
131624      7/23/2025      8792      7/15/2030      10555061.00      0.00      NaN      Y
NaN      Y      NaN      9/3/2025      55555      55555      0.00      0.00      NaN      Y
161257      2/4/2026      8795      7/15/2030      155253605.00      0.00      NaN      Y
NaN      Y      NaN      2/4/2026      51      55555      0.00      0.00      NaN      Y
=== balances (13, 34)
['ACCOUNT_BAL_MST_ID', 'GLCODE', 'NEW_AC_NUMBER', 'SUB_AC_NUMBER', 'TRANSACTION_DATE', 'PRINCIPAL_BALANCE', 'RECEIPT_AMOUNT',
'PAYMENT_AMOUNT', 'OD_INT_REC_PAY', 'INT_REC', 'SANCTION_AMOUNT', 'REPORT_BALANCE', 'ALNX_TYPE', 'OD_INT_RECEIPT', 'OD_INT_P
ACCOUNT_BAL_MST_ID GLCODE NEW_AC_NUMBER SUB_AC_NUMBER TRANSACTION_DATE PRINCIPAL_BALANCE RECEIPT_AMOUNT PAYMENT_AMOUNT
OD_INT_REC_PAY INT_REC SANCTION_AMOUNT REPORT_BALANCE ALNX_TYPE OD_INT_RECEIPT OD_INT_PAYMENT INT_RECEIPT INT_PAYMENT AC_PRINCIP
260832 1050102 000104430000084 1 8/31/2025 -321319701.92 0.00 -321319701.92 0.00 -321319701.92
0.00 0.00 314900900.00 -321319701.92 LN 0.00 0.00 0.00 0.00 -32
268425 1050102 000104430000084 1 9/30/2025 -318881092.33 10440990.60 -8002381.01
0.00 0.00 314900900.00 -318881092.33 LN 0.00 0.00 0.00 0.00 -31
289024 1050102 000104430000084 1 10/31/2025 -299571053.46 27172439.52 -7862400.65
0.00 0.00 314900900.00 -299571053.46 LN 0.00 0.00 0.00 0.00 -29
292818 1050102 000104430000084 1 11/30/2025 -296684297.92 10442000.00 -7555244.46
0.00 0.00 314900900.00 -296684297.92 LN 0.00 0.00 0.00 0.00 -29
325211 1050102 000104430000084 1 12/31/2025 -304864196.88 0.00 -8179898.96
0.00 0.00 314900900.00 -304864196.88 LN 0.00 0.00 0.00 0.00 -30
=== plan (18, 27)
['INST_DET_ID', 'NEW_AC_NUMBER', 'SUB_ACCOUNT_NO', 'TRANSACTION_DATE', 'INSTALLMENT_TYPE', 'INSTALLMENT_PAY_TYPE',
'INSTALLMENT_PAY_MODE', 'INSTALLMENT_START_DATE', 'INSTALLMENT_AMT', 'INSTALLMENT_END_DATE', 'INTT_INST_START_DATE',
'INTT_INST_AMT', '
INST_DET_ID NEW_AC_NUMBER SUB_ACCOUNT_NO TRANSACTION_DATE INSTALLMENT_TYPE INSTALLMENT_PAY_TYPE INSTALLMENT_PAY_MODE
INSTALLMENT_START_DATE INSTALLMENT_AMT INSTALLMENT_END_DATE INTT_INST_START_DATE INTT_INST_AMT
```

RE									
	8009	000104430000084	1	8/8/2025	O	1	M		
	9/1/2025	9840997.00		NaN				0.00	Installment Details through Disburse
	8010	000104430000084	1	8/8/2025	D	1	M		
	9/1/2025	10137730.00		NaN				0.00	Installment Details through Disburse
	8011	000104430000084	1	8/8/2025	D	1	M		
	9/1/2025	10332254.00		NaN				0.00	Installment Details through Disburse
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App									

Tool call: Bash

```

description: Find rate, fee and charge tables and account-level rate columns in the dictionary results
--- command ---
cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Request Responses 6 Oct 2026/Dupleix/Dupleix" && echo "=== DD_07 candidate tables (rows>0)"; tr -d '\r' < DD_07_tables_by_subject.csv | awk -F',' '{ $3+0 > 0' | cut -c1-120 | grep -i -v "deposit\saving\locker\share" | head -60; echo "=== DD_03: columns in ACMASTER/ACLOANMASTER with rate/policy/fee/margin"; tr -d '\r' < DD_03_columns.csv | awk -F',' '{ ($2=="ACMASTER" || $2=="ACLOANMASTER") && (toupper($4) ~ /RATE|POLICY|FEE|MARGIN|PLR|FLOAT|CHARGE|INTT|INT_/)' | cut -d'"' -f4,8,12,22,24 | head -40; echo "=== tables named CHARGE/FEE/PLR/PENAL/RATE in DD_02 with rows"; tr -d '\r' < DD_02_tables.csv | awk -F',' '{ toupper($3) ~ /CHARGE|FEE|PLR|RATE|PENAL|INTT|INTEREST/ && $4+0 > 0' | cut -c1-110 | head -30

```

Result of Bash

```
=== DD_07 candidate tables (rows>0)
"2","ACCOUNT_BALANCE","75607",""
"4","ACCOUNT_STATUS_HISTORY","10482",""
"5","ACLOANMASTER","7154",""
"6","AC_MIN_BALANCE","2268",""
"14","AUTO_CHARGES_MASTER","1",""
"20","BRANCH_DAY_STATUS","813",""
"21","CG_PARTY_LOAN_DETAILS","2",""
"31","CUMVOUCH","395162",""
"32","CUMVOUCH_310724","81",""
"33","CUMVOUCH_3110","742",""
"34","CUMVOUCH_BAK","19780",""
"35","CUMVOUCH_BK240824","12464",""
"36","CUMVOUCH_CHET13062025","71311",""
"37","CUMVOUCH_MW","58",""
"38","CUMVOUCH_NF","3217",""
"40","CURRENCY_RATE_MASTER","3",""
"41","CURR_RATE_MASTER_HIST","2",""
"43","CUST_ORG_AUDITOR_INFO","105",""
"44","CUST_OTH_LOAN_DETAILS","171",""
"45","DAILY_INTEREST_ACCRUAL","54604",""
"52","DIFF_INTEREST_PRODUCT","7521",""
"53","DIFF_INTEREST_SUMMARY","29",""
"58","EMPLOYEE_STATUS_HISTORY","22",""
"59","EMP_INSU_CONTRI_RATE","21",""
"63","FIXLOAN_INST_CHART","95609",""
"64","FIXLOAN_INST_CHART1","3211",""
"65","FIXLOAN_INST_CHART_141124","9339",""
"66","FIXLOAN_INST_CHART_300824","4977",""
"69","FIXLOAN_OVERDUE_DETAILS","81829",""
"70","FIX_INST_CHART28SEP","9106",""
"72","FORGIVE_INTEREST_DETAILS","2",""
"73","GL_OPENING_BALANCE","598",""
"74","GL_OPENING_BALANCE_020CT24","357",""
"75","GL_OPENING_BALANCE_120423","331",""
"76","GL_OPENING_BALANCE_BK240824","331",""
"106","G_LOAN_BOOK_MAPPING","44",""
"135","INTEREST_PRODUCT","9319",""
"137","INTEREST_SLAB_DETAILS","11487",""
"138","INTEREST_SLAB_MASTER","94",""
"139","INTEREST_SUMMARY","3391",""
"143","INTERNAL_USER","10",""
"148","INV_INSTITUTE_MASTER","16",""
"153","LEASE_FEE_DETAILS","1",""
"154","LEASE_FEE_SCHEDULE","120",""
"155","LEASE_INTEREST_SCHEDULE","120",""
"156","LOANACCOUNT","593",""
"157","LOANACCOUNT_MEGA_SEED","95",""
"158","LOANACCOUNT_MEGA_WORKING","116",""
"168","LOAN_BOOK_DETAILS_ALL","725460",""
"169","LOAN_BOOK_MAPPING","29",""
"170","LOAN_DISBURSMENT_SCHEDULE","6880",""
"172","LOAN_INSURANCE_DETAILS","73",""
"173","LOAN_INT_SUBVEN_DETAILS","70331",""
"174","LOAN_INT_SUBVEN_DETAILS_CHET13062025","8350",""
"175","LOAN_INT_SUBVEN_SETTING","12",""
"177","LOAN_SANCTION_FORWARD","6192",""
"178","LOAN_SECURITY_DETAILS","3482",""
"185","LOS_DISB_CHARGES_POST","10052",""
"186","MAIIC_GL_INTEREST_RECOGNIZED","3996","Interest income actually posted to the GL, per loan, per month and per year
"189","MULTI_INST_DETAILS","8544",""
=== DD_03: columns in ACMASER/ACLOANMASTER with rate/policy/fee/margin
ACLOANMASTER"MARGIN_PERCENT"22"0"
ACLOANMASTER"INT_REC_REVERSE"1"0"
ACMASTER"INTEREST_POLICY"1"0"
ACMASTER"INTEREST_RATE"22"0"
ACMASTER"FLOATING_FLAG"1"0"
ACMASTER"LAST_PB_PRINT_DATE"7"10862"
ACMASTER"LAST_PB_PRINT_VVOID"22"10862"
ACMASTER"LAST_TODINT_DATE"7"10862"
ACMASTER"AC_SIMINT_BAL"22"5736"
ACMASTER"BR_SIMINT_BAL"22"3202"
ACMASTER"AC_INT_PAID"22"3202"
ACMASTER"BR_INT_PAID"22"3202"
ACMASTER"INT_TRANSFER_TOAC"18"10862"
ACMASTER"INT_TRF_SUBACC"22"10862"
=== tables named CHARGE/FEE/PLR/PENAL/RATE in DD_02 with rows
"110","VGCBS","AUTO_CHARGES_MASTER","1","2026-06-13",""
"250","VGCBS","CURRENCY_RATE_MASTER","3","2026-06-13",""
"251","VGCBS","CURR_RATE_MASTER_HIST","2","2026-06-13",""
"273","VGCBS","DAILY_INTEREST_ACCRUAL","54604","2026-07-29",""
"291","VGCBS","DIFF_INTEREST_PRODUCT","7521","2026-06-13",""
"292","VGCBS","DIFF_INTEREST_SUMMARY","29","2026-06-13",""
"347","VGCBS","EMP_INSU_CONTRI_RATE","21","2026-06-13",""
"408","VGCBS","FORGIVE_INTEREST_DETAILS","2","2026-07-03",""
"762","VGCBS","INTEREST_PRODUCT","9319","2026-09-09",""
"764","VGCBS","INTEREST_SLAB_DETAILS","11487","2026-06-13",""
"765","VGCBS","INTEREST_SLAB_MASTER","94","2026-06-13",""
"766","VGCBS","INTEREST_SUMMARY","3391","2026-09-09",""
```

```
"792","VGCBS","INV_SHARE_MARKET_RATE","16","2026-06-13",""  
"839","VGCBS","LEASE_FEE_DETAILS","1","2026-06-13",""  
"840","VGCBS","LEASE_FEE_SCHEDULE","120","2026-06-13",""  
"841","VGCBS","LEASE_INTEREST_SCHEDULE","120","2026-06-13",""  
"891","VGCBS","LOS_DISB_CHARGES_POST","10052","2026-06-13",""  
"895","VGCBS","MAIIC_GL_INTEREST_RECOGNIZED","3996","2026-08-25","Interest income actually posted to the GL, p  
"1005","VGCBS","PLR_MASTER","46","2026-06-13",""  
"1006","VGCBS","PLR_MASTER_09JAN","26","2026-06-13",""  
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

description: List the columns of the newly found rate, fee, accrual and loan book tables

```
--- command ---  
cd "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Request Responses 6 Oct 2026/  
Duplex/Duplex" && for t in PLR_MASTER LOS_DISB_CHARGES_POST DAILY_INTEREST_ACCRUAL LOAN_BOOK_DETAILS_ALL  
ACCOUNT_STATUS_HISTORY INTEREST_SLAB_MASTER INTEREST_SLAB_DETAILS FORGIVE_INTEREST_DETAILS AUTO_CHARGES_MASTER INTEREST_SUMMARY;  
do echo "==" $t: $(tr -d '\r' < DD_03_columns.csv | awk -F"," 'v-t="$t" '$2==t {printf "%s(%s) ", $4, $5}'); done 2>&1 | cut  
-c1-900
```

Result of Bash

```
== PLR_MASTER: PLR_MST_ID(NUMBER) TRAN_DATE(DATE) APPLI_FROM_DATE(DATE) PLR_RATE(NUMBER) MIN_VERIATION_RATE(NUMBER)  
MAX_VARIATION_RATE(NUMBER) ACTIVE_FLAG(VARCHAR2) PARTICULAR(VARCHAR2) DELETE_FLAG(VARCHAR2) OPERATION_ID(VARCHAR2)  
OPERATION_DATE(DATE) ENTER_USER_ID(NUMBER) ENTER_DESC(VARCHAR2) DELETE_USER_ID(NUMBER) DELETE_DESC(VARCHAR2)  
PASS_USER_ID(NUMBER) PASS_DESC(VARCHAR2)  
== LOS_DISB_CHARGES_POST: LO_DISB_CHARG_DET_ID(NUMBER) DISB_CHARGES_NAME(VARCHAR2) NEW_AC_NUMBER(VARCHAR2)  
SUB_AC_NUMBER(NUMBER) ACTUAL_CHARGES_AMT(NUMBER) APLLI_CHARGES_AMT(NUMBER) STAX_PER(NUMBER) STAX_AMOUNT(NUMBER) SBC_PER(NUMBER)  
SBC_AMOUNT(NUMBER) POST_FLAG(VARCHAR2) ENTER_USER_ID(NUMBER) ENTER_DESC(VARCHAR2) POST_USERID(NUMBER) BATCH_NUMBER(NUMBER)  
PARTICULAR(VARCHAR2) INCOME_GL(VARCHAR2)  
== DAILY_INTEREST_ACCRUAL: INT_ACCRU_DET_ID(NUMBER) OP_BRANCH_MST_ID(NUMBER) GLCODE(VARCHAR2) NEW_AC_NUMBER(VARCHAR2)  
SUB_AC_NUMBER(NUMBER) TRANSACTION_DATE(DATE) INTEREST_AMOUNT(NUMBER) DRCR(VARCHAR2) DR_POSTING_GL(VARCHAR2)  
CR_POSTING_GL(VARCHAR2) BATCH_NUMBER(NUMBER)  
== LOAN_BOOK_DETAILS_ALL: LOAN_BOOK_DET_ID_A(NUMBER) TRANSACTION_DATE(DATE) ASONDATE(DATE) BRANCH_MST_ID(NUMBER)  
CUSTOMER_ID(NUMBER) GLCODE(VARCHAR2) NEW_AC_NUMBER(VARCHAR2) SUB_AC_NUMBER(NUMBER) LOAN_TYPE(VARCHAR2) VALUE_DATE(DATE)  
MATURITY_DATE(DATE) TENOR_YRS(NUMBER) MORAT_MONTHS(VARCHAR2) INTEREST_RATE(NUMBER) APPROVED(NUMBER) DISBURSD(NUMBER)  
NOT_YET_DISBURS(NUMBER) PRINCIPAL(NUMBER) COLTRAL_TYPE(VARCHAR2) CHARGE_AMOUNT(NUMBER) COLLATERAL_VALUE(NUMBER)  
INTEREST_TO_DATE(NUMBER) REPAYMENT(NUMBER) CARRYING_AMOUNT(NUMBER) ARREA_PRINCIPAL(NUMBER) ARREAS_INTEREST(NUMBER)  
ARREAS_TOTAL(NUMBER) ARREAS_CATEGORY(VARCHAR2) DAY_1_30(NUMBER) DAY_31_91(NUMBER) DAY_91_180(NUMBER) DAY_181_270(NUMBER)  
DAY_271_360(NUMBER) SEG_MENT(VARCHAR2) INDUSTRY_CODE(VARCHAR2) IND_DESCR(VARCHAR2) PHONE_1(VARCHAR2) ARM(VARCHAR2)  
BUSINESS_UNIT(VARCHAR2) ENTER_USER_ID(NUMBER) ENTER_DESC(VARCHAR2) INSTALLMENT_AMT(NUMBER)  
== ACCOUNT_STATUS_HISTORY: AC_STATUS_MST_ID(NUMBER) OP_BRANCH_MST_ID(NUMBER) TRANSACTION_DATE(DATE) NEW_AC_NUMBER(VARCHAR2)  
SUB_AC_NUMBER(NUMBER) STATUS_CODE(VARCHAR2) STATUS_EFF_DATE(DATE) FREEZE_DR_AMOUNT(NUMBER) STATUS_CHANGE_REASON(VARCHAR2)  
ACTIVE_FLAG(VARCHAR2) DELETE_FLAG(VARCHAR2) OPERATION_ID(VARCHAR2) OPERATION_DATE(DATE) ENTER_USER_ID(NUMBER)  
ENTER_DESC(VARCHAR2) DELETE_USER_ID(NUMBER) DELETE_DESC(VARCHAR2) PASS_USER_ID(NUMBER) PASS_DESC(VARCHAR2) RENEW_FLAG(VARCHAR2)  
SUB_AC_FLAG(VARCHAR2) PREV_STATUS_CODE(VARCHAR2)  
== INTEREST_SLAB_MASTER: INTEREST_RATE_SLAB_ID(NUMBER) SCHEME_MST_ID(NUMBER) TRAN_DATE(DATE) APPLICABLE_FROM_DATE(DATE)  
APPLICABLE_UPTO_DATE(DATE) MIN_AMOUNT(NUMBER) MAX_AMOUNT(NUMBER) MIN_PERIOD(NUMBER) MIN_PERIOD_UNIT(VARCHAR2)  
MAX_PERIOD(NUMBER) MAX_PERIOD_UNIT(VARCHAR2) PLR_MST_ID(NUMBER) INTEREST_RATE(NUMBER) PREMATURE_RATE(NUMBER)  
EXPIRY_PENAL_RATE(NUMBER) OVERDRAWN_PENAL_RATE(NUMBER) UNUTILIZE_PENAL_RATE(NUMBER) STOCK_PENAL_RATE(NUMBER)  
NON_REVIEW_PENAL_RATE(NUMBER) INST_OVERDUE_PENAL_RATE(NUMBER) ABOVE_FDR_PERCENT(NUMBER) OTHER_RATE(NUMBER)  
ACTIVE_FLAG(VARCHAR2) DELETE_FLAG(VARCHAR2) OPERATION_ID(VARCHAR2) OPERATION_DATE(DATE) ENTER_USER_ID(NUMBER)  
ENTER_DESC(VARCHAR2) DELETE_USER_ID(NUMBER) DELETE_DESC(VARCHAR2) PASS_USER_ID(NUMBER) PASS_DESC(VARCHAR2)  
MIN_CIBIL_SCORE(NUMBER) MAX_CIBIL_SCORE(NUMBER) EXCESS_INST_PENAL_RATE(NUMBER) MIN_INTEREST_RATE(NUMBER) MAX_INTEREST_RATE(NUMBER)  
== INTEREST_SLAB_DETAILS: INTEREST_RATE_DET_ID(NUMBER) NEW_AC_NUMBER(VARCHAR2) SUB_ACCOUNT_NO(NUMBER) TRAN_DATE(DATE)  
APPLICABLE_FROM_DATE(DATE) MIN_AMOUNT(NUMBER) MAX_AMOUNT(NUMBER) INTEREST_POLICY(VARCHAR2) INTEREST_PRODUCT_FREQUENCY(VARCHAR2)  
INTEREST_PAYMENT_FREQUENCY(VARCHAR2) PLR_RATE(NUMBER) VARIATION_RATE(NUMBER) INTEREST_RATE(NUMBER) PREMATURE_RATE(NUMBER)  
EXPIRY_PENAL_RATE(NUMBER) OVERDRAWN_PENAL_RATE(NUMBER) UNUTILIZE_PENAL_RATE(NUMBER) STOCK_PENAL_RATE(NUMBER)  
NON_REVIEW_PENAL_RATE(NUMBER) INST_OVERDUE_PENAL_RATE(NUMBER) ADDITIONAL_SANC_RATE(NUMBER) ACTIVE_FLAG(VARCHAR2)  
DELETE_FLAG(VARCHAR2) OPERATION_ID(VARCHAR2) OPERATION_DATE(DATE) ENTER_USER_ID(NUMBER) ENTER_DESC(VARCHAR2)  
DELETE_USER_ID(NUMBER) DELETE_DESC(VARCHAR2) PASS_USER_ID(NUMBER) PASS_DESC(VARCHAR2) OLD_PRIMARY_KEY(NUMBER)  
MIN_PERIOD(NUMBER) MIN_PERIOD_UNIT(VARCHAR2) MAX_PERIOD(NUMBER) MAX_PERIOD_UNIT(VARCHAR2) E  
== FORGIVE_INTEREST_DETAILS: FORG_INT_DET_ID(NUMBER) OP_BRMST_ID(NUMBER) TRANSACTION_DATE(DATE) NEW_AC_NUMBER(VARCHAR2)  
SUB_AC_NUMBER(NUMBER) INT_CHG_FLAG(VARCHAR2) ORIGINAL_AMT(NUMBER) FORGIVE_AMOUNT(NUMBER) REF_BATCH_NO(NUMBER)  
ACTIVE_FLAG(VARCHAR2) STATUS_DATE(DATE) DELETE_FLAG(VARCHAR2) OPERATION_ID(VARCHAR2) OPERATION_DATE(DATE) ENTER_USER_ID(NUMBER)  
ENTER_DESC(VARCHAR2) DELETE_USER_ID(NUMBER) DELETE_DESC(VARCHAR2) VERIFY_FLAG(VARCHAR2) VERIFY_USER_ID(NUMBER)  
VERIFY_DESC(VARCHAR2) PASS_USER_ID(NUMBER) PASS_DESC(VARCHAR2)  
== AUTO_CHARGES_MASTER: CHARGES_MST_ID(NUMBER) CHARGES_MODULE_TYPE(VARCHAR2) GLCODE(VARCHAR2) SCHEME_ID(NUMBER)  
FROM_AMOUNT(NUMBER) TO_AMOUNT(NUMBER) CHARGES_FROMDATE(DATE) CHARGES_UPTODATE(DATE) CHARGES_TYPE(VARCHAR2)  
CHARGES_SUB_TYPE(VARCHAR2) CHQBOOK_TYPE(VARCHAR2) CHARGES_AMOUNT(NUMBER) CHARGES_TO_EMPLOYEE(VARCHAR2)  
DEDUCT_SERVICE_TAX(VARCHAR2) SEPERATE_STAX_ENTRY(VARCHAR2) DBCR(VARCHAR2) CACLGTRF(VARCHAR2) MIN_AMT_TOPOST(NUMBER)  
CHARGES_EXMP_GRADE(VARCHAR2) INCOME_GLCODE(VARCHAR2) INCOME_ACCNO(VARCHAR2) ACTIVE_FLAG(VARCHAR2) STATUS_DATE(DATE)  
OPERATION_ID(VARCHAR2) OPERATION_DATE(DATE) ENTER_USER_ID(NUMBER) ENTER_DESC(VARCHAR2) PASS_USER_ID(NUMBER) PASS_DESC(VARCHAR2)  
CHARGES_DESC(VARCHAR2) STAX_CRGLCODE(VARCHAR2) STAX_CRACCNO(VARCHAR2) FROM_MONTH(NUMBER) UPTO_MONTH(NUMBER)  
PERCENT_AMT_FLAG(VARCHAR2) NPA_CHARGES_APPLIFLAG(VARCHAR2) CHARGE_ON_EXCESSAMT(VARCHAR2) CHG_OMIT_COUNT(NUMBER)  
== INTEREST_SUMMARY: INTEREST_MST_ID(NUMBER) OPERATION_BRANCH(NUMBER) GLCODE(VARCHAR2) NEW_AC_NUMBER(VARCHAR2)  
SUB_AC_NUMBER(NUMBER) TRANSACTION_DATE(DATE) INTEREST_FROM_DATE(DATE) INTEREST_UPTO_DATE(DATE) INTEREST_RATE(NUMBER)  
PRINCIPAL_PRODUCT(NUMBER) NO_OF_DAYS(NUMBER) NO_OF_MONTHS(NUMBER) INTEREST_AMOUNT(NUMBER) OVERDUE_PRODUCT(NUMBER)  
OVDNO_OF_DAYS(NUMBER) OVD_INT_RATE(NUMBER) PENAL_RATE(NUMBER) OVERDUE_INTEREST(NUMBER) TOTAL_INTEREST(NUMBER)  
CALC_USER_ID(NUMBER) PARTICULARS(VARCHAR2) POST_FLAG(VARCHAR2) POSTED_FLAG(VARCHAR2) INT_TRF_GLCODE(VARCHAR2)  
INT_TRF_NEWACNO(VARCHAR2) INT_TRF_SUBACNO(NUMBER) OVD_INT_FLAG(VARCHAR2) LIEN_GLCODE(VARCHAR2) LIEN_NEWACNO(VARCHAR2)  
LIEN_SUBACNO(NUMBER) INT_FROM_ACOPTD(NUMBER) INT_PAID(NUMBER) PRODUCT_FROM_DATE(DATE) PRODUCT_UPTO_DATE(DATE)  
BRANCH_MST_ID(NUMBER) INTEREST_PAID(NUMBER) ACTUAL_INTEREST(NUMBER) TDS_AMOUNT(NUMBER) INV_GOV_T_TRAN_ID(NUMBER)  
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Write

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.sql

--- content ---

```
-----
-- MAIIC E-Banker: FOLLOW-UP EXTRACTS, 6 October 2026
-- Prepared by Dupleix Institute from the data dictionary results received today.
--
-- Every statement is a SELECT. Nothing is created, changed or deleted.
-- The two ALTER SESSION lines change only how this session displays dates and
-- numbers. Run them first: the dictionary results came back with dates like
-- 7/15/2025, which we would rather not have to guess at.
--
-- Scope = every account on a MAIIC or FInES scheme (scheme ids from DD_09).
-- Export each result as CSV under the name shown. Do not open in Excel.
-- Account numbers must stay as text with their leading zeros.
-----
```

```
ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY-MM-DD HH24:MI:SS';
ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.,';
```

```
-----
-- PRIORITY 1: the five that replace Extracts A, B and C
-----
```

```
-- FU_01_ledger_all.csv
-- Every posting on every in-scope account, all GL codes, all dates, signed,
-- with the narration. About 3,700 rows. This replaces Extract B.
SELECT cv.*
FROM   cumvouch cv
WHERE  cv.delete_flag = 'N'
AND    cv.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY cv.new_ac_number, cv.sub_account_no, cv.transaction_date, cv.cumvouch_det_id;
```

```
-- FU_02_account_master_all.csv
-- Every column of the account master for the in-scope accounts (181 rows,
-- 141 columns). Includes INTEREST_POLICY, FLOATING_FLAG and INTEREST_RATE at
-- account level. This replaces most of Extract A.
SELECT a.*
FROM   acmaster a
WHERE  a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145)
ORDER BY a.new_ac_number, a.sub_ac_number;
```

```
-- FU_03_loan_master_all.csv
-- Every column of the loan master for the same accounts (80 columns).
-- Includes MARGIN_PERCENT, the sanction date and the moratorium fields.
SELECT al.*
FROM   acloanmaster al
WHERE  al.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY al.new_ac_number, al.sub_account_no;
```

```
-- FU_04_rate_setup_per_account.csv
-- The interest rate set-up held against each account, with its effective date,
-- the PLR rate, the variation (spread) and the policy. This is the rate history
-- per loan.
SELECT d.*
FROM   interest_slab_details d
WHERE  d.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY d.new_ac_number, d.applicable_from_date, d.interest_rate_det_id;
```

```
-- FU_05_interest_posted_by_account.csv
-- The interest calculation behind each posting: rate, from and to dates,
-- principal product, days, overdue and penal interest. This replaces Extract C
-- and gives the "how" behind every interest figure.
SELECT s.*
FROM   interest_summary s
WHERE  s.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY s.new_ac_number, s.interest_from_date, s.interest_mst_id;
```

```
-----
-- PRIORITY 2: rates, fees and the loan book history
-----
```

```
-- FU_06_plr_master.csv
-- The prime lending rate history (46 rows). The source of the rate change file.
SELECT * FROM plr_master ORDER BY appli_from_date, plr_mst_id;
```

```
-- FU_07_disbursement_charges.csv
-- Charges posted at disbursement: name, amount, income GL. This is the fee
```

```

-- table we have been asking for. Whole table (about 10,000 rows, all products)
-- so that nothing is filtered out by a wrong assumption.
SELECT * FROM los_disb_charges_post ORDER BY new_ac_number, lo_disb_charg_det_id;

-- FU_08_loan_book_history.csv
-- The stored loan book report, every as-on date, for the in-scope accounts.
-- This should fill the months we do not have (November 2024 to November 2025).
SELECT l.*
FROM loan_book_details_all l
WHERE l.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                          WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY l.asondate, l.new_ac_number;

-- If FU_08 is too large, run this month-end-only version instead:
-- SELECT l.* FROM loan_book_details_all l
-- WHERE l.asondate = LAST_DAY(l.asondate)
-- AND l.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
--                        WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
-- ORDER BY l.asondate, l.new_ac_number;

-- FU_09_balance_history.csv
-- The balance history table for the in-scope accounts (month-end rows with
-- principal, receipts, payments and interest fields).
SELECT b.*
FROM account_balance b
WHERE b.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                          WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY b.new_ac_number, b.transaction_date, b.account_bal_mst_id;

-- FU_10_instalment_chart_current.csv
-- The current instalment chart for every in-scope account, with the amount
-- recovered against each instalment. Current version only (DELETE_FLAG = 'N').
SELECT f.*
FROM fixloan_inst_chart f
WHERE f.delete_flag = 'N'
AND f.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                       WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY f.new_ac_number, f.installment_date, f.inst_chart_det_id;

=====
-- PRIORITY 3: the rest, when convenient
=====

-- FU_11_instalment_plans.csv
SELECT m.*
FROM multi_inst_details m
WHERE m.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                          WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY m.new_ac_number, m.inst_det_id;

-- FU_12_disbursement_schedules.csv
SELECT d.*
FROM loan_disbursement_schedule d
WHERE d.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                          WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY d.new_ac_number, d.disb_schedule_date, d.loan_disb_det_id;

-- FU_13_status_history.csv
-- Every status change on the in-scope accounts, with the reason. Should explain
-- the H and F codes.
SELECT h.*
FROM account_status_history h
WHERE h.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                          WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY h.new_ac_number, h.status_eff_date, h.ac_status_mst_id;

-- FU_14_daily_interest_accrual.csv
-- The daily accrual log for the in-scope accounts.
SELECT d.*
FROM daily_interest_accrual d
WHERE d.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                          WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY d.new_ac_number, d.transaction_date, d.int_accru_det_id;

-- FU_15_slab_master.csv
-- The scheme-level rate slabs (94 rows). Whole table.
SELECT * FROM interest_slab_master ORDER BY scheme_mst_id, applicable_from_date, interest_rate_slab_id;

-- FU_16_forgiven_interest.csv (2 rows)
SELECT * FROM forgive_interest_details ORDER BY transaction_date;
-- FU_17_auto_charges.csv (1 row)
SELECT * FROM auto_charges_master;

```

```
-- FU_18_date_time_check.csv
-- Whether the dates the extracts filter on carry a time of day.
SELECT 'CUMVOUCH.TRANSACTION_DATE' AS date_column,
       COUNT(*) AS rows_checked,
       SUM(CASE WHEN transaction_date <> TRUNC(transaction_date) THEN 1 ELSE 0 END) AS rows_with_time_of_day
FROM   cumvouch
UNION ALL
SELECT 'ACCOUNT_BALANCE.TRANSACTION_DATE', COUNT(*),
       SUM(CASE WHEN transaction_date <> TRUNC(transaction_date) THEN 1 ELSE 0 END)
FROM   account_balance
UNION ALL
SELECT 'FIXLOAN_INST_CHART.INSTALLMENT_DATE', COUNT(*),
       SUM(CASE WHEN installment_date <> TRUNC(installment_date) THEN 1 ELSE 0 END)
FROM   fixloan_inst_chart;

-----
-- End of file. Eighteen result sets.
-----
```

Result of Write

File created successfully at: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.sql (file state is current in your context – no need to Read it back)

CLAUDE (2026-10-06 13:33:35 UTC)

Written for: you and Barry on the call; the SQL file is for Barry to paste.

Barry's results settle the main open questions, and they show the tables that let us stop asking for extracts A, B and C in their current form. I've put the follow-up queries in a new file in the same folder, in priority order.

What the results tell us

- **Interest Policy is found, and it explains the repricing.** It is on the scheme table: MAIIC Agricultural and Industrial are P (Link with PLR); FlnES, Term Loans and Investment loans are S (slab). That matches the loan books exactly. The account master also carries INTEREST_POLICY and FLOATING_FLAG per account.
- **The Fixed/Variable label is confirmed inverted.** The raw flag is F on every MAIIC scheme and I on every FlnES scheme.
- **The margin exists:** ACLOANMASTER.MARGIN_PERCENT, populated on every row.
- **The ledger is signed and the narration says everything.** Debits are negative. Ebenezer Midian's drawdown shows the arrangement fee (9,000,000) and legal fee (5,900,000) as separate disbursement postings with the fee named, and each interest posting names the period and rate ("IntDR Fm 08-AUG-25 To 31-AUG-25 @ 31%").
- **Transaction code 120** (6 rows, 31 December 2025) totals MWK 85,166,683.31, exactly the B to C difference. Codes 308 to 311 are never used. Code 306 (37 debit rows, MWK 1.28bn) is still unexplained.
- **Restructures and write-offs:** both tables are empty, so the blank restructure date is real.
- **Tables we did not know existed:** the PLR rate history, a per-account rate set-up table with the spread and effective date, the interest calculation log (rate, dates, days, product), the disbursement charges table with the income GL, the stored loan book report for every as-on date (which should fill the missing November 2024 to November 2025 months), the status history, and a daily accrual log.

What Barry can do now

1. **Set the date format first.** His exports came out as 7/15/2025. The two ALTER SESSION lines at the top of the new file fix that for the session.
2. **Run FOLLOW-UP extracts for Barry - 6 Oct 2026.sql, Priority 1 first** (five queries, all small):
 - FU_01: the full signed ledger for every in-scope account, all GL codes, all dates, with narrations. About 3,700 rows. This replaces Extract B.
 - FU_02 and FU_03: every column of the account master and loan master for the 181 accounts. This replaces Extract A.
 - FU_04: the per-account rate set-up with effective dates and spread.
 - FU_05: the interest calculation log. This replaces Extract C.
3. **Priority 2 while he is still on:** the PLR history, the disbursement charges table, the loan book history (there is a month-end-only version if the full one is too big), the balance history and the current instalment chart.

Questions to ask him on the call

- What transaction code **306** is (a debit on the loan, 37 postings), and **300** and **343** (two credits each).
- What status **H** and **F** mean. FU_13 should also show it.
- Whether the **"(Manual Int)" schemes** are ever used; they all have zero accounts.

- Which backup copies of the ledger (CUMVOUCH_BAK, CUMVOUCH_CHET13062025, and so on) hold anything the live table does not, since go-live was October 2024 and the live ledger starts July 2024.

One caution: the follow-up queries use only column names confirmed in Barry's dictionary results, but they are still untested. If one fails, the error message will name the column.

Files are in 2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC:

- FOLLOW-UP extracts for Barry - 6 Oct 2026.sql

USER (2026-10-06 13:34:16 UTC)

save this session's complete chat history on the raw scripts folder with the build scripts so that another staff member can follow even with other AI tools

Tool call: Edit

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\build_transcript.py
--- old_string ---
END_MARK = "save in the subfolder a pdf with this claude session"
--- new_string ---
END_MARK = "save this session's complete chat history"
{
  "replace_all": false
}
```

Result of Edit

The file C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\build_transcript.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: Edit

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\build_transcript.py
--- old_string ---
HEADER = f"""This is the record of the part of a Claude Code session (5 October 2026) that analysed the three raw query scripts
MAIIC sent for Extracts A, B and C. It is saved so that other reviewers (Codex, Copilot) can check the work.
```

It starts at the user's request to analyse the scripts and ends with the delivery of the PDF analysis report. Earlier, unrelated parts of the same session are left out.

What is included: every user message, every reply shown to the user, every tool call with its full input (commands, scripts and file contents), and every tool result in full.

What is left out: the assistant's internal reasoning, automatic system reminders (they carry unrelated workspace context), the text of the PDF skill, and image outputs (page renders used for visual checks).

What to review:

1. The reading of the three scripts (FACILITY_REGISTER, CASHFLOW_REGISTER, INTEREST_RECON), reproduced in full in the tool results below.
2. The evidence tests run on Extracts A, B and C (the maiic_check scripts and their outputs).
3. The 24 data dictionary queries. They were written without access to the Oracle database and have NOT been run.
4. The statements in the PDF report MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf.

Known correction made during the session: the first analysis said Extract A shows one disbursement tranche per loan. A later test showed that was wrong (58 of 314 populated rows list two to ten entries); the report and the run-order SQL file carry the corrected finding. The earlier wrong statement is left in this transcript where it was made.

```
Times are UTC. Source: the session log {os.path.basename(SESSION)}, entries {start} to {end - 1}. Events: {len(events)}."""
--- new_string ---
HEADER = f"""This is the complete record of the part of a Claude Code session (5 and 6 October 2026) that dealt with the three
raw query scripts MAIIC sent for Extracts A, B and C. It is saved so that another member of staff, with or without an AI tool,
can follow what was done, in what order, and why.
```

It starts at the request to analyse the scripts and runs to the analysis of the data dictionary results MAIIC returned on 6 October 2026. Earlier, unrelated parts of the same session are left out.

The work falls into five stages, in time order:

1. Analysis of the three scripts and tests against the delivered extract files (5 October).
2. The data dictionary query pack and its run order.
3. The plain-language PDF report, version 1, and the session transcript and build files saved for review.
4. The independent review by Codex, the response to it, and version 2 of the report and query pack, including suggested changes to MAIIC's scripts and to the Duplex engine code.
5. The data dictionary results returned by MAIIC on 6 October, what they showed, and the follow-up extract queries written from them.

What is included: every user message, every reply shown to the user, every tool call with its full input (commands, scripts and file contents), and every tool result in full. Where a long result was stored by the tooling in a side file, that file is reproduced here.

What is left out: the assistant's internal reasoning, automatic system reminders (they carry unrelated workspace context), the text of the PDF skill, and image outputs (page renders used for visual checks).

How to follow it: the files produced along the way are in the same folder as this transcript. The Build files subfolder holds the Python scripts, with a README. The data dictionary results themselves are in the Query Request Responses 6 Oct 2026 folder next to this one.

Corrections made during the session, left in place where they were made: (a) the first analysis said Extract A shows one disbursement tranche per loan, which a later test showed was wrong; (b) version 1 of the report overstated several points that the Codex review identified, and version 2 corrects them. Both are explained in the report's Appendix E.

```
Times are UTC. Source: the session log {os.path.basename(SESSION)}, entries {start} to {end - 1}. Events: {len(events)}."""
{
  "replace_all": false
}
```

Result of Edit

The file C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\build_transcript.py has been updated successfully. (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Rebuild the transcript to cover the whole MAIIC part of the session, and check it

```
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && sed -i 's/title="Claude session transcript: MAIIC raw query scripts", author="/title="Claude session transcript:
MAIIC raw query scripts (complete)", author="/ build_transcript.py && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python
build_transcript.py 2>&1 | tail -4 && ./songenv/Scripts/python -c "
from pypdf import PdfReader
p = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf'
r = PdfReader(p); print('pages', len(r.pages))
t = ' '.join((pg.extract_text() or '') for pg in r.pages)
for needle in ['system-reminder', 'wadzanairombe', 'ZNBS', 'Codex', 'FOLLOW-UP extracts', 'INTEREST_POLICY', 'What Barry can do
now']]:
    print(needle, '->', t.count(needle))
"
{
  "timeout": 600000
}
```

Result of Bash

```
events {'user': 14, 'tool': 82, 'result': 82, 'assistant': 20} | chars 691235
first: USER (2026-10-05 13:02:59 UTC) | last: CLAUDE (2026-10-06 13:33:35 UTC) | Written for: you and Barry on the call; the
SQL file is for Barry to paste. Bar
written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf
pages 133
system-reminder -> 6
wadzanairombe -> 2
ZNBS -> 2
Codex -> 24
FOLLOW-UP extracts -> 4
INTEREST_POLICY -> 11
What Barry can do now -> 1
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

```
description: Find where unrelated workspace text leaked into the transcript
--- command ---
cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC" && grep -n
-i "wadzanairombe\|ZNBS\|system-reminder" Claude_Session_Transcript_Raw_Queries_2026-10-05.md | cut -c1-160
```

Result of Bash

```
3669: t = re.sub(r"", "", t, flags=re.S)
3718: body = re.sub(r"", "", body, flags=re.S).rstrip()
3885: for needle in ['system-reminder', 'wadzanairombe', 'ZNBS', 'PROC_BUILD_MAIIC_FACILITY_REGISTER', 'DD_161', '13-page PDF']:
3903: system-reminder -> 0
3904: wadzanairombe -> 0
3905: ZNBS -> 0
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

```
description: Refresh the build files folder and README to match the complete transcript
--- command ---
cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Build
files" && cp "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp\htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad/build_transcript.py" . && PYTHONIOENCODING=utf-8 "/c/Users/wadza/AppData/Local/Temp/claude/
c--xampp\htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad/songenv/Scripts/python" - <<'PYEOF'
p = "README.md"; s = open(p, encoding="utf-8").read()
old = "- **The session transcript covers version 1 only.** It ends before the Codex review. The review, the extra tests and the
version 2 changes are not in it."
new = ("- **The session transcript is complete.** `Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf` (and the `.md` copy)
runs from the first request on 5 October 2026 to the analysis of MAIIC's data dictionary results on 6 October, including the
Codex review and version 2. Its opening page lists the five stages of the work.\n"
"- **The 6 October analysis of the dictionary results was done with short inline scripts**, not saved files. They are
reproduced in full in the transcript, in the tool calls after the heading for that day. The results they read are in `..\..\
Query Request Responses 6 Oct 2026\Dupleix\Dupleix`. \n"
"- **Reading order for a newcomer:** the report PDF first (version 2), then the transcript, then the scripts here as
needed.")
assert old in s; s = s.replace(old, new)
open(p, "w", encoding="utf-8").write(s); print("readme updated")
PYEOF
ls -la --time-style=long-iso .. | awk '{print $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15, $16}' | tail -8
```

Result of Bash

```
<stdin>:4: SyntaxWarning: "\" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\"\""? A
raw string is also an option.
readme updated
0 2026-10-06 15:27 ..
0 2026-10-05 17:48 Build files
709474 2026-10-06 15:34 Claude_Session_Transcript_Raw_Queries_2026-10-05.md
499920 2026-10-06 15:34 Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf
8925 2026-10-06 15:33 FOLLOW-UP extracts for Barry - 6 Oct 2026.sql
22821 2026-10-05 17:38 MAIIC_EBanker_Data_Dictionary_Queries.sql
147029 2026-10-05 17:47 MAIIC_Raw_Query_Scripts_Analysis_2026-10-05.pdf
23577 2026-10-05 17:39 RUN ORDER - data dictionary queries by priority.sql
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

CLAUDE (2026-10-06 13:35:39 UTC)

Written for: another member of staff picking this up, with or without an AI tool.

The complete transcript is saved in the Query Requests to MAIIC folder, replacing the earlier partial one. It now runs from the first request on 5 October to this afternoon's analysis of Barry's dictionary results: 133 pages as PDF, with a Markdown copy of the same content for AI tools, which read plain text more reliably.

What it contains:

- Every message, every reply, every tool call with its full input (commands, scripts and file contents) and every result in full. Long results that the tooling had stored in side files are reproduced in full.

- An opening page that explains the five stages of the work in order, how to follow it, which files sit alongside it, and the two corrections made along the way.
- Left out, and stated on that page: my internal reasoning, automatic workspace reminders (checked: none of that leaked through), and the page images used for visual checks.

The Build files subfolder is refreshed:

- The transcript builder is the version that produced this file.
- The README now says the transcript is complete, gives a reading order for a newcomer (report first, then transcript, then scripts), and notes that today's analysis of the dictionary results was done with short inline scripts that appear in full in the transcript rather than as saved files.

One thing to be aware of: the transcript records tool outputs as they were, including a few runs that failed and were re-run. That is deliberate, so the record shows the work as it happened.

Files are in 2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC:

- Claude_Session_Transcript_Raw_Queries_2026-10-05.pdf
- Claude_Session_Transcript_Raw_Queries_2026-10-05.md
- Build files\README.md

USER (2026-10-06 13:49:18 UTC)

Priority 2 while he is still on: the PLR history, the disbursement charges table, the loan book history (there is a month-end-only version if the full one is too big), the balance history and the current instalment chart. => is the priority 2 query available

CLAUDE (2026-10-06 13:49:39 UTC)

Yes. Priority 2 is in the same file as Priority 1, FOLLOW-UP extracts for Barry - 6 Oct 2026.sql, under the heading "PRIORITY 2: rates, fees and the loan book history". It is five queries:

```
| Query | Save as | What it returns |
|---|---|---|
| FU_06 | DD_06_plr_master.csv... (file says `FU_06_plr_master.csv`) | The PLR rate history, 46 rows, whole table |
| FU_07 | `FU_07_disbursement_charges.csv` | The disbursement charges table with the income GL, whole table, about 10,000 rows |
| FU_08 | `FU_08_loan_book_history.csv` | The stored loan book report for the in-scope accounts, every as-on date. The month-end-only version is directly below it, commented out; uncomment and run that instead if the full one is too big |
| FU_09 | `FU_09_balance_history.csv` | The balance history for the in-scope accounts |
| FU_10 | `FU_10_instalment_chart_current.csv` | The current instalment chart for the in-scope accounts, with the amount recovered per instalment |
```

Each is a plain SELECT * with the same scope filter as Priority 1 (scheme ids from DD_09), so Barry can paste them one at a time. The two ALTER SESSION lines at the top of the file should already be in place from Priority 1; if he opened a new session, run them again first.

USER (2026-10-06 13:50:30 UTC)

FOLLOW-UP extracts for Barry - 6 Oct 2026.sql, == open it

Tool call: PowerShell

description: Open the follow-up extracts SQL file in Notepad

--- command ---

```
Start-Process notepad.exe -ArgumentList '"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.sql"'
```

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-06 13:50:39 UTC)

It should now be open in Notepad. Priority 2 starts about a third of the way down, under the heading "PRIORITY 2: rates, fees and the loan book history" (FU_06 to FU_10).

USER (2026-10-06 14:23:28 UTC)

Can the query request start by listing all the csv files to be sent to us under each priority numbers

Tool call: Edit

```
file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.sql
--- old_string ---
-- Scope = every account on a MAIIC or FinES scheme (scheme ids from DD_09).
-- Export each result as CSV under the name shown. Do not open in Excel.
-- Account numbers must stay as text with their leading zeros.
-----
--- new_string ---
-- Scope = every account on a MAIIC or FinES scheme (scheme ids from DD_09).
-- Export each result as CSV under the name shown. Do not open in Excel.
-- Account numbers must stay as text with their leading zeros.
--
-- FILES TO SEND BACK (18 in all)
--
-- PRIORITY 1 (replaces Extracts A, B and C)
--   FU_01_ledger_all.csv           every posting, all GL codes, signed, with narration
--   FU_02_account_master_all.csv   account master, every column, 181 accounts
--   FU_03_loan_master_all.csv      loan master, every column (includes MARGIN_PERCENT)
--   FU_04_rate_setup_per_account.csv rate set-up per account: policy, PLR, spread, effective date
--   FU_05_interest_posted_by_account.csv the calculation behind each interest posting
--
-- PRIORITY 2 (rates, fees and the loan book history)
--   FU_06_plr_master.csv           the PLR rate history, 46 rows
--   FU_07_disbursement_charges.csv charges posted at disbursement, with income GL
--   FU_08_loan_book_history.csv     the stored loan book report, every as-on date
--                                   (or the month-end-only version if too large)
--   FU_09_balance_history.csv       balance history, month-end rows
--   FU_10_instalment_chart_current.csv current instalment chart with amount recovered
--
-- PRIORITY 3 (when convenient)
--   FU_11_instalment_plans.csv
--   FU_12_disbursement_schedules.csv
--   FU_13_status_history.csv        every status change with its reason (explains H and F)
--   FU_14_daily_interest_accrual.csv
--   FU_15_slab_master.csv           scheme-level rate slabs, 94 rows
--   FU_16_forgiven_interest.csv     2 rows
--   FU_17_auto_charges.csv          1 row
--   FU_18_date_time_check.csv       whether dates carry a time of day
-----
{
  "replace_all": false
}
```

Result of Edit

The file C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.sql has been updated successfully. (file state is current in your context – no need to Read it back)

CLAUDE (2026-10-06 14:24:01 UTC)

Done. The file now opens with a "FILES TO SEND BACK" list: all 18 CSV names grouped under Priority 1, 2 and 3, each with a one-line description, before the session settings and the queries.

Notepad will still show the old version until you close and reopen the file (or if it prompts to reload, accept).

USER (2026-10-06 14:24:23 UTC)

open the updated file lets see

Tool call: PowerShell

description: Open the updated follow-up extracts file in Notepad

--- command ---

```
Start-Process notepad.exe -ArgumentList '"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.sql"'
```

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-06 14:24:31 UTC)

The updated file is open in a new Notepad window. The "FILES TO SEND BACK" list is near the top, right after the scope note; if an older window of the same file is still open, close that one.

USER (2026-10-06 14:26:24 UTC)

```
ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY-MM-DD HH24:MI:SS';
```

```
ALTER SESSION SET NLS_NUMERIC_CHARACTERS = ',';
```

=> if they are running every query one by one, to they need to run those alter queries on top of each query

CLAUDE (2026-10-06 14:26:31 UTC)

No. The two ALTER SESSION lines apply to the whole database session, so Barry runs them once after logging in and every query he runs afterwards in that same connection uses them.

He only needs to run them again if:

- he disconnects or reconnects, or opens a new worksheet that uses a separate connection;
- SQL Developer is restarted.

One caution: the date format only affects queries that return date columns as dates, which is all of ours. If his export tool has its own date setting (SQL Developer has one under Preferences, Database, NLS), that setting can override the session. The quick check is FU_06, the PLR history: if APPLI_FROM_DATE comes out as 2024-11-05 the setting is working; if it comes out as 11/5/2024 it is not, and he should set the same format in the tool's NLS preferences and re-export.

USER (2026-10-06 14:26:58 UTC)

how does hoe restore to what was there before if he wants

CLAUDE (2026-10-06 14:27:04 UTC)

He doesn't need to do anything. The ALTER SESSION settings last only as long as his connection: they disappear when he disconnects or closes SQL Developer, and nothing is stored in the database. The next time he logs in, the session starts with the normal defaults.

If he wants to put them back during the same session without reconnecting, he can run:

```
ALTER SESSION SET NLS_DATE_FORMAT = 'DD-MON-RR';  
ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.,';
```

DD-MON-RR is Oracle's usual default, and the numeric setting is already the default on an English-language database, so that line changes nothing either way. If he wants to see exactly what his session has before changing anything, this shows it:

```
SELECT parameter, value FROM nls_session_parameters  
WHERE parameter IN ('NLS_DATE_FORMAT', 'NLS_NUMERIC_CHARACTERS');
```

USER (2026-10-06 14:27:30 UTC)

no there is no need to put them back during the session

CLAUDE (2026-10-06 14:27:34 UTC)

Agreed. Run them once at the start, run the queries, and when he disconnects they're gone on their own.

USER (2026-10-06 14:29:13 UTC)

okay we also want the query lists to be numbered as will did before so the file namea must have a prifix to show the query number and the priority number. I also want a comment to explain the alter statements to give barry the confidence that they only last for the session login. Can you update the queryies and show me

Tool call: Write

file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.sql

```
--- content ---
-----
-- MAIIC E-Banker: FOLLOW-UP EXTRACTS, 6 October 2026
-- Prepared by Dupleix Institute from the data dictionary results received today.
--
-- Every query is a SELECT. Nothing is created, changed or deleted.
--
-- Scope = every account on a MAIIC or FInES scheme (scheme ids from DD_09).
-- Run the queries in the order numbered below. Export each result as CSV under
-- the file name shown: the prefix gives the priority and the run number, so
-- P2_07 is priority 2, run 7. Do not open the CSV files in Excel. Account
-- numbers must stay as text with their leading zeros.
--
-- FILES TO SEND BACK (18 in all)
--
-- PRIORITY 1 (replaces Extracts A, B and C)
--   P1_01_ledger_all.csv           every posting, all GL codes, signed, with narration
--   P1_02_account_master_all.csv   account master, every column, 181 accounts
--   P1_03_loan_master_all.csv      loan master, every column (includes MARGIN_PERCENT)
--   P1_04_rate_setup_per_account.csv rate set-up per account: policy, PLR, spread, effective date
--   P1_05_interest_posted_by_account.csv the calculation behind each interest posting
--
-- PRIORITY 2 (rates, fees and the loan book history)
--   P2_06_plr_master.csv           the PLR rate history, 46 rows
--   P2_07_disbursement_charges.csv charges posted at disbursement, with income GL
--   P2_08_loan_book_history.csv     the stored loan book report, every as-on date
--                                   (or the month-end-only version if too large)
--   P2_09_balance_history.csv       balance history, month-end rows
--   P2_10_instalment_chart_current.csv current instalment chart with amount recovered
--
-- PRIORITY 3 (when convenient)
--   P3_11_instalment_plans.csv
--   P3_12_disbursement_schedules.csv
--   P3_13_status_history.csv        every status change with its reason (explains H and F)
--   P3_14_daily_interest_accrual.csv
--   P3_15_slab_master.csv           scheme-level rate slabs, 94 rows
--   P3_16_forgiven_interest.csv     2 rows
--   P3_17_auto_charges.csv          1 row
--   P3_18_date_time_check.csv       whether dates carry a time of day
-----

=====
-- RUN 0 | SESSION SETTINGS: run these two lines once, after logging in.
=====
-- What they do: they tell THIS login session to display dates as 2026-10-06
-- 14:30:00 and numbers with a dot for the decimal point. Without them the
-- exports come out as 7/15/2025, which we then have to guess at.
--
-- What they do NOT do: they change nothing in the database. No table, no
-- setting and no other user is affected. They last only as long as this
-- connection: when you disconnect or close SQL Developer they are gone, and the
-- next login starts with the normal defaults. There is nothing to put back.
--
-- Run once, then run the queries below in the same connection. If you reconnect
-- or open a new connection, run these two lines again first.
--
-- To see what the session is using at any time:
-- SELECT parameter, value FROM nls_session_parameters
-- WHERE parameter IN ('NLS_DATE_FORMAT', 'NLS_NUMERIC_CHARACTERS');

ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY-MM-DD HH24:MI:SS';
ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.,';

=====
-- PRIORITY 1: the five that replace Extracts A, B and C
=====
-- RUN 1 of 18 | PRIORITY 1 | save as P1_01_ledger_all.csv
-- Every posting on every in-scope account, all GL codes, all dates, signed,
-- with the narration. About 3,700 rows. This replaces Extract B.
SELECT cv.*
FROM cumvouch cv
WHERE cv.delete_flag = 'N'
AND cv.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                        WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY cv.new_ac_number, cv.sub_account_no, cv.transaction_date, cv.cumvouch_det_id;

-- RUN 2 of 18 | PRIORITY 1 | save as P1_02_account_master_all.csv
-- Every column of the account master for the in-scope accounts (181 rows,
-- 141 columns). Includes INTEREST_POLICY, FLOATING_FLAG and INTEREST_RATE at
-- account level. This replaces most of Extract A.
SELECT a.*
FROM acmaster a
WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145)
ORDER BY a.new_ac_number, a.sub_ac_number;
```

```

-- RUN 3 of 18 | PRIORITY 1 | save as P1_03_loan_master_all.csv
-- Every column of the loan master for the same accounts (80 columns).
-- Includes MARGIN_PERCENT, the sanction date and the moratorium fields.
SELECT a1.*
FROM acloanmaster a1
WHERE a1.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY a1.new_ac_number, a1.sub_account_no;

-- RUN 4 of 18 | PRIORITY 1 | save as P1_04_rate_setup_per_account.csv
-- The interest rate set-up held against each account, with its effective date,
-- the PLR rate, the variation (spread) and the policy. This is the rate history
-- per loan.
SELECT d.*
FROM interest_slab_details d
WHERE d.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY d.new_ac_number, d.applicable_from_date, d.interest_rate_det_id;

-- RUN 5 of 18 | PRIORITY 1 | save as P1_05_interest_posted_by_account.csv
-- The interest calculation behind each posting: rate, from and to dates,
-- principal product, days, overdue and penal interest. This replaces Extract C
-- and gives the "how" behind every interest figure.
SELECT s.*
FROM interest_summary s
WHERE s.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY s.new_ac_number, s.interest_from_date, s.interest_mst_id;

-----
-- PRIORITY 2: rates, fees and the loan book history
-----

-- RUN 6 of 18 | PRIORITY 2 | save as P2_06_plr_master.csv
-- The prime lending rate history (46 rows). The source of the rate change file.
SELECT * FROM plr_master ORDER BY appli_from_date, plr_mst_id;

-- RUN 7 of 18 | PRIORITY 2 | save as P2_07_disbursement_charges.csv
-- Charges posted at disbursement: name, amount, income GL. This is the fee
-- table we have been asking for. Whole table (about 10,000 rows, all products)
-- so that nothing is filtered out by a wrong assumption.
SELECT * FROM los_disb_charges_post ORDER BY new_ac_number, lo_disb_charg_det_id;

-- RUN 8 of 18 | PRIORITY 2 | save as P2_08_loan_book_history.csv
-- The stored loan book report, every as-on date, for the in-scope accounts.
-- This should fill the months we do not have (November 2024 to November 2025).
SELECT l.*
FROM loan_book_details_all l
WHERE l.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY l.asondate, l.new_ac_number;

-- If RUN 8 is too large, run this month-end-only version instead and save it
-- under the same file name, P2_08_loan_book_history.csv:
-- SELECT l.* FROM loan_book_details_all l
-- WHERE l.asondate = LAST_DAY(l.asondate)
-- AND l.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
--                          WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
-- ORDER BY l.asondate, l.new_ac_number;

-- RUN 9 of 18 | PRIORITY 2 | save as P2_09_balance_history.csv
-- The balance history table for the in-scope accounts (month-end rows with
-- principal, receipts, payments and interest fields).
SELECT b.*
FROM account_balance b
WHERE b.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY b.new_ac_number, b.transaction_date, b.account_bal_mst_id;

-- RUN 10 of 18 | PRIORITY 2 | save as P2_10_instalment_chart_current.csv
-- The current instalment chart for every in-scope account, with the amount
-- recovered against each instalment. Current version only (DELETE_FLAG = 'N').
SELECT f.*
FROM fixloan_inst_chart f
WHERE f.delete_flag = 'N'
AND f.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                         WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY f.new_ac_number, f.installment_date, f.inst_chart_det_id;

-----
-- PRIORITY 3: the rest, when convenient
-----
-- RUN 11 of 18 | PRIORITY 3 | save as P3_11_instalment_plans.csv

```

```

SELECT m.*
FROM multi_inst_details m
WHERE m.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY m.new_ac_number, m.inst_det_id;

-- RUN 12 of 18 | PRIORITY 3 | save as P3_12_disbursement_schedules.csv
SELECT d.*
FROM loan_disbursement_schedule d
WHERE d.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY d.new_ac_number, d.disb_schedule_date, d.loan_disb_det_id;

-- RUN 13 of 18 | PRIORITY 3 | save as P3_13_status_history.csv
-- Every status change on the in-scope accounts, with the reason. Should explain
-- the H and F codes.
SELECT h.*
FROM account_status_history h
WHERE h.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY h.new_ac_number, h.status_eff_date, h.ac_status_mst_id;

-- RUN 14 of 18 | PRIORITY 3 | save as P3_14_daily_interest_accrual.csv
-- The daily accrual log for the in-scope accounts.
SELECT d.*
FROM daily_interest_accrual d
WHERE d.new_ac_number IN (SELECT a.new_ac_number FROM acmaster a
                           WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145))
ORDER BY d.new_ac_number, d.transaction_date, d.int_accru_det_id;

-- RUN 15 of 18 | PRIORITY 3 | save as P3_15_slab_master.csv
-- The scheme-level rate slabs (94 rows). Whole table.
SELECT * FROM interest_slab_master ORDER BY scheme_mst_id, applicable_from_date, interest_rate_slab_id;

-- RUN 16 of 18 | PRIORITY 3 | save as P3_16_forgiven_interest.csv (2 rows)
SELECT * FROM forgive_interest_details ORDER BY transaction_date;

-- RUN 17 of 18 | PRIORITY 3 | save as P3_17_auto_charges.csv (1 row)
SELECT * FROM auto_charges_master;

-- RUN 18 of 18 | PRIORITY 3 | save as P3_18_date_time_check.csv
-- Whether the dates the extracts filter on carry a time of day.
SELECT 'CUMVOUCH.TRANSACTION_DATE' AS date_column,
       COUNT(*) AS rows_checked,
       SUM(CASE WHEN transaction_date <> TRUNC(transaction_date) THEN 1 ELSE 0 END) AS rows_with_time_of_day
FROM cumvouch
UNION ALL
SELECT 'ACCOUNT_BALANCE.TRANSACTION_DATE', COUNT(*),
       SUM(CASE WHEN transaction_date <> TRUNC(transaction_date) THEN 1 ELSE 0 END)
FROM account_balance
UNION ALL
SELECT 'FIXLOAN_INST_CHART.INSTALLMENT_DATE', COUNT(*),
       SUM(CASE WHEN installment_date <> TRUNC(installment_date) THEN 1 ELSE 0 END)
FROM fixloan_inst_chart;

-----
-- End of file. Eighteen result sets, P1_01 to P3_18.
-----

```

Tool call: PowerShell

description: Open the updated file in Notepad

--- command ---

```
Start-Process notepad.exe -ArgumentList '"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.sql"'
```

Result of Write

The file C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.sql has been updated successfully. (file state is current in your context – no need to Read it back)

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-06 14:30:20 UTC)

Updated and opened in a new Notepad window (close the older window of the same file).

What changed:

- **File names now carry the priority and run number:** P1_01_ledger_all.csv through P3_18_date_time_check.csv. The list at the top uses the same names, so the files sort in run order when they come back.
- **Every query is headed the same way as before:** RUN 7 of 18 | PRIORITY 2 | save as P2_07_disbursement_charges.csv.
- **The session settings have their own block, "RUN 0",** with a plain explanation for Barry: what the two lines do, that they change nothing in the database and affect no other user, that they last only for this connection and vanish on disconnect with nothing to put back, and the one-line query that shows what the session is using.
- **The month-end alternative for run 8** now says to save it under the same name, P2_08_loan_book_history.csv, so we always receive one file for that slot.

USER (2026-10-06 14:32:10 UTC)

which tiems on the data requests where closed by these queries?

CLAUDE (2026-10-06 14:32:53 UTC)

Measured against the two lists we sent MAIIC (the "still open" items in the 24 September follow-up email and the ten open clarifications in the research report), this is where things stand. "Closed" means answered by the dictionary results already received; "closes on receipt" means the table has been found and the follow-up query returns it; "open" still needs a person.

From the 24 September follow-up email

#	Item	Status	How
1	Interest Policy, Floating Flag, calculation and instalment basis, moratorium, scheme code per loan	Closed at scheme level; per account closes on receipt	DD_10 gives every scheme setting; P1_02 and P1_03 give the account-level policy, flag and moratorium fields. Moratorium type still to be confirmed from the loan master columns.
2	Loan Book Report for every month from October 2024	Closes on receipt, coverage to verify	`LOAN\_BOOK\_DETAILS\_ALL` holds the stored report by as-on date (P2_08).
3	GL-wise Interest Posted Report monthly from October 2024	Closes on receipt	`INTEREST\_SUMMARY` gives rate, dates, days and amount per posting (P1_05), and the ledger gives the postings themselves (P1_01).
4	The completed fee template	Largely closes on receipt	`LOS\_DISB\_CHARGES\_POST` is the fee table (P2_07), and the ledger narrations name fees deducted at drawdown (P1_01). Fees Finance handled outside the system would still need the template.
5	Materiality threshold used in the 2024 and 2025 assessments	Open	Needs Dr Thom.
6	Which record is the contractual one	Open	Needs a written answer from MAIIC.
7	Future extracts as CSV with year-first dates	Closed by process	The session settings at RUN 0.
Vendor	Audit Trail bulk export of rate history	Replaced	`INTEREST\_SLAB\_DETAILS` per account (P1_04) and `PLR\_MASTER` (P2_06).
Vendor	Reschedule Report	Closed, nothing to report	The reschedule table has no rows.
Vendor	Loan Book Report change request	Largely unnecessary	P1_02 and P1_03 return every field we asked to have added.

From the research report's ten clarifications

#	Question	Status	How
1	Is the EMI chart stored?	Closed	Stored, with a row at the start date and the amount recovered (DD_16d).
2	Meaning of transaction types	Partly closed	Signs known for every code (DD_12); 120 equals the B to C difference; 308 to 311 never used. 306, 300 and 343 still need the vendor, though the narrations in P1_01 will probably say.
3	Status codes H and F	Open; P3_13 may close it	The status history carries a change reason.
4	NUM_INSTALLMENTS is tenor plus one	Closed	The chart's first row sits at the start date with a zero amount.
5	One disbursement tranche per loan	Closed	Schedule table; actual drawdowns are the 301 postings (P1_01).
6	Sub-account always 1, restructure date blank	Closed	Both restructure tables are empty; restructures are not recorded in E-Banker.
7	Scheme per account	Closed	`SCHEME\_MST\_ID` on the account master; DD_09 lists them.
8	Repeatable rate change extract	Closes on receipt	`PLR\_MASTER` (P2_06).
9	Go-live date and past loan books	Mostly closes on receipt	Ledger postings start 31 July 2024 (DD_12); P2_08 for the loan books.
10	Loan book customisation effort	No longer needed	See the vendor row above.

Also closed on receipt: the reference rate and spread per loan (PLR_RATE and VERIATION_RATE in P1_04, MARGIN_PERCENT in P1_03) and the per-drawdown dates (P1_01).

Still open with no table found yet: the LOS application and process reference numbers, and the funded/non-funded flag. Those may sit in a separate LOS schema.

USER (2026-10-06 14:36:15 UTC)

write an email to Barry and the other project members acknowledging the raw query requests and how they have managed to close some gaps as show below. However, please the following 18 ammended queries to resubmit the Extract A, B, C which would resolve the reconciliation issues. Be grateful that the submissions have unlocked most of the open items. Rephrase as you see fit

Tool call: Write

```
file_path: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Correspondence - Information Requests\2026-10-06
Thank you for the dictionary results and the 18 follow-up extracts (draft).md
--- content ---
# Draft email: dictionary results received, and the 18 follow-up extracts

**To:** Barry Makumba; Dr Thomson Kumwenda; Tamanda; Dibwa
**Cc:** Kundai Muriwo; Wadzanai Rombe
**Subject:** MAIIC EIR: thank you for the dictionary results, and the 18 extracts that replace Extracts A, B and C
**Attachment:** FOLLOW-UP extracts for Barry - 6 Oct 2026.sql

---
```

Dear Barry, Dr Thom, Tamanda and Dibwa,

Thank you, Barry, for turning the data dictionary queries round so quickly. All twenty-four results arrived this afternoon, every one of them ran, and between them they have unlocked most of what has been open since September. This is the most progress we have made in one day on the data side of the project.

What the results have settled

- **Interest Policy** is found. It is held on each scheme, and on each account. MAIIC Agricultural and Industrial loans are "Link with PLR", so they follow the Reserve Bank rate; FInES, Term and Investment loans are on a slab rate, which is why they never moved. That matches the loan books exactly and ends the guesswork on repricing.
- **The Fixed/Variable label** in Extract A was the wrong way round. The raw flag is "floating" on every MAIIC scheme and "fixed" on every FInES scheme. The script's translation had the two swapped. No harm done, now that we know.
- **The spread over the prime rate exists** as a field on every loan, and the per-account rate set-up table carries the PLR rate, the spread and the date each rate took effect. That is the rate history we were going to ask the vendor to export from the audit trail.
- **The ledger is signed and self-describing.** Debits are negative, credits positive, and each posting's narration names what it is: "arrangement fee", "legal fees", "IntDR Fm 08-AUG-25 To 31-AUG-25 @ 31%". The debit/credit question and most of the "what does this posting mean" questions answer themselves.
- **The fee table exists:** the disbursement charges table holds each charge by name, amount and income GL. The fee template we sent may now only be needed for anything handled outside the system.
- **The loan book history is stored** for every as-on date, which should fill the months from November 2024 to November 2025 that we do not have.
- **The interest calculation log exists**, with the rate, the period, the days and the amount behind every interest posting. That is the "how" behind Extract C.
- **Restructures and write-offs:** both tables are empty, so the blank restructure date in Extract A is real, and the restructure history lives in Tamanda's register, as we understood.
- **The instalment chart is stored**, with the amount recovered against each instalment, and the chart starts with a row at the loan's start date, which explains why the instalment count was always one more than the tenor.
- **Transaction code 120** on six accounts at 31 December 2025 adds up to the cent to the difference we had found between Extracts B and C.

What we now ask for: eighteen extracts that replace Extracts A, B and C

Rather than re-run the three extract procedures, we would like to take the data straight from the tables the dictionary has shown us. The attached file holds eighteen read-only queries, numbered in the order to run them and grouped in three priorities. Each one is a plain SELECT; nothing is created, changed or deleted.

- **Priority 1 (runs 1 to 5)** replaces the three extracts: the full signed ledger with narrations, the account master and loan master with every column, the rate set-up per account, and the interest calculation log. These resolve the reconciliation issues we raised: direction of postings, interest charged against cash received, the rate history, and the fees.
- **Priority 2 (runs 6 to 10)**: the PLR history, the disbursement charges, the stored loan book history, the balance history and the current instalment chart.
- **Priority 3 (runs 11 to 18)**: the remaining supporting tables, when convenient.

Two practical points, both explained in the file:

1. **Run the two session settings first (RUN 0).** They make dates come out as 2026-10-06 rather than 7/15/2025, and numbers with a dot for the decimal point. They change nothing in the database, affect no other user, and last only for that login; when Barry disconnects they are gone and there is nothing to put back.
2. **Please save each result as CSV under the file name shown** (P1_01 to P3_18, so the prefix tells us the priority and the run), and do not open the files in Excel before sending. Excel is what swapped days and months in the earlier extracts and it drops the leading zeros from account numbers.

Barry, the Priority 1 queries are small (the ledger is about 3,700 rows) and should run in minutes. If any query fails, the error will name the column, and a screenshot of it is all we need to fix it.

Three things only you can tell us

- What transaction codes ****306****, ****300**** and ****343**** are. We can see their direction but not their meaning.
- What account status ****H**** and ****F**** mean. The status history (run 13) may show the reason, but a one-line answer would be quicker.
- Whether the "(Manual Int)" schemes are ever used. They all have zero accounts today.

Still open from the September list

- The materiality threshold used in the 2024 and 2025 EIR assessments (Dr Thom).
- Written confirmation of which record is the contractual one: the signed offer letter or the system.
- The LOS application and process reference numbers, and the funded/non-funded flag, which we have not yet found in this schema. They may sit with the origination system.

Thank you again. The results have moved us from asking what E-Banker holds to being able to say exactly which table and column we need, and the eighteen queries are the proof of that.

Kind regards,

Edward Mazibuko CA(SA)

Result of Write

File created successfully at: C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Correspondence - Information Requests\2026-10-06 Thank you for the dictionary results and the 18 follow-up extracts (draft).md (file state is current in your context – no need to Read it back)

CLAUDE (2026-10-06 14:37:11 UTC)

Written for: Barry, Dr Thom, Tamanda and Dibwa at MAIIC, from Edward, with Kundai and you in copy.

The draft is saved alongside the earlier email drafts, and the text is below ready to paste. It names the attachment as the follow-up SQL file.

Subject: MAIIC EIR: thank you for the dictionary results, and the 18 extracts that replace Extracts A, B and C

Dear Barry, Dr Thom, Tamanda and Dibwa,

Thank you, Barry, for turning the data dictionary queries round so quickly. All twenty-four results arrived this afternoon, every one of them ran, and between them they have unlocked most of what has been open since September. This is the most progress we have made in one day on the data side of the project.

What the results have settled

- **Interest Policy is found.** It is held on each scheme, and on each account. MAIIC Agricultural and Industrial loans are "Link with PLR", so they follow the Reserve Bank rate; FlnES, Term and Investment loans are on a slab rate, which is why they never moved. That matches the loan books exactly and ends the guesswork on repricing.
- **The Fixed/Variable label in Extract A was the wrong way round.** The raw flag is "floating" on every MAIIC scheme and "fixed" on every FlnES scheme. The script's translation had the two swapped. No harm done, now that we know.
- **The spread over the prime rate exists** as a field on every loan, and the per-account rate set-up table carries the PLR rate, the spread and the date each rate took effect. That is the rate history we were going to ask the vendor to export from the audit trail.
- **The ledger is signed and self-describing.** Debits are negative, credits positive, and each posting's narration names what it is: "arrangement fee", "legal fees", "IntDR Fm 08-AUG-25 To 31-AUG-25 @ 31%". The debit/credit question and most of the "what does this posting mean" questions answer themselves.
- **The fee table exists:** the disbursement charges table holds each charge by name, amount and income GL. The fee template we sent may now only be needed for anything handled outside the system.
- **The loan book history is stored** for every as-on date, which should fill the months from November 2024 to November 2025 that we do not have.
- **The interest calculation log exists**, with the rate, the period, the days and the amount behind every interest posting. That is the "how" behind Extract C.
- **Restructures and write-offs:** both tables are empty, so the blank restructure date in Extract A is real, and the restructure history lives in Tamanda's register, as we understood.
- **The instalment chart is stored**, with the amount recovered against each instalment, and the chart starts with a row at the loan's start date, which explains why the instalment count was always one more than the tenor.
- **Transaction code 120** on six accounts at 31 December 2025 adds up to the cent to the difference we had found between Extracts B and C.

What we now ask for: eighteen extracts that replace Extracts A, B and C

Rather than re-run the three extract procedures, we would like to take the data straight from the tables the dictionary has shown us. The attached file holds eighteen read-only queries, numbered in the order to run them and grouped in three priorities. Each one is a plain SELECT; nothing is created, changed or deleted.

- **Priority 1 (runs 1 to 5)** replaces the three extracts: the full signed ledger with narrations, the account master and loan master with every column, the rate set-up per account, and the interest calculation log. These resolve the reconciliation issues we raised: direction of postings, interest charged against cash received, the rate history, and the fees.
- **Priority 2 (runs 6 to 10):** the PLR history, the disbursement charges, the stored loan book history, the balance history and the current instalment chart.
- **Priority 3 (runs 11 to 18):** the remaining supporting tables, when convenient.

Two practical points, both explained in the file:

1. **Run the two session settings first (RUN 0).** They make dates come out as 2026-10-06 rather than 7/15/2025, and numbers with a dot for the decimal point. They change nothing in the database, affect no other user, and last only for that login; when Barry disconnects they are gone and there is nothing to put back.
2. **Please save each result as CSV under the file name shown** (P1_01 to P3_18, so the prefix tells us the priority and the run), and do not open the files in Excel before sending. Excel is what swapped days and months in the earlier extracts and it drops the leading zeros from account numbers.

Barry, the Priority 1 queries are small (the ledger is about 3,700 rows) and should run in minutes. If any query fails, the error will name the column, and a screenshot of it is all we need to fix it.

Three things only you can tell us

- What transaction codes **306**, **300** and **343** are. We can see their direction but not their meaning.
- What account status **H** and **F** mean. The status history (run 13) may show the reason, but a one-line answer would be quicker.
- Whether the "(Manual Int)" schemes are ever used. They all have zero accounts today.

Still open from the September list

- The materiality threshold used in the 2024 and 2025 EIR assessments (Dr Thom).
- Written confirmation of which record is the contractual one: the signed offer letter or the system.
- The LOS application and process reference numbers, and the funded/non-funded flag, which we have not yet found in this schema. They may sit with the origination system.

Thank you again. The results have moved us from asking what E-Banker holds to being able to say exactly which table and column we need, and the eighteen queries are the proof of that.

Kind regards,

Edward Mazibuko CA(SA)

Dupleix Institute

Files are in 3. Project Execution\Correspondence - Information Requests:

- 2026-10-06 Thank you for the dictionary results and the 18 follow-up extracts (draft).md

USER (2026-10-06 14:37:11 UTC)

We want to attach the queries file as both pdf, .sql and .txt on the same email so the email must be clear that we want these submitted by 10am tomorrow

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\sql_to_pdf.py
--- content ---
"""Make PDF and TXT copies of the follow-up SQL file for the email."""
import shutil
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import BaseDocTemplate, PageTemplate, Frame, Preformatted, Paragraph, Spacer

D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC"
BASE = D + r"\FOLLOW-UP extracts for Barry - 6 Oct 2026"
text = open(BASE + ".sql", encoding="utf-8").read().replace("\r\n", "\n")
shutil.copyfile(BASE + ".sql", BASE + ".txt")

F = r"C:\Windows\Fonts"
pdfmetrics.registerFont(TTFont("Mono", F + r"\consola.ttf"))
pdfmetrics.registerFont(TTFont("Seg", F + r"\segoeui.ttf"))
pdfmetrics.registerFont(TTFont("Seg-B", F + r"\segoeui.ttf"))
GREY = colors.HexColor("#5B6470"); LINE = colors.HexColor("#C9D1DB"); NAVY = colors.HexColor("#1B2A41")
st_title = ParagraphStyle("t", fontName="Seg-B", fontSize=16, leading=20, textColor=NAVY, spaceAfter=4)
st_p = ParagraphStyle("p", fontName="Seg", fontSize=9.5, leading=13, textColor=GREY, spaceAfter=8)
st_mono = ParagraphStyle("m", fontName="Mono", fontSize=8, leading=10.2)

story = [Paragraph("MAIIC E-Banker: follow-up extracts, 6 October 2026", st_title),
          Paragraph("PDF copy of the file FOLLOW-UP extracts for Barry - 6 Oct 2026.sql, for reading. Please run the queries "
                    "from the .sql or .txt attachment, which carry the same text. Eighteen read-only queries, numbered in run"
                    "order.", st_p),
          Spacer(1, 4)]
lines = text.split("\n")
for i in range(0, len(lines), 60):
    story.append(Preformatted("\n".join(lines[i:i + 60]), st_mono, maxLineLength=104, splitChars=" ", newLineChars=""))

def footer(canvas, doc):
    canvas.saveState(); canvas.setStrokeColor(LINE); canvas.setLineWidth(0.5)
    canvas.line(16 * mm, 12 * mm, A4[0] - 16 * mm, 12 * mm)
    canvas.setFont("Seg", 8); canvas.setFillColor(GREY)
    canvas.drawString(16 * mm, 8 * mm, "FOLLOW-UP extracts for Barry, 6 October 2026 · Duplex Institute · read-only"
                     "queries")
    canvas.drawRightString(A4[0] - 16 * mm, 8 * mm, f"Page {doc.page}"); canvas.restoreState()

doc = BaseDocTemplate(BASE + ".pdf", pagesize=A4, leftMargin=16 * mm, rightMargin=16 * mm, topMargin=14 * mm, bottomMargin=18 *
mm,
                    title="MAIIC E-Banker: follow-up extracts, 6 October 2026", author="Duplex Institute")
doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, id="f",
leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)],
onPage=footer)])
doc.build(story)
print("written .pdf and .txt")
```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\sql_to_pdf.py (file state is current in your context – no need to Read it back)

Tool call: Bash

```
description: Create the PDF and TXT copies of the follow-up queries file
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python sql_to_pdf.py && ./songenv/Scripts/python -c "
from pypdf import PdfReader
r = PdfReader(r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
FOLLOW-UP extracts for Barry - 6 Oct 2026.pdf'); print('pages', len(r.pages))
" && ls -la --time-style=long-iso "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients\Raw Query Scripts/
Query Requests to MAIIC/" | grep FOLLOW | awk '{print $5, $6, $7, $8, $9, $10, $11, $12, $13, $14, $15}'
{
  "timeout": 120000
}
```

Result of Bash

```
written .pdf and .txt
pages 4
72862 2026-10-06 16:37 FOLLOW-UP extracts for Barry - 6 Oct 2026.pdf
12475 2026-10-06 16:30 FOLLOW-UP extracts for Barry - 6 Oct 2026.sql
12475 2026-10-06 16:37 FOLLOW-UP extracts for Barry - 6 Oct 2026.txt
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

CLAUDE (2026-10-06 14:37:42 UTC)

You've hit your session limit · resets 7:10pm (Africa/Johannesburg)

USER (2026-10-06 14:42:35 UTC)

do i have the \$250 credit now

CLAUDE (2026-10-06 14:42:41 UTC)

You've hit your session limit · resets 7:10pm (Africa/Johannesburg)

USER (2026-10-06 20:57:39 UTC)

continue

Tool call: Bash

description: Read the recipient addresses from the earlier emails to MAIIC

--- command ---

```
cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/3. Project Execution/Correspondence - Information Requests" && for f in
"2026-09-11 MAIIC EIR - Consolidated Information Request UPDATED (to MAIIC).eml" "RE_UPDATED_MAIIC IFRS 9 consolidated
information request, revised 11 September 2026 after your loan books and amortisation schedules. Replaces the version sent
earlier today.eml"; do echo "== $f"; grep -a -i -m 12 "^From:|^To:|^Cc:|^X-Unsent|^Content-Type:" "$f" | cut -c1-400; done
```

Result of Bash

```
== 2026-09-11 MAIIC EIR - Consolidated Information Request UPDATED (to MAIIC).eml
From: Edward Mazibuko <edward@dupleixinstitute.com>
To: Tamanda Sitimawina <tsitimawina@maiic.mw>,
Cc: Thomson Kumwenda <tkumwenda@maiic.mw>,
X-Unsent: 1
Content-Type: multipart/mixed; boundary="=====0441268703832996125=="
Content-Type: multipart/alternative;
Content-Type: text/plain; charset="utf-8"
Content-Type: text/html; charset="utf-8"
Content-Type: application/vnd.openxmlformats-officedocument.wordprocessingml.document
== RE_UPDATED_MAIIC IFRS 9 consolidated information request, revised 11 September 2026 after your loan books and amortisation
schedules. Replaces the version sent earlier today.eml
From: Tamanda Sitimawina <tsitimawina@maiic.mw>
To: edward <edward@dupleixinstitute.com>, Vanessa Magaleta
CC: Thomson Kumwenda <tkumwenda@maiic.mw>, wadzanai
Content-Type: multipart/mixed;
Content-Type: multipart/related;
Content-Type: multipart/alternative;
Content-Type: text/plain; charset="us-ascii"
From: edward@dupleixinstitute.com <edward@dupleixinstitute.com>
To: Tamanda Sitimawina <tsitimawina@maiic.mw>; Vanessa Magaleta <vmagaleta@maiic.mw>; Barry Makumba <bmakumba@maiic.mw>
Cc: Thomson Kumwenda <tkumwenda@maiic.mw>; wadzanai <wadzanai@dupleixinstitute.com>; kundai <kundai@dupleixinstitute.com>;
'themba' <themba@dupleixinstitute.com>
Content-Type: text/html; charset="us-ascii"
Content-Type: image/jpeg; name="image001.jpg"
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\make_email.py
--- content ---
"""Build the email to MAIIC as an Outlook-openable .eml (X-Unsent) with the three attachments, and refresh the .md draft."""
import os, re, html
from email.message import EmailMessage
from email.utils import formatdate

Q = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC"
C = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\3. Project Execution\Correspondence - Information Requests"
BASE = "FOLLOW-UP extracts for Barry - 6 Oct 2026"
SUBJECT = "MAIIC EIR: thank you for the dictionary results, and 18 extracts to replace Extracts A, B and C, please by 10:00
tomorrow"

BODY = """Dear Barry, Dr Thom, Tamanda and Dibwa,

Thank you, Barry, for turning the data dictionary queries round so quickly. All twenty-four results arrived this afternoon,
every one of them ran, and between them they have unlocked most of what has been open since September. This is the most
progress we have made in one day on the data side of the project.

## What the results have settled

- **Interest Policy is found.** It is held on each scheme, and on each account. MAIIC Agricultural and Industrial loans are
"Link with PLR", so they follow the Reserve Bank rate; FinES, Term and Investment loans are on a slab rate, which is why they
never moved. That matches the loan books exactly and ends the guesswork on repricing.
- **The Fixed/Variable label in Extract A was the wrong way round.** The raw flag is "floating" on every MAIIC scheme and
"fixed" on every FinES scheme. The script's translation had the two swapped. No harm done, now that we know.
- **The spread over the prime rate exists** as a field on every loan, and the per-account rate set-up table carries the PLR
rate, the spread and the date each rate took effect. That is the rate history we were going to ask the vendor to export from
the audit trail.
- **The ledger is signed and self-describing.** Debits are negative, credits positive, and each posting's narration names what
it is: "arrangement fee", "legal fees", "IntDR Fm 08-AUG-25 To 31-AUG-25 @ 31%". The debit/credit question and most of the
"what does this posting mean" questions answer themselves.
- **The fee table exists:** the disbursement charges table holds each charge by name, amount and income GL. The fee template we
sent may now only be needed for anything handled outside the system.
- **The loan book history is stored** for every as-on date, which should fill the months from November 2024 to November 2025
that we do not have.
- **The interest calculation log exists**, with the rate, the period, the days and the amount behind every interest posting.
That is the "how" behind Extract C.
- **Restructures and write-offs:** both tables are empty, so the blank restructure date in Extract A is real, and the
restructure history lives in Tamanda's register, as we understood.
- **The instalment chart is stored**, with the amount recovered against each instalment, and the chart starts with a row at the
loan's start date, which explains why the instalment count was always one more than the tenor.
- **Transaction code 120** on six accounts at 31 December 2025 adds up to the cent to the difference we had found between
Extracts B and C.

## What we now ask for: eighteen extracts that replace Extracts A, B and C, by 10:00 tomorrow

Rather than re-run the three extract procedures, we would like to take the data straight from the tables the dictionary has
shown us. The attached file holds eighteen read-only queries, numbered in the order to run them and grouped in three
priorities. Each one is a plain SELECT; nothing is created, changed or deleted.

**Please could we have all eighteen result files by 10:00 tomorrow, Tuesday 7 October.** The queries are small (the ledger is
about 3,700 rows) and should run in minutes, and having them in the morning lets us load and check them the same day. If time
runs short, send Priority 1 and 2 by 10:00 and Priority 3 by close of business.

- **Priority 1 (runs 1 to 5)** replaces the three extracts: the full signed ledger with narrations, the account master and loan
master with every column, the rate set-up per account, and the interest calculation log. These resolve the reconciliation
issues we raised: direction of postings, interest charged against cash received, the rate history, and the fees.
- **Priority 2 (runs 6 to 10):** the PLR history, the disbursement charges, the stored loan book history, the balance history
and the current instalment chart.
- **Priority 3 (runs 11 to 18):** the remaining supporting tables.

The same file is attached three ways, so that whichever is easiest to open works: as .sql to run, as .txt to copy from, and as
.pdf to read.

Two practical points, both explained in the file:

1. **Run the two session settings first (RUN 0).** They make dates come out as 2026-10-06 rather than 7/15/2025, and numbers
with a dot for the decimal point. They change nothing in the database, affect no other user, and last only for that login; when
Barry disconnects they are gone and there is nothing to put back.
2. **Please save each result as CSV under the file name shown** (P1_01 to P3_18, so the prefix tells us the priority and the
run), and do not open the files in Excel before sending. Excel is what swapped days and months in the earlier extracts and it
drops the leading zeros from account numbers.

If any query fails, the error will name the column, and a screenshot of it is all we need to fix it.

## Three things only you can tell us

- What transaction codes **306**, **300** and **343** are. We can see their direction but not their meaning.
- What account status **H** and **F** mean. The status history (run 13) may show the reason, but a one-line answer would be
quicker.
- Whether the "(Manual Int)" schemes are ever used. They all have zero accounts today.

## Still open from the September list

- The materiality threshold used in the 2024 and 2025 EIR assessments (Dr Thom).
- Written confirmation of which record is the contractual one: the signed offer letter or the system.
- The LOS application and process reference numbers, and the funded/non-funded flag, which we have not yet found in this
```

schema. They may sit with the origination system.

Thank you again. The results have moved us from asking what E-Banker holds to being able to say exactly which table and column we need, and the eighteen queries are the proof of that.

Kind regards,

Edward Mazibuko CA(SA)
Dupleix Institute
""

```
# ---- refresh the markdown draft
md = f""""# Draft email: dictionary results received, and the 18 follow-up extracts
**To:** Barry Makumba; Dr Thomson Kumwenda; Tamanda Sitimawina; Dibwa
**Cc:** Vanessa Magaleta; Kundai Muriwo; Wadzanai Rombe
**Subject:** {SUBJECT}
**Attachments:** {BASE}.sql, {BASE}.txt, {BASE}.pdf

---

{BODY}""""
open(os.path.join(C, "2026-10-06 Thank you for the dictionary results and the 18 follow-up extracts (draft).md"), "w",
encoding="utf-8").write(md)

# ---- html body
def inline(t):
    t = html.escape(t, quote=False)
    return re.sub(r"\\\"*(.+)\\\"*", r"<b>\1</b>", t)

out, buf, kind = [], [], None
def flush():
    global buf, kind
    if buf and kind == "ul": out.append("<ul>" + "".join(f"<li>{x}</li>" for x in buf) + "</ul>")
    if buf and kind == "ol": out.append("<ol>" + "".join(f"<li>{x}</li>" for x in buf) + "</ol>")
    buf, kind = [], None
for line in BODY.split("\n"):
    s = line.strip()
    if not s:
        flush(); continue
    if s.startswith("## "):
        flush(); out.append(f"<p><b>{inline(s[3:])}</b></p>"); continue
    m = re.match(r"^~ (.*)", s)
    if m:
        if kind != "ul": flush(); kind = "ul"
        buf.append(inline(m.group(1))); continue
    m = re.match(r"^\d+\.. (.*)", s)
    if m:
        if kind != "ol": flush(); kind = "ol"
        buf.append(inline(m.group(1))); continue
    flush(); out.append(f"<p>{inline(s)}</p>")
flush()
HTML = ('<html><body style="font-family:Calibri,Segoe UI,sans-serif;font-size:11pt;line-height:1.4">'
+ "\n".join(out) + "</body></html>")

# ---- the .eml
msg = EmailMessage()
msg["From"] = "Edward Mazibuko <edward@dupleixinstitute.com>"
msg["To"] = ("Barry Makumba <bmakumba@maic.mw>, Thomson Kumwenda <tkumwenda@maic.mw>, "
"Tamanda Sitimawina <tsitimawina@maic.mw>")
msg["Cc"] = ("Vanessa Magaleta <vmagaleta@maic.mw>, Kundai Muriwo <kundai@dupleixinstitute.com>, "
"Wadzanai Rombe <wadzanai@dupleixinstitute.com>")
msg["Subject"] = SUBJECT
msg["Date"] = formatdate(localtime=True)
msg["X-Unsent"] = "1"
msg.set_content(BODY)
msg.add_alternative(HTML, subtype="html")
for ext, mt in ((".sql", ("application", "sql")), (".txt", ("text", "plain")), (".pdf", ("application", "pdf"))):
    p = os.path.join(Q, BASE + ext)
    with open(p, "rb") as fh:
        msg.add_attachment(fh.read(), maintype=mt[0], subtype=mt[1], filename=BASE + ext)
eml = os.path.join(Q, "EMAIL - 18 follow-up extracts by 10am 7 Oct 2026 (open in Outlook and send).eml")
with open(eml, "wb") as fh:
    fh.write(bytes(msg))
print("written", eml, os.path.getsize(eml))
```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\make_email.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Build the ready-to-send email file with the three attachments

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python make_email.py && grep -a -c "^Content-Disposition: attachment" "c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/EMAIL - 18 follow-up extracts by 10am 7 Oct 2026 (open in Outlook and send).eml"
{
  "timeout": 120000
}
```

Result of Bash

written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\EMAIL - 18 follow-up extracts by 10am 7 Oct 2026 (open in Outlook and send).eml 146227
3
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

CLAUDE (2026-10-06 20:59:25 UTC)

Written for: Barry, Dr Thom, Tamanda and Dibwa at MAIIC, sent from Edward's account.

The email is built as an Outlook message file with the three attachments already on it, saved in the raw queries folder. Double-click it and Outlook opens it as an unsent message; check the recipients, then send.

What's on it:

- **To:** Barry, Dr Thom, Tamanda (addresses taken from the 11 September thread). **Cc:** Vanessa Magaleta, Kundai, you. **Dibwa is not on it** because I have no address for them; add it in Outlook before sending.
- **Subject** carries the deadline: "...18 extracts to replace Extracts A, B and C, please by 10:00 tomorrow".
- **The deadline paragraph** asks for all eighteen files by 10:00 on Tuesday 7 October, and says that if time runs short, Priority 1 and 2 by 10:00 and Priority 3 by close of business.
- **Attachments:** the queries file as .sql (to run), .txt (to copy from) and .pdf (to read), with a line in the body explaining why there are three.
- The body is otherwise the draft you approved, as HTML with a plain-text copy.

Two cautions:

- **New Outlook and .eml files:** classic Outlook opens an unsent .eml as an editable draft. New Outlook sometimes opens it read-only; if that happens, use "Forward" or copy the text from the saved draft and attach the three files by hand. I can't test this from here.
- **Sender:** the file names Edward as sender. Whichever account opens it will send from that account.

On your other question: I can't see your account's billing or credits from inside this session. That shows under your plan or billing settings on claude.ai.

Files are in 2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC:

- EMAIL - 18 follow-up extracts by 10am 7 Oct 2026 (open in Outlook and send).eml
- FOLLOW-UP extracts for Barry - 6 Oct 2026.pdf
- FOLLOW-UP extracts for Barry - 6 Oct 2026.txt
- FOLLOW-UP extracts for Barry - 6 Oct 2026.sql

The text draft, updated to match, is in 3. Project Execution\Correspondence - Information Requests.

USER (2026-10-06 21:02:04 UTC)

open the pdf

Tool call: PowerShell

description: Open the follow-up extracts PDF in the default viewer

```
--- command ---
Invoke-Item -LiteralPath "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\FOLLOW-UP extracts for Barry - 6 Oct 2026.pdf"
```

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-06 21:02:25 UTC)

The PDF should now be open in your default viewer. It's four pages: the file list and session-settings note on page 1, then the eighteen queries in run order.

USER (2026-10-06 21:10:02 UTC)

from the results of the queries, can we successfully build the EIR profiles for both pre-migration and post migration clients?

CLAUDE (2026-10-06 21:10:49 UTC)

Not from what we hold today: the results so far are the dictionary and one sample loan. But they show that the data exists for the post-migration book, and they show exactly what E-Banker cannot give us for the pre-migration book. The split matters because the two groups need different sources.

An EIR profile needs, per loan: the cash paid out (dates and amounts, net of fees deducted), the fees themselves, the contractual rate and every change to it, the contractual schedule, the actual repayments, and the carrying amount at each month-end.

Post-migration loans (originated in E-Banker, from mid-2024)

Yes, once Priority 1 and 2 arrive, with three checks still to pass.

```
| Component | Where it comes from |
|---|---|
| Cash out, by drawdown | Ledger postings of type 301 (P1_01), each dated, and the fee deductions named in the narration |
| Fees | Disbursement charges table (P2_07), cross-checked to the fee narrations in the ledger |
| Rate at origination and every change | Per-account rate set-up with effective dates (P1_04), the PLR history (P2_06), and the
rate quoted in every interest narration |
| Contractual schedule | The stored instalment chart (P2_10), with the amount recovered per instalment |
| Actual repayments | Type 305 postings in the ledger |
| Month-end carrying amount | Balance history (P2_09) and the stored loan book (P2_08) |
| Interest actually charged, for the reconciliation | Interest calculation log (P1_05) |
```

The checks: that the fee table covers every fee the narrations show; what codes 306, 300 and 343 are; and whether the per-account rate table holds the full history or only the current rate.

Pre-migration loans (the take-on book, about 100 loans from 2020)

Only partly from E-Banker. The system was loaded at take-on with the original dates and sanctioned amounts, but not with the original cash flows. What is missing from E-Banker has to come from elsewhere:

```
| Component | E-Banker | Has to come from |
|---|---|---|
| Original drawdown dates and amounts | No. The ledger starts in July 2024 with the take-on balances | The take-on amortisation
schedules (103 blocks) and the offer letters |
| Fees at origination | No. The charges table starts with E-Banker | The 2024 and 2025 EIR assessments (67 facilities with
fees) and Finance's records |
| Rate history before October 2024 | No | The dated rate steps in the take-on schedules and the PLR history (P2_06) |
| Repayments before October 2024 | No | The take-on schedules carry actual repayments |
| Everything from October 2024 onward | Yes, as for post-migration loans | |
```

The one thing that blocks this today is the link: only 4 of the 103 take-on blocks carry a loan name and none carries an account number. We need a mapping from Finance, block by block, to E-Banker account numbers. Without it the pre-migration half of each profile cannot be joined to the post-migration half.

What to ask for

- From Tamanda: the account number for each of the 103 take-on blocks, and the fee paid at origination for each.
- From Barry, in addition to the eighteen: whether the per-account rate table holds history or only the current rate (P1_04 will show it), and what the July and August 2024 postings of types 300 and 301 represent, since those look like the take-on entries.

So the honest answer is: post-migration, yes, pending the extracts and the checks; pre-migration, yes for the period from October 2024, and for the years before only once Finance gives us the mapping and the origination fees.

USER (2026-10-06 21:13:40 UTC)

open the email to barry

Tool call: PowerShell

description: Open the email file in Outlook

--- command ---

```
Invoke-Item -LiteralPath "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\EMAIL - 18 follow-up extracts by 10am 7 Oct 2026 (open in Outlook and send).eml"
```

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-06 21:13:52 UTC)

The email file has been sent to Outlook (whichever program is set to open .eml files). Before sending, add Dibwa's address, since I had none for them, and check that the three attachments show at the bottom.

If Outlook opened it read-only rather than as a draft, let me know and I'll give you the text in a form you can paste into a new message instead.

USER (2026-10-07 07:55:29 UTC)

continue

CLAUDE (2026-10-07 07:55:44 UTC)

You've hit your session limit · resets 10:50am (Africa/Johannesburg)

USER (2026-10-07 08:51:31 UTC)

"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Follow-Up Scripts Resutls" has the results to the follow up scripts from Barry. Can you do a thorough audit of these scripts and check the date formats so that we know the way format. We want a comprehensive report in pdf also showing the gaps covered, if we are now ready to build the system, if the system can now successfully rebuild the repayment profiles or rely on the repayment profiles in the system for EIR and Present value of repayments discounting, the suggested changes to the code, if there are still errors on the dates, what may be causing that and what is the fix, try and reconcile the GLs to the extracts etc

Tool call: Bash

description: List the follow-up results Barry sent

--- command ---

```
cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Follow-Up Scripts Resutls" 2>/dev/null || cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/" && ls -la --time-style=long-iso . | awk '{print $5, $6, $7, $8, $9, $10, $11, $12}'; find . -type f | head -40; echo "--- line counts"; for f in *.csv *.txt *.xlsx 2>/dev/null; do [ -f "$f" ] && printf "%8s %s\n" "$(wc -l < "$f")" "$f"; done 2>/dev/null | head -40
```

Result of Bash [error]

Exit code 2

```
/usr/bin/bash: eval: line 1: syntax error near unexpected token `2'
```

```
/usr/bin/bash: eval: line 1: `cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Follow-Up Scripts Resutls" 2>/dev/null || cd "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/" && ls -la --time-style=long-iso . | awk '{print $5, $6, $7, $8, $9, $10, $11, $12}'; find . -type f | head -40; echo "--- line counts"; for f in *.csv *.txt *.xlsx 2>/dev/null; do [ -f "$f" ] && printf "%8s %s\n" "$(wc -l < "$f")" "$f"; done 2>/dev/null | head -40'
```

Tool call: PowerShell

description: List the follow-up result files with sizes and line counts

--- command ---

```
$d = "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Follow-Up Scripts Resutls"; Get-ChildItem -LiteralPath $d -Recurse -File | ForEach-Object { [pscustomobject]@{ KB = [math]::Round($_.Length/1KB,1); Lines = (Get-Content -LiteralPath $_.FullName | Measure-Object -Line).Lines; Modified = $_.LastWriteTime.ToString("yyyy-MM-dd HH:mm"); Name = $_.FullName.Replace($d + '\', '') } } | Format-Table -AutoSize { "timeout": 180000 }
```

Result of PowerShell

	KB	Lines	Modified		Name
	--	-----	-----		----
1044.5	7930	2026-10-07	10:33		Dupleix_02.zip
2169.4	3641	2026-10-07	10:35		P1_01_ledger_all.csv
163.5	185	2026-10-07	10:35		P1_02_account_master_all.csv
80.9	185	2026-10-07	10:35		P1_03_loan_master_all.csv
358.3	1292	2026-10-07	10:35		P1_04_rate_setup_per_account.csv
26.2	83	2026-10-07	10:35		P1_05_interest_posted_by_account.csv
	8	50	2026-10-07	10:35	P2_06_plr_master.csv
503.1	3652	2026-10-07	10:35		P2_07_disbursement_charges.csv
5490.7	12415	2026-10-07	10:35		P2_08_loan_book_history.csv
782.1	2550	2026-10-07	10:35		P2_09_balance_history.csv
710.1	2743	2026-10-07	10:35		P2_10_instalment_chart_current.csv
350.6	1959	2026-10-07	10:35		P3_11_instalment_plans.csv
42.2	252	2026-10-07	10:35		P3_12_disbursement_schedules.csv
45.6	284	2026-10-07	10:35		P3_13_status_history.csv
741.5	7490	2026-10-07	10:35		P3_14_daily_interest_accrual.csv
33.8	101	2026-10-07	10:35		P3_15_slab_master.csv
	0.8	3	2026-10-07	10:35	P3_16_forgiven_interest.csv
	0.9	2	2026-10-07	10:35	P3_17_auto_charges.csv
	0.2	4	2026-10-07	10:35	P3_18_date_time_check.csv

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\fu_profile.py
--- content ---
"""Profile every follow-up result file: shape, columns, date formats, key counts."""
import re, os, sys, zipfile, json
import pandas as pd
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 50); pd.set_option("display.max_colwidth", 40)
D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Follow-Up
Scripts Resultsl"
PAT = {
    "iso_datetime": re.compile(r"^\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}$"),
    "iso_date": re.compile(r"^\d{4}-\d{2}-\d{2}$"),
    "m/d/yyyy": re.compile(r"^\d{1,2}/\d{1,2}/\d{4}$"),
    "dd-mon-yy": re.compile(r"^\d{2}-[A-Za-z]{3}-\d{2}$"),
    "m/d/yyyy h:mm": re.compile(r"^\d{1,2}/\d{1,2}/\d{4} \d{1,2}:\d{2}$"),
}
with zipfile.ZipFile(os.path.join(D, "Dupleix_02.zip")) as z:
    print("ZIP contents:", [(i.filename, i.file_size) for i in z.infolist()][1:30])
report = {}
for f in sorted(os.listdir(D)):
    if not f.endswith(".csv"): continue
    df = pd.read_csv(os.path.join(D, f), dtype=str, keep_default_na=False)
    if df.columns[0].strip() == "": df = df.iloc[:, 1:]
    info = {"rows": len(df), "cols": len(df.columns)}
    datecols = {}
    for c in df.columns:
        s = df[c][df[c].str.strip() != ""]
        if len(s) == 0: continue
        kinds = {}
        for k, p in PAT.items():
            n = int(s.str.match(p).sum())
            if n: kinds[k] = n
        if kinds and sum(kinds.values()) >= 0.5 * len(s):
            datecols[c] = dict(kinds, nonblank=len(s), sample=s.iloc[0])
            if "iso_datetime" in kinds:
                with_time = int(~s.str.endswith(" 00:00:00").sum()); datecols[c]["with_time_of_day"] = with_time
    info["date_columns"] = datecols
    report[f] = info
    print(f"=== {f}: {len(df)} rows x {len(df.columns)} cols")
    print("    columns:", ", ".join(df.columns[:45]) + (" ..." if len(df.columns) > 45 else ""))
    for c, v in datecols.items():
        print(f"    DATE {c}: {v}")
    if "NEW_AC_NUMBER" in df.columns:
        print("    accounts:", df.NEW_AC_NUMBER.nunique(), "| sample:", df.NEW_AC_NUMBER.iloc[0], "| all 15 chars:",
bool((df.NEW_AC_NUMBER.str.len() == 15).all()))
json.dump(report, open("fu_profile.json", "w"), indent=1, default=str)
```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\fu_profile.py (file state is current in your context – no need to Read it back)

Tool call: Bash

```
description: Profile every result file: rows, columns, date formats, accounts
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python fu_profile.py 2>&1 | cut -c1-330
{
    "timeout": 300000
}
```

Result of Bash

```
ZIP contents: [('Dupleix_02/', 0), ('Dupleix_02/P1_01_ledger_all.csv', 2221448), ('Dupleix_02/P1_02_account_master_all.csv', 167405), ('Dupleix_02/P1_03_loan_master_all.csv', 82822), ('Dupleix_02/P1_04_rate_setup_per_account.csv', 366867), ('Dupleix_02/P1_05_interest_posted_by_account.csv', 26788), ('Dupleix_02/P2_06_plr_master', 26788)]

=== P1_01_ledger_all.csv: 3640 rows x 83 cols
columns: CUMVOUCH_DET_ID, CB_BRANCH_MST_ID, T_BRANCH_MST_ID, O_BRANCH_MST_ID, CB_GLCODE, AC_GLCODE, NEW_AC_NUMBER, SUB_ACCOUNT_NO, CB_GLTYPE, CB_ALNX_TYPE, TRANSACTION_DATE, CB_EFFECTIVE_DATE, TRANSACTION_TIME, TRANSAMT, INTEREST_AMOUNT, VSCRNO, BATCH_TYPE, BATCH_NUMBER, OPERATION_ID, OPERATION_DATE, DBCR, CACLGTRF, CACLGTRF_
DATE TRANSACTION_DATE: {'m/d/yyyy': 3640, 'nonblank': 3640, 'sample': '7/31/2024'}
DATE CB_EFFECTIVE_DATE: {'m/d/yyyy': 3640, 'nonblank': 3640, 'sample': '7/31/2024'}
DATE OPERATION_DATE: {'m/d/yyyy': 3533, 'nonblank': 3533, 'sample': '7/31/2024'}
DATE CHEQUE_DATE: {'m/d/yyyy': 233, 'nonblank': 233, 'sample': '11/6/2024'}
accounts: 132 | sample: 000104420000003 | all 15 chars: True

=== P1_02_account_master_all.csv: 184 rows x 141 cols
columns: ACCOUNT_MST_ID, BRANCH_MST_ID, GLCODE, OLD_AC_NUMBER, NEW_AC_NUMBER, SUB_AC_NUMBER, AGENT_NUMBER, SCHEME_MST_ID, CUSTOMER_ID, ACCOUNT_NAME, ACCOUNT_NAME_OL, TRANSACTION_DATE, ACCOUNT_OPEN_DATE, STATUS_CODE, ACCOUNT_STATUS_DATE, FREEZE_DR_AMOUNT, STATUS_CHANGE_REASON, CHEQUE_BOOK_FLAG, TOD_APPLI_FLAG, INTEREST_CALC_FL
DATE TRANSACTION_DATE: {'m/d/yyyy': 184, 'nonblank': 184, 'sample': '3/28/2024'}
DATE ACCOUNT_OPEN_DATE: {'m/d/yyyy': 184, 'nonblank': 184, 'sample': '7/28/2020'}
DATE ACCOUNT_STATUS_DATE: {'m/d/yyyy': 184, 'nonblank': 184, 'sample': '7/29/2020'}
DATE INTEREST_STOP_DATE: {'m/d/yyyy': 40, 'nonblank': 40, 'sample': '12/11/2025'}
DATE LAST_INTEREST_DATE: {'m/d/yyyy': 168, 'nonblank': 168, 'sample': '8/29/2024'}
DATE STATUS_DUE_DATE: {'m/d/yyyy': 139, 'nonblank': 139, 'sample': '3/28/2024'}
DATE OPERATION_DATE: {'m/d/yyyy': 184, 'nonblank': 184, 'sample': '3/28/2024'}
DATE LAST_TR_DATE: {'m/d/yyyy': 132, 'nonblank': 132, 'sample': '8/31/2024'}
DATE PREV_INTPOST_DATE: {'m/d/yyyy': 118, 'nonblank': 118, 'sample': '8/29/2024'}
DATE LAST_CR_DATE: {'m/d/yyyy': 79, 'nonblank': 79, 'sample': '8/31/2024'}
DATE EMAIL_STATE_UPTODT: {'m/d/yyyy': 1, 'nonblank': 1, 'sample': '4/1/2024'}
accounts: 184 | sample: 000104420000003 | all 15 chars: True

=== P1_03_loan_master_all.csv: 184 rows x 80 cols
columns: ACLOAN_MST_ID, NEW_AC_NUMBER, SUB_ACCOUNT_NO, RESOLUTION_NO, RESOLUTION_DATE, SANCTION_AMOUNT, SANCTION_DATE, PERIOD_DAYS, PERIOD_MONTHS, PERIOD_YEARS, EXPIRY_DATE, FUNDED_FLAG, AC_RENEW_TRANDATE, RENEW_PERIOD_DAYS, RENEW_PERIOD_MONTHS, RENEW_PERIOD_YEARS, RENEW_EXP_DATE, DOCU_START_DATE, DOCU_PERIOD_DAYS, DOCU_PERIO
DATE RESOLUTION_DATE: {'m/d/yyyy': 104, 'nonblank': 104, 'sample': '7/28/2020'}
DATE SANCTION_DATE: {'m/d/yyyy': 183, 'nonblank': 183, 'sample': '7/28/2020'}
DATE EXPIRY_DATE: {'m/d/yyyy': 183, 'nonblank': 183, 'sample': '7/27/2022'}
DATE OPERATION_DATE: {'m/d/yyyy': 183, 'nonblank': 183, 'sample': '7/28/2020'}
accounts: 184 | sample: 000104420000003 | all 15 chars: True

=== P1_04_rate_setup_per_account.csv: 1291 rows x 38 cols
columns: INTEREST_RATE_DET_ID, NEW_AC_NUMBER, SUB_ACCOUNT_NO, TRAN_DATE, APPLICABLE_FROM_DATE, MIN_AMOUNT, MAX_AMOUNT, INTEREST_POLICY, INTEREST_PRODUCT_FREQUENCY, INTEREST_PAYMENT_FREQUENCY, PLR_RATE, VARIATION_RATE, INTEREST_RATE, PREMATURE_RATE, EXPIRY_PENAL_RATE, OVERDRAWN_PENAL_RATE, UNUTILIZE_PENAL_RATE, STOCK_PENAL_RAT
DATE TRAN_DATE: {'m/d/yyyy': 1291, 'nonblank': 1291, 'sample': '3/28/2024'}
DATE APPLICABLE_FROM_DATE: {'m/d/yyyy': 1291, 'nonblank': 1291, 'sample': '7/28/2020'}
DATE OPERATION_DATE: {'m/d/yyyy': 1291, 'nonblank': 1291, 'sample': '3/28/2024'}
accounts: 183 | sample: 000104420000003 | all 15 chars: True

=== P1_05_interest_posted_by_account.csv: 82 rows x 43 cols
columns: INTEREST_MST_ID, OPERATION_BRANCH, GLCODE, NEW_AC_NUMBER, SUB_AC_NUMBER, TRANSACTION_DATE, INTEREST_FROM_DATE, INTEREST_UPTO_DATE, INTEREST_RATE, PRINCIPAL_PRODUCT, NO_OF_DAYS, NO_OF_MONTHS, INTEREST_AMOUNT, OVERDUE_PRODUCT, OVDNO_OF_DAYS, OVD_INT_RATE, PENAL_RATE, OVERDUE_INTEREST, TOTAL_INTEREST, CALC_USER_ID, PART
DATE TRANSACTION_DATE: {'m/d/yyyy': 82, 'nonblank': 82, 'sample': '8/31/2026'}
DATE INTEREST_FROM_DATE: {'m/d/yyyy': 82, 'nonblank': 82, 'sample': '8/31/2026'}
DATE INTEREST_UPTO_DATE: {'m/d/yyyy': 82, 'nonblank': 82, 'sample': '8/31/2026'}
DATE PRODUCT_FROM_DATE: {'m/d/yyyy': 81, 'nonblank': 81, 'sample': '8/1/2026'}
DATE PRODUCT_UPTO_DATE: {'m/d/yyyy': 82, 'nonblank': 82, 'sample': '8/31/2026'}
accounts: 82 | sample: 000104420000010 | all 15 chars: True

=== P2_06_plr_master.csv: 49 rows x 17 cols
columns: PLR_MST_ID, TRAN_DATE, APPLI_FROM_DATE, PLR_RATE, MIN_VARIATION_RATE, MAX_VARIATION_RATE, ACTIVE_FLAG, PARTICULAR, DELETE_FLAG, OPERATION_ID, OPERATION_DATE, ENTER_USER_ID, ENTER_DESC, DELETE_USER_ID, DELETE_DESC, PASS_USER_ID, PASS_DESC
DATE TRAN_DATE: {'m/d/yyyy': 49, 'nonblank': 49, 'sample': '9/12/2025'}
DATE APPLI_FROM_DATE: {'m/d/yyyy': 49, 'nonblank': 49, 'sample': '3/4/2020'}
DATE OPERATION_DATE: {'m/d/yyyy': 26, 'nonblank': 26, 'sample': '9/12/2025'}

=== P2_07_disbursement_charges.csv: 3651 rows x 17 cols
columns: LO_DISB_CHARG_DET_ID, DISB_CHARGES_NAME, NEW_AC_NUMBER, SUB_AC_NUMBER, ACTUAL_CHARGES_AMT, APPLI_CHARGES_AMT, STAX_PER, STAX_AMOUNT, SBC_PER, SBC_AMOUNT, POST_FLAG, ENTER_USER_ID, ENTER_DESC, POST_USERID, BATCH_NUMBER, PARTICULAR, INCOME_GL
accounts: 1218 | sample: 000104420000064 | all 15 chars: True

=== P2_08_loan_book_history.csv: 12414 rows x 46 cols
columns: LOAN_BOOK_DET_ID_A, TRANSACTION_DATE, ASONDATE, BRANCH_MST_ID, CUSTOMER_ID, GLCODE, NEW_AC_NUMBER, SUB_AC_NUMBER, LOAN_TYPE, VALUE_DATE, MATURITY_DATE, TENOR_YRS, MORAT_MONTHS, INTEREST_RATE, APPROVED, DISBURSD, NOT_YET_DISBURS, PRINCIPAL, COLTRAL_TYPE, CHARGE_AMOUNT, COLLATERAL_VALUE, INTEREST_TO_DATE, REPAYMENT, CA
DATE TRANSACTION_DATE: {'m/d/yyyy': 12414, 'nonblank': 12414, 'sample': '9/23/2026'}
DATE ASONDATE: {'m/d/yyyy': 12414, 'nonblank': 12414, 'sample': '12/31/2024'}
DATE VALUE_DATE: {'m/d/yyyy': 11179, 'nonblank': 11179, 'sample': '7/23/2020'}
DATE MATURITY_DATE: {'m/d/yyyy': 11179, 'nonblank': 11179, 'sample': '7/31/2026'}
DATE OVERDUE_PRINCI_DATE: {'m/d/yyyy': 10652, 'nonblank': 10652, 'sample': '8/1/2024'}
accounts: 144 | sample: 000104420000003 | all 15 chars: True

=== P2_09_balance_history.csv: 2549 rows x 34 cols
columns: ACCOUNT_BAL_MST_ID, GLCODE, NEW_AC_NUMBER, SUB_AC_NUMBER, TRANSACTION_DATE, PRINCIPAL_BALANCE, RECEIPT_AMOUNT, PAYMENT_AMOUNT, OD_INT_REC_PAY, INT_REC, SANCTION_AMOUNT, REPORT_BALANCE, ALNX_TYPE, OD_INT_RECEIPT, OD_INT_PAYMENT, INT_RECEIPT, INT_PAYMENT, AC_PRINCIPAL_BALANCE, AC_RECEIPT_AMOUNT, AC_PAYMENT_AMOUNT, AC_O
DATE TRANSACTION_DATE: {'m/d/yyyy': 2549, 'nonblank': 2549, 'sample': '7/31/2024'}
accounts: 131 | sample: 000104420000003 | all 15 chars: True

=== P2_10_instalment_chart_current.csv: 2742 rows x 38 cols
columns: INST_CHART_DET_ID, NEW_AC_NUMBER, SUB_ACCOUNT_NO, TRANSACTION_DATE, INST_DET_ID, INST_SR_NO, INSTALLMENT_DATE, INSTALLMENT_AMT, INST_RECOVERED, LAST_OVD_CALC_DATE, REMARK, ACTIVE_FLAG, STATUS_CHANGE_DATE, STATUS_REASON, DELETE_FLAG, OPERATION_ID, OPERATION_DATE, ENTER_USER_ID, ENTER_DESC, DELETE_USER_ID, DELETE_DESC,
DATE TRANSACTION_DATE: {'m/d/yyyy': 2742, 'nonblank': 2742, 'sample': '3/28/2024'}
DATE INSTALLMENT_DATE: {'m/d/yyyy': 2742, 'nonblank': 2742, 'sample': '7/28/2020'}
DATE OPERATION_DATE: {'m/d/yyyy': 1588, 'nonblank': 1588, 'sample': '11/1/2025'}
```

```

accounts: 75 | sample: 000104420000003 | all 15 chars: True
=== P3_11_installment_plans.csv: 1958 rows x 27 cols
columns: INST_DET_ID, NEW_AC_NUMBER, SUB_ACCOUNT_NO, TRANSACTION_DATE, INSTALLMENT_TYPE, INSTALLMENT_PAY_TYPE,
INSTALLMENT_PAY_MODE, INSTALLMENT_START_DATE, INSTALLMENT_AMT, INSTALLMENT_END_DATE, INTT_INST_START_DATE, INTT_INST_AMT,
REMARK, DELETE_FLAG, OPERATION_ID, OPERATION_DATE, ENTER_USER_ID, ENTER_DESC, DELETE_USER_ID,
DATE TRANSACTION_DATE: {'m/d/yyyy': 1958, 'nonblank': 1958, 'sample': '3/28/2024'}
DATE INSTALLMENT_START_DATE: {'m/d/yyyy': 1958, 'nonblank': 1958, 'sample': '2/1/2021'}
DATE INSTALLMENT_END_DATE: {'m/d/yyyy': 1051, 'nonblank': 1051, 'sample': '1/31/2021'}
DATE OPERATION_DATE: {'m/d/yyyy': 1454, 'nonblank': 1454, 'sample': '3/28/2024'}
DATE DEMAND_START_DATE: {'m/d/yyyy': 149, 'nonblank': 149, 'sample': '10/1/2025'}
accounts: 160 | sample: 000104420000003 | all 15 chars: True
=== P3_12_disbursement_schedules.csv: 251 rows x 20 cols
columns: LOAN_DISB_DET_ID, NEW_AC_NUMBER, SUB_ACCOUNT_NO, DISB_SCHEDULE_DATE, DISB_SCHEDULE_AMOUNT, STATUS_FLAG, REMARKS,
DELETE_FLAG, OPERATION_ID, OPERATION_DATE, ENTER_USER_ID, ENTER_DESC, DELETE_USER_ID, DELETE_DESC, PASS_USER_ID, PASS_DESC,
OLD_PRIMARY_KEY, PAC_PURP_MST_ID, PAC_SCH_MST_ID, SANC_MST_ID
DATE DISB_SCHEDULE_DATE: {'m/d/yyyy': 251, 'nonblank': 251, 'sample': '7/28/2020'}
DATE OPERATION_DATE: {'m/d/yyyy': 152, 'nonblank': 152, 'sample': '9/5/2025'}
accounts: 161 | sample: 000104420000003 | all 15 chars: True
=== P3_13_status_history.csv: 283 rows x 22 cols
columns: AC_STATUS_MST_ID, OP_BRANCH_MST_ID, TRANSACTION_DATE, NEW_AC_NUMBER, SUB_AC_NUMBER, STATUS_CODE, STATUS_EFF_DATE,
FREEZE_DR_AMOUNT, STATUS_CHANGE_REASON, ACTIVE_FLAG, DELETE_FLAG, OPERATION_ID, OPERATION_DATE, ENTER_USER_ID, ENTER_DESC,
DELETE_USER_ID, DELETE_DESC, PASS_USER_ID, PASS_DESC, RENEW_FLAG, SUB_AC_FLAG, PR
DATE TRANSACTION_DATE: {'m/d/yyyy': 283, 'nonblank': 283, 'sample': '7/29/2020'}
DATE STATUS_EFF_DATE: {'m/d/yyyy': 283, 'nonblank': 283, 'sample': '7/28/2020'}
DATE OPERATION_DATE: {'m/d/yyyy': 127, 'nonblank': 127, 'sample': '12/11/2025'}
accounts: 184 | sample: 000104420000003 | all 15 chars: True
=== P3_14_daily_interest_accrual.csv: 7489 rows x 11 cols
columns: INT_ACCRU_DET_ID, OP_BRANCH_MST_ID, GLCODE, NEW_AC_NUMBER, SUB_AC_NUMBER, TRANSACTION_DATE, INTEREST_AMOUNT, DRCR,
DR_POSTING_GL, CR_POSTING_GL, BATCH_NUMBER
DATE TRANSACTION_DATE: {'m/d/yyyy': 7489, 'nonblank': 7489, 'sample': '11/1/2024'}
accounts: 14 | sample: 000104420000005 | all 15 chars: True
=== P3_15_slab_master.csv: 100 rows x 44 cols
columns: INTEREST_RATE_SLAB_ID, SCHEME_MST_ID, TRAN_DATE, APPLICABLE_FROM_DATE, APPLICABLE_UPTO_DATE, MIN_AMOUNT, MAX_AMOUNT,
MIN_PERIOD, MIN_PERIOD_UNIT, MAX_PERIOD, MAX_PERIOD_UNIT, PLR_MST_ID, INTEREST_RATE, PREMATURE_RATE, EXPIRY_PENAL_RATE,
OVERDRAWN_PENAL_RATE, UNUTILIZE_PENAL_RATE, STOCK_PENAL_RATE, NON_REVIEW_PENAL_RA
DATE TRAN_DATE: {'m/d/yyyy': 100, 'nonblank': 100, 'sample': '7/21/2013'}
DATE APPLICABLE_FROM_DATE: {'m/d/yyyy': 100, 'nonblank': 100, 'sample': '1/1/2010'}
DATE APPLICABLE_UPTO_DATE: {'m/d/yyyy': 68, 'nonblank': 68, 'sample': '1/1/2025'}
DATE OPERATION_DATE: {'m/d/yyyy': 98, 'nonblank': 98, 'sample': '7/21/2013'}
=== P3_16_forgiven_interest.csv: 2 rows x 23 cols
columns: FORG_INT_DET_ID, OP_BRMST_ID, TRANSACTION_DATE, NEW_AC_NUMBER, SUB_AC_NUMBER, INT_CHG_FLAG, ORIGINAL_AMT,
FORGIVE_AMOUNT, REF_BATCH_NO, ACTIVE_FLAG, STATUS_DATE, DELETE_FLAG, OPERATION_ID, OPERATION_DATE, ENTER_USER_ID, ENTER_DESC,
DELETE_USER_ID, DELETE_DESC, VERIFY_FLAG, VERIFY_USER_ID, VERIFY_DESC, PASS_USER_ID, P
DATE TRANSACTION_DATE: {'m/d/yyyy': 2, 'nonblank': 2, 'sample': '6/30/2026'}
DATE STATUS_DATE: {'m/d/yyyy': 2, 'nonblank': 2, 'sample': '6/30/2026'}
accounts: 2 | sample: 000104460000061 | all 15 chars: True
=== P3_17_auto_charges.csv: 1 rows x 42 cols
columns: CHARGES_MST_ID, CHARGES_MODULE_TYPE, GLCODE, SCHEME_ID, FROM_AMOUNT, TO_AMOUNT, CHARGES_FROMDATE, CHARGES_UPTODATE,
CHARGES_TYPE, CHARGES_SUB_TYPE, CHQBOOK_TYPE, CHARGES_AMOUNT, CHARGES_TO_EMPLOYEE, DEDUCT_SERVICE_TAX, SEPERATE_STAX_ENTRY,
DBCR, CACLGTRF, MIN_AMT_TOPOST, CHARGES_EXMP_GRADE, INCOME_GLCODE, INCOME_ACCN
DATE CHARGES_FROMDATE: {'m/d/yyyy': 1, 'nonblank': 1, 'sample': '4/1/2022'}
DATE CHARGES_UPTODATE: {'m/d/yyyy': 1, 'nonblank': 1, 'sample': '12/31/2050'}
=== P3_18_date_time_check.csv: 3 rows x 3 cols
columns: DATE_COLUMN, ROWS_CHECKED, ROWS_WITH_TIME_OF_DAY
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\fu_load.py
--- content ---
"""Shared loader for the follow-up result files (month-first dates, signed amounts)."""
import os, re
import pandas as pd

D = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Follow-Up
Scripts Resultls"
X = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Database extracts"
FILES = {
    "ledger": "P1_01_ledger_all.csv", "acm": "P1_02_account_master_all.csv", "alm": "P1_03_loan_master_all.csv",
    "rate": "P1_04_rate_setup_per_account.csv", "isum": "P1_05_interest_posted_by_account.csv", "plr": "P2_06_plr_master.csv",
    "chg": "P2_07_disbursement_charges.csv", "lb": "P2_08_loan_book_history.csv", "bal": "P2_09_balance_history.csv",
    "chart": "P2_10_instalment_chart_current.csv", "plan": "P3_11_instalment_plans.csv", "disb":
    "P3_12_disbursement_schedules.csv",
    "status": "P3_13_status_history.csv", "accr": "P3_14_daily_interest_accrual.csv", "slab": "P3_15_slab_master.csv",
    "forg": "P3_16_forgiven_interest.csv", "auto": "P3_17_auto_charges.csv", "dtchk": "P3_18_date_time_check.csv",
}
MDY = re.compile(r"^\d{1,2}/\d{1,2}/\d{4}$")

def load(key, parse_dates=True):
    df = pd.read_csv(os.path.join(D, FILES[key]), dtype=str, keep_default_na=False)
    if df.columns[0].strip() == "":
        df = df.iloc[:, 1:]
    df.columns = [c.strip() for c in df.columns]
    for c in df.columns:
        s = df[c]
        nb = s[s.str.strip() != ""]
        if len(nb) and nb.str.match(MDY).all():
            if parse_dates:
                df[c] = pd.to_datetime(s.where(s.str.strip() != "", None), format="%m/%d/%Y")
            elif len(nb) and nb.str.match(r"^\d{1,2}/\d{1,2}/\d{4}$").all() and c not in ("NEW_AC_NUMBER", "OLD_AC_NUMBER", "GLCODE",
"AC_GLCODE", "CB_GLCODE", "INCOME_GL", "DR_POSTING_GL", "CR_POSTING_GL", "INT_TRF_GLCODE"):
                df[c] = pd.to_numeric(s.where(s.str.strip() != "", None))
    return df

def load_all():
    return {k: load(k) for k in FILES}
```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\fu_load.py (file state is current in your context – no need to Read it back)

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\fu_a1.py
--- content ---
"""Audit 1: date formats, coverage of each table, key distributions."""
import re
import pandas as pd, numpy as np
from fu_load import load, FILES, D, X, MDY
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 40); pd.set_option("display.max_colwidth", 50)

print("=== 1. DATE FORMAT PROOF: first component > 12 (would mean day-first) vs second component > 12 (month-first)")
tot_first, tot_second, tot = 0, 0, 0
for k, f in FILES.items():
    raw = pd.read_csv(f"{D}\\{f}", dtype=str, keep_default_na=False)
    for c in raw.columns:
        s = raw[c][raw[c].str.strip() != ""]
        if len(s) and s.str.match(MDY).all():
            a = s.str.split("/").str[0].astype(int); b = s.str.split("/").str[1].astype(int)
            tot_first += int((a > 12).sum()); tot_second += int((b > 12).sum()); tot += len(s)
print(f"    date values checked {tot:,} | first part > 12: {tot_first} | second part > 12: {tot_second:,}")
print("    P3_18:", load("dtchk").to_dict("records"))

acm, alm, led, bal, lb, chart, plan, rate, chg, disb, status, accr, isum = (load(k) for k in
    ("acm", "alm", "ledger", "bal", "lb", "chart", "plan", "rate", "chg", "disb", "status", "accr", "isum"))
scope = set(acm.NEW_AC_NUMBER)
print("\n=== 2. SCOPE: accounts in account master", len(scope), "| by scheme:", acm.SCHEME_MST_ID.value_counts().to_dict())
print("    status:", acm.STATUS_CODE.value_counts().to_dict())
print("    INTEREST_POLICY per account:", acm.INTEREST_POLICY.value_counts().to_dict(), "| FLOATING_FLAG:",
    acm.FLOATING_FLAG.value_counts().to_dict())
print("    policy by scheme:"); print(pd.crosstab(acm.SCHEME_MST_ID, acm.INTEREST_POLICY).to_string())
print("    opened before 2024-07-01 (pre-migration):", int((acm.ACCOUNT_OPEN_DATE < "2024-07-01").sum()), "| on/after:",
    int((acm.ACCOUNT_OPEN_DATE >= "2024-07-01").sum()))
print("    open date range:", acm.ACCOUNT_OPEN_DATE.min().date(), "to", acm.ACCOUNT_OPEN_DATE.max().date())
print("    MARGIN_PERCENT:", alm.MARGIN_PERCENT.describe()[["count", "min", "max"]].to_dict(), "| FUNDED_FLAG:",
    alm.FUNDED_FLAG.value_counts().to_dict())
print("    loan master moratorium cols:", [c for c in alm.columns if "MOR" in c.upper()])

print("\n=== 3. COVERAGE per table (accounts of the 184)")
def cov(name, df):
    s = set(df.NEW_AC_NUMBER); print(f"    {name:<22} accounts {len(s & scope):3d} | outside scope {len(s - scope)} | rows
{len(df):,}")
    return s
for name, df in (("ledger", led), ("balance history", bal), ("loan book history", lb), ("chart (current)", chart), ("instalment
plans", plan), ("rate set-up", rate), ("charges", chg), ("disb schedule", disb), ("status history", status), ("daily accrual",
accr), ("interest summary", isum)):
    cov(name, df)
noled = acm[~acm.NEW_AC_NUMBER.isin(led.NEW_AC_NUMBER)]
print("    accounts with NO ledger rows:", len(noled), "| status:", noled.STATUS_CODE.value_counts().to_dict(), "| opened:",
    noled.ACCOUNT_OPEN_DATE.min().date(), "to", noled.ACCOUNT_OPEN_DATE.max().date())
nochart = acm[~acm.NEW_AC_NUMBER.isin(chart.NEW_AC_NUMBER)]
print("    accounts with NO current chart:", len(nochart), "| status:", nochart.STATUS_CODE.value_counts().to_dict(), "|
pre-migration among them:", int((nochart.ACCOUNT_OPEN_DATE < "2024-07-01").sum()))

print("\n=== 4. LEDGER: date range", led.TRANSACTION_DATE.min().date(), "to", led.TRANSACTION_DATE.max().date(), "| GL codes:",
    led.AC_GLCODE.value_counts().to_dict())
print("    trantype x sign:"); t = led.assign(sign=np.sign(led.TRANSAMT)); print(pd.crosstab(t.TRANTYPE, t.sign).to_string())
print("    DBCR values:", led.DBCR.value_counts().to_dict() if "DBCR" in led.columns else "n/a")
print("    TRANSACTION_DATE == CB_EFFECTIVE_DATE on", int((led.TRANSACTION_DATE == led.CB_EFFECTIVE_DATE).sum()), "of", len(led),
    "| == OPERATION_DATE on", int((led.TRANSACTION_DATE == led.OPERATION_DATE).sum()))
late = led[led.OPERATION_DATE.notna() & (led.OPERATION_DATE > led.TRANSACTION_DATE)]
print("    back-dated postings (operation date after transaction date):", len(late), "| by type:",
    late.TRANTYPE.value_counts().to_dict(), "| max days:", int((late.OPERATION_DATE - late.TRANSACTION_DATE).dt.days.max()))
for tt in ("300", "306", "343", "120", "400", "401", "900", "901"):
    sub = led[led.TRANTYPE == tt]
    if len(sub): print(f"        type {tt}: {len(sub)} rows, sample narrations:", sub.PARTICULARS.head(3).tolist())
print("    fee narrations among 301:", int(led[(led.TRANTYPE == "301")].PARTICULARS.str.contains("fee|charge|legal|arrang|insur|
commit", case=False).sum()), "of", int((led.TRANTYPE == "301").sum()))

print("\n=== 5. LOAN BOOK HISTORY: as-on dates")
lbd = lb.groupby("ASONDATE").agg(accounts=("NEW_AC_NUMBER", "nunique"), rows=("NEW_AC_NUMBER", "size"))
print("    distinct as-on dates:", len(lbd), "| first", lbd.index.min().date(), "last", lbd.index.max().date())
print("    month-ends among them:", int((lbd.index == lbd.index + pd.offsets.MonthEnd(0)).sum()), "| by year-month (accounts):",
    {str(k)[:7]: int(v) for k, v in lb.groupby(lb.ASONDATE.dt.to_period("M")).NEW_AC_NUMBER.nunique().items()})
print("    duplicate account rows on one as-on date:", int(lb.duplicated(["ASONDATE", "NEW_AC_NUMBER", "SUB_AC_NUMBER"]).sum()))

print("\n=== 6. BALANCE HISTORY: dates")
bd = bal.groupby(bal.TRANSACTION_DATE.dt.to_period("M")).NEW_AC_NUMBER.nunique()
print("    months:", {str(k): int(v) for k, v in bd.items()})
print("    all month-end?", bool((bal.TRANSACTION_DATE == bal.TRANSACTION_DATE + pd.offsets.MonthEnd(0)).all()), "| dup
account-date:", int(bal.duplicated(["NEW_AC_NUMBER", "SUB_AC_NUMBER", "TRANSACTION_DATE"]).sum()))

print("\n=== 7. RATE SET-UP: rows per account", rate.groupby("NEW_AC_NUMBER").size().describe()[["min", "50%",
    "max"]].to_dict(), "| APPLICABLE_FROM range", rate.APPLICABLE_FROM_DATE.min().date(), rate.APPLICABLE_FROM_DATE.max().date())
print("    DELETE_FLAG:", rate.DELETE_FLAG.value_counts().to_dict(), "| ACTIVE_FLAG:", rate.ACTIVE_FLAG.value_counts().to_dict(),
    "| policy:", rate.INTEREST_POLICY.value_counts().to_dict())
print("    sample Ebenezer:"); print(rate[rate.NEW_AC_NUMBER == "000104430000084"][["APPLICABLE_FROM_DATE", "INTEREST_POLICY",
    "PLR_RATE", "VERIATION_RATE", "INTEREST_RATE", "ACTIVE_FLAG", "DELETE_FLAG"]].to_string(index=False))

print("\n=== 8. CHART (current): columns with amounts:", [c for c in chart.columns if "AMT" in c or "BAL" in c or "RECOV" in c])
print("    rows per account", chart.groupby("NEW_AC_NUMBER").size().describe()[["min", "50%", "max"]].to_dict(), "|
```

```

DELETE_FLAG:", chart.DELETE_FLAG.value_counts().to_dict(), "| ACTIVE_FLAG:", chart.ACTIVE_FLAG.value_counts().to_dict())
print("    zero-amount rows:", int((chart.INSTALLMENT_AMT == 0).sum()), "| rows with INST_RECOVERED>0:",
int((chart.INST_RECOVERED > 0).sum()), "| partially recovered:", int(((chart.INST_RECOVERED > 0) & (chart.INST_RECOVERED <
chart.INSTALLMENT_AMT)).sum()))

print("\n== 9. INTEREST SUMMARY:", len(isum), "rows | dates", isum.TRANSACTION_DATE.min().date(),
isum.TRANSACTION_DATE.max().date(), "| POSTED_FLAG:", isum.POSTED_FLAG.value_counts().to_dict() if "POSTED_FLAG" in isum else
"")
print("== 10. CHARGES: in-scope accounts", chg.NEW_AC_NUMBER.isin(scope).sum(), "rows |",
chg[chg.NEW_AC_NUMBER.isin(scope)].DISB_CHARGES_NAME.value_counts().head(8).to_dict())
print("    POST_FLAG:", chg[chg.NEW_AC_NUMBER.isin(scope)].POST_FLAG.value_counts().to_dict(), "| INCOME_GL:",
chg[chg.NEW_AC_NUMBER.isin(scope)].INCOME_GL.value_counts().head(6).to_dict())
print("== 11. STATUS HISTORY: reasons for H and F:")
print(status[status.STATUS_CODE.isin(["H", "F", "D"])].groupby(["STATUS_CODE", "STATUS_CHANGE_REASON"]).size().to_string())
print("== 12. DAILY ACCRUAL: accounts", accr.NEW_AC_NUMBER.nunique(), "| dates", accr.TRANSACTION_DATE.min().date(),
accr.TRANSACTION_DATE.max().date(), "| DRCR:", accr.DRCR.value_counts().to_dict())

```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\fu_a1.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Audit 1: date-format proof, scope, coverage and key distributions

--- command ---

```

cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python fu_a1.py 2>&1 | cut -c1-330
{
  "timeout": 300000
}

```

Result of Bash

```
=== 1. DATE FORMAT PROOF: first component > 12 (would mean day-first) vs second component > 12 (month-first)
date values checked 100,504 | first part > 12: 0 | second part > 12: 66,762
P3_18: [{ 'DATE_COLUMN': 'CUMVOUCH.TRANSACTION_DATE', 'ROWS_CHECKED': 409552, 'ROWS_WITH_TIME_OF_DAY': 0 }, { 'DATE_COLUMN':
'ACCOUNT_BALANCE.TRANSACTION_DATE', 'ROWS_CHECKED': 79375, 'ROWS_WITH_TIME_OF_DAY': 0 }, { 'DATE_COLUMN':
'FIXLOAN_INST_CHART.INSTALLMENT_DATE', 'ROWS_CHECKED': 97249, 'ROWS_WITH_TIME_OF_DAY': 0 } ]

=== 2. SCOPE: accounts in account master 184 | by scheme: {90: 51, 92: 51, 86: 41, 84: 16, 143: 7, 88: 5, 140: 4, 142: 3, 141:
3, 144: 2, 94: 1}
status: { 'A': 89, 'C': 46, 'D': 25, 'H': 21, 'F': 3 }
INTEREST_POLICY per account: { 'S': 127, 'P': 42, 'M': 15 } | FLOATING_FLAG: { 'I': 103, 'F': 81 }
policy by scheme:
INTEREST_POLICY M P S
SCHEME_MST_ID
84 8 8 0
86 7 34 0
88 0 0 5
90 0 0 51
92 0 0 51
94 0 0 1
140 0 0 4
141 0 0 3
142 0 0 3
143 0 0 7
144 0 0 2
opened before 2024-07-01 (pre-migration): 109 | on/after: 75
open date range: 2020-03-04 to 2026-08-28
MARGIN_PERCENT: { 'count': 184.0, 'min': 0.0, 'max': 0.0 } | FUNDED_FLAG: { 'N': 184 }
loan master moratorium cols: [ 'PMOROTORIUM_PERIOD', 'PMOROTORIUM_UNIT', 'IMOROTORIUM_PERIOD', 'IMOROTORIUM_UNIT',
'MORT_PERIOD_SEP_INT' ]

=== 3. COVERAGE per table (accounts of the 184)
ledger accounts 132 | outside scope 0 | rows 3,640
balance history accounts 131 | outside scope 0 | rows 2,549
loan book history accounts 144 | outside scope 0 | rows 12,414
chart (current) accounts 75 | outside scope 0 | rows 2,742
instalment plans accounts 160 | outside scope 0 | rows 1,958
rate set-up accounts 183 | outside scope 0 | rows 1,291
charges accounts 52 | outside scope 1166 | rows 3,651
disb schedule accounts 161 | outside scope 0 | rows 251
status history accounts 184 | outside scope 0 | rows 283
daily accrual accounts 14 | outside scope 0 | rows 7,489
interest summary accounts 82 | outside scope 0 | rows 82
accounts with NO ledger rows: 52 | status: { 'C': 46, 'A': 6 } | opened: 2021-11-01 to 2025-10-29
accounts with NO current chart: 109 | status: { 'A': 53, 'C': 44, 'H': 6, 'D': 6 } | pre-migration among them: 56

=== 4. LEDGER: date range 2022-09-01 to 2026-09-30 | GL codes: { '1050202': 1303, '1050201': 1023, '1050102': 724, '1050101':
435, '1050401': 150, '3065': 5 }
trantype x sign:
sign -1.0 0.0 1.0
TRANSTYPE
120 0 0 6
300 0 0 2
301 214 0 0
303 2158 1 0
305 0 2 1210
306 37 0 0
343 0 0 2
400 1 0 0
401 0 0 5
900 1 0 0
901 0 0 1
DBCR values: { 'DR': 2412, 'CR': 1228 }
TRANSACTION_DATE == CB_EFFECTIVE_DATE on 3640 of 3640 | == OPERATION_DATE on 3516
back-dated postings (operation date after transaction date): 17 | by type: {303: 13, 301: 4} | max days: 699
fee narrations among 301: 0 of 0

=== 5. LOAN BOOK HISTORY: as-on dates
distinct as-on dates: 21 | first 2024-12-31 last 2026-08-31
month-ends among them: 21 | by year-month (accounts): { '2024-12': 85, '2025-01': 86, '2025-02': 86, '2025-03': 86,
'2025-04': 92, '2025-05': 99, '2025-06': 101, '2025-07': 107, '2025-08': 108, '2025-09': 118, '2025-10': 119, '2025-11': 125,
'2025-12': 123, '2026-01': 123, '2026-02': 123, '2026-03': 125, '2026-04': 127, '2026-
duplicate account rows on one as-on date: 10053

=== 6. BALANCE HISTORY: dates
months: { '2024-07': 78, '2024-08': 78, '2024-09': 76, '2024-10': 78, '2024-11': 80, '2024-12': 82, '2025-01': 84, '2025-02':
84, '2025-03': 84, '2025-04': 84, '2025-05': 92, '2025-06': 95, '2025-07': 99, '2025-08': 103, '2025-09': 104, '2025-10': 105,
'2025-11': 109, '2025-12': 116, '2026-01': 118, '2026-02': 118, '2026-03':
all month-end? True | dup account-date: 1
=== 7. RATE SET-UP: rows per account { 'min': 1.0, '50%': 1.0, 'max': 54.0 } | APPLICABLE_FROM range 2020-02-01 2026-09-03
DELETE_FLAG: { 'N': 1185, 'Y': 106 } | ACTIVE_FLAG: { 'Y': 1212, 'N': 79 } | policy: { 'P': 998, 'M': 167, 'S': 126 }
sample Ebenezer:
APPLICABLE_FROM_DATE INTEREST_POLICY PLR_RATE VERIATION_RATE INTEREST_RATE ACTIVE_FLAG DELETE_FLAG
2025-07-15 P 0.0 0.0 31.2 Y N
2025-07-16 P 25.3 0.0 31.2 Y N
2025-07-23 P 25.3 0.0 31.2 Y N
2025-09-03 P 25.3 0.0 31.2 Y N
2025-10-01 P 25.4 0.1 31.3 Y N
2025-11-01 P 25.3 -0.1 31.2 Y N
2026-02-04 P 24.7 -0.6 30.6 N Y
```

2026-02-04	P	24.7	-0.6	30.6	N	Y
2026-02-04	P	24.7	-0.6	30.6	Y	N
2026-03-05	P	23.7	-1.0	29.6	Y	N
2026-03-09	P	22.4	-1.3	28.3	Y	N
2026-04-20	P	20.8	-1.6	26.7	Y	N
2026-05-06	P	20.6	-0.2	26.5	Y	N
2026-06-03	P	20.4	-0.2	26.3	Y	N
2026-07-03	P	20.5	0.1	26.4	Y	N
2026-08-05	P	20.8	0.3	26.7	Y	N
2026-09-03	P	21.2	0.4	27.1	Y	N

```

=== 8. CHART (current): columns with amounts: ['INSTALLMENT_AMT', 'INST_RECOVERED', 'INTR_INSTALLMENT_AMT',
'PRIN_INSTALLMENT_AMT', 'EXP_BALANCE', 'INTEREST_RECOVERED', 'PENAL_RECOVERED', 'ACT_INT_RECOVERY', 'RECOV_CUMVOID',
'INT_RECOVERY_REUPDATE']
rows per account {'min': 3.0, '50%': 28.0, 'max': 73.0} | DELETE_FLAG: {'N': 2742} | ACTIVE_FLAG: {'Y': 2742}
zero-amount rows: 659 | rows with INST_RECOVERED>0: 436 | partially recovered: 269

```

```

=== 9. INTEREST SUMMARY: 82 rows | dates 2026-08-31 2026-08-31 | POSTED_FLAG: {'Y': 82}
=== 10. CHARGES: in-scope accounts 155 rows | {'Agreement Fee (%)': 52, 'Legal Fee (%)': 52, 'Other Fee (%)': 51}
POST_FLAG: {'N': 155} | INCOME_GL: {'': 155}

```

```

=== 11. STATUS HISTORY: reasons for H and F:

```

STATUS_CODE	STATUS_CHANGE_REASON	
D	Account closure	1
	Finalization	1
	Foreclosure	15
	LAPSED	1
	PAID UP	25
	Paid Up	4
F	Repayment reconciliations	1
	PAID UP	1
	Paid Up	2

```

=== 12. DAILY ACCRUAL: accounts 14 | dates 2024-11-01 2026-09-29 | DRCR: {'CR': 7259, 'DR': 230}
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\fu_a2.py
--- content ---
"""Audit 2: reconciliations between the ledger, the balance history, the loan book history, Extract C and the terms."""
import re
import pandas as pd, numpy as np
from fu_load import load, X
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 40); pd.set_option("display.max_colwidth", 60)
led, bal, lb, acm, alm, chart, chg, rate = (load(k) for k in ("ledger", "bal", "lb", "acm", "alm", "chart", "chg", "rate"))
led["TRANSTYPE"] = led.TRANSTYPE.astype(int)
key = ["NEW_AC_NUMBER", "SUB_ACCOUNT_NO"]
led = led.sort_values(["NEW_AC_NUMBER", "SUB_ACCOUNT_NO", "TRANSACTION_DATE", "CUMVOUCH_DET_ID"])
own = led[led.AC_GLCODE == led.NEW_AC_NUMBER.map(acm.set_index("NEW_AC_NUMBER").GLCODE)]
print("ledger rows on the loan's own GL:", len(own), "of", len(led), "| other GL rows:",
led[~led.index.isin(own.index)].AC_GLCODE.value_counts().to_dict())
own = own.copy(); own["run"] = own.groupby(key).TRANSAMT.cumsum()

# (a) ledger running balance vs balance history at month-end
me = own.groupby(key + [own.TRANSACTION_DATE.dt.to_period("M")]).run.last().reset_index().rename(columns={"TRANSACTION_DATE":
"period"})
b = bal.copy(); b["period"] = b.TRANSACTION_DATE.dt.to_period("M"); b = b.rename(columns={"SUB_AC_NUMBER": "SUB_ACCOUNT_NO"})
# carry the ledger balance forward into months with no posting
allm = []
for k, g in me.groupby(key):
    g = g.set_index("period").run
    idx = pd.period_range(g.index.min(), "2026-09", freq="M")
    allm.append(pd.DataFrame({"NEW_AC_NUMBER": k[0], "SUB_ACCOUNT_NO": k[1], "period": idx, "ledger_bal":
g.reindex(idx).ffill().values}))
allm = pd.concat(allm)
m = b.merge(allm, on=key + ["period"], how="left")
m["diff"] = m.PRINCIPAL_BALANCE - m.ledger_bal
ok = (m["diff"].abs() < 1)
print(f"\n(a) BALANCE HISTORY vs LEDGER running balance: {len(m):,} account-months | agree within 1: {int(ok.sum()):,} |
differ: {int(~ok.sum()):,} | no ledger: {int(m.ledger_bal.isna().sum()):,}")
bad = m[~ok & m.ledger_bal.notna()]
print("    differing accounts:", bad.NEW_AC_NUMBER.nunique(), "| first differing months:",
bad.groupby("NEW_AC_NUMBER").period.min().astype(str).value_counts().head(5).to_dict())
print(bad.sort_values("diff", key=abs, ascending=False)[["NEW_AC_NUMBER", "period", "PRINCIPAL_BALANCE", "ledger_bal",
"diff"]].head(8).to_string(index=False))

# (b) loan book history (latest run per as-on) vs ledger
lbx = lb.sort_values("TRANSACTION_DATE").drop_duplicates(["ASONDATE", "NEW_AC_NUMBER", "SUB_AC_NUMBER"], keep="last").copy()
print(f"\n(b) LOAN BOOK HISTORY: {len(lb):,} rows -> {len(lbx):,} after keeping the latest run per as-on date | runs per as-on
date:", lb.groupby("ASONDATE").TRANSACTION_DATE.nunique().describe()[["min", "max"]].to_dict())
lbx["period"] = lbx.ASONDATE.dt.to_period("M"); lbx = lbx.rename(columns={"SUB_AC_NUMBER": "SUB_ACCOUNT_NO"})
mb = lbx.merge(allm, on=key + ["period"], how="left")
for col in ("PRINCIPAL", "CARRYING_AMOUNT"):
    d = mb[col] + mb.ledger_bal # ledger balance is negative (debit)
    print(f"    {col} vs -ledger: agree within 1 on {int((d.abs() < 1).sum()):,} of {int(mb.ledger_bal.notna().sum()):,} with a
ledger balance")
mb["d_carry"] = mb.CARRYING_AMOUNT + mb.ledger_bal
print(mb[(mb.d_carry.abs() >= 1) & mb.ledger_bal.notna()].sort_values("d_carry", key=abs, ascending=False)[["NEW_AC_NUMBER",
"ASONDATE", "PRINCIPAL", "CARRYING_AMOUNT", "ledger_bal", "d_carry"]].head(6).to_string(index=False))
print("    loan book vs balance history, same month, PRINCIPAL vs PRINCIPAL_BALANCE:", end=" ")
mm = lbx.merge(b[key + ["period", "PRINCIPAL_BALANCE"]], on=key + ["period"], how="inner")
print(f"    {int(((mm.PRINCIPAL + mm.PRINCIPAL_BALANCE).abs() < 1).sum()):,} of {len(mm):,} agree")

# (c) ledger interest vs Extract C run 41
C = pd.read_csv(X + r"\ExtractC_Jan2025-July2026.xlsx".replace(".xlsx", ".csv"), dtype=str) if False else pd.read_excel(X + r"\
ExtractC_Jan2025-July2026.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
C = C[(C.RUN_ID == 41) & (C.PERIOD_TYPE == "MONTH")]
C["period"] = pd.PeriodIndex(year=C.PERIOD_YEAR.astype(int), month=C.PERIOD_MONTH.astype(int), freq="M")
li = own[own.TRANSTYPE.isin([120, 303, 308, 309, 310, 311]).groupby(["NEW_AC_NUMBER",
own.TRANSACTION_DATE.dt.to_period("M")]).TRANSAMT.sum().mul(-1).rename("ledger_int").reset_index().rename(columns={"TRANSACTION_
DATE": "period"})
mc = C.merge(li, left_on=["LOAN_ACCOUNT_NUMBER", "period"], right_on=["NEW_AC_NUMBER", "period"], how="outer", indicator=True)
mc["diff"] = mc.INTEREST_INCOME_POSTED.fillna(0) - mc.ledger_int.fillna(0)
print(f"\n(c) EXTRACT C (run 41) vs LEDGER types 120+303 by account-month: {len(mc):,} | both: {int((mc._merge ==
'both').sum()):,} | only C: {int((mc._merge == 'left_only').sum()):,} | only ledger: {int((mc._merge == 'right_only').sum()):,} |
agree within 1: {int((mc['diff'].abs() < 1).sum()):,}")
print("    totals Jan-2025 to Jul-2026: C", f"{C.INTEREST_INCOME_POSTED.sum():.2f}", "| ledger", f"{li[(li.period >= '2025-01')
& (li.period <= '2026-07')].ledger_int.sum():.2f}")
print(mc[(mc['diff'].abs() >= 1)][["LOAN_ACCOUNT_NUMBER", "NEW_AC_NUMBER", "period", "INTEREST_INCOME_POSTED", "ledger_int",
"diff", "_merge"]].head(6).to_string(index=False))
oth = own[own.TRANSTYPE == 306]
print("    type 306 narrations:", oth.PARTICULARS.str[:60].value_counts().head(5).to_dict())

# (d) recompute each interest posting from the narration: balance before x rate x days / 365
pat = re.compile(r"Fm (\d{2}-[A-Za-z]{3}-\d{2}) To (\d{2}-[A-Za-z]{3}-\d{2}) @ ?([\d.]+)%", re.I)
own["prev"] = own.groupby(key).run.shift(1).fillna(0)
ii = own[own.TRANSTYPE == 303].copy()
ex = ii.PARTICULARS.str.extract(pat)
ii["d_from"] = pd.to_datetime(ex[0], format="%d-%b-%y", errors="coerce"); ii["d_to"] = pd.to_datetime(ex[1], format="%d-%b-%y",
errors="coerce"); ii["rate"] = pd.to_numeric(ex[2], errors="coerce")
ii["days"] = (ii.d_to - ii.d_from).dt.days + 1
ii["expected"] = -(ii.prev) * ii.rate / 100 * ii.days / 365
ii["ratio"] = ii.TRANSAMT / ii.expected
parsed = ii.rate.notna()
close = parsed & ((ii.ratio - 1).abs() < 0.005)
```

```

print(f"\n(d) RECOMPUTE 303 POSTINGS from narration (balance before x rate x days/365): {len(ii):,} postings | narration parsed
{int(parsed.sum()):,} | within 0.5%: {int(close.sum()):,} | within 2%: {int((parsed & ((ii.ratio - 1).abs() < 0.02)).sum()):,}")
print("    rate in narration vs rate set-up table in force (active, not deleted):", end=" ")
rs = rate[(rate.DELETE_FLAG == "N")].sort_values("APPLICABLE_FROM_DATE")
hits = 0; tested = 0
for _, r in ii[parsed].iterrows():
    cand = rs[(rs.NEW_AC_NUMBER == r.NEW_AC_NUMBER) & (rs.APPLICABLE_FROM_DATE <= r.d_to)]
    if len(cand):
        tested += 1; hits += int(abs(cand.iloc[-1].INTEREST_RATE - r.rate) < 0.051)
print(f"{hits} of {tested} agree")
far = ii[parsed & ~close]
print("    not within 0.5%: days used vs calendar month, sample:"); print(far[["NEW_AC_NUMBER", "TRANSACTION_DATE", "prev",
"rate", "days", "TRANSAMT", "expected", "ratio"]].head(6).round(2).to_string(index=False))
print("    first-month postings (posting dated same month as first 301):", end=" ")
first301 = own[own.TRANTYPE == 301].groupby("NEW_AC_NUMBER").TRANSACTION_DATE.min()
fm = ii[ii.TRANSACTION_DATE.dt.to_period("M") == first301].dt.to_period("M")
print(len(fm), "| within 0.5%:", int(((fm.ratio - 1).abs() < 0.005).sum()))

# (e) disbursements vs sanction, fees in narrations vs charges table
d301 = own[own.TRANTYPE == 301].groupby("NEW_AC_NUMBER").TRANSAMT.sum().mul(-1)
s = alm.set_index("NEW_AC_NUMBER").SANCTION_AMOUNT
cmp = pd.concat([d301.rename("drawn"), s.rename("sanction")], axis=1).dropna()
print(f"\n(e) DISBURSEMENTS: accounts with 301 postings {len(d301)} | drawn = sanction on {int(((cmp.drawn -
cmp.sanction).abs() < 1).sum())} | drawn < sanction on {int((cmp.drawn < cmp.sanction - 1).sum())} | drawn > sanction on
{int((cmp.drawn > cmp.sanction + 1).sum())}")
fee301 = own[(own.TRANTYPE == 301) & own.PARTICULARS.str.contains("fee|charge|legal|arrang|insur|commit|process", case=False,
na=False)]
print("    301 rows whose narration names a fee:", len(fee301), "on", fee301.NEW_AC_NUMBER.nunique(), "accounts | total",
f"{-fee301.TRANSAMT.sum():,.2f}")
print("    sample:", fee301.PARTICULARS.str[:50].head(6).tolist())
ch = chg[chg.NEW_AC_NUMBER.isin(acm.NEW_AC_NUMBER)]
print("    charges table in-scope: accounts", ch.NEW_AC_NUMBER.nunique(), "| ACTUAL_CHARGES_AMT total",
f"{ch.ACTUAL_CHARGES_AMT.sum():,.2f}", "| APLLI total", f"{ch.APLLI_CHARGES_AMT.sum():,.2f}", "| rows with amount > 0:",
int((ch.ACTUAL_CHARGES_AMT > 0).sum()))
both = set(fee301.NEW_AC_NUMBER) & set(ch.ACTUAL_CHARGES_AMT > 0).NEW_AC_NUMBER
print("    accounts with fee narration AND a charges-table amount:", len(both), "| narration only:",
len(set(fee301.NEW_AC_NUMBER) - set(ch[ch.ACTUAL_CHARGES_AMT > 0].NEW_AC_NUMBER)), "| charges only:",
len(set(ch[ch.ACTUAL_CHARGES_AMT > 0].NEW_AC_NUMBER) - set(fee301.NEW_AC_NUMBER)))
for a in list(both)[:3]:
    print("        ", a, "narration fees", f"{-fee301[fee301.NEW_AC_NUMBER == a].TRANSAMT.sum():,.2f}", "| charges",
ch[ch.NEW_AC_NUMBER == a][["DISB_CHARGES_NAME", "ACTUAL_CHARGES_AMT", "APLLI_CHARGES_AMT"]].values.tolist())

# (f) Extract A dates vs the master now
A = pd.read_excel(X + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
A = A[A["AS_OF_DATE"].astype(str).str[:10] == "31-12-2025"].set_index("LOAN_ACCOUNT_NUMBER")
def parse_a(v):
    if isinstance(v, pd.Timestamp): return v, "excel"
    try: return pd.to_datetime(str(v)[:10], format="%d-%m-%Y"), "text"
    except Exception: return pd.NaT, "bad"
rows = []
for k, v in A.LOAN_START_DATE.items():
    d, kind = parse_a(v); real = acm.set_index("NEW_AC_NUMBER").ACCOUNT_OPEN_DATE.get(k)
    if real is None or pd.isna(d): continue
    swapped = pd.NaT
    if kind == "excel":
        try: swapped = pd.Timestamp(year=d.year, month=d.month, day=d.day)
        except Exception: pass
    rows.append((kind, d == real, swapped == real))
f = pd.DataFrame(rows, columns=["kind", "match", "match_after_swap"])
print(f"\n(f) EXTRACT A loan start date vs ACCOUNT_OPEN_DATE now:"); print(f.groupby("kind").agg(n=("match", "size"),
match=("match", "sum"), match_after_swap=("match_after_swap", "sum")).to_string())

```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\fu_a2.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Audit 2: reconcile ledger to balances, loan book, Extract C, narration rates, sanctions and fees

--- command ---

```

cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python fu_a2.py 2>&1 | cut -c1-330; ls "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Trial Balances" | head -30
{
  "timeout": 600000
}

```

Result of Bash

ledger rows on the loan's own GL: 3640 of 3640 | other GL rows: {}

(a) BALANCE HISTORY vs LEDGER running balance: 2,549 account-months | agree within 1: 2,533 | differ: 16 | no ledger: 0

differing accounts: 1		first differing months: {'2025-05': 1}		
NEW_AC_NUMBER	period	PRINCIPAL_BALANCE	ledger_bal	diff
000104450000015	2025-05	-27303.21	-1854936.83	1827633.62
000104450000015	2025-06	658534.33	-1169099.29	1827633.62
000104450000015	2025-07	658534.33	-1169099.29	1827633.62
000104450000015	2025-08	3053316.65	1225683.03	1827633.62
000104450000015	2025-09	3053316.65	1225683.03	1827633.62
000104450000015	2025-10	3053316.65	1225683.03	1827633.62
000104450000015	2025-11	3053316.65	1225683.03	1827633.62
000104450000015	2025-12	3053316.65	1225683.03	1827633.62

```
(b) LOAN BOOK HISTORY: 12,414 rows -> 2,361 after keeping the latest run per as-on date | runs per as-on date: {'min': 1.0, 'max': 12.0}
```

PRINCIPAL vs -ledger: agree within 1 on 15 of 2,264 with a ledger balance
CARRYING AMOUNT vs -ledger: agree within 1 on 2,264 of 2,264 with a ledger balance

```
Empty DataFrame
Columns: [NEW_AC_NUMBER, ASONDATE, PRINCIPAL, CARRYING_AMOUNT, ledger_bal, d_carry]
Index: []
```

```
loan book vs balance history, same month, PRINCIPAL vs PRINCIPAL_BALANCE: 15 of 2,159 agree
```

Traceback (most recent call last):

File "C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\fu_a2.py", line 49, in <module>

```
C["period"] = pd.PeriodIndex(year=C.PERIOD_YEAR.astype(int), month=C.PERIOD_MONTH.astype(int), freq="M")
```

```
TypeError: PeriodIndex.new () got an unexpected keyword argument 'year'
```

```

Trial Balance_28 February 2025.xls
Trial Balance_28 February 2026.xls
Trial Balance_30 April 2025.xls
Trial Balance_30 April 2026.xls
Trial Balance_30 June 2025.xls
Trial Balance_30 June 2026.xls
Trial Balance_30 November 2025.xls
Trial Balance_30 September 2025.xls
Trial Balance_31 August 2025.xls
Trial Balance_31 August 2026.xls
Trial Balance_31 December 2025.xls
Trial Balance_31 January 2026.xls
Trial Balance_31 July 2025.xls
Trial Balance_31 July 2026.xls
Trial Balance_31 March 2025.xls
Trial Balance_31 March 2026.xls
Trial Balance_31 May 2025.xls
Trial Balance_31 May 2026.xls
Trial Balance_31 October 2025.xls

```

```
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

```
description: Rerun the remaining reconciliations
```

```

--- command ---

```

```
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && sed -i 's/C\[\"period\"\] = pd.PeriodIndex(year=C.PERIOD_YEAR.astype(int), month=C.PERIOD_MONTH.astype(int), freq=\"M\")/C\[\"period\"\] = pd.to_datetime(C.PERIOD_YEAR.astype(int).astype(str) + \"-\" + C.PERIOD_MONTH.astype(int).astype(str).str.zfill(2) + \"-01\").dt.to_period(\"M\")/' fu_a2.py && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python fu_a2.py 2>&1 | sed -n '/^(c)/,$p' | cut -c1-330
{
  "timeout": 600000
}
```

Result of Bash

(c) EXTRACT C (run 41) vs LEDGER types 120+303 by account-month: 2,101 | both: 1,564 | only C: 0 | only ledger: 537 | agree within 1: 1,565

totals Jan-2025 to Jul-2026: C 5,293,988,207.06 | ledger 5,293,988,207.06

LOAN_ACCOUNT_NUMBER	NEW_AC_NUMBER	period	INTEREST_INCOME_POSTED	ledger_int	diff	_merge
NaN	000104420000003	2024-07	NaN	1004624.69	-1.004625e+06	right_only
NaN	000104420000005	2024-07	NaN	237378159.64	-2.373782e+08	right_only
NaN	000104420000005	2024-08	NaN	10999883.62	-1.099988e+07	right_only
NaN	000104420000005	2024-09	NaN	10314500.08	-1.031450e+07	right_only
NaN	000104420000005	2024-10	NaN	10708141.13	-1.070814e+07	right_only
NaN	000104420000005	2024-11	NaN	10648316.70	-1.064832e+07	right_only

type 306 narrations: {'To Trf reversal': 11, 'To Trf Reversal for duplicate payments': 3, 'To Trf JVD AGRO LIMITED as per closure quotation 31 Mar 2026': 2, 'To Trf Reverse of miss posting transaction': 1, 'To Trf Reversal for missposting': 1}

(d) RECOMPUTE 303 POSTINGS from narration (balance before x rate x days/365): 2,159 postings | narration parsed 2,047 | within 0.5%: 1,102 | within 2%: 1,257

rate in narration vs rate set-up table in force (active, not deleted): 1659 of 2047 agree

not within 0.5%: days used vs calendar month, sample:

NEW_AC_NUMBER	TRANSACTION_DATE	prev rate	days	TRANSAMT	expected	ratio	
000104420000005	2024-09-30	-3.782209e+08	32.45	30.0	-10314500.08	-10087616.79	1.02
000104420000005	2025-07-31	-2.938184e+08	32.65	31.0	-12262419.44	-8147624.85	1.51
000104420000005	2025-11-30	-1.300009e+08	32.65	30.0	-8134792.21	-3488653.98	2.33
000104420000006	2024-09-30	-1.309184e+07	10.00	30.0	-105599.23	-107604.17	0.98
000104420000006	2024-10-31	-1.345636e+07	10.00	31.0	-112087.85	-114286.92	0.98
000104420000007	2024-08-31	-1.716643e+08	32.10	31.0	-4711598.78	-4680085.91	1.01

first-month postings (posting dated same month as first 301): 123 | within 0.5%: 33

(e) DISBURSEMENTS: accounts with 301 postings 131 | drawn = sanction on 112 | drawn < sanction on 13 | drawn > sanction on 6

301 rows whose narration names a fee: 43 on 18 accounts | total 1,655,425,555.84

sample: ['To Trf Arrangement fee on loan', 'To Trf Legal fee on loan disbursement', 'To Trf Insurance on loan', 'To Trf Arrangement fees', 'To Trf Legal Fees', 'To Trf Insurance']

charges table in-scope: accounts 52 | ACTUAL_CHARGES_AMT total 350,668,023.00 | APLLI total 350,668,023.00 | rows with amount > 0: 84

accounts with fee narration AND a charges-table amount: 13 | narration only: 5 | charges only: 25

000104430000080 narration fees 2,553,902.60 | charges [['Agreement Fee (%)', 1500000.0, 1500000.0], ['Legal Fee (%)', 855000.0, 855000.0], ['Other Fee (%)', 0.0, 0.0]]

000104460000117 narration fees 5,641,647.00 | charges [['Agreement Fee (%)', 2103705.0, 2103705.0], ['Legal Fee (%)', 3537942.0, 3537942.0], ['Other Fee (%)', 0.0, 0.0]]

000104430000086 narration fees 10,520,000.00 | charges [['Agreement Fee (%)', 7500000.0, 7500000.0], ['Legal Fee (%)', 3265000.0, 3265000.0], ['Other Fee (%)', 0.0, 0.0]]

(f) EXTRACT A loan start date vs ACCOUNT_OPEN_DATE now:

```
n match match_after_swap
kind
text 108 108 0
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

description: Inspect the trial balance layout and find the loan and interest income lines

```
--- command ---
cd "/C:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && ./songenv/Scripts/python -m pip install -q xlrtd 2>&1 | tail -1; PYTHONIOENCODING=utf-8 ./songenv/Scripts/python - <<'PYEOF' 2>&1 | cut -c1-250
import pandas as pd
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 20); pd.set_option("display.max_colwidth", 45)
T = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Trial Balances"
x = pd.ExcelFile(T + r"\Trial Balance_31 January 2025.xls")
print(x.sheet_names)
df = x.parse(x.sheet_names[0], header=None)
print(df.shape); print(df.head(12).to_string())
m = df.astype(str).apply(lambda r: r.str.contains("1050101|1050102|1050201|1050202|1050401|4215|4216|4218|4219|3065|4871|4873",
regex=True).any(), axis=1)
print(df[m].to_string())
PYEOF
{
  "timeout": 120000
}
```

Result of Bash

```
['rpt_Trial_Balance_Malawi']  
(177, 4)
```

	0	1	2	3
0	NaN	2025-01-01 00:00:00	NaN	NaN
1	NaN	GL Title	Debit	Credit
2	NaN	1017...NBS Cash Collection	868,069,514.47	0.00
3	NaN	1018...NBS Current A/C 21011223	39,835,328.76	0.00
4	NaN	1001...CDH Inv Bank 0050401042201	122,242,681.13	0.00
5	NaN	1022...AGRA- FCDA	220,328,718.72	0.00
6	NaN	1010...Money Market CDHIB Call Account	2,345,075,967.54	0.00
7	NaN	1011...Money Market FDH Discount House	2,100,515,658.89	0.00
8	NaN	1002...CDH Inv Bank PIA 0050401044100	30,795,478.29	0.00
9	NaN	1005...FDI Project Payment-USD	4.67	0.00
10	NaN	1014...Money Market Continental Asset Management	1,286,404,450.06	0.00
11	NaN	1019...NBS USD A/C 21011436	1,846,377.70	0.00
	0	1	2	3
31	NaN	1050101...MAIIC Agricultural Loans	1,642,334,523.43	0.00
32	NaN	1050102...MAIIC Industrial Loans	2,312,356,889.21	0.00
33	NaN	1050201...FInES Agricultural Loans	1,857,622,600.94	0.00
34	NaN	1050202...FInES Industrial Loans	2,667,566,257.31	0.00
60	NaN	3065...NASCOMEX Limited	2,321,428,571.00	0.00
122	NaN	4219...Interest On FInES Industrial Loans	0.00	23,309,775.88
124	NaN	4215...Interest On MAIIC Agricultural Loans	0.00	47,120,571.28
125	NaN	4216...Interest On MAIIC Industrial Loans	0.00	51,993,819.10
133	NaN	4871...Legal Fees	0.00	8,908,500.00
134	NaN	4873...Arrangement Fees	0.00	14,500,000.00

Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\fu_a3.py
--- content ---
"""Audit 3: trial balance against the ledger (balance sheet and income), interest summary, fees, chart, Extract A dates."""
import os, re, glob
import pandas as pd, numpy as np
from fu_load import load, X
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 40); pd.set_option("display.max_colwidth", 60)
T = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Trial Balances"
led, acm, alm, chart, chg, isum = (load(k) for k in ("ledger", "acm", "alm", "chart", "chg", "isum"))
led["TRANTYPE"] = led.TRANTYPE.astype(int)
key = ["NEW_AC_NUMBER", "SUB_ACCOUNT_NO"]
led = led.sort_values(key + ["TRANSACTION_DATE", "CUMVOUCH_DET_ID"]); led["run"] = led.groupby(key).TRANSAMT.cumsum()
led["period"] = led.TRANSACTION_DATE.dt.to_period("M")

# ---- trial balances
def num(v):
    try: return float(str(v).replace(",",""))
    except Exception: return np.nan
tb = {}
for f in glob.glob(T + r"\Trial Balance*.xls"):
    m = re.search(r"_(\d{1,2})_w+ \d{4})", f); per = pd.to_datetime(m.group(1), format="%d %B %Y").to_period("M")
    df = pd.read_excel(f, header=None)
    rows = {}
    for _, r in df.iterrows():
        t = str(r[1])
        mm = re.match(r"^(\\d{4,7})\\.\\.\\.\\.\\.(.*)", t)
        if mm: rows[mm.group(1)] = (mm.group(2).strip(), num(r[2]), num(r[3]))
    tb[per] = rows
pers = sorted(tb)
print("TB months:", pers[0], "to", pers[-1], "| count", len(pers))
loan_gls = ["1050101", "1050102", "1050201", "1050202", "1050401"]
print("\n(g) BALANCE SHEET: TB debit balance per loan GL vs sum of ledger running balances at month-end")
gl_bal = []
for per in pers:
    for gl in loan_gls:
        sub = led[(led.AC_GLCODE == gl) & (led.period <= per)]
        ledger = -sub.groupby(key).run.last().sum() if len(sub) else 0.0
        tbv = tb[per].get(gl, (None, np.nan, np.nan))
        gl_bal.append((str(per), gl, tbv[1], ledger, (tbv[1] - ledger) if pd.notna(tbv[1]) else np.nan))
gb = pd.DataFrame(gl_bal, columns=["month", "gl", "tb_debit", "ledger", "diff"])
print(" months x GL with TB figure:", int(gb.tb_debit.notna().sum()), "| agree within 1:", int((gb["diff"].abs() < 1).sum()),
"| differ:", int((gb["diff"].abs() >= 1).sum()))
print(gb[gb["diff"].abs() >= 1].round(2).to_string(index=False))
print(" total loan book at 2025-12:", f"{gb[gb.month == '2025-12'].tb_debit.sum():.2f}", "TB vs ledger", f"{gb[gb.month ==
'2025-12'].ledger.sum():.2f}")

print("\n(h) INCOME: TB interest income accounts and whether they are monthly or cumulative")
inc = {gl: t for gl, t in tb[pers[-1]].items() if gl.startswith("42")}
print(" 42xx accounts in the latest TB:", {k: v[0] for k, v in inc.items()})
MAP = {"4215": "1050101", "4216": "1050102", "4218": "1050201", "4219": "1050202"}
for gl in tb[pers[-1]]:
    if gl.startswith("42") and "Term" in tb[pers[-1]][gl][0]: MAP[gl] = "1050401"
rows = []
for i, per in enumerate(pers):
    for igl, lgl in MAP.items():
        cr = tb[per].get(igl, (None, np.nan, np.nan))[2]
        prev = tb[pers[i - 1]].get(igl, (None, np.nan, 0.0))[2] if i > 0 and pers[i - 1].year == per.year else 0.0
        move = cr - (prev if pd.notna(prev) else 0.0) if pd.notna(cr) else np.nan
        ledger = -led[(led.AC_GLCODE == lgl) & (led.period == per) & led.TRANTYPE.isin([120, 303]).TRANSAMT.sum()
        rows.append((str(per), igl, lgl, cr, move, ledger))
ic = pd.DataFrame(rows, columns=["month", "income_gl", "loan_gl", "tb_credit_balance", "tb_movement_ytd_basis",
"ledger_interest"])
ic["diff_vs_balance"] = ic.tb_credit_balance - ic.ledger_interest; ic["diff_vs_movement"] = ic.tb_movement_ytd_basis -
ic.ledger_interest
print(" rows:", len(ic), "| ledger = TB balance (monthly basis):", int((ic.diff_vs_balance.abs() < 1).sum()), "| ledger = TB
movement (YTD basis):", int((ic.diff_vs_movement.abs() < 1).sum()))
print(ic[ic.month.isin(["2025-01", "2025-02", "2025-12", "2026-01", "2026-08"])]].round(2).to_string(index=False))
ic.to_csv("tb_income_recon.csv", index=False); gb.to_csv("tb_balance_recon.csv", index=False)
fees = {gl: t for gl, t in tb[pers[-1]].items() if gl in ("4871", "4872", "4873")}
print(" fee income accounts latest TB:", fees)

print("\n(i) INTEREST SUMMARY (Aug 2026) vs ledger 303 of 31 Aug 2026")
l8 = led[(led.TRANTYPE == 303) & (led.TRANSACTION_DATE == "2026-08-31")].groupby("NEW_AC_NUMBER").TRANSAMT.sum().mul(-1)
j = isum.set_index("NEW_AC_NUMBER")[["INTEREST_AMOUNT", "OVERDUE_INTEREST", "TOTAL_INTEREST", "INTEREST_RATE", "NO_OF_DAYS",
"OVD_INT_RATE", "PENAL_RATE"]].join(l8.rename("ledger_303"), how="outer")
print(" accounts:", len(j), "| TOTAL_INTEREST = ledger:", int((j.TOTAL_INTEREST - j.ledger_303).abs() < 1).sum()), "|
INTEREST_AMOUNT = ledger:", int((j.INTEREST_AMOUNT - j.ledger_303).abs() < 1).sum()), "| rows with overdue interest > 0:",
int((j.OVERDUE_INTEREST > 0).sum()))
print(" overdue interest total Aug-2026:", f"{j.OVERDUE_INTEREST.sum():.2f}", "of total", f"{j.TOTAL_INTEREST.sum():.2f}",
"| penal rates used:", j.PENAL_RATE.value_counts().head(4).to_dict())

print("\n(j) FEES: the largest fee-like 301 narrations")
fee301 = led[(led.TRANTYPE == 301) & led.PARTICULARS.str.contains("fee|charge|legal|arrang|insur|commit|process", case=False,
na=False)]
print(fee301.sort_values("TRANSAMT")[["NEW_AC_NUMBER", "TRANSACTION_DATE", "TRANSAMT",
"PARTICULARS"]].head(8).to_string(index=False))
ins = fee301[fee301.PARTICULARS.str.contains("insur", case=False)]
print(" insurance rows:", len(ins), f"{-ins.TRANSAMT.sum():.2f}", "| arrangement:",
```

```

f"{-fee301[fee301.PARTICULARS.str.contains('arrang', case=False)].TRANSAMT.sum():,.2f}", "| legal:",
f"{-fee301[fee301.PARTICULARS.str.contains('legal', case=False)].TRANSAMT.sum():,.2f}")
ch = chg[chg.NEW_AC_NUMBER.isin(acm.NEW_AC_NUMBER) & (chg.ACTUAL_CHARGES_AMT > 0)]
print("   charges table (in scope, amount > 0): by name", ch.groupby("DISB_CHARGES_NAME").ACTUAL_CHARGES_AMT.agg(["count",
"sum"]).round(0).to_dict())
first301 = led[led.TRANTYPE == 301].groupby("NEW_AC_NUMBER").TRANSACTION_DATE.min()
ch_acc = ch.groupby("NEW_AC_NUMBER").ACTUAL_CHARGES_AMT.sum()
print("   charges-table accounts with first drawdown in ledger:", int(ch_acc.index.isin(first301.index).sum()), "of",
len(ch_acc), "| first drawdown year of those:", first301[first301.index.isin(ch_acc.index)].dt.year.value_counts().to_dict())

print("\n(k) CHART: internal consistency and convention")
chart = chart.sort_values(["NEW_AC_NUMBER", "INSTALLMENT_DATE", "INST_SR_NO"])
g = chart.groupby("NEW_AC_NUMBER")
first = g.head(1); last = g.tail(1)
drawn = -led[led.TRANTYPE == 301].groupby("NEW_AC_NUMBER").TRANSAMT.sum()
print("   first row amount 0 on", int((first.INSTALLMENT_AMT == 0).sum()), "of", len(first), "| first EXP_BALANCE = drawn on",
int((first.set_index("NEW_AC_NUMBER").EXP_BALANCE - drawn.reindex(first.NEW_AC_NUMBER).values).abs().lt(1).sum()), "| last
EXP_BALANCE = 0 on", int((last.EXP_BALANCE.abs() < 1).sum()))
print("   sum of principal parts = first EXP_BALANCE on", int(((g.PRIN_INSTALLMENT_AMT.sum() -
first.set_index('NEW_AC_NUMBER').EXP_BALANCE).abs() < 1).sum()), "| principal + interest = instalment on",
int(((chart.PRIN_INSTALLMENT_AMT + chart.INTR_INSTALLMENT_AMT - chart.INSTALLMENT_AMT).abs() < 0.02).sum()), "of", len(chart))
chart["prev_bal"] = g.EXP_BALANCE.shift(1); chart["prev_date"] = g.INSTALLMENT_DATE.shift(1)
rt = acm.set_index("NEW_AC_NUMBER").INTEREST_RATE
c2 = chart[chart.prev_bal.notna() & (chart.INTR_INSTALLMENT_AMT > 0)].copy()
c2["rate"] = c2.NEW_AC_NUMBER.map(rt); c2["days"] = (c2.INSTALLMENT_DATE - c2.prev_date).dt.days
c2["monthly"] = c2.prev_bal * c2.rate / 100 / 12; c2["daily"] = c2.prev_bal * c2.rate / 100 * c2.days / 365
for name in ("monthly", "daily"):
    r = c2.INTR_INSTALLMENT_AMT / c2[name]
    print(f"   chart interest vs {name} convention at the current rate: within 0.5% on {int(((r - 1).abs() < 0.005).sum()):,}
of {len(c2):,}")
print("   level instalment (same INSTALLMENT_AMT on all non-zero rows) on", int((chart[chart.INSTALLMENT_AMT >
0].groupby("NEW_AC_NUMBER").INSTALLMENT_AMT.nunique() == 1).sum()), "of", chart.NEW_AC_NUMBER.nunique(), "accounts")
# actual repayments vs chart due, by account, cumulative to Sep 2026
due = chart[chart.INSTALLMENT_DATE <= "2026-09-30"].groupby("NEW_AC_NUMBER").INSTALLMENT_AMT.sum()
paid = led[led.TRANTYPE.isin([305, 306])].groupby("NEW_AC_NUMBER").TRANSAMT.sum()
cmp = pd.concat([due.rename("due_to_date"), paid.rename("paid_to_date")], axis=1).dropna()
cmp["paid_over_due"] = cmp.paid_to_date / cmp.due_to_date.replace(0, np.nan)
print("   accounts with chart and repayments:", len(cmp), "| paid within 5% of due:", int(((cmp.paid_over_due - 1).abs() <
0.05).sum()), "| paid < 80% of due:", int((cmp.paid_over_due < 0.8).sum()), "| paid > 120%:", int((cmp.paid_over_due >
1.2).sum()))
print("   recovered per chart (INST_RECOVERED sum) vs paid:", int(((chart.groupby('NEW_AC_NUMBER').INST_RECOVERED.sum() -
paid.reindex(chart.NEW_AC_NUMBER.unique())).abs() < 1).sum()), "agree")

print("\n(l) EXTRACT A: Excel-converted dates against the true dates now in hand")
A = pd.read_excel(X + r"\Extract A.xlsx", dtype={"LOAN_ACCOUNT_NUMBER": str})
A = A[A["AS_OF_DATE"].astype(str).str[:10] == "31-12-2025"]
real = acm.set_index("NEW_AC_NUMBER").ACCOUNT_OPEN_DATE
n_excel = n_match = n_swap = n_text = n_textmatch = 0
for k, v in zip(A.LOAN_ACCOUNT_NUMBER, A.LOAN_START_DATE):
    if k not in real.index: continue
    if hasattr(v, "year"):
        n_excel += 1; d = pd.Timestamp(v)
        if d == real[k]: n_match += 1
    else:
        try:
            if pd.Timestamp(year=d.year, month=d.day, day=d.month) == real[k]: n_swap += 1
        except Exception: pass
    elif isinstance(v, str):
        n_text += 1; n_textmatch += int(pd.to_datetime(v, format="%d-%m-%Y", errors="coerce") == real[k])
print(f"   text dates {n_text}: match {n_textmatch} | Excel-converted dates {n_excel}: match as read {n_match}, match after
swapping day and month {n_swap}")

```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\fu_a3.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Audit 3: trial balance reconciliation, interest components, fees, chart conventions and Extract A dates

--- command ---

```

cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python fu_a3.py 2>&1 | cut -c1-300
{
  "timeout": 600000
}

```

Result of Bash

TB months: 2025-01 to 2026-08 | count 20

(g) BALANCE SHEET: TB debit balance per loan GL vs sum of ledger running balances at month-end
months x GL with TB figure: 0 | agree within 1: 0 | differ: 0

Empty DataFrame

Columns: [month, gl, tb_debit, ledger, diff]

Index: []

total loan book at 2025-12: 0.00 TB vs ledger 14,841,212,779.54

(h) INCOME: TB interest income accounts and whether they are monthly or cumulative

42xx accounts in the latest TB: {}

rows: 80 | ledger = TB balance (monthly basis): 0 | ledger = TB movement (YTD basis): 0

month	income_gl	loan_gl	tb_credit_balance	tb_movement_ytd_basis	ledger_interest	diff_vs_balance	diff_vs_movement
2025-01	4215	1050101	NaN	NaN	47120571.28	NaN	NaN
2025-01	4216	1050102	NaN	NaN	51993819.10	NaN	NaN
2025-01	4218	1050201	NaN	NaN	-0.00	NaN	NaN
2025-01	4219	1050202	NaN	NaN	23309775.88	NaN	NaN
2025-02	4215	1050101	NaN	NaN	40949129.45	NaN	NaN
2025-02	4216	1050102	NaN	NaN	58363917.03	NaN	NaN
2025-02	4218	1050201	NaN	NaN	30248160.98	NaN	NaN
2025-02	4219	1050202	NaN	NaN	20300319.35	NaN	NaN
2025-12	4215	1050101	NaN	NaN	60086643.19	NaN	NaN
2025-12	4216	1050102	NaN	NaN	151902197.02	NaN	NaN
2025-12	4218	1050201	NaN	NaN	7845936.26	NaN	NaN
2025-12	4219	1050202	NaN	NaN	27108433.21	NaN	NaN
2026-01	4215	1050101	NaN	NaN	22052068.05	NaN	NaN
2026-01	4216	1050102	NaN	NaN	167288651.29	NaN	NaN
2026-01	4218	1050201	NaN	NaN	8516780.78	NaN	NaN
2026-01	4219	1050202	NaN	NaN	31620701.01	NaN	NaN
2026-08	4215	1050101	NaN	NaN	21061455.01	NaN	NaN
2026-08	4216	1050102	NaN	NaN	123761547.70	NaN	NaN
2026-08	4218	1050201	NaN	NaN	7030827.30	NaN	NaN
2026-08	4219	1050202	NaN	NaN	31652711.33	NaN	NaN

fee income accounts latest TB: {}

(i) INTEREST SUMMARY (Aug 2026) vs ledger 303 of 31 Aug 2026

accounts: 82 | TOTAL_INTEREST = ledger: 82 | INTEREST_AMOUNT = ledger: 82 | rows with overdue interest > 0: 0

overdue interest total Aug-2026: 0.00 of total 475,983,556.79 | penal rates used: {0.0: 82}

(j) FEES: the largest fee-like 301 narrations

NEW_AC_NUMBER TRANSACTION_DATE TRANSAMT
PARTICULARS

000104460000120	2026-03-31	-570808129.0	To Trf Loan drawdown forJVD Agroprocessing Limited
000104430000089	2025-10-31	-285466250.0	To Trf Loan drawdown amount-Anchor Processor Limited
000106050000007	2026-03-25	-200000000.0	To Trf Partial Loan Drawdown -Milele Agroprocessing Limited
000106050000007	2026-06-02	-158000000.0	To Trf Milele Agroprocessing Limited- Forth Partial loan draw down
000106050000006	2026-01-19	-90000000.0	To Trf Arrangement fee on loan drawdown
000106050000007	2026-03-25	-60570181.0	To Trf Partial loan drawdown- Milelel Agroprocessing Limited
000106050000006	2026-01-19	-49259000.0	To Trf Legal fees on loan draw down
000106050000007	2026-04-14	-46317560.0	To Trf Milele Agroprocessing Ltd Partial Loan Drawdown: Payment made directly to supplier

insurance rows: 6 4,419,598.84 | arrangement: 197,537,942.00 | legal: 73,922,725.00

charges table (in scope, amount > 0): by name {'count': {'Agreement Fee (%)': 38, 'Legal Fee (%)': 37, 'Other Fee (%)': 9}, 'sum': {'Agreement Fee (%)': 205417295.0, 'Legal Fee (%)': 136217758.0, 'Other Fee (%)': 9032970.0}}

charges-table accounts with first drawdown in ledger: 27 of 38 | first drawdown year of those: {2025: 21, 2024: 3, 2026: 3}

(k) CHART: internal consistency and convention

first row amount 0 on 56 of 75 | first EXP_BALANCE = drawn on 15 | last EXP_BALANCE = 0 on 74

sum of principal parts = first EXP_BALANCE on 26 | principal + interest = instalment on 2240 of 2742

chart interest vs monthly convention at the current rate: within 0.5% on 1,094 of 2,635

chart interest vs daily convention at the current rate: within 0.5% on 28 of 2,635

level instalment (same INSTALLMENT_AMT on all non-zero rows) on 30 of 75 accounts

accounts with chart and repayments: 69 | paid within 5% of due: 23 | paid < 80% of due: 21 | paid > 120%: 8

recovered per chart (INST_RECOVERED sum) vs paid: 0 agree

(l) EXTRACT A: Excel-converted dates against the true dates now in hand

text dates 108: match 108 | Excel-converted dates 73: match as read 2, match after swapping day and month 71

Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

Tool call: Bash

description: Debug the trial balance row parsing

```
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python - <<'PYEOF' 2>&1 | cut -c1-200
import pandas as pd, re, glob
T = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Trial Balances"
f = glob.glob(T + r"\Trial Balance_31 January 2025.xls")[0]
df = pd.read_excel(f, header=None)
print(df.shape, list(df.columns))
for i in (31, 122):
    print([repr(v) for v in df.iloc[i].tolist()])
PYEOF
{
"timeout": 60000
}
```

Result of Bash

```
(177, 4) [0, 1, 2, 3]
['np.float64(nan)', '1050101...MAIIC Agricultural Loans', '1,642,334,523.43', '0.00']
['np.float64(nan)', '4219...Interest On FInES Industrial Loans', '0.00', '23,309,775.88']
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

description: Fix the trial balance parsing and the fee narration pattern, then rerun those sections

```
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && sed -i 's/ t = str(r\[1\])$/ t = str(r[1]).strip()/; s/fee|charge|legal|arrang|insur|commit|process/\\\\bfees?\\\\b|charge|legal|arrang|insur|commit/' fu_a3.py && grep -n 'str(r\[1\])\\|fees?' fu_a3.py | head -3 && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python fu_a3.py 2>&1 | sed -n '/^(g)/,/(i)/p;/(j)/,/(k)/p' | cut -c1-300
{
"timeout": 600000
}
```

Result of Bash

```
23:         t = str(r[1]).strip()
72:fee301 = led[(led.TRANSTYPE == 301) & led.PARTICULARS.str.contains("\bfees?\b|charge|legal|arrang|insur|commit", case=False,
na=False)]
(g) BALANCE SHEET: TB debit balance per loan GL vs sum of ledger running balances at month-end
    months x GL with TB figure: 90 | agree within 1: 50 | differ: 40
    month    gl      tb_debit      ledger      diff
2025-01 1050201 1.857623e+09 1.857223e+09 400000.0
2025-01 1050202 2.667566e+09 2.668566e+09 -100000.0
2025-02 1050201 1.847789e+09 1.847389e+09 400000.0
2025-02 1050202 2.631092e+09 2.632092e+09 -100000.0
2025-03 1050201 1.828550e+09 1.828150e+09 400000.0
2025-03 1050202 2.600664e+09 2.601664e+09 -100000.0
2025-04 1050201 1.797985e+09 1.797585e+09 400000.0
2025-04 1050202 2.560297e+09 2.561297e+09 -100000.0
2025-05 1050201 1.771494e+09 1.771094e+09 400000.0
2025-05 1050202 3.089874e+09 3.090874e+09 -100000.0
2025-06 1050201 1.607054e+09 1.606654e+09 400000.0
2025-06 1050202 3.043024e+09 3.044024e+09 -100000.0
2025-07 1050201 1.574152e+09 1.573752e+09 400000.0
2025-07 1050202 3.063257e+09 3.064257e+09 -100000.0
2025-08 1050201 1.568759e+09 1.568359e+09 400000.0
2025-08 1050202 3.047445e+09 3.048445e+09 -100000.0
2025-09 1050201 1.551709e+09 1.551309e+09 400000.0
2025-09 1050202 3.028701e+09 3.029701e+09 -100000.0
2025-10 1050201 1.468126e+09 1.467726e+09 400000.0
2025-10 1050202 2.935598e+09 2.936598e+09 -100000.0
2025-11 1050201 1.209281e+09 1.208881e+09 400000.0
2025-11 1050202 2.886701e+09 2.887701e+09 -100000.0
2025-12 1050201 1.402393e+09 1.401993e+09 400000.0
2025-12 1050202 3.808289e+09 3.809289e+09 -100000.0
2026-01 1050201 1.381902e+09 1.381502e+09 400000.0
2026-01 1050202 3.989620e+09 3.990620e+09 -100000.0
2026-02 1050201 1.364280e+09 1.363880e+09 400000.0
2026-02 1050202 3.966645e+09 3.967645e+09 -100000.0
2026-03 1050201 1.276554e+09 1.276154e+09 400000.0
2026-03 1050202 4.194843e+09 4.195843e+09 -100000.0
2026-04 1050201 1.250932e+09 1.250532e+09 400000.0
2026-04 1050202 4.127854e+09 4.128854e+09 -100000.0
2026-05 1050201 1.322416e+09 1.322016e+09 400000.0
2026-05 1050202 4.073961e+09 4.074961e+09 -100000.0
2026-06 1050201 1.286033e+09 1.285633e+09 400000.0
2026-06 1050202 3.939997e+09 3.940997e+09 -100000.0
2026-07 1050201 1.267396e+09 1.266996e+09 400000.0
2026-07 1050202 3.841888e+09 3.842888e+09 -100000.0
2026-08 1050201 1.579269e+09 1.578869e+09 400000.0
2026-08 1050202 3.825301e+09 3.826301e+09 -100000.0
    total loan book at 2025-12: 14,840,612,779.54 TB vs ledger 14,841,212,779.54

(h) INCOME: TB interest income accounts and whether they are monthly or cumulative
    42xx accounts in the latest TB: {'4205': 'Investment Interest Income -TBill', '4211': 'Investment Income USD CALL ACCOUNT',
'4218': 'Interest On FInES Agricultural Loans', '4219': 'Interest On FInES Industrial Loans', '4207': 'Interest On Staff Loans',
'4215': 'Interest On MAIIC Agricultural Loan
    rows: 100 | ledger = TB balance (monthly basis): 9 | ledger = TB movement (YTD basis): 83
    month income_gl loan_gl tb_credit_balance tb_movement_ytd_basis ledger_interest diff_vs_balance diff_vs_movement
2025-01 4215 1050101 4.712057e+07 47120571.28 47120571.28 0.000000e+00 0.0
2025-01 4216 1050102 5.199382e+07 51993819.10 51993819.10 -0.000000e+00 -0.0
2025-01 4218 1050201 NaN NaN -0.00 NaN NaN
2025-01 4219 1050202 2.330978e+07 23309775.88 23309775.88 0.000000e+00 0.0
2025-01 42019 1050401 NaN NaN -0.00 NaN NaN
2025-02 4215 1050101 8.806970e+07 40949129.45 40949129.45 4.712057e+07 0.0
2025-02 4216 1050102 1.103577e+08 58363917.03 58363917.03 5.199382e+07 -0.0
2025-02 4218 1050201 3.024816e+07 30248160.98 30248160.98 0.000000e+00 0.0
2025-02 4219 1050202 4.361010e+07 20300319.35 20300319.35 2.330978e+07 0.0
2025-02 42019 1050401 NaN NaN -0.00 NaN NaN
2025-12 4215 1050101 NaN NaN 60086643.19 NaN NaN
2025-12 4216 1050102 NaN NaN 151902197.02 NaN NaN
2025-12 4218 1050201 NaN NaN 7845936.26 NaN NaN
2025-12 4219 1050202 NaN NaN 27108433.21 NaN NaN
2025-12 42019 1050401 NaN NaN 39093897.99 NaN NaN
2026-01 4215 1050101 2.205207e+07 22052068.05 22052068.05 0.000000e+00 0.0
2026-01 4216 1050102 1.672887e+08 167288651.29 167288651.29 0.000000e+00 0.0
2026-01 4218 1050201 8.516781e+06 8516780.78 8516780.78 -0.000000e+00 -0.0
2026-01 4219 1050202 3.162070e+07 31620701.01 31620701.01 0.000000e+00 0.0
2026-01 42019 1050401 2.846468e+07 28464681.12 28464681.12 0.000000e+00 0.0
2026-08 4215 1050101 1.648335e+08 21061455.01 21061455.01 1.437721e+08 -0.0
2026-08 4216 1050102 1.140382e+09 123761547.70 123761547.70 1.016620e+09 0.0
2026-08 4218 1050201 5.834259e+07 7030827.30 7030827.30 5.131176e+07 -0.0
2026-08 4219 1050202 2.538596e+08 31652711.33 31652711.33 2.222069e+08 0.0
2026-08 42019 1050401 1.598164e+09 292477015.45 292477015.45 1.305687e+09 0.0
    fee income accounts latest TB: {'4871': ('Legal Fees', 0.0, 243389041.0), '4872': ('Consultancy Fees', 317250.0, 0.0),
'4873': ('Arrangement Fees', 0.0, 414947879.0)}
```

(i) INTEREST SUMMARY (Aug 2026) vs ledger 303 of 31 Aug 2026

(j) FEES: the largest fee-like 301 narrations

NEW_AC_NUMBER	TRANSACTION_DATE	TRANSAMT	PARTICULARS
000106050000006	2026-01-19	-90000000.0	To Trf Arrangement fee on loan drawdown
000106050000006	2026-01-19	-49259000.0	To Trf Legal fees on loan draw down
000104430000082	2025-07-31	-30000000.0	To Trf Pinnacle Financial services Limited-Arrangement fees
000104430000085	2025-08-08	-30000000.0	To Trf Arrangement fees- Mphunzitsi SACCO

```
000104430000084      2025-08-08  -9000000.0              To Trf arrangement fee-Ebenezer
000106050000005      2025-12-23  -9000000.0              To Trf Arrangement fees on loan drawdown
000104430000086      2025-08-27  -7500000.0              To Trf Being arrangement fees
000104420000064      2025-09-05  -7500000.0              To Trf Arrangement fee on loan
insurance rows: 6 4,419,598.84 | arrangement: 197,537,942.00 | legal: 73,922,725.00
charges table (in scope, amount > 0): by name {'count': {'Agreement Fee (%)': 38, 'Legal Fee (%)': 37, 'Other Fee (%)': 9},
'sum': {'Agreement Fee (%)': 205417295.0, 'Legal Fee (%)': 136217758.0, 'Other Fee (%)': 9032970.0}}
charges-table accounts with first drawdown in ledger: 27 of 38 | first drawdown year of those: {2025: 21, 2024: 3, 2026: 3}

(k) CHART: internal consistency and convention
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\fu_a4.py
--- content ---
"""Audit 4: the income rows that did not tie, pre- and post-migration coverage, status codes, rate history depth, plans."""
import pandas as pd, numpy as np
from fu_load import load
pd.set_option("display.width", 250); pd.set_option("display.max_columns", 40); pd.set_option("display.max_colwidth", 60)
led, acm, alm, rate, chart, plan, status, lb, disb = (load(k) for k in ("ledger", "acm", "alm", "rate", "chart", "plan",
"status", "lb", "disb"))
led["TRANTYPE"] = led.TRANTYPE.astype(int)
ic = pd.read_csv("tb_income_recon.csv", dtype={"income_gl": str, "loan_gl": str})
print("\n(h2) income rows that did not tie on the YTD basis:")
print(ic[~(ic.diff_vs_movement.abs() < 1)][["month", "income_gl", "loan_gl", "tb_credit_balance", "tb_movement_ytd_basis",
"ledger_interest", "diff_vs_movement"]].round(2).to_string(index=False))
gb = pd.read_csv("tb_balance_recon.csv", dtype={"gl": str})
print("    balance differences by GL (distinct values):", gb[gb["diff"].abs() >= 1].groupby("gl")["diff"].unique().to_dict())

print("\n(m) PRE- AND POST-MIGRATION")
first301 = led[led.TRANTYPE == 301].groupby("NEW_AC_NUMBER").TRANSACTION_DATE.min()
a = acm.set_index("NEW_AC_NUMBER")[["ACCOUNT_OPEN_DATE", "STATUS_CODE", "SCHEME_MST_ID"]].copy()
a["first_301"] = first301; a["pre"] = a.ACCOUNT_OPEN_DATE < "2024-07-01"
a["takeon"] = a.first_301.dt.strftime("%Y-%m-%d").eq("2024-07-31")
print("    pre-migration accounts:", int(a.pre.sum()), "| of which first ledger posting is the 31-Jul-2024 take-on:", int((a.pre
& a.takeon).sum()), "| with no ledger rows:", int((a.pre & a.first_301.isna()).sum()), "| status of pre-migration:",
a[a.pre].STATUS_CODE.value_counts().to_dict())
print("    post-migration accounts:", int(~a.pre.sum()), "| with 301 postings:", int(~a.pre & a.first_301.notna()).sum()), "|
status:", a[~a.pre].STATUS_CODE.value_counts().to_dict())
r = rate[rate.DELETE_FLAG == "N"]
hist = r.groupby("NEW_AC_NUMBER").agg(rows=("APPLICABLE_FROM_DATE", "size"), first=("APPLICABLE_FROM_DATE", "min"),
last=("APPLICABLE_FROM_DATE", "max"))
pre_hist = hist[hist.index.isin(a[a.pre].index)]
print("    rate history for pre-migration accounts:", len(pre_hist), "accounts | with a rate dated before Jul-2024:",
int((pre_hist["first"] < "2024-07-01").sum()), "| median rows:", pre_hist.rows.median(), "| MAIIC (policy P) accounts with >5
rate rows:", int((hist.rows > 5).sum()))
takeon = led[(led.TRANTYPE == 301) & (led.TRANSACTION_DATE == "2024-07-31")]
print("    take-on postings 31-Jul-2024:", len(takeon), "accounts", takeon.NEW_AC_NUMBER.nunique(), "| total",
f"(-takeon.TRANSAMT.sum():.2f)", "| narration sample:", takeon.PARTICULARS.str[:45].head(3).tolist())
t300 = led[led.TRANTYPE == 300]; print("    type 300 rows:", t300[["NEW_AC_NUMBER", "TRANSACTION_DATE", "TRANSAMT",
"PARTICULARS"]].values.tolist())
t343 = led[led.TRANTYPE == 343]; print("    type 343 rows:", t343[["NEW_AC_NUMBER", "TRANSACTION_DATE", "TRANSAMT",
"PARTICULARS"]].values.tolist())
t120 = led[led.TRANTYPE == 120]; print("    type 120 narrations:", t120.PARTICULARS.str[:60].tolist()[:3])

print("\n(n) STATUS: codes in status history:", status.STATUS_CODE.value_counts().to_dict())
cur = acm.set_index("NEW_AC_NUMBER").STATUS_CODE
h = status[status.NEW_AC_NUMBER.isin(cur[cur == "H"].index)].sort_values(["NEW_AC_NUMBER", "STATUS_EFF_DATE"])
print("    H accounts: latest status-history row per account, code and reason:")
print(h.groupby("NEW_AC_NUMBER").tail(1).groupby(["STATUS_CODE", "STATUS_CHANGE_REASON"]).size().to_string())
print("    H accounts with a balance in the ledger at Sep-2026:", end=" ")
led = led.sort_values(["NEW_AC_NUMBER", "TRANSACTION_DATE", "CUMVOUCH_DET_ID"]); lastbal =
led.groupby("NEW_AC_NUMBER").TRANSAMT.sum()
print(int((lastbal.reindex(cur[cur == "H"].index).fillna(0).abs() > 1).sum()), "of", int((cur == "H").sum()), "| F:",
int((lastbal.reindex(cur[cur == "F"].index).fillna(0).abs() > 1).sum()), "of", int((cur == "F").sum()), "| D:",
int((lastbal.reindex(cur[cur == "D"].index).fillna(0).abs() > 1).sum()), "of", int((cur == "D").sum()))
print("    INTEREST_STOP_DATE set on", int(acm.INTEREST_STOP_DATE.notna().sum()), "accounts; by status:",
acm[acm.INTEREST_STOP_DATE.notna()].STATUS_CODE.value_counts().to_dict())

print("\n(o) TERMS NEEDED TO GENERATE A SCHEDULE, per account")
alm2 = alm.set_index("NEW_AC_NUMBER")
cols = {"SANCTION_AMOUNT": alm2.SANCTION_AMOUNT.notna() & (alm2.SANCTION_AMOUNT > 0), "SANCTION_DATE":
alm2.SANCTION_DATE.notna(), "EXPIRY_DATE": alm2.EXPIRY_DATE.notna(),
"tenor (months)": (alm2.PERIOD_YEARS.fillna(0) * 12 + alm2.PERIOD_MONTHS.fillna(0)) > 0, "INTEREST_RATE (account)":
acm.set_index("NEW_AC_NUMBER").INTEREST_RATE.notna(),
"PMOROTORIUM_PERIOD": alm2.PMOROTORIUM_PERIOD.notna(), "instalment plan row": alm2.index.isin(plan[plan.DELETE_FLAG ==
"N"].NEW_AC_NUMBER), "current chart": alm2.index.isin(chart.NEW_AC_NUMBER)}
for k, v in cols.items(): print(f"    {k:<26} {int(v.sum())} of {len(alm2)}")
p = plan[plan.DELETE_FLAG == "N"]
print("    plan INSTALLMENT_TYPE:", p.INSTALLMENT_TYPE.value_counts().to_dict(), "| PAY_MODE:",
p.INSTALLMENT_PAY_MODE.value_counts().to_dict(), "| rows per account:", p.groupby("NEW_AC_NUMBER").size().describe()[["min",
"50%", "max"]].to_dict())
print("    MORT_PERIOD_SEP_INT values:", alm.MORT_PERIOD_SEP_INT.value_counts().to_dict())
print("    moratorium period (P) distribution:", alm.PMOROTORIUM_PERIOD.value_counts().head(6).to_dict(), "| I:",
alm.IMOROTORIUM_PERIOD.value_counts().head(6).to_dict())
print("    active accounts with no current chart, by scheme:", acm[(acm.STATUS_CODE == "A") &
~acm.NEW_AC_NUMBER.isin(chart.NEW_AC_NUMBER)].SCHEME_MST_ID.value_counts().to_dict())
lbx = lb.sort_values("TRANSACTION_DATE").drop_duplicates(["ASONDATE", "NEW_AC_NUMBER", keep="last"])
print("    loan book MORAT_MONTHS values:", lbx.MORAT_MONTHS.value_counts().head(8).to_dict())
print("    loan book TENOR_YRS sample:", lbx.TENOR_YRS.value_counts().head(6).to_dict())
```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\fu_a4.py (file state is current in your context – no need to Read it back)

Tool call: Bash

```
description: Audit 4: income rows not tying, pre-migration coverage, status codes and schedule terms
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python fu_a4.py 2>&1 | cut -c1-320
{
  "timeout": 300000
}
```

Result of Bash

(h2) income rows that did not tie on the YTD basis:

month	income_gl	loan_gl	tb_credit_balance	tb_movement_ytd_basis	ledger_interest	diff_vs_movement
2025-01	4218	1050201	NaN	NaN	-0.00	NaN
2025-01	42019	1050401	NaN	NaN	-0.00	NaN
2025-02	42019	1050401	NaN	NaN	-0.00	NaN
2025-03	42019	1050401	NaN	NaN	-0.00	NaN
2025-04	42019	1050401	NaN	NaN	-0.00	NaN
2025-05	42019	1050401	NaN	NaN	-0.00	NaN
2025-06	42019	1050401	NaN	NaN	-0.00	NaN
2025-07	42019	1050401	NaN	NaN	-0.00	NaN
2025-08	42019	1050401	NaN	NaN	-0.00	NaN
2025-09	42019	1050401	NaN	NaN	-0.00	NaN
2025-10	42019	1050401	NaN	NaN	-0.00	NaN
2025-11	42019	1050401	NaN	NaN	-0.00	NaN
2025-12	4215	1050101	NaN	NaN	60086643.19	NaN
2025-12	4216	1050102	NaN	NaN	151902197.02	NaN
2025-12	4218	1050201	NaN	NaN	7845936.26	NaN
2025-12	4219	1050202	NaN	NaN	27108433.21	NaN
2025-12	42019	1050401	NaN	NaN	39093897.99	NaN

balance differences by GL (distinct values): {'1050201': array([400000. , 400000.00000024, 399999.99999976]), '1050202': array([-100000. , -999999.99999952, -1000000.00000048])}

(m) PRE- AND POST-MIGRATION

pre-migration accounts: 109 | of which first ledger posting is the 31-Jul-2024 take-on: 77 | with no ledger rows: 32 | status of pre-migration: {'A': 34, 'C': 31, 'H': 21, 'D': 20, 'F': 3}

post-migration accounts: 75 | with 301 postings: 54 | status: {'A': 55, 'C': 15, 'D': 5}

rate history for pre-migration accounts: 108 accounts | with a rate dated before Jul-2024: 106 | median rows: 1.0 | MAIIC (policy P) accounts with >5 rate rows: 40

take-on postings 31-Jul-2024: 78 accounts 78 | total 8,297,388,309.25 | narration sample: ['To Trf Opening Account Balance Disbursement amo', 'To Trf Opening Account Balance Disbursement amo', 'To Trf Opening Account Balance Disbursement amo']

type 300 rows: [['000104420000003', Timestamp('2024-08-31 00:00:00'), 10244624.69, 'By Trf Micholess Creameries Write off'], ['000104450000012', Timestamp('2024-08-31 00:00:00'), 12041809.63, 'By Trf Maluso Cooperative Write off']]

type 343 rows: [['000104460000117', Timestamp('2026-03-12 00:00:00'), 9009942.39, 'By Trf Reversal of credit transactions to account 000104460000117'], ['000106050000003', Timestamp('2026-03-12 00:00:00'), 65444499.06, 'By Trf Reversal of credit transactions to account 000106050000003']]

type 120 narrations: ['By Trf Diff Int. Credit From 01/01/2025 To 31/12/2025 By ROI', 'By Trf Diff Int. Credit From 01/01/2025 To 31/12/2025 By ROI', 'By Trf Diff Int. Credit From 01/01/2025 To 31/12/2025 By ROI']

(n) STATUS: codes in status history: {'A': 185, 'D': 48, 'C': 47, 'F': 3}

H accounts: latest status-history row per account, code and reason:

STATUS_CODE	STATUS_CHANGE_REASON
D	Account closure 1
	Foreclosure 3
	PAID UP 13
	Paid Up 4

H accounts with a balance in the ledger at Sep-2026: 3 of 21 | F: 0 of 3 | D: 23 of 25

INTEREST_STOP_DATE set on 40 accounts; by status: {'D': 20, 'H': 17, 'F': 2, 'A': 1}

(o) TERMS NEEDED TO GENERATE A SCHEDULE, per account

SANCTION_AMOUNT	183 of 184
SANCTION_DATE	183 of 184
EXPIRY_DATE	183 of 184
tenor (months)	183 of 184
INTEREST_RATE (account)	184 of 184
PMOROTORIUM_PERIOD	184 of 184
instalment plan row	159 of 184
current chart	75 of 184

plan INSTALLMENT_TYPE: {'I': 502, 'O': 235, 'D': 18} | PAY_MODE: {'M': 672, 'H': 49, 'Q': 25, 'Y': 9} | rows per account: {'min': 1.0, '50%': 2.0, 'max': 37.0}

MORT_PERIOD_SEP_INT values: {'Y': 66, 'N': 60, '': 58}

moratorium period (P) distribution: {0: 96, 3: 46, 6: 27, 12: 8, 9: 2, 4: 2} | I: {0: 126, 3: 42, 6: 10, 12: 3, 8: 1, 9: 1}

active accounts with no current chart, by scheme: {92: 31, 143: 7, 86: 3, 141: 3, 140: 2, 142: 2, 144: 2, 88: 1, 90: 1, 94: 1}

loan book MORAT_MONTHS values: {0.0: 861, 3.0: 591, 6.0: 445, 12.0: 139, 4.0: 42, 18.0: 21, 9.0: 14, 13.0: 14}

loan book TENOR_YRS sample: {2: 941, 4: 517, 5: 395, 3: 305, 1: 123, 6: 68}

Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\fu_a5.py
--- content ---
"""Audit 5: rebuild every interest posting from daily balances (the way E-Banker's product method works), and the Dec-2025
TB."""
import re, glob
import pandas as pd, numpy as np
from fu_load import load
pd.set_option("display.width", 250); pd.set_option("display.max_colwidth", 60)
led = load("ledger"); led["TRANSTYPE"] = led.TRANSTYPE.astype(int)
key = ["NEW_AC_NUMBER", "SUB_ACCOUNT_NO"]
led = led.sort_values(key + ["TRANSACTION_DATE", "CUMVOUCH_DET_ID"])
pat = re.compile(r"Fm (\d{2}-[A-Za-z]{3}-\d{2}) To (\d{2}-[A-Za-z]{3}-\d{2}) @ ?([\d.]+)%", re.I)
res = []
for acct, g in led.groupby("NEW_AC_NUMBER"):
    # daily closing balance from all postings on the loan's own GL (interest capitalised included)
    daily = g.groupby("TRANSACTION_DATE").TRANSAMT.sum()
    idx = pd.date_range(daily.index.min(), "2026-09-30")
    close = daily.reindex(idx).fillna(0).cumsum()          # balance at end of each day
    for _, r in g[g.TRANSTYPE == 303].iterrows():
        m = pat.search(str(r.PARTICULARS))
        if not m: res.append((acct, r.TRANSACTION_DATE, r.TRANSAMT, np.nan, np.nan, np.nan, "no narration")); continue
        d0 = pd.to_datetime(m.group(1), format="%d-%b-%y"); d1 = pd.to_datetime(m.group(2), format="%d-%b-%y"); rate =
float(m.group(3))
        # balance before this interest posting on each day: the day's closing balance, with the interest row itself excluded on
its own day
        win = close.loc[d0:d1].copy()
        if r.TRANSACTION_DATE in win.index: win.loc[r.TRANSACTION_DATE:] -= r.TRANSAMT
        # same-day interest postings by other rows on the posting date are rare; ignore
        prod_close = -win.sum()          # closing-balance convention
        open_ = close.shift(1).reindex(win.index).fillna(0.0)
        if r.TRANSACTION_DATE in open_.index: open_.loc[r.TRANSACTION_DATE + pd.Timedelta(days=1):] -= r.TRANSAMT
        prod_open = -open_.sum()          # opening-balance convention
        exp_c = prod_close * rate / 100 / 365; exp_o = prod_open * rate / 100 / 365
        res.append((acct, r.TRANSACTION_DATE, -r.TRANSAMT, exp_c, exp_o, rate, "ok"))
df = pd.DataFrame(res, columns=["acct", "date", "posted", "exp_close", "exp_open", "rate", "note"])
ok = df[df.note == "ok"].copy()
for c in ("exp_close", "exp_open"):
    r = ok.posted / ok[c].replace(0, np.nan)
    print(f"{c}: within 0.1% on {int(((r - 1).abs() < 0.001).sum())}, | within 1% on {int(((r - 1).abs() < 0.01).sum())}, |
of {len(ok)}, postings with a narration")
ok["best"] = np.minimum((ok.posted / ok.exp_close.replace(0, np.nan) - 1).abs(), (ok.posted / ok.exp_open.replace(0, np.nan) -
1).abs())
print("either convention within 0.1%:", int((ok.best < 0.001).sum()), "| within 1%:", int((ok.best < 0.01).sum()), "| amount
within 1% of total posted:", f"{ok[ok.best < 0.01].posted.sum() / ok.posted.sum():.1%}")
far = ok[ok.best >= 0.01]
print("postings not within 1%:", len(far), "on", far.acct.nunique(), "accounts | by year:",
far.date.dt.year.value_counts().to_dict())
print("    their share of all interest posted:", f"{far.posted.sum() / ok.posted.sum():.1%}")
print(far.sort_values("posted", ascending=False)[["acct", "date", "posted", "exp_close", "exp_open",
"rate"]].head(8).round(2).to_string(index=False))
print("no-narration postings:", int((df.note != "ok").sum()), "| total", f"{-df[df.note != 'ok'].posted.sum():.2f}" if
(df.note != "ok").any() else "")
df.to_csv("interest_rebuild.csv", index=False)

# Dec 2025 TB
T = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Trial Balances"
x = pd.read_excel(T + r"\Trial Balance_31 December 2025.xls", header=None)
print("\nDec-2025 TB shape", x.shape); print(x.head(6).to_string())
m = x.astype(str).apply(lambda r: r.str.contains("1050101|4215|4216|42019", regex=True).any(), axis=1)
print(x[m].to_string())
```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\fu_a5.py (file state is current in your context – no need to Read it back)

Tool call: Bash

```
description: Rebuild every interest posting from daily balances and check the December 2025 trial balance
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python fu_a5.py 2>&1 | cut -c1-260
{
  "timeout": 600000
}
```

Result of Bash

```
C:\Users\wadza\AppData\Local\Temp\claude\c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\
fu_a5.py:40: UserWarning: obj.round has no effect with datetime, timedelta, or period dtypes. Use obj.dt.round(...) instead.
    print(far.sort_values("posted", ascending=False)[["acct", "date", "posted", "exp_close", "exp_open",
"rate"]].head(8).round(2).to_string(index=False))
exp_close: within 0.1% on 1,574 | within 1% on 1,823 | of 2,047 postings with a narration
exp_open: within 0.1% on 1,015 | within 1% on 1,692 | of 2,047 postings with a narration
either convention within 0.1%: 1595 | within 1%: 1828 | amount within 1% of total posted: 91.6%
postings not within 1%: 219 on 93 accounts | by year: {2025: 90, 2024: 76, 2026: 53}
    their share of all interest posted: 8.4%
      acct      date      posted      exp_close      exp_open      rate
000104430000071 2025-12-31 26499597.74 27412057.32 27466035.75 30.2
000104430000071 2026-03-31 24108764.97 23695693.95 23751011.79 27.3
000104430000071 2026-04-30 21867978.48 21037459.22 21063497.10 25.7
000104430000068 2025-07-31 18953401.20 7521650.11 7530039.13 33.2
000104430000062 2026-03-31 17326644.46 17063524.48 17085841.19 30.4
000104430000062 2026-04-30 16110667.49 15563077.81 15563077.81 28.8
000104430000082 2026-03-31 13433727.39 13216787.09 13216787.09 28.3
000104430000085 2026-03-31 13154740.51 12955468.11 12955468.11 30.2
no-narration postings: 112 | total 1,837,859,679.99

Dec-2025 TB shape (128, 4)
      0      1      2      3
0 NaN      2025-12-01 00:00:00      NaN      NaN
1 NaN      GL Title      Debit      Credit
2 NaN      1017...NBS Cash Collection 339,514,040.32      0.00
3 NaN      1018...NBS Current A/C 21011223 193,081,999.20      0.00
4 NaN      1046...First Capital Bank 28,486,995.05      0.00
5 NaN      1001...CDH Inv Bank 0050401042201      0.00 68,950,245.91
      0      1      2      3
33 NaN      1050101...MAIIC Agricultural Loans 1,368,737,810.68 0.00
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Bash

```
description: Look at the interest postings without the standard narration
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp\htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python - <<'PYEOF' 2>&1 | cut -c1-240
import pandas as pd, re
from fu_load import load
led = load("ledger"); led["TRANSTYPE"] = led.TRANSTYPE.astype(int)
pat = re.compile(r"Fm (\d{2}-[A-Za-z]{3}-\d{2}) To (\d{2}-[A-Za-z]{3}-\d{2}) @ ?([\d.]+)%", re.I)
i3 = led[led.TRANSTYPE == 303]
nn = i3[~i3.PARTICULARS.str.contains(pat)]
print("303 without the standard narration:", len(nn), "| by date:",
nn.TRANSACTION_DATE.dt.strftime("%Y-%m").value_counts().head(5).to_dict())
print("samples:", nn.PARTICULARS.str[:70].value_counts().head(6).to_dict())
print("amount:", f"{-nn.TRANSAMT.sum():.2f}")
d = pd.read_csv("interest_rebuild.csv")
print("rebuild rows:", len(d), "| posted total", f"{d.posted.sum():.2f}")
PYEOF
{
  "timeout": 120000
}
```

Result of Bash

```
<stdin>:6: UserWarning: This pattern is interpreted as a regular expression, and has match groups. To actually get the groups,
use str.extract.
303 without the standard narration: 112 | by date: {'2024-07': 78, '2025-12': 22, '2025-01': 2, '2022-09': 1, '2025-05': 1}
samples: {'To Trf Opening interest charge': 79, 'To Trf Diff Int. Debit From 01/01/2025 To 31/12/2025 By ROI 31.3 %': 2, 'To
Trf Diff Int. Debit From 01/01/2025 To 31/12/2025 By ROI 33.3 %': 2, 'To Trf Diff Int. Debit From 01/01/2025 To 30/
amount: 1,837,859,679.99
rebuild rows: 2159 | posted total 4,526,723,541.58
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\fu_report.py
--- content ---
""""Audit report on the 18 follow-up extracts received from MAIIC on 7 October 2026 (PDF, reportlab).""""
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.lib.enums import TA_LEFT
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import (BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Table, TableStyle,
                                PageBreak, KeepTogether, Image, ListFlowable, ListItem, CondPageBreak)

F = r"C:\Windows\Fonts"
pdfmetrics.registerFont(TTFont("Seg", F + r"\segoeui.ttf")); pdfmetrics.registerFont(TTFont("Seg-B", F + r"\segoeui.ttf"))
pdfmetrics.registerFont(TTFont("Seg-I", F + r"\segoeuii.ttf")); pdfmetrics.registerFont(TTFont("Seg-BI", F + r"\segoeuii.ttf"))
pdfmetrics.registerFontFamily("Seg", normal="Seg", bold="Seg-B", italic="Seg-I", boldItalic="Seg-BI")
pdfmetrics.registerFont(TTFont("Mono", F + r"\consola.ttf"))
NAVY = colors.HexColor("#1B2A41"); ORANGE = colors.HexColor("#E07B00"); GREY = colors.HexColor("#5B6470")
LIGHT = colors.HexColor("#F4F6F9"); LINE = colors.HexColor("#C9D1DB"); AMBER = colors.HexColor("#FFF4E0"); GREEN =
colors.HexColor("#EAF6EC")
S = {
    "title": ParagraphStyle("title", fontName="Seg-B", fontSize=24, leading=29, textColor=NAVY, spaceAfter=4),
    "sub": ParagraphStyle("sub", fontName="Seg", fontSize=13, leading=17, textColor=GREY, spaceAfter=10),
    "meta": ParagraphStyle("meta", fontName="Seg", fontSize=9.5, leading=13, textColor=GREY),
    "h1": ParagraphStyle("h1", fontName="Seg-B", fontSize=15, leading=19, textColor=NAVY, spaceBefore=14, spaceAfter=6,
keepWithNext=1),
    "h2": ParagraphStyle("h2", fontName="Seg-B", fontSize=11.5, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4,
keepWithNext=1),
    "h2t": ParagraphStyle("h2t", fontName="Seg-B", fontSize=11.5, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4),
    "p": ParagraphStyle("p", fontName="Seg", fontSize=10, leading=14.2, spaceAfter=6, alignment=TA_LEFT),
    "li": ParagraphStyle("li", fontName="Seg", fontSize=10, leading=14),
    "cell": ParagraphStyle("cell", fontName="Seg", fontSize=8.6, leading=11.4),
    "cellb": ParagraphStyle("cellb", fontName="Seg-B", fontSize=8.6, leading=11.4),
    "head": ParagraphStyle("head", fontName="Seg-B", fontSize=8.8, leading=11.4, textColor=colors.white),
    "box": ParagraphStyle("box", fontName="Seg", fontSize=10, leading=14.2, spaceAfter=3),
    "note": ParagraphStyle("note", fontName="Seg-I", fontSize=8.8, leading=12, textColor=GREY, spaceAfter=6),
}
W = A4[0] - 36 * mm
story = []
def H1(t): story.append(Paragraph(t, S["h1"]))
def H2(t): story.append(Paragraph(t, S["h2"]))
def H2T(t): story.append(CondPageBreak(62 * mm)); story.append(Paragraph(t, S["h2t"]))
def P(t): story.append(Paragraph(t, S["p"]))
def NOTE(t): story.append(Paragraph(t, S["note"]))
def GAP(h=4): story.append(Spacer(1, h))
def M(t): return f'<font name="Mono" size="8">{t}</font>'
def BUL(items, numbered=False):
    story.append(ListFlowable([ListItem(Paragraph(i, S["li"]), leftIndent=14, spaceAfter=3) for i in items],
                              bulletType="1" if numbered else "bullet", bulletFontName="Seg", bulletFontSize=9,
                              start=None if numbered else "*", leftIndent=14, bulletColor=ORANGE if not numbered else NAVY));
GAP(4)
def TAB(header, rows, widths, bold_first=False, keep=False):
    data = [[Paragraph(h, S["head"]) for h in header]]
    for r in rows:
        data.append([Paragraph(str(c), S["cellb"]) if (bold_first and i == 0) else S["cell"]) for i, c in enumerate(r)]
    t = Table(data, colWidths=[W * x for x in widths], repeatRows=1)
    st = [("BACKGROUND", (0, 0), (-1, 0), NAVY), ("VALIGN", (0, 0), (-1, -1), "TOP"), ("LEFTPADDING", (0, 0), (-1, -1), 5),
("RIGHTPADDING", (0, 0), (-1, -1), 5),
        ("TOPPADDING", (0, 0), (-1, -1), 4), ("BOTTOMPADDING", (0, 0), (-1, -1), 5), ("LINEBELOW", (0, 0), (-1, -1), 0.4,
LINE), ("BOX", (0, 0), (-1, -1), 0.6, LINE)]
    for i in range(1, len(data)):
        if i % 2 == 0: st.append(("BACKGROUND", (0, i), (-1, i), LIGHT))
    t.setStyle(TableStyle(st)); story.append(KeepTogether([t] if keep else t); GAP(8)
def BOX(paras, colour=AMBER):
    t = Table([[[Paragraph(x, S["box"]) for x in paras]]], colWidths=[W])
    t.setStyle(TableStyle([("BACKGROUND", (0, 0), (-1, -1), colour), ("LINEBEFORE", (0, 0), (0, -1), 3, ORANGE), ("LEFTPADDING",
(0, 0), (-1, -1), 10),
        ("RIGHTPADDING", (0, 0), (-1, -1), 10), ("TOPPADDING", (0, 0), (-1, -1), 8), ("BOTTOMPADDING", (0,
0), (-1, -1), 6)]))
    story.append(t); GAP(2)

# ----- cover
LOGO = r"C:\Users\wadza\OneDrive\2026\Tenders\PFA\Excel & Data Analysis Training\Basic Excel Course\3 Source Files\img\
dupleix-logo.png"
try:
    im = Image(LOGO); r = im.imageHeight / im.imageWidth; im.drawWidth = 42 * mm; im.drawHeight = 42 * mm * r; im.hAlign =
"LEFT"; story.append(im); GAP(10)
except Exception: pass
story.append(Paragraph("MAIIC E-Banker follow-up extracts", S["title"]))
story.append(Paragraph("Audit of the eighteen result files, the reconciliations they allow, and whether the EIR build can
start", S["sub"]))
story.append(Paragraph("Prepared by Dupleix Institute for MAIIC Finance and ICT and the Dupleix team · 7 October 2026",
S["meta"]))
story.append(Paragraph("Based on: the eighteen CSV files received from Barry Makumba on 7 October 2026 (P1_01 to P3_18); the
twenty monthly trial balances "
    "January 2025 to August 2026; Extracts A and C as delivered in August 2026. Every figure below was
produced by scripts kept in the Build files folder.", S["meta"])))
```

```

GAP(8)
BOX(["<b>What this report is.</b> An audit of the data as received: what each file holds, whether the dates can be trusted, how
the files tie to one another and to the trial balance, "
    "and what that means for building the EIR engine. It is not yet the EIR calculation itself, and it does not change any
code; section 12 lists the code changes it calls for.")]

# ----- 1 summary
H1("1. Summary")
P("All eighteen files arrived, every one ran, and together they change the position of the project. For the first time we can
tie the loan ledger to the "
    "balance history, to the stored loan book, to the interest extract and to the trial balance, and the ties are exact or
explained. The answers to the "
    "questions asked of this audit are:")
TAB(["Question", "Answer"], [
    ["<b>Are the dates right, and what is the format?</b>", "Yes. Every one of the 100,504 date values in the eighteen files is
in the same format, month first (7/15/2025), with no time of day. None is transposed. The session setting we asked for did not
take effect, so the files are not year-first, but a consistent month-first format is safe to read once the reader is told the
format. Section 3."],
    ["<b>Which gaps are now covered?</b>", "Interest Policy, the rate history per loan, the PLR history, signed postings with
narrations, the fee table, the loan book history from December 2024, the balance history from July 2024, the instalment chart
with recoveries, and the meaning of most transaction codes. Section 4."],
    ["<b>Do the extracts reconcile to the GL?</b>", "Largely yes. The ledger agrees with the balance history on 2,533 of 2,549
account-months; with the loan book's carrying amount on all 2,264 account-months; with Extract C to the cent (MWK 5,293,988,
207.06); and with the trial balance exactly for three of the five loan GLs in every month, the other two carrying a constant
difference of +400,000 and -1,000,000 that predates 2025. Interest income in the trial balance agrees with the ledger in every
month that can be compared. Section 5."],
    ["<b>Can we rebuild the interest the system charged?</b>", "Yes, to 0.1% on 1,595 of the 2,047 standard interest postings,
and to 1% on 1,828 (91.6% of the value), from daily balances and the rate in the narration. The rest are months with a
mid-month rate change, which the dated rate table resolves. Section 6."],
    ["<b>Can we rely on the system's repayment profiles?</b>", "No. Only 75 of 184 accounts have a current instalment chart,
only 30 of those are level instalments, and actual repayments are within 5% of the chart on 23 of 69 accounts. The engine must
generate the contractual profile from the loan terms, which are present for 183 of 184 accounts, and use the chart only as a
check. Section 7."],
    ["<b>Are we ready to build?</b>", "Yes for the 75 loans originated in E-Banker. For the 109 loans taken on in July 2024,
the system gives everything from take-on onward (balance, rates, cash flows) but not the original drawdowns or fees, which must
come from the take-on workbook and Finance. Section 8."],
    ["<b>What should change in the code?</b>", "New importers for the signed ledger, rate history, PLR, balance history and
loan book history; a strict month-first date reader; cash classified by transaction code; an interest-rebuild routine on daily
balances and dated rates; and three automated reconciliations. Section 12."],
], [0.30, 0.70])

# ----- 2 received
H1("2. What was received")
TAB(["File", "Rows", "Accounts", "What it holds", "Period"], [
    ["P1_01 ledger", "3,640", "132", "Every posting on the loan accounts, signed, with narration. 2,412 debits, 1,228 credits",
"Jul 2024 to Sep 2026 (one 2022 row)",
    ["P1_02 account master", "184", "184", "Every column of the account master: policy, flag, rate, status, dates", "Current"],
    ["P1_03 loan master", "184", "184", "Sanction amount and date, tenor, expiry, moratorium fields", "Current"],
    ["P1_04 rate set-up", "1,291", "183", "Rate history per account: policy, PLR rate, rate, effective date", "Feb 2020 to Sep
2026"],
    ["P1_05 interest summary", "82", "82", "The calculation behind each interest posting, but for the latest month only", "Aug
2026 only"],
    ["P2_06 PLR master", "49", "", "The prime lending rate history", "Mar 2020 to Sep 2025"],
    ["P2_07 disbursement charges", "3,651", "52 in scope", "Fee lines by name and amount (whole table, 1,218 accounts)",
"Current"],
    ["P2_08 loan book history", "12,414", "144", "The stored loan book report, several runs per month-end (2,361 unique rows)",
"Dec 2024 to Aug 2026, 21 month-ends"],
    ["P2_09 balance history", "2,549", "131", "Month-end balance, receipts, payments, interest fields", "Jul 2024 onward,
monthly"],
    ["P2_10 instalment chart", "2,742", "75", "Current chart: date, instalment, principal and interest parts, expected balance,
recovered", "Current"],
    ["P3_11 instalment plans", "1,958", "160", "Instalment plan versions: type, frequency, start date, amount", "2021 onward"],
    ["P3_12 disbursement schedules", "251", "161", "Planned disbursements", ""],
    ["P3_13 status history", "283", "184", "Every status change with its reason", "2020 onward"],
    ["P3_14 daily accrual", "7,489", "14", "Daily accrual log, fourteen accounts only", "Nov 2024 to Sep 2026"],
    ["P3_15 slab master", "100", "", "Scheme-level rate slabs", "2010 onward"],
    ["P3_16 forgiven interest", "2", "2", "Interest forgiveness entries", "Jun 2026"],
    ["P3_17 auto charges", "1", "", "One charge rule", ""],
    ["P3_18 date-time check", "3", "", "No date carries a time of day: 0 of 409,552 ledger rows, 79,375 balance rows, 97,249
chart rows", ""],
], [0.19, 0.08, 0.10, 0.41, 0.22], bold_first=True)
P("Two files are thinner than hoped. The interest summary holds only the latest month, so it cannot replace Extract C for
history; it is not needed, because the "
    "ledger's own interest postings carry the period and rate in their narration. The daily accrual log covers fourteen accounts
only.")

# ----- 3 dates
H1("3. The dates: format, proof and what to do")
H2("What the files contain")
P("Every date column in every file is written the same way: month, day, year, with slashes and no time of day, for example 7/15/
2025 and 8/8/2025. We tested "
    "all 100,504 non-blank date values: in none of them is the first number greater than 12, and in 66,762 the second number is
greater than 12. That proves the "
    "order is month first throughout. Run 18 confirms that none of the dates in the three big tables carries a time of day, so
the \"on or before\" and "
    "\"between\" tests in the extract scripts lose nothing at the end of a day.")
H2("Why this is different from the August extracts")
P("The August extracts were damaged on the way out: Excel re-read day-first text as month-first and silently swapped days and
months on part of each column, "
    "leaving a mixture that could not be trusted. We can now prove that. Extract A's loan start dates come in two kinds: 108 that
stayed as text, all 108 of ")

```

"which match the true opening date now in the account master; and 73 that Excel had converted, of which 2 match as read and 71 match only after the day and "

"month are swapped back. The follow-up files have not been through Excel, so every column is one format and nothing is swapped.")

H2("Why the files are not year-first, and the fix")

P("The two session settings we asked Barry to run would have produced 2025-07-15. The files show that the export tool used its own date preference instead. "

"In SQL Developer the grid and the CSV export follow Tools, Preferences, Database, NLS, Date Format, which overrides the session setting. That is harmless "

"here, because month-first with a four-digit year is unambiguous and consistent. For future extracts Barry can set that preference to YYYY-MM-DD once, and "

"the files will arrive year-first.")

BOX(["The rule for reading these files: parse every date column with the explicit format month/day/year. Never let a reader guess, and reject any "

"file in which a date column mixes real dates with text, because that is the signature of an Excel round trip. Account numbers are text with leading "

"zeros (fifteen characters, every row, every file) and must be read as text."])

----- 4 gaps

H1("4. The gaps these files close")

TAB(["Gap we had", "What closes it", "Evidence"], [

["Interest Policy per loan", "Account master and rate set-up", "P 42 accounts (all MAIIC Agricultural and Industrial), S 127 (FinES, Term and Investment), M 15 (manual). Matches the loan book behaviour"],

["The spread over the prime rate", "Rate set-up table", "MARGIN_PERCENT on the loan master is zero everywhere; the spread is the difference between INTEREST_RATE and PLR_RATE in the rate set-up (Ebenezer Midian: 31.2 - 25.3 = 5.9)],

["Rate history per loan", "Rate set-up table", "1,291 rows, 183 accounts, effective dates from February 2020; 106 of 108 pre-migration accounts have a rate dated before July 2024; MAIIC accounts carry up to 54 dated steps"],

["Reference rate history", "PLR master", "49 rows, March 2020 to September 2025"],

["Direction of postings", "Ledger", "Signed amounts and a DBCR column. Disbursements, interest charges and reversals are debits (negative); repayments and adjustments are credits"],

["Meaning of transaction codes", "Ledger narrations", "See section 6: 300 write-off, 306 reversal of a repayment, 343 reversal of credits, 120 year-end rate-difference credit, 303 interest charge, 301 drawdown"],

["Fees", "Charges table and ledger narrations", "Charges table: 84 lines with an amount on 52 of our accounts, MWK 350.7m. Narrations name fees deducted at drawdown on 18 accounts. Section 9"],

["Loan book for the missing months", "Loan book history", "21 month-ends from December 2024; November 2024 and earlier are not in it, but the balance history covers July 2024 onward"],

["Month-end balances", "Balance history", "2,549 account-months from July 2024; ties to the ledger on 2,533"],

["The instalment chart with recoveries", "Chart extract", "75 accounts with a current chart, with principal and interest parts and an expected balance per row"],

["Status H and F", "Status history and balances", "Not in the status history as codes. The last recorded change on every H account is to D with the reason Paid Up or Foreclosure; 18 of 21 H accounts carry a nil balance and 17 have an interest stop date. H looks like \"paid up, awaiting closure\". Vendor to confirm"],

["Restructures and write-offs", "Ledger", "The two write-offs (Micholess Creameries 10,244,624.69 and Maluso Cooperative 12,041,809.63, August 2024) exist as type 300 ledger credits although the write-off tables are empty"],

], [0.22, 0.22, 0.56])

P("Still outstanding: the LOS application and process references, the meaning of code H from the vendor, the origination data for the take-on loans (section 8), "

"and the two constant GL differences (section 5).")

----- 5 reconciliations

H1("5. Reconciliations")

P("Each test joins two sources that were produced independently. A tie means both are complete and consistent; a difference is a lead.")

TAB(["#", "Test", "Result", "What it tells us"], [

["a", "Ledger running balance against the balance history, every account-month", "2,549 account-months; 2,533 agree within MWK 1; the 16 that differ are one account, Zaithwa Farms (000104450000015), by 1,827,633.62 from May 2025", "The ledger extract is complete for 130 of 131 accounts. Zaithwa Farms has a balance movement with no posting in the ledger; ask Barry which posting is missing or whether the balance was adjusted directly"],

["b", "Ledger running balance against the loan book's carrying amount", "2,264 account-months; 2,264 agree. The loan book's PRINCIPAL column agrees on 15 only", "The stored loan book is the ledger balance. PRINCIPAL is a different measure (it excludes capitalised interest) and must not be used as the carrying amount"],

["c", "Ledger interest (types 303 and 120) against Extract C, run 41", "1,564 account-months, all agree; totals January 2025 to July 2026 equal to the cent, MWK 5,293,988,207.06. The ledger has 537 further account-months outside Extract C's window", "Extract C is exactly the ledger's interest postings on the loan's own GL. It is no longer needed"],

["d", "Trial balance loan GL balances against the sum of ledger balances, 20 month-ends", "1050101, 1050102 and 1050401 agree to the cent in every month. 1050201 is 400,000.00 higher in the TB in every month; 1050202 is 1,000,000.00 lower in every month", "The ledger extract covers the whole of three GLs. The two constant differences predate January 2025 and sit outside the account-level ledger: most likely a GL-level journal or a take-on difference. Ask Barry for the postings on those two GLs with no account number"],

["e", "Trial balance interest income against ledger interest, by GL, by month", "The TB income accounts are year-to-date. On that basis the ledger agrees in every month that has a TB line: 83 rows. The 17 rows without a line are FinES Agricultural in January 2025 (nil), the Term Loan account before December 2025 (nil), and December 2025, whose TB carries no income lines", "Interest charged to the loans in the ledger is what the GL recognised as income. The reconciliation the engine must perform is EIR interest against these figures"],

["f", "Interest summary (August 2026) against the ledger's August interest", "82 accounts, all equal; no overdue or penal component in any of them", "The posting is the plain accrual; penalties are not mixed into it"],

["g", "Drawdowns against sanctioned amounts", "131 accounts with drawdowns: equal on 112, less on 13, more on 6", "The six over-drawn are worth a look; they may be fee or insurance add-ons posted as drawdowns"],

], [0.03, 0.22, 0.38, 0.37])

P("Total loan book at 31 December 2025: trial balance MWK 14,840,612,779.54; ledger MWK 14,841,212,779.54; difference 600,000.00, being the two constant items.")

----- 6 interest rebuild

H1("6. Rebuilding the interest the system charged")

P("Each monthly interest posting carries its own narration: \"IntDR Fm 08-AUG-25 To 31-AUG-25 @ 31%\". We rebuilt every posting from the ledger alone: the "

"balance at the end of each day in the stated period, summed, times the stated rate, divided by 365.")

TAB(["Measure", "Result"], [

["Interest postings (type 303)", "2,159, totalling MWK 4,526,723,541.58"],

["With the standard narration", "2,047. The other 112 are 79 \"Opening interest charge\" rows at take-on (31 July 2024) and 22 year-end \"Diff Int. Debit ... By ROI\" rows (December 2025), together MWK 1,837,859,679.99"],

```

["Rebuilt within 0.1%", "1,595 of 2,047"],
["Rebuilt within 1%", "1,828 of 2,047, which is 91.6% of the value posted"],
["Not within 1%", "219 postings on 93 accounts, 8.4% of the value. These are months in which the rate changed part-way
through (the rate set-up shows steps dated 5 and 9 March 2026 for the same loan, for example); the narration carries only the
closing rate"],
["Narration rate against the rate set-up table", "1,659 of 2,047 agree with the rate in force on the posting date"],
["Convention", "Daily closing balance, actual days, 365-day year. The monthly rate/12 convention does not reproduce the
postings"],
], [0.30, 0.70])
P("Conclusion: the engine can reproduce E-Banker's own accrual from the ledger and the dated rate table, which is what makes a
reconciliation of EIR interest to "
"GL interest explainable line by line. The remaining step is to apply the dated rate steps inside a month rather than one
rate per month.")
H2("The transaction codes, now explained")
TAB(["Code", "Rows", "Sign", "Meaning, from the narrations"], [
["301", "214", "Debit", "Drawdown, including fee and insurance deductions posted as separate drawdown lines; the 78 take-on
lines of 31 July 2024 are \"Opening Account Balance Disbursement amount\", MWK 8,297,388,309.25"],
["303", "2,159", "Debit", "Monthly interest charge, capitalised to the loan; plus the take-on opening interest and the
year-end rate-difference debits"],
["305", "1,212", "Credit", "Repayment"],
["306", "37", "Debit", "Reversal of a repayment (\"reversal\", \"reversal for duplicate payments\", \"reverse of miss
posting\")"],
["120", "6", "Credit", "\"Diff Int. Credit From 01/01/2025 To 31/12/2025 By ROI\": year-end credit correcting interest for
a rate change; equals the Extract B to C difference"],
["300", "2", "Credit", "Write-off (Micholess Creameries, Maluso Cooperative), August 2024"],
["343", "2", "Credit", "\"Reversal of credit transactions to account\", 12 March 2026"],
["900 / 901", "1 / 1", "Debit / Credit", "A fee of 10,350,000 posted on 6 May 2025 and reversed on 12 May 2025"],
["400 / 401", "1 / 5", "", "Postings on GL 3065, NASCOMEX Limited, an equity-type instrument, not a loan"],
], [0.10, 0.08, 0.12, 0.70], keep=True)

# ----- 7 repayment profiles
H1("7. Repayment profiles: rebuild them, or rely on the system?")
P("The effective interest rate needs the contractual cash flows at origination, and the present value tests need the expected
cash flows discounted at that rate. "
"The question is whether E-Banker's stored instalment chart can serve as those cash flows.")
TAB(["Test on the stored chart", "Result"], [
["Accounts with a current chart", "75 of 184. Of the 89 active accounts, 53 have no current chart"],
["Shape of the chart", "56 of 75 begin with a zero row at the start date; 74 of 75 end with an expected balance of nil;
principal plus interest equals the instalment on 2,240 of 2,742 rows"],
["Level instalments", "30 of 75 accounts; the rest step up or down"],
["Interest convention in the chart", "Expected balance times the current rate divided by 12 reproduces 1,094 of 2,635
interest parts within 0.5%; the actual-days convention reproduces 28. The chart is a monthly-rate schedule, the ledger accrues
daily: the two will never agree exactly"],
["Actual repayments against the chart", "Of 69 accounts with both, 23 have paid within 5% of what the chart says was due to
date; 21 have paid less than 80%; 8 have paid more than 120%"],
["Recovered-to-date field", "Does not agree with cash received on any account"],
], [0.32, 0.68])
H2("What this means for the engine")
BUL([
"<b>Generate the contractual profile from the terms.</b> The terms are present: sanction amount, sanction date, expiry and
tenor on 183 of 184 accounts; rate "
"on all 184; principal and interest moratorium periods on all 184; an instalment plan (frequency, start date, type) on 159.
The generator should follow the chart's "
"own convention, level instalment at rate/12 on the expected balance, with the moratorium shapes the loan master and plan
describe.",
"<b>Use the stored chart as the check, not the source.</b> Where a chart exists, the generated schedule should reproduce it;
where it does not, the two "
"conventions (monthly in the chart, daily in the ledger) explain the gap.",
"<b>Take actual cash flows from the ledger.</b> Drawdowns are type 301, repayments are type 305 net of 306. Repayment
behaviour departs from the schedule on "
"two accounts in three, so the amortised cost after origination must roll forward on actual cash, never on the schedule.",
"<b>Discount expected cash flows at the EIR.</b> For the present value of future repayments (impairment and modification
tests), the expected cash flows are "
"the generated contractual schedule from the current balance, adjusted for known arrears and moratoria, discounted at the
original EIR. The stored chart "
"cannot be relied on for this because it is missing for most accounts and does not reflect what has been paid.",
])

# ----- 8 ready
H1("8. Are we ready to build: pre- and post-migration")
TAB(["Component", "Post-migration (75 accounts, opened from July 2024)", "Pre-migration (109 accounts, opened before July
2024)"], [
["Cash out, by drawdown", "In the ledger: 54 accounts with type 301 postings, dated, with fee lines named", "Not in
E-Banker. 77 accounts were taken on with one \"opening balance\" line on 31 July 2024; 32 had already closed. Original
drawdowns come from the take-on workbook"],
["Fees at origination", "Charges table (38 accounts with amounts) and the ledger narrations (18 accounts); section 9", "Not
in E-Banker. From the 2024 and 2025 EIR assessments and Finance's records"],
["Rate at origination and changes", "Rate set-up table with effective dates; PLR history; the rate in every interest
narration", "Rate set-up table carries a rate dated before July 2024 for 106 of 108; the take-on workbook carries the dated
steps"],
["Contractual schedule", "Generated from the terms (section 7)", "Generated from the original terms, which the loan master
holds (sanction date, tenor), from the original date"],
["Actual cash flows", "Ledger, complete", "Ledger from 31 July 2024; the take-on workbook before"],
["Carrying amount at month-end", "Balance history from July 2024; loan book from December 2024", "Same, from take-on;
before that the take-on workbook"],
["Interest recognised, for the reconciliation", "Ledger type 303 and 120, tying to the TB", "Same, from July 2024"],
["Linking key", "Account number, present everywhere", "Account number for the E-Banker side; the take-on workbook has no
account numbers and needs the mapping from Finance"],
], [0.20, 0.40, 0.40])
BOX(["<b>Verdict.</b> Post-migration: ready, subject to the three items in section 11. Pre-migration: ready from 31 July 2024
onward; the origination side "

```

"(original drawdowns, fees, pre-2024 rate steps) needs the take-on workbook mapped to account numbers, and the fee records from Finance. The 46 closed "

"accounts need no profile. The 21 H and 3 F accounts are paid up and need only closing entries."], GREEN)

----- 9 fees

H1("9. Fees: two sources that overlap but do not yet agree")

TAB(["Source", "What it shows"], [
["Charges table (P2_07), our accounts", "52 accounts, 155 lines, 84 with an amount: Agreement Fee MWK 205,417,295 on 38 accounts, Legal Fee 136,217,758 on 37, Other Fee 9,032,970 on 9. POST_FLAG is N and INCOME_GL is blank on every line, so this table records the fees agreed, not their posting"],

["Ledger narrations (P1_01), type 301", "43 drawdown lines on 18 accounts name a fee: arrangement fees 197,537,942, legal fees 73,922,725, insurance 4,419,598.84. These are the amounts actually deducted from the advance"],

["Overlap", "13 accounts appear in both; 5 have a fee narration but no charges line; 25 have a charges line but no fee narration (most of them are take-on loans whose drawdown predates the ledger)"],

["Trial balance fee income, August 2026 year-to-date", "4873 Arrangement Fees 414,947,879; 4871 Legal Fees 243,389,041"],
], [0.30, 0.70])

P("For the EIR the figure that matters is the fee actually deducted at drawdown, which is the ledger line. Where an account has a charges-table amount but no "

"ledger line, the fee was either deducted before July 2024 (take-on loans) or not deducted through the loan account. The fee template remains the way to "

"confirm the latter. A separate reconciliation of the fee income accounts to these two sources is still to be done.")

----- 10 status

H1("10. Status codes")

TAB(["Code", "Accounts", "What the data shows"], [
["A Active", "89", "6 have no ledger postings at all: approved, not yet drawn"],

["C Closed", "46", "No ledger rows for 46 of them; closed before July 2024 or settled"],

["D Dormant", "25", "23 of 25 carry a balance; 20 have an interest stop date. In E-Banker \"dormant\" appears to mean non-accruing, which is the staging population"],

["H", "21", "Not a code in the status history. The last recorded change on each is to D, reason Paid Up (17), Foreclosure (3) or Account closure (1); 18 of 21 have a nil balance; 17 have an interest stop date. Reads as paid up, awaiting closure. Vendor to confirm"],

["F", "3", "All three nil balance, reason Paid Up"],
], [0.12, 0.10, 0.78], keep=True)

----- 11 open

H1("11. What is still open, and what to ask")

TAB(["Item", "Ask", "Who"], [
["Constant GL differences: 1050201 +400,000.00, 1050202 -1,000,000.00, every month", "The postings on those two GLs that

carry no account number, since January 2025 or earlier", "Barry"],

["Zaithwa Farms (000104450000015): balance history moves by 1,827,633.62 in May 2025 with no ledger posting", "The missing posting, or the balance adjustment", "Barry"],

["Six over-drawn accounts (drawn exceeds sanction)", "Whether the excess is fee or insurance add-on", "Credit"],

["Code H", "Confirm \"paid up, awaiting closure\"", "Vendor"],

["Take-on loans: original drawdowns, fees, pre-2024 rate steps", "The take-on workbook blocks mapped to account numbers, and the origination fee per loan", "Tamanda"],

["Interest summary and daily accrual tables hold the latest month and 14 accounts only", "Confirm they are purged monthly; not needed if so", "Barry"],

["Year-end \"Diff Int ... By ROI\" debits (22) and credits (6) of December 2025", "What triggered them: a rate correction applied retrospectively?", "Finance"],

["Export date format", "Set SQL Developer's NLS date format preference to YYYY-MM-DD so future files arrive year-first", "Barry"],

["Loan book history runs", "Several runs are stored per month-end (up to 12); we take the latest. Confirm that is the right one", "Barry"],
], [0.44, 0.44, 0.12])

----- 12 code

H1("12. Suggested changes to the engine code")

P("These follow from the audit. They are for the Duplex team; nothing has been changed yet. File names refer to the MAICC-IFRS9 repository.")

TAB(["#", "Change", "Why", "Priority"], [
["1", "A strict date reader for MAIIC files: parse with the explicit month/day/year format; refuse a column that

mixes real dates and text; refuse ambiguous formats. Replace the flexible fallback in " + M("MappedFileReader.php") + " for these feeds.", "The August files were damaged by a guessing reader downstream of Excel. These files are safe only if read with the declared format.", "High"],

["2", "A ledger importer for P1_01: signed amount, transaction type code, narration, GL code, posting number. Classify cash by code: 301 out, 305 in, 306 reversal of 305, 303 accrual (not cash), 120 and year-end 303 adjustments, 300 write-off, 343 reversal. Replace " + M("ContractTransactionImportService.php") + " and the label-based list in " + M("EirRevenueService.php") + ".", "Direction and meaning now come from the data. The present code counts interest charges as cash collected.", "High"],

["3", "Rate history importer for P1_04 and P2_06: dated rate steps per account and the PLR series; derive the rate type from INTEREST_POLICY (P floating, S and M fixed); drop the derivation from Extract A's label.", "Interest Policy and the dated rates are now in hand; the old label was inverted.", "High"],

["4", "Interest rebuild routine daily closing balance from the ledger, times the rate in force each day from the rate steps, over 365. Report each posting as rebuilt, within tolerance, or unexplained.", "Reproduces 91.6% of interest to 1% already; with dated steps it should reach the rest. This is the explainable half of the EIR-to-GL reconciliation.", "High"],

["5", "Schedule generator aligned to the chart's convention level instalment at rate/12 on the expected balance, zero opening row, principal and interest moratorium periods from the loan master, frequency from the plan. Compare to the stored chart where one exists and report the variance.", "The engine must generate profiles for all accounts; only 75 have a chart.", "High"],

["6", "Balance history and loan book importers (P2_09, P2_08, latest run per month-end), used as independent month-end balances.", "Both tie to the ledger; they give the carrying amount for months with no posting and the missing-months loan book.", "Medium"],

["7", "Three automated reconciliations run on every load: ledger balance to balance history per account-month; ledger balance to the TB per GL per month-end; ledger interest to TB interest income (year-to-date) per GL per month.", "All three tie today; running them on each load catches a broken extract before it reaches a calculation.", "Medium"],

["8", "Retire the Extract A, B and C importers once the new feeds are loaded, or keep them read-only for lineage.", "The ledger reproduces Extract C to the cent; the account and loan masters replace Extract A; the signed ledger replaces Extract B.", "Medium"],

["9", "Fee capture read fees from the type 301 lines whose narration names a fee, and from the charges table, and reconcile both to the fee income GLs; keep the fee template for fees handled outside the account.", "Two sources overlap on 13

```

accounts only.", "Medium"],
["10", "<b>Status handling:</b> store the raw code; treat D as non-accruing; treat H and F as paid-up pending vendor
confirmation; use INTEREST_STOP_DATE as the non-accrual date.", "The data now explains the codes well enough to act on.",
"Low"],
], [0.03, 0.40, 0.45, 0.12])

# ----- 13 next
H1("13. Next steps")
BUL([
    "Send Barry the four asks in section 11 that are his: the GL-only postings on 1050201 and 1050202, the Zaithwa Farms
posting, the purge behaviour of the two small tables, and the date preference.",
    "Ask Tamanda for the take-on workbook mapping to account numbers and the origination fee per take-on loan.",
    "Make the code changes in section 12, items 1 to 5 first, and load the eighteen files.",
    "Run the EIR for the 75 post-migration loans, reconcile EIR interest to the trial balance by GL and month, and present that
reconciliation as the first engine output.",
    "Extend to the take-on loans as the mapping and fees arrive.",
], numbered=True)

# ----- appendix
story.append(PageBreak())
H1("Appendix A. The tests behind this report")
TAB(["Test", "Result"], [
    ["Date values checked across all files", "100,504; first component never above 12; second component above 12 in 66,762"],
    ["Date columns with a time of day (run 18)", "0 of 409,552 ledger rows; 0 of 79,375 balance rows; 0 of 97,249 chart rows"],
    ["Extract A loan start dates against the account master", "Text dates 108: all match. Excel-converted 73: 2 match as read,
71 match after swapping day and month"],
    ["Scope", "184 accounts: A 89, C 46, D 25, H 21, F 3. Opened before July 2024: 109; from July 2024: 75"],
    ["Interest Policy per account", "S 127, P 42, M 15. Scheme 84: P 8, M 8; scheme 86: P 34, M 7; all FInES, Term and
Investment schemes S"],
    ["Ledger", "3,640 rows, 132 accounts, 2,412 debits, 1,228 credits, all on the loan's own GL; 17 postings entered after
their transaction date (13 interest, 4 drawdowns), up to 699 days"],
    ["Ledger against balance history", "2,549 account-months; 2,533 within MWK 1; 16 differ, all Zaithwa Farms, 1,827,633.62
from May 2025"],
    ["Ledger against loan book carrying amount", "2,264 of 2,264 agree; PRINCIPAL agrees on 15"],
    ["Loan book history runs", "12,414 rows, 2,361 unique account and month-end; up to 12 runs per month-end; latest run kept"],
    ["Ledger against Extract C run 41", "1,564 account-months, all agree; both total 5,293,988,207.06"],
    ["Trial balance loan GLs against ledger", "90 GL-months; 50 exact; 1050201 +400,000.00 and 1050202 -1,000,000.00 in all 20
months"],
    ["Trial balance interest income against ledger", "83 of 83 comparable GL-months agree on a year-to-date basis; 17 have no
TB line"],
    ["Interest rebuild from daily balances and narration rate", "2,047 standard postings: 1,595 within 0.1%, 1,828 within 1%
(91.6% of value); 219 not within 1% on 93 accounts"],
    ["Narration rate against rate set-up in force", "1,659 of 2,047 agree"],
    ["Interest summary August 2026 against ledger", "82 of 82 equal; no overdue interest"],
    ["Drawdowns against sanction", "131 accounts: equal 112, less 13, more 6"],
    ["Fees in ledger narrations", "43 lines, 18 accounts: arrangement 197,537,942; legal 73,922,725; insurance 4,419,598.84"],
    ["Charges table, our accounts", "52 accounts, 155 lines, 84 with amounts totalling 350,668,023; POST_FLAG N, INCOME_GL
blank on all"],
    ["Chart", "75 accounts, 2,742 rows; zero first row 56; nil closing balance 74; parts add to instalment 2,240; monthly
convention 1,094 of 2,635; level 30 of 75"],
    ["Repayments against chart due to date", "69 accounts: within 5% 23; below 80% 21; above 120% 8"],
    ["Terms available for generation", "Sanction amount, date, expiry, tenor 183 of 184; rate 184; moratorium fields 184; plan
159; chart 75"],
    ["Take-on", "78 opening-balance drawdown lines on 31 July 2024, 77 accounts, 8,297,388,309.25; 79 opening interest charge
lines"],
    ["Rate history for pre-migration accounts", "108 accounts with rows; 106 with a rate dated before July 2024"],
], [0.40, 0.60])

def footer(canvas, doc):
    canvas.saveState(); canvas.setStrokeColor(LINE); canvas.setLineWidth(0.5)
    canvas.line(18 * mm, 14 * mm, A4[0] - 18 * mm, 14 * mm)
    canvas.setFont("Seg", 8); canvas.setFillColor(GREY)
    canvas.drawString(18 * mm, 9.5 * mm, "MAIIC E-Banker follow-up extracts: audit · Dupleix Institute · 7 October 2026")
    canvas.drawRightString(A4[0] - 18 * mm, 9.5 * mm, f"Page {doc.page}"); canvas.restoreState()

OUT = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\
MAIIC_FollowUp_Extracts_Audit_2026-10-07.pdf"
doc = BaseDocTemplate(OUT, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=20 * mm,
    title="MAIIC E-Banker follow-up extracts: audit", author="Dupleix Institute")
doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, id="f",
    leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)], onPage=footer)])
doc.build(story)
print("written", OUT)

```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\fu_report.py (file state is current in your context – no need to Read it back)

Tool call: Bash

```
description: Build the audit PDF, render a contact sheet, and save the audit scripts to the build files folder
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python fu_report.py && ./songenv/Scripts/python -c "
import pypdfium2 as pdfium
from PIL import Image
p = r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_FollowUp_Extracts_Audit_2026-10-07.pdf'
pdf = pdfium.PdfDocument(p); n = len(pdf); print('pages', n)
ims = [pdf[i].render(scale=0.5).to_pil() for i in range(n)]
w, h = ims[0].size; cols = 5; rows = (n + cols - 1) // cols
sheet = Image.new('RGB', (w * cols, h * rows), 'white')
for i, im in enumerate(ims): sheet.paste(im, ((i % cols) * w, (i // cols) * h))
sheet.save('contact_fu.png'); pdf[0].render(scale=1.0).to_pil().save('fu_p1.png')
" && DST="/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Build files" && cp fu_load.py fu_profile.py fu_a1.py fu_a2.py fu_a3.py fu_a4.py fu_a5.py fu_report.py "$DST/" && ls "$DST" | wc -l
{
  "timeout": 300000
}
```

Result of Bash

```
written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\MAIIC_FollowUp_Extracts_Audit_2026-10-07.pdf
pages 11
23
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Read

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude/c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\contact_fu.png
```

Result of Read

```
[image output: not reproduced in this transcript]
```

CLAUDE (2026-10-07 09:06:28 UTC)

Report built (11 pages). Adding the audit scripts to the build-files README, then summarising.

Tool call: Bash

```
description: Record the audit scripts in the build files README
--- command ---
cd "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Build files" && PYTHONIOENCODING=utf-8 "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad/songenv/Scripts/python" - <<'PYEOF'
p = "README.md"; s = open(p, encoding="utf-8").read()
add = ""
## 7 October 2026: audit of the eighteen follow-up extracts

The result files are in `...\Follow-Up Scripts Resutls`. The audit report is `...\MAIIC_FollowUp_Extracts_Audit_2026-10-07.pdf`. These scripts produced it; run them from this folder in the order listed (they need `pandas`, `openpyxl`, `xlrd` and `reportlab`).

| File | Purpose |
|---|---|
| `fu_load.py` | Shared loader: reads every result file, parses dates with the explicit month/day/year format, keeps account numbers as text |
| `fu_profile.py` | Rows, columns and date formats of every file; proves the month-first order |
| `fu_a1.py` | Coverage per table, scope, interest policy, ledger signs, loan book and balance history months, rate set-up, chart shape |
| `fu_a2.py` | Reconciliations: ledger to balance history, to loan book carrying amount, to Extract C; interest recomputed from narrations; drawdowns and fees; Extract A dates |
| `fu_a3.py` | Trial balance to ledger (loan GL balances and interest income); interest summary; fee sources; chart conventions; Extract A Excel dates. Writes `tb_balance_recon.csv` and `tb_income_recon.csv` |
| `fu_a4.py` | Income rows that did not tie, pre- and post-migration coverage, status codes, terms available for schedule generation |
| `fu_a5.py` | Rebuilds every interest posting from daily balances and the narration rate; writes `interest_rebuild.csv`; checks the December 2025 trial balance |
| `fu_report.py` | Builds the audit PDF |
| ""
open(p, "w", encoding="utf-8").write(s.rstrip() + "\n" + add); print("readme updated")
PYEOF
ls -la --time-style=long-iso "." | grep -i "audit\|follow" | awk '{print $5, $6, $7, $8, $9, $10, $11, $12, $13, $14}'
```

Result of Bash

```
<stdin>:5: SyntaxWarning: "\F" is an invalid escape sequence. Such sequences will not work in the future. Did you mean "\\F"? A raw string is also an option.
readme updated
146227 2026-10-06 22:59 EMAIL - 18 follow-up extracts by 10am
72862 2026-10-06 16:37 FOLLOW-UP extracts for Barry - 6 Oct
12475 2026-10-06 16:30 FOLLOW-UP extracts for Barry - 6 Oct
12475 2026-10-06 16:37 FOLLOW-UP extracts for Barry - 6 Oct
0 2026-10-07 10:39 Follow-Up Scripts Resutls
105034 2026-10-07 11:06 MAIIC_FollowUp_Extracts_Audit_2026-10-07.pdf
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

CLAUDE (2026-10-07 09:06:57 UTC)

Written for: MAIIC Finance and ICT and the Dupleix team.

The audit report is saved as an 11-page PDF next to the results folder, and every figure in it comes from scripts now in the Build files folder. The headline answers:

Dates. All 100,504 date values across the 18 files are one format: month/day/year, no time of day, none transposed. The proof is that the first number is never above 12 and the second is above 12 in 66,762 values. The session setting didn't take effect because SQL Developer's own NLS preference overrides it; that's harmless with a consistent format, and the fix for next time is to set that preference to YYYY-MM-DD. I also proved the August damage: of Extract A's 73 Excel-converted start dates, 71 match the true date only after swapping day and month.

Reconciliations. These are the strongest results we've had:

- Ledger to balance history: 2,533 of 2,549 account-months tie; the 16 exceptions are one account (Zaithwa Farms, 1,827,633.62 from May 2025).
- Ledger to the stored loan book's carrying amount: 2,264 of 2,264.
- Ledger to Extract C: all 1,564 account-months, totals equal to the cent (MWK 5,293,988,207.06). Extract C is no longer needed.
- Ledger to the trial balance: three loan GLs tie to the cent in all 20 months; 1050201 is +400,000 and 1050202 is -1,000,000 in every month, a pre-2025 item outside the account ledger.
- Trial balance interest income (year-to-date) ties to the ledger in every month that has a TB line.

Rebuilding interest. From daily ledger balances and the rate in each posting's narration, 1,595 of 2,047 postings rebuild within 0.1% and 1,828 within 1% (91.6% of value). The rest are months with a mid-month rate change, which the dated rate table resolves. Convention: daily closing balance, actual days, 365.

Repayment profiles. Do not rely on the system's. Only 75 of 184 accounts have a current chart, 30 of those are level, and actual repayments are within 5% of the chart on 23 of 69 accounts. The engine must generate profiles from the terms (present for 183 of 184 accounts) and use the chart as a check. Actual cash comes from the ledger: 301 out, 305 in net of 306.

Ready to build? Yes for the 75 loans originated in E-Banker. For the 109 take-on loans, everything from 31 July 2024 is in the system; original drawdowns, fees and pre-2024 rate steps need the take-on workbook mapped to account numbers (Tamanda).

Codes explained from narrations: 300 write-off, 306 reversal of a repayment, 343 reversal of credits, 120 year-end rate-difference credit. H accounts read as paid-up awaiting closure (18 of 21 nil balance), pending the vendor.

Code changes (section 12): a strict month-first date reader, a signed-ledger importer with cash classified by code, rate-history and PLR importers with rate type from INTEREST_POLICY, an interest-rebuild routine, a schedule generator on the chart's convention, and the three reconciliations above run automatically on every load.

Two things to flag: the interest summary and daily accrual tables hold only the latest month and 14 accounts, so they don't replace Extract C (the ledger does); and the loan book history stores up to 12 runs per month-end, so the latest run must be selected.

Files are in 2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC:

- MAIIC_FollowUp_Extracts_Audit_2026-10-07.pdf
- Build files\README.md (the eight fu_*.py audit scripts are listed there)

USER (2026-10-07 09:11:23 UTC)

what do we need to do from now?

CLAUDE (2026-10-07 09:11:34 UTC)

Four things, in this order. The first two run in parallel.

1. Clear the last data items (this week)

- Barry: the GL-only postings on 1050201 and 1050202 (the +400,000 and –1,000,000), the missing Zaithwa Farms posting, and the SQL Developer date preference set to YYYY-MM-DD.
- Tamanda: the take-on workbook blocks mapped to account numbers, and the origination fee per take-on loan. This is the only thing standing between us and the pre-migration profiles.
- Vendor: confirm code H. Credit: the six accounts drawn above sanction.

2. Change the engine (this week and next)

The five High items in section 12 of the audit: the strict date reader, the signed-ledger importer with cash classified by transaction code, the rate-history and PLR importers with the rate type taken from Interest Policy, the interest-rebuild routine on daily balances, and the schedule generator aligned to the chart's convention. Then load the eighteen files and the twenty trial balances.

3. Produce the first real output (as soon as 2 is done)

Run the EIR for the 75 post-migration loans and reconcile EIR interest to the trial balance by GL and by month. That reconciliation, line by line, is the deliverable that shows Dr Thom and the auditors that the engine works. The three automated checks (ledger to balance history, ledger to TB balances, ledger interest to TB income) should pass on the load before anyone looks at an EIR.

4. Extend to the take-on book (when Tamanda's mapping arrives)

Join each take-on block to its account, load the original drawdowns and fees, and run the same calculation and reconciliation for the 109 pre-migration loans.

One decision for you and Edward before step 3: which record is the contractual one for the EIR, the signed offer letter or the system. The data now supports either; the choice has to be written down.

USER (2026-10-07 09:14:15 UTC)

Barry: the GL-only postings on 1050201 and 1050202 (the +400,000 and –1,000,000) => explain more on this one

CLAUDE (2026-10-07 09:14:33 UTC)

Here is what the test did and what the result means.

The test. For each of the 20 month-ends from January 2025 to August 2026, I added up the running balance of every loan account in the ledger extract, GL code by GL code, and compared the total with the debit balance the trial balance shows for that GL. If the extract holds every posting on every account in a GL, the two must be equal, because the GL balance is nothing more than the sum of its account balances.

The result. For 1050101 (MAIIC Agricultural), 1050102 (MAIIC Industrial) and 1050401 (MAIIC Term Loan) the two are equal to the cent in every one of the 20 months. For the two FInES GLs they differ, but by the same amount every month:

- 1050201 FInES Agricultural: the trial balance is 400,000.00 higher than the accounts.
- 1050202 FInES Industrial: the trial balance is 1,000,000.00 lower than the accounts.

Why a constant difference is informative. If postings were missing from the extract month by month, the difference would grow or shrink as those postings happened. A difference that is identical in January 2025 and in August 2026 means nothing has changed in the gap since before January 2025: whatever caused it happened earlier, once, and has sat there ever since. The round amounts point the same way.

The likely causes. Three possibilities, all of which Barry can see and we cannot:

1. A journal posted straight to the GL code without a loan account number, so it is in the GL balance but not in any account's ledger. The 400,000 and 1,000,000 look like such entries.
2. A take-on difference from the July 2024 migration: the GL opening balance was loaded with one figure and the account balances with another.
3. A posting on an account that is not in our scope, for example a loan on a scheme whose name does not begin with MAIIC or FInES but that sits on a FInES GL code.

What to ask Barry. Every posting on those two GL codes that has no account number, or whose account is not one of the 184, from the start of E-Banker. This query does it:

```
SELECT cv.*
FROM   cumvouch cv
WHERE  cv.ac_glcode IN ('1050201', '1050202')
AND    cv.delete_flag = 'N'
```

```
AND (cv.new_ac_number IS NULL
      OR cv.new_ac_number NOT IN (SELECT a.new_ac_number FROM acmaster a
                                   WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,140,141,142,143,144,145)))
ORDER BY cv.ac_glcode, cv.transaction_date, cv.cumvouch_det_id;
```

If it returns nothing, the difference is in the GL opening balance from the migration, and the second question is what the GL opening balance for each of the two codes was on 31 July 2024 and how it was arrived at.

Why it matters. The 600,000 net is immaterial to any EIR figure. It matters because the reconciliation of the engine to the trial balance has to tie to the cent to be useful to the auditors, and an explained 400,000 is a reconciling item, while an unexplained one is a question mark against the whole control.

USER (2026-10-07 09:18:17 UTC)

Can you write a plain language report to Barry to explain these account differences like a teacher and the query to run

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\gl_note.py
--- content ---
"""Plain-language note to Barry on the two FInES GL differences, with the queries to run."""
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Table, TableStyle, Preformatted, Image

F = r"C:\Windows\Fonts"
pdfmetrics.registerFont(TTFont("Seg", F + r"\segoeui.ttf")); pdfmetrics.registerFont(TTFont("Seg-B", F + r"\segoeuib.ttf"))
pdfmetrics.registerFont(TTFont("Seg-I", F + r"\segoeuii.ttf")); pdfmetrics.registerFont(TTFont("Seg-BI", F + r"\segoeuiiz.ttf"))
pdfmetrics.registerFontFamily("Seg", normal="Seg", bold="Seg-B", italic="Seg-I", boldItalic="Seg-BI")
pdfmetrics.registerFont(TTFont("Mono", F + r"\consola.ttf"))
NAVY = colors.HexColor("#1B2A41"); ORANGE = colors.HexColor("#E07B00"); GREY = colors.HexColor("#5B6470"); LIGHT =
colors.HexColor("#F4F6F9"); LINE = colors.HexColor("#C9D1DB"); AMBER = colors.HexColor("#FFF4E0")
st = {"title": ParagraphStyle("t", fontName="Seg-B", fontSize=20, leading=25, textColor=NAVY, spaceAfter=4),
      "meta": ParagraphStyle("m", fontName="Seg", fontSize=9.5, leading=13, textColor=GREY, spaceAfter=10),
      "h": ParagraphStyle("h", fontName="Seg-B", fontSize=12, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4,
keepWithNext=1),
      "p": ParagraphStyle("p", fontName="Seg", fontSize=10.5, leading=15, spaceAfter=7),
      "cell": ParagraphStyle("c", fontName="Seg", fontSize=9.2, leading=12), "head": ParagraphStyle("hd", fontName="Seg-B",
fontSize=9.2, leading=12, textColor=colors.white),
      "mono": ParagraphStyle("mo", fontName="Mono", fontSize=8.3, leading=10.6)}
W = A4[0] - 36 * mm
s = []
def P(t): s.append(Paragraph(t, st["p"]))
def H(t): s.append(Paragraph(t, st["h"]))
def TAB(header, rows, widths):
    data = [[Paragraph(h, st["head"]) for h in header]] + [[Paragraph(str(c), st["cell"]) for c in r] for r in rows]
    t = Table(data, colWidths=[W * x for x in widths], repeatRows=1)
    sty = [{"BACKGROUND", (0, 0), (-1, 0), NAVY}, {"VALIGN", (0, 0), (-1, -1), "TOP"}, {"LEFTPADDING", (0, 0), (-1, -1), 5},
{"RIGHTPADDING", (0, 0), (-1, -1), 5},
{"TOPPADDING", (0, 0), (-1, -1), 4}, {"BOTTOMPADDING", (0, 0), (-1, -1), 5}, {"LINEBELOW", (0, 0), (-1, -1), 0.4,
LINE}, {"BOX", (0, 0), (-1, -1), 0.6, LINE}]
    for i in range(2, len(data), 2): sty.append(("BACKGROUND", (0, i), (-1, i), LIGHT))
    t.setStyle(TableStyle(sty)); s.append(t); s.append(Spacer(1, 8))
def CODE(t):
    tb = Table([Preformatted(t, st["mono"])], colWidths=[W])
    tb.setStyle(TableStyle([{"BACKGROUND", (0, 0), (-1, -1), LIGHT}, {"BOX", (0, 0), (-1, -1), 0.5, LINE}, {"LEFTPADDING", (0,
0), (-1, -1), 8}, {"RIGHTPADDING", (0, 0), (-1, -1), 8}, {"TOPPADDING", (0, 0), (-1, -1), 6}, {"BOTTOMPADDING", (0, 0), (-1,
-1), 6}]))
    s.append(tb); s.append(Spacer(1, 8))

LOGO = r"C:\Users\wadza\OneDrive\2026\Tenders\PFA\Excel & Data Analysis Training\Basic Excel Course\3 Source Files\img\
dupleix-logo.png"
try:
    im = Image(LOGO); r = im.imageHeight / im.imageWidth; im.drawWidth = 36 * mm; im.drawHeight = 36 * mm * r; im.hAlign =
"LEFT"; s.append(im); s.append(Spacer(1, 8))
except Exception: pass
s.append(Paragraph("Two small differences between the loan accounts and the general ledger", st["title"]))
s.append(Paragraph("A note for Barry Makumba from Dupleix Institute · 7 October 2026 · with the two queries that will
settle it", st["meta"]))

H("1. First, thank you")
P("The eighteen files you sent this morning are the best data we have had on this project. We have been able to check them
against each other and against "
  "the monthly trial balances, and almost everything ties to the cent. This note is about the one thing that does not, because
it is small, it is interesting, "
  "and you are the only person who can see the cause.")

H("2. The idea behind the check, in one picture")
P("A general ledger code such as 1050201 (FInES Agricultural Loans) is a drawer. Inside the drawer are the individual loan
accounts. The balance on the drawer "
  "should always equal the sum of the balances of the accounts inside it, because every posting that changes a loan account
also changes the drawer by the "
  "same amount. Like this:")
TAB(["", "Loan accounts inside GL 1050201 (illustration)", "Balance"], [
    ["", "Account A", "1,000"], ["", "Account B", "2,500"], ["", "Account C", "500"],
    ["", "<b>Sum of the accounts</b>", "<b>4,000</b>"], ["", "<b>Balance of GL 1050201 in the trial balance</b>", "<b>4,000</
b>"],
], [0.05, 0.70, 0.25])
P("We did exactly this with your files. We took every posting in the ledger file (P1_01), added them up account by account,
then GL by GL, at each of the "
  "twenty month-ends from January 2025 to August 2026, and compared the total with the trial balance for that month.")

H("3. What we found")
TAB(["GL code", "Name", "Result over the 20 month-ends"], [
    ["1050101", "MAIIC Agricultural Loans", "Equal to the cent in every month"],
    ["1050102", "MAIIC Industrial Loans", "Equal to the cent in every month"],
    ["1050401", "MAIIC Term Loan", "Equal to the cent in every month"],
    ["1050201", "FInES Agricultural Loans", "The trial balance is <b>400,000.00 higher</b> than the accounts, in every month"],
    ["1050202", "FInES Industrial Loans", "The trial balance is <b>1,000,000.00 lower</b> than the accounts, in every month"],
], [0.14, 0.30, 0.56])
P("Three of the five drawers are perfect, which tells us the ledger file is complete for them. The two FInES drawers each carry
a difference, and the "
```

```

"difference is the same figure in January 2025 as in August 2026.")

H("4. Why \"the same every month\" is the clue")
P("If postings were missing from the file as the months went by, the difference would move: it would be one number in March and another in July. A "
    "difference that never moves means nothing has gone wrong since January 2025. Whatever caused it happened once, before January 2025, and has sat in "
    "the drawer ever since. The round amounts, 400,000 and 1,000,000, point the same way: they look like entries made by hand, not like interest or repayments.")
P("So we are not looking for a flaw in your extract. We are looking for one or two old entries that sit in the GL but not in any loan account.")

H("5. The three places it can be")
TAB(["Possibility", "What it would look like", "How the first query shows it"], [
    ["A journal posted straight to the GL code, with no loan account number", "A posting on 1050201 or 1050202 whose account number is blank", "It appears in the query result with an empty NEW_AC_NUMBER"],
    ["A posting on an account that is not one of the 184 loans we extract", "A loan on a scheme whose name does not start with MAIIC or FInES, but which sits on a FInES GL code", "It appears in the query result with an account number we do not have"],
    ["A difference from the July 2024 take-on", "The GL opening balance was loaded with one figure and the loan accounts with another", "The first query returns nothing; the second query shows the opening balance"],
], [0.30, 0.38, 0.32])

H("6. The two queries to run")
P("Both are plain SELECTs. They read, and change nothing. Run the session settings first as before, then export each result as CSV.")
P("<b>Query 1: postings on the two GL codes that do not belong to one of our 184 accounts.</b> Save as GL_01_fines_postings_without_account.csv.")
CODE("""SELECT cv.*
FROM   cumvouch cv
WHERE  cv.ac_glcode IN ('1050201', '1050202')
AND    cv.delete_flag = 'N'
AND    (cv.new_ac_number IS NULL
        OR cv.new_ac_number NOT IN (SELECT a.new_ac_number FROM acmaster a
                                   WHERE a.scheme_mst_id IN (84,85,86,87,88,89,90,91,92,93,94,95,
                                                           140,141,142,143,144,145)))

ORDER BY cv.ac_glcode, cv.transaction_date, cv.cumvouch_det_id;""")
P("<b>Query 2: the GL opening balances loaded at take-on.</b> Save as GL_02_fines_opening_balances.csv. The table GL_OPENING_BALANCE appeared in your "
    "dictionary results; if its GL code column has a different name, the error message will say so.")
CODE("""SELECT *
FROM   gl_opening_balance
WHERE  glcode IN ('1050101', '1050102', '1050201', '1050202', '1050401')
ORDER BY glcode;""")
P("If query 1 returns rows, the sum of their amounts on each GL code should be 400,000 and 1,000,000 (allowing for sign), and the question is answered. If it "
    "returns nothing, query 2 shows the opening balances, and the question becomes how those two figures were arrived at in July 2024.")

H("7. Why we care about 600,000 in a book of 15 billion")
P("The net difference is 600,000 on a loan book of about MWK 15 billion, so it changes no result. It matters for a different reason. The engine we are building "
    "has to reconcile to the trial balance to the cent so that the auditors can rely on it. A difference we can name is a reconciling item they will accept in a "
    "sentence. A difference nobody can explain is a question mark against the whole control. Two queries now save a long conversation later.")
s.append(Spacer(1, 6))
s.append(Paragraph("Thank you again. The results have moved the project further in a day than the previous two months.", st["p"]))
s.append(Paragraph("Edward Mazibuko and Wadzanai Rombe, Duplex Institute", st["meta"]))

def footer(c, d):
    c.saveState(); c.setStrokeColor(LINE); c.setLineWidth(0.5); c.line(18 * mm, 14 * mm, A4[0] - 18 * mm, 14 * mm)
    c.setFont("Seg", 8); c.setFillColor(GREY); c.drawString(18 * mm, 9.5 * mm, "Note to Barry: the two FInES GL differences · Duplex Institute · 7 October 2026")
    c.drawRightString(A4[0] - 18 * mm, 9.5 * mm, f"Page {d.page}"); c.restoreState()
OUT = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to Barry - the two FInES GL differences - 7 Oct 2026.pdf"
doc = BaseDocTemplate(OUT, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=20 * mm, title="Note to Barry: the two FInES GL differences", author="Duplex Institute")
doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, id="f", leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)], onPage=footer)])
doc.build(s); print("written", OUT)

```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\gl_note.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Build the note to Barry as a PDF and check the opening-balance table's column name

```
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python gl_note.py && ./songenv/Scripts/python -c "
import pypdfium2 as pdfium
p = r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to Barry - the two FInES GL differences - 7 Oct 2026.pdf'
pdf = pdfium.PdfDocument(p); print('pages', len(pdf)); pdf[1].render(scale=0.9).to_pil().save('gl_p2.png')
" && grep -i "GLCODE\|GL_CODE" "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Request Responses 6 Oct 2026/Dupleix/Dupleix/DD_03_columns.csv" | grep "GL_OPENING_BALANCE\" | cut -d'"'"' -f4,8 | head -3; cp gl_note.py "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Build files/"
{
  "timeout": 120000
}
```

Result of Bash

written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to Barry - the two FInES GL differences - 7 Oct 2026.pdf
pages 3
GL_OPENING_BALANCE"GLCODE
G_GL_OPENING_BALANCE"GLCODE
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

CLAUDE (2026-10-07 09:19:36 UTC)

The check I just ran was a small one: before putting Query 2 in the note, I looked up the GL_OPENING_BALANCE table in Barry's dictionary results (DD_03) to confirm that its GL code column really is called GLCODE. It is, so the query in the note will run as written and Barry won't hit a "column does not exist" error.

The bigger check the note itself explains is the drawer-and-contents test: for each loan GL code, the balance in the trial balance should equal the sum of the balances of the loan accounts inside it. Three GLs pass to the cent every month; the two FInES GLs carry a fixed 400,000 and 1,000,000 difference that never moves, so it dates from before January 2025.

Written for: Barry, in plain language, with a worked illustration.

The note is saved as a 3-page PDF:

- Why the check works, with a one-picture example of a GL "drawer" and the accounts inside it.
- What was found, GL by GL.
- Why "the same difference every month" means a single old entry rather than a flaw in his extract.
- The three places it can be, and how each query reveals it.
- The two queries, with file names to save under.
- Why 600,000 matters in a 15 billion book: an explained reconciling item against an unexplained one.

Files are in 2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC:

- Note to Barry - the two FInES GL differences - 7 Oct 2026.pdf

USER (2026-10-07 09:21:13 UTC)

Give me the full path for theis file

CLAUDE (2026-10-07 09:21:17 UTC)

C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to Barry - the two FInES GL differences - 7 Oct 2026.pdf

USER (2026-10-07 09:33:48 UTC)

open the pdf file

Tool call: PowerShell

description: Open the note to Barry in the default PDF viewer

```
--- command ---
Invoke-Item -LiteralPath "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to Barry - the two FInES GL differences - 7 Oct 2026.pdf"
```

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-07 09:34:05 UTC)

It should now be open in your PDF viewer.

USER (2026-10-07 09:40:50 UTC)

the missing Zaithwa Farms posting => explain more. What do you mean by missing

CLAUDE (2026-10-07 09:41:25 UTC)

"Missing" means this: two independent records of the same account disagree by exactly one amount from one month, and that amount does not appear as a posting in the ledger file.

The two records. E-Banker keeps the ledger (every posting) and, separately, a month-end balance table. For 130 of the 131 accounts in the balance table, adding up the ledger postings gives the month-end balance exactly, every month, 2,533 account-months in all. Zaithwa Farms (000104450000015) is the one exception.

What the numbers show. Up to April 2025 the two agree. From May 2025 the balance table is 1,827,633.62 better for the customer than the ledger, and stays so:

Month-end	Balance table	Sum of ledger postings	Difference
May 2025	27,303.21 owed	1,854,936.83 owed	1,827,633.62
Jun to Jul 2025	658,534.33 in credit	1,169,099.29 owed	1,827,633.62
Aug 2025 onward	3,053,316.65 in credit	1,225,683.03 in credit	1,827,633.62

So in May 2025 something reduced what the customer owed by 1,827,633.62 in the balance table, and there is no posting of that amount for that account in the ledger file. (Both records agree the customer has been in credit, meaning overpaid, since August 2025; they disagree on how much.)

What "missing" could mean, in order of likelihood:

1. The posting exists in the ledger but was excluded from the extract, most likely because it carries DELETE_FLAG = 'Y' (the extract asked for 'N' only), or because it sits on a different sub-account number.
2. The posting was made to a different account number, for example the old account number, and the balance table was updated for this one.
3. Someone corrected the balance table directly without a posting. That would be a control weakness worth knowing about.

What Barry can run. First, every posting for the account, including deleted ones:

```
SELECT cv.*
FROM cumvouch cv
WHERE cv.new_ac_number = '000104450000015'
ORDER BY cv.transaction_date, cv.cumvouch_det_id;
```

Then, any posting of that amount in May 2025 on any account, in case it was posted elsewhere:

```
SELECT cv.*
FROM cumvouch cv
WHERE ABS(cv.transamt) = 1827633.62
AND cv.transaction_date BETWEEN DATE '2025-04-01' AND DATE '2025-06-30'
ORDER BY cv.transaction_date;
```

Why it matters. Not for the money, which is under two million kwacha. It matters because the balance table is the figure MAIIC reports, and the ledger is the figure the engine will calculate from. For 130 accounts they are the same thing; for this one they are not, and we need to know which is right before the engine treats this customer as 1.2 million or 3.1 million in credit.

USER (2026-10-07 09:43:51 UTC)

write a plain language pdf again for this, including the alter date queries and explaining fully why we need these queries run

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\zaithwa_note.py
--- content ---
"""Plain-language note to Barry on the Zaithwa Farms balance difference, with the queries to run."""
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Table, TableStyle, Preformatted, Image

F = r"C:\Windows\Fonts"
pdfmetrics.registerFont(TTFont("Seg", F + r"\segoeui.ttf")); pdfmetrics.registerFont(TTFont("Seg-B", F + r"\segoeuib.ttf"))
pdfmetrics.registerFont(TTFont("Seg-I", F + r"\segoeuii.ttf")); pdfmetrics.registerFont(TTFont("Seg-BI", F + r"\segoeuiiz.ttf"))
pdfmetrics.registerFontFamily("Seg", normal="Seg", bold="Seg-B", italic="Seg-I", boldItalic="Seg-BI")
pdfmetrics.registerFont(TTFont("Mono", F + r"\consola.ttf"))
NAVY = colors.HexColor("#1B2A41"); ORANGE = colors.HexColor("#E07B00"); GREY = colors.HexColor("#5B6470"); LIGHT =
colors.HexColor("#F4F6F9"); LINE = colors.HexColor("#C9D1DB")
st = {"title": ParagraphStyle("t", fontName="Seg-B", fontSize=20, leading=25, textColor=NAVY, spaceAfter=4),
      "meta": ParagraphStyle("m", fontName="Seg", fontSize=9.5, leading=13, textColor=GREY, spaceAfter=10),
      "h": ParagraphStyle("h", fontName="Seg-B", fontSize=12, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4,
keepWithNext=1),
      "p": ParagraphStyle("p", fontName="Seg", fontSize=10.5, leading=15, spaceAfter=7),
      "cell": ParagraphStyle("c", fontName="Seg", fontSize=9.2, leading=12), "head": ParagraphStyle("hd", fontName="Seg-B",
fontSize=9.2, leading=9.2, textColor=colors.white),
      "mono": ParagraphStyle("mo", fontName="Mono", fontSize=8.3, leading=10.6)}
W = A4[0] - 36 * mm
s = []
def P(t): s.append(Paragraph(t, st["p"]))
def H(t): s.append(Paragraph(t, st["h"]))
def TAB(header, rows, widths):
    data = [[Paragraph(h, st["head"]) for h in header]] + [[Paragraph(str(c), st["cell"]) for c in r] for r in rows]
    t = Table(data, colWidths=[W * x for x in widths], repeatRows=1)
    sty = [{"BACKGROUND", (0, 0), (-1, 0), NAVY}, {"VALIGN", (0, 0), (-1, -1), "TOP"}, {"LEFTPADDING", (0, 0), (-1, -1), 5},
{"RIGHTPADDING", (0, 0), (-1, -1), 5},
{"TOPPADDING", (0, 0), (-1, -1), 4}, {"BOTTOMPADDING", (0, 0), (-1, -1), 5}, {"LINEBELOW", (0, 0), (-1, -1), 0.4,
LINE}, {"BOX", (0, 0), (-1, -1), 0.6, LINE}]
    for i in range(2, len(data), 2): sty.append(("BACKGROUND", (0, i), (-1, i), LIGHT))
    t.setStyle(TableStyle(sty)); s.append(t); s.append(Spacer(1, 8))
def CODE(t):
    tb = Table([[Preformatted(t, st["mono"])]], colWidths=[W])
    tb.setStyle(TableStyle([{"BACKGROUND", (0, 0), (-1, -1), LIGHT}, {"BOX", (0, 0), (-1, -1), 0.5, LINE}, {"LEFTPADDING", (0,
0), (-1, -1), 8}, {"RIGHTPADDING", (0, 0), (-1, -1), 8}, {"TOPPADDING", (0, 0), (-1, -1), 6}, {"BOTTOMPADDING", (0, 0), (-1,
-1), 6}]))
    s.append(tb); s.append(Spacer(1, 8))

LOGO = r"C:\Users\wadza\OneDrive\2026\Tenders\PFA\Excel & Data Analysis Training\Basic Excel Course\3 Source Files\img\
dupleix-logo.png"
try:
    im = Image(LOGO); r = im.imageHeight / im.imageWidth; im.drawWidth = 36 * mm; im.drawHeight = 36 * mm * r; im.hAlign =
"LEFT"; s.append(im); s.append(Spacer(1, 8))
except Exception: pass
s.append(Paragraph("One account where the balance table and the ledger disagree: Zaithwa Farms", st["title"]))
s.append(Paragraph("A note for Barry Makumba from Dupleix Institute · 7 October 2026 · with the queries that will settle
it", st["meta"]))

H("1. The check this came from")
P("E-Banker keeps two records of every loan. The first is the ledger (the table CUMVOUCH): every posting ever made, one row
each. The second is the balance "
    "table (ACCOUNT_BALANCE): one row per account per month-end, with the balance on that day. The second should always be the
sum of the first, because a "
    "month-end balance is nothing more than every posting up to that day added together.")
P("We tested this with the two files you sent, P1_01 (the ledger) and P2_09 (the balance table): 131 accounts, 2,549
account-months. For 130 accounts the "
    "two agree to the cent in every month. That is a strong result; it means the ledger file is complete and the balance table is
faithful to it. "
    "One account is the exception.")

H("2. What the numbers show for Zaithwa Farms, account 000104450000015")
P("Up to April 2025 the two records agree. From May 2025 they differ by exactly 1,827,633.62, and they go on differing by that
same amount every month "
    "after that. In the table below, \"owed\" means the customer owes MAIIC; \"in credit\" means the customer has paid more than
was owed.")
TAB(["Month-end", "Balance table (P2_09)", "Sum of the ledger postings (P1_01)", "Difference"], [
    ["April 2025 and earlier", "agree", "agree", "nil"],
    ["May 2025", "27,303.21 owed", "1,854,936.83 owed", "1,827,633.62"],
    ["June and July 2025", "658,534.33 in credit", "1,169,099.29 owed", "1,827,633.62"],
    ["August 2025 onward", "3,053,316.65 in credit", "1,225,683.03 in credit", "1,827,633.62"],
], [0.22, 0.26, 0.30, 0.22])
P("In words: during May 2025 something reduced what the customer owed by 1,827,633.62 in the balance table, and no posting of
that amount for that account "
    "is in the ledger file. Both records agree the customer has been in credit since August 2025; they disagree on by how much.")

H("3. What \"missing\" means, and the three places the entry can be")
P("We are not saying money is missing. We are saying a posting is missing from the file, or was never made. There are three
ways that happens:")
TAB(["Possibility", "What it would look like", "Which query shows it"], [
    ["The posting exists but was left out of the extract", "A row on this account marked deleted (DELETE_FLAG = Y), which our
```

```

extract excluded, or a row on a different sub-account number", "Query 1 lists every row on the account, deleted or not, on
every sub-account"],
["The posting was made to a different account number", "A credit of 1,827,633.62 in May 2025 on another account, for
example the old account number, while the balance table was updated for this one", "Query 2 finds any posting of that amount in
April to June 2025, on any account"],
["The balance table was changed without a posting", "Queries 1 and 2 both return nothing of the amount", "Then the balance
row itself is the only record. Query 3 shows who entered it and when"],
], [0.30, 0.42, 0.28])

H("4. Before the queries: the two date settings, and why")
P("Please run these two lines once, after logging in and before the queries. The files you sent on 7 October had their dates
written month first (7/15/2025). "
    "That is readable, but year-first (2025-07-15) is the form that no program can misread, and it is what we have asked for in
every request. The lines below "
    "ask your session to display dates that way and numbers with a dot for the decimal point.")
CODE("""ALTER SESSION SET NLS_DATE_FORMAT = 'YYYY-MM-DD HH24:MI:SS';
ALTER SESSION SET NLS_NUMERIC_CHARACTERS = '.,';""")
P("What they do: they change how <b>your session</b> displays dates and numbers. What they do not do: they change nothing in
the database, no table, no "
    "setting, no other user. They last only until you disconnect; the next login starts with the normal defaults, and there is
nothing to put back.")
P("One more thing we learned from the 7 October files: the export still came out month-first even though the settings were run.
That is because SQL "
    "Developer's export follows its own preference, under Tools, Preferences, Database, NLS, Date Format. If you set that
preference to YYYY-MM-DD once, every "
    "export from then on will be year-first and the session lines will not be needed again.")

H("5. The three queries")
P("All three are plain SELECTs. They read and change nothing. Export each result as CSV under the name shown, and please do not
open the files in Excel "
    "before sending: that is what swapped days and months in the August extracts.")
P("<b>Query 1: every posting on the account, including deleted rows and every sub-account.</b> Save as ZF_01_all_postings.csv.")
CODE("""SELECT cv.*
FROM   cumvouch cv
WHERE  cv.new_ac_number = '000104450000015'
ORDER BY cv.transaction_date, cv.cumvouch_det_id;""")
P("<b>Query 2: any posting of 1,827,633.62 between April and June 2025, on any account.</b> Save as ZF_02_amount_search.csv.")
CODE("""SELECT cv.*
FROM   cumvouch cv
WHERE  ABS(cv.transamt) = 1827633.62
AND    cv.transaction_date BETWEEN DATE '2025-04-01' AND DATE '2025-06-30'
ORDER BY cv.transaction_date, cv.cumvouch_det_id;""")
P("<b>Query 3: the balance table rows for the account, with the audit columns.</b> Save as ZF_03_balance_rows.csv. This shows
who entered each month-end "
    "row and when, which tells us whether the May 2025 row was produced by the month-end run or keyed afterwards.")
CODE("""SELECT b.*
FROM   account_balance b
WHERE  b.new_ac_number = '000104450000015'
ORDER BY b.transaction_date, b.account_bal_mst_id;""")

H("6. Why we need these run")
P("The amount is small: under two million kwacha on a book of fifteen billion. The reason is not the money.")
P("The balance table is the figure MAIIC reports. The ledger is the figure our engine will calculate from, posting by posting.
For 130 accounts those are the "
    "same thing, and that is what lets the engine's results be reconciled to MAIIC's books to the cent. For this one account they
are not the same thing, so "
    "before the engine runs we have to know which record is right: is the customer 1.2 million in credit, or 3.1 million? If a
posting was deleted or sits "
    "elsewhere, the ledger is simply incomplete for this account and we add the row. If the balance table was changed by hand,
that is something Finance "
    "and the auditors need to know about, and the engine will follow the ledger.")
P("Either way the answer is one line in our reconciliation instead of an unexplained difference, and the three queries take a
few minutes.")
s.append(Spacer(1, 6))
s.append(Paragraph("Thank you again for the 7 October files. They have let us tie the ledger to the balance table, to the
stored loan book and to the trial balance, and this account is the only one that did not tie.", st["p"]))
s.append(Paragraph("Edward Mazibuko and Wadzanai Rombe, Dupleix Institute", st["meta"]))

def footer(c, d):
    c.saveState(); c.setStrokeColor(LINE); c.setLineWidth(0.5); c.line(18 * mm, 14 * mm, A4[0] - 18 * mm, 14 * mm)
    c.setFont("Seg", 8); c.setFill(GREY); c.drawString(18 * mm, 9.5 * mm, "Note to Barry: Zaithwa Farms balance difference
    · Dupleix Institute · 7 October 2026")
    c.drawRightString(A4[0] - 18 * mm, 9.5 * mm, f"Page {d.page}"); c.restoreState()
OUT = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to
Barry - Zaithwa Farms balance difference - 7 Oct 2026.pdf"
doc = BaseDocTemplate(OUT, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=20 * mm,
title="Note to Barry: Zaithwa Farms balance difference", author="Dupleix Institute")
doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, id="f",
leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)], onPage=footer)])
doc.build(s); print("written", OUT)

```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\cclaude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\zaithwa_note.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Build the Zaithwa Farms note as a PDF and confirm the balance table has audit columns

```
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python zaithwa_note.py && ./songenv/Scripts/python -c "
import pypdfium2 as pdfium
p = r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to Barry - Zaithwa Farms balance difference - 7 Oct 2026.pdf'
print('pages', len(pdfium.PdfDocument(p)))" && cp zaithwa_note.py "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Build files/" && grep -c "ACCOUNT_BAL_MST_ID\|ENTER_USER_ID" "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Follow-Up Scripts Results/P2_09_balance_history.csv" | head -1
{
  "timeout": 120000
}
```

Result of Bash

written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to Barry - Zaithwa Farms balance difference - 7 Oct 2026.pdf
pages 3
1
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

CLAUDE (2026-10-07 09:45:08 UTC)

Written for: Barry, in plain language.

The note is saved as a 3-page PDF. It covers:

- **The check it came from:** the ledger and the month-end balance table are two records of the same thing, and they agree for 130 of 131 accounts.
- **The numbers for Zaithwa Farms,** month by month, showing the fixed 1,827,633.62 gap from May 2025 and what "owed" and "in credit" mean.
- **What "missing" means:** not missing money, but a posting that is either excluded from the extract (deleted flag or other sub-account), posted to another account number, or never made because the balance table was changed by hand. Each possibility is matched to the query that reveals it.
- **The two date settings,** with a full explanation: what they change (how his session displays dates and numbers), what they do not change (nothing in the database, no other user), that they vanish on disconnect, and why the 7 October files still came out month-first (SQL Developer's own preference overrides the session; set it once to YYYY-MM-DD).
- **Three queries:** every posting on the account including deleted rows; any posting of that amount in April to June 2025 on any account; and the balance table rows for the account with their audit columns, which show who entered the May row and when.
- **Why we need them run:** the balance table is what MAIIC reports, the ledger is what the engine calculates from, and for this account they differ, so we must know whether the customer is 1.2 million or 3.1 million in credit before the engine runs.

Full path:

C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to Barry - Zaithwa Farms balance difference - 7 Oct 2026.pdf

USER (2026-10-07 09:45:34 UTC)

open the pdf

Tool call: PowerShell

description: Open the Zaithwa Farms note in the default PDF viewer

```
--- command ---
Invoke-Item -LiteralPath "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Note to Barry - Zaithwa Farms balance difference - 7 Oct 2026.pdf"
```

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-07 09:45:41 UTC)

It should now be open in your PDF viewer.

Vendor: confirm code H. Credit: the six accounts drawn above sanction. => explain this more and how we can resolve it

Tool call: Bash

```
description: Identify the six accounts drawn above sanction and the H accounts' recorded reasons
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/
scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python - <<'PYEOF' 2>&1 | cut -c1-230
import pandas as pd
from fu_load import load
pd.set_option("display.width", 250); pd.set_option("display.max_colwidth", 70)
led, acm, alm, lb = (load(k) for k in ("ledger", "acm", "alm", "lb")); led["TRANSTYPE"] = led.TRANSTYPE.astype(int)
d = led[led.TRANSTYPE == 301].groupby("NEW_AC_NUMBER").TRANSAMT.sum().mul(-1)
a = alm.set_index("NEW_AC_NUMBER"); n = acm.set_index("NEW_AC_NUMBER")
over = d[d > a.SANCTION_AMOUNT.reindex(d.index) + 1]
lbx = lb.sort_values("TRANSACTION_DATE").drop_duplicates(["ASONDATE", "NEW_AC_NUMBER"], keep="last")
latest = lbx.sort_values("ASONDATE").groupby("NEW_AC_NUMBER").tail(1).set_index("NEW_AC_NUMBER")
for k, v in over.items():
    print(f"\n{k} {n.ACCOUNT_NAME[k]} | scheme {n.SCHEME_MST_ID[k]} | status {n.STATUS_CODE[k]} | opened
{n.ACCOUNT_OPEN_DATE[k].date()}")
    print(f"    sanction {a.SANCTION_AMOUNT[k]:.2f} (sanction date {a.SANCTION_DATE[k].date()}) | drawn {v:.2f} | excess {v -
a.SANCTION_AMOUNT[k]:.2f} | loan book APPROVED {latest.APPROVED.get(k, float('nan')):.2f} DISBURSD {latest.DISBURSD.get(k,
float('nan')):.2f}")
    for _, r in led[(led.NEW_AC_NUMBER == k) & (led.TRANSTYPE == 301)].iterrows():
        print(f"        {r.TRANSACTION_DATE.date()} {-r.TRANSAMT:>18,.2f} {r.PARTICULARS[:65]}")
print("\nH accounts:", list(n[n.STATUS_CODE == 'H'].index[:5]), "...")
h = n[n.STATUS_CODE == "H"]; print("H: STATUS_CHANGE_REASON values:", h.STATUS_CHANGE_REASON.value_counts().to_dict(), " |
ACCOUNT_STATUS_DATE range", h.ACCOUNT_STATUS_DATE.min().date(), h.ACCOUNT_STATUS_DATE.max().date())
PYEOF
{
  "timeout": 120000
}
```

Result of Bash

```
000104420000005 MUSECO | scheme 84 | status D | opened 2020-05-13
    sanction 370,000,000.00 (sanction date 2020-05-13) | drawn 405,097,989.86 | excess 35,097,989.86 | loan book APPROVED 0.00
DISBURSD 0.00
    2024-07-31      405,097,989.86 To Trf Opening Account Balance Disbursement amount

000104420000009 Mach Milk Limited | scheme 84 | status D | opened 2020-10-13
    sanction 65,023,440.00 (sanction date 2020-10-13) | drawn 81,075,926.78 | excess 16,052,486.78 | loan book APPROVED 81,075,
926.78 DISBURSD 81,075,926.78
    2024-07-31      81,075,926.78 To Trf Opening Account Balance Disbursement amount

000104430000002 Edge View Academy Limited | scheme 86 | status A | opened 2020-03-04
    sanction 300,000,000.00 (sanction date 2020-03-04) | drawn 331,395,845.48 | excess 31,395,845.48 | loan book APPROVED 300,
000,000.00 DISBURSD 331,395,845.48
    2024-07-31      331,395,845.48 To Trf Opening Account Balance Disbursement amount

000104430000054 VNC Bricks & Construction Group Ltd | scheme 86 | status D | opened 2022-10-19
    sanction 20,000,000.00 (sanction date 2022-10-19) | drawn 200,000,000.00 | excess 180,000,000.00 | loan book APPROVED 0.00
DISBURSD 0.00
    2024-07-31      200,000,000.00 To Trf Opening Account Balance Disbursement amount

000104450000020 Kayola Processors | scheme 90 | status H | opened 2024-03-01
    sanction 12,543,063.04 (sanction date 2024-03-01) | drawn 12,543,065.00 | excess 1.96 | loan book APPROVED 0.00 DISBURSD 0.00
    2024-07-31      12,543,065.00 To Trf Opening Account Balance Disbursement amount

000104460000070 Mchinji Civil Servants SACCO | scheme 92 | status H | opened 2023-07-31
    sanction 50,000,000.00 (sanction date 2023-07-20) | drawn 100,000,000.00 | excess 50,000,000.00 | loan book APPROVED 0.00
DISBURSD 0.00
    2024-07-31      100,000,000.00 To Trf Opening Account Balance Disbursement amount

H accounts: ['000104420000007', '000104430000004', '000104450000020', '000104450000022', '000104450000023'] ...
H: STATUS_CHANGE_REASON values: {'Paid Up': 21} | ACCOUNT_STATUS_DATE range 2025-12-11 2026-06-30
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Both are small, and the data now says what each one is.

Code H

What we know. 21 accounts carry status H. On every one of them the account master's own reason field says "Paid Up", with status dates from December 2025 to June 2026; 18 of the 21 have a nil balance in the ledger, and 17 have an interest stop date. The status history table never records H as a code: the last change it holds for these accounts is to D with the reason Paid Up or Foreclosure.

What is missing. Only the label. Everything in the data says H is "paid up, not yet formally closed", but the manual does not list it, so we are reading behaviour, not a definition. The three H accounts that still carry a small balance are the ones to check.

How to resolve it. One question to Virtual Galaxy, and it can go through Barry: "In E-Banker, what does account status code H stand for, and what has to happen for an H account to become C (Closed)?" Until the answer comes, the engine treats H and F as settled: no accrual, no profile, closing entries only. The answer will not change a number; it will let us write the status rule down with the vendor's words rather than ours.

The six accounts drawn above sanction

What the test did. For every account with drawdown postings in the ledger, I compared the total drawn (type 301) with the sanctioned amount in the loan master. On 112 accounts they are equal; on 13 the customer has drawn less; on 6 the drawn figure is higher. Those six are the ones I flagged.

What they turn out to be. All six are take-on loans. Each has a single drawdown line dated 31 July 2024, "Opening Account Balance Disbursement amount", and that line is the balance brought into E-Banker at migration, not the money originally lent. A migration balance can exceed the sanction because it includes interest that had been capitalised and arrears that had built up over the years before. So "drawn above sanction" is the wrong description: the ledger holds the take-on balance, not the original drawdown.

Account	Customer	Sanction	Take-on balance	Excess	Reading
000104420000005	MUSECO	370,000,000	405,097,989.86	35,097,989.86	Balance with capitalised interest
000104430000002	Edge View Academy	300,000,000	331,395,845.48	31,395,845.48	Same; the loan book shows approved 300m, disbursed 331.4m
000104420000009	Mach Milk	65,023,440	81,075,926.78	16,052,486.78	Same
000104450000020	Kayola Processors	12,543,063.04	12,543,065.00	1.96	Rounding
000104460000070	Mchinji Civil Servants SACCO	50,000,000	100,000,000	50,000,000	Exactly double: a sanction increase not recorded, or a wrong sanction figure
000104430000054	VNC Bricks and Construction	20,000,000	200,000,000	180,000,000	Exactly ten times: almost certainly a mis-keyed sanction (20,000,000 for 200,000,000)

How to resolve it. Two different actions:

- For the first three, nothing to resolve with Credit. The original drawdowns and the interest capitalised before July 2024 must come from the take-on workbook, which is the same request already with Tamanda. The engine will use the original drawdown for the EIR and the take-on balance as the carrying amount at 31 July 2024.
- For Mchinji and VNC Bricks, ask Credit (Dibwa) to confirm the approved amount from the offer letter and, if the loan master is wrong, have Barry correct SANCTION_AMOUNT. Both are also dormant or paid up, so the EIR effect is limited to history, but the loan master should say what the letter says.

Kayola is a two-kwacha rounding and needs nothing.

So the item on the list shrinks to: one vendor question, two offer letters to check, and a sanction figure to correct if the letters confirm it.

USER (2026-10-07 09:56:08 UTC)

Which offer letter to ask for now

CLAUDE (2026-10-07 09:56:24 UTC)

Two, both from Credit (Dibwa):

- Mchinji Civil Servants SACCO**, account 000104460000070, FlNES Industrial Loans, sanctioned 20 July 2023. The loan master says 50,000,000; the take-on balance was 100,000,000. We need the letter, and any later letter increasing the facility, to know whether the approved amount was 50m or 100m.
- VNC Bricks and Construction Group Ltd**, account 000104430000054, MAIIC Industrial Loans, sanctioned 19 October 2022. The loan master says 20,000,000; the take-on balance was 200,000,000. We need the letter to confirm whether the approved amount was 200,000,000 and the loan master is mis-keyed.

The other four need no letter: three are take-on balances that include capitalised interest, and one is a two-kwacha rounding.

USER (2026-10-07 09:58:08 UTC)

Can you write a full pdf document with the two offer letter requests including the manual 10 samples we asked for before in plain language. UUnderstood?

Tool call: PowerShell

description: List the offer letters already received

--- command ---

```
Get-ChildItem -LiteralPath "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Offer Letters" -Recurse -File  
| Select-Object LastWriteTime, Length, Name | Format-Table -AutoSize
```

Result of PowerShell

LastWriteTime	Length	Name
17/09/2026 14:52:48	244720	EBENEZER MIDIAN LTD.pdf
17/09/2026 14:51:53	183730	GOJET Investments.pdf
17/09/2026 14:52:05	193442	JAT GROUP LTD.pdf
17/09/2026 14:51:15	243782	JVD AGRO LIMITED.pdf
17/09/2026 14:53:37	250582	Lake Malawi Aquaculture Limited.pdf
17/09/2026 16:48:58	448430	MAIIC Offer Letters - Analysis Report.pdf
17/09/2026 15:43:59	33216	MAIIC Offer Letters - Loan Terms Analysis.xlsx
17/09/2026 17:09:40	182458	MAIIC Offer Letters - Repayment Schedules and EIR.xlsx
17/09/2026 14:53:24	234504	MALAWI POLICE SACCO.pdf
17/09/2026 14:52:12	182836	Microloan Foundation Limited.pdf
17/09/2026 14:52:33	243810	Milele Agroprocessing Limited.pdf
17/09/2026 14:53:30	234657	Mphunzitsi SACCO.pdf
17/09/2026 14:52:26	207673	PROMENADE MEDICAL CENTRE.pdf

Tool call: Write

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\
scratchpad\offer_note.py
--- content ---
"""Plain-language request to Credit for the signed offer letters: the ten sample loans and the two sanction checks."""
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.lib import colors
from reportlab.lib.styles import ParagraphStyle
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.platypus import BaseDocTemplate, PageTemplate, Frame, Paragraph, Spacer, Table, TableStyle, Image, ListFlowable,
ListItem

F = r"C:\Windows\Fonts"
pdfmetrics.registerFont(TTFont("Seg", F + r"\segoeui.ttf")); pdfmetrics.registerFont(TTFont("Seg-B", F + r"\segoeuib.ttf"))
pdfmetrics.registerFont(TTFont("Seg-I", F + r"\segoeuii.ttf")); pdfmetrics.registerFont(TTFont("Seg-BI", F + r"\segoeuiiz.ttf"))
pdfmetrics.registerFontFamily("Seg", normal="Seg", bold="Seg-B", italic="Seg-I", boldItalic="Seg-BI")
NAVY = colors.HexColor("#1B2A41"); ORANGE = colors.HexColor("#E07B00"); GREY = colors.HexColor("#5B6470"); LIGHT =
colors.HexColor("#F4F6F9"); LINE = colors.HexColor("#C9D1DB"); AMBER = colors.HexColor("#FFF4E0")
st = {"title": ParagraphStyle("t", fontName="Seg-B", fontSize=20, leading=25, textColor=NAVY, spaceAfter=4),
      "meta": ParagraphStyle("m", fontName="Seg", fontSize=9.5, leading=13, textColor=GREY, spaceAfter=10),
      "h": ParagraphStyle("h", fontName="Seg-B", fontSize=12, leading=15, textColor=ORANGE, spaceBefore=10, spaceAfter=4,
keepWithNext=1),
      "p": ParagraphStyle("p", fontName="Seg", fontSize=10.5, leading=15, spaceAfter=7), "li": ParagraphStyle("li",
fontName="Seg", fontSize=10.5, leading=15),
      "cell": ParagraphStyle("c", fontName="Seg", fontSize=9.2, leading=12), "head": ParagraphStyle("hd", fontName="Seg-B",
fontSize=9.2, leading=12, textColor=colors.white),
      "box": ParagraphStyle("b", fontName="Seg", fontSize=10.5, leading=15, spaceAfter=3)}
W = A4[0] - 36 * mm
s = []
def P(t): s.append(Paragraph(t, st["p"]))
def H(t): s.append(Paragraph(t, st["h"]))
def BUL(items):
    s.append(ListFlowable([ListItem(Paragraph(i, st["li"]), leftIndent=14, spaceAfter=3) for i in items], bulletType="bullet",
start="•", leftIndent=14, bulletColor=ORANGE, bulletFontSize=9)); s.append(Spacer(1, 4))
def TAB(header, rows, widths):
    data = [[Paragraph(h, st["head"]) for h in header]] + [[Paragraph(str(c), st["cell"]) for c in r] for r in rows]
    t = Table(data, colWidths=[W * x for x in widths], repeatRows=1)
    sty = [{"BACKGROUND", (0, 0), (-1, 0), NAVY}, {"VALIGN", (0, 0), (-1, -1), "TOP"}, {"LEFTPADDING", (0, 0), (-1, -1), 5},
{"RIGHTPADDING", (0, 0), (-1, -1), 5},
{"TOPPADDING", (0, 0), (-1, -1), 4}, {"BOTTOMPADDING", (0, 0), (-1, -1), 5}, {"LINEBELOW", (0, 0), (-1, -1), 0.4,
LINE}, {"BOX", (0, 0), (-1, -1), 0.6, LINE}]
    for i in range(2, len(data), 2): sty.append(("BACKGROUND", (0, i), (-1, i), LIGHT))
    t.setStyle(TableStyle(sty)); s.append(t); s.append(Spacer(1, 8))
def BOX(paras):
    t = Table([[[Paragraph(x, st["box"]) for x in paras]]], colWidths=[W])
    t.setStyle(TableStyle([{"BACKGROUND", (0, 0), (-1, -1), AMBER}, {"LINEBEFORE", (0, 0), (0, -1), 3, ORANGE}, {"LEFTPADDING",
(0, 0), (-1, -1), 10}, {"RIGHTPADDING", (0, 0), (-1, -1), 10}, {"TOPPADDING", (0, 0), (-1, -1), 8}, {"BOTTOMPADDING", (0, 0),
(-1, -1), 6}]))
    s.append(t); s.append(Spacer(1, 8))

LOGO = r"C:\Users\wadza\OneDrive\2026\Tenders\PFA\Excel & Data Analysis Training\Basic Excel Course\3 Source Files\img\
dupleix-logo.png"
try:
    im = Image(LOGO); r = im.imageHeight / im.imageWidth; im.drawWidth = 36 * mm; im.drawHeight = 36 * mm * r; im.hAlign =
"LEFT"; s.append(im); s.append(Spacer(1, 8))
except Exception: pass
s.append(Paragraph("The offer letters we are asking Credit for", st["title"]))
s.append(Paragraph("A note for Dibwa and the Credit team from Dupleix Institute · 7 October 2026 · twelve letters in two
groups, and why each one matters", st["meta"]))

H("1. Why offer letters, when we now have the system data")
P("Since Monday we have received the full data dictionary of E-Banker and eighteen extracts from it, and we can now tie the
loan ledger to the trial balance "
  "to the cent. That settles what the <b>system</b> holds. It does not settle what the <b>contract</b> says, and the effective
interest rate (EIR) is "
  "calculated on the contract: the amount approved, the rate and how it is built up, the fees deducted, and the repayment terms
the customer signed for.")
P("The system record and the signed letter are not always the same. We have already found two of the ten sample loans where the
rate in the core differs "
  "from the rate typed on the letter (Ebenezer Midian 31% in the core against 25.1% on the letter; JAT Group 33% against 25.3%),
and two take-on loans where "
  "the approved amount in the system looks mis-keyed. Only the signed letter can say which is right. That is why we ask for the
letters themselves.")
BOX(["<b>What \"signed offer letter\" means here:</b> the final version accepted and signed by the customer, with every
schedule and annex, as kept on the "
  "credit file. A draft or system-generated letter is useful but is not the contract. The ten letters we received on 17
September appear to be the "
  "unsigned issue versions; if any of them is in fact the signed version, please just say so and we will not ask again for
that one."])

H("2. Group A: the signed letters for the ten sample loans")
P("These are the ten loans we agreed to follow end to end, from credit approval to the ledger. For each we need the signed
letter so that we can read, "
  "from the contract itself:")
BUL(["the <b>approved amount</b> and the drawdown terms (single or in tranches);",
  "the <b>interest rate</b>, and how it is made up: the reference rate, the spread above it (the margin), and whether the
rate is fixed or moves with the Reserve Bank rate;",
  "the <b>fees</b>: arrangement, legal, insurance and any other charge deducted from the advance, with the amount or
```

```

percentage;",
    "the <b>repayment terms</b>: frequency, first instalment date, tenor, and any moratorium on principal or on interest;",
    "the <b>signature date</b>, which fixes the contract date for the EIR.")]
TAB(["#", "Customer", "Account", "Product", "Why this one"], [
    ["1", "Ebenezer Midian Ltd", "000104430000084", "MAIIC Industrial", "Core rate 31% against 25.1% on the letter we hold;
five tranches; the loan we have rebuilt in most detail"],
    ["2", "GOJET Investments", "000104450000098", "FinES Agricultural", "Originated after July 2026; no interest posted yet"],
    ["3", "JAT Group Ltd", "000104430000087", "MAIIC Industrial", "Core rate 33% against 25.3% on the letter; drawdowns dated
before the application"],
    ["4", "JVD Agro Limited", "000104460000120", "FinES Industrial", "Capitalising moratorium; interest rebuilds to the cent
from April 2026"],
    ["5", "Lake Malawi Aquaculture Limited", "000106050000009", "MAIIC Term Loan", "Approved 1,055m, drawn 297m: the part-drawn
case"],
    ["6", "Malawi Police SACCO", "000106050000006", "MAIIC Term Loan", "Fees of 90m arrangement and 49.3m legal deducted at
drawdown in January 2026"],
    ["7", "Microloan Foundation Limited", "000104460000116", "FinES Industrial", "First-month interest matches to the cent;
clean test case"],
    ["8", "Milele Agroprocessing Limited", "000106050000007", "MAIIC Term Loan", "Drawn in several tranches inside the month"],
    ["9", "Mphunzitsi SACCO", "000106050000014", "MAIIC Term Loan", "Originated after July 2026; 30m arrangement fee at
drawdown"],
    ["10", "Promenade Medical Centre", "000106050000004", "MAIIC Term Loan", "No interest posted for two months after drawdown,
then a catch-up posting"],
], [0.04, 0.26, 0.17, 0.17, 0.36])
P("Where a loan was varied after signature (a rate change, a restructure, a top-up), please include the variation letter as
well. Where fees were not on the "
    "letter but were deducted, a note of the amount and the authority for it is enough.")

H("3. Group B: two letters to settle an approved amount")
P("When E-Banker went live, each existing loan was loaded with one opening balance line dated 31 July 2024. For most loans that
balance is a little above the "
    "approved amount, because it includes interest capitalised over the years before, and that is expected. Two loans are
different: the opening balance is a "
    "round multiple of the approved amount in the system, which looks like either a facility increase that was never recorded or
a mis-keyed figure.")
TAB(["Customer", "Account", "Product", "Approved amount in the system", "Balance loaded on 31 July 2024", "What the letter will
tell us"], [
    ["Mchinji Civil Servants SACCO", "000104460000070", "FinES Industrial, sanctioned 20 July 2023", "50,000,000", "100,000,
000", "Whether the facility was 50m or 100m, or was increased by a later letter"],
    ["VNC Bricks and Construction Group Ltd", "000104430000054", "MAIIC Industrial, sanctioned 19 October 2022", "20,000,000",
"200,000,000", "Whether 20,000,000 in the system is a typing slip for 200,000,000"],
], [0.20, 0.15, 0.19, 0.14, 0.14, 0.18])
P("For each, please send the signed letter and any later letter that changed the amount. If the letter shows the system figure
is wrong, Barry can correct "
    "the sanction amount in the loan master, and the engine will then agree with the contract. Both loans are no longer accruing
(one dormant, one paid up), so "
    "this affects history and the audit trail rather than current income, but the record should say what the letter says.")

H("4. What we will do with them")
BUL(["Read every term into a table, loan by loan, next to the system's version of the same term, and list every difference.",
    "Use the letter's fees and rate build-up to calculate each loan's effective interest rate, and reconcile the interest that
rate produces to what the ledger posted.",
    "Bring the differences to you and Dr Thom with a recommendation on which record the engine should treat as the contract,
so that the choice is written down before the first results go out.",
    "Return the two sanction questions to Barry with the letter's figure, so that the loan master can be corrected if
needed."])

H("5. How to send them")
P("PDF scans are fine; one file per loan, named with the customer's name, is ideal. If the file holds confidential personal
details beyond the loan terms, "
    "those pages can be left out; we need the facility terms, the fee schedule and the signature page.")
P("Thank you. The ten sample loans are the thread that runs through the whole engine, and the letters are the one document the
system cannot give us.")
s.append(Spacer(1, 6))
s.append(Paragraph("Edward Mazibuko and Wadzanai Rombe, Dupleix Institute", st["meta"]))

def footer(c, d):
    c.saveState(); c.setStrokeColor(LINE); c.setLineWidth(0.5); c.line(18 * mm, 14 * mm, A4[0] - 18 * mm, 14 * mm)
    c.setFont("Seg", 8); c.setFill(GREY); c.drawString(18 * mm, 9.5 * mm, "Request to Credit: the twelve offer letters ·
Dupleix Institute · 7 October 2026")
    c.drawRightString(A4[0] - 18 * mm, 9.5 * mm, f"Page {d.page}"); c.restoreState()
    OUT = r"C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Request
to Credit - the twelve offer letters - 7 Oct 2026.pdf"
    doc = BaseDocTemplate(OUT, pagesize=A4, leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=20 * mm,
title="Request to Credit: the twelve offer letters", author="Dupleix Institute")
    doc.addPageTemplates([PageTemplate(id="p", frames=[Frame(doc.leftMargin, doc.bottomMargin, doc.width, doc.height, id="f",
leftPadding=0, rightPadding=0, topPadding=0, bottomPadding=0)], onPage=footer)])
    doc.build(s); print("written", OUT)

```

Result of Write

File created successfully at: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\offer_note.py (file state is current in your context – no need to Read it back)

Tool call: Bash

description: Build the offer letter request as a PDF

```
--- command ---
cd "/c/Users/wadza/AppData/Local/Temp/claude/c--xampp\htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python offer_note.py && ./songenv/Scripts/python -c "
import pypdfium2 as pdfium
p = r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Request to Credit - the twelve offer letters - 7 Oct 2026.pdf'
print('pages', len(pdfium.PdfDocument(p)))" && cp offer_note.py "/c/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC/Build files/"
{
  "timeout": 120000
}
```

Result of Bash

written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Request to Credit - the twelve offer letters - 7 Oct 2026.pdf
pages 3
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App

CLAUDE (2026-10-07 09:59:44 UTC)

Written for: Dibwa and the Credit team at MAIIC, in plain language.

Understood, and done. The request is a 3-page PDF covering twelve letters in two groups:

- **Why letters at all, now that we have the system data:** the EIR is calculated on the contract, and we already know of two sample loans where the core rate differs from the letter (Ebenezer Midian 31% against 25.1%, JAT Group 33% against 25.3%). A box explains that "signed" means the customer-accepted version from the credit file, and that the ten letters received on 17 September look like unsigned issue versions.
- **Group A, the ten sample loans:** a table with customer, account number, product and the reason each one is on the list, plus the five things we will read from each letter (amount and drawdown terms, rate build-up and margin, fees, repayment terms and moratorium, signature date). It also asks for any later variation letters.
- **Group B, the two sanction checks:** Mchinji Civil Servants SACCO (50m in the system, 100m loaded at take-on) and VNC Bricks (20m against 200m), with what each letter will tell us and what happens next if the system is wrong.
- **What we will do with them,** and how to send them (PDF scans, one per loan, facility terms, fee schedule and signature page are enough).

One assumption to check with Edward before sending: I treated the 17 September letters as unsigned versions, because the September request to Credit was for the signed ones. If some of those were already the signed copies, the note says so can be answered in a line.

Full path:

C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Request to Credit - the twelve offer letters - 7 Oct 2026.pdf

USER (2026-10-07 10:00:05 UTC)

ensure the request explains why we need the 10 sample letters and the extra two referencing to the original ask

Tool call: Bash

```
description: Add the reference to the original ask and rebuild the request
--- command ---
cd "/c:/Users/wadza/AppData/Local/Temp/claude/c--xampp-htdocs-Stress-Testing-App/ee984144-7b28-4df1-9a6e-560f112869e9/scratchpad" && PYTHONIOENCODING=utf-8 ./songenv/Scripts/python - <<'PYEOF'
p = "offer_note.py"; s = open(p, encoding="utf-8").read()
def rep(old, new):
    global s
    assert s.count(old) == 1, old[:50]; s = s.replace(old, new)
    rep('H("1. Why offer letters, when we now have the system data")'',
    'H("1. Where this request comes from")
P("This is not a new ask. On 11 September 2026 our consolidated information request listed the signed offer letters among the documents needed for the "
    "effective interest rate, because the fees and the margin are not held in the core system. At the EIR meeting on 24 September it was agreed that Credit "
    "would provide <b>the signed offer letters for the ten sample loans, showing the fees and the margin</b>, alongside the system walkthrough. Ten letters "
    "reached us on 17 September; they appear to be the issue versions rather than the signed copies, so the 24 September item is still open. This note repeats "
    "that ask with the reasons set out in full, and adds two further letters that this week's audit of the system data has made necessary.")
TAB(["Group", "What", "Where it was asked", "Status"], [
    ["A", "The signed offer letters for the ten sample loans, with fees and margin", "11 September consolidated request; agreed at the 24 September meeting (item 3 of the follow-up)", "Open: unsigned versions received 17 September"],
    ["B", "Two further letters, to settle the approved amount on two take-on loans", "New: from the audit of the 7 October extracts", "New"],
], [0.08, 0.37, 0.35, 0.20])
H("2. Why offer letters, when we now have the system data")''')
rep('H("2. Group A: the signed letters for the ten sample loans")
P("These are the ten loans we agreed to follow end to end, from credit approval to the ledger. For each we need the signed letter so that we can read, "
    "from the contract itself:")'',
    'H("3. Group A: the signed letters for the ten sample loans, as agreed on 24 September")
P("These are the ten loans chosen in September as the sample for the whole engine: the same ten whose EMI charts Barry provided, whose interest we "
    "rebuilt from the ledger, and which the walkthrough followed from approval to posting. The reason the signed letter is needed for each is the same "
    "reason it was asked for in September: the fees deducted at drawdown and the margin over the reference rate are the two terms that make an effective "
    "interest rate differ from the rate on the system, and neither is held in the core. From each signed letter we will read:")''')
rep('H("3. Group B: two letters to settle an approved amount")'', 'H("4. Group B: two further letters, arising from this week's audit")''')
rep('H("4. What we will do with them")'', 'H("5. What we will do with them")''')
rep('H("5. How to send them")'', 'H("6. How to send them")''')
open(p, "w", encoding="utf-8").write(s); print("patched")
PYEOF
PYTHONIOENCODING=utf-8 ./songenv/Scripts/python offer_note.py && ./songenv/Scripts/python -c "
import pypdfium2 as pdfium
p = r'C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Request to Credit - the twelve offer letters - 7 Oct 2026.pdf'
pdf = pdfium.PdfDocument(p); print('pages', len(pdf)); pdf[0].render(scale=0.9).to_pil().save('offer_p1.png')" && cp offer_note.py "/c:/Users/wadza/OneDrive/2026/Projects/MAIIC/2. Documents from clients/Raw Query Scripts/Query Requests to MAIIC/Build files/"
{
    "timeout": 120000
}
```

Result of Bash

```
patched
written C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Request to Credit - the twelve offer letters - 7 Oct 2026.pdf
pages 3
Shell cwd was reset to c:\xampp\htdocs\Stress-Testing-App
```

Tool call: Read

```
file_path: C:\Users\wadza\AppData\Local\Temp\claude\c--xampp-htdocs-Stress-Testing-App\ee984144-7b28-4df1-9a6e-560f112869e9\scratchpad\offer_p1.png
```

Result of Read

[image output: not reproduced in this transcript]

CLAUDE (2026-10-07 10:00:39 UTC)

Updated and rebuilt. The request now opens with "Where this request comes from", which ties both groups to the original ask:

- **Group A, the ten sample loans:** stated as not a new ask. It cites the 11 September consolidated request and the agreement at the 24 September EIR meeting (item 3 of the follow-up: the signed offer letters for the ten sample loans, showing the fees and the margin), notes that the ten letters received on 17 September appear to be issue versions rather than signed copies, and so marks the item as still open. The Group A heading now says "as agreed on 24 September", and its first paragraph explains the reason in full: these ten are the sample the whole engine follows, and the fees and

margin they carry are the two terms the core system does not hold.

- **Group B, the two extra letters:** marked as new, arising from the 7 October audit, with the sanction discrepancies that prompted them.

A small table under section 1 shows the two groups side by side with where each was asked and its status. Same file name and path as before.

USER (2026-10-07 10:01:02 UTC)

open the pdf

Tool call: PowerShell

description: Open the offer letter request in the default PDF viewer

--- command ---

```
Invoke-Item -LiteralPath "C:\Users\wadza\OneDrive\2026\Projects\MAIIC\2. Documents from clients\Raw Query Scripts\Query Requests to MAIIC\Request to Credit - the twelve offer letters - 7 Oct 2026.pdf"
```

Result of PowerShell

(PowerShell completed with no output)

CLAUDE (2026-10-07 10:01:09 UTC)

It should now be open in your PDF viewer.